

Data Science I with python

STAT 303-1-Sec20

Lizhen Shi

Table of contents

Preface	18
I Getting started	19
1 Setting up your environment with VS Code	20
1.1 Learning Objectives	20
1.2 Why Choose Visual Studio Code (VS Code)?	20
1.3 Quick Start: Setting Up Python & VS Code	21
1.4 Full Setup: Reliable Python Environment in VS Code	22
1.5 Jupyter Notebooks in VS Code	24
1.6 Rendering notebook as HTML using Quarto	25
1.6.1 Installing and Verifying Quarto	25
1.6.2 Converting the Notebook to HTML	26
1.7 Independent Study	26
1.7.1 Step-by-Step Instructions	27
1.8 Reference	28
2 Python Environments and Package Management	29
2.1 Learning Objectives	29
2.2 Data Science packages in Python	29
2.3 Python Virtual Environments	30
2.4 Install Data Science Packages Within Your Environment	30
2.4.1 How to Install Packages	31
2.4.2 Backing Up and Sharing Your Environment	32
2.5 Dependency Conflicts	36
2.5.1 Why do dependency conflicts happen?	36
2.5.2 How to avoid dependency conflicts	36
2.6 How to Resolve Conflicts and Manage Environments	36
2.6.1 Use <code>conda</code> for Robust Environment Management	36
2.6.2 Use <code>poetry</code> for Modern Python Projects	38
2.6.3 Difference Between <code>conda</code> and <code>poetry</code>	39
2.6.4 Modern Python Package Managers: <code>pip</code> , <code>conda</code> , <code>poetry</code> , <code>uv</code>	40
2.7 Independent Study: Practice Environment Setup with <code>pip</code>	41
2.7.1 Instructions	41

2.8	Resources	42
3	Enhancing Workflow in Jupyter Notebooks	43
3.1	Learning Objectives	43
3.2	Magic Commands	43
3.2.1	What are Magic Commands?	43
3.2.2	Line Magic	44
3.2.3	Cell Magic Commands	47
3.3	Shell Commands in Jupyter Notebooks	47
3.3.1	Using Shell Commands	48
3.4	Installing Required Packages Within Your Notebook	48
3.5	File Paths in Python	50
3.5.1	Absolute Path	50
3.5.2	Relative Path	50
3.5.3	Methods to Specify File Paths in Windows	51
3.5.4	File paths in macOS and Linux	52
3.6	Interacting with the OS and Filesystem	53
3.7	Navigate between directories and understand . and .. in paths	55
3.8	Independent Study	55
3.8.1	Setting Up Your Data Science Workspace	55
3.9	References and Further Learning	56
II	Python refresher	57
4	Python Basics	58
4.1	Python Variables	58
4.1.1	Rules for variable names	58
4.1.2	Dynamic Typing in Python Variables	59
4.1.3	Multiple Variable Assignment in Python	59
4.2	Built-in data types	60
4.3	Python Standard Library	61
4.4	Third-Party Packages (Libraries)	63
4.4.1	Importing a Library	63
4.5	User-defined functions	64
4.5.1	Creating and using functions	64
4.5.2	Variable scope: Local and global Variables	65
4.5.3	Function Arguments	66
4.6	Control Flow	68
4.6.1	Conditional Statements	68
4.6.2	Loops	74
4.7	Object Oriented Programming	80

5	Data Structures	83
5.1	Primitives vs. Containers	83
5.2	Core Built-in Data Structures	83
5.2.1	Sequences (ordered, indexable)	83
5.2.2	Sets (unordered, unique)	84
5.2.3	Mappings (key-value)	84
5.3	Iterables vs. Iterators	85
5.4	Common, Efficient Operations	85
5.4.1	Comprehensions	85
5.4.2	Membership & Lookups	87
5.4.3	Unpacking	87
5.4.4	Sorting in Python Iterables	89
5.4.5	Lambda Functions in Python	91
5.4.6	<code>enumerate()</code>	92
5.4.7	<code>zip()</code>	93
5.4.8	Common Functions for Iterables	94
5.5	Independent Study	95
5.5.1	Sorting the data	95
5.5.2	<code>lambda</code>	95
5.5.4	Student data	96
III	Core library foundations	98
6	Reading Data	99
6.1	Types of Data: Structured vs. Unstructured	99
6.1.1	Structured Data	99
6.1.2	Unstructured Data	100
6.2	Introduction to Pandas	100
6.3	Reading CSV Files with Pandas	100
6.3.1	Understanding CSV Files	100
6.3.2	CSV File Structure Example	101
6.3.3	Getting Started: Importing Pandas	101
6.3.4	Loading Data with <code>pd.read_csv()</code>	101
6.3.5	Indexing	103
6.3.6	Understanding Pandas Data Structures	105
6.3.7	Series - Your First Pandas Data Structure	105
6.4	DataFrame Exploration: Attributes and Methods	107
6.4.1	Attributes vs Methods: What's the Difference?	107
6.4.2	Essential DataFrame Attributes	107
6.5	Putting It All Together: Understanding Pandas Data Structures	113
6.5.1	DataFrame vs Series: The Complete Picture	113
6.5.2	Key Concepts You Now Understand	113

6.5.3	What You Can Do Now	114
6.6	Writing Data to CSV Files	114
6.6.1	Basic CSV Export	114
6.6.2	Important CSV Export Parameters	115
6.6.3	Best Practices for CSV Export	116
6.6.4	Complete Export Example	117
6.6.5	Verifying Your Export	117
6.7	Reading Other Data Formats	118
6.7.1	Reading Text Files (.txt)	119
6.7.2	Reading JSON Data	121
6.7.3	Reading Data from Web URLs	122
6.8	Independent Study	132
6.8.1	Practice exercise 1: Reading .csv data	132
6.8.2	Practice exercise 2: Reading .txt data	132
6.8.3	Practice exercise 3: Reading HTML data	133
6.8.4	Practice exercise 4: Reading JSON data	133
7	Pandas Fundamentals	134
7.1	Introduction to Pandas Fundamentals	134
7.1.1	Pandas in the Data Science Ecosystem	134
7.1.2	Core Pandas Capabilities	135
7.2	Creating Pandas Series & DataFrames	135
7.2.1	Method 1: Reading from External Data Sources	135
7.2.2	Method 2: Creating from Python Data Structures	136
7.3	Data Selection and Filtering	139
7.3.1	Basic Selection	139
7.3.2	Advanced Selection: .loc[] and .iloc[]	145
7.3.3	Finding Extremes in pandas: Values, Labels, and Positions	151
7.4	Sorting & Ranking: Organize data for analysis	154
7.4.1	Why Sorting and Ranking Matter	154
7.4.2	Basic Sorting with sort_values()	154
7.4.3	Sorting by Multiple Columns	155
7.4.4	Ranking Methods with rank()	157
7.4.5	Percentile Ranking	158
7.4.6	Advanced Sorting Techniques	161
7.4.7	Top-N and Bottom-N Selection	163
7.4.8	Performance Comparison: Sorting Methods	165
7.4.9	Real-World Sorting Scenarios	165
7.4.10	*Key Takeaways: Sorting & Ranking Mastery**	168
7.5	Adding & Removing: Modify DataFrame structure	169
7.5.1	Why DataFrame Structure Modification Matters	169
7.5.2	Adding Columns to DataFrames	169
7.5.3	Removing Columns from DataFrames	171

7.5.4	Removing Rows from DataFrames	173
7.5.5	Method Comparison Summary	174
7.6	Data Types: Working with Different Data Formats	175
7.6.1	Why Data Types Matter in Data Science	175
7.6.2	Complete Pandas Data Types Reference	175
7.6.3	Data Type Detection and Inspection	175
7.6.4	Smart Data Type Filtering with <code>select_dtypes()</code>	177
7.6.5	Data Type Conversion	180
7.6.6	Working with DateTime Data: The <code>.dt</code> Accessor	184
7.6.7	Working with <code>object</code> Data	188
7.7	Working with Numerical Data	192
7.7.1	Basic Descriptive Statistics	192
7.7.2	Individual Statistical Methods	197
7.7.3	Summary: Working with Numerical Data	203
7.8	Understanding <code>inplace=True</code> vs <code>inplace=False</code>	203
7.8.1	Why This Matters: Memory, Performance, and Safety	203
7.8.2	Core Concept: Modify vs Create	204
7.8.3	Common Methods Supporting <code>inplace</code> Parameter	205
7.9	Independent Practice	209
7.9.1	Practice exercise 1	209
7.9.2	Practice exercise 2	213
8	Pandas Intermediate	218
8.1	Learning Objective	218
8.2	Arithmetic Operations: Within and Between DataFrames	219
8.2.1	Why Arithmetic Operations Matter in Data Science	219
8.2.2	Arithmetic Operations Within a DataFrame	219
8.2.3	Arithmetic Operations Between DataFrames	223
8.2.4	Advanced Arithmetic Techniques	228
8.2.5	Error Handling and Edge Cases	229
8.2.6	Performance and Method Comparison	233
8.2.7	Are Arithmetic Operations Vectorized?	233
8.3	Correlation Operations: Understanding Relationships Between Variables	234
8.3.1	What is Correlation?	234
8.3.2	Correlation Coefficient Interpretation	234
8.3.3	Correlation Types	235
8.3.4	Pandas Correlation Methods	236
8.3.5	Summary: Correlation Operations	244
8.3.6	Are Correlation Operations Vectorized?	245
8.4	Advanced Operations: Function Application & Data Transformation	245
8.4.1	<code>apply()</code> : The Swiss Army Knife of Data Transformation	246
8.4.2	<code>map()</code> : Efficient Element-wise Transformations (on a Series)	248
8.4.3	<code>replace()</code> : Replace specific values with other values	251

8.4.4	Performance Comparison	252
8.5	Core Vectorized Operations in Pandas	253
8.5.1	Why Vectorization Matters	253
8.5.2	Categories of Vectorized Operations	254
8.5.3	Arithmetic Operations	254
8.5.4	Comparison Operations	255
8.5.5	Boolean Indexing & Filtering	257
8.5.6	Aggregation Operations	259
8.5.7	String Operations (<code>.str</code> accessor)	261
8.5.8	DateTime Operations (<code>.dt</code> accessor)	262
8.5.9	Mathematical Functions (NumPy Integration)	263
8.5.10	Performance Comparison: Loops vs Vectorization	264
8.5.11	Summary: Vectorization Best Practices	268
8.6	Summary: Performance-First Mindset	270
8.6.1	Key Takeaways	270
8.6.2	Looking Ahead: Why NumPy Matters	270
8.6.3	The Performance Pyramid	270
8.7	Lambda Functions: Inline Power for Data Transformation	271
8.7.1	Syntax & Structure	271
8.7.2	Key Characteristics	271
8.7.3	Real-World Lambda Applications	271
8.7.4	Performance Benchmarking: Lambda vs Named Functions vs Vectorized	277
8.8	Advanced NLP Preprocessing	280
8.8.1	the <code>re</code> module in Python is used for regular expression	280
8.8.2	The NLTK Library for Advanced NLP	284
8.9	Independent Study	312
8.9.1	Practice exercise 1	312
8.9.2	Practice exercise 2	316
9	NumPy Fundamentals	325
9.1	Learning Objectives	325
9.2	Getting Started with NumPy	326
9.3	Why NumPy?	326
9.3.1	NumPy Arrays are Memory Efficient: Homogeneity & Contiguous Storage	327
9.3.2	NumPy arrays are fast	328
9.4	Building Blocks: NumPy Arrays Fundamentals	329
9.4.1	Array Creation: Your Complete Toolkit	329
9.4.2	Understanding Array Attributes	334
9.5	Data Types and Memory Optimization	335
9.5.1	Common NumPy Data Types	336
9.5.2	Upcasting	337
9.6	Array Indexing and Slicing: Accessing Your Data	338
9.6.1	Array Indexing	338

9.6.2	Array Slicing	339
9.6.3	Combining Indexing and Slicing	340
9.6.4	Boolean Mask: Conditional Selection	340
9.6.5	Indexing & Slicing Quick Reference	341
9.7	Advanced Selection Methods	341
9.7.1	Conditional Selection with <code>np.where()</code> and <code>np.select()</code>	341
9.7.2	<code>np.where()</code>	342
9.7.3	<code>np.select</code> : Multi-way Conditional Logic	343
9.7.4	Finding Extremes with <code>np.argmin()</code> and <code>np.argmax()</code>	345
9.7.5	Finding the Top- <i>n</i> with <code>np.argsort()</code>	349
9.8	Array Operations: Mathematical Power at Scale	351
9.8.1	Arithmetic Operations	351
9.8.2	Aggregate Functions: Statistical Summaries	358
9.9	Array Reshaping: Transforming Data Dimensions	360
9.9.1	Core Reshaping Methods	360
9.9.2	Reshaping Quick Reference	372
9.10	Array Concatenation: Joining Arrays Together	374
9.10.1	Understanding Axes in Concatenation	374
9.10.2	<code>np.concatenate()</code> : The Universal Joiner	374
9.10.3	Specialized Stacking Functions	383
9.10.4	Advanced Concatenation Techniques	387
9.10.5	Concatenation Quick Reference	390
9.10.6	Common Pitfalls and Solutions	391
9.11	Independent Practice	391
9.11.1	Capitals & Distance from Washington, D.C.	391
9.11.2	Bonus Task: Top 10 Nearest & Farthest Capitals from Washington, D.C. (Haversine)	392
10	NumPy Intermediate	394
10.1	Learning Objectives	394
10.2	Vectorization in NumPy	394
10.2.1	Why Vectorization Matters	395
10.3	Vectorized Operations in NumPy	399
10.3.1	Types of Vectorized Operations	399
10.3.2	Matrix Multiplication in NumPy with Vectorization	402
10.4	Converting Between NumPy Arrays and Pandas DataFrames	404
10.4.1	Decision Framework: When to Use Which Library	405
10.5	Conversion Methods: Pandas → NumPy	412
10.5.1	The Two-Way Street	412
10.5.2	Save & Restore Metadata Manually	412
10.5.3	Pandas DataFrame → NumPy Array	415
10.5.4	NumPy Array → Pandas DataFrame	420

10.6	Random Number Generation in NumPy	422
10.6.1	Why NumPy Random Over Python Random?	423
10.6.2	Core Random Number Functions	423
10.6.3	Uniform Distribution: <code>np.random.rand()</code> and <code>np.random.uniform()</code> .	424
10.6.4	Normal (Gaussian) Distribution: <code>np.random.randn()</code> and <code>np.random.normal()</code>	425
10.6.5	Integer Random Numbers: <code>np.random.randint()</code>	426
10.6.6	Random Selection: <code>np.random.choice()</code>	427
10.6.7	Other Useful Distributions	428
10.6.8	Reproducibility: Random Seeds	429
10.6.9	Performance: NumPy vs Python Random	429
10.6.10	Quick Reference Guide	430
10.7	Independent Study	431
10.7.1	Practice Exercise 1: Shopping Optimization with Vectorization	431
10.7.2	Practice Exercise 2: Movie Rating Analysis with Matrix Multiplication	432
10.7.3	Practice Exercise 3: Simulation Study with Random Number Generation	433
10.7.4	Practice Exercise 4: Bootstrapping for Confidence Intervals	434
11	Data Visualization Fundamentals	435
11.1	Learning Objectives	435
11.2	The Art of Visualization: Choosing the Right Plot Type	436
11.2.1	Data Classification for Visualization	436
11.2.2	The Role of Visualization in Data Analysis	436
11.2.3	Quick Reference: Data Types and Visualization Mapping	438
11.3	Visualization Tools: The Python Ecosystem	439
11.3.1	The Three-Library Approach	439
11.3.2	Why These Three?	439
11.3.3	Library Comparison at a Glance	439
11.3.4	Step 1: Import Libraries	440
11.3.5	Step 2: Understanding the Imports	440
11.3.6	Step 3: Configure Your Environment	441
11.3.7	Step 4: Understanding the Configuration	441
11.3.8	Step 5: Verify Your Setup	442
11.3.9	Step 6: Pro Tips for Working with These Libraries	443
11.3.10	You're Ready!	445
11.4	Basic Plotting with Pandas	445
11.4.1	Dataset: COVID-19 Cases	446
11.4.2	Step 1: Your First Pandas Plot	446
11.4.3	Step 2: Plotting Multiple Variables	448
11.4.4	Step 3: Customizing Your Plots	449
11.4.5	Step 4: Different Plot Types	450
11.4.6	Step 5: Choosing the Right Plot Type	454
11.4.7	Additional Resources	455
11.4.8	Summary: Pandas Plotting Strengths & Limitations	455

11.4.9	Practice Exercise: Apply Your Skills	456
11.5	Data Plotting with Matplotlib pyplot Interface	456
11.5.1	What is Matplotlib?	457
11.5.2	Step 1: Understanding pyplot	457
11.5.3	Step 2: Your First pyplot Plot	458
11.5.4	Step 3: Customizing Axes	460
11.5.5	Step 4: Plotting Multiple Lines	461
11.5.6	Step 5: Adding Labels, Titles, and Legends	463
11.5.7	Step 6: Styling Lines and Markers	465
11.5.8	Step 7: Controlling Figure Size	470
11.5.9	Step 8: Other Plot Types with pyplot	471
11.5.10	What's Next?	480
11.6	Plotting with Seaborn	480
11.6.1	Step 1: Why Choose Seaborn Over Matplotlib?	481
11.6.2	Step 2: Setup and Aesthetic Customization	482
11.6.3	Step 3: Distribution Plots	483
11.6.4	Step 4: Relational Plots - Scatter Plots	484
11.6.5	Step 5: Distribution Plots - Histograms	488
11.6.6	Step 6: Categorical Plots - Bar Plots	491
11.6.7	Step 7: Box Plots - Distribution Summary	494
11.6.8	Step 8: Matrix Visualizations - Heatmaps	496
11.6.9	Step 9: Correlation Matrices	499
11.6.10	Interpreting Correlation Coefficients	504
11.6.11	Summary: Seaborn Best Practices	505
11.6.12	Practice Exercises	507
11.7	Chapter Summary	507
11.7.1	Visualization Fundamentals	508
11.7.2	Complete Plot Type Reference	508
11.7.3	When to Use Each Library	509
11.7.4	Essential Best Practices	509
11.7.5	Pro Tip: Combine Libraries!	510
11.8	Independent Study	511
11.8.1	Practice exercise 1	511
11.8.2	Practice exercise 2	514
12	Data Visualization Intermediate	519
12.1	Chapter Overview	519
12.2	Matplotlib's Two Interfaces	519
12.2.1	Compare Pyplot vs Object-Oriented Interfaces	520
12.2.2	Understand the Matplotlib Object Hierarchy	523
12.2.3	Create and Customize Plots with the OOP Interface	527
12.3	Mastering Subplots	529
12.3.1	Types of Subplot Layouts	530

12.3.2	Create Simple Subplot Grids	530
12.3.3	Control Subplot Layout and Spacing	531
12.3.4	Integrate Matplotlib with Pandas and Seaborn	532
12.3.5	Create Nested Subplots (Insets)	535
12.3.6	Advanced Formatting with Custom Tick Formatters	537
12.4	Creating Subplots with Seaborn	540
12.4.1	Using <code>Facetgrid</code>	540
12.4.2	Using <code>Pairplot</code>	543
12.5	Geospatial Plotting	546
12.5.1	Static Plots with GeoPandas	546
12.5.2	Dataset: Bicycle Sharing in Chicago	549
12.5.3	Adding the divvy station to the plot	550
12.5.4	Change the chicago shapefile	551
12.5.5	Interactive Plotting	553
12.5.6	Adding the divvy station on the interactive <code>Folium.Map</code>	555
12.6	Independent Study	556
12.6.1	Practice exercise 1	556

IV From messy to insight 558

13 Data Cleaning 559

13.1	Common Data Quality Issues	559
13.2	Handling Missing Data	560
13.2.1	Dataset Introduction	560
13.2.2	Identifying missing values in a dataframe	561
13.2.3	Types of Missing Values	563
13.2.4	Methods for Handling Missing Values	564
13.2.5	Imputing Missing Values	567
13.3	Dealing with Duplicate Records	581
13.3.1	Creating a Sample Dataset with Duplicates	581
13.3.2	Identifying Duplicate Records	582
13.3.3	Removing Duplicate Records	585
13.3.4	Applying to Real Data	587
13.4	Outlier detection and handling	587
13.4.1	Outlier detection	588
13.4.2	Common Methods for Handling outliers	593
13.5	Independent Study	595
13.5.1	Exercise 1: Missing Value Imputation Comparison	595
13.5.2	Exercise 2: Outlier Detection and Impact Analysis	595
13.5.3	Exercise 3: Combined Data Cleaning Pipeline	596

14 Data Preparation	597
14.1 Why Data Preparation Matters	597
14.2 Dataset Introduction	598
14.3 Data Binning	598
14.3.1 Equal-width Binning	602
14.3.2 Equal-size (quantile) Binning	609
14.3.3 Custom Binning	614
14.4 Encoding Categorical Variables	619
14.4.1 Label Encoding	619
14.4.2 One-Hot Encoding	620
14.5 Feature Scaling	622
14.5.1 How to Scale Features in Practice	623
14.6 Independent Study	625
14.6.1 Practice exercise 1	625
14.6.2 Practice exercise 2	625
 15 Data Grouping and Aggregation	 626
15.1 Why Grouping and Aggregation Matter in Data Science	626
15.2 Categorical Aggregation	627
15.2.1 One-way Aggregation: <code>value_counts()</code>	628
15.2.2 Two-way Aggregation: <code>crosstab()</code>	629
15.3 <code>groupby()</code> : Grouping by a Single Column	632
15.3.1 Syntax of <code>groupby()</code>	632
15.3.2 Example: Grouping by Continent	632
15.3.3 Understanding the GroupBy Object	633
15.3.4 Exploring GroupBy Objects: Attributes and Methods	633
15.4 Data Aggregation Within Groups	636
15.4.1 Common Aggregation Functions	636
15.4.2 Example: Calculating Mean Statistics by Country	637
15.4.3 Calculating Other Statistics	638
15.5 Multiple and Custom Aggregations Using <code>agg()</code>	639
15.5.1 Multiple Aggregations on a Single Column	639
15.5.2 Custom Aggregation Functions	640
15.5.3 Renaming Aggregated Columns	641
15.6 Multiple Aggregations on Multiple Columns	642
15.7 Different Aggregations for Different Columns	643
15.7.1 Combining Dictionary Syntax with Custom Column Names	643
15.8 Grouping by Multiple Columns	644
15.8.1 Basic Syntax for Grouping by Multiple Columns	644
15.8.2 Understanding Hierarchical (Multi-Level) Indexing	645
15.8.3 Subsetting Data in a Hierarchical Index	646
15.8.4 Grouping by multiple columns and aggregating multiple variables	648

15.9	Advanced Operations within groups: <code>apply()</code> , <code>transform()</code> , and <code>filter()</code> . .	649
15.9.1	Using <code>apply()</code> on groups	649
15.9.2	Using <code>transform()</code> on Groups	651
15.9.3	Using <code>filter()</code> on Groups	656
15.10	Sampling data by group	657
15.11	<code>corr()</code> : Correlation by group	658
15.12	Independent Study	659
15.12.1	Practice exercise 1	659
16	Data Reshaping	661
16.1	Introduction to Data Reshaping	661
16.1.1	Why Data Reshaping Matters	661
16.1.2	Understanding Wide vs. Long Format	662
16.1.3	Reshaping Tools in Pandas	664
16.2	Pivoting from Long to Wide: <code>pivot_table()</code>	664
16.2.1	What is Pivoting?	665
16.2.2	Example 1: Simple Pivot with Single Index	665
16.2.3	Example 2: Multi-Level Index (Hierarchical Pivot)	666
16.2.4	Practical Use: Temporal Comparisons	667
16.2.5	Visualizing Pivot Tables with Heatmaps	668
16.3	Melting from Wide to Long: <code>melt()</code>	672
16.3.1	Why Melt Your Data?	672
16.3.2	Example 1: Basic Melt	673
16.3.3	Example 2: Using Melted Data for Visualization	675
16.3.4	Pivot Melt: The Full Circle	676
16.4	Stacking and Unstacking using <code>stack()</code> and <code>unstack()</code>	678
16.4.1	Understanding the <code>level</code> in multi-level index and columns	680
16.4.2	Stacking (Columns to Rows): <code>stack()</code>	682
16.4.3	Unstacking (Rows to Columns): <code>unstack()</code>	683
16.4.4	Transposing using <code>.T</code> attribute	684
16.5	Summary of Common Reshaping Functions	684
16.6	Converting from Multi-Level Index to Single-Level Index DataFrame	685
16.7	Independent Study	686
16.7.1	Preparing GDP per capita data	686
16.7.2	Preparing population data	688
17	Combining Datasets	689
17.1	Introduction: Why Combine Datasets?	689
17.1.1	Common Scenarios Requiring Data Combination	689
17.1.2	The Three Main Combination Methods	690
17.1.3	Setup: Loading Required Libraries	690
17.2	Method 1: Merging with <code>merge()</code>	690
17.2.1	Basic Syntax	691

17.2.2	Example Datasets: Lord of the Rings Fellowship	691
17.2.3	Example 1: Basic Merge with Automatic Key Detection	692
17.2.4	Example 2: Merging with Different Column Names	693
17.2.5	Example 3: Merging on Multiple Columns	694
17.2.6	Understanding Join Types: The <code>how</code> Parameter	695
17.3	Method 2: Joining with <code>join()</code>	702
17.3.1	Key Difference: <code>merge()</code> vs <code>join()</code>	702
17.3.2	Important: Handling Overlapping Columns	702
17.3.3	Setting a Meaningful Index for Joining	703
17.3.4	Changing Join Type in <code>join()</code>	705
17.4	Method 3: Concatenating with <code>concat()</code>	705
17.4.1	The Two Directions: <code>axis</code> Parameter	705
17.4.2	Concatenating Along Rows (Vertical Stacking)	706
17.4.3	Horizontal Concatenation (Combining Columns)	709
17.5	Summary: Choosing the Right Method	711
17.5.1	Decision Tree:	711
17.6	Handling Missing Values After Combining Datasets	711
17.6.1	Why Missing Values Occur	711
17.6.2	Strategies for Handling Missing Values	712
17.6.3	Practical Example: Handling Missing Values	714
17.6.4	Best Practices for Missing Value Handling	715
17.7	Independent Study	716
17.7.1	Merging GDP per capita and population datasets	716
17.7.2	Merging datasets with <i>similar</i> values in the <i>key</i> column	716

Appendices 719

A Assignment A 719

Instructions	719
A.1 GDP per Capita (35 pts)	720
A.1.1 List Comprehension	720
A.1.3 Dictionary	721
A.2 Student Survey (40 pts)	722
A.2.1 Marriage age responses	722
A.2.2 Majors/Minors: Nested lists of student majors.	723
A.2.3 Starting salary expectations	724
A.3 Starbucks Drinks (20 pts)	724
A.4 AI Use Disclosure (5 pts)	726
A.4.1 AI Use Template	726

B Assignment B 728

Instructions	728
------------------------	-----

B.1	GDP per capita & social indicators	728
B.2	GDP per capita vs social indicators	730
B.3	Medical data	733
B.4	Bonus question	734
B.5	AI Use Disclosure	735
B.5.1	AI Use Template	736
C	Assignment C	737
	Instructions	737
C.1	Air quality sensors (30 pts)	738
C.1.1	Data	738
C.1.2	First sensor	738
C.1.3	Second sensor	738
C.1.4	First two sensors	739
C.1.5	Third sensor	739
C.1.6	All 50 sensors	740
C.2	Sales (16 pts)	740
C.2.1	Create first array	741
C.2.2	Create second array	741
C.2.3	Multiply arrays	741
C.2.4	Sum array elements	741
C.3	Starbucks restaurant (35 pts)	742
C.3.1	Most consumed drink type (name) and state with highest consumption (8 pts)	742
C.3.2	Highest total fat consumption drink type in Illinois (10 pts)	742
C.3.3	States with lowest total fat and highest protein consumption (10 pts)	743
C.3.4	Counting states with protein $> 2 \times$ fat consumption (7 pts)	743
C.4	Exercise minutes (16 pts)	743
C.4.1	Sequential computation without NumPy	744
C.4.2	Parallel computation with NumPy	744
C.4.3	Time saving with NumPy	744
D	Assignment D	745
	Instructions	745
	Exploring factors associated with profitability of a movie	746
D.1	Time trend	746
D.1.1	Month of release	746
D.1.2	Month of release with number of movies in each genre	747
D.1.3	Month of release with proportion of movies in each genre	748
D.1.4	Year of release with genre	748
D.2	Associations	749
D.2.1	Pairplot / heatmap	749

D.2.2	Nested associations	750
D.2.3	Profit based on movie director	750
D.3	Distributions	751
D.3.1	Distribution of profit based on genre (boxplots)	751
D.3.2	Distribution of profit based on genre (density plots)	751
D.4	Insights	752
Bonus question: Stock Market		753
Active investing		753
Passive Investing		753
Tasks		754
Return		754
Risk		754
Sharpe Ratio		754
D.5	Return on investment	755
D.5.1	Impatient investor (daily)	755
D.5.2	Patient Investor (yearly)	755
D.5.3	From daily to yearly	756
E Assignment E		757
Instructions		757
E.1	Missing value imputation	757
E.1.1	Number of missing values	758
E.1.2	Indices of missing values	758
E.1.3	Correlation of continuous variables with price	758
E.1.4	Missing value imputation using correlated continuous variable	758
E.1.5	Correlation of categorical variables with price	759
E.1.6	Missing value imputation using correlated categorical variable	759
E.1.7	Missing value imputation using correlated continuous variable within the categories of correlated categorical variable	760
E.1.8	Bonus question: Missing value imputation with KNN	760
E.1.9	Ultra-bonus question	762
E.2	Binning	763
E.2.1	Trend with house_age	763
E.2.2	Trend with bins of house_age	763
E.2.3	Trend with number_convenience_stores	764
E.2.4	Trend with bins of number_convenience_stores	764
E.2.5	Size of number_convenience_stores bins	764
E.3	Outliers	764
E.3.1	Identifying outlying prices	764
E.3.2	Quick EDA	764

F	Assignment F	765
	Instructions	765
F.1	Canadian Fish Biodiversity	766
F.1.1	Top 3 projects	766
F.1.2	Missing value imputation with <code>groupby()</code>	766
F.1.3	Living conditions	768
F.1.4	Fish diversity	768
F.2	GDP, surplus, and compensation	770
F.2.1	Combining datasets	770
F.2.2	Time trend: GDP with region	770
G	Datasets & Templates	771

Preface

Welcome to **STAT 303-1 (Data Science with Python I), Section 20** at Northwestern University.

This book is designed to support your journey into data science, blending foundational concepts with hands-on practice. It builds on the original work of **Professor Arvind Krishna**, whose materials inspired the structure and spirit of this resource. The content has been thoughtfully updated to meet the evolving needs of students and the goals of the course.

In this first course, you will:

- Learn to use essential Python libraries for data science, including but not limited to **NumPy**, **Pandas**, and **Matplotlib**. These three are the main focus, but you will also be introduced to other useful libraries and tools as needed for real-world data analysis.
- Develop skills in **Exploratory Data Analysis (EDA)**, transforming messy, real-world datasets into meaningful insights through visualization, summarization, and pattern discovery

The book is organized to guide you step-by-step:

- **Chapters 1–3**: Set up your coding environment in VS Code, understand Python environments, and enhance your workflow
- **Chapters 4–5**: Review key concepts from STAT 201. If you need a refresher, visit the [STAT 201 ebook](#).
- New material begins in **Chapter 6 – *Reading Data***

Throughout the quarter, this resource will be updated and refined to improve clarity, depth, and alignment with our teaching objectives.

As a **living document**, your feedback and suggestions are always welcome. Contributions from students, instructors, and the broader academic community help make this book stronger and more effective.

If you have ideas for improving the textbook, please share your feedback or suggestions using our dedicated [Textbook Improvement Form](#). Your input is invaluable in making this resource better for everyone.

Thank you for joining us on this learning adventure. We hope this book becomes a valuable companion as you build your skills and confidence in data science with Python.

Part I

Getting started

1 Setting up your environment with VS Code

<IPython.core.display.Image object>

1.1 Learning Objectives

Setting up your Python data science environment is the foundation for all your work in this course. A well-configured setup will save you time, prevent errors, and make your workflow smoother. Whether you're new to VS Code or Python, these steps will help you build a reliable environment for coding, analysis, and reproducibility.

By the end of this chapter, you will be able to:

- Install and configure **Visual Studio Code (VS Code)** for Python and Jupyter Notebooks.
- Create and manage a **virtual environment** for this course.
- Install essential **Python libraries** for data science.
- Verify that your environment is working properly.

1.2 Why Choose Visual Studio Code (VS Code)?

Choosing the right editor can make a big difference in your productivity and learning experience. VS Code is a top choice for data science and Python development because it combines power, flexibility, and ease of use.

- **Free and Lightweight**
VS Code is free, open-source, and runs efficiently on most systems without being resource-heavy.
- **Cross-Platform**
Works consistently on Windows, macOS, and Linux.

- **Rich Python Support**

With the official Python extension, you get:

- Syntax highlighting and IntelliSense (auto-completion, code hints).
- Built-in debugging tools.
- Easy integration with virtual environments and Jupyter Notebooks.

- **Integrated Jupyter Notebook Support**

You can open and run `.ipynb` notebooks directly in VS Code.

- **Customizable and Extensible**

Large marketplace of extensions (e.g., formatting, linting, Git, Docker, AI tools).

- **Version Control Integration**

Built-in Git and GitHub support for managing and sharing code.

- **Unified Workflow**

One place for editing, running, debugging, documenting, and version-controlling Python code.

Whether you're just starting out or working on advanced projects, VS Code provides a modern, unified environment for all your coding needs.

1.3 Quick Start: Setting Up Python & VS Code

VS Code is a versatile editor. To use it for data science, you'll complete a few extra setup steps (e.g., Python & Jupyter extensions). Follow this step-by-step setup guide to get started quickly and avoid common pitfalls.

Step 1: Install Python

- Download and install the latest version from python.org.
- On Windows, check the box “**Add Python to PATH.**” This makes Python available in your terminal.

Step 2: Install VS Code & Extensions

- Download VS Code from code.visualstudio.com.
- Open VS Code, go to the Extensions panel (`Ctrl+Shift+X`), and install:
 - **Python (Microsoft)**
 - **Jupyter (Microsoft)**

Step 3: Create a Course Folder

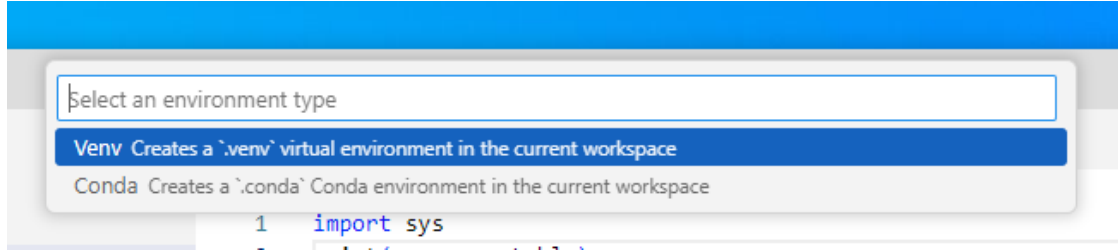
- Make a folder for your notebooks (e.g., `datasci` on Desktop or Documents).
- Open this folder in VS Code.

Step 4: Create Your First Notebook

- In VS Code, go to **File** → **New File** → **Save as** → `test.ipynb`.
- Add a code cell:

```
print("hello world")
```

- Run the cell. VS Code may prompt you to select or create a Python environment.



- If prompted, create a new environment using the GUI (recommended for beginners).
- Make sure the kernel (top right) shows **.venv (Python 3.xx.x)**.
- If you see `hello world` printed, your setup is working!

If everything works, you can skip the Full Setup below.

If you run into issues, continue with the detailed setup and troubleshooting steps in the next section.

This quick start helps you get coding fast, but the full setup ensures your environment is robust and reproducible for all course projects.

1.4 Full Setup: Reliable Python Environment in VS Code

If the quick start didn't work, or you want a more robust setup, follow these step-by-step instructions to ensure your Python environment is reproducible and ready for data science.

Step 1: Install Python

- Go to the [official Python website](https://www.python.org/) and download the latest version for your operating system.
- **Important:** During installation, check the box “**Add Python to PATH**” so Python is available in your terminal.
- After installation, restart your terminal in VS Code.

- Verify installation by running:

```
python --version
```

You should see the installed Python version.

Step 2: Install VS Code Extensions

- Open VS Code.
- Go to Extensions (Ctrl+Shift+X).
- Install:
 - **Python** (by Microsoft)
 - **Jupyter** (by Microsoft)

Step 3: Set Up Your Workspace

- Create a new folder for your course work (e.g., STAT303-1 on Desktop or Documents).
- Open the folder in VS Code (File > Open Folder).

Step 4: Create a Notebook for Your Work

- In VS Code, go to File > New File and select Jupyter Notebook.
- Save the file as `your_notebook.ipynb`.
- Add a code cell:

```
print("hello world")
```

- Make sure to select the `.venv` kernel for your notebook (see next step).

Step 5A: Create a Virtual Environment (GUI Method)

- Run the cell. If prompted, create a Python environment using `venv` in the current workspace.
- Wait for Jupyter to start and install necessary components (e.g., ipykernel).
- After installation, ensure the selected kernel in the top right of the notebook says **.venv (Python 3.xx.x)**.
- If not, select the `.venv` kernel manually.

If you successfully created the environment using the GUI, skip Step 5B.

Step 5B: Create a Virtual Environment (Command Line Method)

- If not prompted to create your environment, use the terminal:
 - Open the terminal in VS Code.

- Run:

```
python -m venv .venv
```

- Activate the environment:

- * On Windows:

```
.\.venv\Scripts\activate
```

- * On macOS/Linux:

```
source .venv/bin/activate
```

- You should see (.venv) at the start of your terminal prompt.
- Install essential packages:

```
pip install numpy pandas matplotlib
```

Step 6: Verify Your Setup

- Run the code cell again. You should see `hello world` printed out.
- In the terminal, check your Python version and installed packages:

```
python --version  
pip list
```

Troubleshooting Tips

- If the kernel does not show `.venv`, restart VS Code and ensure the environment is activated.
- Make sure the Python and Jupyter extensions are installed and enabled.
- If you see errors, check that your notebook is saved with the `.ipynb` extension and the correct kernel is selected.

A reliable environment is essential for reproducible data science. These steps will help you avoid common issues and set yourself up for success throughout the course.

1.5 Jupyter Notebooks in VS Code

After setting up your environment, you can take full advantage of Jupyter Notebooks directly in VS Code. This integration lets you:

- Create, edit, and run `.ipynb` notebooks without leaving VS Code
- Use interactive code cells for data analysis, visualization, and documentation
- Access rich outputs (plots, tables, markdown) inline with your code

- Switch kernels to use different Python environments for each notebook
- Debug, refactor, and version-control your notebooks with built-in tools

Getting Started:

- Open or create a notebook file (.ipynb) in VS Code
- Make sure the correct Python kernel/environment is selected (top right corner)
- Add code and markdown cells as needed
- Run cells individually or all at once

Learn More:

- Explore the official guide: [Jupyter Notebooks in VS Code](#)

Jupyter Notebooks in VS Code provide a powerful, flexible workflow for data science and Python development.

1.6 Rendering notebook as HTML using Quarto

Quarto is designed for high-quality, customizable, and publishable outputs, making it suitable for reports, blogs, or presentations. We'll use Quarto to print the *.ipynb* file as HTML for your assignment submission.

1.6.1 Installing and Verifying Quarto

- Download and install Quarto from the [official website](#).
- Follow the installation instructions for your operating system.
- Open your terminal within VS Code:
 - Go to **Terminal** -> **New Terminal**.
- Run the following command to verify that Quarto and its dependencies are correctly installed: `quarto --version`
- You should see the installed quarto version. If the command line doesn't work, you might see an error message like:
 - ``quarto is not recognized``
 - ``quarto command is not found``

This issue is often caused by Quarto not being added to the PATH environment variable. Similar to how you added the Python path to the environment variable above, you need to add the Quarto path to the system environment variable so that the command can be recognized by your operating system's shell.

On Windows, if you used the default installation path (without changing it), Quarto is installed in: `C:\Users\<USER>\AppData\Local\Programs\Quarto\bin`

Note:

Before moving forward, ensure that the command `quarto --version` successfully prints the version of your installed quarto. If it does not, you may need to troubleshoot your quarto installation or add it to the PATH environment variable.

1.6.2 Converting the Notebook to HTML

Check the procedure for rendering a notebook as HTML [here](#). You have several options to format the file. Here are some points to remember when using Quarto to render your notebook as HTML:

1. The `Raw NBConvert` cell type is used to render different code formats into HTML or LaTeX. This information is stored in the notebook metadata and converted appropriately. **Use this cell type to put the desired formatting settings for the HTML file.**
2. In the formatting settings, remember to use the setting `embed-resources: true`. This will ensure that the rendered HTML file is self-contained, and is not dependent on other files. This is especially important when you are sending the HTML file to someone, or uploading it somewhere. If the file is self-contained, then you can send the file by itself without having to attach the dependent files with it.

Once you have entered the desired formatting setting in the `Raw NBConvert` cell, you are ready to render the notebook to HTML. Open the terminal, navigate to the directory containing the notebook (*.ipynb file*), and use the command: `quarto render filename.ipynb --to html`.

1.7 Independent Study

Goal: Get hands-on experience creating a reproducible Python environment using `pip` and `.venv` in VS Code.

1.7.1 Step-by-Step Instructions

Step 1: Create Your Workspace

- Make a folder named STAT303_1 for your course work.
- Open this folder in VS Code.
- Create a new notebook: `setup_test.ipynb`.
- Add a code cell:

```
print("Hello, VS Code & .venv!")
```

- Run the cell. If prompted, create/select a Python environment (choose `.venv`).

Step 2: Create the `.venv` Environment

- If VS Code does not prompt you, open the terminal in VS Code.
- Run:

```
python -m venv .venv
```

- Activate the environment:

```
.\.venv\Scripts\activate
```

- Install essential packages:

```
pip install numpy pandas matplotlib
```

Step 3: Verify Environment Consistency

- In the terminal, check your Python version:

```
python --version
```

- In the notebook, run:

```
import sys
print(sys.executable)
```

- Make sure the Python path in the notebook matches the one in your terminal. This confirms you are using the same environment.

Step 4: Generate html from the notebook using Quarto

- In the terminal, make sure you are in the directory that contains your notebook

```
quarto render setup_test.ipynb --to html
```

- Make sure the html file was generated in the same directory

Troubleshooting Tips

If you run into issues (e.g., kernel not showing `.venv`, code cell errors):

- Restart VS Code.
- Deactivate/reactivate `.venv`.
- Make sure **Python** & **Jupyter** extensions are installed.
- Save your notebook with the `.ipynb` extension.
- If the kernel is missing, use the kernel picker (top right) to select `.venv`.
- If you see an error like *“No such file or directory”* or *“No valid input files passed to render”*:
 1. Check your current location:
 - macOS / Linux: `pwd`
 - Windows: `cd`
 2. List files in the current directory:
 - macOS / Linux: `ls`
 - Windows: `dir`
 3. If the notebook isn’t there:
 - Use `cd` to move into the correct folder, or
 - Provide the full (absolute) path to the file.

Extra:

- Try installing a package (e.g., `seaborn`) and import it in a new cell to test.
- Practice switching kernels to see how environments affect your code.
- Practice creating a `.conda` env and use `conda` to install the `seaborn` package

1.8 Reference

- [Getting Started with VS Code](#)
- [Jupyter Notebooks in VS Code](#)
- [Install Python Packages](#)
- [Managing environments with `conda`](#)

2 Python Environments and Package Management

<IPython.core.display.Image object>

2.1 Learning Objectives

Managing Python environments and packages is essential for reproducible, error-free data science. Understanding these concepts will help you avoid common pitfalls and collaborate more effectively. This chapter will guide you through the tools and best practices for setting up, sharing, and troubleshooting Python environments.

By the end of this chapter , you will be able to:

- Explain why virtual environments are important.
- Install essential data science packages in your Python environment.
- Create a `requirements.txt` file and use it to quickly recreate an environment.
- Recognize why dependency conflicts happen.
- Describe how tools like `conda` or `poetry` can help resolve dependency conflicts.

2.2 Data Science packages in Python

Python has a rich ecosystem of packages that make data analysis easier and more powerful. In this course, you will install and use several core packages:

- **NumPy** → numerical computing and array operations
- **pandas** → data manipulation and analysis

- **Matplotlib** → data visualization
- **Seaborn** → statistical data visualization (built on top of Matplotlib)

These packages give you the essential tools to load, clean, analyze, and visualize data.

Before using them, you need to install them in your Python environment.

In your previous VS Code setup lesson, you created a Python environment using `.venv` and selected it as the kernel to run your notebook. This created a folder called `.venv`, which contains:

- A dedicated Python interpreter for your project
- A `Lib` folder where all packages you install are stored

Any packages you install (e.g., with `pip install ...`) will only affect this environment, keeping your project isolated from others.

Using virtual environments is considered best practice for Python development, especially in data science projects. This isolation helps you:

- Avoid version conflicts between packages
- Keep project dependencies separate
- Make your code more reproducible and shareable

Before we dive in, let's clarify what a Python environment is and why isolation matters.

2.3 Python Virtual Environments

A Python virtual environment is an **isolated workspace** with its own Python interpreter and its own set of installed packages.

This isolation makes it easy to manage dependencies for different projects and prevents package version conflicts.

Why use virtual environments?

- Keep each project's dependencies separate
- Avoid breaking code by accidentally upgrading/downgrading packages
- Share your code with others and ensure it works the same way on their machine

Virtual environments are a key tool for professional, reproducible Python workflows.

2.4 Install Data Science Packages Within Your Environment

Python's power for data science comes from its rich ecosystem of packages. These packages provide essential tools for scientific computing, data analysis, visualization, and machine learning.

Packages like `numpy`, `pandas`, and `matplotlib` are not included in the Python standard library—you need to install them in your environment before you can use them.

Test your setup: Add the following code to your `test.ipynb` notebook to check if your environment is ready:

```
import numpy as np
import pandas as pd
print("Setup complete!")
```

If you see a `ModuleNotFoundError`, it means the package is not installed in your current environment. Use `pip install` to add missing packages.

Tip: Always install packages inside your active virtual environment to keep your project isolated and reproducible.

2.4.1 How to Install Packages

There are two main ways to install data science packages in your environment:

2.4.1.1 Installing from the Terminal

- **Open a New Terminal:** Check that your terminal is using the same environment as your notebook kernel.
 - If you see `(.venv)` at the start of the prompt, your virtual environment is active and matches the notebook kernel.
 - If you see `(base)`, it means the base conda environment is active (common if you have Anaconda installed).
 - To confirm which Python is being used, run:
 - * On macOS/Linux: `which python`
 - * On Windows: `where.exe python`

Note: If you have both Anaconda and VS Code installed, environments can sometimes conflict. If the terminal and notebook use different environments, packages may be installed in the wrong place, causing import errors in your notebook.

- **Install packages with `pip`:**

```
pip install numpy pandas
```

2.4.1.2 Installing from the Notebook

You can also install packages directly from a Jupyter Notebook cell using a magic command. This is convenient for quick installs without leaving the notebook interface.

- Add a new code cell and run:

```
pip install numpy pandas
```

Best Practice: Always make sure you are installing packages into the correct environment. Double-check your kernel and terminal environment before running installation commands.

2.4.2 Backing Up and Sharing Your Environment

Reproducibility is a cornerstone of good data science. To ensure your code works for you, your collaborators, and on other machines, you need a way to recreate your Python environment exactly.

Backing up your environment makes it easy to share your work and avoid “it works on my machine” problems.

How to back up your environment:

Step 1: **Create a `requirements.txt` file**

- This file lists all installed packages and their versions, so anyone can recreate your environment.

Run the following command in your terminal to list all installed packages and their versions:

```
pip freeze
```

```
asttokens==2.4.1
colorama==0.4.6
comm==0.2.2
contourpy==1.3.0
cyclor==0.12.1
debugpy==1.8.6
decorator==5.1.1
executing==2.1.0
fonttools==4.54.1
ipykernel==6.29.5
ipython==8.27.0
jedi==0.19.1
```



```
jupyter_client==8.6.3
jupyter_core==5.7.2
kiwisolver==1.4.7
matplotlib==3.9.2
matplotlib-inline==0.1.7
nest-asyncio==1.6.0
numpy==2.1.1
packaging==24.1
pandas==2.2.3
parso==0.8.4
pillow==10.4.0
platformdirs==4.3.6
prompt_toolkit==3.0.48
psutil==6.0.0
pure_eval==0.2.3
Pygments==2.18.0
pyparsing==3.1.4
python-dateutil==2.9.0.post0
pytz==2024.2
pywin32==306
pyzmq==26.2.0
six==1.16.0
stack-data==0.6.3
tornado==6.4.1
traitlets==5.14.3
tzdata==2024.2
wcwidth==0.2.13
```

Note: you may need to restart the kernel to use updated packages.

Using the redirection operator `>`, you can save the output of `pip freeze` to a `requirement.txt`. This file can be used to install the same versions of packages in a different environment.

```
pip freeze > requirement.txt
```

Note: you may need to restart the kernel to use updated packages.

Let's check whether the `requirement.txt` is in the current working directory

```
%ls
```

Volume in drive C is Windows
Volume Serial Number is A80C-7DEC

Directory of c:\Users\lsi8012\OneDrive - Northwestern University\FA24\303-1\test_env

```
09/27/2024  02:25 PM    <DIR>          .
09/27/2024  02:25 PM    <DIR>          ..
09/27/2024  07:44 AM    <DIR>          .venv
09/27/2024  01:42 PM    <DIR>          images
09/27/2024  02:25 PM                695 requirement.txt
09/27/2024  02:25 PM            21,352 venv_setup.ipynb
                2 File(s)            22,047 bytes
                4 Dir(s)  166,334,562,304 bytes free
```

Step 2: Share the file

- Send the `requirements.txt` file to your collaborator, or save it for future use.

Step 3: Recreate the environment elsewhere

- On a new machine or environment, run:

```
pip install -r requirements.txt
```

```
pip install -r requirement.txt
```

```
Requirement already satisfied: asttokens==2.4.1 in c:\users\lsi8012\onedrive - northwestern u
Requirement already satisfied: colorama==0.4.6 in c:\users\lsi8012\onedrive - northwestern u
Requirement already satisfied: comm==0.2.2 in c:\users\lsi8012\onedrive - northwestern unive
Requirement already satisfied: contourpy==1.3.0 in c:\users\lsi8012\onedrive - northwestern u
Requirement already satisfied: cycler==0.12.1 in c:\users\lsi8012\onedrive - northwestern un
Requirement already satisfied: debugpy==1.8.6 in c:\users\lsi8012\onedrive - northwestern un
Requirement already satisfied: decorator==5.1.1 in c:\users\lsi8012\onedrive - northwestern u
Requirement already satisfied: executing==2.1.0 in c:\users\lsi8012\onedrive - northwestern u
Requirement already satisfied: fonttools==4.54.1 in c:\users\lsi8012\onedrive - northwestern
Requirement already satisfied: ipykernel==6.29.5 in c:\users\lsi8012\onedrive - northwestern
Requirement already satisfied: ipython==8.27.0 in c:\users\lsi8012\onedrive - northwestern u
Requirement already satisfied: jedi==0.19.1 in c:\users\lsi8012\onedrive - northwestern unive
Requirement already satisfied: jupyter_client==8.6.3 in c:\users\lsi8012\onedrive - northwes
Requirement already satisfied: jupyter_core==5.7.2 in c:\users\lsi8012\onedrive - northwester
Requirement already satisfied: kiwisolver==1.4.7 in c:\users\lsi8012\onedrive - northwestern
Requirement already satisfied: matplotlib==3.9.2 in c:\users\lsi8012\onedrive - northwestern
Requirement already satisfied: matplotlib-inline==0.1.7 in c:\users\lsi8012\onedrive - north
```

```

Requirement already satisfied: nest-asyncio==1.6.0 in c:\users\lsi8012\onedrive - northwestern uni
Requirement already satisfied: numpy==2.1.1 in c:\users\lsi8012\onedrive - northwestern uni
Requirement already satisfied: packaging==24.1 in c:\users\lsi8012\onedrive - northwestern uni
Requirement already satisfied: pandas==2.2.3 in c:\users\lsi8012\onedrive - northwestern uni
Requirement already satisfied: parso==0.8.4 in c:\users\lsi8012\onedrive - northwestern uni
Requirement already satisfied: pillow==10.4.0 in c:\users\lsi8012\onedrive - northwestern uni
Requirement already satisfied: platformdirs==4.3.6 in c:\users\lsi8012\onedrive - northwestern uni
Requirement already satisfied: prompt_toolkit==3.0.48 in c:\users\lsi8012\onedrive - northwestern uni
Requirement already satisfied: psutil==6.0.0 in c:\users\lsi8012\onedrive - northwestern uni
Requirement already satisfied: pure_eval==0.2.3 in c:\users\lsi8012\onedrive - northwestern uni
Requirement already satisfied: Pygments==2.18.0 in c:\users\lsi8012\onedrive - northwestern uni
Requirement already satisfied: pyparsing==3.1.4 in c:\users\lsi8012\onedrive - northwestern uni
Requirement already satisfied: python-dateutil==2.9.0.post0 in c:\users\lsi8012\onedrive - northwestern uni
Requirement already satisfied: pytz==2024.2 in c:\users\lsi8012\onedrive - northwestern uni
Requirement already satisfied: pywin32==306 in c:\users\lsi8012\onedrive - northwestern uni
Requirement already satisfied: pyzmq==26.2.0 in c:\users\lsi8012\onedrive - northwestern uni
Requirement already satisfied: six==1.16.0 in c:\users\lsi8012\onedrive - northwestern uni
Requirement already satisfied: stack-data==0.6.3 in c:\users\lsi8012\onedrive - northwestern uni
Requirement already satisfied: tornado==6.4.1 in c:\users\lsi8012\onedrive - northwestern uni
Requirement already satisfied: traitlets==5.14.3 in c:\users\lsi8012\onedrive - northwestern uni
Requirement already satisfied: tzdata==2024.2 in c:\users\lsi8012\onedrive - northwestern uni
Requirement already satisfied: wcwidth==0.2.13 in c:\users\lsi8012\onedrive - northwestern uni
Note: you may need to restart the kernel to use updated packages.

```

Once you run the install command, all packages listed in your `requirements.txt` file will be installed, matching the exact versions you used in your original environment.

This ensures your code runs the same way everywhere—whether on your computer, a collaborator’s machine, or a server.

Why is this important?

- Guarantees everyone is using the same package versions, reducing errors and “works on my machine” problems.
- Makes it easy to set up your project on a new computer, server, or cloud environment.
- Simplifies troubleshooting and collaboration—everyone starts from the same setup.

Pro Tip:

- Update your `requirements.txt` after installing or upgrading packages to keep it current.
- Consider version-controlling your `requirements.txt` file (e.g., with Git) so changes are tracked and shared with collaborators.

A well-maintained environment file is essential for reproducible, shareable, and professional data science projects.

2.5 Dependency Conflicts

A **dependency conflict** occurs when two or more packages in your Python environment require different or incompatible versions of the same library.

This can cause errors, unexpected behavior, or even break your code.

2.5.1 Why do dependency conflicts happen?

- Many Python packages rely on other packages (called *dependencies*) to work.
- If you install packages that depend on different versions of the same dependency, they may not work together.
- This is especially common in data science, where libraries evolve quickly and have complex interdependencies.

Example: Suppose you install two packages:

- PackageA requires `numpy==1.24.0`
- PackageB requires `numpy==2.2.0`

If you install both with `pip`, the **last specified version wins**. One of the packages may then fail because it cannot use the version of `numpy` that was actually installed.

2.5.2 How to avoid dependency conflicts

- Use **virtual environments** to isolate dependencies for each project.
- Check package requirements before installing new libraries.
- Use tools like **Poetry** or **Conda** to detect and resolve conflicts automatically.

2.6 How to Resolve Conflicts and Manage Environments

2.6.1 Use conda for Robust Environment Management

`conda` is a powerful tool for managing both Python environments and packages, especially in data science workflows. It helps you avoid and resolve dependency conflicts by handling package versions and system dependencies more effectively than `pip` alone.

Why use conda?

- Create fully isolated environments for different projects
- Install packages and all their dependencies from the Anaconda repository
- Easily switch between environments for different tasks
- Export and share environment configurations for reproducibility

How conda prevents dependency conflicts:

- When you install a package, conda automatically checks for compatible versions of all dependencies and installs them together.
- If a conflict is detected, conda will warn you, suggest solutions, or prevent incompatible installations.

Essential conda commands:

- Create a new environment:

```
conda create --name myenv numpy pandas matplotlib
```

- Activate an environment:

```
conda activate myenv
```

- Install a package in an environment:

```
conda install seaborn
```

- List all environments:

```
conda env list
```

- Export environment configuration:

```
conda env export > environment.yml
```

- Recreate an environment from a file:

```
conda env create -f environment.yml
```

Example: Suppose you need to work on two projects that require different versions of `scikit-learn`. You can create two separate environments:

```
conda create --name projectA scikit-learn=0.24  
conda create --name projectB scikit-learn=1.2
```

Each environment will have its own compatible dependencies, so you avoid conflicts.

Summary: Using `conda` is highly recommended for managing complex dependencies, system-level packages, and reproducible environments in data science.

2.6.2 Use poetry for Modern Python Projects

poetry is a modern tool for managing Python dependencies and packaging. It streamlines environment setup, ensures reproducibility, and makes sharing your project easy.

Why use poetry?

- Automatically creates and manages virtual environments for each project
- Uses a `pyproject.toml` file to specify and lock project requirements
- Prevents dependency conflicts with a lock file (`poetry.lock`)
- Simplifies publishing packages to PyPI and sharing with others

How poetry helps with dependency management:

- Resolves and installs compatible versions of all dependencies automatically
- Keeps your environment reproducible and up-to-date
- Makes it easy to add, update, or remove packages

Essential poetry commands:

Command / File	Description
<code>poetry install</code>	Installs all dependencies. If a <code>poetry.lock</code> file exists, installs exact versions from it. If not, resolves dependencies, creates the <code>poetry.lock</code> file, and installs.
<code>poetry add</code> <code><package></code>	Adds a package, resolves the new dependency tree, updates the <code>pyproject.toml</code> and updates the <code>poetry.lock</code> file.
<code>poetry remove</code> <code><package></code>	Removes a package, resolves the new dependency tree, updates the <code>pyproject.toml</code> and updates the <code>poetry.lock</code> file.
<code>poetry show</code>	Lists all installed packages (and their versions) as recorded in the <code>poetry.lock</code> file.
<code>poetry show</code> <code>--tree</code>	Displays the dependency tree based on the resolved dependencies in the <code>poetry.lock</code> file.
<code>poetry show</code> <code>--outdated</code>	Compares versions in the <code>poetry.lock</code> file against the latest available on the remote repository.
<code>poetry run</code> <code><command></code>	Runs a command in the virtual environment whose packages are defined by the <code>poetry.lock</code> file.
<code>poetry.lock</code> (File)	The most important file for reproducibility. Pins exact versions of every package and dependency. Always commit this to version control to ensure all developers and deployments use identical dependencies.

Core Files to Know

- **pyproject.toml:**
The *declarative* file where you define your project’s metadata, dependencies, and build system.
This file is meant to be edited by humans.
- **poetry.lock:**
The *definitive record* of the exact versions of every package that makes your project work.
This file is generated and managed by Poetry.
You should commit this to version control to ensure everyone (and your deployments) uses identical dependencies.

Summary: `poetry` is highly recommended for modern Python projects where you want reliable dependency management, easy environment setup, and reproducible results.

2.6.3 Difference Between `conda` and `poetry`

Both `conda` and `poetry` are tools for managing Python environments and dependencies, but they have different strengths and use cases.

conda:

- Manages both Python environments and packages, including non-Python dependencies (e.g., C libraries, compilers)
- Works with multiple languages (Python, R, etc.)
- Uses the Anaconda repository, which includes many scientific and data science packages
- Great for data science workflows, especially when you need packages with compiled code or system-level dependencies
- Handles complex dependency resolution and environment isolation

poetry:

- Focuses on Python projects and packages
- Uses `pyproject.toml` and `poetry.lock` for reproducible builds
- Automatically creates and manages virtual environments
- Excellent for modern Python application development and publishing to PyPI
- Handles dependency resolution and version locking for Python packages only

Key differences:

- `conda` can install system-level and non-Python dependencies; `poetry` only manages Python packages
- `conda` environments can include packages from the Anaconda repository; `poetry` uses PyPI
- `poetry` is ideal for pure Python projects and reproducible builds; `conda` is better for scientific computing and mixed-language projects

When to use each:

- Use **conda** if you need scientific libraries, compiled code, or non-Python dependencies
- Use **poetry** for modern Python projects, apps, and libraries where you want easy dependency management and publishing

Summary Table:

Feature	conda	poetry
Language support	Python, R, more	Python only
Non-Python dependencies	Yes	No
Environment isolation	Yes	Yes
Dependency resolution	Excellent	Excellent
Reproducibility	Good	Excellent
Publishing to PyPI	No	Yes

Choose the tool that best fits your project needs!

2.6.4 Modern Python Package Managers: `pip`, `conda`, `poetry`, `uv`

Python package management has evolved rapidly, making it easier to install, update, and manage dependencies for any project.

- **pip** is the standard tool for installing Python packages from PyPI. It works well for most projects and is simple to use.
- **.venv** helps you create isolated environments so each project has its own dependencies.
- **conda** offers advanced environment and package management, especially for scientific computing and projects with non-Python dependencies.
- **poetry** provides modern dependency management, automatic environment creation, and reproducibility for Python projects.
- **uv** is a new, high-performance package manager written in Rust. It's designed for speed and supports modern workflows. Learn more: [uv GitHub page](#)

For this course:

- You'll use **pip** and **.venv** for installing packages and managing your environment.
- As you tackle larger or more complex projects, consider exploring **conda**, **poetry**, and **uv** for better performance, reproducibility, and ease of use.

Summary:

- Choose the tool that fits your workflow and project needs.

- Stay curious—new tools like `uv` are making Python development faster and more reliable.

A good package manager and environment strategy will save you time, prevent headaches, and make your code more reproducible and shareable.

2.7 Independent Study: Practice Environment Setup with `pip`

Objective: Build hands-on skills in creating, managing, and verifying Python environments and packages using `pip`.

2.7.1 Instructions

Step 1: Create Your Workspace

- Make a folder named `test_pip_env`.
- Open the folder in VS Code.

Step 2: Create a Virtual Environment

- create a `.venv` environment

Step 3: Install Required Packages

- With the environment active, install packages using `pip`:

```
pip install numpy pandas
```

Step 4: Export Your Environment Configuration

- Save your environment's package list to a file:

```
pip freeze > stat303_env.txt
```

Step 5: Deactivate and Remove the Environment

- Deactivate the environment:

```
deactivate
```

- Remove the environment folder to clean up.

Step 6: Recreate the Environment from the Exported File

- Create a new environment and activate it.

- Install packages from your saved file:

```
pip install -r stat303_env.txt
```

- Verify that `numpy`, `pandas`, `matplotlib`, and (if installed) `scikit-learn` are present.

Step 7: Run a Jupyter Notebook

- Create a notebook in the recreated environment.
- Import `numpy`, `pandas`, `matplotlib`, and `scikit-learn`.
- Print the versions of these packages to confirm setup.
- Generate html file from the notebook using `quarto`

This exercise will help you master environment management and reproducibility—key skills for any data scientist.

2.8 Resources

- [Tutorials in Python Packaging User Guide](#)
- [Poetry Documentation](#)
- [uv GitHub page](#)

3 Enhancing Workflow in Jupyter Notebooks

<IPython.core.display.Image object>

3.1 Learning Objectives

In this lecture, we'll explore how to optimize your workflow in Jupyter notebooks by leveraging **magic commands** and **shell commands**, understanding **file paths**, and interacting with the **filesystem** using the **os** module.

By completing this lecture, you will be able to:

- Run shell commands directly within a notebook.
- Navigate and manipulate files and directories efficiently.
- Integrate external tools and scripts seamlessly into your workflow.

By mastering these techniques, you'll be able to work more efficiently and handle complex data science tasks with ease.

3.2 Magic Commands

3.2.1 What are Magic Commands?

Magic commands in Jupyter are shortcuts that extend the functionality of the notebook environment. There are two types of magic commands:

- **Line magics:** Commands that operate on a single line.
- **Cell magics:** Commands that operate on the entire cell.

You can access the full list of magic commands by typing:

```
# show all the available magic commands on the system
%lsmagic
```

Available line magics:

```
%alias %alias_magic %autoawait %autocall %automagic %autosave %bookmark %cd %clear %
```

Available cell magics:

```
%%! %%HTML %%SVG %%bash %%capture %%cmd %%code_wrap %%debug %%file %%html %%javasc
```

Automagic is ON, % prefix IS NOT needed for line magics.

3.2.2 Line Magic

In Jupyter notebooks, line magic commands are invoked by placing a single percentage sign (%) in front of the statement, allowing for quick, inline operations.

3.2.2.1 %time: Timing the execution of code

In data science, it is often crucial to evaluate the performance of specific code snippets or algorithms, and the %time magic command provides a simple and efficient way to measure the execution time of individual statements, helping you identify bottlenecks and optimize your code for better performance.

```
def my_dot(a, b):
    """
    Compute the dot product of two vectors

    Args:
        a (ndarray (n,)): input vector
        b (ndarray (n,)): input vector with same dimension as a

    Returns:
        x (scalar):
    """
    x=0
    for i in range(a.shape[0]):
        x = x + a[i] * b[i]
    return x
```

```
import numpy as np
np.random.seed(1)
a = np.random.rand(10000000) # very large arrays
b = np.random.rand(10000000)
```

Let's use `%time` to measure the execution time of a single line of code.

```
# Example: Timing a list comprehension
%time np.dot(a, b)
```

```
CPU times: total: 15.6 ms
Wall time: 32.9 ms
```

```
2501072.5816813153
```

```
%time my_dot(a, b)
```

```
CPU times: total: 1.88 s
Wall time: 1.86 s
```

```
2501072.5816813707
```

To capture the output of `%time` (or `%timeit`), you cannot directly assign it to a variable as it's a magic command that prints the result to the notebook's output. However, you can use Python's built-in `time` module to manually time your code and assign the execution time to a variable.

Here's how you can do it using the `time` module:

```
import time
tic = time.time() # capture start time
c = np.dot(a, b)
toc = time.time() # capture end time

print(f"np.dot(a, b) = {c:.4f}")
print(f"Vectorized version duration: {1000*(toc-tic):.4f} ms ")

tic = time.time() # capture start time
c = my_dot(a,b)
toc = time.time() # capture end time

print(f"my_dot(a, b) = {c:.4f}")
print(f"loop version duration: {1000*(toc-tic):.4f} ms ")

del(a);del(b) #remove these big arrays from memory
```

```
np.dot(a, b) = 2501072.5817
Vectorized version duration: 2.9922 ms
my_dot(a, b) = 2501072.5817
loop version duration: 2004.3225 ms
```

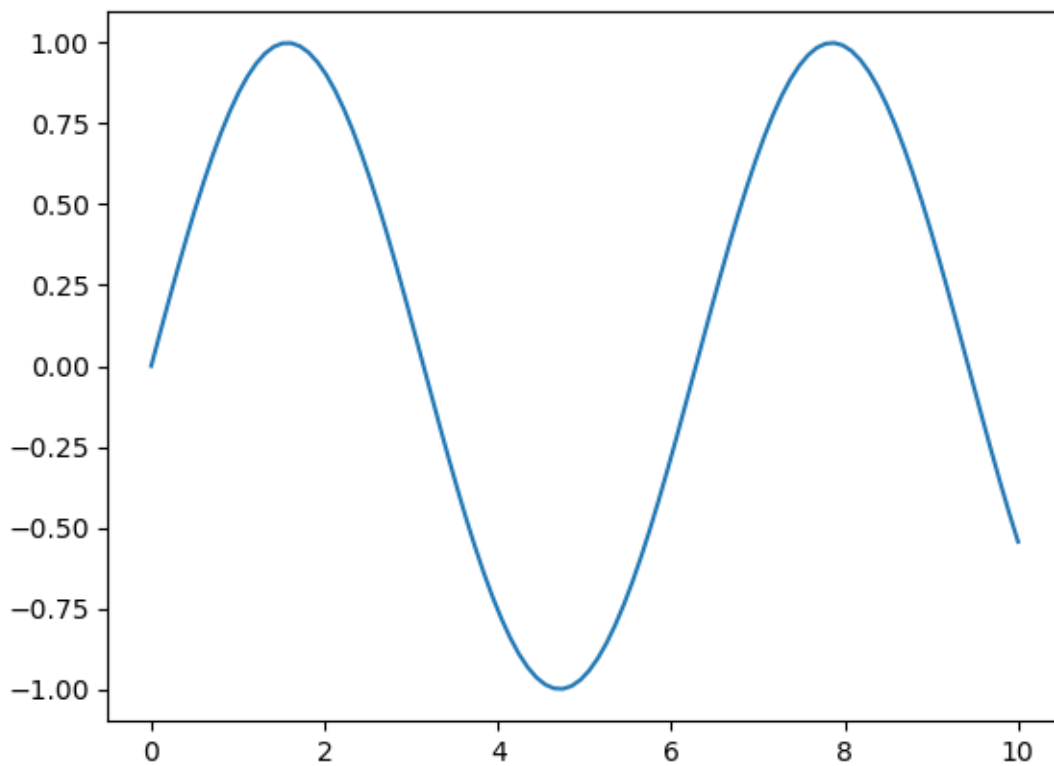
3.2.2.2 %matplotlib inline: Displaying plots inline

This command allows you to embed plots within the notebook.

```
# Example: Using %matplotlib inline to display a plot
import matplotlib.pyplot as plt
import numpy as np

%matplotlib inline

x = np.linspace(0, 10, 100)
y = np.sin(x)
plt.plot(x, y);
```



3.2.3 Cell Magic Commands

While cell magic commands are denoted with double percentage signs (%%) at the beginning of the cell, enabling you to apply commands to the entire cell for more complex tasks. Note that a cell magic command in Jupyter notebook has to be the first line in the code cell!

3.2.3.1 %%time: Timing cell execution

This cell magic is useful for measuring the execution time of an entire cell.

```
%%time
# Example: Timing a cell with matrix multiplication

A = np.random.rand(1000, 1000)
B = np.random.rand(1000, 1000)
C = np.dot(A, B)
```

CPU times: total: 219 ms

Wall time: 23 ms

Question: there are several timing magic commands that can be confusing due to their similarities, they are %time,%timeit, %%time, and %timeit. Do your own research on the differences among them

3.3 Shell Commands in Jupyter Notebooks

What are Shell Commands?

Shell commands let you interact directly with your computer's operating system from within a Jupyter notebook. This means you can manage files, check your environment, and run system tools without leaving your notebook. To run a shell command in Jupyter, start the line with an exclamation mark (!). For example, !ls lists files on macOS/Linux, while !dir does the same on Windows.

How does it work?

- Python code is executed by the IPython kernel inside the notebook.
- Shell commands (lines starting with !) are sent to your computer's shell (like Bash, Command Prompt, or PowerShell), not to Python.
- This separation means you can use both Python and shell commands in the same notebook, but they run in different environments.

3.3.1 Using Shell Commands

3.3.1.1 Print the current working directory

To see where your notebook is running, use: - On macOS/Linux: `!pwd` - On Windows: `!cd`

This helps you understand where files will be saved or loaded from.

```
!cd
```

```
!cd
```

```
c:\Users\lsi8012\mydev\STAT303-1-class-notes
```

3.3.1.2 `!ls` (`!dir` on Windows): Listing files and directories

To view all files and folders in your current directory, use a shell command:

- On macOS/Linux: `!ls`
- On Windows: `!dir`

This is useful for quickly checking what data, scripts, or notebooks are available in your workspace. If you want to see hidden files (those starting with a dot), use `!ls -a` on macOS/Linux or `!dir /a` on Windows.

Tip: If you get an error, double-check that you are using the correct command for your operating system.

3.4 Installing Required Packages Within Your Notebook

When working in Jupyter notebooks, you often need to install new Python packages. There are two main ways to do this:

1. Shell Command (not recommend):

- Use an exclamation mark (!) before the command:
 - `!pip install package_name`
- This runs the command in your system shell, which may not always install the package in the same environment as your notebook kernel.

2. Magic Command:

- Use a percent sign (%) before the command:
 - `%pip install package_name`
- This is Jupyter-specific and ensures the package is installed in the environment where your notebook is running.

Example:

- `%pip install numpy` # Installs numpy in the notebook’s environment
- `!pip install numpy` # Installs numpy using the system shell (may differ from notebook environment)

Unlike shell commands, the `%pip` magic command is designed specifically for Jupyter notebooks. It automatically installs packages into the exact environment your notebook kernel is using, reducing confusion and installation errors.

```
%pip install numpy
```

```
Requirement already satisfied: numpy in c:\users\lsi8012\appdata\local\anaconda3\lib\site-pa
Note: you may need to restart the kernel to use updated packages.
```

In most cases, you don’t need to type % before `pip install numpy` because Jupyter’s “automagic” feature is enabled by default. Automagic lets you use line magics without the % (unless their name conflicts with Python variables or commands).

How automagic works:

- If you type `pip install numpy`, Jupyter will interpret it as `%pip install numpy` automatically.

Best practice:

- Use `%pip install ...` for clarity and to avoid conflicts.
- Automagic is convenient, but explicit magic commands are safer in shared or complex notebooks.

Note: Cell magics still require the %% prefix.

3.5 File Paths in Python

Understanding how to specify file paths in Python is essential for loading and saving data. File paths tell Python where to find or store your files (e.g., datasets, results, or scripts).

There are two main types of file paths: **absolute paths** and **relative paths**.

- **Absolute Path:** Gives the complete location of a file or folder from the root of your file system. It always points to the same place, no matter where your code is running.
- **Relative Path:** Specifies the location of a file or folder in relation to your current working directory. It is shorter and more flexible, making your code easier to share and reuse.

Choosing the right type of path helps avoid errors and makes your code more portable across different computers and operating systems.

3.5.1 Absolute Path

An **absolute path** provides the full address to a file or directory, starting from the root of your system. It does **not** depend on where your code is running.

Example (Windows):

```
# Absolute path example (Windows)
file_path = r"C:\Users\Username\Documents\data.csv"
```

```
!conda env list
```

```
# conda environments:
#
base                  *  C:\Users\lsi8012\AppData\Local\anaconda3
```

The path associated with each conda env is absolute path.

3.5.2 Relative Path

A **relative path** describes the location of a file or folder in relation to your current working directory. It is shorter, more flexible, and makes your code easier to share and run on different computers.

Relative paths are especially useful in projects with organized folder structures, because they allow you to move your code and data together without changing file references.

To find your current working directory in your notebook, you can use a magic command:

- Magic command: `%pwd`

Knowing your current working directory helps you understand where Python will look for files when using relative paths.

```
# Magic command
%pwd
```

```
'c:\\Users\\lsi8012\\Documents\\Courses\\FA24\\DataScience_Intro_python_fa24_Sec20_21'
```

Example of a Relative Path:

```
# Example of relative path
file_path = "sample.txt" # Relative to the current directory
```

The relative path `sample.txt` means that there is a file in the current working directory.

3.5.3 Methods to Specify File Paths in Windows

Specifying file paths correctly is crucial for avoiding errors and making your code portable. Windows uses backslashes (`\`) to separate folders, but in Python, a single backslash is an escape character (e.g., `\n` for newline), which can lead to mistakes if not handled properly.

Here are four reliable ways to specify file paths in Windows:

3.5.3.1 Method 1: Using Escaped Backslashes

Use double backslashes (`\\`) to prevent Python from interpreting them as escape characters.

```
file_path = "C:\\Users\\Username\\Documents\\data.csv"
```

3.5.3.2 Method 2: Using Raw Strings

Prefix the path with `r` to tell Python to treat backslashes as literal characters.

```
file_path = r"C:\Users\Username\Documents\data.csv"
```

3.5.3.3 Method 3: Using forward slashes (/)

Python accepts forward slashes (/) in file paths, even on Windows. This is often the simplest and most portable method.

```
file_path = "C:/Users/Username/Documents/data.csv"
```

3.5.3.4 Method 4: Using `os.path.join`

Use `os.path.join()` to build paths programmatically. This method automatically uses the correct separator for your operating system, making your code cross-platform.

Using these methods helps prevent bugs, makes your code easier to share, and ensures it works on different computers and operating systems.

```
import os
file_path = os.path.join("C:", "Users", "Username", "Documents", "data.csv")
```

This method works across different operating systems because `os.path.join` automatically uses the correct path separator (\ for Windows and /for Linux/Mac).

3.5.4 File paths in macOS and Linux

macOS and Linux use forward slashes (/) as path separators, which is exactly what Python expects by default. This means you can specify file paths directly, like `/Users/yourname/Documents/data.csv`, without worrying about escape characters or compatibility issues.

Why is this helpful? - Forward slashes work seamlessly in Python on macOS, Linux, and even Windows. - You avoid common errors caused by backslashes (which are escape characters in Python). - Code written with forward slashes is portable and recommended for all platforms.

3.5.4.1 Best Practices for File Paths in Data Science

- Prefer **relative paths** within your project folders—this makes your code portable and easy to share or move.
- Use **absolute paths** only for files outside your project or when you need a fixed location.
- Always check your **current working directory** before reading or writing files to avoid confusion and errors.

- Avoid hardcoding file paths directly in your code; use variables or configuration files for flexibility.
- For cross-platform compatibility, use **forward slashes (/)** in file paths or build paths programmatically with `os.path.join()`.
- Document your file structure and path conventions in your project README to help collaborators.
- When sharing code, test file paths on both Windows and macOS/Linux to ensure portability.

3.6 Interacting with the OS and Filesystem

In data science projects, you often work with data files (such as CSV, Excel, or JSON) stored in various folders. Managing these files efficiently is essential for reproducible workflows and organized projects.

The Python `os` module provides powerful tools to interact with your operating system and manage files and directories directly from your notebook. With `os`, you can: - Check your current working directory - List files and folders - Create, rename, or delete directories - Move between folders - Check if files or folders exist before using them

Let's import the `os` module and explore some of its most useful functions for data science tasks.

```
import os
```

We can get the location of the current working directory using the `os.getcwd()` function.

```
os.getcwd()
```

```
'c:\\Users\\lsi8012\\mydev\\STAT303-1-class-notes'
```

The command `os.chdir('..')` in Python changes the current working directory to the parent directory of the current one.

Note that `..` as the path notation for the parent directory is universally true across all major operating systems, including Windows, macOS, and Linux. It allows you to move one level up in the directory hierarchy, which is very useful when navigating directories programmatically, especially in scripts where directory traversal is needed.

```
os.chdir('..')
```

```
os.getcwd()
```

```
'c:\\Users\\lsi8012\\mydev'
```

`os.chdir()` is used to change the current working directory.

For example: `os.chdir('./week2')`

`./week2` is a relative path:

- `.` refers to the current directory.
- `week2` is a folder inside the current directory.

The `os.listdir()` function in Python returns a list of all files and directories in the specified path. If no path is provided, it returns the contents of the current working directory.

```
os.listdir()
```

```
['limonstar-web',  
'llm2005spring',  
'llm2005spring_back',  
'mcdc_2025su',  
'mcdc_2025su_new',  
'STAT201',  
'STAT303-1-class-notes',  
'STAT303-2-class-notes',  
'STAT303-3-class-notes',  
'stat359_su25',  
'stat362']
```

Check whether a specific folder/file exist in the current working directory

```
'data' in os.listdir('.')
```

```
False
```

3.7 Navigate between directories and understand . and .. in paths

In Python, you can use the `os` module to work with directories and paths, and it's important to understand how relative paths work:

- `.` refers to the **current directory**
- `..` refers to the **parent directory**

This knowledge is essential for organizing files and writing portable code.

From the command line, you can use the `cd` (change directory) command to move between folders:

- `cd <folder_name>` moves **down** into a child folder
- `cd ..` moves **up** to the parent folder

By combining these commands, you can navigate to the desired location.

3.8 Independent Study

3.8.1 Setting Up Your Data Science Workspace

To reinforce and apply the skills from this lecture, complete the following hands-on tasks:

1. Set Up Your Workspace

- Create a folder named `STAT303-1` to organize all course materials.
- Set up a dedicated `.venv` environment for your coursework to keep dependencies isolated.
- Organize your files into subfolders for datasets, assignments, project, quizzes, and lectures. The layout looks like below

```
stat303-1/  
  .venv/           # Python virtual environment  
  datasets/  
  project/  
  homework assignments/  
  small assignments/  
  lectures/
```

- Use the `os` module or shell commands in your notebook to create these directories programmatically.

2. Practice Magic Commands

- Use `%timeit` to measure the execution time of a simple Python function in your notebook.
- Explore `%lsmagic` to discover all available magic commands and experiment with a few.

3. Run Shell Commands

- Use `!ls` (or `!dir` on Windows) to list the contents of the directories you created.
- Use `!pwd` (or `!cd` on Windows) to print your current working directory.

4. Navigate between directoiress using `.` and `..`

Tip: Document your process and any issues you encounter. This will help you troubleshoot and share your workflow with others.

3.9 References and Further Learning

- **IPython Magic Commands**

[IPython Documentation – Magic Commands](#)

A complete list of available line and cell magics, such as `%time`, `%pip`, and `%%bash`.

- **File and Directory Management in Python**

[Python `os` module documentation](#)

Covers functions like `os.getcwd()`, `os.chdir()`, and path handling.

Part II

Python refresher

4 Python Basics

<IPython.core.display.Image object>

Before diving into new material, this chapter provides a quick refresher on key Python concepts covered in STAT201, which is the prerequisite for this course. If you have not taken STAT201, or if you feel you need to strengthen your Python skills, please review chapters 3–8 in the [STAT201 book](#). A solid understanding of Python basics will help you succeed in this course and make it easier to follow the advanced topics ahead.

4.1 Python Variables

Variables are fundamental building blocks in Python—they allow you to store, update, and reference data throughout your code. Choosing clear and descriptive variable names makes your code easier to read, debug, and share with others.

4.1.1 Rules for variable names

- Variable names must start with a letter (a–z, A–Z) or an underscore (_). They cannot begin with a number.
- Names can include letters, digits, and underscores (`a_variable`, `profit_margin`, `the_3_musketeers`).
- Variable names are case-sensitive: `score`, `Score`, and `SCORE` are all different variables.
- Avoid using Python reserved words (like `for`, `if`, `class`, etc.) as variable names.

Tip: Use descriptive names that reflect the purpose of the variable. This helps you and others understand your code at a glance.

Here are some examples of good variable names:

```
# Examples of valid variable names

a_variable = 23
is_today_Saturday = False
my_favorite_car = "Delorean"
the_3_musketeers = ["Athos", "Porthos", "Aramis"]
```

If you use an invalid variable name, Python will raise a `SyntaxError` and stop running your code. Always follow the naming rules to avoid these errors and keep your code readable.

4.1.2 Dynamic Typing in Python Variables

Python variables are dynamically typed, meaning you don't need to declare their type before using them. You can assign a value of any type to a variable, and even change its type later in your code:

```
x = 5          # x is an integer
x = "cat"      # now x is a string
```

To check the type of a variable, use the built-in `type()` function:

```
type(x)
```

This flexibility makes Python easy to use, but it's important to keep track of your variable types to avoid confusion in your code.

```
# Print the variables defined above their values and their types
print("a_variable:", a_variable, "| type:", type(a_variable))
print("is_today_Saturday:", is_today_Saturday, "| type:", type(is_today_Saturday))
print("my_favorite_car:", my_favorite_car, "| type:", type(my_favorite_car))
print("the_3_musketeers:", the_3_musketeers, "| type:", type(the_3_musketeers))
```

```
a_variable: 23 | type: <class 'int'>
is_today_Saturday: False | type: <class 'bool'>
my_favorite_car: Delorean | type: <class 'str'>
the_3_musketeers: ['Athos', 'Porthos', 'Aramis'] | type: <class 'list'>
```

4.1.3 Multiple Variable Assignment in Python

Python allows you to assign values to several variables at once in a single line. This technique is especially useful for initializing related variables and can make your code cleaner and more efficient.

Example:

```
color1, color2, color3 = "red", "green", "blue"
```

After this assignment: - color1 is “red” - color2 is “green” - color3 is “blue”

This approach works for any number of variables, as long as the number of values matches the number of variable names.

```
color1, color2, color3 = "red", "green", "blue"
print(color1)
print(color3)
```

```
red
blue
```

The same value can be assigned to multiple variables by chaining multiple assignment operations within a single statement.

```
color4 = color5 = color6 = "magenta"
print(color4)
print(color6)
```

```
magenta
magenta
```

4.2 Built-in data types

Python has several built-in data types for storing different kinds of information in variables.

```
<IPython.core.display.Image object>
```

Primitive Types

In Python, **integers, floats, booleans, and None** are often called *primitive data types* because they represent a single value.

Container (Data Structure) Types

Types such as **strings, lists, tuples, sets, and dictionaries** are *containers* because they can hold multiple values (characters in a string, items in a list, key-value pairs in a dictionary, etc.). We'll explore these container types in more detail in the next chapter.

Identifying Types

You can check the type of any object using Python's built-in `type()` function. For example:

```
print(type(42))          # int
print(type(3.14))        # float
print(type(True))        # bool
print(type(None))         # NoneType
print(type("hello"))     # str (a sequence / container of characters)
print(type([1, 2, 3]))   # list
```

```
<class 'int'>
<class 'float'>
<class 'bool'>
<class 'NoneType'>
<class 'str'>
<class 'list'>
```

4.3 Python Standard Library

The [Python Standard Library](#) provides a wide range of modules and built-in functions that support everyday programming tasks. These tools allow you to write code that is both efficient and readable without reinventing common functionality.

Examples of built-in functions: - `print()`: Displays output to the screen. - `len()`: Returns the length of an object (like a list or string). - `type()`: Shows the type of a variable. - `sum()`: Adds up all items in an iterable (like a list). - `range()`: Generates a sequence of numbers, often used in loops.

You can explore more built-in functions and modules in the official documentation. Using these tools makes your code more readable and powerful.

Example:

`range()`: The `range()` function returns a sequence of evenly-spaced integer values. It is commonly used in `for` loops to define the sequence of elements over which the iterations are performed.

Below is an example where the `range()` function is used to create a sequence of whole numbers upto 10:

```
print(list(range(1,10)))
```

```
[1, 2, 3, 4, 5, 6, 7, 8, 9]
```

Date and Time:

Python includes a powerful built-in module called `datetime` for working with dates and times. This module lets you create, manipulate, and format date/time objects easily.

- You can get the current date and time.
- You can perform arithmetic with dates (e.g., add days, subtract dates).
- You can format dates and times for display or parsing.

This is essential for tasks like timestamping data, scheduling, or analyzing time series.

```
import datetime as dt
```

```
#Defining a date-time object  
dt_object = dt.datetime(2022, 9, 20, 11,30,0)
```

Information about date and time can be accessed with the relevant attribute of the `datetime` object.

```
dt_object.day, dt_object.year
```

```
(20, 2022)
```

Formatting Dates and Times:

The `strftime` method in the `datetime` module lets you convert a `datetime` object into a readable string using custom formats. This is useful for displaying dates in a way that matches your needs (e.g., for reports, logs, or user interfaces).

Common format codes include: - `%Y`: 4-digit year (e.g., 2025) - `%m`: 2-digit month (01-12) - `%d`: 2-digit day (01-31) - `%H`: Hour (00-23) - `%M`: Minute (00-59) - `%S`: Second (00-59)

See the [Python documentation](#) for more formatting options.

```
dt_object.strftime('%m/%d/%Y')
```

```
'09/20/2022'
```

```
dt_object.strftime('%m/%d/%y %H:%M')
```

```
'09/20/22 11:30'
```

```
dt_object.strftime('%h-%d-%Y')
```

'Sep-20-2022'

4.4 Third-Party Packages (Libraries)

While Python has many useful [built-in functions](#) like `print()`, `abs()`, `max()`, and `sum()`, these are often not enough for data analysis. Third-party libraries extend Python's capabilities and are essential for scientific computing and data science.

Popular libraries and their main uses:

1. **NumPy**: Efficient numerical operations, arrays, and mathematical functions. Essential for scientific and data analysis tasks.
2. **Pandas**: Powerful data manipulation and analysis. DataFrames and Series make reading, cleaning, and transforming data easy.
3. **Matplotlib & Seaborn**: Data visualization. Matplotlib creates a wide range of plots; Seaborn builds on Matplotlib for attractive statistical graphics.
4. **SciPy**: Advanced scientific computing—optimization, integration, statistics, and more.
5. **Scikit-learn**: Machine learning—tools for preprocessing, classification, regression, clustering, and model evaluation.
6. **Statsmodels**: Statistical modeling and inference (focuses on explanation, not just prediction).

How to use these libraries: 1. **Install the libraries** : Covered by Chapter 2 and 3 2. **Import the libraries** in your Python script or Jupyter notebook. 3. **Use their functions and classes** to analyze data, visualize results, and build models.

These libraries form the foundation of modern data science workflows in Python. You will gain plenty of hands-on experience with each of them throughout this course sequence.

4.4.1 Importing a Library

Use the `import` keyword to bring a library into your Python code.

Example:

```
import numpy as np
```

Aliases like `np` make code shorter and easier to read.

```
import numpy as np
np.arange(8)
```

```
array([0, 1, 2, 3, 4, 5, 6, 7])
```

Importing in Python: Key Styles

- Import a whole module:

```
import math
```

- Import specific items:

```
from random import randint
```

- Use an alias:

```
import pandas as pd
```

- Rename an imported item:

```
from os.path import join as join_path
```

Pick the style that fits your needs and keeps your code readable.

4.5 User-defined functions

A function is a reusable set of instructions that takes one or more inputs, performs some operations, and often returns an output. **Indeed, while python's standard library and ecosystem libraries offer a wealth of pre-defined functions for a wide range of tasks, there are situations where defining your own functions is not just beneficial but necessary.**

4.5.1 Creating and using functions

You can define a new function using the `def` keyword.

```
def say_hello():
    print('Hello there!')
    print('How are you?')
```


Note the round brackets or parentheses () and colon : after the function's name. Both are essential parts of the syntax. The function's *body* contains an indented block of statements. The statements inside a function's body are not executed when the function is defined. To execute the statements, we need to *call* or *invoke* the function.

```
say_hello()
```

```
Hello there!  
How are you?
```

```
def say_hello_to(name):  
    print('Hello ', name)  
    print('How are you?')
```

```
say_hello_to('Lizhen')
```

```
Hello Lizhen  
How are you?
```

```
name = input ('Please enter your name: ')  
say_hello_to(name)
```

```
Please enter your name: George  
Hello George  
How are you?
```

4.5.2 Variable scope: Local and global Variables

Local variable: When we declare variables inside a function, these variables will have a local scope (within the function). We cannot access them outside the function. These types of variables are called local variables. For example,

```
def greet():  
    message = 'Hello' # local variable  
    print('Local', message)  
greet()
```

```
Local Hello
```

```
# print(message) # try to access message variable outside greet() function, uncomment this line
```

As `message` was defined within the function `greet()`, it is local to the function, and cannot be called outside the function.

Global variable: A variable declared outside of the function or in global scope is known as a global variable. This means that a global variable can be accessed inside or outside of the function.

Let's see an example of how a global variable is created.

```
message = 'Hello' # declare global variable

def greet():
    print('Local', message) # declare local variable

greet()
print('Global', message)
```

```
Local Hello
Global Hello
```

4.5.3 Function Arguments

4.5.3.1 Named Arguments

When calling functions with multiple arguments, using *named* arguments improves clarity and reduces mistakes. You can also split long function calls across multiple lines for readability.

Example:

```
def greet(name, message):
    print(f"{message}, {name}!")

greet(name="Alice", message="Hello")
```

Named arguments make your code easier to understand and maintain.

Here is an example:

```
def loan_emi(amount, duration, rate, down_payment=0):
    loan_amount = amount - down_payment
    emi = loan_amount * rate * ((1+rate)**duration) / (((1+rate)**duration)-1)
    return emi
```

```
emi1 = loan_emi(
    amount=1260000,
    duration=8*12,
    rate=0.1/12,
    down_payment=3e5
)
```

```
emi1
```

```
14567.19753389219
```

4.5.3.2 Optional Arguments

Functions with optional arguments offer more flexibility in how you can use them. You can call the function with or without the argument, and if there is no argument in the function call, then a default value is used.

```
emi2 = loan_emi(
    amount=1260000,
    duration=8*12,
    rate=0.1/12)
```

```
emi2
```

```
19119.4467632335
```

4.5.3.3 *args and **kwargs

We can pass a variable number of arguments to a function using two special symbols in Python:

- ***args** for variable-length **positional arguments**
- ****kwargs** for variable-length **keyword arguments**

This is useful when you want your function to accept a variety of arguments.

```
def myFun(*args,**kwargs):
    print("args: ", args)
    print("kwargs: ", kwargs)

# Now we can use both *args ,**kwargs
# to pass arguments to this function :
myFun('John',22,'cs',name="John",age=22,major="cs")
```

```
args: ('John', 22, 'cs')
kwargs: {'name': 'John', 'age': 22, 'major': 'cs'}
```

4.6 Control Flow

Control flow statements (like if, for, and while) let you decide how your code runs based on conditions and loops. Control flow in Python includes both conditional statements and iteration loops to manage program logic.

4.6.1 Conditional Statements

As in other languages, python has [built-in keywords](#) that provide conditional flow of control in the code.

4.6.1.1 Branching with if, else and elif

One of the most powerful features of programming languages is *branching*: the ability to make decisions and execute a different set of statements based on whether one or more conditions are true.

The if statement

In Python, branching is implemented using the if statement, which is written as follows:

```
if condition:
    statement1
    statement2
```

The `condition` can be a value, variable or expression. If the condition evaluates to `True`, then the statements within the *if block* are executed. Notice the four spaces before `statement1`, `statement2`, etc. The spaces inform Python that these statements are associated with the `if` statement above. This technique of structuring code by adding spaces is called *indentation*.

Indentation: Python relies heavily on *indentation* (white space before a statement) to define code structure. This makes Python code easy to read and understand. You can run into problems if you don't use indentation properly. Indent your code by placing the cursor at the start of the line and pressing the `Tab` key once to add 4 spaces. Pressing `Tab` again will indent the code further by 4 more spaces, and press `Shift+Tab` will reduce the indentation by 4 spaces.

For example, let's write some code to check and print a message if a given number is even.

```
a_number = 34
```

```
if a_number % 2 == 0:
    print("We're inside an if block")
    print('The given number {} is even.'.format(a_number))
```

```
We're inside an if block
The given number 34 is even.
```

The else statement

We may want to print a different message if the number is not even in the above example. This can be done by adding the `else` statement. It is written as follows:

```
if condition:
    statement1
    statement2
else:
    statement4
    statement5
```

If `condition` evaluates to `True`, the statements in the `if` block are executed. If it evaluates to `False`, the statements in the `else` block are executed.

```
if a_number % 2 == 0:
    print('The given number {} is even.'.format(a_number))
else:
    print('The given number {} is odd.'.format(a_number))
```

The given number 34 is even.

The elif statement

Python also provides an `elif` statement (short for “else if”) to chain a series of conditional blocks. The conditions are evaluated one by one. For the first condition that evaluates to `True`, the block of statements below it is executed. The remaining conditions and statements are not evaluated. So, in an `if`, `elif`, `elif...` chain, at most one block of statements is executed, the one corresponding to the first condition that evaluates to `True`.

```
today = 'Wednesday'
```

```
if today == 'Sunday':
    print("Today is the day of the sun.")
elif today == 'Monday':
    print("Today is the day of the moon.")
elif today == 'Tuesday':
    print("Today is the day of Tyr, the god of war.")
elif today == 'Wednesday':
    print("Today is the day of Odin, the supreme diety.")
elif today == 'Thursday':
    print("Today is the day of Thor, the god of thunder.")
elif today == 'Friday':
    print("Today is the day of Frigga, the goddess of beauty.")
elif today == 'Saturday':
    print("Today is the day of Saturn, the god of fun and feasting.")
```

Today is the day of Odin, the supreme diety.

In the above example, the first 3 conditions evaluate to `False`, so none of the first 3 messages are printed. The fourth condition evaluates to `True`, so the corresponding message is printed. The remaining conditions are skipped. Try changing the value of `today` above and re-executing the cells to print all the different messages.

Using if, elif, and else together

You can also include an `else` statement at the end of a chain of `if`, `elif...` statements. This code within the `else` block is evaluated when none of the conditions hold true.

```
a_number = 49
```

```

if a_number % 2 == 0:
    print('{} is divisible by 2'.format(a_number))
elif a_number % 3 == 0:
    print('{} is divisible by 3'.format(a_number))
elif a_number % 5 == 0:
    print('{} is divisible by 5'.format(a_number))
else:
    print('All checks failed!')
    print('{} is not divisible by 2, 3 or 5'.format(a_number))

```

```

All checks failed!
49 is not divisible by 2, 3 or 5

```

Non-Boolean Conditions

Note that conditions do not necessarily have to be booleans. In fact, a condition can be any value. The value is converted into a boolean automatically using the `bool` operator. Any value in Python can be converted to a Boolean using the `bool` function.

Only the following values evaluate to **False** (they are often called *falsy* values):

1. The value **False** itself
2. The integer **0**
3. The float **0.0**
4. The empty value **None**
5. The empty text **""**
6. The empty list **[]**
7. The empty tuple **()**
8. The empty dictionary **{}**
9. The empty set **set()**
10. The empty range **range(0)**

Everything else evaluates to **True** (a value that evaluates to **True** is often called a *truthy* value).

```

if '':
    print('The condition evaluted to True')
else:
    print('The condition evaluted to False')

```

```
The condition evaluted to False
```

```

if 'Hello':
    print('The condition evaluted to True')
else:
    print('The condition evaluted to False')

```

The condition evaluted to True

```

if { 'a': 34 }:
    print('The condition evaluted to True')
else:
    print('The condition evaluted to False')

```

The condition evaluted to True

```

if None:
    print('The condition evaluted to True')
else:
    print('The condition evaluted to False')

```

The condition evaluted to False

Nested conditional statements

The code inside an if block can also include an if statement inside it. This pattern is called **nesting** and is used to check for another condition after a particular condition holds true.

```

a_number = 15

```

```

if a_number % 2 == 0:
    print("{} is even".format(a_number))
    if a_number % 3 == 0:
        print("{} is also divisible by 3".format(a_number))
    else:
        print("{} is not divisibule by 3".format(a_number))
else:
    print("{} is odd".format(a_number))
    if a_number % 5 == 0:
        print("{} is also divisible by 5".format(a_number))
    else:
        print("{} is not divisibule by 5".format(a_number))

```



```
15 is odd
15 is also divisible by 5
```

Notice how the `print` statements are indented by 8 spaces to indicate that they are part of the inner `if/else` blocks.

Nested `if, else` statements are often confusing to read and prone to human error. It's good to avoid nesting whenever possible, or limit the nesting to 1 or 2 levels.

Shorthand if conditional expression

A frequent use case of the `if` statement involves testing a condition and setting a variable's value based on the condition.

Python provides a shorter syntax, which allows writing such conditions in a single line of code. It is known as a *conditional expression*, sometimes also referred to as a *ternary operator*. It has the following syntax:

```
x = true_value if condition else false_value
```

It has the same behavior as the following `if-else` block:

```
if condition:
    x = true_value
else:
    x = false_value
```

Let's try it out for the example above.

```
parity = 'even' if a_number % 2 == 0 else 'odd'

print('The number {} is {}'.format(a_number, parity))
```

The number 15 is odd.

The `pass` statement

`if` statements cannot be empty, there must be at least one statement in every `if` and `elif` block. We can use the `pass` statement to do nothing and avoid getting an error.

```
a_number = 9
```

```
# please uncomment the code below and see the error message
# if a_number % 2 == 0:

# elif a_number % 3 == 0:
#     print('{} is divisible by 3 but not divisible by 2')
```

As there must be at least one statement within the `if` block, the above code throws an error.

```
if a_number % 2 == 0:
    pass
elif a_number % 3 == 0:
    print('{} is divisible by 3 but not divisible by 2'.format(a_number))
```

9 is divisible by 3 but not divisible by 2

4.6.2 Loops

4.6.2.1 while loops

Another powerful feature of programming languages, closely related to branching, is running one or more statements multiple times. This feature is often referred to as *iteration* or *looping*, and there are two ways to do this in Python: using `while` loops and `for` loops.

`while` loops have the following syntax:

```
while condition:
    statement(s)
```

Statements in the code block under `while` are executed repeatedly as long as the `condition` evaluates to `True`. Generally, one of the statements under `while` makes some change to a variable that causes the condition to evaluate to `False` after a certain number of iterations.

Let's try to calculate the factorial of 100 using a `while` loop. The factorial of a number `n` is the product (multiplication) of all the numbers from 1 to `n`, i.e., $1*2*3*\dots*(n-2)*(n-1)*n$.

```

result = 1
i = 1

while i <= 10:
    result = result * i
    i = i+1

print('The factorial of 100 is: {}'.format(result))

```

The factorial of 100 is: 3628800

4.6.2.2 Infinite Loops

Suppose the condition in a `while` loop always holds true. In that case, Python repeatedly executes the code within the loop forever, and the execution of the code never completes. This situation is called an infinite loop. It generally indicates that you’ve made a mistake in your code. For example, you may have provided the wrong condition or forgotten to update a variable within the loop, eventually falsifying the condition.

If your code is *stuck* in an infinite loop during execution, just press the “Stop” button on the toolbar (next to “Run”) or select “Kernel > Interrupt” from the menu bar. This will *interrupt* the execution of the code. The following two cells both lead to infinite loops and need to be interrupted.

```
# INFINITE LOOP - INTERRUPT THIS CELL
```

```

result = 1
i = 1

while i <= 100:
    result = result * i
    # forgot to increment i

```

```
# INFINITE LOOP - INTERRUPT THIS CELL
```

```

result = 1
i = 1

while i > 0 : # wrong condition
    result *= i
    i += 1

```

4.6.2.3 break and continue statements

In Python, `break` and `continue` statements can alter the flow of a normal loop.

We can use the `break` statement within the loop's body to immediately stop the execution and `break` out of the loop. with the `continue` statement. If the condition evaluates to `True`, then the loop will move to the next iteration.

```
i = 1
result = 1

while i <= 100:
    result *= i
    if i == 42:
        print('Magic number 42 reached! Stopping execution..')
        break
    i += 1

print('i:', i)
print('result:', result)
```

```
Magic number 42 reached! Stopping execution..
i: 42
result: 1405006117752879898543142606244511569936384000000000
```

```
i = 1
result = 1

while i < 8:
    i += 1
    if i % 2 == 0:
        print('Skipping {}'.format(i))
        continue
    print('Multiplying with {}'.format(i))
    result = result * i

print('i:', i)
print('result:', result)
```

```
Skipping 2
Multiplying with 3
Skipping 4
```

```
Multiplying with 5
Skipping 6
Multiplying with 7
Skipping 8
i: 8
result: 105
```

In the example above, the statement `result = result * i` inside the loop is skipped when `i` is even, as indicated by the messages printed during execution.

Logging: The process of adding `print` statements at different points in the code (*often within loops and conditional statements*) for inspecting the values of variables at various stages of execution is called logging. As our programs get larger, they naturally become prone to human errors. Logging can help in verifying the program is working as expected. In many cases, `print` statements are added while writing & testing some code and are removed later.

Task: Guess the output and explain it.

```
# Use of break statement inside the loop

for val in "string":
    if val == "i":
        break
    print(val)

print("The end")
```

```
s
t
r
The end
```

```
# Program to show the use of continue statement inside loops

for val in "string":
    if val == "i":
        continue
    print(val)

print("The end")
```

s
t
r
n
g
The end

4.6.2.4 for loops

A `for` loop is used for iterating or looping over sequences, i.e., lists, tuples, dictionaries, strings, and *ranges*. `For` loops have the following syntax:

```
for value in sequence:  
    statement(s)
```

The statements within the loop are executed once for each element in `sequence`. Here's an example that prints all the element of a list.

```
days = ['Monday', 'Tuesday', 'Wednesday', 'Thursday', 'Friday']  
  
for day in days:  
    print(day)
```

Monday
Tuesday
Wednesday
Thursday
Friday

```
# Looping over a string  
for char in 'Monday':  
    print(char)
```

M
o
n
d
a
y

```
# Looping over a dictionary
person = {
    'name': 'John Doe',
    'sex': 'Male',
    'age': 32,
    'married': True
}

for key, value in person.items():
    print("Key:", key, ",", "Value:", value)
```

```
Key: name , Value: John Doe
Key: sex , Value: Male
Key: age , Value: 32
Key: married , Value: True
```

4.6.2.5 Iterating using range

The `range` function is used to create a sequence of numbers that can be iterated over using a `for` loop. It can be used in 3 ways:

- `range(n)` - Creates a sequence of numbers from 0 to `n-1`
- `range(a, b)` - Creates a sequence of numbers from `a` to `b-1`
- `range(a, b, step)` - Creates a sequence of numbers from `a` to `b-1` with increments of `step`

Let's try it out.

```
for i in range(4):
    print(i)
```

```
0
1
2
3
```

```
for i in range(3, 8):
    print(i)
```

```
3
4
5
6
7
```

```
for i in range(3, 14, 4):
    print(i)
```

```
3
7
11
```

Ranges are used for iterating over lists when you need to track the index of elements while iterating.

```
a_list = ['Monday', 'Tuesday', 'Wednesday', 'Thursday', 'Friday']

for i in range(len(a_list)):
    print('The value at position {} is {}'.format(i, a_list[i]))
```

```
The value at position 0 is Monday.
The value at position 1 is Tuesday.
The value at position 2 is Wednesday.
The value at position 3 is Thursday.
The value at position 4 is Friday.
```

4.7 Object Oriented Programming

Python is an object-oriented programming language. In layman terms, it means that every number, string, data structure, function, class, module, etc., exists in the python interpreter as a python object. An object may have attributes and methods associated with it. For example, let us define a variable that stores an integer:

```
var = 2
```

The variable `var` is an object that has attributes and methods associated with it. For example a couple of its attributes are `real` and `imag`, which store the real and imaginary parts respectively, of the object `var`:


```
print("Real part of 'var': ",var.real)
print("Real part of 'var': ",var.imag)
```

```
Real part of 'var':  2
Real part of 'var':  0
```

Attribute: An attribute is a value associated with an object, defined within the class of the object.

Method: A method is a function associated with an object, defined within the class of the object, and has access to the attributes associated with the object.

For looking at attributes and methods associated with an object, say `obj`, press tab key after typing `obj.`

Consider the example below of a class *example_class*:

```
class example_class:
    class_name = 'My Class'
    def my_method(self):
        print('Hello World!')

e = example_class()
```

In the above class, `class_name` is an attribute, while `my_method` is a method.

4.7.0.1 Call by Reference in Python

Python uses *call by object reference* (also called *call by sharing*). When you assign an object to a variable, the variable points to the object in memory—not a copy.

Changes made through one reference affect the original object. This is important to remember when working with mutable types like lists and dictionaries.

```
x = [5,3]
```

The variable name `x` is a reference to the memory location where the object `[5, 3]` is stored. Now, suppose we assign `x` to a new variable `y`:

```
y = x
```

In the above statement the variable name `y` now refers to the same object `[5,3]`. The object `[5,3]` does **not** get copied to a new memory location referred by `y`. To prove this, let us add an element to `y`:

```
y.append(4)
print(y)
```

```
[5, 3, 4]
```

```
print(x)
```

```
[5, 3, 4]
```

When we changed `y`, note that `x` also changed to the same object, showing that `x` and `y` refer to the same object, instead of referring to different copies of the same object.

5 Data Structures

<IPython.core.display.Image object>

In data science, you'll shape raw data **before modeling**. These built-ins are the foundation that explain why `Series/DataFrame` (pandas) and `ndarray` (NumPy) behave the way they do.

5.1 Primitives vs. Containers

- **Primitive (single value):** `int`, `float`, `bool`, `None`
- **Containers (hold multiple values):** `str`, `list`, `tuple`, `set`, `dict`

Tip: Check any object's type with `type(x)`.

5.2 Core Built-in Data Structures

5.2.1 Sequences (ordered, indexable)

Types: `list`, `tuple`, `str`

```
# list (mutable)
nums = [10, 20, 20, 30]
nums.append(40)      # mutate in place
nums[0] = 11         # item assignment
nums_slice = nums[1:3] # slicing

# tuple (immutable)
pt = (42.0, -1.5)     # cannot reassign pt[0]

# string (immutable sequence of characters)
s = "data"
s2 = s.upper()        # returns a new string; s unchanged
```

When to use:

- **list**: grow/shrink, frequent edits, ordered data
- **tuple**: fixed-size records, function returns, hashable as dict keys
- **str**: text processing (we'll also use `nltk` later)

5.2.2 Sets (unordered, unique)

Types: set

```
a = {1, 2, 2, 3}      # {1, 2, 3}
b = {3, 4}
a | b                 # union -> {1, 2, 3, 4}
a & b                 # intersection -> {3}
a - b                 # difference -> {1, 2}
```

{1, 2}

When to use:

- Deduplication, membership tests, fast set algebra.

5.2.3 Mappings (key–value)

Types: dict

```
student = {"name": "Alex", "year": 3, "major": "Stats"}
student["year"] = 4          # update
student["gpa"] = 3.7         # insert
for k, v in student.items(): # iterate keys & values
    print(k, "->", v)
```

```
name -> Alex
year -> 4
major -> Stats
gpa -> 3.7
```

When to use:

- Labeled data, lookups, configuration/state.

5.3 Iterables vs. Iterators

In Python, an **iterable** is any object capable of returning its members one at a time, such as lists, tuples, strings, and dictionaries. You can loop over iterables using a **for** loop.

An **iterator** is an object that represents a stream of data; it produces the next value when you call `next()` on it. Iterators are created from iterables using the `iter()` function.

Key differences: - Iterables can be looped over, but do not remember their position. - Iterators remember their position and can only be advanced one item at a time.

Example:

```
my_list = [1, 2, 3]
my_iter = iter(my_list)
print(next(my_iter)) # 1
print(next(my_iter)) # 2
print(next(my_iter)) # 3
```

Understanding the difference helps you write efficient loops and work with data streams in Python.

5.4 Common, Efficient Operations

5.4.1 Comprehensions

Comprehensions are a concise way to create lists, sets, or dictionaries from iterables by applying an expression to each item in an iterable (such as a list, tuple, or range) and optionally filtering the items based on a condition. They are a powerful and efficient way to generate new collections without the need for explicit loops.

Basic Syntax:

```
new_list = [expression for item in iterable if condition]
```

- **expression:** What you want to include in the new list (or set/dict).
- **item:** Represents each element in the iterable as the comprehension iterates through it.
- **iterable:** The source of elements (list, tuple, range, etc.).
- **condition (optional):** A filter to control which items are included. If omitted, all items are included.

Why use comprehensions? - More readable and concise than loops. - Often faster than equivalent for-loops. - Preferred for simple transformations and filtering.

List comprehension example:

Create a list that has squares of natural numbers from 5 to 15.

```
sqrt_natural_no_5_15 = [(x**2) for x in range(5,16)]  
print(sqrt_natural_no_5_15)
```

```
[25, 36, 49, 64, 81, 100, 121, 144, 169, 196, 225]
```

Create a list of tuples, where each tuple consists of a natural number and its square, for natural numbers ranging from 5 to 15.

```
sqrt_natural_no_5_15 = [(x,x**2) for x in range(5,16)]  
print(sqrt_natural_no_5_15)
```

```
[(5, 25), (6, 36), (7, 49), (8, 64), (9, 81), (10, 100), (11, 121), (12, 144), (13, 169), (14, 196)]
```

Creating a list of words that start with the letter 'a' in a given list of words.

```
words = ['apple', 'banana', 'avocado', 'grape', 'apricot']  
a_words = [word for word in words if word.startswith('a')]  
print(a_words)
```

```
['apple', 'avocado', 'apricot']
```

Set comprehension:

```
unique_lengths = {len(word) for word in ['cat', 'dog', 'mouse']}  
uniq_initials = {name[0].upper() for name in ["amy","ann","bob","amy"]}  
print(uniq_initials)  
print(unique_lengths)
```

```
{'B', 'A'}  
{3, 5}
```

Dictionary comprehension:

```
word_lengths = {word: len(word) for word in ['cat', 'dog', 'mouse']}
print(word_lengths)
```

```
{'cat': 3, 'dog': 3, 'mouse': 5}
```

Comprehensions are preferred for simple transformations and filtering.

5.4.2 Membership & Lookups

Membership operations let you check if an item exists in a collection (like a list, set, or dictionary). Use `in` and `not in` for these checks. Lookups retrieve values by key or index, and are fastest in sets and dictionaries.

Examples:

- `3 in [1, 2, 3] → True` (checks if 3 is in the list)
- `'a' in {'a': 1, 'b': 2} → True` (checks if 'a' is a key in the dictionary)
- `5 not in {1, 2, 3} → True` (checks if 5 is not in the set)

Tip:

- Dictionary and set lookups are very fast (constant time).
- List and tuple membership checks are slower (linear time).
- For safe dictionary lookups, use `.get(key)` to avoid errors if the key is missing.

5.4.3 Unpacking

Unpacking lets you assign elements of a collection (like a list, tuple, or string) to multiple variables in a single step. This makes your code more readable and concise, especially when working with structured data.

Basic Syntax:

```
pt = (3, 4)
x, y = pt # x=3, y=4

a, b, c = [1, 2, 3] # a=1, b=2, c=3
```

You can also use unpacking in loops and with function arguments.

Extended Unpacking: Python allows you to use the `*` operator to capture multiple elements:

```
# Extended unpacking:
first, *rest = [10, 20, 30, 40] # first=10, rest=[20, 30, 40]

a, *mid, z = [1,2,3,4]

print(rest)
print(mid)
```

```
[20, 30, 40]
[2, 3]
```

Unpacking in Loops:

```
pairs = [(1, 'a'), (2, 'b'), (3, 'c')]
for num, char in pairs:
    print(num, char)
```

```
1 a
2 b
3 c
```

Unpacking with Dictionaries:

```
student = {"name": "Alex", "year": 3}
for key, value in student.items():
    print(key, value)
```

```
name Alex
year 3
```

Why use unpacking?

- Makes code cleaner and more readable
- Quick variable assignment
- Useful for working with structured data (e.g., tuples, lists, dicts)
- Avoids manual indexing

Tip: Unpacking works with any iterable, including lists, tuples, strings, and even the results of functions that return multiple values.

5.4.4 Sorting in Python Iterables

Sorting is a common task when working with data. Python provides flexible ways to order lists, tuples, and other iterables.

5.4.4.1 `sorted()` (built-in function)

- Returns a **new sorted list** from any iterable.
- Works with lists, tuples, strings, sets, and more.
- Does **not modify** the original iterable.

```
nums = [3, 1, 4, 1, 5]
print(sorted(nums))      # [1, 1, 3, 4, 5]
print(nums)              # [3, 1, 4, 1, 5] (unchanged)

word = "python"
print(sorted(word))      # ['h', 'n', 'o', 'p', 't', 'y']
```

```
[1, 1, 3, 4, 5]
[3, 1, 4, 1, 5]
['h', 'n', 'o', 'p', 't', 'y']
```

5.4.4.2 `.sort()` (list method)

- **In-place sort:** modifies the list itself.
- Only available for lists (not other iterables).
- Returns `None`.

```
nums = [3, 1, 4, 1, 5]
nums.sort()
print(nums)      # [1, 1, 3, 4, 5]
```

```
[1, 1, 3, 4, 5]
```

5.4.4.3 Reverse Sorting

Both `sorted()` and `.sort()` accept a `reverse` argument.

```
nums = [3, 1, 4, 1, 5]
print(sorted(nums, reverse=True))
```

```
[5, 4, 3, 1, 1]
```

5.4.4.4 Sorting with a key

The `key` parameter lets you customize sorting logic.

```
# Sort by string length
words = ["pear", "apple", "banana", "kiwi"]
print(sorted(words, key=len))
# ['kiwi', 'pear', 'apple', 'banana']

# Sort by last character
print(sorted(words, key=lambda w: w[-1]))
# ['banana', 'pear', 'apple', 'kiwi']
```

```
['pear', 'kiwi', 'apple', 'banana']
['banana', 'apple', 'kiwi', 'pear']
```

5.4.4.5 Sorting Complex Data

For lists of tuples or dicts, use `key`.

```
# Sort by the second element of each tuple
pairs = [("a", 3), ("b", 1), ("c", 2)]
print(sorted(pairs, key=lambda t: t[1]))
# [('b', 1), ('c', 2), ('a', 3)]

# Sort list of dicts by a field
students = [
    {"name": "Alice", "grade": 85},
    {"name": "Bob", "grade": 92},
    {"name": "Chen", "grade": 78}
]
print(sorted(students, key=lambda s: s["grade"]))
```

```
[('b', 1), ('c', 2), ('a', 3)]
[{'name': 'Chen', 'grade': 78}, {'name': 'Alice', 'grade': 85}, {'name': 'Bob', 'grade': 92}]
```

Sorting strings alphabetically vs. numerically:

```
nums = ["10", "2", "1"]
print(sorted(nums))
print(sorted(nums, key=int))
```

```
['1', '10', '2']
['1', '2', '10']
```

5.4.5 Lambda Functions in Python

Sometimes you only need a **tiny one-off function** for a specific task (like sorting by length or filtering items). Writing a full `def` feels heavy.

That's where **lambda functions** help.

Syntax

```
lambda parameters: expression
```

- Creates an anonymous function (no name required).
- Must contain a single expression (not multiple statements).
- Automatically returns the value of the expression.

```
square = lambda x: x**2
print(square(5))    # 25
```

25

Using Lambda Functions

- With `sorted()`

```
words = ["pear", "apple", "banana", "kiwi"]

# Sort by word length
print(sorted(words, key=lambda w: len(w)))

# Sort by last character
print(sorted(words, key=lambda w: w[-1]))
```

```
['pear', 'kiwi', 'apple', 'banana']
['banana', 'apple', 'kiwi', 'pear']
```

- With `map()` and `filter()`

```
nums = [1, 2, 3, 4, 5]

# Square each number
print(list(map(lambda x: x**2, nums)))

# Keep only even numbers
print(list(filter(lambda x: x % 2 == 0, nums)))
```

```
[1, 4, 9, 16, 25]
[2, 4]
```

- With `reduce()` (from `functools`)

```
from functools import reduce

nums = [1, 2, 3, 4, 5]
product = reduce(lambda x, y: x * y, nums)
print(product)    # 120
```

120

Key Takeaways:

- `lambda` = quick, throwaway function for one-liners
- use `def` if the function
 - has multiple steps
 - is reused often

5.4.6 `enumerate()`

The `enumerate()` function is a built-in Python tool that adds a counter to any iterable, returning pairs of (index, item) as you loop. This is especially useful when you need both the item and its position in a loop.

Syntax:

```
for index, value in enumerate(iterable, start=0):  
    # use index and value
```

- **iterable:** Any sequence (list, tuple, string, etc.)
- **start:** Optional, sets the starting index (default is 0)

Why use `enumerate()`?

- Makes code cleaner and less error-prone
- Avoids manual index tracking with a separate variable
- Works with lists, tuples, strings, and more

Example:

```
fruits = ['apple', 'banana', 'cherry']  
for idx, fruit in enumerate(fruits):  
    print(idx, fruit)
```

```
0 apple  
1 banana  
2 cherry
```

Tip: You can set the starting index with the `start` argument, e.g. `enumerate(my_list, start=1)`.

Use `enumerate()` for readable, efficient loops when you need both index and value.

5.4.7 `zip()`

The `zip()` function combines two or more iterables (like lists, tuples, or strings) into tuples, pairing elements by their position. This is useful for parallel iteration, creating pairs, or merging data from multiple sources.

Syntax:

```
zip(iterable1, iterable2, ...)
```

- Each tuple contains one element from each iterable, matched by position.
- Stops at the shortest iterable.

Why use zip()? - Parallel iteration over multiple sequences - Pairing related data (e.g., names and scores) - Creating dictionaries from two lists

Example:

```
names = ['Alice', 'Bob', 'Chen']
scores = [85, 92, 78]
for name, score in zip(names, scores):
    print(name, score)
```

```
Alice 85
Bob 92
Chen 78
```

```
# Creating a dictionary from two lists:
gradebook = dict(zip(names, scores))
print(gradebook)
```

```
{'Alice': 85, 'Bob': 92, 'Chen': 78}
```

Tip: - You can use `zip(*zipped)` to unzip a list of tuples back into separate lists. - If the input iterables are different lengths, `zip()` stops at the shortest one.

5.4.8 Common Functions for Iterables

Python provides a variety of built-in functions to operate on iterables, making it easy to manipulate, process, and analyze collections like lists, tuples, strings, sets, and dictionaries. Below is a list of commonly used built-in functions specifically designed for iterables.

Function	Description	Example
<code>len()</code>	Returns the number of elements in an iterable.	<code>len([1, 2, 3]) → 3</code>
<code>min()</code>	Returns the smallest element in an iterable.	<code>min([3, 1, 4]) → 1</code>
<code>max()</code>	Returns the largest element in an iterable.	<code>max([3, 1, 4]) → 4</code>
<code>sum()</code>	Returns the sum of elements in an iterable (numeric types only).	<code>sum([1, 2, 3]) → 6</code>
<code>sorted()</code>	Returns a sorted list from an iterable (does not modify the original).	<code>sorted([3, 1, 2]) → [1, 2, 3]</code>
<code>reversed()</code>	Returns an iterator that accesses the elements of an iterable in reverse.	<code>list(reversed([1, 2, 3])) → [3, 2, 1]</code>

Function	Description	Example
<code>enumerate()</code>	Returns an iterator of tuples containing indices and elements of the iterable.	<code>list(enumerate(['a', 'b', 'c'])) → [(0, 'a'), (1, 'b'), (2, 'c')]</code>
<code>all()</code>	Returns True if all elements of the iterable are true (or if empty).	<code>all([True, 1, 'a']) → True</code>
<code>any()</code>	Returns True if any element of the iterable is true.	<code>any([False, 0, 'b']) → True</code>
<code>str.join(iterable)</code>	Joins the elements of an iterable (e.g., list, tuple) into a single string, using the given string as a separator.	<code>''.join(['a', 'b', 'c']) → 'abc'</code>

5.5 Independent Study

5.5.1 Sorting the data

- Sort [10, 2, 33, 25, 7] in descending order.
- Given words = ["data", "python", "AI", "science"], sort alphabetically ignoring case.
- Sort [("Ann", 22), ("Bob", 19), ("Chen", 22)] by age, preserving name order when ages match.

5.5.2 lambda

- Use lambda with `sorted()` to order the list:

```
nums = [-3, 1, -2, 5, 0]
```

- Use lambda with `filter()` to keep only names starting with a vowel:

```
names = ["Alice", "Bob", "Eve", "Uma", "Sam"]
```

- Use lambda with `map()` to convert Celsius to Fahrenheit:

```
temps_c = [0, 20, 37, 100]
```

5.5.3

You are given a list of strings representing student names. Some students appear more than once because they signed up for multiple activities.

Write a Python function that:

1. **Removes duplicate names** while *preserving the first occurrence order*.
2. **Returns a list of tuples** (index, name_length) for each **unique name**, where:
 - index is the original position of the name in the list (first occurrence).
 - name_length is the length of the name.

Finally, sort the output list by name_length in descending order.

```
names = ["Alice", "Bob", "Alice", "Charlie", "Bob", "Dave"]
```

5.5.4 Student data

```
# CELL 1: Student Data Setup
students = [
    (101, "Alice", "STAT303", 85),
    (102, "Bob", "STAT301", 92),
    (103, "Charlie", "STAT303", 78),
    (104, "Diana", "STAT301", 95),
    (105, "Eve", "STAT303", 88),
    (106, "Frank", "STAT304", 82),
    (107, "Grace", "STAT301", 90),
    (108, "Hank", "STAT304", 87),
    (109, "Ivy", "STAT303", 91),
    (101, "Alice", "STAT301", 88),
    (101, "Alice", "STAT304", 82),
    (101, "Alice", "STAT302", 86),
    (102, "Bob", "STAT303", 79),
    (102, "Bob", "STAT304", 85),
    (102, "Bob", "STAT302", 83),
    (103, "Charlie", "STAT301", 91),
    (103, "Charlie", "STAT304", 84),
    (103, "Charlie", "STAT302", 80),
    (104, "Diana", "STAT303", 89),
```



```
(104, "Diana", "STAT304", 93),
(104, "Diana", "STAT302", 96),
(105, "Eve", "STAT301", 87),
(105, "Eve", "STAT304", 90),
(105, "Eve", "STAT302", 85),
(106, "Frank", "STAT303", 76),
(106, "Frank", "STAT301", 81),
(106, "Frank", "STAT302", 79),
(107, "Grace", "STAT303", 84),
(107, "Grace", "STAT304", 88),
(107, "Grace", "STAT302", 91),
(108, "Hank", "STAT303", 80),
(108, "Hank", "STAT301", 83),
(108, "Hank", "STAT302", 82),
(109, "Ivy", "STAT301", 94),
(109, "Ivy", "STAT304", 89),
(109, "Ivy", "STAT302", 92),
(110, "Jack", "STAT303", 83),
(110, "Jack", "STAT301", 86),
(110, "Jack", "STAT304", 79),
(110, "Jack", "STAT302", 81),
(111, "Karen", "STAT303", 96),
(111, "Karen", "STAT301", 92),
(111, "Karen", "STAT304", 94),
(111, "Karen", "STAT302", 98),
(112, "Leo", "STAT303", 72),
(112, "Leo", "STAT301", 75),
(112, "Leo", "STAT304", 78),
(112, "Leo", "STAT302", 74)
```

]

- What is the data type for this variable
- How many unique students in the dataset
- How many unique courses in the dataset
- what is the grade range?
- What is the average grade?

Part III

Core library foundations

6 Reading Data

<IPython.core.display.Image object>

6.1 Types of Data: Structured vs. Unstructured

Understanding the format of your data is crucial before you can analyze it effectively. Data exists in two primary formats:

6.1.1 Structured Data

Structured data is organized in a tabular format with clearly defined relationships between data elements. Key characteristics include:

- **Rows** represent individual **observations** (records or data points)
- **Columns** represent **variables** (attributes or features)
- Data follows a consistent schema or format
- Easy to search, query, and analyze using standard tools

Example: The dataset below contains movie information where each row represents one movie (observation) and each column contains specific attributes like title, rating, or budget (variables). Since all data in a row relates to the same movie, this is also called **relational data**.

	Title	US Gross	Production Budget	Release Date	Major Genre
0	The Shawshank Redemption	28241469	25000000	Sep 23 1994	Drama
1	Inception	285630280	160000000	Jul 16 2010	Horror/Thriller
2	One Flew Over the Cuckoo's Nest	108981275	4400000	Nov 19 1975	Comedy
3	The Dark Knight	533345358	185000000	Jul 18 2008	Action/Adventure
4	Schindler's List	96067179	25000000	Dec 15 1993	Drama

Common formats: CSV files, Excel spreadsheets, SQL databases

6.1.2 Unstructured Data

Unstructured data lacks a predefined organizational structure, making it more challenging to analyze with traditional methods.

Examples include: Text documents, images, audio files, videos, social media posts, sensor data

While harder to analyze directly, unstructured data can often be converted into structured formats for analysis (e.g., extracting features from images or sentiment scores from text).

In this sequence, we will focus on analyzing structured data.

6.2 Introduction to Pandas

Pandas is Python's premier library for data manipulation and analysis. Think of it as Excel for Python - it provides powerful tools for working with tabular data (like spreadsheets) and supports reading from various file formats:

- **CSV** (Comma-Separated Values)
- **Excel** spreadsheets (.xlsx, .xls)
- **JSON** (JavaScript Object Notation)
- **HTML** tables
- **SQL** databases
- And many more!

6.3 Reading CSV Files with Pandas

6.3.1 Understanding CSV Files

CSV (Comma-Separated Values) is the most popular format for storing structured data. Here's what makes CSV files special:

- **Simple structure:** Each row represents one record/observation
- **Comma delimiter:** Values within a row are separated by commas
- **Plain text:** Human-readable and lightweight
- **Universal compatibility:** Supported by virtually all data tools

Pro Tip: When you open a CSV file in Excel, the commas are automatically converted to column separators, so you won't see them visually!

6.3.2 CSV File Structure Example

Here's a sample CSV file containing daily COVID-19 data for Italy:

```
date,new_cases,new_deaths,new_tests
2020-04-21,2256.0,454.0,28095.0
2020-04-22,2729.0,534.0,44248.0
2020-04-23,3370.0,437.0,37083.0
2020-04-24,2646.0,464.0,95273.0
2020-04-25,3021.0,420.0,38676.0
2020-04-26,2357.0,415.0,24113.0
2020-04-27,2324.0,260.0,26678.0
2020-04-28,1739.0,333.0,37554.0
...
```

Structure breakdown: - **Header row:** Column names (date, new_cases, new_deaths, new_tests) - **Data rows:** Each row contains values for one day - **Delimiter:** Commas separate each value within a row

6.3.3 Getting Started: Importing Pandas

Before we can work with data, we need to import the pandas library. By convention, pandas is imported with the alias `pd` - this is a universal practice that makes your code more readable and concise.

Why use `pd`?

- **Shorter syntax:** `pd.read_csv()` instead of `pandas.read_csv()`
- **Industry standard:** Everyone in the data science community uses this alias
- **Cleaner code:** Makes your scripts more readable

```
import pandas as pd
import os
```

6.3.4 Loading Data with `pd.read_csv()`

What is `pd.read_csv()`?

The `pd.read_csv()` function is your gateway to loading CSV data into Python. It converts CSV files into a pandas **DataFrame** - think of it as a powerful, interactive spreadsheet that you can manipulate with code.

Key Features:

- **Automatic data type detection:** Pandas intelligently guesses column types (numbers, text, dates)
- **Header recognition:** Automatically uses the first row as column names
- **Missing data handling:** Gracefully handles empty cells
- **Flexible parsing:** Can handle various CSV formats and delimiters

```
movie_ratings = pd.read_csv('./datasets/movie_ratings.csv')
movie_ratings.head()
```

	Title	US Gross	Worldwide Gross	Production Budget	Release Date	MPAA Rating
0	Opal Dreams	14443	14443	9000000	Nov 22 2006	PG/PG-13
1	Major Dundee	14873	14873	3800000	Apr 07 1965	PG/PG-13
2	The Informers	315000	315000	18000000	Apr 24 2009	R
3	Buffalo Soldiers	353743	353743	15000000	Jul 25 2003	R
4	The Last Sin Eater	388390	388390	2200000	Feb 09 2007	PG/PG-13

Understanding the DataFrame Display:

When you display a DataFrame, you'll notice several key components:

1. **The Index Column (leftmost):** The first column shows row indices (0, 1, 2, 3...). When you create a DataFrame without specifying an index, pandas automatically assigns a **RangeIndex**, which is a sequence of integers starting from 0.
2. **Column Headers:** The top row shows column names from your CSV file
3. **Data Cells:** The intersection of rows and columns contains your actual data
4. **Data Types:** Pandas automatically detects whether columns contain numbers, text, dates, etc.

Key Point: The index is **not** part of your original data - it's added by pandas to help identify and access rows efficiently.

```
# Examine the DataFrame index in detail
print(" DataFrame Index Information:")
print(f"Index type: {type(movie_ratings.index)}")
print(f"Index values: {movie_ratings.index}")
print(f"Index range: {movie_ratings.index[0]} to {movie_ratings.index[-1]}")
print(f"Total rows: {len(movie_ratings.index)}")

# Show the index as a list for first 10 rows
print(f"\nFirst 10 index values: {movie_ratings.index[:10].tolist()}")
```

```
DataFrame Index Information:
Index type: <class 'pandas.core.indexes.range.RangeIndex'>
Index values: RangeIndex(start=0, stop=2228, step=1)
Index range: 0 to 2227
Total rows: 2228
```

```
First 10 index values: [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
```

6.3.5 Indexing

Alternatively, we can use the `index_col` parameter in the `pd.read_csv()` function to set a specific column as the `index` when reading a CSV file in pandas. This allows us to directly specify which column should be used as the index of the resulting DataFrame.

```
movie_ratings = pd.read_csv('./datasets/movie_ratings.csv', index_col='Title')
movie_ratings.head()
```

Title	US Gross	Worldwide Gross	Production Budget	Release Date	MPAA Rating	Source
Opal Dreams	14443	14443	9000000	Nov 22 2006	PG/PG-13	Adapted from
Major Dundee	14873	14873	3800000	Apr 07 1965	PG/PG-13	Adapted from
The Informers	315000	315000	18000000	Apr 24 2009	R	Adapted from
Buffalo Soldiers	353743	353743	15000000	Jul 25 2003	R	Adapted from
The Last Sin Eater	388390	388390	2200000	Feb 09 2007	PG/PG-13	Adapted from

Let's check out the index again

```
movie_ratings.index
```

```
Index(['Opal Dreams', 'Major Dundee', 'The Informers', 'Buffalo Soldiers',
      'The Last Sin Eater', 'The City of Your Final Destination', 'The Claim',
      'Texas Rangers', 'Ride With the Devil', 'Karakter',
      ...,
      'A Cinderella Story', 'D-War', 'Sinbad: Legend of the Seven Seas',
      'Hoodwinked', 'First Knight', 'King Arthur', 'Mulan', 'Robin Hood',
      'Robin Hood: Prince of Thieves', 'Spiceworld'],
      dtype='object', name='Title', length=2228)
```

6.3.5.1 Indexing with `set_index()` and `reset_index()`

- Use `set_index()` to assign an existing column as the DataFrame's index.
- Use `reset_index()` to revert back to the default integer index.

```
movie_ratings.reset_index(inplace=True)
```

```
movie_ratings.index
```

```
RangeIndex(start=0, stop=2228, step=1)
```

```
movie_ratings.set_index("Title").head()
```

Title	US Gross	Worldwide Gross	Production Budget	Release Date	MPAA Rating	Source
Opal Dreams	14443	14443	9000000	Nov 22 2006	PG/PG-13	Adapted from the novel by Susan Cooper
Major Dundee	14873	14873	3800000	Apr 07 1965	PG/PG-13	Adapted from the novel by Louis L'Amour
The Informers	315000	315000	18000000	Apr 24 2009	R	Adapted from the novel by John Le Carré
Buffalo Soldiers	353743	353743	15000000	Jul 25 2003	R	Adapted from the novel by James A. McHale
The Last Sin Eater	388390	388390	2200000	Feb 09 2007	PG/PG-13	Adapted from the novel by James H. Jackson

```
movie_ratings.index
```

```
RangeIndex(start=0, stop=2228, step=1)
```

- The `index` can act like a primary key.
- It controls how `.loc[]`, slicing, and filtering behave.
- Choosing a good index (e.g., IDs, timestamps) makes subsetting much easier and more readable.

The built-in python function `type` can be used to check the datatype of an object:

```
type(movie_ratings)
```

```
pandas.core.frame.DataFrame
```

The Result: A DataFrame

6.3.6 Understanding Pandas Data Structures

Before diving deeper into data manipulation, it's crucial to understand the **two fundamental data structures** that pandas provides. Think of these as the building blocks for all data science operations.

6.3.7 Series - Your First Pandas Data Structure

A `pandas.Series` is like a **smart column** of data - imagine a single column from an Excel spreadsheet, but with superpowers!

Definition: A Series is a **one-dimensional array** of data with an associated **index** (labels for each data point).

Key Characteristics:

Feature	Description	Example
One-dimensional Indexed	Holds data in a single column/row Each element has a label/index	[85, 92, 78, 95] ['Alice', 'Bob', 'Carol', 'David']
Homogeneous Size immutable	Ideally same data type (but flexible) Fixed size once created (values can change)	All numbers or all text 4 elements stay 4 elements

**** Think of it as**:** A single column from your DataFrame with smart labeling capabilities.

```
# Creating a Series from the 'Title' column
print(" Extracting a Series from our DataFrame:")
print("=" * 50)

movie_titles = movie_ratings['Title']
print(" Movie Titles Series (first 10):")
print(movie_titles.head(10))
print()

# Let's explore what we just created
print(" Series Details:")
print(f" Type: {type(movie_titles)}")
print(f" Shape: {movie_titles.shape}")
print(f" Size: {movie_titles.size}")
print(f" Data Type: {movie_titles.dtype}")
print(f" Name: {movie_titles.name}")
```

Extracting a Series from our DataFrame:

=====

Movie Titles Series (first 10):

```
0          Opal Dreams
1      Major Dundee
2      The Informers
3      Buffalo Soldiers
4      The Last Sin Eater
5  The City of Your Final Destination
6          The Claim
7      Texas Rangers
8      Ride With the Devil
9          Karakter
```

Name: Title, dtype: object

Series Details:

Type: <class 'pandas.core.series.Series'>

Shape: (2228,)

Size: 2228

Data Type: object

Name: Title

6.3.7.1 DataFrame: Two-Dimensional Data

A **DataFrame** is the star of pandas - it's like a complete spreadsheet or database table:

Key Characteristics:

- **Two-dimensional:** Organized in rows and columns
- **Heterogeneous:** Different columns can have different data types
- **Labeled axes:** Both rows (index) and columns have names/labels
- **Flexible:** Easy to filter, sort, group, and analyze

Structure:

- **Rows (Index):** Each row represents one **observation** (e.g., one movie, customer, transaction)
- **Columns:** Each column represents one **variable/feature** (e.g., title, price, rating, date)
- **Cells:** Individual data points where rows and columns intersect

Default Behavior:

- **Row indices:** Automatically numbered 0, 1, 2, 3... (but customizable)
- **Column names:** Taken from the first row of your CSV file (the header)

- **Data types:** Automatically detected (numbers, text, dates, etc.)

6.4 DataFrame Exploration: Attributes and Methods

Now that you understand what DataFrames and Series are, let's learn how to **explore and understand** your data. Think of these tools as your data detective kit!

6.4.1 Attributes vs Methods: What's the Difference?

Type	How to Use	What It Does	Example
Attributes	<code>df.attribute</code> (no parentheses)	Shows properties	<code>df.shape</code>
Methods	<code>df.method()</code> (with parentheses)	Performs actions	<code>df.head()</code>

Pro Tip: Use `dir(your_dataframe)` to see all available attributes and methods!

6.4.2 Essential DataFrame Attributes

These attributes help you quickly understand your dataset's structure and characteristics.

6.4.2.1 Data Types with `dtypes`

What it shows: The data type of each column

Why it matters: Understanding data types helps you:

- Identify if numbers are stored as text (and need conversion)
- Know which operations work with each column
- Optimize memory usage and performance
- Catch data quality issues early

```
# Examine data types of all columns
print(" Data Types in Our Movie Dataset:")
print("=" * 40)
print(movie_ratings.dtypes)
print()
```

```
# Count how many columns of each type we have
print(" Data Type Summary:")
dtype_counts = movie_ratings.dtypes.value_counts()
for dtype, count in dtype_counts.items():
    print(f" {dtype}: {count} columns")
```

```
Title                object
US Gross             int64
Worldwide Gross      int64
Production Budget    int64
Release Date         object
MPAA Rating          object
Source               object
Major Genre          object
Creative Type         object
IMDB Rating          float64
IMDB Votes           int64
dtype: object
```

6.4.2.2 Understanding Pandas Data Types

Pandas Type	Python Equivalent	Real-World Example	When You'll See It
object	string/mixed	Movie titles, categories, mixed data	Text columns, categorical data
int64	int	Year (2023), count (50 movies)	Whole numbers, counts, IDs
float64	float	Rating (7.5), budget (\$50.5M)	Decimal numbers, measurements
bool	True/False	Is_Popular, Has_Sequel	Yes/no questions
datetime64	datetime	Release dates, timestamps	Dates and times

Common Data Type Issues:

- **Numbers stored as 'object':** Often means there are non-numeric characters (like '\$' or ',')
- **Dates as 'object':** Need to convert using `pd.to_datetime()`
- **Missing values in integers:** Forces conversion to float64

6.4.2.3 Dataset Dimensions with `shape`

What it shows: Returns dimensions as (rows, columns)

Why it matters:

- Understand your dataset size at a glance
- Know if you have a “tall” (many rows) or “wide” (many columns) dataset
- Estimate memory requirements
- Spot issues (e.g., fewer rows than expected after filtering)

```
# Check the dimensions of our movie dataset
print(f"Dataset shape: {movie_ratings.shape}")
print(f"Number of rows (movies): {movie_ratings.shape[0]}")
print(f"Number of columns (features): {movie_ratings.shape[1]}")
```

```
Dataset shape: (2228, 11)
Number of rows (movies): 2228
Number of columns (features): 11
```

6.4.2.3.1 `columns`

Purpose: Returns the column names as an Index object

Why useful: See what variables/features are available in your dataset

```
# View all column names
print("Available columns:")
for i, col in enumerate(movie_ratings.columns, 1):
    print(f"{i}. {col}")
```

```
Available columns:
1. Title
2. US Gross
3. Worldwide Gross
4. Production Budget
5. Release Date
6. MPAA Rating
7. Source
8. Major Genre
9. Creative Type
10. IMDB Rating
11. IMDB Votes
```

6.4.2.3.2 index

Purpose: Returns the row labels (index) of the DataFrame

Why useful: Understand how rows are labeled and indexed

```
# Examine the index
print(f"Index type: {type(movie_ratings.index)}")
print(f"Index range: {movie_ratings.index[0]} to {movie_ratings.index[-1]}")
print(f"First 10 index values: {movie_ratings.index[:10].tolist()}")
```

Index type: <class 'pandas.core.indexes.range.RangeIndex'>

Index range: 0 to 2227

First 10 index values: [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]

6.4.2.4 Essential DataFrame Methods

6.4.2.4.1 info()

Purpose: Provides a comprehensive summary of the DataFrame

Returns: Information about data types, non-null counts, and memory usage

```
# Get comprehensive dataset information
movie_ratings.info()
```

<class 'pandas.core.frame.DataFrame'>

RangeIndex: 2228 entries, 0 to 2227

Data columns (total 11 columns):

#	Column	Non-Null Count	Dtype
0	Title	2228 non-null	object
1	US Gross	2228 non-null	int64
2	Worldwide Gross	2228 non-null	int64
3	Production Budget	2228 non-null	int64
4	Release Date	2228 non-null	object
5	MPAA Rating	2228 non-null	object
6	Source	2228 non-null	object
7	Major Genre	2228 non-null	object
8	Creative Type	2228 non-null	object
9	IMDB Rating	2228 non-null	float64
10	IMDB Votes	2228 non-null	int64

dtypes: float64(1), int64(4), object(6)

memory usage: 191.6+ KB

What `info()` tells you:

- **Data types** for each column
- **Non-null count** (helps identify missing data)
- **Memory usage** (useful for large datasets)
- **Total entries** and columns

6.4.2.4.2 `describe()`

Purpose: Generate descriptive statistics for numerical columns

Returns: Count, mean, std, min, 25%, 50%, 75%, max for each numeric column

```
# Get statistical summary of numerical columns
movie_ratings.describe()
```

	US Gross	Worldwide Gross	Production Budget	IMDB Rating	IMDB Votes
count	2.228000e+03	2.228000e+03	2.228000e+03	2228.000000	2228.000000
mean	5.076370e+07	1.019370e+08	3.816055e+07	6.239004	33585.154847
std	6.643081e+07	1.648589e+08	3.782604e+07	1.243285	47325.651561
min	0.000000e+00	8.840000e+02	2.180000e+02	1.400000	18.000000
25%	9.646188e+06	1.320737e+07	1.200000e+07	5.500000	6659.250000
50%	2.838649e+07	4.266892e+07	2.600000e+07	6.400000	18169.000000
75%	6.453140e+07	1.200000e+08	5.300000e+07	7.100000	40092.750000
max	7.601676e+08	2.767891e+09	3.000000e+08	9.200000	519541.000000

Key Statistics Explained:

- **count:** Number of non-missing values
- **mean:** Average value
- **std:** Standard deviation (measure of spread)
- **min/max:** Smallest and largest values
- **25%, 50%, 75%:** Quartiles (percentiles)

Tip: Use `describe(include='all')` to include non-numeric columns in the summary!

6.4.2.4.3 `head()` and `tail()`

Purpose: View the first or last few rows of your dataset

Default: Shows 5 rows, but you can specify any number

```
# View first 3 rows
print("First 3 movies:")
movie_ratings.head(3)
```

First 3 movies:

	Title	US Gross	Worldwide Gross	Production Budget	Release Date	MPAA Rating	Source
0	Opal Dreams	14443	14443	9000000	Nov 22 2006	PG/PG-13	Adapted
1	Major Dundee	14873	14873	3800000	Apr 07 1965	PG/PG-13	Adapted
2	The Informers	315000	315000	18000000	Apr 24 2009	R	Adapted

```
# View last 3 rows
print("Last 3 movies:")
movie_ratings.tail(3)
```

Last 3 movies:

	Title	US Gross	Worldwide Gross	Production Budget	Release Date	Source
2225	Robin Hood	105269730	310885538	210000000	May 14 2010	Original
2226	Robin Hood: Prince of Thieves	165493908	390500000	50000000	Jun 14 1991	Original
2227	Spiceworld	29342592	56042592	25000000	Jan 23 1998	Original

6.4.2.4.4 `sample()`

Purpose: Get a random sample of rows from your DataFrame **Why useful:** Explore data patterns without bias toward top/bottom rows

```
# Get 5 random movies from the dataset
print("Random sample of 5 movies:")
movie_ratings.sample(5)
```


6.4.2.5 Quick Reference: Most Used Attributes & Methods

Attribute/Method	Purpose	Example Usage
<code>.shape</code>	Get dimensions (rows, cols)	<code>df.shape</code>
<code>.dtypes</code>	Check data types	<code>df.dtypes</code>
<code>.columns</code>	View column names	<code>df.columns</code>
<code>.info()</code>	Comprehensive overview	<code>df.info()</code>
<code>.describe()</code>	Statistical summary	<code>df.describe()</code>
<code>.head(n)</code>	First n rows	<code>df.head(10)</code>
<code>.tail(n)</code>	Last n rows	<code>df.tail(3)</code>
<code>.sample(n)</code>	Random n rows	<code>df.sample(5)</code>

6.5 Putting It All Together: Understanding Pandas Data Structures

Now that we've explored DataFrames and Series in detail, let's consolidate your understanding with a comprehensive comparison and key takeaways.

6.5.1 DataFrame vs Series: The Complete Picture

Aspect	DataFrame	Series
Dimensions	2D (rows × columns)	1D (single column)
Purpose	Complete dataset/table	Single variable/column
Structure	Multiple columns, each can be different type	Single column of one data type
Access	<code>df['column']</code> or <code>df.column</code>	Direct indexing <code>series[0]</code>
Example	Entire movie database	Just the 'Rating' column
Relationship	Contains multiple Series	One column extracted from DataFrame

6.5.2 Key Concepts You Now Understand

6.5.2.1 DataFrames are Collections of Series

```
# When you do this:
ratings = movie_ratings['Rating'] # You get a Series
titles = movie_ratings['Title']   # You get another Series
```

```
# The DataFrame is made up of these Series working together!
```

6.5.2.2 Index: The Hidden Hero

- Every row has a label (0, 1, 2, 3... by default)
- Links related data across different columns
- Enables precise data selection and alignment

6.5.2.3 Data Types Matter

- Numerical columns (`int64`, `float64`) → Mathematical operations
- Text columns (`object`) → String operations, categories
- Date columns (`datetime64`) → Time-based analysis
- Boolean columns (`bool`) → True/False filtering

6.5.3 What You Can Do Now

Load data from CSV files into DataFrames

Understand the structure using `.shape`, `.dtypes`, `.info()`

Extract specific columns as Series for focused analysis

Identify data quality issues through data type inspection

Navigate between 1D and 2D data structures confidently

6.6 Writing Data to CSV Files

After cleaning, analyzing, or transforming your data, you'll often need to **save your results** for future use or sharing. Pandas makes this easy with the `to_csv()` method.

6.6.1 Basic CSV Export

The simplest way to export a DataFrame is using the `to_csv()` method with just a filename:

```
# Basic export - saves DataFrame to CSV file
movie_ratings.to_csv('./datasets/movie_ratings_exported.csv')
print(" Data exported successfully!")
```

Data exported successfully!

```
# Verify the file was created
if './datasets/movie_ratings_exported.csv' in [f'./datasets/{f}' for f in os.listdir('./datasets')]:
    print(" File successfully exported!")
else:
    print(" Export failed")
```

File successfully exported!

6.6.2 Important CSV Export Parameters

By default, pandas includes row indices in the exported CSV. Often you don't want this:

The `to_csv()` method has many useful parameters to customize your output:

6.6.2.1 Index Control

```
# Export WITHOUT row indices (most common)
movie_ratings.to_csv('./datasets/movies_no_index.csv', index=False)

# Export WITH row indices (default behavior)
movie_ratings.to_csv('./datasets/movies_with_index.csv', index=True)
```

6.6.2.2 Column Selection

Export only specific columns:

```
# Export only selected columns
columns_to_export = ['Title', 'IMDB Rating', 'Worldwide Gross']
movie_ratings[columns_to_export].to_csv('./datasets/movies_selected_columns.csv', index=False)
```

6.6.2.3 Custom Separators

Change the delimiter (useful for different regional standards):

```
# Use semicolon instead of comma (common in European locales)
movie_ratings.to_csv('./datasets/movies_semicolon.csv', sep=';', index=False)

# Use tab separator (creates TSV file)
movie_ratings.to_csv('./datasets/movies_tab.txt', sep='\t', index=False)
```

6.6.2.4 Handling Missing Values

Control how missing/null values are represented:

```
# Replace NaN values with custom text
movie_ratings.to_csv('./datasets/movies_custom_na.csv',
                    index=False,
                    na_rep='Missing')
```

6.6.2.5 Encoding Specification

Ensure proper character encoding (important for international data):

```
# Specify UTF-8 encoding (recommended for international characters)
movie_ratings.to_csv('./datasets/movies_utf8.csv',
                    index=False,
                    encoding='utf-8')
```

6.6.3 Best Practices for CSV Export

Practice	Why Important	Example
Always use <code>index=False</code>	Prevents unnecessary row numbers	<code>df.to_csv('file.csv', index=False)</code>
Specify encoding	Ensures international characters display correctly	<code>encoding='utf-8'</code>
Use descriptive filenames	Makes files easy to identify later	<code>'cleaned_movie_data_2024.csv'</code>
Check file paths	Avoid overwriting important files	Use relative or absolute paths carefully
Handle missing data	Define how NaN values should appear	<code>na_rep='N/A'</code> or <code>na_rep=''</code>

6.6.4 Complete Export Example

Here's a comprehensive example combining multiple parameters:

```
# Professional CSV export with all best practices
export_filename = './datasets/movie_analysis_final.csv'

# Select relevant columns for export
columns_for_analysis = [
    'Title', 'IMDB Rating', 'IMDB Votes',
    'US Gross', 'Worldwide Gross', 'Production Budget'
]

# Export with professional settings
movie_ratings[columns_for_analysis].to_csv(
    export_filename,
    index=False,          # No row numbers
    encoding='utf-8',     # International characters
    na_rep='N/A',        # Clear missing value indicator
    float_format='%.2f'   # 2 decimal places for numbers
)

print(f" Analysis data exported to: {export_filename}")
print(f" Exported {len(columns_for_analysis)} columns for {len(movie_ratings)} movies")
```

```
Analysis data exported to: ./datasets/movie_analysis_final.csv
Exported 6 columns for 2228 movies
```

6.6.5 Verifying Your Export

Always verify your exported data looks correct:

```
# Read back the exported file to verify
verification_df = pd.read_csv(export_filename)

print(f"\n Exported dimensions: {verification_df.shape}")

print(" Exported file preview:")
verification_df.head(3)
```

Exported dimensions: (2228, 6)

Exported file preview:

	Title	IMDB Rating	IMDB Votes	US Gross	Worldwide Gross	Production Budget
0	Opal Dreams	6.5	468	14443	14443	9000000
1	Major Dundee	6.7	2588	14873	14873	3800000
2	The Informers	5.2	7595	315000	315000	18000000

6.7 Reading Other Data Formats

While **CSV is the most popular format** for structured data, real-world data comes in many different formats. As a data analyst, you'll frequently encounter various file types and data sources that require different reading approaches.

Common Data Formats Overview

Format	Extension	Use Case	Pandas Function
CSV	.csv	Most common tabular data	<code>pd.read_csv()</code>
Text	.txt	Tab-separated or custom delimiters	<code>pd.read_csv()</code> with <code>sep</code> parameter
JSON	.json	API data, nested structures	<code>pd.read_json()</code>
Excel	.xlsx, .xls	Spreadsheet data	<code>pd.read_excel()</code>
URLs	Various	Live data from web APIs	Various functions

In this section, we'll explore how to:

- **Read text files** with different delimiters (tabs, semicolons, etc.)
- **Handle JSON data** from APIs and web services
- **Access live data** directly from URLs and web pages

Key Insight: The same `pd.read_csv()` function can read many text-based formats by adjusting parameters!

6.7.1 Reading Text Files (.txt)

Text files offer **maximum flexibility** for storing tabular data with custom delimiters.

Key Differences from CSV:

- **CSV files:** Always use commas (,) as delimiters. Let's read a file where values are separated by tab characters:
- **Text files:** Can use **any character** as a delimiter

6.7.1.1 Reading Tab-Separated Files

Common Delimiters in Text Files:

- **Space ()** → Simple space-separated files
- **Tab character (\t)** → Creates TSV (Tab-Separated Values) files- **Pipe symbol (|)** → Used when data contains commas
- **Semicolon (;)** → Common in European data files

```
# Read tab-separated file using \t as separator
# Note: We still use pd.read_csv() but specify the separator
movie_ratings_txt = pd.read_csv('./datasets/movie_ratings.txt', sep='\t')

movie_ratings_txt.head()
```

	Unnamed: 0	Title	US Gross	Worldwide Gross	Production Budget	Release Date	M
0	0	Opal Dreams	14443	14443	9000000	Nov 22 2006	P
1	1	Major Dundee	14873	14873	3800000	Apr 07 1965	P
2	2	The Informers	315000	315000	18000000	Apr 24 2009	R
3	3	Buffalo Soldiers	353743	353743	15000000	Jul 25 2003	R
4	4	The Last Sin Eater	388390	388390	2200000	Feb 09 2007	P

Key Points:

- Use the **same** `pd.read_csv()` function for text files
- Specify the delimiter with the `sep` parameter
- `\t` represents the tab character

6.7.1.2 Common Separator Examples

- The function automatically handles headers and data types

```
# Different separator examples (these are demonstrations)

# Semicolon-separated (common in Europe)
# df_semicolon = pd.read_csv('file.txt', sep=';')

# Pipe-separated (when data contains commas)
# df_pipe = pd.read_csv('file.txt', sep='|')

# Space-separated
# df_space = pd.read_csv('file.txt', sep=' ')

# Multiple spaces or mixed whitespace
# df_whitespace = pd.read_csv('file.txt', sep='\s+')

print(" Tip: Use sep='\s+' for files with irregular spacing!")
```

Tip: Use sep='\s+' for files with irregular spacing!

6.7.1.3 Automatic Delimiter Detection

Pandas can automatically detect delimiters using the Python engine:

```
# Let pandas automatically detect the delimiter
# This uses the 'python' engine with a 'sniffer' tool
try:
    auto_detected = pd.read_csv('./datasets/movie_ratings.txt',
                                sep=None,                    # Auto-detect
                                engine='python')             # Use Python engine
    print(" Delimiter detected automatically!")
    print(f" Shape: {auto_detected.shape}")
except FileNotFoundError:
    print(" File not found - this is just a demonstration")
```

```
Delimiter detected automatically!
Shape: (2228, 12)
```

When to use automatic detection:

- When you're unsure about the delimiter
- For quick data exploration
- Avoid in production code (be explicit for reliability)

Pro Tip: Always check the [pandas documentation](#) for parameter details!

6.7.2 Reading JSON Data

JSON (JavaScript Object Notation) is the **standard format for web APIs** and modern data exchange.

Why JSON is Everywhere:

- **Web APIs:** Most modern APIs return JSON data
- **Mobile Apps:** Standard format for app-server communication
- **Cloud Services:** AWS, Google Cloud, Azure all use JSON
- **IoT Devices:** Smart devices send data in JSON format

Key Advantages over CSV:

Feature	CSV	JSON
Structure	Flat tables only	Nested / hierarchical
Data Types	Text and numbers	Objects, arrays, booleans
Metadata	Must be stored separately or encoded in headers	Supports metadata inline (keys + values in the same document)
Complex / Nested Records	Must be flattened into multiple columns	Native support for nested objects and arrays

6.7.2.1 Reading JSON with Pandas

- **Data type inference** (numbers, dates, etc.)

`pd.read_json()` automatically converts JSON data into pandas DataFrames:

- **DataFrame conversion** when possible
- **Smart detection** of JSON structure

Automatic Processing:

- **Local files:** `pd.read_json('data.json')`
- **URLs:** `pd.read_json('https://api.example.com/data')`

- **Various orientations:** Records, index, values, etc.
- **JSON strings:** Direct JSON text

6.7.2.2 Real Example: TED Talks Dataset

Let's load a real JSON dataset containing TED Talks information:

```
# Load TED Talks data from GitHub (JSON format)

url = 'https://raw.githubusercontent.com/cwkenwaysun/TEDmap/master/data/TED_Talks.json'
print(" Loading TED Talks data from JSON...")
tedtalks_data = pd.read_json(url)
print(f" Loaded {tedtalks_data.shape[0]} TED Talks with {tedtalks_data.shape[1]} features!")
```

```
Loading TED Talks data from JSON...
Loaded 2474 TED Talks with 16 features!
```

```
# Explore the TED Talks dataset

print(f" Dataset Info:")
tedtalks_data.head(3)

print(f"Shape: {tedtalks_data.shape}")
print("\n First few TED Talks:")
print(f"Columns: {list(tedtalks_data.columns)}")
```

```
Dataset Info:
Shape: (2474, 16)
```

```
First few TED Talks:
Columns: ['id', 'speaker', 'headline', 'URL', 'description', 'transcript_URL', 'month_filmed']
```

6.7.3 Reading Data from Web URLs

Modern data science often involves accessing live data from web APIs (Application Programming Interfaces). This allows you to get real-time information directly into your analysis pipeline.

Using the `requests` Library

The `requests` library is the standard tool for making HTTP requests in Python. You'll need to install it if not already available:

```
pip install requests
```

Basic API Request Pattern: 1. **Send request** to the API URL 2. **Check response** status (success/error) 3. **Parse JSON data** into usable format 4. **Convert to DataFrame** for analysis

Let's demonstrate with the **CoinGecko API**, which provides cryptocurrency market data:

```
import requests

# Define the API endpoint
api_url = 'https://api.coingecko.com/api/v3/coins/markets'

# Add parameters to get USD prices for top 10 cryptocurrencies
params = {
    'vs_currency': 'usd', # Price in US Dollars
    'order': 'market_cap_desc', # Sort by market cap
    'per_page': 10, # Get top 10 coins
    'page': 1, # First page
    'sparkline': False # Don't include price history
}

print(" Requesting cryptocurrency data from CoinGecko API...")

# Send GET request with parameters
response = requests.get(api_url, params=params)

# Check HTTP status codes
print(f" Response Status Code: {response.status_code}")

if response.status_code == 200:
    print(" Request successful!")

    # Parse JSON response
    crypto_data = response.json()

    # Display basic info about the response
    print(f" Received data for {len(crypto_data)} cryptocurrencies")

    # Show raw JSON structure (first item only)
```

```

print(f"\n Sample JSON structure:")
print(f"Keys available: {list(crypto_data[0].keys())}")

else:
    # Handle different error codes
    error_messages = {
        400: "Bad Request - Check your parameters",
        401: "Unauthorized - API key might be required",
        403: "Forbidden - Access denied",
        404: "Not Found - Invalid endpoint",
        429: "Rate Limited - Too many requests",
        500: "Server Error - Try again later"
    }

    error_msg = error_messages.get(response.status_code, "Unknown error")
    print(f" Request failed: {error_msg}")
    crypto_data = None

```

Requesting cryptocurrency data from CoinGecko API...

Response Status Code: 200

Request successful!

Received data for 10 cryptocurrencies

Sample JSON structure:

Keys available: ['id', 'symbol', 'name', 'image', 'current_price', 'market_cap', 'market_cap

```

# Convert JSON data to pandas DataFrame (if successful)
if crypto_data:
    # Method 1: Convert JSON directly to DataFrame
    crypto_df = pd.DataFrame(crypto_data)

    print(f"\n Cryptocurrency Market Data:")
    print(f"Dataset shape: {crypto_df.shape}")

    # Select and display key columns
    key_columns = ['name', 'symbol', 'current_price', 'market_cap', 'price_change_percentage']
    display_df = crypto_df[key_columns].copy()

    # Format for better readability
    display_df['current_price'] = display_df['current_price'].apply(lambda x: f"${x:,.2f}")
    display_df['market_cap'] = display_df['market_cap'].apply(lambda x: f"${x:,.0f}")

```

```

display_df['price_change_percentage_24h'] = display_df['price_change_percentage_24h'].ap

# Rename columns for display
display_df.columns = ['Name', 'Symbol', 'Current Price', 'Market Cap', '24h Change %']

print(f"\n Top 10 Cryptocurrencies by Market Cap:")
display(display_df)

# Method 2: Manual processing for learning
print(f"\n Price Summary:")
for coin in crypto_data[:5]: # Show first 5
    name = coin['name']
    symbol = coin['symbol'].upper()
    price = coin['current_price']
    change_24h = coin['price_change_percentage_24h']

    # Format change with color indicator
    change_indicator = " " if change_24h > 0 else " "
    print(f"{change_indicator} {name} ({symbol}): ${price:,.2f} | 24h: {change_24h:.2f}%")

else:
    print(" No data to process due to API request failure")

```

Cryptocurrency Market Data:
Dataset shape: (10, 26)

Top 10 Cryptocurrencies by Market Cap:

	Name	Symbol	Current Price	Market Cap	24h Change %
0	Bitcoin	btc	\$114,167.00	\$2,275,903,872,681	0.13%
1	Ethereum	eth	\$4,133.97	\$499,142,371,776	-1.06%
2	Tether	usdt	\$1.00	\$174,703,594,573	-0.04%
3	XRP	xrp	\$2.86	\$170,592,210,519	-1.02%
4	BNB	bnb	\$1,006.92	\$140,084,147,935	-1.12%
5	Solana	sol	\$208.39	\$113,247,264,117	-1.54%
6	USDC	usdc	\$1.00	\$73,396,062,741	0.00%
7	Lido Staked Ether	steth	\$4,133.28	\$35,272,520,571	-0.90%
8	Dogecoin	doge	\$0.23	\$34,944,052,528	-0.81%
9	TRON	trx	\$0.33	\$31,555,873,939	-0.92%

Price Summary:

Bitcoin (BTC): \$114,167.00 | 24h: 0.13%
Ethereum (ETH): \$4,133.97 | 24h: -1.06%
Tether (USDT): \$1.00 | 24h: -0.04%
XRP (XRP): \$2.86 | 24h: -1.02%
BNB (BNB): \$1,006.92 | 24h: -1.12%

6.7.3.1 Read data from wikipedia

```
api_url = 'https://en.wikipedia.org/wiki/List_of_countries_by_GDP_(nominal)_per_capita'
# Use pandas to read all tables from the webpage
wiki_response = requests.get(api_url)
# Check HTTP status codes
print(f" Response Status Code: {wiki_response.status_code}")
if wiki_response.status_code == 200:
    print(" Request successful!")
else:
    # output error message based on the status code
    error_messages = {
        400: "Bad Request - Check your parameters",
        401: "Unauthorized - API key might be required",
        403: "Forbidden - Access denied",
        404: "Not Found - Invalid endpoint",
        429: "Rate Limited - Too many requests",
        500: "Server Error - Try again later"
    }
    error_msg = error_messages.get(wiki_response.status_code, "Unknown error")
    print(f" Request failed: {error_msg}")
```

Response Status Code: 403

Request failed: Forbidden - Access denied

6.7.3.1.1 Understanding HTTP 403 Errors with Wikipedia

Why Wikipedia Returns 403 “Forbidden” Errors:

Wikipedia and many other websites block requests that appear to be automated bots or scrapers. When you make a request without proper headers, the server sees it as suspicious and blocks it with a 403 error.

Common Causes of 403 Errors:

- **Missing User-Agent:** No browser identification
- **Suspicious headers:** Looks like automated scraping
- **Rate limiting:** Too many requests too quickly
- **IP blocking:** Your IP has been flagged

6.7.3.1.2 Solution: Add Proper Headers

The key is to make your request look like it's coming from a real web browser by adding appropriate headers:

```
headers = {
    'User-Agent': 'My Data Analysis Project (your_email@example.com)'
}
url = 'https://en.wikipedia.org/wiki/List_of_countries_by_GDP_(nominal)'

# Pass the headers to read_html
tables = pd.read_html(url, header=0, attrs=headers)
```

Let's define a helper function for this purpose

```
def read_html_tables_from_url(url, match=None, user_agent=None, timeout=10, **kwargs):
    """
    Read HTML tables from a website URL with error handling and customizable options.

    Parameters:
    -----
    url : str
        The website URL to scrape tables from
    match : str, optional
        String or regex pattern to match tables (passed to pd.read_html)
    user_agent : str, optional
        Custom User-Agent header. If None, uses a default Chrome User-Agent
    timeout : int, default 10
        Request timeout in seconds
    **kwargs : dict
        Additional arguments passed to pd.read_html()

    Returns:
    -----
```

```

list of DataFrames or None
    List of pandas DataFrames (one per table found), or None if error occurs

Examples:
-----
# Basic usage
tables = read_html_tables_from_url("https://en.wikipedia.org/wiki/List_of_countries_by_GL

# Filter tables containing specific text
tables = read_html_tables_from_url(url, match="Country")

# With custom user agent
tables = read_html_tables_from_url(url, user_agent="MyBot/1.0")
"""
import pandas as pd
import requests

# Default User-Agent if none provided
if user_agent is None:
    user_agent = 'Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, 1.

headers = {'User-Agent': user_agent}

try:
    print(f" Fetching data from: {url}")

    # Make the HTTP request
    response = requests.get(url, headers=headers, timeout=timeout)
    response.raise_for_status() # Raise an exception for bad status codes (4xx or 5xx)

    print(f" Successfully retrieved webpage (Status: {response.status_code})")

    # Parse HTML tables
    read_html_kwargs = {'match': match} if match else {}
    read_html_kwargs.update(kwargs) # Add any additional arguments

    tables = pd.read_html(response.content, **read_html_kwargs)

    print(f" Found {len(tables)} table(s) on the page")

    # Display table information
    if tables:

```



```

        print(f"\n Table Overview:")
        for i, table in enumerate(tables):
            print(f" Table {i}: {table.shape} (rows × columns)")
            if i >= 4: # Limit output for readability
                print(f" ... and {len(tables) - 5} more tables")
                break

        return tables

except requests.exceptions.HTTPError as err:
    print(f" HTTP Error: {err}")
    return None
except requests.exceptions.ConnectionError as err:
    print(f" Connection Error: {err}")
    return None
except requests.exceptions.Timeout as err:
    print(f" Timeout Error: Request took longer than {timeout} seconds")
    return None
except requests.exceptions.RequestException as err:
    print(f" Request Error: {err}")
    return None
except ValueError as err:
    print(f" No tables found on the page or parsing error: {err}")
    return None
except Exception as e:
    print(f" An unexpected error occurred: {e}")
    return None

```

Now let's demonstrate how to use our new function with different options:

```

# Example 1: Basic usage - Get all tables from GDP Wikipedia page
url = 'https://en.wikipedia.org/wiki/List_of_countries_by_GDP_(nominal)_per_capita'
tables = read_html_tables_from_url(url)

if tables:
    print(f"\n Successfully retrieved {len(tables)} tables!")

    # Display info about the largest table
    largest_table = max(tables, key=len)
    print(f"\n Largest table has {largest_table.shape[0]} rows and {largest_table.shape[1]} columns")
    print(f"Columns: {list(largest_table.columns)}")
else:

```

```
print(" Failed to retrieve tables")
```

```
Fetching data from: https://en.wikipedia.org/wiki/List_of_countries_by_GDP_(nominal)_per_capita
Successfully retrieved webpage (Status: 200)
Found 6 table(s) on the page
```

```
Table Overview:
```

```
Table 0: (1, 2) (rows × columns)
Table 1: (224, 4) (rows × columns)
Table 2: (9, 2) (rows × columns)
Table 3: (13, 2) (rows × columns)
Table 4: (2, 2) (rows × columns)
... and 1 more tables
```

```
Successfully retrieved 6 tables!
```

```
Largest table has 224 rows and 4 columns
```

```
Columns: ['Country/Territory', 'IMF (2025)[a][5]', 'World Bank (2022-24)[6]', 'United Nations']
Successfully retrieved webpage (Status: 200)
Found 6 table(s) on the page
```

```
Table Overview:
```

```
Table 0: (1, 2) (rows × columns)
Table 1: (224, 4) (rows × columns)
Table 2: (9, 2) (rows × columns)
Table 3: (13, 2) (rows × columns)
Table 4: (2, 2) (rows × columns)
... and 1 more tables
```

```
Successfully retrieved 6 tables!
```

```
Largest table has 224 rows and 4 columns
```

```
Columns: ['Country/Territory', 'IMF (2025)[a][5]', 'World Bank (2022-24)[6]', 'United Nations']
```

```
# Example 2: Filtered search - Only get tables containing 'Country'
filtered_tables = read_html_tables_from_url(url, match='Country')

if filtered_tables:
    print(f"\n Found {len(filtered_tables)} table(s) containing 'Country'")

    # Examine the first filtered table
```

```

gdp_table = filtered_tables[0]
print(f"\n GDP Table Details:")
print(f"Shape: {gdp_table.shape}")
print(f"Columns: {list(gdp_table.columns)}")
print("\n Sample data:")
display(gdp_table.head(3))
else:
    print(" No tables found containing 'Country'")

```

Fetching data from: [https://en.wikipedia.org/wiki/List_of_countries_by_GDP_\(nominal\)_per_capita](https://en.wikipedia.org/wiki/List_of_countries_by_GDP_(nominal)_per_capita)
 Successfully retrieved webpage (Status: 200)
 Found 1 table(s) on the page

Table Overview:
 Table 0: (224, 4) (rows × columns)

Found 1 table(s) containing 'Country'

GDP Table Details:
 Shape: (224, 4)
 Columns: ['Country/Territory', 'IMF (2025)[a][5]', 'World Bank (2022–24)[6]', 'United Nations (2023)[7]']

Sample data:
 Successfully retrieved webpage (Status: 200)
 Found 1 table(s) on the page

Table Overview:
 Table 0: (224, 4) (rows × columns)

Found 1 table(s) containing 'Country'

GDP Table Details:
 Shape: (224, 4)
 Columns: ['Country/Territory', 'IMF (2025)[a][5]', 'World Bank (2022–24)[6]', 'United Nations (2023)[7]']

Sample data:

	Country/Territory	IMF (2025)[a][5]	World Bank (2022–24)[6]	United Nations (2023)[7]
0	Monaco	—	256581	256581
1	Liechtenstein	—	207973	207150

	Country/Territory	IMF (2025)[a][5]	World Bank (2022–24)[6]	United Nations (2023)[7]
2	Luxembourg	140941	137516	128936

6.8 Independent Study

6.8.1 Practice exercise 1: Reading .csv data

Read the file *Top 10 Albums By Year.csv*. This file contains the top 10 albums for each year from 1990 to 2021. Each row corresponds to a unique album.

6.8.1.1

Print the first 5 rows of the data.

6.8.1.2

How many rows and columns are there in the data?

6.8.1.3

Print the summary statistics of the data, and answer the following questions:

1. What proportion of albums have 15 or lesser tracks? Mention a range for the proportion.
2. What is the mean length of a track (in minutes)?

6.8.2 Practice exercise 2: Reading .txt data

Read the file *bestseller_books.txt*. It contains top 50 best-selling books on amazon from 2009 to 2019. Identify the delimiter without opening the file with Notepad or a text-editing software. How many rows and columns are there in the dataset?

Alternatively, you can use the argument `sep = None`, and `engine = 'python'`. The default engine is C. However, the ‘python’ engine has a ‘sniffer’ tool which may identify the delimiter automatically.

6.8.3 Practice exercise 3: Reading HTML data

Read the table(s) consisting of attendance of spectators in FIFA worlds cup from this [page](#). Read only those table(s) that have the word '*reaching*' in them. How many rows and columns are there in the table(s)?

6.8.4 Practice exercise 4: Reading JSON data

Read the movies dataset from [here](#). How many rows and columns are there in the data?

7 Pandas Fundamentals

<IPython.core.display.Image object>

7.1 Introduction to Pandas Fundamentals

In the **Reading Data** chapter, we learned how to load data files into pandas DataFrames. Now we'll dive deeper into pandas' powerful capabilities for data manipulation and analysis.

Pandas is an essential tool in the data scientist or data analyst's toolkit due to its ability to handle and transform data with ease and efficiency.

7.1.1 Pandas in the Data Science Ecosystem

Pandas doesn't work in isolation - it's designed to integrate seamlessly with other Python libraries:

Library	Integration with Pandas	Purpose
NumPy	Built on NumPy arrays	Mathematical operations and array processing
Matplotlib	Direct plotting from DataFrames	Data visualization and charts
Seaborn	Native DataFrame support	Statistical visualizations
Scikit-learn	Seamless data flow	Machine learning algorithms
SciPy	Statistical analysis	Advanced statistical functions

7.1.2 Core Pandas Capabilities

Throughout this chapter, you'll master these essential pandas operations:

- **Creating Series & DataFrames:** From Python data structures
- **Filtering & Subsetting:** Extract exactly the data you need
- **Sorting & Ranking:** Organize data for analysis
- **Adding & Removing:** Modify DataFrame structure
- **Data Types:** Work with different types of data

Learning Path: We'll start with fundamental concepts and progressively build toward advanced data manipulation techniques, giving you a complete toolkit for real-world data analysis.

Now let's start by importing the essential Pandas library:

```
import pandas as pd
```

7.2 Creating Pandas Series & DataFrames

There are **two primary approaches** for creating pandas data structures, each serving different purposes in your data analysis workflow:

7.2.1 Method 1: Reading from External Data Sources

In real-world data science, you'll typically work with data stored in external files or databases. This is the most common approach for production analysis.

Popular Data Sources:

Source Type	Pandas Function	Use Case
CSV Files	<code>pd.read_csv()</code>	Most common - structured tabular data
Excel Files	<code>pd.read_excel()</code>	Business data, reports with multiple sheets

Source Type	Pandas Function	Use Case
JSON Files	<code>pd.read_json()</code>	API data, web services, nested data
SQL Databases	<code>pd.read_sql()</code>	Enterprise databases, large datasets
HTML Tables	<code>pd.read_html()</code>	Web scraping, online data tables

Pro Tip: We covered these methods extensively in the **Reading Data** chapter. If you need a refresher on loading external data, refer back to that section!

7.2.2 Method 2: Creating from Python Data Structures

When you need to create data programmatically or work with small datasets for testing and learning, you can build DataFrames and Series directly from Python objects.

Benefits of this approach:

- **Testing & Learning:** Perfect for experimenting with pandas features
- **Data Generation:** Create synthetic data for analysis
- **Small Datasets:** Handle simple, manual data entry
- **Prototyping:** Quick data structure creation for proof-of-concepts

Let's explore how to create pandas data structures from scratch!

7.2.2.1 Creating Series from Python Data Structures

A **Series** is pandas' 1D data structure - think of it as a smart, labeled list or array.

7.2.2.1.1 Method A: From a Python List

The simplest way to create a Series is from a Python list. Pandas automatically assigns integer indices starting from 0.

```
#Defining a Pandas Series
series_example = pd.Series(['these', 'are', 'english', 'words'])
series_example
```



```

0      these
1       are
2    english
3     words
dtype: object

```

Key Observation: Notice the automatic integer indices (0, 1, 2, 3) on the left!

Custom Indexing: You can provide meaningful labels instead of default numbers using the `index` parameter:

```

#Defining a Pandas Series with custom row labels
series_example = pd.Series(['these','are','english','words'], index = range(101,105))
series_example

```

```

101     these
102      are
103   english
104    words
dtype: object

```

7.2.2.1.2 Method B: From a Python Dictionary

Dictionaries are perfect for creating Series with **meaningful labels**. The dictionary keys automatically become the Series index, and values become the data points.

Perfect for: Time series data, named categories, any key-value relationships

```

# Real-world example: US GDP per capita by year (1960-2021)
# Notice some years are missing - pandas handles this gracefully!
GDP_per_capita_dict = {'1960':3007,'1961':3067,'1962':3244,'1963':3375,'1964':3574,'1965':38

```

```

#Example 2: Creating a Pandas Series from a Dictionary
GDP_per_capita_series = pd.Series(GDP_per_capita_dict)
GDP_per_capita_series.head()

```

```

1960    3007
1961    3067
1962    3244
1963    3375
1964    3574
dtype: int64

```

7.2.2.2 Creating a DataFrame from Python Data Structures

7.2.2.2.1 Method A: From a list of Python Dictionary

You can create a DataFrame where keys are column names and values are lists representing column data.

```
#List of dictionary consisting of 52 playing cards of the deck
deck_list_of_dictionaries = [{'value':i, 'suit':c}
for c in ['spades', 'clubs', 'hearts', 'diamonds']
for i in range(2,15)]
```

```
#Example 3: Creating a Pandas DataFrame from a List of dictionaries
deck_df = pd.DataFrame(deck_list_of_dictionaries)
deck_df.head()
```

	value	suit
0	2	spades
1	3	spades
2	4	spades
3	5	spades
4	6	spades

7.2.2.2.2 Method B: From a Python Dictionary

You can create a DataFrame where keys are column names and values are lists representing column data.

```
#Example 4: Creating a Pandas DataFrame from a Dictionary
dict_data = {'A': [1, 2, 3, 4, 5],
             'B': [20, 10, 50, 40, 30],
             'C': [100, 200, 300, 400, 500]}

dict_df = pd.DataFrame(dict_data)
dict_df
```

	A	B	C
0	1	20	100
1	2	10	200
2	3	50	300

	A	B	C
3	4	40	400
4	5	30	500

7.3 Data Selection and Filtering

Now that you understand series and dataframe are two fundamental data structures in pandas, let's master the art of **extracting exactly the data you need**. Data selection and filtering are fundamental skills that you'll use in every pandas project.

7.3.1 Basic Selection

7.3.1.1 Extracting Column(s)

The first step when working with a DataFrame is often to extract one or more columns. To do this effectively, it's helpful to understand the internal structure of a DataFrame. Conceptually, you can think of a DataFrame as a dictionary of lists, where the keys are column names, and the values are lists or arrays containing data for the respective columns.

Loading Our Practice Dataset

Let's work with a real movie ratings dataset to practice our selection techniques:

```
import pandas as pd
movie_ratings = pd.read_csv('./datasets/movie_ratings.csv')
movie_ratings.head()
```

	Title	US Gross	Worldwide Gross	Production Budget	Release Date	MPAA Rating
0	Opal Dreams	14443	14443	9000000	Nov 22 2006	PG/PG-13
1	Major Dundee	14873	14873	3800000	Apr 07 1965	PG/PG-13
2	The Informers	315000	315000	18000000	Apr 24 2009	R
3	Buffalo Soldiers	353743	353743	15000000	Jul 25 2003	R
4	The Last Sin Eater	388390	388390	2200000	Feb 09 2007	PG/PG-13

7.3.1.1.1 Method 1: Single Column Extraction

Each column represents a **feature** of your data. There are **two ways** to extract a single column:

A) Bracket Notation: `df['column_name']` (Recommended)

- Always works
- Handles column names with spaces
- Clear and explicit

B) Dot Notation: `df.column_name` (Limited use)

- Fails with spaces in column names
- Conflicts with DataFrame methods
- Shorter syntax for simple names

```
movie_ratings.Title
```

```
0           Opal Dreams
1         Major Dundee
2         The Informers
3        Buffalo Soldiers
4        The Last Sin Eater
...
2223          King Arthur
2224             Mulan
2225          Robin Hood
2226  Robin Hood: Prince of Thieves
2227          Spiceworld
Name: Title, Length: 2228, dtype: object
```

```
# Understanding the DataFrame structure:
# DataFrames work like dictionaries with enhanced features!
movie_ratings_dict = {
    'Title': ['Opal Dreams', 'Major Dundee', 'The Informers', 'Buffalo Soldiers', 'The Last
    'US Gross': [14443, 14873, 315000, 353743, 388390],
    'Worldwide Gross': [14443, 14873, 315000, 353743, 388390],
    'Production Budget': [9000000, 3800000, 18000000, 15000000, 2200000]
}
```

Dictionary Analogy:

Just like accessing dictionary values by key:

```
movie_ratings_dict['Title']
```

```
['Opal Dreams',  
 'Major Dundee',  
 'The Informers',  
 'Buffalo Soldiers',  
 'The Last Sin Eater']
```

DataFrame Column Extraction:

Similarly, we extract DataFrame columns by column name:

```
movie_ratings['Title']
```

```
0           Opal Dreams  
1           Major Dundee  
2           The Informers  
3           Buffalo Soldiers  
4           The Last Sin Eater  
...  
2223          King Arthur  
2224             Mulan  
2225          Robin Hood  
2226  Robin Hood: Prince of Thieves  
2227             Spiceworld  
Name: Title, Length: 2228, dtype: object
```

7.3.1.1.2 Method 2: Multiple Column Selection

Key Rule: Use a **list of column names** inside the brackets:

```
# Single column → Series  
df['column']  
  
# Multiple columns → DataFrame  
df[['col1', 'col2']]
```

Notice: Double brackets `[[ ]]` create a list, which tells pandas you want multiple columns!

```
movie_ratings[['Title', 'US Gross', 'Worldwide Gross' ]]
```

	Title	US Gross	Worldwide Gross
0	Opal Dreams	14443	14443
1	Major Dundee	14873	14873
2	The Informers	315000	315000
3	Buffalo Soldiers	353743	353743
4	The Last Sin Eater	388390	388390
...	...	...	...
2223	King Arthur	51877963	203877963
2224	Mulan	120620254	303500000
2225	Robin Hood	105269730	310885538
2226	Robin Hood: Prince of Thieves	165493908	390500000
2227	Spiceworld	29342592	56042592

7.3.1.2 Extracting Row(s)

In many cases, we need to filter rows based on specific conditions or a combination of multiple conditions. Next, let's explore how to use these conditions effectively to extract rows that meet our criteria, whether it's a single condition or multiple conditions combined

7.3.1.2.1 Single Condition Filtering

Filter rows in a DataFrame based on **one logical condition**.

This is the most common way to extract a subset of data, such as selecting all rows where a column's values meet a specific criterion (e.g., greater than, equal to, not equal).

For Example:

```
# extracting the rows that have IMDB Rating greater than 8
movie_ratings[movie_ratings['IMDB Rating'] > 8]
```

	Title	US Gross	Worldwide Gross	Production Budget	Release Date	MPAA Ra
21	Gandhi, My Father	240425	1375194	5000000	Aug 03 2007	Other
56	Ed Wood	5828466	5828466	18000000	Sep 30 1994	R
67	Requiem for a Dream	3635482	7390108	4500000	Oct 06 2000	Other
164	Trainspotting	16501785	24000785	3100000	Jul 19 1996	R
181	The Wizard of Oz	28202232	28202232	2777000	Aug 25 2039	G
...	...	...	...	...	...	...

	Title	US Gross	Worldwide Gross	Production Budget	Release Date	MPAA Ra
2090	Finding Nemo	339714978	867894287	94000000	May 30 2003	G
2092	Toy Story 3	410640665	1046340665	200000000	Jun 18 2010	G
2094	Avatar	760167650	2767891499	237000000	Dec 18 2009	PG/PG-13
2130	Scarface	44942821	44942821	25000000	Dec 09 1983	Other
2194	The Departed	133311000	290539042	90000000	Oct 06 2006	R

7.3.1.2.2 Multiple Conditions Filtering

When filtering with more than one condition, use **logical operators**:

Operator	Purpose	Example
&	AND – both conditions must be True	<code>(df['A'] > 5) & (df['B'] < 10)</code>
\	OR – at least one condition must be True	<code>(df['A'] > 5) \ (df['B'] < 10)</code>
~	NOT – negates the condition	<code>~(df['A'] > 5)</code>

Important Rule: Always wrap each condition in **parentheses ()** to avoid operator precedence errors.

```
# extracting the rows that have IMDB Rating greater than 8 and US Gross less than 1000000
movie_ratings[(movie_ratings['IMDB Rating'] > 8) & (movie_ratings['US Gross'] < 1000000)]
```

	Title	US Gross	Worldwide Gross	Production Budget	Release Date	MPAA Rating
21	Gandhi, My Father	240425	1375194	5000000	Aug 03 2007	Other
636	Lake of Fire	25317	25317	6000000	Oct 03 2007	Other

7.3.1.2.3 Negating Conditions with the ~ Operator

The tilde (~) operator in pandas is used to **negate boolean conditions**, making it especially useful when you want to *exclude* certain rows.

Instead of selecting rows that satisfy a condition, ~ allows you to filter rows that **do not** meet that condition. This is helpful for refining queries and focusing on the remaining data.

For example, if you want all rows where **IMDB Rating** is *not* equal to 8, you can write:

```
# Excluding the rows that have IMDB Rating that equals 8 using the tilde ~
movie_ratings[~(movie_ratings['IMDB Rating'] == 8)]
```

	Title	US Gross	Worldwide Gross	Production Budget	Release Date	
0	Opal Dreams	14443	14443	9000000	Nov 22 2006	I
1	Major Dundee	14873	14873	3800000	Apr 07 1965	I
2	The Informers	315000	315000	18000000	Apr 24 2009	I
3	Buffalo Soldiers	353743	353743	15000000	Jul 25 2003	I
4	The Last Sin Eater	388390	388390	2200000	Feb 09 2007	I
...	...	...	...	...	...	...
2223	King Arthur	51877963	203877963	90000000	Jul 07 2004	I
2224	Mulan	120620254	303500000	90000000	Jun 19 1998	C
2225	Robin Hood	105269730	310885538	210000000	May 14 2010	I
2226	Robin Hood: Prince of Thieves	165493908	390500000	50000000	Jun 14 1991	I
2227	Spiceworld	29342592	56042592	25000000	Jan 23 1998	I

Alternative: Using !=

Another way to exclude rows where a column equals a certain value is to use the **not-equal operator !=**:

```
# using the != operator
movie_ratings[movie_ratings['IMDB Rating'] != 8]
```

	Title	US Gross	Worldwide Gross	Production Budget	Release Date	
0	Opal Dreams	14443	14443	9000000	Nov 22 2006	I
1	Major Dundee	14873	14873	3800000	Apr 07 1965	I
2	The Informers	315000	315000	18000000	Apr 24 2009	I
3	Buffalo Soldiers	353743	353743	15000000	Jul 25 2003	I
4	The Last Sin Eater	388390	388390	2200000	Feb 09 2007	I
...	...	...	...	...	...	...
2223	King Arthur	51877963	203877963	90000000	Jul 07 2004	I
2224	Mulan	120620254	303500000	90000000	Jun 19 1998	C
2225	Robin Hood	105269730	310885538	210000000	May 14 2010	I
2226	Robin Hood: Prince of Thieves	165493908	390500000	50000000	Jun 14 1991	I
2227	Spiceworld	29342592	56042592	25000000	Jan 23 1998	I

7.3.2 Advanced Selection: `.loc[]` and `.iloc[]`

When you need **precise control** over both rows and columns, pandas provides two powerful indexers that let you select exactly what you need.

7.3.2.1 `.loc[]` vs `.iloc[]`: The Key Difference

Indexer	Uses	Best For	Example
<code>.loc[]</code>	Labels (names)	Human-readable selection	<code>df.loc['row_name', 'column_name']</code>
<code>.iloc[]</code>	Positions (numbers)	Programmatic selection	<code>df.iloc[0, 1]</code>

Mental Model

Think of your DataFrame like a **spreadsheet**:

- `.loc[]` works like clicking on **named** rows/columns (A1, B2, etc.)
- `.iloc[]` works like clicking on **position** numbers (row 0, column 1, etc.)

Memory Tip:

- `loc` = **L**abels/**L**ocation names
- `iloc` = **i**nteger **l**ocations

Preparing Our Data: Sorting by IMDB Rating

To demonstrate `.loc[]` and `.iloc[]` effectively, let's first sort our movies by rating so we can easily select the best ones:

```
movie_ratings_sorted = movie_ratings.sort_values(by = 'IMDB Rating', ascending = False)
movie_ratings_sorted.head()
```

	Title	US Gross	Worldwide Gross	Production Budget	Release Date	MPA
182	The Shawshank Redemption	28241469	28241469	25000000	Sep 23 1994	R
2084	Inception	285630280	753830280	160000000	Jul 16 2010	PG
790	Schindler's List	96067179	321200000	25000000	Dec 15 1993	R
1962	Pulp Fiction	107928762	212928762	8000000	Oct 14 1994	R
561	The Dark Knight	533345358	1022345358	185000000	Jul 18 2008	PG

7.3.2.2 Using .loc[] - Label-Based Selection

Syntax: `df.loc[row_indexer, column_indexer]`

Key Features:

- Uses **actual labels** (index names, column names)
- **Inclusive** of both endpoints in slicing
- Can combine with boolean conditions
- Most **intuitive** for data analysis

Example: Let's subset the Title, Worldwide Gross, Production Budget, and IMDB Rating of the top 3 movies using specific row indices that obtained from the above sorting table.

```
# Select specific rows by index and specific columns by name
movies_subset = movie_ratings_sorted.loc[[182, 2084, 790], ['Title', 'IMDB Rating', 'US Gross']]
print(" Selected movies with key financial and rating data:")
movies_subset
```

Selected movies with key financial and rating data:

	Title	IMDB Rating	US Gross	Worldwide Gross	Production Budget
182	The Shawshank Redemption	9.2	28241469	28241469	25000000
2084	Inception	9.1	285630280	753830280	160000000
790	Schindler's List	8.9	96067179	321200000	25000000

The `:` symbol in `.loc` is a slicing operator that represents a range or all elements in the specified dimension (rows or columns). Use `:` alone to select all rows/columns, or with start/end points to slice specific parts of the DataFrame.

```
# Select ALL rows (:) but only specific columns
movies_subset = movie_ratings_sorted.loc[:, ['Title', 'Worldwide Gross', 'Production Budget']]
print(" All movies with financial and rating columns only:")
movies_subset
```

All movies with financial and rating columns only:

	Title	Worldwide Gross	Production Budget	IMDB Rating
182	The Shawshank Redemption	28241469	25000000	9.2
2084	Inception	753830280	160000000	9.1
790	Schindler's List	321200000	25000000	8.9
1962	Pulp Fiction	212928762	8000000	8.9
561	The Dark Knight	1022345358	185000000	8.9
...	...	...	...	...
1051	Glitter	4273372	8500000	2.0
1495	Disaster Movie	34690901	20000000	1.7
1116	Crossover	7009668	5600000	1.7
805	From Justin to Kelly	4922166	12000000	1.6
1147	Super Babies: Baby Geniuses 2	9109322	20000000	1.4

```
# Select a RANGE of rows (182 to 561) and specific columns
movies_subset = movie_ratings_sorted.loc[182:561, ['Title', 'Worldwide Gross', 'Production Budget', 'IMDB Rating']]
print("Movies from index 182 to 561 (inclusive):")
movies_subset
```

Movies from index 182 to 561 (inclusive):

	Title	Worldwide Gross	Production Budget	IMDB Rating
182	The Shawshank Redemption	28241469	25000000	9.2
2084	Inception	753830280	160000000	9.1
790	Schindler's List	321200000	25000000	8.9
1962	Pulp Fiction	212928762	8000000	8.9
561	The Dark Knight	1022345358	185000000	8.9

7.3.2.3 Combining .loc with Boolean Conditions

Power Move: Filter rows AND select columns simultaneously using boolean logic!

```
# extracting the rows that have IMDB Rating greater than 8 or US Gross less than 1000000, on the same time
movie_ratings[(movie_ratings['IMDB Rating'] > 8) & (movie_ratings['US Gross'] < 1000000)][['Title', 'Worldwide Gross', 'Production Budget', 'IMDB Rating']]

#using loc to extract the rows that have IMDB Rating greater than 8 or US Gross less than 1000000, on the same time
movie_ratings.loc[(movie_ratings['IMDB Rating'] > 8) & (movie_ratings['US Gross'] < 1000000)][['Title', 'Worldwide Gross', 'Production Budget', 'IMDB Rating']]
```

	Title	IMDB Rating
21	Gandhi, My Father	8.1
636	Lake of Fire	8.4

7.3.2.4 Using .iloc[] - Position-Based Selection

Syntax: `df.iloc[row_positions, column_positions]`

Key Features:

- Uses **integer positions** (0-based indexing like Python lists)
- **Exclusive** of end point in slicing (like Python slicing)
- Great for **systematic sampling** or **first/last N records**
- Think: “integer location”

Position Type	Example	Description
Single	<code>df.iloc[0, 1]</code>	First row, second column
List	<code>df.iloc[[0, 2], [1, 3]]</code>	Specific positions
Slice	<code>df.iloc[0:3, 1:4]</code>	Range (exclusive end)
All	<code>df.iloc[:, :]</code>	All rows and columns

Memory Tip:

- `iloc` = integer positions,
- `loc` = label names

<IPython.core.display.Image object>

```
# let's check the movie_ratings_sorted DataFrame
movie_ratings_sorted.head()
```

	Title	US Gross	Worldwide Gross	Production Budget	Release Date	MP
182	The Shawshank Redemption	28241469	28241469	25000000	Sep 23 1994	R
2084	Inception	285630280	753830280	160000000	Jul 16 2010	PG
790	Schindler's List	96067179	321200000	25000000	Dec 15 1993	R
1962	Pulp Fiction	107928762	212928762	8000000	Oct 14 1994	R
561	The Dark Knight	533345358	1022345358	185000000	Jul 18 2008	PG

After sorting, the position-based index changes, while the label-based index remains unchanged. Let's pass the position-based index to `iloc` to retrieve the top 2 rows from the `movie_ratings_sorted` DataFrame.

```
# Get first 2 rows by POSITION (0 and 1), all columns
top_2_movies = movie_ratings_sorted.iloc[0:2, :]
print(" Top 2 movies by position:")
top_2_movies
```

Top 2 movies by position:

	Title	US Gross	Worldwide Gross	Production Budget	Release Date	MPAA R
182	The Shawshank Redemption	28241469	28241469	25000000	Sep 23 1994	R
2084	Inception	285630280	753830280	160000000	Jul 16 2010	PG

It is important to note that the endpoint is excluded in an `iloc` slice.

For comparison, let's pass the same argument to `loc` and see what it returns.

```
# Same range [0:2] with loc - uses LABELS, includes endpoint
loc_result = movie_ratings_sorted.loc[0:2, :]
print(" Using .loc[0:2,:] - includes rows with labels 0, 1, AND 2:")
loc_result
```

Using `.loc[0:2,:]` - includes rows with labels 0, 1, AND 2:

	Title	US Gross	Worldwide Gross	Production Budget	Release Date	MPAA R
0	Opal Dreams	14443	14443	9000000	Nov 22 2006	PG/PG-1
851	Star Trek: Generations	75671262	120000000	38000000	Nov 18 1994	PG/PG-1
140	Tuck Everlasting	19161999	19344615	15000000	Oct 11 2002	PG/PG-1
708	De-Lovely	13337299	18396382	4000000	Jun 25 2004	PG/PG-1
705	Flyboys	13090630	14816379	60000000	Sep 22 2006	PG/PG-1
...	...	...	...	...	...	...
955	The Brothers Solomon	900926	900926	10000000	Sep 07 2007	R
1637	Drumline	56398162	56398162	20000000	Dec 13 2002	PG/PG-1
1610	Hollywood Homicide	30207785	51107785	75000000	Jun 13 2003	PG/PG-1
569	Doom	28212337	54612337	70000000	Oct 21 2005	R
2	The Informers	315000	315000	18000000	Apr 24 2009	R

As you can see, `:` is used in the same way as in `loc` to denote a range of the index. All rows between label indices 0 and 2, inclusive of both ends, are returned.

```
# Select first 10 rows and specific column positions [0, 2, 3, 9]
movies_iloc_subset = movie_ratings_sorted.iloc[0:10, [0, 2, 3, 9]]
print(" First 10 movies with columns at positions 0, 2, 3, 9:")
movies_iloc_subset
```

First 10 movies with columns at positions 0, 2, 3, 9:

	Title	Worldwide Gross	Production Budget	IMDB R
182	The Shawshank Redemption	28241469	25000000	9.2
2084	Inception	753830280	160000000	9.1
790	Schindler's List	321200000	25000000	8.9
1962	Pulp Fiction	212928762	8000000	8.9
561	The Dark Knight	1022345358	185000000	8.9
2092	Toy Story 3	1046340665	200000000	8.9
487	The Lord of the Rings: The Fellowship of the Ring	868621686	109000000	8.8
335	Fight Club	100853753	65000000	8.8
497	The Lord of the Rings: The Return of the King	1133027325	94000000	8.8
1081	C'era una volta il West	5321508	5000000	8.8

Critical Thinking Question: Can you use `.iloc` for conditional filtering (like `df.iloc[df['column'] > 5]`)?

Answer: **No!** `.iloc` only accepts **integer positions**, not boolean conditions. For conditional filtering, you must use `.loc` or boolean indexing directly.

7.3.2.5 Summary: `.loc` vs `.iloc` - Choose Your Weapon!

Feature	<code>.loc</code> (Label-Based)	<code>.iloc</code> (Position-Based)
Uses	Labels/Names	Integer Positions
Slicing	<code>[start:end]</code> includes end	<code>[start:end]</code> excludes end
Examples	<code>df.loc['row_name', 'col_name']</code>	<code>df.iloc[0, 1]</code>
Conditions	<code>df.loc[df['col'] > 5]</code>	No boolean conditions
Best For	Business logic filtering	Systematic sampling

Mental Model:

- `.loc` = “Give me the data **by name/label**”
- `.iloc` = “Give me the data **by position number**”

7.3.3 Finding Extremes in pandas: Values, Labels, and Positions

7.3.3.1 Finding extreme values

When you need to find extreme values, pandas offers powerful methods:

Method	Purpose	Returns
<code>min()</code>	Actual minimum value	The minimum value itself
<code>max()</code>	Actual maximum value	The maximum value itself

```
# find the max/min worldwide gross
max_gross = max(movie_ratings["Worldwide Gross"])
min_gross = min(movie_ratings["Worldwide Gross"])

print("Highest gross:", max_gross)
print("Minimum gross:", min_gross)
```

```
Highest gross: 2767891499
Minimum gross: 884
```

7.3.3.2 Finding extreme labels

Method	Purpose	Returns	Use With
<code>idxmax()</code>	Label of maximum	Index label	<code>.loc</code>
<code>idxmin()</code>	Label of minimum	Index label	<code>.loc</code>

Pro Tip: Use `idx` methods to **locate** the extreme values, then use `.loc` to get the full row!

`min()` / `max()` vs. `idxmin()` / `idxmax()`

- Use `min()` / `max()` when you only need the **extreme value** itself.
- Use `idxmin()` / `idxmax()` when you need to know **where that extreme occurs** — i.e., the **index label**, so you can fetch the corresponding row.

```
# Find the INDEX LABELS of movies with max/min worldwide gross
max_gross_index = movie_ratings_sorted['Worldwide Gross'].idxmax()
min_gross_index = movie_ratings_sorted['Worldwide Gross'].idxmin()

print(" Highest grossing movie index:", max_gross_index)
print(" Lowest grossing movie index:", min_gross_index)

# Now get the full details of these movies
print("\n Highest grossing movie:")
print(movie_ratings_sorted.loc[max_gross_index, ['Title', 'Worldwide Gross']])

print("\n Lowest grossing movie:")
print(movie_ratings_sorted.loc[min_gross_index, ['Title', 'Worldwide Gross']])
```

```
Highest grossing movie index: 2094
Lowest grossing movie index: 896
```

```
Highest grossing movie:
Title           Avatar
Worldwide Gross  2767891499
Name: 2094, dtype: object
```

```
Lowest grossing movie:
Title           In Her Line of Fire
Worldwide Gross              884
Name: 896, dtype: object
```

Workflow: `idxmin()/idxmax()` return **index labels** → use with `.loc` to extract full rows

Example: Find the index, then get the actual value:

```
# Get the actual gross values using the indices we found
print(" Max worldwide gross:", f"${movie_ratings_sorted.loc[max_gross_index, 'Worldwide Gross']}")
print(" Min worldwide gross:", f"${movie_ratings_sorted.loc[min_gross_index, 'Worldwide Gross']}")
```

```
Max worldwide gross: $2,767,891,499
Min worldwide gross: $884
```


7.3.3.3 Finding extreme positions

When you need **position numbers** instead of **label names**, you need to use `argmax()` and `argmin()`

Method	Purpose	Returns	Use With
<code>argmax()</code>	Position of maximum	Integer index (0-based)	<code>.iloc</code>
<code>argmin()</code>	Position of minimum	Integer index (0-based)	<code>.iloc</code>

```
# Find POSITIONS (not labels) of movies with max/min worldwide gross
max_position = movie_ratings_sorted['Worldwide Gross'].argmax()
min_position = movie_ratings_sorted['Worldwide Gross'].argmin()

print(" Max gross at position:", max_position)
print(" Min gross at position:", min_position)

# Use iloc with positions to get the actual values
print("\n Max worldwide gross:", f"${movie_ratings_sorted.iloc[max_position, 2]:,.0f}")
print(" Min worldwide gross:", f"${movie_ratings_sorted.iloc[min_position, 2]:,.0f}")

# Get full movie details using positions
print("\n Movies with extreme gross values:")
print("Highest:", movie_ratings_sorted.iloc[max_position]['Title'])
print("Lowest:", movie_ratings_sorted.iloc[min_position]['Title'])
```

```
Max gross at position: 48
Min gross at position: 2149
```

```
Max worldwide gross: $2,767,891,499
Min worldwide gross: $884
```

```
Movies with extreme gross values:
Highest: Avatar
Lowest: In Her Line of Fire
```

Pro Tips for Min/Max Operations

Scenario	Recommended Method	Reason
Custom/Non-unique indices	<code>idxmax()</code> / <code>idxmin()</code>	Returns meaningful labels
Simple position-based work	<code>argmax()</code> / <code>argmin()</code>	Direct integer positions
Find max/min across rows	<code>df.idxmax(axis=1)</code>	Column name with max value per row
Find max/min across columns	<code>df.idxmax(axis=0)</code>	Row index with max value per column

Memory Aid:

- `idx` → **labels** → use with `.loc`
- `arg` → **positions** → use with `.iloc`

7.4 Sorting & Ranking: Organize data for analysis

7.4.1 Why Sorting and Ranking Matter

Sorting and ranking are fundamental operations in data analysis that help you:

- **Identify top performers** (best movies, highest sales)
- **Understand distributions** (from lowest to highest values)
- **Find outliers** (extreme values at the ends)
- **Create ordered presentations** (leaderboards, reports)

7.4.2 Basic Sorting with `sort_values()`

Syntax: `df.sort_values(by='column_name', ascending=True/False)`

Parameter	Purpose	Options
<code>by</code>	Column(s) to sort by	Single column or list of columns
<code>ascending</code>	Sort direction	<code>True</code> (low→high), <code>False</code> (high→low)
<code>na_position</code>	Where to put NaN values	'first' or 'last'
<code>inplace</code>	Modify original DataFrame	<code>True</code> or <code>False</code>

```
# Sort movies by IMDB Rating (ascending - lowest to highest)
movies_by_rating_asc = movie_ratings.sort_values(by='IMDB Rating', ascending=True)
print(" Movies sorted by IMDB Rating (worst to best):")
print(movies_by_rating_asc[['Title', 'IMDB Rating']].head())

print("\n" + "="*50)

# Sort movies by IMDB Rating (descending - highest to lowest)
movies_by_rating_desc = movie_ratings.sort_values(by='IMDB Rating', ascending=False)
print(" Movies sorted by IMDB Rating (best to worst):")
print(movies_by_rating_desc[['Title', 'IMDB Rating']].head())
```

Movies sorted by IMDB Rating (worst to best):

	Title	IMDB Rating
1147	Super Babies: Baby Geniuses 2	1.4
805	From Justin to Kelly	1.6
1495	Disaster Movie	1.7
1116	Crossover	1.7
1051	Glitter	2.0

=====

Movies sorted by IMDB Rating (best to worst):

	Title	IMDB Rating
182	The Shawshank Redemption	9.2
2084	Inception	9.1
790	Schindler's List	8.9
1962	Pulp Fiction	8.9
561	The Dark Knight	8.9

7.4.3 Sorting by Multiple Columns

Real-world scenario: Sort movies by **genre first**, then by **worldwide gross** within each genre

Syntax: `df.sort_values(by=['column1', 'column2'], ascending=[True, False])`

`movie_ratings`

	Title	US Gross	Worldwide Gross	Production Budget	Release Date
0	Opal Dreams	14443	14443	9000000	Nov 22 2006

	Title	US Gross	Worldwide Gross	Production Budget	Release Date	
1	Major Dundee	14873	14873	3800000	Apr 07 1965	1
2	The Informers	315000	315000	18000000	Apr 24 2009	1
3	Buffalo Soldiers	353743	353743	15000000	Jul 25 2003	1
4	The Last Sin Eater	388390	388390	2200000	Feb 09 2007	1
...	...	...	...	...	...	...
2223	King Arthur	51877963	203877963	90000000	Jul 07 2004	1
2224	Mulan	120620254	303500000	90000000	Jun 19 1998	0
2225	Robin Hood	105269730	310885538	210000000	May 14 2010	1
2226	Robin Hood: Prince of Thieves	165493908	390500000	50000000	Jun 14 1991	1
2227	Spiceworld	29342592	56042592	25000000	Jan 23 1998	1

```
# Multi-column sorting: Genre (A-Z), then Worldwide Gross (high to low)
movies_multi_sort = movie_ratings.sort_values(
    by=['Major Genre', 'Worldwide Gross'],
    ascending=[True, False] # Genre A-Z, Gross high-to-low
)

# Pro tip: Check how many unique genres we have
print(f"\n Total unique genres: {movie_ratings['Major Genre'].nunique()}")
print(" Genres:", sorted(movie_ratings['Major Genre'].unique()))

print(" Movies sorted by Major Genre (A-Z), then by Worldwide Gross (high-to-low):")
movies_multi_sort[['Major Genre', 'Title', 'Worldwide Gross']].head(10)
```

Total unique genres: 6

Genres: ['Action/Adventure', 'Comedy', 'Documentary', 'Drama', 'Horror/Thriller', 'Western/

Movies sorted by Major Genre (A-Z), then by Worldwide Gross (high-to-low):

	Major Genre	Title	Worldwide Gross
2094	Action/Adventure	Avatar	2767891499
497	Action/Adventure	The Lord of the Rings: The Return of the King	1133027325
895	Action/Adventure	Pirates of the Caribbean: Dead Man's Chest	1065659812
2092	Action/Adventure	Toy Story 3	1046340665
496	Action/Adventure	Alice in Wonderland	1023291110
561	Action/Adventure	The Dark Knight	1022345358
495	Action/Adventure	Harry Potter and the Sorcerer's Stone	976457891

	Major Genre	Title	Worldwide Gross
894	Action/Adventure	Pirates of the Caribbean: At World's End	960996492
494	Action/Adventure	Harry Potter and the Order of the Phoenix	938468864
493	Action/Adventure	Harry Potter and the Half-Blood Prince	937499905

7.4.4 Ranking Methods with rank()

Purpose: Assign numerical ranks to values (1st, 2nd, 3rd, etc.)

Method	Handling Ties	Example (values: [1, 2, 2, 4])
'average'	Average of tied ranks	[1.0, 2.5, 2.5, 4.0]
'min'	Minimum rank for ties	[1, 2, 2, 4]
'max'	Maximum rank for ties	[1, 3, 3, 4]
'first'	Order of appearance	[1, 2, 3, 4]
'dense'	No gaps in ranks	[1, 2, 2, 3]

Use Cases: Create leaderboards, percentile rankings, performance tiers

```
# Create rankings for IMDB ratings (higher rating = better rank)
movie_ratings['Rating_Rank'] = movie_ratings['IMDB Rating'].rank(ascending=False, method='min')

# Create rankings for worldwide gross (higher gross = better rank)
movie_ratings['Gross_Rank'] = movie_ratings['Worldwide Gross'].rank(ascending=False, method='min')

# Show top 10 movies by IMDB rating with their ranks
top Rated = movie_ratings.nsmallest(10, 'Rating_Rank')
print(" Top 10 Movies by IMDB Rating:")
print(top Rated[['Title', 'IMDB Rating', 'Rating_Rank', 'Worldwide Gross', 'Gross_Rank']].to_string(index=False))

print("\n" + "="*60)

# Compare: Movies that are highly rated vs highly grossing
print("\n Rating vs Gross Performance Analysis:")
comparison = movie_ratings[['Title', 'IMDB Rating', 'Rating_Rank', 'Worldwide Gross', 'Gross_Rank']]
print(comparison.to_string(index=False))
```

Top 10 Movies by IMDB Rating:

	Title	IMDB Rating	Rating_Rank	Worldwide Gross
	The Shawshank Redemption	9.2	1.0	28241469

	Inception	9.1	2.0	753830280
	The Dark Knight	8.9	3.0	1022345358
	Schindler's List	8.9	3.0	321200000
	Pulp Fiction	8.9	3.0	212928762
	Toy Story 3	8.9	3.0	1046340665
	Cidade de Deus	8.8	7.0	28763397
	Fight Club	8.8	7.0	100853753
The Lord of the Rings: The Fellowship of the Ring		8.8	7.0	868621686
The Lord of the Rings: The Return of the King		8.8	7.0	1133027325

=====

Rating vs Gross Performance Analysis:

	Title	IMDB Rating	Rating_Rank	Worldwide Gross	Gross_Rank
	Opal Dreams	6.5	994.0	14443	2223.0
	Major Dundee	6.7	828.0	14873	2222.0
	The Informers	5.2	1808.0	315000	2179.0
	Buffalo Soldiers	6.9	665.0	353743	2176.0
	The Last Sin Eater	5.7	1535.0	388390	2172.0
The City of Your Final Destination		6.6	914.0	493296	2165.0
	The Claim	6.5	994.0	622023	2154.0
	Texas Rangers	5.0	1873.0	623374	2152.0
	Ride With the Devil	6.4	1058.0	630779	2149.0
	Karakter	7.8	167.0	713413	2142.0

7.4.5 Percentile Ranking

Percentile ranking allows you to convert raw ranks into **percentiles (0–100%)**, giving a clearer sense of each value's **relative position** in the dataset.

This is especially useful when comparing across datasets of different sizes or when you want a normalized measure of ranking.

In pandas, you can compute percentile ranks using the `rank()` method with the parameter `pct=True`:

```
# Calculate percentile rankings (0-100%)
movie_ratings['Rating_Percentile'] = movie_ratings['IMDB Rating'].rank(pct=True) * 100
movie_ratings['Gross_Percentile'] = movie_ratings['Worldwide Gross'].rank(pct=True) * 100

# Interpret percentiles
print(" Percentile Interpretation Guide:")
print("• 90th+ percentile = Top 10% (Excellent)")
```

```

print("• 75th+ percentile = Top 25% (Very Good)")
print("• 50th+ percentile = Above Average")
print("• Below 50th = Below Average")

print("\n Top performers in both rating AND gross:")
top_both = movie_ratings[
    (movie_ratings['Rating_Percentile'] >= 90) &
    (movie_ratings['Gross_Percentile'] >= 90)
][['Title', 'IMDB Rating', 'Rating_Percentile', 'Worldwide Gross', 'Gross_Percentile']]

if len(top_both) > 0:
    print(top_both.round(1).to_string(index=False))
else:
    print(" No movies in top 10% for both rating AND gross!")

# Show some examples with percentile context
print("\n Sample movies with percentile context:")
sample = movie_ratings[['Title', 'IMDB Rating', 'Rating_Percentile', 'Worldwide Gross', 'Gross_Percentile']]
print(sample.round(1).to_string(index=False))

```

Percentile Interpretation Guide:

- 90th+ percentile = Top 10% (Excellent)
- 75th+ percentile = Top 25% (Very Good)
- 50th+ percentile = Above Average
- Below 50th = Below Average

Top performers in both rating AND gross:

	Title	IMDB Rating	Rating_Percentile	Worldwide Gross
	The Green Mile	8.4	98.3	
	Shutter Island	8.0	94.8	
	The Curious Case of Benjamin Button	8.0	94.8	
	Jurassic Park 3	7.9	93.3	
	Gone with the Wind	8.2	96.9	
	The Exorcist	8.1	96.1	
	The Bourne Ultimatum	8.2	96.9	
	Jaws	8.3	97.7	
	Shrek	8.0	94.8	
	How to Train Your Dragon	8.2	96.9	
	Casino Royale	8.0	94.8	
	Forrest Gump	8.6	99.1	
	The Lord of the Rings: The Fellowship of the Ring	8.8	99.6	
	Jurassic Park	7.9	93.3	

The Lord of the Rings: The Two Towers	8.7	99.4
The Lord of the Rings: The Return of the King	8.8	99.6
Batman Begins	8.3	97.7
X2	7.8	91.5
300	7.8	91.5
Iron Man	7.9	93.3
The Dark Knight	8.9	99.8
A Beautiful Mind	8.0	94.8
The Pursuit of Happyness	7.8	91.5
Schindler's List	8.9	99.8
The Fugitive	7.8	91.5
Star Trek	8.2	96.9
Pirates of the Caribbean: The Curse of the Black Pearl	8.0	94.8
As Good as it Gets	7.8	91.5
Inglourious Basterds	8.4	98.3
Se7en	8.7	99.4
American Beauty	8.6	99.1
Toy Story	8.2	96.9
Slumdog Millionaire	8.3	97.7
Raiders of the Lost Ark	8.7	99.4
The Last Samurai	7.8	91.5
Gladiator	8.3	97.7
The Matrix	8.7	99.4
The Hangover	7.9	93.3
Saving Private Ryan	8.5	98.8
Toy Story 2	8.0	94.8
Aladdin	7.8	91.5
Terminator 2: Judgment Day	8.5	98.8
WALL-E	8.5	98.8
Ratatouille	8.1	96.1
The Incredibles	8.1	96.1
The Sixth Sense	8.2	96.9
Up	8.4	98.3
Inception	9.1	100.0
The Lion King	8.2	96.9
ET: The Extra-Terrestrial	7.9	93.3
Finding Nemo	8.2	96.9
Toy Story 3	8.9	99.8
Avatar	8.3	97.7
The Departed	8.5	98.8

Sample movies with percentile context:

Title	IMDB Rating	Rating_Percentile	Worldwide Gross	Gross_Percentile
-------	-------------	-------------------	-----------------	------------------

Opal Dreams	6.5	54.0	14443	0.3
Major Dundee	6.7	61.0	14873	0.3
The Informers	5.2	18.2	315000	2.2
Buffalo Soldiers	6.9	68.4	353743	2.4
The Last Sin Eater	5.7	29.9	388390	2.6

7.4.6 Advanced Sorting Techniques

7.4.6.1 Custom Sorting with `sort_values(key=function)`

The `key` parameter in `sort_values()` lets you apply a **transformation function** to column values **before sorting**.

This is useful when the natural order of values is not the same as the order you want to sort by.

```
# Custom sorting: Sort movie titles by length (shortest to longest)
movies_by_title_length = movie_ratings.sort_values(
    by='Title',
    key=lambda x: x.str.len() # Sort by title length, not alphabetically
)

print(" Movies sorted by title length (shortest to longest):")
title_length_sample = movies_by_title_length[['Title', 'IMDB Rating']].head(8)
for idx, row in title_length_sample.iterrows():
    print(f" '{row['Title']}' ({len(row['Title'])} chars) - {row['IMDB Rating']}")

print("\n" + "="*50)

# Sort by absolute deviation from average rating (find most "average" movies)
avg_rating = movie_ratings['IMDB Rating'].mean()
movies_by_avg_deviation = movie_ratings.sort_values(
    by='IMDB Rating',
    key=lambda x: abs(x - avg_rating) # Sort by distance from average
)

print(f"\n Movies closest to average rating ({avg_rating:.1f}):")
avg_movies = movies_by_avg_deviation[['Title', 'IMDB Rating']].head(5)
for idx, row in avg_movies.iterrows():
    deviation = abs(row['IMDB Rating'] - avg_rating)
    print(f" '{row['Title']}' - Rating: {row['IMDB Rating']} (±{deviation:.2f} from avg)")
```

Movies sorted by title length (shortest to longest):

```
'9' (1 chars) - 7.8
'54' (2 chars) - 5.6
'Up' (2 chars) - 8.4
'Pi' (2 chars) - 7.5
'X2' (2 chars) - 7.8
'21' (2 chars) - 6.7
'Woo' (3 chars) - 3.4
'Saw' (3 chars) - 7.7
```

=====

Movies closest to average rating (6.2):

```
'Beyond Borders' - Rating: 6.2 ( $\pm 0.04$  from avg)
'Come Early Morning' - Rating: 6.2 ( $\pm 0.04$  from avg)
'Bad Boys II' - Rating: 6.2 ( $\pm 0.04$  from avg)
'Drop Dead Gorgeous' - Rating: 6.2 ( $\pm 0.04$  from avg)
'The Sisterhood of the Traveling Pants 2' - Rating: 6.2 ( $\pm 0.04$  from avg)
```

7.4.6.2 Index Sorting with `sort_index()`

Sort DataFrame by **row indices** or **column names** rather than values

```
# First, let's see the current index order
print(" Current index range:")
print(f"First 5 indices: {list(movie_ratings.index[:5])}")
print(f>Last 5 indices: {list(movie_ratings.index[-5:])}")

# Sort by index in ascending order (restore original row order)
movies_index_sorted = movie_ratings.sort_index()
print(f"\n After sorting by index (ascending):")
print(f"First 5 indices: {list(movies_index_sorted.index[:5])}")

# Sort by index in descending order
movies_index_desc = movie_ratings.sort_index(ascending=False)
print(f"\n After sorting by index (descending):")
print(f"First 5 indices: {list(movies_index_desc.index[:5])}")

# Sort columns alphabetically by column names
print(f"\n Original column order:")
print(f"Columns: {list(movie_ratings.columns)}")
```

```

movies_cols_sorted = movie_ratings.sort_index(axis=1) # axis=1 for columns
print(f"\n Columns after alphabetical sorting:")
print(f"Columns: {list(movies_cols_sorted.columns)}")

```

Current index range:

First 5 indices: [0, 1, 2, 3, 4]

Last 5 indices: [2223, 2224, 2225, 2226, 2227]

After sorting by index (ascending):

First 5 indices: [0, 1, 2, 3, 4]

After sorting by index (descending):

First 5 indices: [2227, 2226, 2225, 2224, 2223]

Original column order:

Columns: ['Title', 'US Gross', 'Worldwide Gross', 'Production Budget', 'Release Date', 'MPAA']

Columns after alphabetical sorting:

Columns: ['Creative Type', 'Gross_Percentile', 'Gross_Rank', 'IMDB Rating', 'IMDB Votes', 'MPAA']

7.4.7 Top-N and Bottom-N Selection

When you want the **largest or smallest N values** in a Series or DataFrame, pandas provides efficient methods that avoid a full sort:

- **nlargest(n)** → returns the top **n** rows with the highest values
- **nsmallest(n)** → returns the bottom **n** rows with the lowest values

These are **faster than sorting** the entire DataFrame with `.sort_values()`, especially for large datasets.

```

# Top 5 highest-grossing movies (using nlargest)
print(" Top 5 Highest-Grossing Movies:")
top_gross = movie_ratings.nlargest(5, 'Worldwide Gross')[['Title', 'Worldwide Gross', 'IMDB Rating']]
for idx, row in top_gross.iterrows():
    print(f" {row['Title']} - ${row['Worldwide Gross']:,.0f} ( {row['IMDB Rating']})")

print("\n" + "="*50)

# Top 5 highest-rated movies (using nlargest)

```

```

print("\n Top 5 Highest-Rated Movies:")
top_rated = movie_ratings.nlargest(5, 'IMDB Rating')[['Title', 'IMDB Rating', 'Worldwide Gross']]
for idx, row in top_rated.iterrows():
    print(f" {row['Title']} - {row['IMDB Rating']} ({row['Worldwide Gross']:,.0f})")

print("\n" + "="*50)

# Bottom 5 lowest-rated movies (using nsmallest)
print("\n Bottom 5 Lowest-Rated Movies:")
bottom_rated = movie_ratings.nsmallest(5, 'IMDB Rating')[['Title', 'IMDB Rating', 'Worldwide Gross']]
for idx, row in bottom_rated.iterrows():
    print(f" {row['Title']} - {row['IMDB Rating']} ({row['Worldwide Gross']:,.0f})")

print("\n" + "="*50)

# Bottom 5 lowest-grossing movies (using nsmallest)
print("\n Bottom 5 Lowest-Grossing Movies:")
bottom_gross = movie_ratings.nsmallest(5, 'Worldwide Gross')[['Title', 'Worldwide Gross', 'IMDB Rating']]
for idx, row in bottom_gross.iterrows():
    print(f" {row['Title']} - ${row['Worldwide Gross']:,.0f} ( {row['IMDB Rating']})")

```

Top 5 Highest-Grossing Movies:

Avatar - \$2,767,891,499 (8.3)
 Titanic - \$1,842,879,955 (7.4)
 The Lord of the Rings: The Return of the King - \$1,133,027,325 (8.8)
 Pirates of the Caribbean: Dead Man's Chest - \$1,065,659,812 (7.3)
 Toy Story 3 - \$1,046,340,665 (8.9)

=====

Top 5 Highest-Rated Movies:

The Shawshank Redemption - 9.2 (\$28,241,469)
 Inception - 9.1 (\$753,830,280)
 The Dark Knight - 8.9 (\$1,022,345,358)
 Schindler's List - 8.9 (\$321,200,000)
 Pulp Fiction - 8.9 (\$212,928,762)

=====

Bottom 5 Lowest-Rated Movies:

Super Babies: Baby Geniuses 2 - 1.4 (\$9,109,322)
 From Justin to Kelly - 1.6 (\$4,922,166)

Crossover - 1.7 (\$7,009,668)
 Disaster Movie - 1.7 (\$34,690,901)
 Son of the Mask - 2.0 (\$59,918,422)

=====

Bottom 5 Lowest-Grossing Movies:
 In Her Line of Fire - \$884 (3.5)
 The Californians - \$4,134 (5.1)
 Peace, Propaganda and the Promised Land - \$4,930 (3.0)
 Say Uncle - \$5,361 (5.7)
 London - \$12,667 (7.7)

7.4.8 Performance Comparison: Sorting Methods

Method	Best For	Performance	Use Case
<code>sort_values()</code>	Full sorting	Slower for large data	Complete ranking, detailed analysis
<code>nlargest(n)</code>	Top N only	Faster for small N	Leaderboards, top performers
<code>nsmallest(n)</code>	Bottom N only	Faster for small N	Finding outliers, worst performers
<code>rank()</code>	Relative positions	Moderate	Percentiles, competition rankings

Pro Tip: Use `nlargest()`/`nsmallest()` when you only need the top/bottom few records!

7.4.9 Real-World Sorting Scenarios

Let's apply sorting and ranking to solve common business questions:

```
# Scenario 1: Find "Hidden Gems" - High rating but low gross
print(" HIDDEN GEMS: Great movies that didn't make much money")
print("="*55)

# Create efficiency ratio: Rating per dollar (higher = better value)
movie_ratings['Rating_per_Gross'] = movie_ratings['IMDB Rating'] / (movie_ratings['Worldwide Gross'] / 1000000)
```

```

hidden_gems = movie_ratings[
    (movie_ratings['IMDB Rating'] >= 7.5) &
    (movie_ratings['Worldwide Gross'] < movie_ratings['Worldwide Gross'].median())
].nlargest(5, 'IMDB Rating')

for idx, row in hidden_gems.iterrows():
    print(f" {row['Title']}")
    print(f"    Rating: {row['IMDB Rating']} |    Gross: ${row['Worldwide Gross']:,.0f}")
    print()

print("="*55)

# Scenario 2: Find "Blockbuster Hits" - High gross AND high rating
print("\n BLOCKBUSTER HITS: Great movies that made tons of money")
print("="*55)

blockbusters = movie_ratings[
    (movie_ratings['IMDB Rating'] >= 7.0) &
    (movie_ratings['Worldwide Gross'] >= movie_ratings['Worldwide Gross'].quantile(0.8))
].sort_values(['IMDB Rating', 'Worldwide Gross'], ascending=[False, False]).head(5)

for idx, row in blockbusters.iterrows():
    print(f" {row['Title']}")
    print(f"    Rating: {row['IMDB Rating']} |    Gross: ${row['Worldwide Gross']:,.0f}")
    print()

print("="*55)

# Scenario 3: ROI Analysis - Best return on investment
print("\n ROI CHAMPIONS: Best gross-to-budget ratio")
print("="*55)

# Calculate ROI ratio (avoid division by zero)
movie_ratings['ROI_Ratio'] = movie_ratings['Worldwide Gross'] / movie_ratings['Production Budget']

top_roi = movie_ratings.nlargest(5, 'ROI_Ratio')
for idx, row in top_roi.iterrows():
    roi = row['ROI_Ratio']
    print(f" {row['Title']}")
    print(f"    ROI: {roi:.1f}x | Budget: ${row['Production Budget']:,.0f} → Gross: ${row['Worldwide Gross']:,.0f}")
    print()

```

HIDDEN GEMS: Great movies that didn't make much money

=====

The Shawshank Redemption

Rating: 9.2 | Gross: \$28,241,469

Cidade de Deus

Rating: 8.8 | Gross: \$28,763,397

C'era una volta il West

Rating: 8.8 | Gross: \$5,321,508

The Town

Rating: 8.7 | Gross: \$33,180,607

Memento

Rating: 8.7 | Gross: \$39,665,950

=====

BLOCKBUSTER HITS: Great movies that made tons of money

=====

Inception

Rating: 9.1 | Gross: \$753,830,280

Toy Story 3

Rating: 8.9 | Gross: \$1,046,340,665

The Dark Knight

Rating: 8.9 | Gross: \$1,022,345,358

Schindler's List

Rating: 8.9 | Gross: \$321,200,000

Pulp Fiction

Rating: 8.9 | Gross: \$212,928,762

=====

ROI CHAMPIONS: Best gross-to-budget ratio

=====

Paranormal Activity

ROI: 12918.0x | Budget: \$15,000 → Gross: \$193,770,453

Tarnation

ROI: 5330.3x | Budget: \$218 → Gross: \$1,162,014

Super Size Me

ROI: 454.3x | Budget: \$65,000 → Gross: \$29,529,368

The Brothers McMullen

ROI: 417.1x | Budget: \$25,000 → Gross: \$10,426,506

The Blair Witch Project

ROI: 413.8x | Budget: \$600,000 → Gross: \$248,300,000

7.4.10 *Key Takeaways: Sorting & Ranking Mastery**

Essential Methods

- `sort_values()`: Complete sorting by column values
- `rank()`: Assign numerical ranks (1st, 2nd, 3rd...)
- `nlargest()` / `nsmallest()`: Efficient top/bottom N selection
- `sort_index()`: Sort by row indices or column names

Decision Framework

```
# Quick reference for method selection
if "I need top/bottom N only":
    use_nlargest_or_nsmallest()
elif "I need complete ordering":
    use_sort_values()
elif "I need relative positions":
    use_rank()
elif "I need custom ordering logic":
    use_sort_values_with_key_function()
```

Pro Tips

1. **Performance:** `nlargest(5)` beats `sort_values().head(5)` for large datasets
2. **Multiple columns:** `sort_values(['col1', 'col2'], ascending=[True, False])`
3. **Percentiles:** `rank(pct=True) * 100` for 0-100% percentile scores
4. **Custom logic:** Use `key=lambda x: transform(x)` for complex sorting rules
5. **Stability:** pandas sorting is **stable** (preserves original order for ties)

Next up: Learn how to add and remove your sorted data!

7.5 Adding & Removing: Modify DataFrame structure

7.5.1 Why DataFrame Structure Modification Matters

Real-world data analysis often requires:

- **Adding new columns** (calculated fields, derived metrics)
- **Removing columns** (irrelevant features, memory optimization)
- **Removing rows** (outliers, invalid data, filtering)

7.5.2 Adding Columns to DataFrames

There are multiple ways to add new columns, each with different use cases:

7.5.2.1 Method 1: Direct Assignment

The simplest way to add a column with a constant value or calculated field. The added column will be put at the end of the dataframe

```
# Let's work with our movie dataset to demonstrate column operations
print("Original DataFrame shape:", movie_ratings.shape)
print("Original columns:", list(movie_ratings.columns))

# Method 1a: Add a constant value column
movie_ratings['Data_Source'] = 'IMDB'
print(f"\n Added constant column 'Data_Source'")

# Method 1b: Add calculated column based on existing data
movie_ratings['Profit'] = movie_ratings['Worldwide Gross'] - movie_ratings['Production Budget']
print(f" Added calculated column 'Profit'")

# Show results
print(f"\n New DataFrame shape:", movie_ratings.shape)
print(" Sample of new columns:")
movie_ratings[['Title', 'Profit', 'Data_Source']].head()
```

Original DataFrame shape: (2228, 19)

Original columns: ['Title', 'US Gross', 'Worldwide Gross', 'Production Budget', 'Release Date', 'IMDB Rating', 'IMDB Votes', 'Genre', 'Director', 'Cast', 'Country', 'Language', 'Box Office', 'Runtime', 'MPAA Rating', 'Dolby Digital', 'Dolby Atmos', 'Dolby Vision', 'Dolby Surround 7.1']

Added constant column 'Data_Source'

Added calculated column 'Profit'

New DataFrame shape: (2228, 19)

Sample of new columns:

	Title	Profit	Data_Source
0	Opal Dreams	-8985557	IMDB
1	Major Dundee	-3785127	IMDB
2	The Informers	-17685000	IMDB
3	Buffalo Soldiers	-14646257	IMDB
4	The Last Sin Eater	-1811610	IMDB

7.5.2.2 Method 3: Using insert() for Specific Positioning

When you need to **control exactly where** the new column appears

```
movies_copy['Release Date']
```

```
0      Nov 22 2006
1      Apr 07 1965
2      Apr 24 2009
3      Jul 25 2003
4      Feb 09 2007
```

...

```
2223    Jul 07 2004
2224    Jun 19 1998
2225    May 14 2010
2226    Jun 14 1991
2227    Jan 23 1998
```

Name: Release Date, Length: 2228, dtype: object

```
# Method 3: insert() for precise column positioning
# Create a working copy to demonstrate insert()
movies_copy = movie_ratings.copy()

print(" Original column order (first 5):")
print(list(movies_copy.columns[:5]))

# Insert a column at position 2 (after Title and Genre)
```

```
# 1) Parse the date column (e.g., "Nov 22 2006")
movies_copy["Release Date"] = pd.to_datetime(movies_copy["Release Date"], format="%b %d %Y",

movies_copy.insert(2, 'Release_Decade', (movies_copy['Release Date'].dt.year // 10) * 10)

# Insert another column at the beginning (position 0)
movies_copy.insert(0, 'Movie_ID', range(1, len(movies_copy) + 1))

print("\n New column order (first 6):")
print(list(movies_copy.columns[:6]))

# Note: insert() modifies the DataFrame in-place
print(f"\n DataFrame shape after inserts: {movies_copy.shape}")

print("\n Sample with positioned columns:")
movies_copy[['Movie_ID', 'Title', 'Major Genre', 'Release_Decade', 'IMDB Rating']].head()
```

Original column order (first 5):
['Title', 'US Gross', 'Worldwide Gross', 'Production Budget', 'Release Date']

New column order (first 6):
['Movie_ID', 'Title', 'US Gross', 'Release_Decade', 'Worldwide Gross', 'Production Budget']

DataFrame shape after inserts: (2228, 21)

Sample with positioned columns:

	Movie_ID	Title	Major Genre	Release_Decade	IMDB Rating
0	1	Opal Dreams	Drama	2000	6.5
1	2	Major Dundee	Western/Musical	1960	6.7
2	3	The Informers	Horror/Thriller	2000	5.2
3	4	Buffalo Soldiers	Comedy	2000	6.9
4	5	The Last Sin Eater	Drama	2000	5.7

7.5.3 Removing Columns from DataFrames

The most common and flexible way to remove columns is with `.drop()`.

- By default, `.drop()` works on **rows**, so you need to specify `axis=1` (or `columns=`) to drop columns.
- You can remove **one column** or **multiple columns** at the same time.
- `.drop()` returns a **new DataFrame** unless you use `inplace=True`.

```
# Setup: Create a working copy with extra columns for deletion demo
movies_for_deletion = movie_ratings.copy()

# Add some temporary columns to demonstrate deletion
movies_for_deletion['Temp_Col1'] = 'Delete_Me'
movies_for_deletion['Temp_Col2'] = range(len(movies_for_deletion))
movies_for_deletion['Temp_Col3'] = movies_for_deletion['IMDB Rating'] * 2

print(" DataFrame before deletion:")
print(f"Shape: {movies_for_deletion.shape}")
print(f"Columns: {list(movies_for_deletion.columns)}")

# Using drop() - most common and flexible
print(f"\n Using drop()")

# Drop single column
movies_method1 = movies_for_deletion.drop('Temp_Col1', axis=1)
print(f"After dropping 1 column: {movies_method1.shape}")

# Drop multiple columns
movies_method1b = movies_for_deletion.drop(['Temp_Col1', 'Temp_Col2'], axis=1)
print(f"After dropping 2 columns: {movies_method1b.shape}")

print(f"\n Final comparison - all methods result in same shape: {movies_method1b.shape}")
```

DataFrame before deletion:

Shape: (2228, 22)

Columns: ['Title', 'US Gross', 'Worldwide Gross', 'Production Budget', 'Release Date', 'MPAA

Using drop()

After dropping 1 column: (2228, 21)

After dropping 2 columns: (2228, 20)

Final comparison - all methods result in same shape: (2228, 20)

7.5.4 Removing Rows from DataFrames

There are several ways to remove or filter out unwanted rows in a DataFrame:

- using `drop()`
- boolean filtering

7.5.4.1 Method 1: Using `.drop()`

- Works by specifying the **row index labels** to remove.
- Returns a new DataFrame unless you use `inplace=True`.

7.5.4.2 Method2: Boolean Filtering

- Keep only rows that meet a condition (the most common method in data analysis).
- More flexible than `drop()`, especially when conditions depend on column values.

```
# Row removal demonstrations
print(" Original dataset info:")
print(f"Total movies: {len(movie_ratings)}")
print(f"IMDB Rating range: {movie_ratings['IMDB Rating'].min():.1f} - {movie_ratings['IMDB Rating'].max():.1f}")

# Method 1: Remove rows by index using drop()
print(f"\n Method 1: Remove specific rows by index")
movies_drop_index = movie_ratings.drop([0, 1, 2]) # Remove first 3 movies
print(f"After dropping first 3 rows: {len(movies_drop_index)} movies")

# Method 2: Remove rows by condition (Boolean filtering)
print(f"\n Method 2: Remove rows by condition")

# Remove movies with low ratings (< 5.0)
movies_no_low_rating = movie_ratings[movie_ratings['IMDB Rating'] >= 5.0]
removed_count = len(movie_ratings) - len(movies_no_low_rating)
print(f"Removed {removed_count} movies with rating < 5.0")
print(f"Remaining: {len(movies_no_low_rating)} movies")

# Remove movies with missing/zero gross
movies_with_gross = movie_ratings[movie_ratings['Worldwide Gross'] > 0]
removed_no_gross = len(movie_ratings) - len(movies_with_gross)
print(f"Removed {removed_no_gross} movies with no gross data")
```

```

print(f"Remaining: {len(movies_with_gross)} movies")

# Summary of filtering effects
print(f"\n Filtering Summary:")
print(f"Original dataset: {len(movie_ratings)} movies")
print(f"After rating filter: {len(movies_no_low_rating)} movies ({len(movies_no_low_rating)/len(movies)}%)")
print(f"After gross filter: {len(movies_with_gross)} movies ({len(movies_with_gross)/len(movies)}%)")

```

Original dataset info:
Total movies: 2228
IMDB Rating range: 1.4 - 9.2

Method 1: Remove specific rows by index
After dropping first 3 rows: 2225 movies

Method 2: Remove rows by condition
Removed 324 movies with rating < 5.0
Remaining: 1904 movies
Removed 0 movies with no gross data
Remaining: 2228 movies

Filtering Summary:
Original dataset: 2228 movies
After rating filter: 1904 movies (85.5%)
After gross filter: 2228 movies (100.0%)

7.5.5 Method Comparison Summary

Operation	Method	Best For	In-Place?	Performance
Add Column	<code>df['col'] = value</code>	Simple assignment	Yes	Fastest
Add Column	<code>df.insert()</code>	Specific positioning	Yes	Medium
Remove Columns	<code>df.drop()</code>	Flexible removal	No	Fast
Remove Rows	Boolean filtering	Conditional removal	No	Fast
Remove Rows	<code>df.drop()</code>	Index-based removal	No	Fast

7.6 Data Types: Working with Different Data Formats

7.6.1 Why Data Types Matter in Data Science

Data types are **fundamental** to pandas operations:

- **Functionality:** Type-specific methods (`.str`, `.dt` accessors)
- **Speed:** Vectorized operations work best with proper types
- **Data Integrity:** Prevent incorrect calculations and comparisons

7.6.2 Complete Pandas Data Types Reference

Data Type	Description	Memory Efficient	Example	Common Use Cases
<code>int64</code>	64-bit integers	Efficient	1, 2, 3	Counts, IDs, years
<code>float64</code>	64-bit floating point	Efficient	1.5, 2.7, 3.14	Measurements, ratios
<code>object</code>	Strings/Mixed types	Memory heavy	'text', [1,2,3]	Names, categories
<code>category</code>	Categorical data	Super efficient	'A', 'B', 'C'	Fixed categories
<code>datetime64[ns]</code>	Date and time	Efficient	2024-01-15	Timestamps, dates
<code>timedelta64[ns]</code>	Time differences	Efficient	5 days	Duration, intervals
<code>bool</code>	True/False values	Ultra efficient	True, False	Flags, conditions
<code>string</code>	Pure string type	Moderate	'pandas 2.0+'	Text analysis (new)

7.6.3 Data Type Detection and Inspection

```
# Comprehensive data type exploration
print(" DATA TYPE ANALYSIS")
print("="*40)

# Basic data type information
print(" Basic DataFrame Info:")
print(f"Shape: {movie_ratings.shape}")
print(f"\n Data Types Overview:")
print(movie_ratings.dtypes)
```

```
# Data type categorization
print(f"\n Columns by Data Type:")
numeric_cols = movie_ratings.select_dtypes(include=['int64', 'float64']).columns.tolist()
object_cols = movie_ratings.select_dtypes(include=['object']).columns.tolist()
datetime_cols = movie_ratings.select_dtypes(include=['datetime64']).columns.tolist()

print(f" Numeric columns ({len(numeric_cols)}): {numeric_cols}")
print(f" Object columns ({len(object_cols)}): {object_cols}")
print(f" Datetime columns ({len(datetime_cols)}): {datetime_cols}")
```

DATA TYPE ANALYSIS

=====

Basic DataFrame Info:

Shape: (2228, 19)

Data Types Overview:

Title	object
US Gross	int64
Worldwide Gross	int64
Production Budget	int64
Release Date	object
MPAA Rating	object
Source	object
Major Genre	object
Creative Type	object
IMDB Rating	float64
IMDB Votes	int64
Rating_Rank	float64
Gross_Rank	float64
Rating_Percentile	float64
Gross_Percentile	float64
Rating_per_Gross	float64
ROI_Ratio	float64
Data_Source	object
Profit	int64

dtype: object

Columns by Data Type:

Numeric columns (12): ['US Gross', 'Worldwide Gross', 'Production Budget', 'IMDB Rating', 'IMDB Votes', 'Rating_Rank', 'Gross_Rank', 'Rating_Percentile', 'Gross_Percentile', 'Rating_per_Gross', 'ROI_Ratio', 'Profit']
Object columns (7): ['Title', 'Release Date', 'MPAA Rating', 'Source', 'Major Genre', 'Creative Type', 'Data_Source']
Datetime columns (0): []

7.6.4 Smart Data Type Filtering with select_dtypes()

Purpose: Efficiently select columns based on their data types for targeted analysis

```
# Select columns by data type for targeted analysis
print(" DATA TYPE FILTERING EXAMPLES")
print("="*45)

# 1. Select object (string/categorical) columns
print(" Object/String Columns:")
object_data = movie_ratings.select_dtypes(include='object')
print(f"Columns: {list(object_data.columns)}")
print("Sample data:")
print(object_data.head(3))

print("\n" + "="*45)
```

DATA TYPE FILTERING EXAMPLES

```
=====
Object/String Columns:
Columns: ['Title', 'Release Date', 'MPAA Rating', 'Source', 'Major Genre', 'Creative Type',
Sample data:
```

	Title	Release Date	MPAA Rating	Source \
0	Opal Dreams	Nov 22 2006	PG/PG-13	Adapted screenplay
1	Major Dundee	Apr 07 1965	PG/PG-13	Adapted screenplay
2	The Informers	Apr 24 2009	R	Adapted screenplay

	Major Genre	Creative Type	Data_Source
0	Drama	Fiction	IMDB
1	Western/Musical	Fiction	IMDB
2	Horror/Thriller	Fiction	IMDB

```
=====

# 2. Select numeric columns (int + float)
print(" Numeric Columns:")
numeric_data = movie_ratings.select_dtypes(include='number')
print(f"Columns: {list(numeric_data.columns)}")
print("Sample data:")
print(numeric_data.head(3))

print("\n" + "="*45)
```

```

# 3. Advanced filtering with multiple types
print("Advanced Type Filtering:")

# Select only integers
int_data = movie_ratings.select_dtypes(include=['int64'])
print(f"Integer columns: {list(int_data.columns)}")

# Select only floats
float_data = movie_ratings.select_dtypes(include=['float64'])
print(f"Float columns: {list(float_data.columns)}")

# Exclude object types (get only numeric + datetime)
non_object = movie_ratings.select_dtypes(exclude=['object'])
print(f"Non-object columns: {list(non_object.columns)}")

# 4. Statistical summary by type
print(f"\n Statistical Summary for Numeric Data:")
print(numeric_data.describe().round(2))

print(f"\n Unique Values in Object Columns:")
for col in object_data.columns:
    unique_count = object_data[col].nunique()
    print(f" {col:<15}: {unique_count:>3} unique values")
    if unique_count <= 5: # Show values if few unique
        print(f"     Values: {list(object_data[col].unique())}")

```

Numeric Columns:

Columns: ['US Gross', 'Worldwide Gross', 'Production Budget', 'IMDB Rating', 'IMDB Votes', 'IMDB Votes', 'IMDB Votes']

Sample data:

	US Gross	Worldwide Gross	Production Budget	IMDB Rating	IMDB Votes \
0	14443	14443	9000000	6.5	468
1	14873	14873	3800000	6.7	2588
2	315000	315000	18000000	5.2	7595

	Rating_Rank	Gross_Rank	Rating_Percentile	Gross_Percentile \
0	994.0	2223.0	54.017056	0.269300
1	828.0	2222.0	60.973968	0.314183
2	1808.0	2179.0	18.155296	2.244165

	Rating_per_Gross	ROI_Ratio	Profit
0	450.045005	0.001605	-8985557

```

1          450.480737    0.003914  -3785127
2          16.507937     0.017500 -17685000

```

=====

Advanced Type Filtering:

Integer columns: ['US Gross', 'Worldwide Gross', 'Production Budget', 'IMDB Votes', 'Profit']

Float columns: ['IMDB Rating', 'Rating_Rank', 'Gross_Rank', 'Rating_Percentile', 'Gross_Percentile']

Non-object columns: ['US Gross', 'Worldwide Gross', 'Production Budget', 'IMDB Rating', 'IMDB Votes', 'Profit']

Statistical Summary for Numeric Data:

	US Gross	Worldwide Gross	Production Budget	IMDB Rating \
count	2.228000e+03	2.228000e+03	2.228000e+03	2228.00
mean	5.076370e+07	1.019370e+08	3.816055e+07	6.24
std	6.643081e+07	1.648589e+08	3.782604e+07	1.24
min	0.000000e+00	8.840000e+02	2.180000e+02	1.40
25%	9.646188e+06	1.320737e+07	1.200000e+07	5.50
50%	2.838649e+07	4.266892e+07	2.600000e+07	6.40
75%	6.453140e+07	1.200000e+08	5.300000e+07	7.10
max	7.601676e+08	2.767891e+09	3.000000e+08	9.20

	IMDB Votes	Rating_Rank	Gross_Rank	Rating_Percentile \
count	2228.00	2228.00	2228.00	2228.00
mean	33585.15	1087.83	1114.50	50.02
std	47325.65	646.48	643.31	28.86
min	18.00	1.00	1.00	0.04
25%	6659.25	535.00	557.00	24.71
50%	18169.00	1058.00	1114.50	50.76
75%	40092.75	1653.00	1671.25	74.37
max	519541.00	2228.00	2228.00	100.00

	Gross_Percentile	Rating_per_Gross	ROI_Ratio	Profit
count	2228.00	2228.00	2228.00	2.228000e+03
mean	50.02	6.35	12.59	6.377647e+07
std	28.87	95.26	296.50	1.422922e+08
min	0.04	0.00	0.00	-9.463523e+07
25%	25.03	0.05	0.79	-3.290465e+06
50%	50.02	0.14	1.79	1.450338e+07
75%	75.02	0.45	3.62	7.412914e+07
max	100.00	3959.28	12918.03	2.530891e+09

Unique Values in Object Columns:

```

Title           : 2220 unique values
Release Date    : 1064 unique values

```

```

MPAA Rating      :    4 unique values
  Values: ['PG/PG-13', 'R', 'Other', 'G']
Source           :    2 unique values
  Values: ['Adapted screenplay', 'Original Screenplay']
Major Genre      :    6 unique values
Creative Type    :    2 unique values
  Values: ['Fiction', 'Non-Fiction']
Data_Source      :    1 unique values
  Values: ['IMDB']

```

7.6.5 Data Type Conversion

Often, after the initial reading, we need to convert the datatypes of some of the columns to make them suitable for analysis, as the available functions and operations depend on the column's data type. For example, the datatype of Release Date in the DataFrame `movie_ratings` is object. To perform datetime related computations on this variable, we'll need to convert it to a datetime format. We'll use the Pandas function `to_datetime()` to convert it to a datetime format. Similar functions such as `to_numeric()`, `to_string()` etc., can be used for other conversions.

In Pandas, the `errors='coerce'` parameter is often used in the context of data conversion, specifically when using the `pd.to_numeric` function. This argument tells Pandas to convert values that it can and set the ones it cannot convert to `NaN`. It's a way of gracefully handling errors without raising an exception. Read the textbook for an example

7.6.5.1 Core Conversion Functions

Function	Purpose	Error Handling	Example
<code>pd.to_numeric()</code>	String → Number	<code>errors='coerce'</code>	'123' → 123
<code>pd.to_datetime()</code>	String → DateTime	<code>errors='coerce'</code>	'2024-01-01' → datetime
<code>astype()</code>	Force type change	Raises errors	<code>df['col'].astype('int')</code>
<code>pd.Categorical()</code>	Create categories	N/A	Efficient categorical data

Pro Tip: `errors='coerce'` converts invalid values to `NaN` instead of throwing errors!

```

# Comprehensive data type analysis before conversion
print(" DATA TYPE CONVERSION EXAMPLE")
print("="*40)

```

```

# Check current data type and sample values
print(f" Release Date column analysis:")
print(f"Current data type: {movie_ratings['Release Date'].dtypes}")
print(f"Sample values:")
print(movie_ratings['Release Date'].head())

# Check for problematic values
print(f"\n Data Quality Check:")
print(f"Non-null values: {movie_ratings['Release Date'].count()} / {len(movie_ratings)}")
print(f"Unique values: {movie_ratings['Release Date'].nunique()}")
print(f>Data range: {movie_ratings['Release Date'].min()} to {movie_ratings['Release Date'].max()}")

# Check for potential conversion issues
print(f"\n Potential Issues:")
non_numeric = movie_ratings['Release Date'].apply(lambda x: not str(x).isdigit() if pd.notnull(x))
if non_numeric.any():
    print(f"Non-numeric values found: {non_numeric.sum()}")
    print(f"Examples: {movie_ratings.loc[non_numeric, 'Release Date'].head().tolist()}")
else:
    print(" All values appear to be numeric years")

```

DATA TYPE CONVERSION EXAMPLE

=====

```

Release Date column analysis:
Current data type: object
Sample values:
0    Nov 22 2006
1    Apr 07 1965
2    Apr 24 2009
3    Jul 25 2003
4    Feb 09 2007
Name: Release Date, dtype: object

```

```

Data Quality Check:
Non-null values: 2228 / 2228
Unique values: 1064
Data range: Apr 01 1988 to Sep 30 2006

```

```

Potential Issues:
Non-numeric values found: 2228
Examples: ['Nov 22 2006', 'Apr 07 1965', 'Apr 24 2009', 'Jul 25 2003', 'Feb 09 2007']

```

Next, we'll convert the `Release Date` column in the `DataFrame` to the `datetime` format to facilitate further analysis.

7.6.5.2 Comprehensive Type Conversion Examples

```
# COMPREHENSIVE TYPE CONVERSION EXAMPLES
print(" TYPE CONVERSION DEMONSTRATIONS")
print("="*45)

# 1. DateTime Conversion (Year → Full Date)
print(" 1. DateTime Conversion:")
print(f"Before: {movie_ratings['Release Date'].dtype}")

# Convert years to datetime (assuming January 1st)
movie_ratings['Release Date'] = pd.to_datetime(movie_ratings['Release Date'], format='mixed')
print(f"After: {movie_ratings['Release Date'].dtype}")
print("Sample converted values:")
print(movie_ratings['Release Date'].head())

print("\n" + "="*45)

# 2. Numeric Conversion Example
print(" 2. Numeric Conversion with Error Handling:")

# Create sample data with mixed types for demonstration
sample_data = pd.Series(['100', '200.5', 'invalid', '300', None])
print(f"Original data: {sample_data.tolist()}")

# Safe conversion with error handling
numeric_converted = pd.to_numeric(sample_data, errors='coerce')
print(f"After to_numeric(): {numeric_converted.tolist()}")
print(f>Data type: {numeric_converted.dtype}")

print("\n" + "="*45)

# 3. Custom astype() Conversions
print(" 3. Custom Type Conversions with astype():")

# Create sample numeric data
sample_numeric = pd.Series([1.7, 2.3, 3.9, 4.1, 5.8])
```

```

print(f"Original (float): {sample_numeric.tolist()}")

# Convert float to int (truncates decimals)
int_converted = sample_numeric.astype('int')
print(f"As integer: {int_converted.tolist()}")

# Convert to string
str_converted = sample_numeric.astype('string')
print(f"As string: {str_converted.tolist()}")
print(f"String dtype: {str_converted.dtype}")

print("\n Final DataFrame dtypes after conversions:")
print(movie_ratings.dtypes)

```

TYPE CONVERSION DEMONSTRATIONS

=====

1. DateTime Conversion:

Before: datetime64[ns]

After: datetime64[ns]

Sample converted values:

0 2006-11-22

1 1965-04-07

2 2009-04-24

3 2003-07-25

4 2007-02-09

Name: Release Date, dtype: datetime64[ns]

=====

2. Numeric Conversion with Error Handling:

Original data: ['100', '200.5', 'invalid', '300', None]

After to_numeric(): [100.0, 200.5, nan, 300.0, nan]

Data type: float64

=====

3. Custom Type Conversions with astype():

Original (float): [1.7, 2.3, 3.9, 4.1, 5.8]

As integer: [1, 2, 3, 4, 5]

As string: ['1.7', '2.3', '3.9', '4.1', '5.8']

String dtype: string

Final DataFrame dtypes after conversions:

Title object

US Gross	int64
Worldwide Gross	int64
Production Budget	int64
Release Date	datetime64[ns]
MPAA Rating	object
Source	object
Major Genre	object
Creative Type	object
IMDB Rating	float64
IMDB Votes	int64
Rating_Rank	float64
Gross_Rank	float64
Rating_Percentile	float64
Gross_Percentile	float64
Rating_per_Gross	float64
ROI_Ratio	float64
Data_Source	object
Profit	int64
dtype:	object

7.6.6 Working with DateTime Data: The .dt Accessor

Power Tool: The `.dt` accessor unlocks **30+** **datetime methods** for temporal analysis and feature engineering.

7.6.6.1 Complete DateTime Component Extraction

Component	Method	Output	Business Use Case
Date Parts	<code>.dt.year</code>	2024	Annual trends, cohort analysis
	<code>.dt.month</code>	1-12	Seasonal patterns, monthly reporting
	<code>.dt.day</code>	1-31	Daily analysis, month-end effects
	<code>.dt.quarter</code>	1-4	Quarterly business cycles
Time Parts	<code>.dt.hour</code>	0-23	Hourly patterns, business hours
	<code>.dt.minute</code>	0-59	Minute-level precision
	<code>.dt.second</code>	0-59	Second-level timing
	<code>.dt.weekday</code>	0=Monday, 6=Sunday	Weekday/weekend analysis
Calendar Info	<code>.dt.day_name()</code>	'Monday', 'Tuesday'...	Human-readable days

Component	Method	Output	Business Use Case
Advanced	<code>.dt.month_name()</code>	January', 'February'...	Human-readable months
	<code>.dt.dayofyear</code>	1-366	Day within year
	<code>.dt.week</code>	1-53	Week number
	<code>.dt.is_leap_year</code>	True/False	Leap year detection
	<code>.dt.days_in_month</code>	28-31	Month length

7.6.6.2 Practical DateTime Feature Engineering

```
# COMPREHENSIVE DATETIME FEATURE ENGINEERING
print(" DATETIME FEATURE EXTRACTION")
print("="*45)

# 1. Extract multiple date components
movie_ratings['Release_Year'] = movie_ratings['Release Date'].dt.year
movie_ratings['Release_Month'] = movie_ratings['Release Date'].dt.month
movie_ratings['Release_Quarter'] = movie_ratings['Release Date'].dt.quarter
movie_ratings['Release_Weekday'] = movie_ratings['Release Date'].dt.weekday
movie_ratings['Release_DayName'] = movie_ratings['Release Date'].dt.day_name()
movie_ratings['Release_MonthName'] = movie_ratings['Release Date'].dt.month_name()

# Display extracted features
print(" Extracted DateTime Features:")
datetime_features = ['Release Date', 'Release_Year', 'Release_Month', 'Release_Quarter', 'Re
print(movie_ratings[datetime_features].head())

print("\n" + "="*45)

# 2. Create business-relevant features
print(" Business-Relevant DateTime Features:")

# Decade classification
movie_ratings['Decade'] = (movie_ratings['Release_Year'] // 10) * 10
print(" Movies by Decade:")
print(movie_ratings['Decade'].value_counts().sort_index())

# Era classification
movie_ratings['Era'] = movie_ratings['Release_Year'].apply(
    lambda x: 'Silent Era' if x < 1930 else
```

```

        'Golden Age' if x < 1960 else
        'New Hollywood' if x < 1980 else
        'Modern Era' if x < 2000 else
        'Digital Era'
    )

print("\n Movies by Era:")
print(movie_ratings['Era'].value_counts())

# Season classification
movie_ratings['Release_Season'] = movie_ratings['Release_Month'].apply(
    lambda x: 'Winter' if x in [12, 1, 2] else
              'Spring' if x in [3, 4, 5] else
              'Summer' if x in [6, 7, 8] else
              'Fall'
)

print("\n Movies by Release Season:")
print(movie_ratings['Release_Season'].value_counts())

print("\n Sample with All Features:")
feature_cols = ['Title', 'Release_Year', 'Decade', 'Era', 'Release_Season', 'Release_DayName']
print(movie_ratings[feature_cols].head())

```

DATETIME FEATURE EXTRACTION

=====

Extracted DateTime Features:

	Release Date	Release_Year	Release_Month	Release_Quarter	Release_DayName
0	2006-11-22	2006	11	4	Wednesday
1	1965-04-07	1965	4	2	Wednesday
2	2009-04-24	2009	4	2	Friday
3	2003-07-25	2003	7	3	Friday
4	2007-02-09	2007	2	1	Friday

=====

Business-Relevant DateTime Features:

Movies by Decade:

Decade	
1950	1
1960	6
1970	8
1980	26

```

1990      595
2000     1536
2010       54
2030        2
Name: count, dtype: int64

```

```

Movies by Era:
Era
Digital Era      1592
Modern Era       621
New Hollywood    14
Golden Age        1
Name: count, dtype: int64

```

```

Movies by Release Season:
Release_Season
Fall      631
Summer    571
Winter    521
Spring    505
Name: count, dtype: int64

```

```

Sample with All Features:
      Title  Release_Year  Decade      Era Release_Season \
0   Opal Dreams      2006   2000  Digital Era         Fall
1   Major Dundee     1965   1960  New Hollywood      Spring
2   The Informers     2009   2000  Digital Era      Spring
3  Buffalo Soldiers     2003   2000  Digital Era      Summer
4  The Last Sin Eater     2007   2000  Digital Era      Winter

```

```

Release_DayName
0   Wednesday
1   Wednesday
2   Friday
3   Friday
4   Friday

```

7.6.6.3 Time Duration Calculations

Business Value: Calculate elapsed time, age of records, time-to-event analysis

```
# let's calculate the days since release till Jan 1st 2024
movie_ratings['Days Since Release'] = (pd.Timestamp.today().normalize() - movie_ratings['Release Date']).dt.days
movie_ratings.head()
```

	Title	US Gross	Worldwide Gross	Production Budget	Release Date	MPAA Rating
0	Opal Dreams	14443	14443	9000000	2006-11-22	PG/PG-13
1	Major Dundee	14873	14873	3800000	1965-04-07	PG/PG-13
2	The Informers	315000	315000	18000000	2009-04-24	R
3	Buffalo Soldiers	353743	353743	15000000	2003-07-25	R
4	The Last Sin Eater	388390	388390	2200000	2007-02-09	PG/PG-13

Filtering Data: Use extracted datetime components to filter rows. Aggregation and grouping using datetime components will be covered in future chapters.

```
# Filter rows where the release month is January
january_releases = movie_ratings[movie_ratings['Release Date'].dt.month == 1]
january_releases.head()
```

	Title	US Gross	Worldwide Gross	Production Budget	Release Date	MPAA Rating
15	Thr3e	1008849	1060418	2400000	2007-01-05	PG/PG-13
57	Impostor	6114237	6114237	40000000	2002-01-04	PG/PG-13
62	The Last Station	6616974	6616974	18000000	2010-01-15	R
63	The Big Bounce	6471394	6626115	50000000	2004-01-30	PG/PG-13
84	Not Easily Broken	10572742	10572742	5000000	2009-01-09	PG/PG-13

7.6.7 Working with object Data

In pandas, the **object** data type is a flexible data type that can store a mix of text (strings), mixed types, or arbitrary Python objects. It is commonly used for string data and is a key part of working with categorical or unstructured data in pandas.

Similar to datetime objects having a **dt** accessor, a number of specialized string methods are available when using the **str** accessor. These methods have in general matching names with the equivalent built-in string methods for single elements, but are applied element-wise on each of the values of the columns.

7.6.7.1 the str accessor in pandas

The `str` accessor in pandas provides a wide range of string methods that allow for efficient and convenient text processing on an entire Series of strings. Here are some commonly used `str` methods:

- String splitting: `str.split()`
- String joining: `str.join()`
- Substrings: `str.slice(start, stop)`, `str[0]`
- String Case Conversion: `str.lower()`, `str.upper()`, `str.capitalize()`
- Whitespace Removal: `str.strip()`, `str.lstrip()`, `str.rstrip()`
- Replacing and Removing: `str.replace('old', 'new')`
- Pattern matching and extraction: `str.contains('pattern')`, `startswith('prefix')`, `endswith('suffix')`
- String length and counting: `str.len()`, `str.count()`

Let's use the well-known titanic dataset to illustrate how to manipulate string columns in pandas dataframe next

```
titanic = pd.read_csv('./Datasets/titanic.csv')
titanic.head()
```

	PassengerId	Survived	Pclass	Name	Age	SibSp	Par
0	1	0	3	Braund, Mr. Owen Harris	22.0	1	0
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	38.0	1	0
2	3	1	3	Heikkinen, Miss. Laina	26.0	0	0
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	35.0	1	0
4	5	0	3	Allen, Mr. William Henry	35.0	0	0

```
titanic.Name
```

```
0          Braund, Mr. Owen Harris
1  Cumings, Mrs. John Bradley (Florence Briggs Th...
2          Heikkinen, Miss. Laina
3  Futrelle, Mrs. Jacques Heath (Lily May Peel)
4          Allen, Mr. William Henry
...
886          Montvila, Rev. Juozas
887          Graham, Miss. Margaret Edith
888  Johnston, Miss. Catherine Helen "Carrie"
889          Behr, Mr. Karl Howell
```

```
890                                Dooley, Mr. Patrick
Name: Name, Length: 891, dtype: object
```

The `Name` column varies in length and contains passengers' last names, titles, and first names. By extracting the title from the `Name` column, we could infer the **sex** of the passenger and add it as a new feature. This could potentially be a significant predictor of survival, as the “ladies first” principle was often applied during the Titanic evacuation. Adding this feature may enhance the model's ability to predict whether a passenger survived.

```
# Let's check the length of the name of each passenger
titanic["Name"].str.len()
```

```
0      23
1      51
2      22
3      44
4      24
..
886    21
887    28
888    40
889    21
890    19
Name: Name, Length: 891, dtype: int64
```

```
# check what is the maximum length of the name
titanic.loc[titanic["Name"].str.len().idxmax(), "Name"]
```

```
'Penasco y Castellana, Mrs. Victor de Satode (Maria Josefa Perez de Soto y Vallejo)'
```

```
# get the observations that contains the word 'Mrs'
titanic[titanic.Name.str.contains('Mrs.')]
```

	PassengerId	Survived	Pclass	Name	Age	SibSp	
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	38.0	1	0
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	35.0	1	0
8	9	1	3	Johnson, Mrs. Oscar W (Elisabeth Vilhelmina Berg)	27.0	0	2
9	10	1	2	Nasser, Mrs. Nicholas (Adele Achem)	14.0	1	0
15	16	1	2	Hewlett, Mrs. (Mary D Kingcome)	55.0	0	0

	PassengerId	Survived	Pclass	Name	Age	SibSp	
...	...	...	...	...	...	...	...
871	872	1	1	Beckwith, Mrs. Richard Leonard (Sallie Monypeny)	47.0	1	...
874	875	1	2	Abelson, Mrs. Samuel (Hannah Wozosky)	28.0	1	...
879	880	1	1	Potter, Mrs. Thomas Jr (Lily Alexenia Wilson)	56.0	0	...
880	881	1	2	Shelley, Mrs. William (Imanita Parrish Hall)	25.0	0	...
885	886	0	3	Rice, Mrs. William (Margaret Norton)	39.0	0	...

```
# get the observations that contains the word 'Mrs'
titanic.Name.str.count('Mrs.').sum()
```

129

```
# split the name column into two columns
titanic.Name.str.split(',', expand=True)
```

	0	1
0	Braund	Mr. Owen Harris
1	Cumings	Mrs. John Bradley (Florence Briggs Thayer)
2	Heikkinen	Miss. Laina
3	Futrelle	Mrs. Jacques Heath (Lily May Peel)
4	Allen	Mr. William Henry
...	...	...
886	Montvila	Rev. Juozas
887	Graham	Miss. Margaret Edith
888	Johnston	Miss. Catherine Helen "Carrie"
889	Behr	Mr. Karl Howell
890	Dooley	Mr. Patrick

```
# get the last part of the split
titanic.Name.str.split(',', expand=True).get(1)
```

```
0          Mr. Owen Harris
1  Mrs. John Bradley (Florence Briggs Thayer)
2          Miss. Laina
3  Mrs. Jacques Heath (Lily May Peel)
4          Mr. William Henry
...
```

```

886                                Rev. Juozas
887                                Miss. Margaret Edith
888                                Miss. Catherine Helen "Carrie"
889                                Mr. Karl Howell
890                                Mr. Patrick
Name: 1, Length: 891, dtype: object

```

7.7 Working with Numerical Data

In data science, **80%** of insights come from numerical analysis. So numerical data analysis matters!

7.7.1 Basic Descriptive Statistics

7.7.1.1 The describe() Method - Your Data's Report Card

```
movie_ratings.describe()
```

	US Gross	Worldwide Gross	Production Budget	Release Date	IMDB Rating
count	2.228000e+03	2.228000e+03	2.228000e+03	2228	2228.000000
mean	5.076370e+07	1.019370e+08	3.816055e+07	2002-07-22 01:12:23.267504512	6.239004
min	0.000000e+00	8.840000e+02	2.180000e+02	1953-02-05 00:00:00	1.400000
25%	9.646188e+06	1.320737e+07	1.200000e+07	1999-07-29 12:00:00	5.500000
50%	2.838649e+07	4.266892e+07	2.600000e+07	2002-12-26 00:00:00	6.400000
75%	6.453140e+07	1.200000e+08	5.300000e+07	2006-06-06 18:00:00	7.100000
max	7.601676e+08	2.767891e+09	3.000000e+08	2039-12-15 00:00:00	9.200000
std	6.643081e+07	1.648589e+08	3.782604e+07	NaN	1.243285

```

# Comprehensive analysis of numerical data
print(" NUMERICAL DATA ANALYSIS OVERVIEW")
print("="*50)

# Display basic info about our dataset
print("\n Dataset Overview:")
print(f"Shape: {movie_ratings.shape}")
print(f"Numerical columns: {movie_ratings.select_dtypes(include=['number']).columns.tolist()}")

# Enhanced describe() with interpretation

```



```

print("\n COMPREHENSIVE STATISTICAL SUMMARY:")
print("="*40)
desc_stats = movie_ratings.describe()
print(desc_stats)

# Interpret the results
print("\n Key Insights from describe():")
print("-"*30)

for col in movie_ratings.select_dtypes(include=['number']).columns:
    if col in desc_stats.columns:
        mean_val = desc_stats.loc['mean', col]
        median_val = desc_stats.loc['50%', col]
        std_val = desc_stats.loc['std', col]

        print(f"\n {col}:")
        print(f"    • Average: {mean_val:.2f}")
        print(f"    • Median: {median_val:.2f}")
        print(f"    • Spread (Std): {std_val:.2f}")

        # Interpret skewness (very rough heuristic using mean vs. median)
        if pd.notna(mean_val) and pd.notna(median_val) and pd.notna(std_val) and std_val > 0:
            skew_direction = "right" if mean_val > median_val else "left"
            if abs(mean_val - median_val) > 0.1 * std_val:
                print(f"    • Distribution: Skewed {skew_direction}")
            else:
                print(f"    • Distribution: Roughly symmetric")
        else:
            print(f"    • Distribution: Not assessed (insufficient variation or NaNs)")

```

NUMERICAL DATA ANALYSIS OVERVIEW

=====

Dataset Overview:

Shape: (2228, 29)

Numerical columns: ['US Gross', 'Worldwide Gross', 'Production Budget', 'IMDB Rating', 'IMDB

COMPREHENSIVE STATISTICAL SUMMARY:

=====

	US Gross	Worldwide Gross	Production Budget \
count	2.228000e+03	2.228000e+03	2.228000e+03
mean	5.076370e+07	1.019370e+08	3.816055e+07

min	0.000000e+00	8.840000e+02	2.180000e+02
25%	9.646188e+06	1.320737e+07	1.200000e+07
50%	2.838649e+07	4.266892e+07	2.600000e+07
75%	6.453140e+07	1.200000e+08	5.300000e+07
max	7.601676e+08	2.767891e+09	3.000000e+08
std	6.643081e+07	1.648589e+08	3.782604e+07

	Release Date	IMDB Rating	IMDB Votes	Rating_Rank \
count	2228	2228.000000	2228.000000	2228.000000
mean	2002-07-22 01:12:23.267504512	6.239004	33585.154847	1087.825853
min	1953-02-05 00:00:00	1.400000	18.000000	1.000000
25%	1999-07-29 12:00:00	5.500000	6659.250000	535.000000
50%	2002-12-26 00:00:00	6.400000	18169.000000	1058.000000
75%	2006-06-06 18:00:00	7.100000	40092.750000	1653.000000
max	2039-12-15 00:00:00	9.200000	519541.000000	2228.000000
std	NaN	1.243285	47325.651561	646.475234

	Gross_Rank	Rating_Percentile	Gross_Percentile	Rating_per_Gross \
count	2228.000000	2228.000000	2228.000000	2228.000000
mean	1114.499551	50.022442	50.022442	6.354252
min	1.000000	0.044883	0.044883	0.002999
25%	557.000000	24.708259	25.033662	0.051100
50%	1114.500000	50.763016	50.022442	0.141194
75%	1671.250000	74.371634	75.022442	0.451079
max	2228.000000	100.000000	100.000000	3959.276018
std	643.312910	28.863900	28.873991	95.256652

	ROI_Ratio	Profit	Release_Year	Release_Month \
count	2228.000000	2.228000e+03	2228.000000	2228.000000
mean	12.594830	6.377647e+07	2002.005386	7.101436
min	0.000884	-9.463523e+07	1953.000000	1.000000
25%	0.787529	-3.290465e+06	1999.000000	4.000000
50%	1.787569	1.450338e+07	2002.000000	7.000000
75%	3.623547	7.412914e+07	2006.000000	10.000000
max	12918.030200	2.530891e+09	2039.000000	12.000000
std	296.498330	1.422922e+08	5.524324	3.399271

	Release_Quarter	Release_Weekday	Decade	Days Since Release
count	2228.000000	2228.000000	2228.000000	2228.000000
mean	2.680880	3.713645	1997.127469	8473.949731
min	1.000000	0.000000	1950.000000	-5186.000000
25%	2.000000	4.000000	1990.000000	7058.250000
50%	3.000000	4.000000	2000.000000	8317.000000

75%	4.000000	4.000000	2000.000000	9562.500000
max	4.000000	6.000000	2030.000000	26538.000000
std	1.111257	0.757334	5.918766	2013.967471

Key Insights from describe():

US Gross:

- Average: 50763702.60
- Median: 28386493.50
- Spread (Std): 66430812.73
- Distribution: Skewed right

Worldwide Gross:

- Average: 101937019.30
- Median: 42668922.50
- Spread (Std): 164858850.98
- Distribution: Skewed right

Production Budget:

- Average: 38160549.18
- Median: 26000000.00
- Spread (Std): 37826043.95
- Distribution: Skewed right

IMDB Rating:

- Average: 6.24
- Median: 6.40
- Spread (Std): 1.24
- Distribution: Skewed left

IMDB Votes:

- Average: 33585.15
- Median: 18169.00
- Spread (Std): 47325.65
- Distribution: Skewed right

Rating_Rank:

- Average: 1087.83
- Median: 1058.00
- Spread (Std): 646.48
- Distribution: Roughly symmetric

Gross_Rank:

- Average: 1114.50
- Median: 1114.50
- Spread (Std): 643.31
- Distribution: Roughly symmetric

Rating_Percentile:

- Average: 50.02
- Median: 50.76
- Spread (Std): 28.86
- Distribution: Roughly symmetric

Gross_Percentile:

- Average: 50.02
- Median: 50.02
- Spread (Std): 28.87
- Distribution: Roughly symmetric

Rating_per_Gross:

- Average: 6.35
- Median: 0.14
- Spread (Std): 95.26
- Distribution: Roughly symmetric

ROI_Ratio:

- Average: 12.59
- Median: 1.79
- Spread (Std): 296.50
- Distribution: Roughly symmetric

Profit:

- Average: 63776470.12
- Median: 14503383.00
- Spread (Std): 142292233.66
- Distribution: Skewed right

Release_Year:

- Average: 2002.01
- Median: 2002.00
- Spread (Std): 5.52
- Distribution: Roughly symmetric

Release_Month:

- Average: 7.10
- Median: 7.00
- Spread (Std): 3.40
- Distribution: Roughly symmetric

Release_Quarter:

- Average: 2.68
- Median: 3.00
- Spread (Std): 1.11
- Distribution: Skewed left

Release_Weekday:

- Average: 3.71
- Median: 4.00
- Spread (Std): 0.76
- Distribution: Skewed left

Decade:

- Average: 1997.13
- Median: 2000.00
- Spread (Std): 5.92
- Distribution: Skewed left

Days Since Release:

- Average: 8473.95
- Median: 8317.00
- Spread (Std): 2013.97
- Distribution: Roughly symmetric

7.7.2 Individual Statistical Methods

Beyond `describe()`, pandas offers **granular statistical methods** for specific analysis needs:

Method	Purpose	Business Example
<code>mean()</code>	Average value	Average customer age, mean sales
<code>median()</code>	Middle value	Median income, typical order size
<code>mode()</code>	Most frequent	Most common product rating
<code>std()</code>	Standard deviation	Risk assessment, quality control
<code>var()</code>	Variance	Data volatility, consistency
<code>min()</code> / <code>max()</code>	Range bounds	Performance benchmarks
<code>quantile()</code>	Percentiles	Customer segments, outlier detection

Method	Purpose	Business Example
<code>sum()</code>	Total	Revenue totals, inventory counts
<code>count()</code>	Non-null values	Data completeness

7.7.2.1 Summary Statistics: Rows vs Columns with axis Parameter

Critical Concept: The `axis` parameter controls the **direction** of calculation: - `axis=0` (default): Calculate **down** the rows (column-wise aggregation) - `axis=1`: Calculate **across** the columns (row-wise aggregation)

Memory Trick: - `axis=0` → “**0** rows away” → Column results - `axis=1` → “**1** column away” → Row results

```
movie_ratings.head()
```

	Title	US Gross	Worldwide Gross	Production Budget	Release Date	MPAA Rating
0	Opal Dreams	14443	14443	9000000	2006-11-22	PG/PG-13
1	Major Dundee	14873	14873	3800000	1965-04-07	PG/PG-13
2	The Informers	315000	315000	18000000	2009-04-24	R
3	Buffalo Soldiers	353743	353743	15000000	2003-07-25	R
4	The Last Sin Eater	388390	388390	2200000	2007-02-09	PG/PG-13

```
movie_ratings.columns
```

```
Index(['Title', 'US Gross', 'Worldwide Gross', 'Production Budget',
      'Release Date', 'MPAA Rating', 'Source', 'Major Genre', 'Creative Type',
      'IMDB Rating', 'IMDB Votes', 'Rating_Rank', 'Gross_Rank',
      'Rating_Percentile', 'Gross_Percentile', 'Rating_per_Gross',
      'ROI_Ratio', 'Data_Source', 'Profit', 'Release_Year', 'Release_Month',
      'Release_Quarter', 'Release_Weekday', 'Release_DayName',
      'Release_MonthName', 'Decade', 'Era', 'Release_Season',
      'Days Since Release'],
      dtype='object')
```

```
# INDIVIDUAL STATISTICAL METHODS DEMONSTRATION
print(" INDIVIDUAL STATISTICAL METHODS")
print("="*50)
```

```

# Select numerical columns for analysis
numeric_cols = ['IMDB Rating', 'IMDB Votes']
print(f"\nAnalyzing columns: {numeric_cols}")

print("\n COLUMN-WISE STATISTICS (axis=0 - default):")
print("-"*45)
print("Mean values (average per column):")
print(movie_ratings[numeric_cols].mean())

print("\nMedian values (middle value per column):")
print(movie_ratings[numeric_cols].median())

print("\nStandard deviation (spread per column):")
print(movie_ratings[numeric_cols].std().round(2))

print("\n RANGE AND DISTRIBUTION:")
print("-"*30)
print("Minimum values:")
print(movie_ratings[numeric_cols].min())

print("\nMaximum values:")
print(movie_ratings[numeric_cols].max())

print("\nQuantiles (25%, 50%, 75%):")
print(movie_ratings[numeric_cols].quantile([0.25, 0.5, 0.75]))

# Business interpretation example
print("\n BUSINESS INSIGHTS:")
print("-"*20)
avg_rating = movie_ratings['IMDB Rating'].mean()
median_votes = movie_ratings['IMDB Votes'].median()

print(f" Average movie quality: {avg_rating:.1f}/10")
print(f" Typical movie popularity: {median_votes:,.0f} votes")

```

INDIVIDUAL STATISTICAL METHODS

=====

Analyzing columns: ['IMDB Rating', 'IMDB Votes']

COLUMN-WISE STATISTICS (axis=0 - default):

Mean values (average per column):

IMDB Rating 6.239004

IMDB Votes 33585.154847

dtype: float64

Median values (middle value per column):

IMDB Rating 6.4

IMDB Votes 18169.0

dtype: float64

Standard deviation (spread per column):

IMDB Rating 1.24

IMDB Votes 47325.65

dtype: float64

RANGE AND DISTRIBUTION:

Minimum values:

IMDB Rating 1.4

IMDB Votes 18.0

dtype: float64

Maximum values:

IMDB Rating 9.2

IMDB Votes 519541.0

dtype: float64

Quantiles (25%, 50%, 75%):

	IMDB Rating	IMDB Votes
0.25	5.5	6659.25
0.50	6.4	18169.00
0.75	7.1	40092.75

BUSINESS INSIGHTS:

Average movie quality: 6.2/10

Typical movie popularity: 18,169 votes

```
# AXIS PARAMETER DEMONSTRATION - Critical for Data Science!
print(" UNDERSTANDING THE AXIS PARAMETER")
print("=*50)
```



```

# Create a sample DataFrame for clear demonstration
sample_data = {
    'Q1_Sales': [100, 150, 120, 200],
    'Q2_Sales': [110, 160, 130, 210],
    'Q3_Sales': [105, 155, 125, 205],
    'Q4_Sales': [115, 165, 135, 215]
}
quarterly_sales = pd.DataFrame(sample_data,
                                index=['North', 'South', 'East', 'West'])

print(" Sample Quarterly Sales Data:")
print(quarterly_sales)

print("\n AXIS=0 (Column-wise): Statistics DOWN the rows")
print("-"*50)
print("Mean sales per quarter (across all regions):")
col_means = quarterly_sales.mean(axis=0) # Default behavior
print(col_means)

print("\nSum sales per quarter (total across regions):")
col_sums = quarterly_sales.sum(axis=0)
print(col_sums)

print("\n AXIS=1 (Row-wise): Statistics ACROSS the columns")
print("-"*50)
print("Mean sales per region (across all quarters):")
row_means = quarterly_sales.mean(axis=1)
print(row_means)

print("\nTotal sales per region (across all quarters):")
row_sums = quarterly_sales.sum(axis=1)
print(row_sums)

print("\n BUSINESS INTERPRETATION:")
print("-"*25)
best_quarter = col_sums.idxmax()
best_region = row_sums.idxmax()
print(f" Best performing quarter: {best_quarter} (${col_sums[best_quarter]:,})")
print(f" Best performing region: {best_region} (${row_sums[best_region]:,})")

# Practical example with movie data
print(f"\n MOVIE DATA EXAMPLE:")

```

```

print("-"*20)
print("Average rating and votes per movie (axis=1):")
movie_sample = movie_ratings[['IMDB Rating', 'IMDB Votes']].head(3)
print("Sample movies:")
print(movie_sample)
print("\nAverage across rating dimensions per movie:")
print(movie_sample.mean(axis=1)) # Average across columns for each row

```

UNDERSTANDING THE AXIS PARAMETER

=====

Sample Quarterly Sales Data:

	Q1_Sales	Q2_Sales	Q3_Sales	Q4_Sales
North	100	110	105	115
South	150	160	155	165
East	120	130	125	135
West	200	210	205	215

AXIS=0 (Column-wise): Statistics DOWN the rows

Mean sales per quarter (across all regions):

```

Q1_Sales    142.5
Q2_Sales    152.5
Q3_Sales    147.5
Q4_Sales    157.5
dtype: float64

```

Sum sales per quarter (total across regions):

```

Q1_Sales    570
Q2_Sales    610
Q3_Sales    590
Q4_Sales    630
dtype: int64

```

AXIS=1 (Row-wise): Statistics ACROSS the columns

Mean sales per region (across all quarters):

```

North    107.5
South    157.5
East     127.5
West     207.5
dtype: float64

```

```
Total sales per region (across all quarters):
North      430
South      630
East       510
West       830
dtype: int64
```

BUSINESS INTERPRETATION:

```
-----
Best performing quarter: Q4_Sales ($630)
Best performing region: West ($830)
```

MOVIE DATA EXAMPLE:

```
-----
Average rating and votes per movie (axis=1):
Sample movies:
   IMDB Rating  IMDB Votes
0           6.5         468
1           6.7        2588
2           5.2        7595

Average across rating dimensions per movie:
0      237.25
1    1297.35
2    3800.10
dtype: float64
```

7.7.3 Summary: Working with Numerical Data

Concept	Purpose	Key Methods
Basic Statistics	Data overview	<code>describe()</code> , <code>mean()</code> , <code>median()</code> , <code>std()</code>
Axis Operations	Direction control	<code>axis=0</code> (columns), <code>axis=1</code> (rows)

7.8 Understanding `inplace=True` vs `inplace=False`

7.8.1 Why This Matters: Memory, Performance, and Safety

The `inplace` parameter is one of the **most important concepts** in pandas for:

- **Memory management** (avoid unnecessary copies)
- **Data integrity** (prevent accidental modifications)
- **Performance optimization** (reduce overhead)
- **Debugging** (track data transformations)

7.8.2 Core Concept: Modify vs Create

Parameter	Behavior	Memory	Original Data	Return Value	Use Case
<code>inplace=False</code>	Creates new copy	Higher usage	Unchanged	Modified DataFrame	Safe exploration
<code>inplace=True</code>	Modifies original	Lower usage	Changed permanently	None	Final transformations

Critical Rule: `inplace=True` returns None, not the modified DataFrame!

```
# Comprehensive inplace demonstration
print(" INPLACE PARAMETER DEMONSTRATION")
print("="*50)

# Create sample business data
original_data = pd.DataFrame({
    'Employee_ID': [101, 102, 103, 104, 105],
    'Name': ['Alice', 'Bob', 'Charlie', 'Diana', 'Eva'],
    'Department': ['IT', 'HR', 'IT', 'Finance', 'HR'],
    'Salary': [75000, 65000, 80000, 70000, 68000],
    'Years_Experience': [5, 3, 7, 4, 6]
})

print(" Original DataFrame:")
print(original_data)
print(f" DataFrame ID: {id(original_data)}") # Memory address

# Method 1: inplace=False (DEFAULT)
print(f"\n Method 1: inplace=False (Creates New Copy)")
print("-" * 45)

# Add bonus column without modifying original
df_with_bonus = original_data.assign(Bonus=original_data['Salary'] * 0.1)
print(" New DataFrame with bonus:")
```

```

print(df_with_bonus[['Name', 'Salary', 'Bonus']].head(3))
print(f" New DataFrame ID: {id(df_with_bonus)}")

print(f"\n Original DataFrame (unchanged):")
print(original_data.head(3))
print(f" Original ID still: {id(original_data)}")

# Method 2: inplace=True (Modifies Original)
print(f"\n Method 2: inplace=True (Modifies Original)")
print("-" * 45)

# Store original ID for comparison
original_id = id(original_data)

# Add bonus column in place
original_data['Bonus'] = original_data['Salary'] * 0.1
print(" Original DataFrame after in-place modification:")
print(original_data[['Name', 'Salary', 'Bonus']].head(3))
print(f" Same DataFrame ID: {id(original_data)}")
print(f" ID unchanged: {original_id == id(original_data)}")

```

```

      B
0  4
1  5
2  6
   A  B
0  1  4
1  2  5
2  3  6
      B
0  4
1  5
2  6

```

7.8.3 Common Methods Supporting inplace Parameter

```

# Demonstrate inplace with different pandas methods
print(" METHODS SUPPORTING INPLACE PARAMETER")
print("-"*50)

```

```

# Create sample dataset for demonstrations
sample_df = pd.DataFrame({
    'Product': ['Laptop', 'Phone', 'Tablet', 'Phone', 'Laptop'],
    'Price': [1200, 800, 600, 850, 1100],
    'Category': ['Electronics', 'Electronics', 'Electronics', 'Electronics', 'Electronics'],
    'Stock': [10, 0, 15, 8, 12]
})

print(" Sample Dataset:")
print(sample_df)

# 1. DROP operations
print(f"\n 1. DROP OPERATIONS")
print("-" * 30)

# Drop with inplace=False (default)
df_copy = sample_df.copy()
result = df_copy.drop('Category', axis=1) # inplace=False by default
print(" After drop(inplace=False) - returns new DataFrame:")
print(f"Original shape: {df_copy.shape}, New shape: {result.shape}")

# Drop with inplace=True
df_copy2 = sample_df.copy()
return_value = df_copy2.drop('Category', axis=1, inplace=True)
print(f" After drop(inplace=True) - modifies original:")
print(f"Modified shape: {df_copy2.shape}, Return value: {return_value}")

# 2. RENAME operations
print(f"\n 2. RENAME OPERATIONS")
print("-" * 30)

df_copy3 = sample_df.copy()
# Rename with inplace=False
renamed = df_copy3.rename(columns={'Product': 'Item_Name'})
print(f" rename(inplace=False): Original columns unchanged")
print(f"Original: {list(df_copy3.columns)}")
print(f"New: {list(renamed.columns)}")

# Rename with inplace=True
df_copy3.rename(columns={'Product': 'Item_Name'}, inplace=True)
print(f" rename(inplace=True): Original columns changed")
print(f"Modified: {list(df_copy3.columns)}")

```

```

# 3. SORT operations
print(f"\n 3. SORT OPERATIONS")
print("-" * 30)

df_copy4 = sample_df.copy()
# Sort with inplace=False
sorted_df = df_copy4.sort_values('Price')
print(f" sort_values(inplace=False):")
print(f"Original order preserved: {list(df_copy4['Price'])}")
print(f"New sorted order: {list(sorted_df['Price'])}")

# Sort with inplace=True
df_copy4.sort_values('Price', inplace=True)
print(f" sort_values(inplace=True):")
print(f"Original modified: {list(df_copy4['Price'])}")

# 4. FILLNA operations
print(f"\n 4. FILLNA OPERATIONS")
print("-" * 30)

# Create data with NaN
nan_df = pd.DataFrame({
    'A': [1, None, 3, None, 5],
    'B': [None, 2, None, 4, None]
})

print("Original with NaN:")
print(nan_df)

# fillna with inplace=False
filled = nan_df.fillna(0)
print(f"\n fillna(inplace=False) - original unchanged:")
print("Original still has NaN:", nan_df.isna().sum().sum() > 0)
print("New has no NaN:", filled.isna().sum().sum() == 0)

# fillna with inplace=True
nan_df.fillna(0, inplace=True)
print(f" fillna(inplace=True) - original modified:")
print("Original now has no NaN:", nan_df.isna().sum().sum() == 0)

```

METHODS SUPPORTING INPLACE PARAMETER

=====

Sample Dataset:

	Product	Price	Category	Stock
0	Laptop	1200	Electronics	10
1	Phone	800	Electronics	0
2	Tablet	600	Electronics	15
3	Phone	850	Electronics	8
4	Laptop	1100	Electronics	12

1. DROP OPERATIONS

After drop(inplace=False) - returns new DataFrame:

Original shape: (5, 4), New shape: (5, 3)

After drop(inplace=True) - modifies original:

Modified shape: (5, 3), Return value: None

2. RENAME OPERATIONS

rename(inplace=False): Original columns unchanged

Original: ['Product', 'Price', 'Category', 'Stock']

New: ['Item_Name', 'Price', 'Category', 'Stock']

rename(inplace=True): Original columns changed

Modified: ['Item_Name', 'Price', 'Category', 'Stock']

3. SORT OPERATIONS

sort_values(inplace=False):

Original order preserved: [1200, 800, 600, 850, 1100]

New sorted order: [600, 800, 850, 1100, 1200]

sort_values(inplace=True):

Original modified: [600, 800, 850, 1100, 1200]

4. FILLNA OPERATIONS

Original with NaN:

	A	B
0	1.0	NaN
1	NaN	2.0
2	3.0	NaN
3	NaN	4.0
4	5.0	NaN

fillna(inplace=False) - original unchanged:


```
Original still has NaN: True
New has no NaN: True
    fillna(inplace=True) - original modified:
Original now has no NaN: True
```

Complete Methods Reference

Method	Supports inplace	Default	Memory Impact	Use Case
drop()	Yes	False	High without inplace	Remove columns/rows
dropna()	Yes	False	High without inplace	Remove missing values
fillna()	Yes	False	High without inplace	Fill missing values
rename()	Yes	False	Medium without inplace	Rename columns/index
sort_values()	Yes	False	High without inplace	Sort by values
sort_index()	Yes	False	High without inplace	Sort by index
reset_index()	Yes	False	Medium without inplace	Reset index
set_index()	Yes	False	Medium without inplace	Set new index
replace()	Yes	False	High without inplace	Replace values

7.9 Independent Practice

7.9.1 Practice exercise 1

7.9.1.1

Read the file *Top 10 Albums By Year.csv*. This file contains the top 10 albums for each year from 1990 to 2021. Each row corresponds to a unique album.

```
import pandas as pd

album_data = pd.read_csv('./Datasets/Top 10 Albums By Year.csv')
album_data.head()
```

	Year	Ranking	Artist	Album	Worldwide Sales	CDs
0	1990	8	Phil Collins	Serious Hits... Live!	9956520	1
1	1990	1	Madonna	The Immaculate Collection	30000000	1
2	1990	10	The Three Tenors	Carreras Domingo Pavarotti In Concert 1990	8533000	1
3	1990	4	MC Hammer	Please Hammer Don't Hurt Em	18000000	1
4	1990	6	Movie Soundtrack	Aashiqui	15000000	1

7.9.1.2

Print the summary statistics of the data, and answer the following question:

- Why is **Worldwide Sales** not included in the summary statistics table printed in the above step?

```
album_data.describe()
```

	Year	Ranking	CDs	Tracks	Hours	Minutes	Seconds
count	320.000000	320.00000	320.000000	320.000000	320.000000	320.000000	320.000000
mean	2005.500000	5.50000	1.043750	14.306250	0.941406	56.478500	3388.715625
std	9.247553	2.87678	0.246528	5.868995	0.382895	22.970109	1378.209812
min	1990.000000	1.00000	1.000000	6.000000	0.320000	19.430000	1166.000000
25%	1997.750000	3.00000	1.000000	12.000000	0.740000	44.137500	2648.250000
50%	2005.500000	5.50000	1.000000	13.000000	0.860000	51.555000	3093.500000
75%	2013.250000	8.00000	1.000000	15.000000	1.090000	65.112500	3906.750000
max	2021.000000	10.00000	4.000000	67.000000	5.070000	304.030000	18242.000000

```
album_data['Worldwide Sales'].dtype
```

```
dtype('O')
```

7.9.1.3

Update the DataFrame so that `Worldwide Sales` is included in the summary statistics table. Print the summary statistics table.

Hint: Sometimes it may not be possible to convert an object to `numeric()`. For example, the object `'hi'` cannot be converted to a `numeric()` by the python compiler. To avoid getting an error, use the `errors` argument of `to_numeric()` to force such conversions to `NaN` (missing value).

```
album_data['Worldwide Sales'] = pd.to_numeric(album_data['Worldwide Sales'], errors = 'coerce')
album_data.describe()
```

	Year	Ranking	Worldwide Sales	CDs	Tracks	Hours	Minutes	Seconds
count	320.000000	320.00000	3.190000e+02	320.000000	320.000000	320.000000	320.000000	320.000000
mean	2005.500000	5.50000	1.071093e+07	1.043750	14.306250	0.941406	56.478500	338.000000
std	9.247553	2.87678	7.566796e+06	0.246528	5.868995	0.382895	22.970109	137.000000
min	1990.000000	1.00000	1.909009e+06	1.000000	6.000000	0.320000	19.430000	116.000000
25%	1997.750000	3.00000	5.000000e+06	1.000000	12.000000	0.740000	44.137500	264.000000
50%	2005.500000	5.50000	8.255866e+06	1.000000	13.000000	0.860000	51.555000	309.000000
75%	2013.250000	8.00000	1.400000e+07	1.000000	15.000000	1.090000	65.112500	390.000000
max	2021.000000	10.00000	4.500000e+07	4.000000	67.000000	5.070000	304.030000	182.000000

7.9.1.4

Create a new column called `mean_sales_per_year` that computes the average worldwide sales per year for each album, assuming that the worldwide sales are as of 2022. Print the first 5 rows of the updated DataFrame.

```
album_data['mean_sales_per_year'] = album_data['Worldwide Sales']/(2022-album_data['Year'])
album_data.head()
```

	Year	Ranking	Artist	Album	Worldwide Sales	CDs
0	1990	8	Phil Collins	Serious Hits... Live!	9956520.0	1
1	1990	1	Madonna	The Immaculate Collection	30000000.0	1
2	1990	10	The Three Tenors	Carreras Domingo Pavarotti In Concert 1990	8533000.0	1
3	1990	4	MC Hammer	Please Hammer Don't Hurt Em	18000000.0	1
4	1990	6	Movie Soundtrack	Aashiqui	15000000.0	1

7.9.1.5

Find the album having the highest worldwide sales in the dataset, and its artist.

```
# Find album with highest worldwide sales
max_sales_idx = album_data['Worldwide Sales'].idxmax()
print(f"Sales: {album_data.loc[max_sales_idx, 'Worldwide Sales']}")

print(f"Album: {album_data.loc[max_sales_idx, 'Album']}")
print(f"Artist: {album_data.loc[max_sales_idx, 'Artist']}")
```

```
Sales: 45000000.0
Album: The Bodyguard Soundtrack
Artist: Whitney Houston
```

7.9.1.6

Subset the data to include only Hip-Hop albums. How many Hip_Hop albums are there?

```
# Subset data for Hip-Hop albums

hip_hop_albums = album_data[album_data['Genre'] == 'Hip Hop']
print(f"Number of Hip-Hop albums: {len(hip_hop_albums)}")
print("\nHip-Hop albums:")
hip_hop_albums.head()
```

```
Number of Hip-Hop albums: 42
```

```
Hip-Hop albums:
```

	Year	Ranking	Artist	Album	Worldwide Sales	CDs	Tracks	AL
3	1990	4	MC Hammer	Please Hammer Don't Hurt Em	18000000.0	1	13	0:5
63	1996	6	Fugees	The Score	13860000.0	1	13	1:0
101	2000	1	Eminem	The Marshall Mathers LP	32000000.0	1	18	1:3
102	2000	10	Nelly	Country Grammar	10715000.0	1	17	1:0
120	2002	6	Nelly	Nellyville	11000000.0	1	19	1:3

7.9.1.7

Which album amongst hip-hop has the highest mean sales per year per track, and who is its artist?

```
# Find Hip-Hop album with highest mean sales per year per track

hip_hop_albums = album_data[album_data['Genre'] == 'Hip Hop'].copy()
hip_hop_albums['Worldwide Sales'] = pd.to_numeric(hip_hop_albums['Worldwide Sales'], errors = 'coerce')
hip_hop_albums['mean_sales_per_year'] = hip_hop_albums['Worldwide Sales'] / (2022 - hip_hop_albums['Year'])
max_idx = hip_hop_albums['mean_sales_per_year'].idxmax()
print(f"Album: {hip_hop_albums.loc[max_idx, 'Album']}")

print(f"Mean sales per year: {hip_hop_albums.loc[max_idx, 'mean_sales_per_year']:.2f}")

hip_hop_albums['mean_sales_per_year_per_track'] = hip_hop_albums['mean_sales_per_year'] / hip_hop_albums['Tracks']
print(f"Mean sales per year per track: {hip_hop_albums.loc[max_idx, 'mean_sales_per_year_per_track']:.2f}")
```

```
Album:
Mean sales per year: 3,402,981.00
Mean sales per year per track: 309,361.91
```

7.9.2 Practice exercise 2

7.9.2.1

Read the file *STAT303-1 survey for data analysis.csv*.

```
# Read the survey data

survey_data = pd.read_csv('./Datasets/STAT303-1 survey for data analysis.csv')
print("Survey data loaded successfully!")
print(f"Shape: {survey_data.shape}")
survey_data.head()
```

```
Survey data loaded successfully!
Shape: (192, 51)
```

	Timestamp	fav_alcohol	On average (approx.) how many parties a m
0	2022/09/13 1:43:34 pm GMT-5	I don't drink	1
1	2022/09/13 5:28:17 pm GMT-5	Hard liquor/Mixed drink	3
2	2022/09/13 7:56:38 pm GMT-5	Hard liquor/Mixed drink	3
3	2022/09/13 10:34:37 pm GMT-5	Hard liquor/Mixed drink	12
4	2022/09/14 4:46:19 pm GMT-5	I don't drink	1

7.9.2.2

How many observations and variables are there in the data?

```
# Check number of observations and variables

print(f"Number of observations (rows): {survey_data.shape[0]}")
print(f"Number of variables (columns): {survey_data.shape[1]}")
print(f"\nDataset shape: {survey_data.shape}")
```

Number of observations (rows): 192

Number of variables (columns): 51

Dataset shape: (192, 51)

7.9.2.3

Rename all the columns of the data, except the first two columns, with the shorter names in the list `new_col_names` given below. The order of column names in the list is the same as the order in which the columns are to be renamed starting with the third column from the left.

```
new_col_names = ['parties_per_month', 'do_you_smoke', 'weed', 'are_you_an_introvert_or_extrovert']
```

```
# Rename columns (keeping first two original, replacing the rest)

survey_data.columns = list(survey_data.columns[0:2]) + new_col_names
print(f"New column names: {list(survey_data.columns)}")
print("Columns renamed successfully!")
```

New column names: ['Timestamp', 'fav_alcohol', 'parties_per_month', 'do_you_smoke', 'weed', 'are_you_an_introvert_or_extrovert']
Columns renamed successfully!

7.9.2.4

Rename the following columns again:

1. Rename `do_you_smoke` to `smoke`.
2. Rename `are_you_an_introvert_or_extrovert` to `introvert_extrovert`.

Hint: Use the function `rename()`

```
# Rename specific columns using rename function

survey_data = survey_data.rename(columns={

    'do_you_smoke': 'smoke',

    'are_you_an_introvert_or_extrovert': 'introvert_extrovert'})

print(f"Updated columns: {list(survey_data.columns)}")
print("Specific columns renamed successfully!")
```

```
Updated columns: ['Timestamp', 'fav_alcohol', 'parties_per_month', 'smoke', 'weed', 'introvert_extrovert']
Specific columns renamed successfully!
```

7.9.2.5

Find the proportion of people going to more than 4 parties per month. Use the variable `parties_result`.

```
survey_data['parties_per_month']=pd.to_numeric(survey_data.parties_per_month,errors='coerce')
parties_result = survey_data.loc[survey_data['parties_per_month'] > 4, :].shape[0] / survey_data.shape[0]
round(parties_result, 2)
```

```
0.34
```

7.9.2.6

Among the people who go to more than 4 parties a month, what proportion of them are introverts?

```
# Among people who go to more than 4 parties, what proportion are introverts?r
round(survey_data.loc[(survey_data['parties_per_month']>4) & (survey_data['introvert_extrover
], 2)
```

0.51

7.9.2.7

Find the proportion of people in each category of the variable `how_happy`.

```
# Find proportion of people in each happiness category
survey_data['how_happy'].value_counts(normalize=True).round(2)
```

```
how_happy
Pretty happy    0.70
Very happy      0.15
Not too happy   0.09
Don't know      0.06
Name: proportion, dtype: float64
```

7.9.2.8

Among the people who go to more than 4 parties a month, what proportion of them are either `Pretty happy` or `Very happy`?

```
# Among the people who go to more than 4 parties a month, what proportion of them are either
round(survey_data.loc[(survey_data['parties_per_month']>4) & (survey_data['how_happy'].isin(
], 2)
```

0.91

7.9.2.9

Examine the column `num_insta_followers`. Some numbers in the column contain a comma(,) or a tilde(~). Remove both these characters from the numbers in the column.

Hint: You may use the function `str.replace()` of the Pandas Series class.


```
survey_data_insta = survey_data.copy()
survey_data_insta['num_insta_followers']=survey_data_insta['num_insta_followers'].str.replace
survey_data_insta['num_insta_followers']=survey_data_insta['num_insta_followers'].str.replace
```

7.9.2.10

Convert the column `num_insta_followers` to numeric. Coerce the errors.

```
survey_data_insta.num_insta_followers = pd.to_numeric(survey_data_insta.num_insta_followers,
```

7.9.2.11

What is the mean `internet_hours_per_day` for the top 46 people in terms of number of instagram followers?

```
survey_data_insta.sort_values(by = 'num_insta_followers',ascending=False, inplace=True)
top_insta = survey_data_insta.iloc[:46,:].copy()
top_insta.internet_hours_per_day = pd.to_numeric(top_insta.internet_hours_per_day,errors = 'o
top_insta.internet_hours_per_day.mean().round(2)
```

5.09

7.9.2.12

What is the mean `internet_hours_per_day` for the remaining people?

```
# What is the mean `internet_hours_per_day` for the remaining people?
low_insta = survey_data_insta.sort_values('num_insta_followers',ascending=False).iloc[46:,:]
low_insta.internet_hours_per_day = pd.to_numeric(low_insta.internet_hours_per_day,errors = 'o
low_insta.internet_hours_per_day.mean().round(2)
```

13.12

8 Pandas Intermediate

Now that you’ve learned the **fundamentals of pandas**, this chapter shifts focus to **transformations** — techniques that reshape and enrich your data.

You’ll practice vectorized arithmetic, automatic alignment, conditional logic, and row- vs. column-wise operations that turn raw tables into analysis-ready features.

8.1 Learning Objective

At the end of this chapter, you will learn:

- **Arithmetic Operations** — perform calculations within and between DataFrames
- **Broadcasting** — apply operations efficiently across different shapes with automatic alignment
- **Transformations** — use `map()` and `apply()` to transform your data flexibly
- **Lambda Functions** — create quick, inline transformations with `map()` or `apply()`
- **Advanced NLP Preprocessing** — practice transformations that support your final project

Note on Vectorization:

Vectorization means executing operations in **parallel** (built on top of NumPy) instead of using slow Python loops.

In this chapter, we will **motivate NumPy vectorization** and introduce its benefits. A deeper dive will come in the dedicated **NumPy chapter**.

First of all, let’s import

```
import numpy as np
import pandas as pd
import time
```

8.2 Arithmetic Operations: Within and Between DataFrames

Arithmetic operations are essential when working with **numerical data columns** in pandas.

8.2.1 Why Arithmetic Operations Matter in Data Science

They form the backbone of many common tasks, including:

- **Creating new metrics** — e.g., `profit = revenue - cost`
- **Normalizing data** — z-scores, percentages, or scaling
- **Comparing datasets** — growth rates, ratios, or relative differences
- **Financial analysis** — ROI, margins, compound growth
- **Scientific computations** — applying formulas and transformations

Pandas Advantage: Arithmetic in pandas is **intuitive and concise** — operations align automatically by rows and columns, saving you from manual looping.

8.2.2 Arithmetic Operations Within a DataFrame

Core Concept: Perform calculations **between columns, with constants, or across rows**

Operation	Symbol	Example	Business Use Case
Addition	+	<code>df['A'] + df['B']</code>	Total sales = Online + Retail
Subtraction	-	<code>df['Revenue'] - df['Cost']</code>	Profit calculation
Multiplication	*	<code>df['Price'] * df['Quantity']</code>	Total value
Division	/	<code>df['Profit'] / df['Revenue']</code>	Margin percentage
Power	**	<code>df['Principal'] * (1 + rate)**years</code>	Compound interest

Operation	Symbol	Example	Business Use Case
Floor Division	//	df['Total'] // df['BatchSize']	Number of full batches
Modulo	%	df['ID'] % 10	Grouping by last digit

8.2.2.1 Column-to-Column Operations

```
# Create a business-like DataFrame for demonstrations
business_data = {
    'Product': ['Laptop', 'Phone', 'Tablet', 'Watch', 'Headphones'],
    'Units_Sold': [150, 300, 200, 500, 250],
    'Price_per_Unit': [1200, 800, 600, 400, 200],
    'Cost_per_Unit': [800, 500, 400, 250, 120],
    'Marketing_Spend': [15000, 25000, 18000, 30000, 12000]
}
business_df = pd.DataFrame(business_data)

print(" Original Business Data:")
print(business_df)

# Basic arithmetic operations between columns
business_df['Total_Revenue'] = business_df['Units_Sold'] * business_df['Price_per_Unit']
business_df['Total_Cost'] = business_df['Units_Sold'] * business_df['Cost_per_Unit']
business_df['Gross_Profit'] = business_df['Total_Revenue'] - business_df['Total_Cost']
business_df['Net_Profit'] = business_df['Gross_Profit'] - business_df['Marketing_Spend']

# Calculate ratios and percentages
business_df['Profit_Margin'] = (business_df['Gross_Profit'] / business_df['Total_Revenue']) * 100
business_df['ROI'] = (business_df['Net_Profit'] / business_df['Marketing_Spend']) * 100

print("\n Enhanced Business Data with Calculations:")
print(business_df[['Product', 'Total_Revenue', 'Gross_Profit', 'Net_Profit', 'Profit_Margin', 'ROI']])
```

Original Business Data:

	Product	Units_Sold	Price_per_Unit	Cost_per_Unit	Marketing_Spend
0	Laptop	150	1200	800	15000
1	Phone	300	800	500	25000
2	Tablet	200	600	400	18000

3	Watch	500	400	250	30000
4	Headphones	250	200	120	12000

Enhanced Business Data with Calculations:

	Product	Total_Revenue	Gross_Profit	Net_Profit	Profit_Margin	ROI
0	Laptop	180000	60000	45000	33.33	300.00
1	Phone	240000	90000	65000	37.50	260.00
2	Tablet	120000	40000	22000	33.33	122.22
3	Watch	200000	75000	45000	37.50	150.00
4	Headphones	50000	20000	8000	40.00	66.67

8.2.2.2 Operations with Constants and Scalars

In pandas, you can perform arithmetic operations not only between columns, but also with **constants (scalars)**.

The operation is applied **element-wise** across the entire column or DataFrame — thanks to vectorization.

```
# Operations with constants (scalars)
print("Operations with Constants:")

# Apply 10% discount to all prices
business_df['Discounted_Price'] = business_df['Price_per_Unit'] * 0.9

# Add fixed shipping cost of $50 to all products
business_df['Price_with_Shipping'] = business_df['Price_per_Unit'] + 50

# Calculate tax (8% of revenue)
business_df['Tax_Amount'] = business_df['Total_Revenue'] * 0.08

# Square the units sold (for some statistical analysis)
business_df['Units_Squared'] = business_df['Units_Sold'] ** 2

print("Sample with scalar operations:")
sample_cols = ['Product', 'Price_per_Unit', 'Discounted_Price', 'Price_with_Shipping', 'Tax_Amount']
print(business_df[sample_cols].head(3).to_string(index=False))
```

Operations with Constants:

Sample with scalar operations:

Product	Price_per_Unit	Discounted_Price	Price_with_Shipping	Tax_Amount
Laptop	1200	1080.0	1250	14400.0

Phone	800	720.0	850	19200.0
Tablet	600	540.0	650	9600.0

8.2.2.3 Row-wise Operations (Across Columns)

By default, methods like `.mean()`, `.max()`, and `.min()` operate **down each column** (`axis=0`). However, if you need to perform calculations **across all columns in each row**, set `axis=1`.

```
# Row-wise arithmetic operations (axis=1 for across columns)
print(" Row-wise Operations:")

# Create a sample DataFrame with numerical data
sample_data = {
    'Q1_Sales': [100, 150, 200, 120, 180],
    'Q2_Sales': [110, 140, 220, 130, 190],
    'Q3_Sales': [120, 160, 210, 140, 200],
    'Q4_Sales': [130, 170, 230, 150, 210]
}
quarterly_sales = pd.DataFrame(sample_data,
                                index=['Product_A', 'Product_B', 'Product_C', 'Product_D', 'Product_E'])

print(" Quarterly Sales Data:")
print(quarterly_sales)

# Calculate row-wise statistics
quarterly_sales['Total_Sales'] = quarterly_sales.sum(axis=1) # Sum across columns
quarterly_sales['Average_Sales'] = quarterly_sales[['Q1_Sales', 'Q2_Sales', 'Q3_Sales', 'Q4_Sales']].mean(axis=1)
quarterly_sales['Max_Quarter'] = quarterly_sales[['Q1_Sales', 'Q2_Sales', 'Q3_Sales', 'Q4_Sales']].max(axis=1)
quarterly_sales['Sales_Range'] = quarterly_sales[['Q1_Sales', 'Q2_Sales', 'Q3_Sales', 'Q4_Sales']].max(axis=1) - quarterly_sales[['Q1_Sales', 'Q2_Sales', 'Q3_Sales', 'Q4_Sales']].min(axis=1)

print("\n With Row-wise Calculations:")
print(quarterly_sales.round(2))
```

Row-wise Operations:

Quarterly Sales Data:

	Q1_Sales	Q2_Sales	Q3_Sales	Q4_Sales
Product_A	100	110	120	130
Product_B	150	140	160	170
Product_C	200	220	210	230
Product_D	120	130	140	150
Product_E	180	190	200	210

With Row-wise Calculations:

	Q1_Sales	Q2_Sales	Q3_Sales	Q4_Sales	Total_Sales	Average_Sales	\
Product_A	100	110	120	130	460	115.0	
Product_B	150	140	160	170	620	155.0	
Product_C	200	220	210	230	860	215.0	
Product_D	120	130	140	150	540	135.0	
Product_E	180	190	200	210	780	195.0	

	Max_Quarter	Sales_Range
Product_A	130	30
Product_B	170	30
Product_C	230	30
Product_D	150	30
Product_E	210	30

8.2.2.4 Correlation Operations

8.2.3 Arithmetic Operations Between DataFrames

Key Concept: When operating on **two DataFrames**, pandas **automatically aligns** data by **index** and **column** labels.

8.2.3.1 Alignment Behavior

Scenario	Result	Example
Exact match	Normal operation	<code>df1 + df2</code>
Missing index/column	Fills with NaN	Automatic alignment
Different shapes	Broadcasts when possible	Scalar operations
Using fill_value	Replaces NaN with specified value	<code>df1.add(df2, fill_value=0)</code>

Pro Tip: Use **explicit methods** (`add()`, `sub()`, `mul()`, `div()`) for better control over missing data!

8.2.3.2 Basic DataFrame-to-DataFrame Operations

```

# Create two related business DataFrames for comparison
print(" DataFrame-to-DataFrame Operations:")

# Store A performance data
store_a_data = {
    'Electronics': [50000, 55000, 60000],
    'Clothing': [30000, 32000, 35000],
    'Books': [8000, 8500, 9000]
}
store_a = pd.DataFrame(store_a_data, index=['Q1', 'Q2', 'Q3'])

# Store B performance data
store_b_data = {
    'Electronics': [45000, 50000, 58000],
    'Clothing': [28000, 30000, 33000],
    'Books': [7500, 8000, 8800]
}
store_b = pd.DataFrame(store_b_data, index=['Q1', 'Q2', 'Q3'])

print(" Store A Sales:")
print(store_a)
print("\n Store B Sales:")
print(store_b)

# Basic arithmetic operations between DataFrames
print("\n Arithmetic Operations Between Stores:")

# Total sales across both stores
total_sales = store_a + store_b
print("\n Combined Sales (A + B):")
print(total_sales)

# Difference in performance
sales_diff = store_a - store_b
print("\n Sales Difference (A - B):")
print(sales_diff)

# Growth ratio (Store A relative to Store B)
growth_ratio = store_a / store_b
print("\n Performance Ratio (A / B):")
print(growth_ratio.round(3))

```


DataFrame-to-DataFrame Operations:

Store A Sales:

	Electronics	Clothing	Books
Q1	50000	30000	8000
Q2	55000	32000	8500
Q3	60000	35000	9000

Store B Sales:

	Electronics	Clothing	Books
Q1	45000	28000	7500
Q2	50000	30000	8000
Q3	58000	33000	8800

Arithmetic Operations Between Stores:

Combined Sales (A + B):

	Electronics	Clothing	Books
Q1	95000	58000	15500
Q2	105000	62000	16500
Q3	118000	68000	17800

Sales Difference (A - B):

	Electronics	Clothing	Books
Q1	5000	2000	500
Q2	5000	2000	500
Q3	2000	2000	200

Performance Ratio (A / B):

	Electronics	Clothing	Books
Q1	1.111	1.071	1.067
Q2	1.100	1.067	1.062
Q3	1.034	1.061	1.023

8.2.3.3 Handling Misaligned Data

```
# Demonstrate alignment issues and solutions
print(" Handling Misaligned DataFrames:")

# Create DataFrames with different indices and columns
df1_partial = pd.DataFrame({
    'A': [10, 20, 30],
```

```

    'B': [40, 50, 60]
}, index=[0, 1, 2])

df2_partial = pd.DataFrame({
    'B': [5, 10, 15, 20], # Extra row
    'C': [100, 200, 300, 400] # Different column
}, index=[1, 2, 3, 4]) # Different indices

print(" DataFrame 1:")
print(df1_partial)
print("\n DataFrame 2:")
print(df2_partial)

# Default behavior - creates NaN for missing alignments
print("\n Default Addition (creates NaN):")
result_nan = df1_partial + df2_partial
print(result_nan)

# Using explicit methods with fill_value to handle NaN
print("\n Addition with fill_value=0:")
result_filled = df1_partial.add(df2_partial, fill_value=0)
print(result_filled)

# Other explicit arithmetic methods
print("\n Explicit Methods Available:")
print("• add() - Addition with fill_value option")
print("• sub() - Subtraction with fill_value option")
print("• mul() - Multiplication with fill_value option")
print("• div() - Division with fill_value option")

# Example with subtraction
result_sub = df1_partial.sub(df2_partial, fill_value=1)
print("\n Subtraction with fill_value=1:")
print(result_sub)

```

Handling Misaligned DataFrames:

DataFrame 1:

	A	B
0	10	40
1	20	50
2	30	60

DataFrame 2:

	B	C
1	5	100
2	10	200
3	15	300
4	20	400

Default Addition (creates NaN):

	A	B	C
0	NaN	NaN	NaN
1	NaN	55.0	NaN
2	NaN	70.0	NaN
3	NaN	NaN	NaN
4	NaN	NaN	NaN

Addition with fill_value=0:

	A	B	C
0	10.0	40.0	NaN
1	20.0	55.0	100.0
2	30.0	70.0	200.0
3	NaN	15.0	300.0
4	NaN	20.0	400.0

Explicit Methods Available:

- add() - Addition with fill_value option
- sub() - Subtraction with fill_value option
- mul() - Multiplication with fill_value option
- div() - Division with fill_value option

Subtraction with fill_value=1:

	A	B	C
0	9.0	39.0	NaN
1	19.0	45.0	-99.0
2	29.0	50.0	-199.0
3	NaN	-14.0	-299.0
4	NaN	-19.0	-399.0

8.2.4 Advanced Arithmetic Techniques

8.2.4.1 Broadcasting and Shape Compatibility

```
# Broadcasting: Operations between different shapes
print(" Broadcasting Examples:")

# DataFrame with different dimensions
sales_matrix = pd.DataFrame({
    'Jan': [1000, 1500, 2000],
    'Feb': [1100, 1600, 2100],
    'Mar': [1200, 1700, 2200]
}, index=['Store_A', 'Store_B', 'Store_C'])

# Series for operations
monthly_bonus = pd.Series([100, 150, 200], index=['Jan', 'Feb', 'Mar']) # Column-wise
store_multiplier = pd.Series([1.1, 1.2, 1.05], index=['Store_A', 'Store_B', 'Store_C']) # Row-wise

print(" Original Sales Matrix:")
print(sales_matrix)
print("\n Monthly Bonus (Series):")
print(monthly_bonus)
print("\n Store Multiplier (Series):")
print(store_multiplier)

# Broadcasting operations
print("\n Broadcasting Results:")

# Add monthly bonus to each column
sales_with_bonus = sales_matrix + monthly_bonus
print(" Sales + Monthly Bonus (broadcasting across columns):")
print(sales_with_bonus)

# Multiply each row by store multiplier
sales_adjusted = sales_matrix.mul(store_multiplier, axis=0) # axis=0 for row-wise
print("\n Sales × Store Multiplier (broadcasting across rows):")
print(sales_adjusted.round(0))
```

Broadcasting Examples:
Original Sales Matrix:
 Jan Feb Mar

```
Store_A  1000  1100  1200
Store_B  1500  1600  1700
Store_C  2000  2100  2200
```

```
Monthly Bonus (Series):
Jan      100
Feb      150
Mar      200
dtype: int64
```

```
Store Multiplier (Series):
Store_A    1.10
Store_B    1.20
Store_C    1.05
dtype: float64
```

```
Broadcasting Results:
Sales + Monthly Bonus (broadcasting across columns):
      Jan  Feb  Mar
Store_A 1100 1250 1400
Store_B 1600 1750 1900
Store_C 2100 2250 2400
```

```
Sales × Store Multiplier (broadcasting across rows):
      Jan    Feb    Mar
Store_A 1100.0 1210.0 1320.0
Store_B 1800.0 1920.0 2040.0
Store_C 2100.0 2205.0 2310.0
```

8.2.5 Error Handling and Edge Cases

Common issues and solutions when working with arithmetic operations

```
# Handling common arithmetic errors and edge cases
print(" Error Handling and Edge Cases:")

# Create data with potential issues
problematic_data = pd.DataFrame({
    'Values': [10, 0, -5, None, float('inf')],
    'Divisors': [2, 0, 1, 3, 2],
    'Mixed_Types': ['10', 20, '30', 40, '50'] # Mixed string/numeric
})
```

```

print(" Problematic Data:")
print(problematic_data)
print(f>Data types:\n{problematic_data.dtypes}")

# Issue 1: Division by zero
print("\n Division by Zero Issue:")
try:
    result = problematic_data['Values'] / problematic_data['Divisors']
    print("Result with division by zero:")
    print(result)
except Exception as e:
    print(f>Error: {e}")

# Solution: Handle division by zero
print("\n Safe Division:")
safe_division = problematic_data['Values'].div(problematic_data['Divisors']).replace([float(
print(safe_division)

# Issue 2: Mixed data types
print("\n Mixed Data Types Issue:")
try:
    # Convert string column to numeric first
    mixed_numeric = pd.to_numeric(problematic_data['Mixed_Types'], errors='coerce')
    print("Converted mixed types:")
    print(mixed_numeric)

    # Now arithmetic works
    result = mixed_numeric + 5
    print("After adding 5:")
    print(result)
except Exception as e:
    print(f>Error: {e}")

# Issue 3: NaN and infinity handling
print("\n NaN and Infinity Handling:")

# Create data with NaN and inf
test_data = pd.Series([1, 2, None, float('inf'), float('-inf'), 5])
print("Original data with NaN and infinity:")
print(test_data)

# Fill NaN with specific values

```

```

filled_data = test_data.fillna(0)
print("\nAfter filling NaN with 0:")
print(filled_data)

# Replace infinity with finite values
clean_data = filled_data.replace([float('inf'), float('-inf')], [999999, -999999])
print("\nAfter replacing infinity:")
print(clean_data)

# Best practices summary
print("\n Best Practices for Robust Arithmetic:")
print("1. Check data types before operations: df.dtypes")
print("2. Handle NaN values: fillna(), dropna(), or errors='coerce'")
print("3. Check for infinity: df.replace([inf, -inf], value)")
print("4. Use explicit methods for better control: add(), sub(), mul(), div()")
print("5. Validate results: df.isna().sum(), df.isinf().sum() if hasattr(df, 'isinf')")

```

Error Handling and Edge Cases:

Problematic Data:

	Values	Divisors	Mixed_Types
0	10.0	2	10
1	0.0	0	20
2	-5.0	1	30
3	NaN	3	40
4	inf	2	50

Data types:

Values	float64
Divisors	int64
Mixed_Types	object

dtype: object

Division by Zero Issue:

Result with division by zero:

0	5.0
1	NaN
2	-5.0
3	NaN
4	inf

dtype: float64

Safe Division:

0	5.0
---	-----

```
1    NaN
2   -5.0
3    NaN
4    0.0
dtype: float64
```

Mixed Data Types Issue:

Converted mixed types:

```
0    10
1    20
2    30
3    40
4    50
```

Name: Mixed_Types, dtype: int64

After adding 5:

```
0    15
1    25
2    35
3    45
4    55
```

Name: Mixed_Types, dtype: int64

NaN and Infinity Handling:

Original data with NaN and infinity:

```
0    1.0
1    2.0
2    NaN
3    inf
4   -inf
5    5.0
```

dtype: float64

After filling NaN with 0:

```
0    1.0
1    2.0
2    0.0
3    inf
4   -inf
5    5.0
```

dtype: float64

After replacing infinity:

```
0    1.0
```



```

1          2.0
2          0.0
3    999999.0
4   -999999.0
5          5.0
dtype: float64

```

Best Practices for Robust Arithmetic:

1. Check data types before operations: `df.dtypes`
2. Handle NaN values: `fillna()`, `dropna()`, or `errors='coerce'`
3. Check for infinity: `df.replace([inf, -inf], value)`
4. Use explicit methods for better control: `add()`, `sub()`, `mul()`, `div()`
5. Validate results: `df.isna().sum()`, `df.isinf().sum()` if `hasattr(df, 'isinf')`

8.2.6 Performance and Method Comparison

Operation Type	Operator	Explicit Method	Fill Value	Performance	Use Case
Addition	<code>df1 + df2</code>	<code>df1.add(df2, fill_value=0)</code>	Yes	Fast	Missing data control
Subtraction	<code>df1 - df2</code>	<code>df1.sub(df2, fill_value=0)</code>	Yes	Fast	Missing data control
Multiplication	<code>df1 * df2</code>	<code>df1.mul(df2, fill_value=1)</code>	Yes	Fast	Broadcasting control
Division	<code>df1 / df2</code>	<code>df1.div(df2, fill_value=1)</code>	Yes	Fast	Zero division safety
Power	<code>df1 ** 2</code>	<code>df1.pow(2)</code>	No	Medium	Exponential operations
Floor Division	<code>df1 // df2</code>	<code>df1.floordiv(df2)</code>	No	Medium	Integer division
Modulo	<code>df1 % df2</code>	<code>df1.mod(df2)</code>	No	Medium	Remainder operations

8.2.7 Are Arithmetic Operations Vectorized?

Yes — arithmetic operations in pandas are **vectorized** because they rely on NumPy under the hood.

This means operations are applied **element-wise** across entire columns or DataFrames at once, instead of looping through rows in Python.

Benefits of Vectorization:

- Much faster than Python loops (implemented in optimized C code)
- Cleaner and more concise syntax
- Handles **automatic alignment** on indexes and columns

8.3 Correlation Operations: Understanding Relationships Between Variables

8.3.1 What is Correlation?

Correlation measures the **strength and direction** of the linear relationship between two numerical variables. In business and data science, correlation analysis helps you:

- **Discover patterns** in your data
- **Identify relationships** between metrics
- **Make predictions** based on variable relationships
- **Generate insights** for decision-making

Key Concepts:

- **Correlation Causation:** High correlation doesn't mean one variable causes the other
- **Range:** Correlation values range from -1 to +1
- **Interpretation:** Closer to ± 1 = stronger relationship, closer to 0 = weaker relationship

8.3.2 Correlation Coefficient Interpretation

Range	Strength	Direction	Business Example
+0.8 to +1.0	Very Strong Positive	As X , Y	More ads → More sales
+0.6 to +0.8	Strong Positive	As X , Y	Higher price → Higher quality perception
+0.3 to +0.6	Moderate Positive	As X , Y	Customer age → Spending
-0.1 to +0.3	Weak/No Relationship	Random pattern	Product color → Sales
-0.3 to -0.6	Moderate Negative	As X , Y	Discounts → Profit margin

Range	Strength	Direction	Business Example
-0.6 to -0.8	Strong Negative	As X ↓, Y ↓	Product defects → Customer satisfaction
-0.8 to -1.0	Very Strong Negative	As X ↓, Y ↓	Price increase → Demand

8.3.3 Correlation Types

Pandas provides multiple correlation methods that you can specify with `df.corr(method=...)`. By default, it uses **Pearson**, but it's important to know the alternatives and when to use them:

Method	Best For	Advantages	Limitations
Pearson	Linear relationships	Simple, standard, interpretable	Very sensitive to outliers
Spearman	Monotonic (rank-based) relationships	Handles non-linearity well	Loses information about magnitude
Kendall	Robust rank correlation	Highly resistant to outliers	Slower, computationally intensive

8.3.3.1 Decision Guide: Choosing the Right Correlation

```
# Simple decision tree for correlation choice:

if data_has_outliers or is_non_linear_monotonic:
    use_method = "spearman" # Rank-based, robust to outliers
elif small_sample_size or need_extra_robustness:
    use_method = "kendall" # Most robust, but slower
else:
    use_method = "pearson" # Standard choice for linear correlation
```

Note for this chapter: We will focus on Pearson correlation, since it is the default in pandas and most commonly used in practice.

8.3.4 Pandas Correlation Methods

Pandas offers several methods for calculating correlations:

Method	Purpose	Use Case
<code>df.corr()</code>	Full correlation matrix	Explore all relationships
<code>df['A'].corr(df['B'])</code>	Pairwise correlation	Specific relationship
<code>df.corrwith(series)</code>	Correlate with external series	Compare with benchmark

In this chapter, we'll focus on the **first two methods** (`df.corr()` and pairwise `.corr()`), and **skip `corrwith()`** for simplicity.

```
import numpy as np
import pandas as pd

# Create sample data for correlation demonstration
print(" CORRELATION OPERATIONS DEMONSTRATION")
print("="*50)

# Create sample business dataset
np.random.seed(42) # For reproducible results
n_samples = 100

# Create correlated business metrics
sales_data = pd.DataFrame({
    'Advertising_Spend': np.random.normal(1000, 200, n_samples),
    'Website_Traffic': np.random.normal(5000, 1000, n_samples),
    'Customer_Satisfaction': np.random.normal(4.0, 0.5, n_samples),
    'Product_Price': np.random.normal(50, 10, n_samples)
})

# Add some realistic correlations
sales_data['Sales_Revenue'] = (
    0.8 * sales_data['Advertising_Spend'] +
    0.3 * sales_data['Website_Traffic'] +
    200 * sales_data['Customer_Satisfaction'] +
    np.random.normal(0, 500, n_samples)
)

sales_data['Conversion_Rate'] = (
    0.02 + 0.01 * sales_data['Customer_Satisfaction'] -
```

```

    0.0002 * sales_data['Product_Price'] +
    np.random.normal(0, 0.005, n_samples)
)

# Ensure realistic ranges
sales_data['Customer_Satisfaction'] = sales_data['Customer_Satisfaction'].clip(1, 5)
sales_data['Conversion_Rate'] = sales_data['Conversion_Rate'].clip(0.001, 0.1)

print(f"\nDataset shape: {sales_data.shape}")
print(" Sample Business Dataset:")
sales_data.head()

```

CORRELATION OPERATIONS DEMONSTRATION

=====

Dataset shape: (100, 6)
Sample Business Dataset:

	Advertising_Spend	Website_Traffic	Customer_Satisfaction	Product_Price	Sales_Revenue	Conversion_Rate
0	1099.342831	3584.629258	4.178894	41.710050	1993.427949	0.0580
1	972.347140	4579.354677	4.280392	44.398190	2708.075056	0.0634
2	1129.537708	4657.285483	4.541526	57.472936	3211.742785	0.0469
3	1304.605971	4197.722731	4.526901	56.103703	3231.872098	0.0568
4	953.169325	4838.714288	3.311165	49.790984	2651.350074	0.0399

8.3.4.1 Method 1: Full Correlation Matrix

The **correlation matrix** shows relationships between ALL numerical variables at once:

```

# METHOD 1: FULL CORRELATION MATRIX
print(" CORRELATION MATRIX ANALYSIS")
print("="*40)

# Calculate correlation matrix using Pearson (default method)
print(" PEARSON CORRELATION MATRIX")
print("-"*30)
correlation_matrix = sales_data.corr() # Pearson is default
print(correlation_matrix.round(3))

```

```

# Business interpretation
print("\n BUSINESS INTERPRETATION:")
print("-"*25)

# Find strongest correlations
correlation_pairs = []
for i, col1 in enumerate(correlation_matrix.columns):
    for j, col2 in enumerate(correlation_matrix.columns):
        if i < j: # Avoid duplicates and self-correlation
            corr_val = correlation_matrix.loc[col1, col2]
            correlation_pairs.append((col1, col2, corr_val))

# Sort by absolute correlation strength
correlation_pairs.sort(key=lambda x: abs(x[2]), reverse=True)

print(" Top 3 Strongest Relationships:")
for i, (col1, col2, corr_val) in enumerate(correlation_pairs[:3]):
    strength = "Strong" if abs(corr_val) > 0.7 else "Moderate" if abs(corr_val) > 0.3 else "Weak"
    direction = "Positive" if corr_val > 0 else "Negative"
    print(f"{i+1}. {col1}    {col2}")
    print(f"    Correlation: {corr_val:.3f} ({strength} {direction})")

    if corr_val > 0:
        print(f"        As {col1} increases, {col2} tends to increase")
    else:
        print(f"        As {col1} increases, {col2} tends to decrease")
    print()

```

CORRELATION MATRIX ANALYSIS

=====

PEARSON CORRELATION MATRIX

	Advertising_Spend	Website_Traffic	\	
Advertising_Spend	1.000	-0.136		
Website_Traffic	-0.136	1.000		
Customer_Satisfaction	0.185	-0.037		
Product_Price	-0.170	-0.018		
Sales_Revenue	0.085	0.565		
Conversion_Rate	0.094	-0.114		
	Customer_Satisfaction	Product_Price	Sales_Revenue	\
Advertising_Spend	0.185	-0.170	0.085	

Website_Traffic	-0.037	-0.018	0.565
Customer_Satisfaction	1.000	0.005	0.085
Product_Price	0.005	1.000	0.131
Sales_Revenue	0.085	0.131	1.000
Conversion_Rate	0.696	-0.214	-0.079

	Conversion_Rate
Advertising_Spend	0.094
Website_Traffic	-0.114
Customer_Satisfaction	0.696
Product_Price	-0.214
Sales_Revenue	-0.079
Conversion_Rate	1.000

BUSINESS INTERPRETATION:

-
- Top 3 Strongest Relationships:
1. Customer_Satisfaction Conversion_Rate
Correlation: 0.696 (Moderate Positive)
As Customer_Satisfaction increases, Conversion_Rate tends to increase
 2. Website_Traffic Sales_Revenue
Correlation: 0.565 (Moderate Positive)
As Website_Traffic increases, Sales_Revenue tends to increase
 3. Product_Price Conversion_Rate
Correlation: -0.214 (Weak Negative)
As Product_Price increases, Conversion_Rate tends to decrease

8.3.4.2 Method 2: Pairwise Correlation

For analyzing **specific relationships** between two variables:

```
# METHOD 2: PAIRWISE CORRELATION ANALYSIS
print(" PAIRWISE CORRELATION ANALYSIS")
print("="*50)

# Select two interesting columns for detailed analysis
col1, col2 = 'Advertising_Spend', 'Sales_Revenue'

print(f" Analyzing relationship between '{col1}' and '{col2}'")
print("-"*60)
```

```

# Calculate Pearson correlation for the pair (default method)
correlation_value = sales_data[col1].corr(sales_data[col2])

print(f" Correlation coefficient: {correlation_value:.4f}")

# Detailed interpretation
print(f"\n INTERPRETATION:")
print("-"*20)

def interpret_correlation(corr_val):
    abs_corr = abs(corr_val)
    if abs_corr >= 0.8:
        strength = "Very Strong"
    elif abs_corr >= 0.6:
        strength = "Strong"
    elif abs_corr >= 0.3:
        strength = "Moderate"
    elif abs_corr >= 0.1:
        strength = "Weak"
    else:
        strength = "Very Weak/None"

    direction = "Positive" if corr_val > 0 else "Negative"
    return f"{strength} {direction}"

print(f" Relationship strength: {interpret_correlation(correlation_value)}")

# Business context
print(f"\n BUSINESS CONTEXT:")
print("-"*20)
if abs(correlation_value) > 0.5:
    print(" Strong relationship detected!")
    print(f"    • This suggests {col1} and {col2} move together")
    print(f"    • Could be useful for predictions or business decisions")
    if correlation_value > 0:
        print(f"    • Higher {col1} is associated with higher {col2}")
    else:
        print(f"    • Higher {col1} is associated with lower {col2}")
else:
    print(" Moderate/Weak relationship detected")
    print(f"    • {col1} and {col2} don't have a strong linear relationship")
    print(f"    • Consider other factors or non-linear relationships")

```



```
# Demonstrate different variable pairs
print(f"\n OTHER INTERESTING PAIRS:")
print("-"*25)
pairs_to_check = [
    ('Customer_Satisfaction', 'Sales_Revenue'),
    ('Product_Price', 'Conversion_Rate'),
    ('Website_Traffic', 'Sales_Revenue')
]

for pair_col1, pair_col2 in pairs_to_check:
    pair_corr = sales_data[pair_col1].corr(sales_data[pair_col2])
    print(f"{pair_col1}    {pair_col2}: {pair_corr:.3f}")
```

PAIRWISE CORRELATION ANALYSIS

=====

Analyzing relationship between 'Advertising_Spend' and 'Sales_Revenue'

Correlation coefficient: 0.0847

INTERPRETATION:

Relationship strength: Very Weak/None Positive

BUSINESS CONTEXT:

Moderate/Weak relationship detected

- Advertising_Spend and Sales_Revenue don't have a strong linear relationship
- Consider other factors or non-linear relationships

OTHER INTERESTING PAIRS:

Customer_Satisfaction Sales_Revenue: 0.085
 Product_Price Conversion_Rate: -0.214
 Website_Traffic Sales_Revenue: 0.565

8.3.4.3 Advanced Correlation Analysis: Finding Strong Correlations Programmatically

```
# ADVANCED CORRELATION ANALYSIS
print(" ADVANCED CORRELATION TECHNIQUES")
```

```

print("="*50)

# 1. Find strongest correlations programmatically
print(" 1. FINDING STRONGEST CORRELATIONS")
print("-"*35)

correlation_matrix = sales_data.corr() # Uses Pearson correlation by default

# Create a function to find strong correlations
def find_strong_correlations(corr_matrix, threshold=0.5):
    """Find correlations above threshold, excluding diagonal"""
    strong_corrs = []

    for i in range(len(corr_matrix.columns)):
        for j in range(i+1, len(corr_matrix.columns)): # Avoid duplicates and diagonal
            col1 = corr_matrix.columns[i]
            col2 = corr_matrix.columns[j]
            corr_val = corr_matrix.iloc[i, j]

            if pd.notna(corr_val) and abs(corr_val) >= threshold:
                strong_corrs.append({
                    'Variable1': col1,
                    'Variable2': col2,
                    'Correlation': corr_val,
                    'Strength': abs(corr_val)
                })

    return pd.DataFrame(strong_corrs).sort_values('Strength', ascending=False)

strong_correlations = find_strong_correlations(correlation_matrix, threshold=0.3)

if len(strong_correlations) > 0:
    print(f" Found {len(strong_correlations)} correlations above 0.3 threshold:")
    print(strong_correlations.round(3))

    print(f"\n TOP CORRELATION:")
    if len(strong_correlations) > 0:
        top_corr = strong_correlations.iloc[0]
        print(f"   {top_corr['Variable1']}   {top_corr['Variable2']}")
        print(f"   Strength: {top_corr['Correlation']:.3f}")

        if top_corr['Correlation'] > 0:

```

```

        print("    Positive relationship: both increase/decrease together")
    else:
        print("    Negative relationship: one increases as other decreases")
else:
    print(" No strong correlations found above 0.3 threshold")
    print(" Try lowering threshold or check data quality")

# 2. Correlation matrix summary
print(f"\n 2. CORRELATION MATRIX SUMMARY")
print("-"*30)

if len(correlation_matrix) > 1:
    # Summary statistics of correlations
    corr_values = correlation_matrix.values
    # Get upper triangle (excluding diagonal)
    mask = np.triu(np.ones_like(corr_values, dtype=bool), k=1)
    upper_triangle = corr_values[mask]

    print(f" Matrix size: {correlation_matrix.shape[0]} x {correlation_matrix.shape[1]}")
    print(f" Highest correlation: {np.nanmax(upper_triangle):.3f}")
    print(f" Lowest correlation: {np.nanmin(upper_triangle):.3f}")
    print(f" Average correlation: {np.nanmean(upper_triangle):.3f}")

    # Count correlations by strength
    strong_positive = np.sum(upper_triangle >= 0.5)
    strong_negative = np.sum(upper_triangle <= -0.5)
    moderate = np.sum((upper_triangle >= 0.3) & (upper_triangle < 0.5)) + np.sum((upper_triangle < -0.3) & (upper_triangle >= -0.5))
    weak = np.sum((upper_triangle > -0.3) & (upper_triangle < 0.3))

    print(f"\n CORRELATION STRENGTH DISTRIBUTION:")
    print(f"    Strong (|r| >= 0.5):    {strong_positive + strong_negative}")
    print(f"    Moderate (0.3 < |r| < 0.5): {moderate}")
    print(f"    Weak (|r| < 0.3):        {weak}")

print(f"\n PRO TIPS FOR CORRELATION ANALYSIS:")
print("-"*35)
print(" Always visualize correlations with scatter plots")
print(" Check for non-linear relationships")
print(" Consider domain knowledge when interpreting")
print(" Remember: correlation ≠ causation")
print(" Look for outliers that might affect correlations")
print(" Use different correlation methods for robustness")

```

ADVANCED CORRELATION TECHNIQUES

1. FINDING STRONGEST CORRELATIONS

Found 2 correlations above 0.3 threshold:

	Variable1	Variable2	Correlation	Strength
1	Customer_Satisfaction	Conversion_Rate	0.696	0.696
0	Website_Traffic	Sales_Revenue	0.565	0.565

TOP CORRELATION:

Customer_Satisfaction Conversion_Rate

Strength: 0.696

Positive relationship: both increase/decrease together

2. CORRELATION MATRIX SUMMARY

Matrix size: 6 x 6

Highest correlation: 0.696

Lowest correlation: -0.214

Average correlation: 0.072

CORRELATION STRENGTH DISTRIBUTION:

Strong ($|r| \geq 0.5$): 2

Moderate ($0.3 \leq |r| < 0.5$): 0

Weak ($|r| < 0.3$): 13

PRO TIPS FOR CORRELATION ANALYSIS:

Always visualize correlations with scatter plots

Check for non-linear relationships

Consider domain knowledge when interpreting

Remember: correlation $\neq$ causation

Look for outliers that might affect correlations

Use different correlation methods for robustness

8.3.5 Summary: Correlation Operations

Method	Best For	Advantages	Limitations
<code>df.corr()</code>	Overview of all relationships	Complete picture	Can be overwhelming

Method	Best For	Advantages	Limitations
<code>df['A'].corr(df['B'])</code>	Specific pair analysis	Focused, detailed	Limited scope
<code>df.corrwith(series)</code>	Benchmark comparisons	External validation	Requires external data

Key Takeaways: - Correlation reveals relationships but not causation - Always visualize data alongside correlation coefficients
- Use multiple correlation methods for robust analysis - Consider business context when interpreting results - Strong correlations ($|r| > 0.7$) suggest predictive potential

8.3.6 Are Correlation Operations Vectorized?

Yes! Correlation operations in pandas (and NumPy under the hood) are **vectorized**:

- **No explicit loops:** Correlations are computed using optimized NumPy routines in C, avoiding slow Python row-by-row iteration.
- **Matrix-based computation:** `df.corr()` computes the covariance matrix of all numeric columns at once, then scales it into a correlation matrix.
- **Pairwise efficiency:** Even `df['A'].corr(df['B'])` leverages array-level operations on entire columns, not element-by-element loops.
- **Scalability:** Vectorization makes correlation fast and efficient for large datasets, with memory (not speed) usually being the main limitation.

This is why pandas can compute full $n \times n$ correlation matrices quickly, even for wide datasets.

8.4 Advanced Operations: Function Application & Data Transformation

Pandas provides powerful methods for applying functions to DataFrame rows, columns, or individual elements. These methods are the **cornerstone of advanced data manipulation**, enabling complex transformations that would be cumbersome with basic operations.

8.4.1 apply(): The Swiss Army Knife of Data Transformation

The `apply()` function is one of pandas' most versatile tools. It can apply **any custom or built-in function** across DataFrame axes or to elements in a Series, making it ideal for flexible transformations.

8.4.1.1 Core Concepts

- **Row-wise operations** (`axis=1`) → apply a function across rows (each row passed as a Series)
- **Column-wise operations** (`axis=0`) → apply a function across columns (each column passed as a Series)
- **Element-wise operations** → apply a function to each value in a Series

8.4.1.2 Syntax Patterns

```
# DataFrame apply() - row-wise or column-wise
df.apply(function, axis=1)      # Row-wise (rows as Series)
df.apply(function, axis=0)      # Column-wise (columns as Series)

# Series apply() - element-wise
df['column'].apply(function)     # Each element passed individually

# With additional arguments
df.apply(function, axis=1, args=(arg1, arg2))
```

8.4.1.3 Real-World Example: Financial Analysis

```
# Create a comprehensive financial dataset

# Generate realistic financial data
np.random.seed(42)
n_companies = 100

financial_data = {
    'Company': [f'Company_{i:03d}' for i in range(1, n_companies + 1)],
```

```

'Revenue': np.random.normal(50000, 15000, n_companies).round(0),
'Expenses': np.random.normal(35000, 10000, n_companies).round(0),
'Employees': np.random.randint(10, 500, n_companies),
'Market_Cap': np.random.lognormal(15, 1, n_companies).round(0),
'Industry': np.random.choice(['Tech', 'Healthcare', 'Finance', 'Retail', 'Manufacturing'], n_companies),
'Founded_Year': np.random.randint(1990, 2020, n_companies),
'Country': np.random.choice(['USA', 'Canada', 'UK', 'Germany', 'Japan'], n_companies)
}

df_financial = pd.DataFrame(financial_data)

# Ensure expenses don't exceed revenue (business logic)
def cap_expenses(row):
    """
    Ensure expenses don't exceed 95% of revenue
    This prevents companies from having expenses that are too close to or exceed their revenue
    """
    # Calculate the maximum allowed expenses (95% of revenue)
    max_allowed_expenses = row['Revenue'] * 0.95

    # Return the smaller value: either current expenses or the maximum allowed
    if row['Expenses'] > max_allowed_expenses:
        return max_allowed_expenses
    else:
        return row['Expenses']

# Apply the function to each row
df_financial['Expenses'] = df_financial.apply(cap_expenses, axis=1)

# Calculate profit and profit margin for each company
def calculate_profit(row):
    return row['Revenue'] - row['Expenses']

def calculate_profit_margin(row):
    profit = row['Revenue'] - row['Expenses']
    return round((profit / row['Revenue']) * 100, 2) if row['Revenue'] > 0 else 0

# Apply functions to create new columns
df_financial['Profit'] = df_financial.apply(calculate_profit, axis=1)
df_financial['Profit_Margin'] = df_financial.apply(calculate_profit_margin, axis=1)

```

```
print(f"\nDataset shape: {df_financial.shape}")
print(f"Industries: {df_financial['Industry'].unique()}")

print(" Financial Dataset Sample:")
df_financial.head(10)
```

```
Dataset shape: (100, 10)
Industries: ['Retail' 'Manufacturing' 'Tech' 'Finance' 'Healthcare']
Financial Dataset Sample:
```

	Company	Revenue	Expenses	Employees	Market_Cap	Industry	Founded_Year	Country
0	Company_001	57451.0	20846.0	395	2340214.0	Retail	2010	USA
1	Company_002	47926.0	30794.0	483	2378910.0	Manufacturing	1994	UK
2	Company_003	59715.0	31573.0	282	4491813.0	Tech	1999	Germany
3	Company_004	72845.0	26977.0	113	651982.0	Tech	1999	Germany
4	Company_005	46488.0	33387.0	426	8413782.0	Finance	2008	USA
5	Company_006	46488.0	39041.0	402	11598738.0	Manufacturing	2006	UK
6	Company_007	73688.0	53862.0	308	4767327.0	Retail	2010	UK
7	Company_008	61512.0	36746.0	255	17044719.0	Tech	2003	UK
8	Company_009	42958.0	37576.0	185	867324.0	Retail	1998	Japan
9	Company_010	58138.0	34256.0	48	20660119.0	Tech	2003	Canada

8.4.1.4 How apply() Actually Works

`apply()` is essentially a **Python-level loop** that iterates over each **row** or **column** and applies your function one-by-one.

It's **much slower** than true **vectorized** operations (NumPy ufuncs, pandas built-ins).

Prefer vectorized operations whenever possible.

Use `apply()` only when logic can't be expressed with vectorized methods.

8.4.2 map(): Efficient Element-wise Transformations (on a Series)

`Series.map()` applies a transformation **element by element** and is often faster/simpler than `apply()` for 1-to-1 lookups or simple functions.

8.4.2.1 When to use

- **Value mapping / recoding** (e.g., codes → labels)
- **Category encoding** (labels → numbers)
- **Simple per-value transforms** (string cleanup, parsing, normalization)
- **Lookup from a codebook** stored as a dict or another Series

8.4.2.2 Three Ways to Use map()

```
# Sample data
df = pd.DataFrame({
    'Industry': ['Tech', 'Healthcare', 'Finance', 'Tech', 'Retail'],
    'Size': ['Small', 'Large', 'Medium', 'Small', 'Large'],
    'Revenue': [100000, 150000, 80000, 120000, 90000]
})

# 1. **Dictionary Mapping** - Most Common
industry_codes = {'Tech': 1, 'Healthcare': 2, 'Finance': 3, 'Retail': 4}
df['Industry_Code'] = df['Industry'].map(industry_codes)

# 2. **Function Mapping**
def add_prefix(industry):
    return f"IND_{industry}"

df['Industry_Prefixed'] = df['Industry'].map(add_prefix)

# 3. **Series Mapping**
size_priority = pd.Series({'Small': 3, 'Medium': 2, 'Large': 1})
df['Size_Priority'] = df['Size'].map(size_priority)

print(df)
```

	Industry	Size	Revenue	Industry_Code	Industry_Prefixed	Size_Priority
0	Tech	Small	100000	1	IND_Tech	3
1	Healthcare	Large	150000	2	IND_Healthcare	1
2	Finance	Medium	80000	3	IND_Finance	2
3	Tech	Small	120000	1	IND_Tech	3
4	Retail	Large	90000	4	IND_Retail	1

8.4.2.3 How map() Actually Works:

map() iterates through each element in the Series and applies the mapping function/dictionary lookup individually.

```
# Practical Performance Comparison: Real-World Examples

# Create a realistic dataset for performance testing
np.random.seed(42)
n_rows = 50000

performance_data = pd.DataFrame({
    'customer_id': range(1, n_rows + 1),
    'purchase_amount': np.random.uniform(10, 1000, n_rows),
    'discount_pct': np.random.choice([0, 5, 10, 15, 20], n_rows),
    'category': np.random.choice(['A', 'B', 'C', 'D'], n_rows),
    'rating': np.random.randint(1, 6, n_rows),
    'is_premium': np.random.choice([True, False], n_rows)
})

print(" Performance Test Dataset:")
print(f"Dataset shape: {performance_data.shape}")
print(performance_data.head())

# Define category mapping for testing
category_mapping = {'A': 'Electronics', 'B': 'Clothing', 'C': 'Books', 'D': 'Sports'}

print(f"\n Performance Test Results ({n_rows:,} rows):")
print("-" * 60)
```

```
Performance Test Dataset:
Dataset shape: (50000, 6)
   customer_id  purchase_amount  discount_pct  category  rating  is_premium
0            1      380.794718           0         C         1         True
1            2      951.207163          15         D         1        False
2            3      734.674002           5         D         1         True
3            4      602.671899           5         A         4         True
4            5      164.458454           0         D         3         True
```

```
Performance Test Results (50,000 rows):
```

```
-----
```

```

# Test 1: Simple Mathematical Operations
print(" Test 1: Calculate final price after discount")

# Method 1: VECTORIZED (Fastest)
start_time = time.time()
performance_data['final_price_vectorized'] = performance_data['purchase_amount'] * (1 - perf
vectorized_time = time.time() - start_time

# Method 2: APPLY with function (Slower)
def calculate_final_price(row):
    return row['purchase_amount'] * (1 - row['discount_pct'] / 100)

start_time = time.time()
performance_data['final_price_apply'] = performance_data.apply(calculate_final_price, axis=1
apply_time = time.time() - start_time

print(f" Vectorized:      {vectorized_time:.4f}s (1.0x baseline)")
print(f" Apply function: {apply_time:.4f}s ({apply_time/vectorized_time:.1f}x slower)")

# Verify results are identical
results_match = (performance_data['final_price_vectorized'].round(2) == performance_data['fi
print(f" Results identical: {results_match}")

# Clean up test columns
performance_data.drop(['final_price_vectorized', 'final_price_apply'], axis=1, inplace=True)

```

```

Test 1: Calculate final price after discount
Vectorized:      0.0013s (1.0x baseline)
Apply function: 0.2778s (207.6x slower)
Results identical: True

```

8.4.3 replace() : Replace specific values with other values

```

s_with_nan = pd.Series([1, 2, np.nan, 4])

# map() with na_action
mapped_nan = s_with_nan.map({1: 'one', 2: 'two', 4: 'four'}, na_action='ignore')
print("map() with na_action='ignore':")
print(mapped_nan)

```

```
# replace() can target NaN directly
replaced_nan = s_with_nan.replace(np.nan, 'missing')
print("\nreplace() handling NaN:")
print(replaced_nan)
```

```
map() with na_action='ignore':
```

```
0    one
1    two
2    NaN
3    four
dtype: object
```

```
replace() handling NaN:
```

```
0    1.0
1    2.0
2    missing
3    4.0
dtype: object
```

Note: Unlike `map()` and `apply()`, `replace()` is a **vectorized operation** in pandas and is therefore **much faster** than `map()` or `apply()`.

8.4.4 Performance Comparison

```
# Performance Comparison
print("\n Test 2: Category mapping")

# Method 1: MAP (Optimal for this task)
start_time = time.time()
performance_data['category_name_map'] = performance_data['category'].map(category_mapping)
map_time = time.time() - start_time

# Method 2: APPLY (Overkill for simple mapping)
start_time = time.time()
performance_data['category_name_apply'] = performance_data['category'].apply(lambda x: category_mapping.get(x, x))
apply_time = time.time() - start_time

# Method 3: VECTORIZED replacement (Alternative approach)
start_time = time.time()
performance_data['category_name_replace'] = performance_data['category'].replace(category_mapping)
```

```

replace_time = time.time() - start_time

print(f" Map():          {map_time:.4f}s (1.0x baseline)")
print(f" Apply():        {apply_time:.4f}s ({apply_time/map_time:.1f}x slower)")
print(f" Replace():        {replace_time:.4f}s ({replace_time/map_time:.1f}x slower)")

# Clean up test columns
performance_data.drop(['category_name_apply', 'category_name_replace'], axis=1, inplace=True)

```

Test 2: Category mapping

```

Map():          0.0025s (1.0x baseline)
Apply():        0.0101s (4.0x slower)
Replace():      0.0000s (0.0x slower)

```

Key Takeaway

- `apply()` and `map()` are **not vectorized** — under the hood they behave like Python loops. Pandas iterates through each row or column and calls your function, which can be slow on large datasets.
- For **value substitutions**, use `pandas.replace()` — it's **vectorized** and far faster than writing an `apply()` with a custom function.

8.5 Core Vectorized Operations in Pandas

Pandas is **built on top of NumPy**, which means that **vectorized operations** are actually **compiled C code** running under the hood. This is why vectorized operations can be **orders of magnitude faster** than Python loops!

8.5.1 Why Vectorization Matters

The Speed Difference:

- **Python loop:** Interpreted, slow, one operation at a time
- **Vectorized operation:** Compiled C code, operates on entire arrays at once

Performance gain: Typically **10-100x faster** than equivalent loops!

8.5.2 Categories of Vectorized Operations

Category	Operations	Example
Arithmetic	+, -, *, /, **, //, %	df['total'] = df['price'] * df['qty']
Comparison	>, <, >=, <=, ==, !=	df['expensive'] = df['price'] > 100
Aggregation	.sum(), .mean(), .max(), .std()	df['revenue'].sum()
Boolean Logic	&, \, , ~	df[(df['price'] > 50) & (df['qty'] > 10)]
String Ops	.str.upper(), .str.contains(), etc.	df['name'].str.upper()
DateTime Ops	.dt.year, .dt.month, .dt.dayofweek	df['date'].dt.year
Math Functions	np.sqrt(), np.log(), np.exp()	np.log(df['revenue'])

Let's see each category with practical examples!

8.5.3 Arithmetic Operations

The most common vectorized operations — perform math on entire columns at once.

```
print(" ARITHMETIC OPERATIONS")
print("=" * 60)

# Sample e-commerce data
orders_df = pd.DataFrame({
    'product': ['Widget', 'Gadget', 'Doohickey', 'Thingamajig', 'Whatsit'],
    'price': [29.99, 49.99, 19.99, 99.99, 39.99],
    'quantity': [2, 1, 5, 1, 3],
    'tax_rate': [0.08, 0.08, 0.08, 0.08, 0.08],
    'discount_pct': [10, 0, 15, 5, 20]
})

print("Original order data:")
print(orders_df)

# VECTORIZED: All calculations happen at once on entire columns
```

```
orders_df['subtotal'] = orders_df['price'] * orders_df['quantity']
orders_df['discount_amount'] = orders_df['subtotal'] * (orders_df['discount_pct'] / 100)
orders_df['after_discount'] = orders_df['subtotal'] - orders_df['discount_amount']
orders_df['tax'] = orders_df['after_discount'] * orders_df['tax_rate']
orders_df['total'] = orders_df['after_discount'] + orders_df['tax']

print("\n Calculated columns (all vectorized!):")
orders_df[['product', 'subtotal', 'discount_amount', 'total']].round(2)
```

ARITHMETIC OPERATIONS

=====

Original order data:

	product	price	quantity	tax_rate	discount_pct
0	Widget	29.99	2	0.08	10
1	Gadget	49.99	1	0.08	0
2	Doohickey	19.99	5	0.08	15
3	Thingamajig	99.99	1	0.08	5
4	Whatsit	39.99	3	0.08	20

Calculated columns (all vectorized!):

	product	subtotal	discount_amount	total
0	Widget	59.98	6.00	58.30
1	Gadget	49.99	0.00	53.99
2	Doohickey	99.95	14.99	91.75
3	Thingamajig	99.99	5.00	102.59
4	Whatsit	119.97	23.99	103.65

8.5.4 Comparison Operations

Create boolean masks for filtering, flagging, and categorizing data.

```
print("\n COMPARISON OPERATIONS")
print("=" * 60)

# Student grades data
grades_df = pd.DataFrame({
    'student': ['Alice', 'Bob', 'Carol', 'David', 'Eve', 'Frank'],
    'math': [92, 78, 88, 65, 95, 72],
```

```

    'science': [88, 82, 90, 70, 91, 68],
    'english': [85, 90, 87, 75, 89, 80]
})

print("Student grades:")
print(grades_df)

# VECTORIZED: Compare entire columns at once
grades_df['math_pass'] = grades_df['math'] >= 70
grades_df['high_achiever'] = grades_df['math'] >= 90
grades_df['needs_help'] = grades_df['math'] < 70

# Calculate average and compare
grades_df['average'] = (grades_df['math'] + grades_df['science'] + grades_df['english']) / 3
grades_df['above_average'] = grades_df['average'] > grades_df['average'].mean()

print("\n Comparison results:")
print(grades_df[['student', 'math', 'average', 'math_pass', 'high_achiever', 'above_average']])

# Count students in each category
print(f"\n Statistics:")
print(f" • Students passing math: {grades_df['math_pass'].sum()}")
print(f" • High achievers (math >= 90): {grades_df['high_achiever'].sum()}")
print(f" • Students above class average: {grades_df['above_average'].sum()}")

```

COMPARISON OPERATIONS

```

=====
Student grades:

```

	student	math	science	english
0	Alice	92	88	85
1	Bob	78	82	90
2	Carol	88	90	87
3	David	65	70	75
4	Eve	95	91	89
5	Frank	72	68	80

```


Comparison results:

```

	student	math	average	math_pass	high_achiever	above_average
0	Alice	92	88.333333	True	True	True
1	Bob	78	83.333333	True	False	True
2	Carol	88	88.333333	True	False	True

3	David	65	70.000000	False	False	False
4	Eve	95	91.666667	True	True	True
5	Frank	72	73.333333	True	False	False

Statistics:

- Students passing math: 5
- High achievers (math 90): 2
- Students above class average: 4

8.5.5 Boolean Indexing & Filtering

Combine comparison operations with boolean logic to filter data.

```
print("\n BOOLEAN INDEXING & FILTERING")
print("=" * 60)

# Product inventory data
inventory_df = pd.DataFrame({
    'product_id': range(1, 11),
    'name': ['Widget', 'Gadget', 'Doohickey', 'Thingamajig', 'Whatsit',
            'Gizmo', 'Contraption', 'Device', 'Apparatus', 'Mechanism'],
    'price': [29.99, 149.99, 19.99, 299.99, 39.99, 89.99, 199.99, 59.99, 179.99, 249.99],
    'stock': [45, 12, 8, 3, 67, 23, 15, 5, 28, 9],
    'category': ['A', 'B', 'A', 'C', 'A', 'B', 'C', 'A', 'B', 'C']
})

print("Product inventory:")
print(inventory_df)

# VECTORIZED FILTERING: Multiple conditions combined
# Find expensive, low-stock items
expensive = inventory_df['price'] > 100
low_stock = inventory_df['stock'] < 15
urgent_restock = expensive & low_stock

print("\n Urgent restock needed (expensive + low stock):")
print(inventory_df[urgent_restock][['name', 'price', 'stock']])

# Find good deals in category A
category_a = inventory_df['category'] == 'A'
affordable = inventory_df['price'] < 50
good_deals = category_a & affordable
```

```

print("\n Good deals in Category A (price < $50):")
print(inventory_df[good_deals][['name', 'price', 'category']])

# Complex filter: Either (expensive AND low stock) OR (cheap AND high stock)
cheap = inventory_df['price'] < 50
high_stock = inventory_df['stock'] > 30
complex_filter = (expensive & low_stock) | (cheap & high_stock)

print("\n Complex filter results:")
print(inventory_df[complex_filter][['name', 'price', 'stock']])

print(f"\n Filter statistics:")
print(f" • Total products: {len(inventory_df)}")
print(f" • Urgent restock: {urgent_restock.sum()}")
print(f" • Good deals: {good_deals.sum()}")
print(f" • Complex filter matches: {complex_filter.sum()}")

```

BOOLEAN INDEXING & FILTERING

=====

Product inventory:

	product_id	name	price	stock	category
0	1	Widget	29.99	45	A
1	2	Gadget	149.99	12	B
2	3	Doohickey	19.99	8	A
3	4	Thingamajig	299.99	3	C
4	5	Whatsit	39.99	67	A
5	6	Gizmo	89.99	23	B
6	7	Contraption	199.99	15	C
7	8	Device	59.99	5	A
8	9	Apparatus	179.99	28	B
9	10	Mechanism	249.99	9	C

Urgent restock needed (expensive + low stock):

	name	price	stock
1	Gadget	149.99	12
3	Thingamajig	299.99	3
9	Mechanism	249.99	9

Good deals in Category A (price < \$50):

name	price	category
------	-------	----------

0	Widget	29.99	A
2	Doohickey	19.99	A
4	Whatsit	39.99	A

Complex filter results:

	name	price	stock
0	Widget	29.99	45
1	Gadget	149.99	12
3	Thingamajig	299.99	3
4	Whatsit	39.99	67
9	Mechanism	249.99	9

Filter statistics:

- Total products: 10
- Urgent restock: 3
- Good deals: 3
- Complex filter matches: 5

8.5.6 Aggregation Operations

Reduce columns to summary statistics using vectorized aggregations.

```
print("\n AGGREGATION OPERATIONS")
print("=" * 60)

# Sales data by region
sales_df = pd.DataFrame({
    'region': ['North', 'South', 'East', 'West', 'North', 'South', 'East', 'West'] * 3,
    'month': [1, 1, 1, 1, 2, 2, 2, 2, 3, 3, 3, 3, 1, 1, 1, 1, 2, 2, 2, 2, 3, 3, 3, 3],
    'revenue': [45000, 52000, 48000, 51000, 47000, 54000, 49000, 53000,
                46000, 55000, 50000, 52000, 44000, 53000, 47000, 50000,
                48000, 56000, 51000, 54000, 45000, 57000, 49000, 55000]
})

print("Sales data (first 8 rows):")
print(sales_df.head(8))

# VECTORIZED AGGREGATIONS: Instant calculations on entire columns
print("\n Overall statistics (vectorized):")
print(f" • Total revenue: ${sales_df['revenue'].sum():,.0f}")
print(f" • Average revenue: ${sales_df['revenue'].mean():,.0f}")
print(f" • Median revenue: ${sales_df['revenue'].median():,.0f}")
```

```

print(f"    • Min revenue: ${sales_df['revenue'].min():,.0f}")
print(f"    • Max revenue: ${sales_df['revenue'].max():,.0f}")
print(f"    • Std deviation: ${sales_df['revenue'].std():,.0f}")

# Group by region and aggregate (still vectorized!)
print("\n Revenue by region:")
region_stats = sales_df.groupby('region')['revenue'].agg(['sum', 'mean', 'count'])
print(region_stats)

# Multiple aggregations at once
print("\n Comprehensive statistics:")
all_stats = sales_df.groupby('region')['revenue'].describe()
print(all_stats)

```

AGGREGATION OPERATIONS

=====

Sales data (first 8 rows):

	region	month	revenue
0	North	1	45000
1	South	1	52000
2	East	1	48000
3	West	1	51000
4	North	2	47000
5	South	2	54000
6	East	2	49000
7	West	2	53000

Overall statistics (vectorized):

- Total revenue: \$1,211,000
- Average revenue: \$50,458
- Median revenue: \$50,500
- Min revenue: \$44,000
- Max revenue: \$57,000
- Std deviation: \$3,730

Revenue by region:

	sum	mean	count
region			
East	294000	49000.000000	6
North	275000	45833.333333	6
South	327000	54500.000000	6

West 315000 52500.000000 6

Comprehensive statistics:

	count	mean	std	min	25%	50%	75%	\
region								
East	6.0	49000.000000	1414.213562	47000.0	48250.0	49000.0	49750.0	
North	6.0	45833.333333	1471.960144	44000.0	45000.0	45500.0	46750.0	
South	6.0	54500.000000	1870.828693	52000.0	53250.0	54500.0	55750.0	
West	6.0	52500.000000	1870.828693	50000.0	51250.0	52500.0	53750.0	

max

region	
East	51000.0
North	48000.0
South	57000.0
West	55000.0

8.5.7 String Operations (.str accessor)

Vectorized string manipulation — no loops needed!

```
print("\n STRING OPERATIONS (.str accessor)")
print("=" * 60)

# Customer data with messy strings
customers_df = pd.DataFrame({
    'customer_id': [1, 2, 3, 4, 5],
    'name': [' john DOE ', 'jane SMITH', 'Bob_Jones', 'alice WILLIAMS ', 'CHARLIE brown'],
    'email': ['john@EXAMPLE.com', 'jane@test.COM', 'bob@demo.ORG', 'alice@SITE.net', 'charlie@GMAIL.com'],
    'phone': ['123-456-7890', '(234) 567-8901', '345.678.9012', '456-567-8901', '567-678-9012']
})

print("Original customer data (messy):")
print(customers_df)

# VECTORIZED STRING OPERATIONS: Clean all at once
customers_df['name_clean'] = customers_df['name'].str.strip().str.title()
customers_df['email_clean'] = customers_df['email'].str.lower()
customers_df['first_name'] = customers_df['name_clean'].str.split().str[0]
customers_df['last_name'] = customers_df['name_clean'].str.split().str[-1]
customers_df['phone_clean'] = customers_df['phone'].str.replace(r'[\d]', '', regex=True)
```

```
# Check for patterns
customers_df['has_underscore'] = customers_df['name'].str.contains('_')
customers_df['email_domain'] = customers_df['email'].str.split('@').str[1]

print("\n Cleaned data:")
print(customers_df[['name_clean', 'email_clean', 'first_name', 'last_name', 'email_domain']])

print("\n String pattern checks:")
print(f"    • Customers with underscore in name: {customers_df['has_underscore'].sum()}")
print(f"    • Unique email domains: {customers_df['email_domain'].nunique()}")
```

8.5.8 DateTime Operations (.dt accessor)

Extract time components and perform date calculations — all vectorized!

```
print("\n DATETIME OPERATIONS (.dt accessor)")
print("=" * 60)

# Transaction data with dates
transactions_df = pd.DataFrame({
    'transaction_id': range(1, 7),
    'date': pd.to_datetime([
        '2024-01-15', '2024-02-20', '2024-03-10',
        '2024-06-25', '2024-09-05', '2024-12-18'
    ]),
    'amount': [250, 180, 420, 310, 275, 390]
})

print("Transaction data:")
print(transactions_df)

# VECTORIZED DATETIME OPERATIONS: Extract components
transactions_df['year'] = transactions_df['date'].dt.year
transactions_df['month'] = transactions_df['date'].dt.month
transactions_df['month_name'] = transactions_df['date'].dt.month_name()
transactions_df['quarter'] = transactions_df['date'].dt.quarter
transactions_df['day_of_week'] = transactions_df['date'].dt.day_name()
transactions_df['is_weekend'] = transactions_df['date'].dt.dayofweek >= 5

# Calculate days since first transaction
first_date = transactions_df['date'].min()
```

```

transactions_df['days_since_first'] = (transactions_df['date'] - first_date).dt.days

print("\n Extracted date components:")
print(transactions_df[['date', 'month_name', 'quarter', 'day_of_week', 'days_since_first']])

# Aggregate by quarter
print("\n Revenue by quarter:")
quarterly = transactions_df.groupby('quarter')['amount'].sum()
print(quarterly)

print(f"\n Date insights:")
print(f"    • Transactions on weekends: {transactions_df['is_weekend'].sum()}")
print(f"    • Date range: {transactions_df['date'].min()} to {transactions_df['date'].max()}")
print(f"    • Total days covered: {transactions_df['days_since_first'].max()}")

```

8.5.9 Mathematical Functions (NumPy Integration)

Pandas seamlessly integrates with NumPy functions for advanced math operations.

```

print("\n MATHEMATICAL FUNCTIONS (NumPy + Pandas)")
print("=" * 60)

# Scientific measurement data
measurements_df = pd.DataFrame({
    'sample_id': range(1, 6),
    'concentration': [2.5, 5.0, 10.0, 20.0, 40.0],
    'absorbance': [0.15, 0.32, 0.65, 1.28, 2.45],
    'temperature': [20, 25, 30, 35, 40]
})

print("Laboratory measurements:")
print(measurements_df)

# VECTORIZED MATH OPERATIONS: Apply functions to entire columns
measurements_df['log_concentration'] = np.log10(measurements_df['concentration'])
measurements_df['sqrt_absorbance'] = np.sqrt(measurements_df['absorbance'])
measurements_df['exp_temp'] = np.exp(measurements_df['temperature'] / 100)
measurements_df['temp_squared'] = np.power(measurements_df['temperature'], 2)

# Trigonometric functions (useful for periodic data)
measurements_df['sin_temp'] = np.sin(np.radians(measurements_df['temperature']))

```

```

# Statistical transformations
measurements_df['concentration_zscore'] = (
    (measurements_df['concentration'] - measurements_df['concentration'].mean()) /
    measurements_df['concentration'].std()
)

print("\n Calculated transformations:")
print(measurements_df[['sample_id', 'concentration', 'log_concentration',
                        'absorbance', 'sqrt_absorbance']].round(3))

print("\n Statistical summary:")
print(f" • Mean concentration: {measurements_df['concentration'].mean():.2f}")
print(f" • Geometric mean (via log): {np.exp(np.log(measurements_df['concentration']).mean())}")
print(f" • Correlation (conc. vs abs.): {measurements_df['concentration'].corr(measurements_

```

8.5.10 Performance Comparison: Loops vs Vectorization

Let's see the **dramatic speed difference** between loops and vectorized operations with real timing data.

```

import time

print("\n PERFORMANCE SHOWDOWN: Loops vs Vectorization")
print("=" * 60)

# Create a large dataset for meaningful timing
large_df = pd.DataFrame({
    'price': np.random.uniform(10, 500, 50000),
    'quantity': np.random.randint(1, 20, 50000),
    'tax_rate': 0.08,
    'discount': np.random.uniform(0, 0.3, 50000)
})

print(f"Dataset size: {len(large_df):,} rows")

# METHOD 1: Python loop with iterrows() - SLOW!
print("\n Method 1: Python loop with iterrows()")
start_time = time.time()
total_loop = []
for idx, row in large_df.iterrows():
    subtotal = row['price'] * row['quantity']

```



```

        discount_amt = subtotal * row['discount']
        after_discount = subtotal - discount_amt
        tax = after_discount * row['tax_rate']
        total = after_discount + tax
        total_loop.append(total)
    loop_time = time.time() - start_time
    print(f"    Time: {loop_time:.4f} seconds")

# METHOD 2: List comprehension - Still slow!
print("\n Method 2: List comprehension")
start_time = time.time()
total_list_comp = [
    (row['price'] * row['quantity'] * (1 - row['discount'])) * (1 + row['tax_rate']))
    for idx, row in large_df.iterrows()
]
list_comp_time = time.time() - start_time
print(f"    Time: {list_comp_time:.4f} seconds")

# METHOD 3: apply() - Better but still not optimal
print("\n Method 3: apply() function")
start_time = time.time()
def calculate_total(row):
    subtotal = row['price'] * row['quantity']
    after_discount = subtotal * (1 - row['discount'])
    return after_discount * (1 + row['tax_rate'])
large_df['total_apply'] = large_df.apply(calculate_total, axis=1)
apply_time = time.time() - start_time
print(f"    Time: {apply_time:.4f} seconds")

# METHOD 4: Vectorized operations - FAST!
print("\n Method 4: Vectorized operations")
start_time = time.time()
large_df['total_vectorized'] = (
    large_df['price'] *
    large_df['quantity'] *
    (1 - large_df['discount']) *
    (1 + large_df['tax_rate'])
)
vectorized_time = time.time() - start_time
print(f"    Time: {vectorized_time:.4f} seconds")

# Performance comparison

```

```

print("\n" + "=" * 60)
print(" PERFORMANCE RESULTS")
print("=" * 60)
print(f"{'Method':<30} {'Time (s)':<12} {'Speedup':<12} {'Status'}")
print("-" * 60)

# Prevent ZeroDivisionError by ensuring minimum time value
min_time = 0.0001 # Minimum time threshold to avoid division by zero
safe_vectorized_time = max(vectorized_time, min_time)

# Calculate speedup safely
loop_speedup = loop_time / safe_vectorized_time if safe_vectorized_time > 0 else float('inf')
list_comp_speedup = list_comp_time / safe_vectorized_time if safe_vectorized_time > 0 else float('inf')
apply_speedup = apply_time / safe_vectorized_time if safe_vectorized_time > 0 else float('inf')

print(f"{' iterrows() loop':<30} {loop_time:>10.4f} {loop_speedup:>8.1f}x AVOID")
print(f"{' List comprehension':<30} {list_comp_time:>10.4f} {list_comp_speedup:>8.1f}x")
print(f"{' apply() function':<30} {apply_time:>10.4f} {apply_speedup:>8.1f}x USE SPARK")
print(f"{' Vectorized operations':<30} {vectorized_time:>10.4f} {'1.0x':>8} BEST")

print("\n Key Insight:")
if vectorized_time > 0:
    speedup_factor = loop_time / safe_vectorized_time
    time_reduction = ((loop_time - vectorized_time) / loop_time * 100) if loop_time > 0 else 0
    print(f" Vectorization is {speedup_factor:.0f}x faster than loops!")
    print(f" That's a {time_reduction:.1f}% reduction in execution time.")
else:
    print(f" Vectorization is extremely fast (< 0.0001 seconds)!")
    print(f" Loop time: {loop_time:.4f}s vs Vectorized: {vectorized_time:.6f}s")

# Verify all methods produce same results
print("\n Verification: All methods produce identical results:",
      np.allclose(total_loop, large_df['total_vectorized']))

```

PERFORMANCE SHOWDOWN: Loops vs Vectorization

=====

Dataset size: 50,000 rows

Method 1: Python loop with iterrows()

Time: 0.8976 seconds

Method 2: List comprehension
Time: 0.8976 seconds

Method 2: List comprehension
Time: 0.8167 seconds

Method 3: apply() function
Time: 0.8167 seconds

Method 3: apply() function
Time: 0.4004 seconds

Method 4: Vectorized operations
Time: 0.0000 seconds

=====

PERFORMANCE RESULTS

=====

Method	Time (s)	Speedup	Status
iterrows() loop	0.8976	8975.7x	AVOID
List comprehension	0.8167	8166.9x	AVOID
apply() function	0.4004	4003.8x	USE SPARINGLY
Vectorized operations	0.0000	1.0x	BEST

Key Insight:
Vectorization is extremely fast (< 0.0001 seconds)!
Loop time: 0.8976s vs Vectorized: 0.000000s

Verification: All methods produce identical results: True
Time: 0.4004 seconds

Method 4: Vectorized operations
Time: 0.0000 seconds

=====

PERFORMANCE RESULTS

=====

Method	Time (s)	Speedup	Status
iterrows() loop	0.8976	8975.7x	AVOID
List comprehension	0.8167	8166.9x	AVOID
apply() function	0.4004	4003.8x	USE SPARINGLY

Vectorized operations	0.0000	1.0x	BEST
-----------------------	--------	------	------

Key Insight:

Vectorization is extremely fast (< 0.0001 seconds)!

Loop time: 0.8976s vs Vectorized: 0.000000s

Verification: All methods produce identical results: True

8.5.11 Summary: Vectorization Best Practices

8.5.11.1 Always Try Vectorization First

Use vectorized operations for:

- Arithmetic: `df['total'] = df['price'] * df['qty']`
- Comparisons: `df['high_value'] = df['amount'] > 1000`
- String operations: `df['upper_name'] = df['name'].str.upper()`
- Date operations: `df['year'] = df['date'].dt.year`
- Math functions: `df['log_revenue'] = np.log(df['revenue'])`
- Aggregations: `df['revenue'].sum(), df.groupby('category').mean()`

8.5.11.2 Performance Hierarchy

Speed Ranking (fastest to slowest):

- | | |
|---|-------------------------|
| 1. Vectorized operations (NumPy/Pandas built-ins) | ← Always try first! |
| 2. <code>.map()</code> for simple 1:1 mappings | ← Good for dictionaries |
| 3. <code>.apply()</code> for complex row-wise logic | ← Use when necessary |
| 4. Python loops (iterrows, comprehensions) | ← AVOID AT ALL COSTS |

8.5.11.3 Common Mistakes to Avoid

```
# DON'T: Use loops for arithmetic
for i in range(len(df)):
    df.loc[i, 'total'] = df.loc[i, 'price'] * df.loc[i, 'qty']

# DO: Use vectorized operations
df['total'] = df['price'] * df['qty']

# DON'T: Use apply() for simple operations
df['price_doubled'] = df['price'].apply(lambda x: x * 2)
```

```
# DO: Use direct vectorized operation
df['price_doubled'] = df['price'] * 2

# DON'T: Use iterrows() ever!
for idx, row in df.iterrows(): # 100x slower!
    # ... any operation

# DO: Find a vectorized alternative
df['result'] = some_vectorized_operation
```

8.5.11.4 Quick Decision Guide

“How should I perform this operation?”

1. ****Can I use a basic operator? (+, -, *, /, >, <, etc.)****
→ Use it directly: `df['col1'] + df['col2']`
2. **Is it a string or date operation?**
→ Use `.str` or `.dt` accessor: `df['text'].str.upper()`
3. **Is it a NumPy function?**
→ Apply to column: `np.sqrt(df['values'])`
4. **Is it a simple dictionary mapping?**
→ Use `.map()`: `df['code'].map(mapping_dict)`
5. **Is it a built-in aggregation?**
→ Use aggregation method: `df['sales'].sum()`
6. **Does it require complex multi-column logic?**
→ Use `.apply()` (only when necessary)
7. **None of the above work?**
→ **Re-think your approach!** There’s almost always a vectorized way.

8.5.11.5 Pro Tips

1. **Profile your code:** Use `%%timeit` in Jupyter to measure performance
2. **Start vectorized:** Don’t write loops first “because it’s easier”
3. **Think in columns:** Pandas works best with column-wise operations
4. **Combine operations:** Chain multiple vectorized operations together
5. **Use NumPy:** Many NumPy functions work directly on pandas Series
6. **Avoid loops:** Seriously, avoid them. There’s almost always a better way.

8.6 Summary: Performance-First Mindset

8.6.1 Key Takeaways

1. **Vectorization is King:** Always your first choice - leverages NumPy's C-speed processing
2. **map() for Mappings:** Perfect for dictionaries and simple element-wise transformations
3. **apply() for Complex Logic:** Use when business rules require multiple columns or complex conditions
4. **Avoid Python Loops:** iterrows(), list comprehensions are performance killers

8.6.2 Looking Ahead: Why NumPy Matters

The next chapter will dive deep into **NumPy**, the foundation that makes pandas so powerful. You'll learn:

- **Array Architecture:** How NumPy's C-based arrays enable lightning-fast operations
- **Broadcasting Rules:** Advanced techniques for working with different array shapes
- **Universal Functions (ufuncs):** The secret weapons behind vectorized operations
- **Custom Vectorization:** How to write your own fast, NumPy-powered functions

8.6.3 The Performance Pyramid

```
NumPy Vectorized (Next Chapter!)
  ↑ 10-100x faster
Pandas Vectorized
  ↑ 2-5x faster
map() / apply()
  ↑ 50-200x faster
Python Loops (iterrows, comprehensions)
```

Understanding NumPy will unlock the full potential of pandas and give you the tools to write **production-ready, high-performance** data analysis code!

8.7 Lambda Functions: Inline Power for Data Transformation

In the previous sections, we saw how to define a function and then use `map()` or `apply()` to apply it.

With **lambda functions**, you can skip the definition step and write the function **inline**.

Lambda functions are **anonymous, one-line functions** that provide a concise way to express simple operations without creating a separate `def` block.

They're the **Swiss Army knife** of pandas transformations — compact, versatile, and perfect for quick, one-off operations.

8.7.1 Syntax & Structure

```
lambda arguments: expression
```

8.7.2 Key Characteristics

Feature	Description	Example
Anonymous	No function name required	<code>lambda x: x * 2</code>
Single Expression	Only one expression allowed	<code>lambda x, y: x + y</code>
Inline Definition	Defined at point of use	<code>df.apply(lambda row: row.sum(), axis=1)</code>
Return Value	Automatically returns expression result	<code>lambda x: x ** 2</code> returns squared value
Multiple Arguments	Can accept multiple parameters	<code>lambda x, y, z: x * y + z</code>

8.7.3 Real-World Lambda Applications

In this section, we'll explore **practical use cases of lambda functions** in pandas and demonstrate common transformation patterns.

```
# Create a comprehensive e-commerce dataset for lambda demonstrations

from datetime import datetime, timedelta

np.random.seed(42)
```

```

# Generate realistic e-commerce transaction data
n_transactions = 1000

ecommerce_data = {
    'transaction_id': [f'TXN_{i:05d}' for i in range(1, n_transactions + 1)],
    'customer_id': np.random.randint(1, 200, n_transactions),
    'product_name': np.random.choice([
        'Laptop', 'Smartphone', 'Tablet', 'Headphones', 'Camera',
        'Watch', 'Speaker', 'Monitor', 'Keyboard', 'Mouse'
    ], n_transactions),
    'price': np.random.uniform(50, 2000, n_transactions).round(2),
    'quantity': np.random.randint(1, 5, n_transactions),
    'discount_pct': np.random.choice([0, 5, 10, 15, 20, 25], n_transactions),
    'shipping_cost': np.random.uniform(5, 50, n_transactions).round(2),
    'customer_rating': np.random.choice([1, 2, 3, 4, 5], n_transactions),
    'purchase_date': pd.date_range('2024-01-01', periods=n_transactions, freq='h'),
    'customer_type': np.random.choice(['Premium', 'Regular', 'New'], n_transactions),
    'payment_method': np.random.choice(['Credit Card', 'PayPal', 'Bank Transfer'], n_transactions)
}

df_ecommerce = pd.DataFrame(ecommerce_data)

print(" E-commerce Dataset Sample:")
print(df_ecommerce.head())
print(f"\nDataset shape: {df_ecommerce.shape}")

```

E-commerce Dataset Sample:

	transaction_id	customer_id	product_name	price	quantity	discount_pct	\
0	TXN_00001	103	Camera	1892.00	1	15	
1	TXN_00002	180	Watch	1382.33	1	0	
2	TXN_00003	93	Headphones	1019.49	2	0	
3	TXN_00004	15	Mouse	1254.80	3	5	
4	TXN_00005	107	Smartphone	1744.36	4	10	

	shipping_cost	customer_rating	purchase_date	customer_type	\
0	40.67	3	2024-01-01 00:00:00	New	
1	33.23	2	2024-01-01 01:00:00	New	
2	38.76	3	2024-01-01 02:00:00	New	
3	11.84	2	2024-01-01 03:00:00	New	
4	25.62	4	2024-01-01 04:00:00	New	

payment_method


```

0 Bank Transfer
1      PayPal
2      PayPal
3 Bank Transfer
4   Credit Card

```

Dataset shape: (1000, 11)

8.7.3.1 Simple Mathematical Operations

```

# Example 1: Simple Mathematical Operations with Lambda
print("=== SIMPLE MATHEMATICAL OPERATIONS ===")

# Calculate discounted price
df_ecommerce['discounted_price'] = df_ecommerce.apply(
    lambda row: row['price'] * (1 - row['discount_pct'] / 100), axis=1
)

# Calculate total cost (price + shipping)
df_ecommerce['total_cost'] = df_ecommerce.apply(
    lambda row: row['discounted_price'] * row['quantity'] + row['shipping_cost'], axis=1
)

# Calculate savings amount
df_ecommerce['savings'] = df_ecommerce.apply(
    lambda row: (row['price'] - row['discounted_price']) * row['quantity'], axis=1
)

print(" Price Calculations (First 10 transactions):")
price_cols = ['transaction_id', 'price', 'discount_pct', 'discounted_price', 'quantity', 'total_cost', 'savings']
print(df_ecommerce[price_cols].head(10).to_string(index=False))

```

=== SIMPLE MATHEMATICAL OPERATIONS ===

Price Calculations (First 10 transactions):

transaction_id	price	discount_pct	discounted_price	quantity	total_cost	savings
TXN_00001	1892.00	15	1608.2000	1	1648.8700	283.8000
TXN_00002	1382.33	0	1382.3300	1	1415.5600	0.0000
TXN_00003	1019.49	0	1019.4900	2	2077.7400	0.0000
TXN_00004	1254.80	5	1192.0600	3	3588.0200	188.2200
TXN_00005	1744.36	10	1569.9240	4	6305.3160	697.7440

TXN_00006	1162.69	25	872.0175	1	892.8175	290.6725
TXN_00007	109.25	0	109.2500	4	446.2200	0.0000
TXN_00008	1865.35	5	1772.0825	3	5343.1275	279.8025
TXN_00009	1394.58	20	1115.6640	3	3393.4220	836.7480
TXN_00010	1369.20	25	1026.9000	4	4114.4000	1369.2000

8.7.3.2 Conditional Logic

```
df['category'] = df['score'].apply(lambda x: 'High' if x > 80 else 'Low')
```

```
# Example 2: Conditional Logic with Lambda
print("\n=== CONDITIONAL LOGIC OPERATIONS ===")

# Categorize customers based on spending
df_ecommerce['customer_segment'] = df_ecommerce['total_cost'].apply(
    lambda x: 'High Value' if x > 500 else 'Medium Value' if x > 200 else 'Low Value'
)

# Classify ratings
df_ecommerce['rating_category'] = df_ecommerce['customer_rating'].apply(
    lambda rating: 'Excellent' if rating == 5
                  else 'Good' if rating >= 4
                  else 'Average' if rating == 3
                  else 'Poor'
)

# Determine shipping urgency
df_ecommerce['shipping_urgency'] = df_ecommerce.apply(
    lambda row: 'Express' if row['customer_type'] == 'Premium' and row['total_cost'] > 300
               else 'Standard' if row['customer_type'] == 'Premium'
               else 'Economy', axis=1
)

print(" Customer Segmentation (Sample):")
segment_cols = ['customer_id', 'total_cost', 'customer_segment', 'customer_rating', 'rating_category']
print(df_ecommerce[segment_cols].head(10).to_string(index=False))
```

```
=== CONDITIONAL LOGIC OPERATIONS ===
Customer Segmentation (Sample):
```

customer_id	total_cost	customer_segment	customer_rating	rating_category	shipping_urgency
103	1648.8700	High Value	3	Average	Economy
180	1415.5600	High Value	2	Poor	Economy
93	2077.7400	High Value	3	Average	Economy
15	3588.0200	High Value	2	Poor	Economy
107	6305.3160	High Value	4	Good	Economy
72	892.8175	High Value	3	Average	Economy
189	446.2200	Medium Value	3	Average	Economy
21	5343.1275	High Value	5	Excellent	Economy
103	3393.4220	High Value	3	Average	Economy
122	4114.4000	High Value	2	Poor	Economy

8.7.3.3 String Operations

```
df['clean_name'] = df['name'].apply(lambda x: x.strip().title())
```

```
# Example 3: String Operations with Lambda
print("\n=== STRING OPERATIONS ===")
```

```
# Clean and standardize product names
```

```
df_ecommerce['product_name_clean'] = df_ecommerce['product_name'].apply(
    lambda name: name.strip().upper().replace(' ', '_')
)
```

```
# Create transaction codes
```

```
df_ecommerce['transaction_code'] = df_ecommerce.apply(
    lambda row: f"{row['customer_type'][:3].upper()}--{row['product_name_clean'][:4]}--{row['t"
)
```

```
# Format purchase date for display
```

```
df_ecommerce['purchase_display'] = df_ecommerce['purchase_date'].apply(
    lambda date: date.strftime('%Y-%m-%d %H:%M')
)
```

```
print(" String Processing Results (Sample):")
```

```
string_cols = ['product_name', 'product_name_clean', 'transaction_code', 'purchase_display']
print(df_ecommerce[string_cols].head(8).to_string(index=False))
```

```
=== STRING OPERATIONS ===
```

String Processing Results (Sample):

product_name	product_name_clean	transaction_code	purchase_display
Camera	CAMERA	NEW-CAME-001	2024-01-01 00:00
Watch	WATCH	NEW-WATC-002	2024-01-01 01:00
Headphones	HEADPHONES	NEW-HEAD-003	2024-01-01 02:00
Mouse	MOUSE	NEW-MOUS-004	2024-01-01 03:00
Smartphone	SMARTPHONE	NEW-SMAR-005	2024-01-01 04:00
Monitor	MONITOR	NEW-MONI-006	2024-01-01 05:00
Watch	WATCH	REG-WATC-007	2024-01-01 06:00
Camera	CAMERA	REG-CAME-008	2024-01-01 07:00

8.7.3.4 Multiple Column Operations

```
df['total'] = df.apply(lambda row: row['price'] * row['quantity'], axis=1)
```

```
# Example 4: Performance Optimization & Advanced Lambda Patterns
```

```
print("\n=== ADVANCED LAMBDA PATTERNS ===")
```

```
# Complex feature engineering
```

```
df_ecommerce['value_score'] = df_ecommerce.apply(
```

```
    lambda row: (
```

```
        (row['total_cost'] / 100) * 0.4 + # 40% weight on spending
```

```
        (row['customer_rating'] / 5) * 0.3 + # 30% weight on satisfaction
```

```
        (1 if row['customer_type'] == 'Premium' else 0.5 if row['customer_type'] == 'Regular
```

```
        (min(row['quantity'], 3) / 3) * 0.1 # 10% weight on quantity (capped at 3)
```

```
    ), axis=1
```

```
)
```

```
# Risk assessment using multiple factors
```

```
df_ecommerce['risk_flag'] = df_ecommerce.apply(
```

```
    lambda row: 'High Risk' if (
```

```
        row['total_cost'] > 1000 and
```

```
        row['payment_method'] == 'Bank Transfer' and
```

```
        row['customer_rating'] <= 2
```

```
) else 'Medium Risk' if (
```

```
        row['total_cost'] > 500 or
```

```
        row['customer_rating'] <= 2
```

```
) else 'Low Risk', axis=1
```

```
)
```

```
# Calculate percentile ranking
```

```

total_cost_90th = df_ecommerce['total_cost'].quantile(0.9)
df_ecommerce['is_top_spender'] = df_ecommerce['total_cost'].apply(
    lambda cost: cost >= total_cost_90th
)

print(" Advanced Analytics Results:")
advanced_cols = ['customer_id', 'total_cost', 'value_score', 'risk_flag', 'is_top_spender']
print(df_ecommerce[advanced_cols].head(10).round(2).to_string(index=False))

print(f"\n Summary Statistics:")
print(f"• Average value score: {df_ecommerce['value_score'].mean():.2f}")
print(f"• High risk transactions: {(df_ecommerce['risk_flag'] == 'High Risk').sum()}")
print(f"• Top spenders (90th percentile): {df_ecommerce['is_top_spender'].sum()}")
print(f"• Top spender threshold: ${total_cost_90th:.2f}")

```

=== ADVANCED LAMBDA PATTERNS ===

Advanced Analytics Results:

customer_id	total_cost	value_score	risk_flag	is_top_spender
103	1648.87	6.81	Medium Risk	False
180	1415.56	5.82	Medium Risk	False
93	2077.74	8.56	Medium Risk	False
15	3588.02	14.57	High Risk	False
107	6305.32	25.56	Medium Risk	True
72	892.82	3.78	Medium Risk	False
189	446.22	2.16	Low Risk	False
21	5343.13	21.87	Medium Risk	True
103	3393.42	13.95	Medium Risk	False
122	4114.40	16.78	Medium Risk	False

Summary Statistics:

- Average value score: 9.26
- High risk transactions: 117
- Top spenders (90th percentile): 100
- Top spender threshold: \$4875.15

8.7.4 Performance Benchmarking: Lambda vs Named Functions vs Vectorized

```

# Performance Benchmarking: Lambda vs Named Functions vs Vectorized
import pandas as pd

```

```

import numpy as np
import time

# Create LARGER test data to get measurable times
test_data = pd.Series(np.random.randint(1, 100, 1000000)) # 1M elements instead of 100K

print(" Performance Comparison (1,000,000 operations):")
print("-" * 50)

# Method 1: Lambda function
start_time = time.time()
result_lambda = test_data.apply(lambda x: x ** 2 + 10)
lambda_time = time.time() - start_time

# Method 2: Named function
def square_plus_ten(x):
    return x ** 2 + 10

start_time = time.time()
result_named = test_data.apply(square_plus_ten)
named_time = time.time() - start_time

# Method 3: Vectorized operations (fastest)
start_time = time.time()
result_vectorized = test_data ** 2 + 10
vectorized_time = time.time() - start_time

# Method 4: List comprehension
start_time = time.time()
result_listcomp = pd.Series([x ** 2 + 10 for x in test_data])
listcomp_time = time.time() - start_time

# Safe comparison - avoid division by zero
print(f"Vectorized operations: {vectorized_time:.6f}s (baseline)")
print(f"Lambda function:      {lambda_time:.6f}s ({lambda_time/vectorized_time:.1f}x slower)")
print(f"Named function:        {named_time:.6f}s ({named_time/vectorized_time:.1f}x slower)")
print(f"List comprehension:    {listcomp_time:.6f}s ({listcomp_time/vectorized_time:.1f}x slower)")

# Verify results are identical
print(f"\n Results identical: {result_lambda.equals(result_named) and result_named.equals(result_vectorized)}")

print("\n Performance Insights:")

```

```
print("• Vectorized operations are fastest for simple math")
print("• Named functions have slight overhead advantage over lambdas")
print("• Lambda functions are good compromise between speed and readability")
print("• List comprehensions are generally slower than pandas methods")
```

Performance Comparison (1,000,000 operations):

```
-----
Vectorized operations: 0.002011s (baseline)
Lambda function:      0.263592s (131.1x slower)
Named function:      0.280185s (139.3x slower)
List comprehension:  0.251086s (124.9x slower)
```

Results identical: False

Performance Insights:

- Vectorized operations are fastest for simple math
- Named functions have slight overhead advantage over lambdas
- Lambda functions are good compromise between speed and readability
- List comprehensions are generally slower than pandas methods

8.7.4.1 Lambda Best Practices & Advanced Techniques

Scenario	Good Lambda Usage	Poor Lambda Usage
Simple Operations	<code>df['x2'] = df['x'].apply(lambda x: x * 2)</code>	<code>df['x2'] = df['x'].apply(lambda x: complex_algorithm(x))</code>
Conditional Logic	<code>df['cat'] = df['score'].apply(lambda x: 'High' if x > 80 else 'Low')</code>	<code>lambda x: 'A' if x > 90 else 'B' if x > 80 else 'C' if x > 70 else 'D' if x > 60 else 'F'</code>
Multiple Columns	<code>df['total'] = df.apply(lambda row: row['price'] * row['qty'], axis=1)</code>	Complex calculations with many variables
String Processing	<code>df['clean'] = df['name'].apply(lambda x: x.strip().title())</code>	Complex regex operations

8.7.4.2 Lambda Decision Framework

```
# Use Lambda when:
simple_operation = lambda x: x * 2          # Simple math
conditional = lambda x: 'A' if x > 5 else 'B' # Basic conditions
string_ops = lambda s: s.upper()          # Simple string ops

# Avoid Lambda when:
# Complex logic (use named functions)
# Multiple steps (break into separate operations)
# Reusable logic (define proper functions)
# Available vectorized operations
```

8.8 Advanced NLP Preprocessing

For text data stored in **object (string) columns**, pandas works seamlessly with Python's built-in string functions.

Beyond that, you can also use the **re (regular expressions)** module and the **NLTK (Natural Language Toolkit)** library for more advanced text preprocessing.

These tools are not commonly needed for this sequence course, but they are powerful for tasks such as text cleaning, tokenization, and feature extraction — especially if you choose an NLP-focused topic for your **final project**.

Below, we'll demonstrate how to use the **re** module and NLTK with pandas columns.

8.8.1 the re module in Python is used for regular expression

The **re** module in Python is used for **regular expressions**, which are powerful tools for text analysis and manipulation. Regular expressions allow you to search, match, and manipulate strings based on specific patterns.

Common Use Cases of **re** in String Text Analysis

- Finding patterns in text: `re.search(r'\d{4}-\d{2}-\d{2}', text)` # searches for a date in the format YYYY-MM-DD
- Extracting Specific parts of a string: `re.findall(r'\b[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Z|a-z]', text)` #extracts email addresses from the text.
- Replacing Parts of a String: `re.sub(r'\$\d+', '[price]', text)` #replacing any price formatted as \$ with [price].

8.8.1.0.1 Commonly Used Patterns in re

- `\d`: Matches any digit (0-9).
- `\w`: Matches any alphanumeric character (letters and numbers).
- `\s`: Matches any whitespace character (spaces, tabs, newlines).
- `[a-z]`: Matches any lowercase letter from a to z.
- `[A-Z]`: Matches any uppercase letter from A to Z.
- `*`: Matches 0 or more occurrences of the preceding character.
- `+`: Matches 1 or more occurrences of the preceding character.
- `?`: Matches 0 or 1 occurrence of the preceding character.
- `^`: Matches the beginning of a string.
- `$`: Matches the end of a string.
- `|`: Acts as an OR operator.

```
import requests

def read_html_tables_from_url(url, match=None, user_agent=None, timeout=10, **kwargs):
    """
    Read HTML tables from a website URL with error handling and customizable options.

    Parameters:
    -----
    url : str
        The website URL to scrape tables from
    match : str, optional
        String or regex pattern to match tables (passed to pd.read_html)
    user_agent : str, optional
        Custom User-Agent header. If None, uses a default Chrome User-Agent
    timeout : int, default 10
        Request timeout in seconds
    **kwargs : dict
        Additional arguments passed to pd.read_html()

    Returns:
    -----
    list of DataFrames or None
        List of pandas DataFrames (one per table found), or None if error occurs

    Examples:
    -----
    # Basic usage
    tables = read_html_tables_from_url("https://en.wikipedia.org/wiki/List_of_countries_by_GL")
```

```

# Filter tables containing specific text
tables = read_html_tables_from_url(url, match="Country")

# With custom user agent
tables = read_html_tables_from_url(url, user_agent="MyBot/1.0")
"""

import pandas as pd
import requests

# Default User-Agent if none provided
if user_agent is None:
    user_agent = 'Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, 1.

headers = {'User-Agent': user_agent}

try:
    print(f" Fetching data from: {url}")

    # Make the HTTP request
    response = requests.get(url, headers=headers, timeout=timeout)
    response.raise_for_status() # Raise an exception for bad status codes (4xx or 5xx)

    print(f" Successfully retrieved webpage (Status: {response.status_code})")

    # Parse HTML tables
    read_html_kwargs = {'match': match} if match else {}
    read_html_kwargs.update(kwargs) # Add any additional arguments

    tables = pd.read_html(response.content, **read_html_kwargs)

    print(f" Found {len(tables)} table(s) on the page")

    # Display table information
    if tables:
        print(f"\n Table Overview:")
        for i, table in enumerate(tables):
            print(f" Table {i}: {table.shape} (rows × columns)")
            if i >= 4: # Limit output for readability
                print(f" ... and {len(tables) - 5} more tables")
                break

    return tables

```

```

except requests.exceptions.HTTPError as err:
    print(f" HTTP Error: {err}")
    return None
except requests.exceptions.ConnectionError as err:
    print(f" Connection Error: {err}")
    return None
except requests.exceptions.Timeout as err:
    print(f" Timeout Error: Request took longer than {timeout} seconds")
    return None
except requests.exceptions.RequestException as err:
    print(f" Request Error: {err}")
    return None
except ValueError as err:
    print(f" No tables found on the page or parsing error: {err}")
    return None
except Exception as e:
    print(f" An unexpected error occurred: {e}")
    return None

```

```

# Example 1: Basic usage - Get all tables from GDP Wikipedia page
url = 'https://en.wikipedia.org/wiki/List_of_Chicago_Bulls_seasons'
tables = read_html_tables_from_url(url)
ChicagoBulls = tables[2] if tables else None
ChicagoBulls.head()

```

Fetching data from: https://en.wikipedia.org/wiki/List_of_Chicago_Bulls_seasons
 Successfully retrieved webpage (Status: 200)
 Found 18 table(s) on the page

Table Overview:
 Table 0: (13, 2) (rows × columns)
 Table 1: (1, 5) (rows × columns)
 Table 2: (59, 13) (rows × columns)
 Table 3: (3, 4) (rows × columns)
 Table 4: (9, 2) (rows × columns)
 ... and 13 more tables
 Successfully retrieved webpage (Status: 200)
 Found 18 table(s) on the page

Table Overview:
 Table 0: (13, 2) (rows × columns)

Table 1: (1, 5) (rows × columns)
 Table 2: (59, 13) (rows × columns)
 Table 3: (3, 4) (rows × columns)
 Table 4: (9, 2) (rows × columns)
 ... and 13 more tables

	Season	Team	Conference	Finish	Division	Finish.1	Wins	Losses	Win%	GB	Playoffs
0	1966–67	1966–67	—	—	Western	4th	33	48	0.407	11	Lost Division
1	1967–68	1967–68	—	—	Western	4th	29	53	0.354	27	Lost Division
2	1968–69	1968–69	—	—	Western	5th	33	49	0.402	22	NaN
3	1969–70	1969–70	—	—	Western	3rd[c]	39	43	0.476	9	Lost Division
4	1970–71	1970–71	Western	3rd	Midwest[d]	2nd	51	31	0.622	2	Lost conference

```
# remove all characters between box brackets including the brackets themselves in the columns
import re
def remove_brackets(x):
    return re.sub(r'\[.*?\]', '', x)

# Apply the function to each column separately using map
ChicagoBulls['Division'] = ChicagoBulls['Division'].map(remove_brackets)
ChicagoBulls['Finish'] = ChicagoBulls['Finish'].map(remove_brackets)
ChicagoBulls['Finish.1'] = ChicagoBulls['Finish.1'].map(remove_brackets)

ChicagoBulls.head()
```

	Season	Team	Conference	Finish	Division	Finish.1	Wins	Losses	Win%	GB	Playoffs
0	1966–67	1966–67	—	—	Western	4th	33	48	0.407	11	Lost Division
1	1967–68	1967–68	—	—	Western	4th	29	53	0.354	27	Lost Division
2	1968–69	1968–69	—	—	Western	5th	33	49	0.402	22	NaN
3	1969–70	1969–70	—	—	Western	3rd	39	43	0.476	9	Lost Division
4	1970–71	1970–71	Western	3rd	Midwest	2nd	51	31	0.622	2	Lost conference

8.8.2 The NLTK Library for Advanced NLP

8.8.2.1 What is NLTK?

NLTK (Natural Language Toolkit) is a comprehensive Python library for **natural language processing (NLP)** and **text analysis**. It offers tools for tokenization, preprocessing, part-of-speech tagging, parsing, named-entity recognition, sentiment analysis, and more—useful from classroom demos to research prototypes.

8.8.2.2 Why NLTK Matters in Data Science

Application Domain	NLTK Use Cases	Business Impact
Social Media Analytics	Sentiment, subjectivity, emotion, polarity detection	Customer insights, brand monitoring
Customer Support	Text classification, intent detection	Automation, faster resolution
Content Analysis	Keyword extraction, topic modeling, summarization	SEO optimization, content strategy
Research & Academia	Corpus analysis, linguistic parsing, concordance tools	Rigorous analysis, reproducible studies
E-commerce	Review mining, aspect sentiment, recommendation signals	

8.8.2.3 NLTK Installation and Setup

Let's start by installing NLTK and downloading the essential datasets:

```
pip install nltk
```

Collecting nltk

Downloading nltk-3.9.2-py3-none-any.whl.metadata (3.2 kB)

Collecting click (from nltk)

Downloading click-8.3.0-py3-none-any.whl.metadata (2.6 kB)

Collecting joblib (from nltk)

Using cached joblib-1.5.2-py3-none-any.whl.metadata (5.6 kB)

Collecting regex>=2021.8.3 (from nltk)

Using cached regex-2025.9.18-cp312-cp312-win_amd64.whl.metadata (41 kB)

Collecting tqdm (from nltk)

Using cached tqdm-4.67.1-py3-none-any.whl.metadata (57 kB)

Requirement already satisfied: colorama in c:\users\lsi8012\onedrive - northwestern universi

Downloading nltk-3.9.2-py3-none-any.whl (1.5 MB)

```
----- 0.0/1.5 MB ? eta -:--:--
----- 1.0/1.5 MB 7.1 MB/s eta 0:00:01
----- 1.5/1.5 MB 7.3 MB/s 0:00:00
```

Using cached regex-2025.9.18-cp312-cp312-win_amd64.whl (275 kB)

Downloading click-8.3.0-py3-none-any.whl (107 kB)

Using cached joblib-1.5.2-py3-none-any.whl (308 kB)

Using cached tqdm-4.67.1-py3-none-any.whl (78 kB)

Installing collected packages: tqdm, regex, joblib, click, nltk

[illegible]

Successfully installed click-8.3.0 joblib-1.5.2 nltk-3.9.2 regex-2025.9.18 tqdm-4.67.1
Note: you may need to restart the kernel to use updated packages.

WARNING: Retrying (Retry(total=4, connect=None, read=None, redirect=None, status=None)) after

```
# NLTK Installation and Essential Downloads

# Install NLTK (uncomment if not already installed)
# !pip install nltk

try:
```

```

import nltk
print(" NLTK is already installed")

# Download essential datasets (this might take a few minutes first time)
print(" Downloading essential NLTK data...")

# Download essential components
essential_downloads = [
    'punkt',          # Sentence tokenizer
    'stopwords',      # Stop words for multiple languages
    'vader_lexicon',  # Sentiment analysis
    'averaged_perceptron_tagger', # POS tagger
    'wordnet',        # WordNet lexical database
    'omw-1.4',        # Open Multilingual Wordnet
    'movie_reviews',  # Sample movie reviews for sentiment
]

for item in essential_downloads:
    try:
        nltk.download(item, quiet=True)
        print(f"   Downloaded: {item}")
    except:
        print(f"   Could not download: {item}")

print("\n NLTK Setup Complete!")
print("Ready for natural language processing!")

except ImportError:
    print(" NLTK not found. Please install with: pip install nltk")
except Exception as e:
    print(f" Setup issue: {e}")
    print("You may need internet connection for first-time downloads")

```

```

NLTK is already installed
Downloading essential NLTK data...
Downloaded: punkt
Downloaded: stopwords
Downloaded: vader_lexicon
Downloaded: averaged_perceptron_tagger
Downloaded: wordnet
Downloaded: omw-1.4
Downloaded: movie_reviews

```

```
NLTK Setup Complete!
Ready for natural language processing!
  Downloaded: averaged_perceptron_tagger
  Downloaded: wordnet
  Downloaded: omw-1.4
  Downloaded: movie_reviews
```

```
NLTK Setup Complete!
Ready for natural language processing!
```

```
[nltk_data] Error downloading 'vader_lexicon' from
[nltk_data] <https://raw.githubusercontent.com/nltk/nltk_data/gh-
[nltk_data] pages/packages/sentiment/vader_lexicon.zip>:
[nltk_data] <urlopen error [WinError 10054] An existing connection
[nltk_data] was forcibly closed by the remote host>
[nltk_data] Error downloading 'averaged_perceptron_tagger' from
[nltk_data] <https://raw.githubusercontent.com/nltk/nltk_data/gh-p
[nltk_data] ages/packages/taggers/averaged_perceptron_tagger.zip>:
[nltk_data] <urlopen error [WinError 10054] An existing connection
[nltk_data] was forcibly closed by the remote host>
[nltk_data] Error downloading 'omw-1.4' from
[nltk_data] <https://raw.githubusercontent.com/nltk/nltk_data/gh-
[nltk_data] pages/packages/corpora/omw-1.4.zip>: <urlopen error
[nltk_data] [WinError 10054] An existing connection was forcibly
[nltk_data] closed by the remote host>
[nltk_data] Error downloading 'movie_reviews' from
[nltk_data] <https://raw.githubusercontent.com/nltk/nltk_data/gh-
[nltk_data] pages/packages/corpora/movie_reviews.zip>: <urlopen
[nltk_data] error [WinError 10054] An existing connection was
[nltk_data] forcibly closed by the remote host>
```

8.8.2.4 Tokenization: Breaking Text into Pieces

Tokenization is the foundation of NLP - splitting text into meaningful units (words, sentences, or phrases).

```
# Tokenization Examples: Word and Sentence Level
from nltk.tokenize import word_tokenize, sent_tokenize
from nltk.tokenize import RegexpTokenizer, WhitespaceTokenizer
import pandas as pd
```



```

# Sample text for demonstration
sample_text = """
Natural Language Processing (NLP) is amazing! It helps computers understand human language.
Companies like Google, Amazon, and Microsoft use NLP for various applications.
Did you know that NLTK has been around since 2001? It's a powerful toolkit.
Let's explore tokenization: word-level, sentence-level, and custom patterns.
"""

print(" Original Text:")
print(sample_text.strip())

print("\n" + "="*60)
print(" TOKENIZATION EXAMPLES")
print("="*60)

# 1. SENTENCE TOKENIZATION
sentences = sent_tokenize(sample_text)
print(f"\n Sentence Tokenization ({len(sentences)} sentences):")
for i, sentence in enumerate(sentences, 1):
    print(f" {i}. {sentence.strip()}")

# 2. WORD TOKENIZATION
words = word_tokenize(sample_text)
print(f"\n Word Tokenization ({len(words)} tokens):")
print("First 15 tokens:", words[:15])

# 3. CUSTOM TOKENIZATION
# Only alphabetic tokens (removes punctuation and numbers)
alpha_tokenizer = RegexpTokenizer(r'[a-zA-Z]+')
alpha_tokens = alpha_tokenizer.tokenize(sample_text)
print(f"\n Alphabetic Only ({len(alpha_tokens)} tokens):")
print("First 15 tokens:", alpha_tokens[:15])

# 4. WHITESPACE TOKENIZATION
whitespace_tokenizer = WhitespaceTokenizer()
whitespace_tokens = whitespace_tokenizer.tokenize(sample_text)
print(f"\n Whitespace Tokenization ({len(whitespace_tokens)} tokens):")
print("First 10 tokens:", whitespace_tokens[:10])

# Create DataFrame for comparison
tokenization_comparison = pd.DataFrame({
    'Method': ['Sentences', 'Words (with punct.)', 'Alphabetic only', 'Whitespace'],

```

```

'Token_Count': [len(sentences), len(words), len(alpha_tokens), len(whitespace_tokens)],
'Sample_Tokens': [
    [s[:30] + "..." if len(s) > 30 else s for s in sentences[:2]],
    words[:3],
    alpha_tokens[:3],
    whitespace_tokens[:3]
]
})

print(f"\n Tokenization Comparison:")
print(tokenization_comparison.to_string(index=False))

```

Original Text:

Natural Language Processing (NLP) is amazing! It helps computers understand human language. Companies like Google, Amazon, and Microsoft use NLP for various applications. Did you know that NLTK has been around since 2001? It's a powerful toolkit. Let's explore tokenization: word-level, sentence-level, and custom patterns.

TOKENIZATION EXAMPLES

Sentence Tokenization (6 sentences):

1. Natural Language Processing (NLP) is amazing!
2. It helps computers understand human language.
3. Companies like Google, Amazon, and Microsoft use NLP for various applications.
4. Did you know that NLTK has been around since 2001?
5. It's a powerful toolkit.
6. Let's explore tokenization: word-level, sentence-level, and custom patterns.

Word Tokenization (60 tokens):

First 15 tokens: ['Natural', 'Language', 'Processing', '(', 'NLP', ')', 'is', 'amazing', '!',

Alphabetic Only (48 tokens):

First 15 tokens: ['Natural', 'Language', 'Processing', 'NLP', 'is', 'amazing', 'It', 'helps',

Whitespace Tokenization (45 tokens):

First 10 tokens: ['Natural', 'Language', 'Processing', '(NLP)', 'is', 'amazing!', 'It', 'helps',

Tokenization Comparison:

Method	Token_Count	Sample_Tokens
Sentences	6	[\nNatural Language Processing (... , It helps computers und

Words (with punct.)	60	[Natural, Language,
Alphabetic only	48	[Natural, Language,
Whitespace	45	[Natural, Language,

8.8.2.5 Text Preprocessing: Stop Words and Cleaning

Stop words are common words that typically don't carry significant meaning and are often removed to focus on important content.

```
# Stop Words and Text Cleaning
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize
import string
import pandas as pd

# Sample product reviews for analysis
product_reviews = [
    "This product is absolutely amazing! I love it so much.",
    "The quality is terrible. I want my money back immediately.",
    "It's okay, nothing special but does the job well enough.",
    "Fantastic value for money! Highly recommend to everyone.",
    "Poor customer service and the product broke after one day."
]

print(" Sample Product Reviews:")
for i, review in enumerate(product_reviews, 1):
    print(f" {i}. {review}")

print("\n" + "="*60)
print(" TEXT PREPROCESSING PIPELINE")
print("="*60)

# Get English stop words
stop_words = set(stopwords.words('english'))
print(f"\n English Stop Words ({len(stop_words)} words):")
print("Sample stop words:", sorted(list(stop_words))[:20])

def preprocess_text(text):
    """Complete text preprocessing pipeline"""

    # 1. Convert to lowercase
    text = text.lower()
```

```

# 2. Tokenize
tokens = word_tokenize(text)

# 3. Remove punctuation and non-alphabetic tokens
tokens = [token for token in tokens if token.isalpha()]

# 4. Remove stop words
filtered_tokens = [token for token in tokens if token not in stop_words]

return tokens, filtered_tokens

# Process all reviews
processed_reviews = []

print(f"\n Processing Each Review:")
for i, review in enumerate(product_reviews, 1):
    original_tokens, filtered_tokens = preprocess_text(review)

    processed_reviews.append({
        'Review_ID': f'Review_{i}',
        'Original_Text': review,
        'Original_Tokens': len(original_tokens),
        'After_Cleaning': len(filtered_tokens),
        'Tokens_Removed': len(original_tokens) - len(filtered_tokens),
        'Key_Words': filtered_tokens[:5] # First 5 meaningful words
    })

    print(f"\n    Review {i}:")
    print(f"    Original: {review}")
    print(f"    Key words: {filtered_tokens[:8]}")
    print(f"    Tokens: {len(original_tokens)} → {len(filtered_tokens)} (-{len(original_tokens) - len(filtered_tokens)})")

# Create summary DataFrame
df_preprocessing = pd.DataFrame(processed_reviews)

print(f"\n Preprocessing Summary:")
print(df_preprocessing[['Review_ID', 'Original_Tokens', 'After_Cleaning', 'Tokens_Removed']])

print(f"\n Overall Statistics:")
total_original = df_preprocessing['Original_Tokens'].sum()
total_cleaned = df_preprocessing['After_Cleaning'].sum()
print(f"• Total tokens before cleaning: {total_original}")

```

```
print(f"• Total tokens after cleaning: {total_cleaned}")
print(f"• Tokens removed: {total_original - total_cleaned} ({((total_original - total_cleaned) / total_original) * 100}%)")
print(f"• Average tokens per review (cleaned): {total_cleaned / len(product_reviews):.1f}")
```

Sample Product Reviews:

1. This product is absolutely amazing! I love it so much.
2. The quality is terrible. I want my money back immediately.
3. It's okay, nothing special but does the job well enough.
4. Fantastic value for money! Highly recommend to everyone.
5. Poor customer service and the product broke after one day.

```
=====
TEXT PREPROCESSING PIPELINE
=====
```

English Stop Words (198 words):

Sample stop words: ['a', 'about', 'above', 'after', 'again', 'against', 'ain', 'all', 'am',

Processing Each Review:

Review 1:

Original: This product is absolutely amazing! I love it so much.

Key words: ['product', 'absolutely', 'amazing', 'love', 'much']

Tokens: 10 → 5 (-5)

Review 2:

Original: The quality is terrible. I want my money back immediately.

Key words: ['quality', 'terrible', 'want', 'money', 'back', 'immediately']

Tokens: 10 → 6 (-4)

Review 3:

Original: It's okay, nothing special but does the job well enough.

Key words: ['okay', 'nothing', 'special', 'job', 'well', 'enough']

Tokens: 10 → 6 (-4)

Review 4:

Original: Fantastic value for money! Highly recommend to everyone.

Key words: ['fantastic', 'value', 'money', 'highly', 'recommend', 'everyone']

Tokens: 8 → 6 (-2)

Review 5:

Original: Poor customer service and the product broke after one day.

Key words: ['poor', 'customer', 'service', 'product', 'broke', 'one', 'day']
Tokens: 10 → 7 (-3)

Preprocessing Summary:

Review_ID	Original_Tokens	After_Cleaning	Tokens_Removed
Review_1	10	5	5
Review_2	10	6	4
Review_3	10	6	4
Review_4	8	6	2
Review_5	10	7	3

Overall Statistics:

- Total tokens before cleaning: 48
- Total tokens after cleaning: 30
- Tokens removed: 18 (37.5%)
- Average tokens per review (cleaned): 6.0

8.8.2.6 Stemming and Lemmatization: Finding Word Roots

Both stemming and lemmatization reduce words to their base form, but they work differently: - **Stemming**: Fast, rule-based chopping (running → run) - **Lemmatization**: Slower, dictionary-based reduction (better → good)

```
# Stemming vs Lemmatization Comparison
from nltk.stem import PorterStemmer, SnowballStemmer
from nltk.stem import WordNetLemmatizer
import pandas as pd

# Initialize stemmers and lemmatizer
porter = PorterStemmer()
snowball = SnowballStemmer('english')
lemmatizer = WordNetLemmatizer()

# Test words that demonstrate differences
test_words = [
    'running', 'runs', 'ran', 'runner',          # Run variations
    'better', 'good', 'best',                    # Good variations
    'dogs', 'cats', 'children', 'geese',         # Plural forms
    'walking', 'walked', 'walks',               # Walk variations
    'flying', 'flies', 'flew',                  # Fly variations
    'studies', 'studying', 'studied',            # Study variations
    'beautiful', 'beautifully',                 # Adjective/adverb
]
```

```

    'happiness', 'happier', 'happiest'          # Happy variations
]

print(" STEMMING vs LEMMATIZATION COMPARISON")
print("="*60)

# Process each word
results = []
for word in test_words:
    porter_stem = porter.stem(word)
    snowball_stem = snowball.stem(word)
    lemma = lemmatizer.lemmatize(word)
    lemma_verb = lemmatizer.lemmatize(word, pos='v') # As verb

    results.append({
        'Original': word,
        'Porter_Stem': porter_stem,
        'Snowball_Stem': snowball_stem,
        'Lemma_Noun': lemma,
        'Lemma_Verb': lemma_verb
    })

# Create comparison DataFrame
df_comparison = pd.DataFrame(results)

print(" Detailed Comparison:")
print(df_comparison.to_string(index=False))

print(f"\n Key Differences Analysis:")

# Count unique results for each method
porter_unique = df_comparison['Porter_Stem'].nunique()
snowball_unique = df_comparison['Snowball_Stem'].nunique()
lemma_unique = df_comparison['Lemma_Noun'].nunique()

print(f"• Porter Stemmer: {porter_unique} unique stems")
print(f"• Snowball Stemmer: {snowball_unique} unique stems")
print(f"• WordNet Lemmatizer: {lemma_unique} unique lemmas")

print(f"\n Notable Examples:")

# Find interesting cases where methods differ significantly

```

```

interesting_cases = [
    ('better', 'good'),    # Lemmatization handles irregular forms
    ('studies', 'study'), # Different handling of -ies ending
    ('flying', 'fly'),     # Verb vs noun treatment
    ('geese', 'goose')     # Irregular plurals
]

for original, expected in interesting_cases:
    if original in test_words:
        row = df_comparison[df_comparison['Original'] == original].iloc[0]
        print(f"• '{original}' → Porter: '{row['Porter_Stem']}', Lemma: '{row['Lemma_Noun']}'")

# Real-world application example
sample_sentence = "The children were running happily through the beautiful gardens, studying"
words_to_process = sample_sentence.lower().replace(',', ' ').replace('.', ' ').split()

print(f"\n Real-world Example:")
print(f"Sentence: {sample_sentence}")
print(f"\nWord processing:")

sentence_results = []
for word in words_to_process:
    if word.isalpha(): # Skip punctuation
        stem = porter.stem(word)
        lemma = lemmatizer.lemmatize(word)
        sentence_results.append(f"{word} → stem: {stem}, lemma: {lemma}")

for result in sentence_results[:6]: # Show first 6 words
    print(f" {result}")

print(f"\n Performance Guidelines:")
print("• Use STEMMING when: Speed is crucial, approximate matching is okay")
print("• Use LEMMATIZATION when: Accuracy is important, working with formal text")
print("• Stemming is ~10x faster but less accurate than lemmatization")

```

STEMMING vs LEMMATIZATION COMPARISON

=====

Detailed Comparison:

Original	Porter_Stem	Snowball_Stem	Lemma_Noun	Lemma_Verb
running	run	run	running	run
runs	run	run	run	run
ran	ran	ran	ran	run

runner	runner	runner	runner	runner
better	better	better	better	better
good	good	good	good	good
best	best	best	best	best
dogs	dog	dog	dog	dog
cats	cat	cat	cat	cat
children	children	children	child	children
geese	gees	gees	goose	geese
walking	walk	walk	walking	walk
walked	walk	walk	walked	walk
walks	walk	walk	walk	walk
flying	fli	fli	flying	fly
flies	fli	fli	fly	fly
flew	flew	flew	flew	fly
studies	studi	studi	study	study
studying	studi	studi	studying	study
studied	studi	studi	studied	study
beautiful	beauti	beauti	beautiful	beautiful
beautifully	beauti	beauti	beautifully	beautifully
happiness	happi	happi	happiness	happiness
happier	happier	happier	happier	happier
happiest	happiest	happiest	happiest	happiest

Key Differences Analysis:

- Porter Stemmer: 18 unique stems
- Snowball Stemmer: 18 unique stems
- WordNet Lemmatizer: 25 unique lemmas

Notable Examples:

- 'better' → Porter: 'better', Lemma: 'better' (expected: 'good')
- 'studies' → Porter: 'studi', Lemma: 'study' (expected: 'study')
- 'flying' → Porter: 'fli', Lemma: 'flying' (expected: 'fly')
- 'geese' → Porter: 'gees', Lemma: 'goose' (expected: 'goose')

Real-world Example:

Sentence: The children were running happily through the beautiful gardens, studying the flyin

Word processing:

the → stem: the, lemma: the
children → stem: children, lemma: child
were → stem: were, lemma: were
running → stem: run, lemma: running
happily → stem: happili, lemma: happily

through → stem: through, lemma: through

Performance Guidelines:

- Use STEMMING when: Speed is crucial, approximate matching is okay
- Use LEMMATIZATION when: Accuracy is important, working with formal text
- Stemming is ~10x faster but less accurate than lemmatization

Detailed Comparison:

Original	Porter_Stem	Snowball_Stem	Lemma_Noun	Lemma_Verb
running	run	run	running	run
runs	run	run	run	run
ran	ran	ran	ran	run
runner	runner	runner	runner	runner
better	better	better	better	better
good	good	good	good	good
best	best	best	best	best
dogs	dog	dog	dog	dog
cats	cat	cat	cat	cat
children	children	children	child	children
geese	gees	gees	goose	geese
walking	walk	walk	walking	walk
walked	walk	walk	walked	walk
walks	walk	walk	walk	walk
flying	fli	fli	flying	fly
flies	fli	fli	fly	fly
flew	flew	flew	flew	fly
studies	studi	studi	study	study
studying	studi	studi	studying	study
studied	studi	studi	studied	study
beautiful	beauti	beauti	beautiful	beautiful
beautifully	beauti	beauti	beautifully	beautifully
happiness	happi	happi	happiness	happiness
happier	happier	happier	happier	happier
happiest	happiest	happiest	happiest	happiest

Key Differences Analysis:

- Porter Stemmer: 18 unique stems
- Snowball Stemmer: 18 unique stems
- WordNet Lemmatizer: 25 unique lemmas

Notable Examples:

- 'better' → Porter: 'better', Lemma: 'better' (expected: 'good')
- 'studies' → Porter: 'studi', Lemma: 'study' (expected: 'study')
- 'flying' → Porter: 'fli', Lemma: 'flying' (expected: 'fly')

- 'geese' → Porter: 'gees', Lemma: 'goose' (expected: 'goose')

Real-world Example:

Sentence: The children were running happily through the beautiful gardens, studying the flyi

Word processing:

```
the → stem: the, lemma: the
children → stem: children, lemma: child
were → stem: were, lemma: were
running → stem: run, lemma: running
happily → stem: happili, lemma: happily
through → stem: through, lemma: through
```

Performance Guidelines:

- Use STEMMING when: Speed is crucial, approximate matching is okay
- Use LEMMATIZATION when: Accuracy is important, working with formal text
- Stemming is ~10x faster but less accurate than lemmatization

8.8.2.7 Sentiment Analysis: Understanding Emotions

NLTK provides powerful tools for analyzing sentiment - determining whether text expresses positive, negative, or neutral emotions.

```
# Sentiment Analysis with NLTK VADER
from nltk.sentiment import SentimentIntensityAnalyzer
import pandas as pd
import matplotlib.pyplot as plt
nltk.download('vader_lexicon')

# Initialize VADER sentiment analyzer
analyzer = SentimentIntensityAnalyzer()

# Diverse sample texts for sentiment analysis
sample_texts = [
    # Positive examples
    "This product is absolutely amazing! I love everything about it!",
    "Fantastic customer service and fast delivery. Highly recommend!",
    "Great value for money. Very satisfied with my purchase.",

    # Negative examples
    "Terrible quality, waste of money. Very disappointed.",
    "Poor customer service and the product broke immediately.",
```

```

    "Worst purchase I've ever made. Completely useless.",

    # Neutral examples
    "The product arrived on time. It's okay, nothing special.",
    "Standard quality for the price. Does what it's supposed to do.",
    "Average product with typical features. Works as expected.",

    # Mixed sentiment
    "Great design but poor durability. Mixed feelings about this.",
    "Love the features but hate the price. It's complicated.",
]

print(" SENTIMENT ANALYSIS WITH NLTK VADER")
print("="*60)

# Analyze sentiment for each text
sentiment_results = []

for i, text in enumerate(sample_texts, 1):
    # Get sentiment scores
    scores = analyzer.polarity_scores(text)

    # Determine overall sentiment
    if scores['compound'] >= 0.05:
        overall = 'Positive'
        emoji = '😊'
    elif scores['compound'] <= -0.05:
        overall = 'Negative'
        emoji = '😞'
    else:
        overall = 'Neutral'
        emoji = '😐'

    sentiment_results.append({
        'Text_ID': f'Text_{i:02d}',
        'Text': text[:50] + "..." if len(text) > 50 else text,
        'Positive': round(scores['pos'], 3),
        'Neutral': round(scores['neu'], 3),
        'Negative': round(scores['neg'], 3),
        'Compound': round(scores['compound'], 3),
        'Sentiment': overall,
        'Emoji': emoji,
    })

```

```

        'Full_Text': text
    })

    print(f"{emoji} Text {i}: {overall} (compound: {scores['compound']:.3f})")
    print(f"    \"{text}\"")
    print(f"    Pos: {scores['pos']:.2f}, Neu: {scores['neu']:.2f}, Neg: {scores['neg']:.2f}")
    print()

# Create DataFrame for analysis
df_sentiment = pd.DataFrame(sentiment_results)

print(" SENTIMENT ANALYSIS SUMMARY")
print("="*40)

# Overall statistics
sentiment_counts = df_sentiment['Sentiment'].value_counts()
print(" Sentiment Distribution:")
for sentiment, count in sentiment_counts.items():
    percentage = (count / len(df_sentiment)) * 100
    print(f" {sentiment}: {count} texts ({percentage:.1f}%)")

print(f"\n Score Ranges:")
print(f"• Positive scores: {df_sentiment['Positive'].min():.3f} - {df_sentiment['Positive'].max():.3f}")
print(f"• Negative scores: {df_sentiment['Negative'].min():.3f} - {df_sentiment['Negative'].max():.3f}")
print(f"• Compound scores: {df_sentiment['Compound'].min():.3f} - {df_sentiment['Compound'].max():.3f}")

# Show most extreme examples
print(f"\n Most Extreme Examples:")
most_positive = df_sentiment.loc[df_sentiment['Compound'].idxmax()]
most_negative = df_sentiment.loc[df_sentiment['Compound'].idxmin()]

print(f" Most Positive (score: {most_positive['Compound']:.3f}):")
print(f"    \"{most_positive['Full_Text']}\"")
print(f" Most Negative (score: {most_negative['Compound']:.3f}):")
print(f"    \"{most_negative['Full_Text']}\"")

# Business application example
print(f"\n Business Application Example:")
print("This could be used for:")
print("• Customer review analysis")
print("• Social media monitoring")
print("• Product feedback categorization")

```

```

print("• Brand sentiment tracking")
print("• Customer support prioritization")

# Display summary table
print(f"\n Detailed Results:")
display_df = df_sentiment[['Text_ID', 'Text', 'Compound', 'Sentiment', 'Emoji']].copy()
print(display_df.to_string(index=False))

```

```

[nltk_data] Downloading package vader_lexicon to
[nltk_data] C:\Users\lsi8012\AppData\Roaming\nltk_data...

```

SENTIMENT ANALYSIS WITH NLTK VADER

```

=====
Text 1: Positive (compound: 0.879)
    "This product is absolutely amazing! I love everything about it!"
    Pos: 0.57, Neu: 0.43, Neg: 0.00

Text 2: Positive (compound: 0.771)
    "Fantastic customer service and fast delivery. Highly recommend!"
    Pos: 0.53, Neu: 0.47, Neg: 0.00

Text 3: Positive (compound: 0.862)
    "Great value for money. Very satisfied with my purchase."
    Pos: 0.61, Neu: 0.39, Neg: 0.00

Text 4: Negative (compound: -0.852)
    "Terrible quality, waste of money. Very disappointed."
    Pos: 0.00, Neu: 0.30, Neg: 0.70

Text 5: Negative (compound: -0.710)
    "Poor customer service and the product broke immediately."
    Pos: 0.00, Neu: 0.50, Neg: 0.50

Text 6: Negative (compound: -0.802)
    "Worst purchase I've ever made. Completely useless."
    Pos: 0.00, Neu: 0.41, Neg: 0.59

Text 7: Negative (compound: -0.092)
    "The product arrived on time. It's okay, nothing special."
    Pos: 0.17, Neu: 0.63, Neg: 0.20

```

Text 8: Neutral (compound: 0.000)
"Standard quality for the price. Does what it's supposed to do."
Pos: 0.00, Neu: 1.00, Neg: 0.00

Text 9: Neutral (compound: 0.000)
"Average product with typical features. Works as expected."
Pos: 0.00, Neu: 1.00, Neg: 0.00

Text 10: Negative (compound: -0.382)
"Great design but poor durability. Mixed feelings about this."
Pos: 0.19, Neu: 0.51, Neg: 0.30

Text 11: Negative (compound: -0.535)
"Love the features but hate the price. It's complicated."
Pos: 0.18, Neu: 0.48, Neg: 0.34

SENTIMENT ANALYSIS SUMMARY

=====

Sentiment Distribution:

Negative: 6 texts (54.5%)
Positive: 3 texts (27.3%)
Neutral: 2 texts (18.2%)

Score Ranges:

- Positive scores: 0.000 - 0.615
- Negative scores: 0.000 - 0.699
- Compound scores: -0.852 - 0.879

Most Extreme Examples:

Most Positive (score: 0.879):

"This product is absolutely amazing! I love everything about it!"

Most Negative (score: -0.852):

"Terrible quality, waste of money. Very disappointed."

Business Application Example:

This could be used for:

- Customer review analysis
- Social media monitoring
- Product feedback categorization
- Brand sentiment tracking
- Customer support prioritization

Detailed Results:

Text_ID	Text	Compound	Sentiment	Emoji
Text_01	This product is absolutely amazing! I love everyth...	0.879	Positive	
Text_02	Fantastic customer service and fast delivery. High...	0.771	Positive	
Text_03	Great value for money. Very satisfied with my purc...	0.862	Positive	
Text_04	Terrible quality, waste of money. Very disappointe...	-0.852	Negative	
Text_05	Poor customer service and the product broke immedi...	-0.710	Negative	
Text_06	Worst purchase I've ever made. Completely useless.	-0.802	Negative	
Text_07	The product arrived on time. It's okay, nothing sp...	-0.092	Negative	
Text_08	Standard quality for the price. Does what it's sup...	0.000	Neutral	
Text_09	Average product with typical features. Works as ex...	0.000	Neutral	
Text_10	Great design but poor durability. Mixed feelings a...	-0.382	Negative	
Text_11	Love the features but hate the price. It's complic...	-0.535	Negative	

8.8.2.8 Part-of-Speech (POS) Tagging & Named Entity Recognition

POS tagging identifies grammatical roles of words (noun, verb, adjective), while Named Entity Recognition (NER) finds important entities like names, places, and organizations.

```
# POS Tagging and Named Entity Recognition
import nltk
from nltk.tokenize import word_tokenize
from nltk.tag import pos_tag
from nltk.chunk import ne_chunk
from nltk.tree import Tree
nltk.download('averaged_perceptron_tagger_eng')
nltk.download('maxent_ne_chunker_tab')
nltk.download('maxent_ne_chunker_tab')
nltk.download('words')
nltk.download('words')

import pandas as pd

# Sample business-related text
business_text = """
Apple Inc. is planning to open a new headquarters in Austin, Texas next year.
CEO Tim Cook announced this during a meeting with investors on Wall Street.
The company expects to hire 5,000 new employees, including software engineers
and data scientists. Microsoft and Google are also expanding their operations
in the region. The project will cost approximately $1 billion dollars.
"""
```



```

print(" POS TAGGING & NAMED ENTITY RECOGNITION")
print("="*60)

print(" Sample Text:")
print(business_text.strip())

# 1. POS TAGGING
print(f"\n PART-OF-SPEECH TAGGING:")
tokens = word_tokenize(business_text)
pos_tags = pos_tag(tokens)

# Group by POS types for analysis
pos_groups = {}
for word, pos in pos_tags:
    if pos not in pos_groups:
        pos_groups[pos] = []
    pos_groups[pos].append(word)

print(f"Found {len(set([pos for _, pos in pos_tags]))} different POS tags:")

# Show most common POS categories
pos_description = {
    'NN': 'Noun (singular)',
    'NNS': 'Noun (plural)',
    'NNP': 'Proper noun (singular)',
    'NNPS': 'Proper noun (plural)',
    'VB': 'Verb (base form)',
    'VBD': 'Verb (past tense)',
    'VBG': 'Verb (gerund/present participle)',
    'VBN': 'Verb (past participle)',
    'VBP': 'Verb (present, not 3rd person singular)',
    'VBZ': 'Verb (present, 3rd person singular)',
    'JJ': 'Adjective',
    'JJR': 'Adjective (comparative)',
    'JJS': 'Adjective (superlative)',
    'RB': 'Adverb',
    'DT': 'Determiner',
    'IN': 'Preposition/subordinating conjunction',
    'TO': 'to',
    'CD': 'Cardinal number'
}

```

```

for pos in sorted(pos_groups.keys()):
    if pos in pos_description:
        example_words = pos_groups[pos][:3] # Show first 3 examples
        print(f" {pos} ({pos_description[pos]}): {example_words}")

# 2. NAMED ENTITY RECOGNITION
print(f"\n NAMED ENTITY RECOGNITION:")
named_entities = ne_chunk(pos_tags)

# Extract entities
entities = []
def extract_entities(tree):
    for subtree in tree:
        if isinstance(subtree, Tree): # It's a named entity
            entity_name = ' '.join([token for token, pos in subtree.leaves()])
            entity_label = subtree.label()
            entities.append((entity_name, entity_label))

extract_entities(named_entities)

if entities:
    print(" Detected Entities:")
    entity_types = {}
    for entity, label in entities:
        if label not in entity_types:
            entity_types[label] = []
        entity_types[label].append(entity)

    for label, entity_list in entity_types.items():
        print(f" {label}: {entity_list}")
else:
    print(" No named entities detected with basic NER")

# 3. MANUAL ENTITY EXTRACTION (for demonstration)
print(f"\n MANUAL ENTITY IDENTIFICATION:")

# Find potential entities using POS tags
companies = [word for word, pos in pos_tags if pos == 'NNP' and word in ['Apple', 'Microsoft']]
people = [word for word, pos in pos_tags if pos == 'NNP' and word in ['Tim', 'Cook']]
locations = [word for word, pos in pos_tags if pos == 'NNP' and word in ['Austin', 'Texas',
numbers = [word for word, pos in pos_tags if pos == 'CD']]

```

```

manual_entities = {
    'Companies': companies,
    'People': people,
    'Locations': locations,
    'Numbers': numbers
}

for category, items in manual_entities.items():
    if items:
        print(f" {category}: {items}")

# 4. BUSINESS APPLICATIONS
print(f"\n BUSINESS APPLICATIONS:")
print("POS Tagging can be used for:")
print("• Information extraction from documents")
print("• Improving search relevance")
print("• Text summarization")
print("• Grammar checking tools")
print("• Keyword extraction")

print("\nNamed Entity Recognition can be used for:")
print("• Customer relationship management")
print("• Compliance and regulatory analysis")
print("• News and social media monitoring")
print("• Document classification")
print("• Knowledge graph construction")

# Create summary DataFrame
word_analysis = []
for word, pos in pos_tags[:15]: # First 15 words
    if word.isalpha(): # Skip punctuation
        is_entity = any(word in entity for entity, _ in entities)
        word_analysis.append({
            'Word': word,
            'POS_Tag': pos,
            'POS_Description': pos_description.get(pos, 'Other'),
            'Is_Entity': is_entity
        })

df_analysis = pd.DataFrame(word_analysis)
print(f"\n Sample Word Analysis:")
print(df_analysis.to_string(index=False))

```

```

[nltk_data] Downloading package averaged_perceptron_tagger_eng to
[nltk_data] C:\Users\lsi8012\AppData\Roaming\nltk_data...
[nltk_data] Package averaged_perceptron_tagger_eng is already up-to-
[nltk_data] date!
[nltk_data] Downloading package maxent_ne_chunker_tab to
[nltk_data] C:\Users\lsi8012\AppData\Roaming\nltk_data...
[nltk_data] Package maxent_ne_chunker_tab is already up-to-date!
[nltk_data] Downloading package maxent_ne_chunker_tab to
[nltk_data] C:\Users\lsi8012\AppData\Roaming\nltk_data...
[nltk_data] Package maxent_ne_chunker_tab is already up-to-date!
[nltk_data] Downloading package words to
[nltk_data] C:\Users\lsi8012\AppData\Roaming\nltk_data...
[nltk_data] Unzipping corpora\words.zip.
[nltk_data] Downloading package words to
[nltk_data] C:\Users\lsi8012\AppData\Roaming\nltk_data...
[nltk_data] Package words is already up-to-date!
[nltk_data] Downloading package words to
[nltk_data] C:\Users\lsi8012\AppData\Roaming\nltk_data...
[nltk_data] Package words is already up-to-date!

```

POS TAGGING & NAMED ENTITY RECOGNITION

=====

Sample Text:

Apple Inc. is planning to open a new headquarters in Austin, Texas next year. CEO Tim Cook announced this during a meeting with investors on Wall Street. The company expects to hire 5,000 new employees, including software engineers and data scientists. Microsoft and Google are also expanding their operations in the region. The project will cost approximately \$1 billion dollars.

PART-OF-SPEECH TAGGING:

Found 20 different POS tags:

```

CD (Cardinal number): ['5,000', '1', 'billion']
DT (Determiner): ['a', 'this', 'a']
IN (Preposition/subordinating conjunction): ['in', 'during', 'with']
JJ (Adjective): ['new', 'next', 'new']
NN (Noun (singular)): ['headquarters', 'year', 'meeting']
NNP (Proper noun (singular)): ['Apple', 'Inc.', 'Austin']
NNS (Noun (plural)): ['investors', 'employees', 'engineers']
RB (Adverb): ['also', 'approximately']
TO (to): ['to', 'to']
VB (Verb (base form)): ['open', 'hire', 'cost']
VBD (Verb (past tense)): ['announced']

```

VBG (Verb (gerund/present participle)): ['planning', 'including', 'expanding']
VBP (Verb (present, not 3rd person singular)): ['are']
VBZ (Verb (present, 3rd person singular)): ['is', 'expects']

NAMED ENTITY RECOGNITION:

Detected Entities:

PERSON: ['Apple', 'Microsoft']
ORGANIZATION: ['Inc.', 'CEO Tim Cook']
GPE: ['Austin', 'Texas', 'Google']
FACILITY: ['Wall Street']

MANUAL ENTITY IDENTIFICATION:

Companies: ['Apple', 'Microsoft', 'Google']
People: ['Tim', 'Cook']
Locations: ['Austin', 'Texas', 'Wall', 'Street']
Numbers: ['5,000', '1', 'billion']

BUSINESS APPLICATIONS:

POS Tagging can be used for:

- Information extraction from documents
- Improving search relevance
- Text summarization
- Grammar checking tools
- Keyword extraction

Named Entity Recognition can be used for:

- Customer relationship management
- Compliance and regulatory analysis
- News and social media monitoring
- Document classification
- Knowledge graph construction

Sample Word Analysis:

Word	POS_Tag	POS_Description	Is_Entity
Apple	NNP	Proper noun (singular)	True
is	VBZ	Verb (present, 3rd person singular)	False
planning	VBG	Verb (gerund/present participle)	False
to	TO	to	False
open	VB	Verb (base form)	False
a	DT	Determiner	True
new	JJ	Adjective	False
headquarters	NN	Noun (singular)	False
in	IN	Preposition/subordinating conjunction	True

Austin	NNP	Proper noun (singular)	True
Texas	NNP	Proper noun (singular)	True
next	JJ	Adjective	False
year	NN	Noun (singular)	False

Detected Entities:

PERSON: ['Apple', 'Microsoft']
 ORGANIZATION: ['Inc.', 'CEO Tim Cook']
 GPE: ['Austin', 'Texas', 'Google']
 FACILITY: ['Wall Street']

MANUAL ENTITY IDENTIFICATION:

Companies: ['Apple', 'Microsoft', 'Google']
 People: ['Tim', 'Cook']
 Locations: ['Austin', 'Texas', 'Wall', 'Street']
 Numbers: ['5,000', '1', 'billion']

BUSINESS APPLICATIONS:

POS Tagging can be used for:

- Information extraction from documents
- Improving search relevance
- Text summarization
- Grammar checking tools
- Keyword extraction

Named Entity Recognition can be used for:

- Customer relationship management
- Compliance and regulatory analysis
- News and social media monitoring
- Document classification
- Knowledge graph construction

Sample Word Analysis:

Word	POS_Tag	POS_Description	Is_Entity
Apple	NNP	Proper noun (singular)	True
is	VBZ	Verb (present, 3rd person singular)	False
planning	VBG	Verb (gerund/present participle)	False
to	TO	to	False
open	VB	Verb (base form)	False
a	DT	Determiner	True
new	JJ	Adjective	False
headquarters	NN	Noun (singular)	False
in	IN	Preposition/subordinating conjunction	True
Austin	NNP	Proper noun (singular)	True

Texas	NNP	Proper noun (singular)	True
next	JJ	Adjective	False
year	NN	Noun (singular)	False

8.8.2.9 NLTK Summary & Real-World Applications

8.8.2.9.1 NLTK Capabilities Overview

Feature	Purpose	Business Use Case	Performance
Tokenization	Split text into words/sentences	Document processing, search	Fast
Stop Words	Remove common words	Focus on meaningful content	Fast
Stemming	Reduce words to stems	Search optimization, indexing	Very Fast
Lemmatization	Reduce words to dictionary form	Text analysis, normalization	Medium
Sentiment Analysis	Detect emotions and opinions	Customer feedback, social monitoring	Fast
POS Tagging	Identify grammatical roles	Information extraction	Medium
Named Entity Recognition	Find people, places, organizations	Document analysis, CRM	Medium

8.8.2.9.2 Best Practices & Tips

Performance Optimization:

- Use **stemming** for speed-critical applications
- Use **lemmatization** for accuracy-critical applications
- **Cache** processed results for repeated text analysis
- **Batch process** large datasets instead of one-by-one

Data Quality:

- **Clean text** before processing (remove HTML, special characters)
- **Handle multiple languages** appropriately

- **Validate results** with domain experts
- **Consider context** when interpreting sentiment

NLTK provides an excellent foundation for understanding NLP concepts before moving to more advanced tools!

8.9 Independent Study

8.9.1 Practice exercise 1

8.9.1.1

Read the file *STAT303-1 survey for data analysis.csv*.

```
survey_data = pd.read_csv('./Datasets/STAT303-1 survey for data analysis.csv')
print("Survey data loaded successfully!")
print(f"Shape: {survey_data.shape}")
survey_data.head()
```

Survey data loaded successfully!
Shape: (192, 51)

	Timestamp	fav_alcohol	On average (approx.) how many parties a m
0	2022/09/13 1:43:34 pm GMT-5	I don't drink	1
1	2022/09/13 5:28:17 pm GMT-5	Hard liquor/Mixed drink	3
2	2022/09/13 7:56:38 pm GMT-5	Hard liquor/Mixed drink	3
3	2022/09/13 10:34:37 pm GMT-5	Hard liquor/Mixed drink	12
4	2022/09/14 4:46:19 pm GMT-5	I don't drink	1

8.9.1.2

Rename all the columns of the data, except the first two columns, with the shorter names in the list `new_col_names` given below. The order of column names in the list is the same as the order in which the columns are to be renamed starting with the third column from the left.

```
new_col_names = ['parties_per_month', 'do_you_smoke', 'weed', 'are_you_an_introvert_or_extrovert']
```



```
survey_data.columns = list(survey_data.columns[0:2])+new_col_names
```

8.9.1.3

Rename the following columns again:

1. Rename `do_you_smoke` to `smoke`.
2. Rename `are_you_an_introvert_or_extrovert` to `introvert_extrovert`.

Hint: Use the function `rename()`

```
# Rename specific columns using rename function

survey_data = survey_data.rename(columns={

    'do_you_smoke': 'smoke',

    'are_you_an_introvert_or_extrovert': 'introvert_extrovert'})

print(f"Updated columns: {list(survey_data.columns)}")
print("Specific columns renamed successfully!")
```

```
Updated columns: ['Timestamp', 'fav_alcohol', 'parties_per_month', 'smoke', 'weed', 'introvert_extrovert']
Specific columns renamed successfully!
```

8.9.1.4

Examine the column `num_insta_followers`. Some numbers in the column contain a comma(,) or a tilde(~). Remove both these characters from the numbers in the column.

Hint: You may use the function `str.replace()` of the Pandas Series class.

```
survey_data_insta = survey_data.copy()
survey_data_insta['num_insta_followers']=survey_data_insta['num_insta_followers'].str.replace(',', '')
survey_data_insta['num_insta_followers']=survey_data_insta['num_insta_followers'].str.replace('~', '')
```

```
survey_data_insta.head()
```

	Timestamp	fav_alcohol	parties_per_month	smoke	weed
0	2022/09/13 1:43:34 pm GMT-5	I don't drink	1	No	Occasionally
1	2022/09/13 5:28:17 pm GMT-5	Hard liquor/Mixed drink	3	No	Occasionally
2	2022/09/13 7:56:38 pm GMT-5	Hard liquor/Mixed drink	3	No	Yes
3	2022/09/13 10:34:37 pm GMT-5	Hard liquor/Mixed drink	12	No	No
4	2022/09/14 4:46:19 pm GMT-5	I don't drink	1	No	No

8.9.1.5

Use the cleaned dataset where the column `num_insta_followers` has been updated.

The last four variables in the dataset are:

1. `cant_change_math_ability`
2. `can_change_math_ability`
3. `math_is_genetic`
4. `much_effort_is_lack_of_talent`

Each of the above variables has values - **Agree** / **Disagree**. Replace **Agree** with 1 and **Disagree** with 0.

Hint : You can do it with any one of the following methods:

1. Use the `map()` function
2. Use the `apply()` function with the lambda function

```
# implement a function to convert the column to numeric, forcing errors to NaN

# Define the columns to convert
columns_to_convert = ['cant_change_math_ability', 'can_change_math_ability',
                      'math_is_genetic', 'much_effort_is_lack_of_talent']

# Method 1: Using map() function (avoids for-loop)
for col in columns_to_convert:
    survey_data_insta[col] = survey_data_insta[col].map({'Agree': 1, 'Disagree': 0})

print("Method 1: Using map() function")
print(survey_data_insta[columns_to_convert].head())
print("\nData types:")
print(survey_data_insta[columns_to_convert].dtypes)
```

```
# Method 2: Using apply() function with lambda (avoids for-loop)
# First, let's reload the data to show this method
survey_data_insta2 = survey_data.copy()
survey_data_insta2['num_insta_followers'] = survey_data_insta2['num_insta_followers'].str.replace(' ', '')
survey_data_insta2['num_insta_followers'] = survey_data_insta2['num_insta_followers'].str.replace('.', '')

for col in columns_to_convert:
    survey_data_insta2[col] = survey_data_insta2[col].apply(lambda x: 1 if x == 'Agree' else 0)

print("\n\nMethod 2: Using apply() with lambda function")
print(survey_data_insta2[columns_to_convert].head())
```

Method 1: Using map() function

	cant_change_math_ability	can_change_math_ability	math_is_genetic	\
0	0	1	0	
1	0	1	0	
2	0	1	0	
3	0	1	0	
4	1	0	0	

	much_effort_is_lack_of_talent
0	0
1	0
2	0
3	0
4	0

Data types:

cant_change_math_ability	int64
can_change_math_ability	int64
math_is_genetic	int64
much_effort_is_lack_of_talent	int64
dtype:	object

Method 2: Using apply() with lambda function

	cant_change_math_ability	can_change_math_ability	math_is_genetic	\
0	0	1	0	
1	0	1	0	
2	0	1	0	
3	0	1	0	

4	1	0	0
---	---	---	---

	much_effort_is_lack_of_talent
0	0
1	0
2	0
3	0
4	0

8.9.2 Practice exercise 2

This exercise will motivate vectorized computations with NumPy. Vectorized computations help perform computations more efficiently, and also make the code concise.

Read the (1) quantities of roll, bun, cake and bread required by 3 people - Ben, Barbara & Beth, from *food_quantity.csv*, (2) price of these food items in two shops - Target and Kroger, from *price.csv*. Find out which shop should each person go to minimize their expenses.

```
food_df = pd.read_csv('./datasets/food_quantity.csv')
food_df.head()
```

	Person	roll	bun	cake	bread
0	Ben	6	5	3	1
1	Barbara	3	6	2	2
2	Beth	3	4	3	1

```
price_df = pd.read_csv('./datasets/price.csv')
price_df.head()
```

	Item	Target	Kroger
0	roll	1.5	1.0
1	bun	2.0	2.5
2	cake	5.0	4.5
3	bread	16.0	17.0

8.9.2.1

Compute the expenses of Ben if he prefers to buy all food items from Target

```
# Step 1: Get Ben's quantities
ben_quantities = food_df[food_df['Person'] == 'Ben'].iloc[0] # Get Ben's row
print("Ben's food quantities:")
print(ben_quantities)
print()

# Step 2: Get Target prices
target_prices = price_df.set_index('Item')['Target'] # Set Item as index, get Target column
print("Target prices:")
print(target_prices)
print()

# Step 3: Calculate Ben's total expenses at Target
# Multiply quantities by corresponding Target prices
food_items = ['roll', 'bun', 'cake', 'bread']
total_expense = 0

print("Ben's expense breakdown at Target:")
for item in food_items:
    quantity = ben_quantities[item]
    price = target_prices[item]
    expense = quantity * price
    total_expense += expense
    print(f"{item}: {quantity} × ${price} = ${expense}")

print(f"\nBen's total expense at Target: ${total_expense}")
```

Ben's food quantities:

Person	Ben
roll	6
bun	5
cake	3
bread	1

Name: 0, dtype: object

Target prices:

Item	
roll	1.5

```
bun      2.0
cake     5.0
bread    16.0
Name: Target, dtype: float64
```

Ben's expense breakdown at Target:

```
roll: 6 × $1.5 = $9.0
bun: 5 × $2.0 = $10.0
cake: 3 × $5.0 = $15.0
bread: 1 × $16.0 = $16.0
```

Ben's total expense at Target: \$50.0

8.9.2.2

Compute Ben's total expenses at both Target and Kroger. Which store is cheaper for him?

```
# Step 1: Get Ben's quantities (reuse from previous calculation)
ben_quantities = food_df[food_df['Person'] == 'Ben'].iloc[0]
food_items = ['roll', 'bun', 'cake', 'bread']

# Step 2: Get prices from both stores
target_prices = price_df.set_index('Item')['Target']
kroger_prices = price_df.set_index('Item')['Kroger']

print(" BEN'S EXPENSE COMPARISON")
print("=" * 40)
print("Ben's quantities:", [ben_quantities[item] for item in food_items])
print()

# Step 3: Calculate expenses at both stores
target_total = 0
kroger_total = 0

print("Detailed breakdown:")
print(f"{'Item':<8} {'Qty':<4} {'Target':<8} {'Kroger':<8} {'Target $':<10} {'Kroger $':<10}")
print("-" * 60)

for item in food_items:
    quantity = ben_quantities[item]
```

```

    target_price = target_prices[item]
    kroger_price = kroger_prices[item]

    target_expense = quantity * target_price
    kroger_expense = quantity * kroger_price

    target_total += target_expense
    kroger_total += kroger_expense

    print(f"{item:<8} {quantity:<4} ${target_price:<7} ${kroger_price:<7} ${target_expense:<7} ${kroger_expense:<7}")

print("-" * 60)
print(f"{'TOTAL':<8} {'':<4} {'':<8} {'':<8} ${target_total:<9} ${kroger_total:<9}")

# Step 4: Determine which store is cheaper
print(f"\n COMPARISON RESULTS:")
print(f"Ben's total at Target: ${target_total}")
print(f"Ben's total at Kroger: ${kroger_total}")

if target_total < kroger_total:
    savings = kroger_total - target_total
    print(f" TARGET is cheaper by ${savings}")
    print(f"Ben should shop at TARGET to save money!")
elif kroger_total < target_total:
    savings = target_total - kroger_total
    print(f" KROGER is cheaper by ${savings}")
    print(f"Ben should shop at KROGER to save money!")
else:
    print(" Both stores cost the same for Ben!")

```

BEN'S EXPENSE COMPARISON

=====

Ben's quantities: [6, 5, 3, 1]

Detailed breakdown:

Item	Qty	Target	Kroger	Target \$	Kroger \$
roll	6	\$1.5	\$1.0	\$9.0	\$6.0
bun	5	\$2.0	\$2.5	\$10.0	\$12.5
cake	3	\$5.0	\$4.5	\$15.0	\$13.5
bread	1	\$16.0	\$17.0	\$16.0	\$17.0

TOTAL	\$50.0	\$49.0
-------	--------	--------

COMPARISON RESULTS:

Ben's total at Target: \$50.0

Ben's total at Kroger: \$49.0

KROGER is cheaper by \$1.0

Ben should shop at KROGER to save money!

8.9.2.3

Compute each person's total expenses at both Target and Kroger. Which store is cheaper for them?

```
# Step 1: Set up data
food_items = ['roll', 'bun', 'cake', 'bread']
target_prices = price_df.set_index('Item')['Target']
kroger_prices = price_df.set_index('Item')['Kroger']

print(" COMPLETE EXPENSE ANALYSIS FOR ALL PEOPLE")
print("=" * 60)
print(f"Food items: {food_items}")
print(f"Target prices: {target_prices.values}")
print(f"Kroger prices: {kroger_prices.values}")
print()

# Step 2: Calculate expenses for each person
results = []

for index, person_row in food_df.iterrows():
    person_name = person_row['Person']

    # Calculate total expenses at both stores
    target_total = 0
    kroger_total = 0

    print(f" {person_name.upper()} 'S ANALYSIS:")
    print("-" * 30)
    print(f"{'Item':<8} {'Qty':<4} {'Target $':<10} {'Kroger $':<10}")
    print("-" * 30)
```



```

for item in food_items:
    quantity = person_row[item]
    target_expense = quantity * target_prices[item]
    kroger_expense = quantity * kroger_prices[item]

    target_total += target_expense
    kroger_total += kroger_expense

    print(f"{item:<8} {quantity:<4} ${target_expense:<9.2f} ${kroger_expense:<9.2f}")

print("-" * 30)
print(f"{'TOTAL':<8} {'':<4} ${target_total:<9.2f} ${kroger_total:<9.2f}")

# Determine cheaper store
if target_total < kroger_total:
    cheaper_store = "TARGET"
    savings = kroger_total - target_total
    print(f" {cheaper_store} is cheaper by ${savings:.2f}")
elif kroger_total < target_total:
    cheaper_store = "KROGER"
    savings = target_total - kroger_total
    print(f" {cheaper_store} is cheaper by ${savings:.2f}")
else:
    cheaper_store = "TIE"
    savings = 0
    print(" Both stores cost the same!")

# Store results
results.append({
    'Person': person_name,
    'Target_Total': target_total,
    'Kroger_Total': kroger_total,
    'Cheaper_Store': cheaper_store,
    'Savings': savings
})

print()

# Step 3: Create summary DataFrame
results_df = pd.DataFrame(results)

print(" SUMMARY TABLE:")

```

```

print("=" * 50)
print(results_df.to_string(index=False))

print(f"\n FINAL RECOMMENDATIONS:")
print("=" * 30)
for _, row in results_df.iterrows():
    if row['Cheaper_Store'] == 'TIE':
        print(f"• {row['Person']}: Can shop at either store (same cost)")
    else:
        print(f"• {row['Person']}: Shop at {row['Cheaper_Store']} to save ${row['Savings']:.2f}")

# Step 4: Additional insights
print(f"\n ADDITIONAL INSIGHTS:")
print("=" * 25)
target_shoppers = results_df[results_df['Cheaper_Store'] == 'TARGET']['Person'].tolist()
kroger_shoppers = results_df[results_df['Cheaper_Store'] == 'KROGER']['Person'].tolist()
tie_shoppers = results_df[results_df['Cheaper_Store'] == 'TIE']['Person'].tolist()

if target_shoppers:
    print(f"Target is better for: {'', '.join(target_shoppers)}")
if kroger_shoppers:
    print(f"Kroger is better for: {'', '.join(kroger_shoppers)}")
if tie_shoppers:
    print(f"No difference for: {'', '.join(tie_shoppers)}")

total_savings = results_df['Savings'].sum()
print(f"Total potential savings if everyone shops optimally: ${total_savings:.2f}")

```

COMPLETE EXPENSE ANALYSIS FOR ALL PEOPLE

```

=====
Food items: ['roll', 'bun', 'cake', 'bread']
Target prices: [ 1.5  2.   5.  16. ]
Kroger prices: [ 1.   2.5  4.5 17. ]

```

BEN'S ANALYSIS:

```

-----
Item      Qty  Target $   Kroger $
-----
roll      6    $9.00    $6.00
bun       5    $10.00   $12.50
cake      3    $15.00   $13.50
bread     1    $16.00   $17.00

```

```

-----
TOTAL          $50.00    $49.00
  KROGER is cheaper by $1.00

```

BARBARA'S ANALYSIS:

```

-----
Item      Qty  Target $    Kroger $
-----
roll      3    $4.50    $3.00
bun       6    $12.00    $15.00
cake      2    $10.00    $9.00
bread     2    $32.00    $34.00
-----
TOTAL          $58.50    $61.00
  TARGET is cheaper by $2.50

```

BETH'S ANALYSIS:

```

-----
Item      Qty  Target $    Kroger $
-----
roll      3    $4.50    $3.00
bun       4    $8.00    $10.00
cake      3    $15.00    $13.50
bread     1    $16.00    $17.00
-----
TOTAL          $43.50    $43.50
  Both stores cost the same!

```

SUMMARY TABLE:

```

=====
Person  Target_Total  Kroger_Total  Cheaper_Store  Savings
-----
   Ben           50.0           49.0        KROGER         1.0
Barbara          58.5           61.0        TARGET         2.5
   Beth          43.5           43.5         TIE           0.0

```

FINAL RECOMMENDATIONS:

- ```

=====
• Ben: Shop at KROGER to save $1.00
• Barbara: Shop at TARGET to save $2.50
• Beth: Can shop at either store (same cost)

```

ADDITIONAL INSIGHTS:

```

=====

```

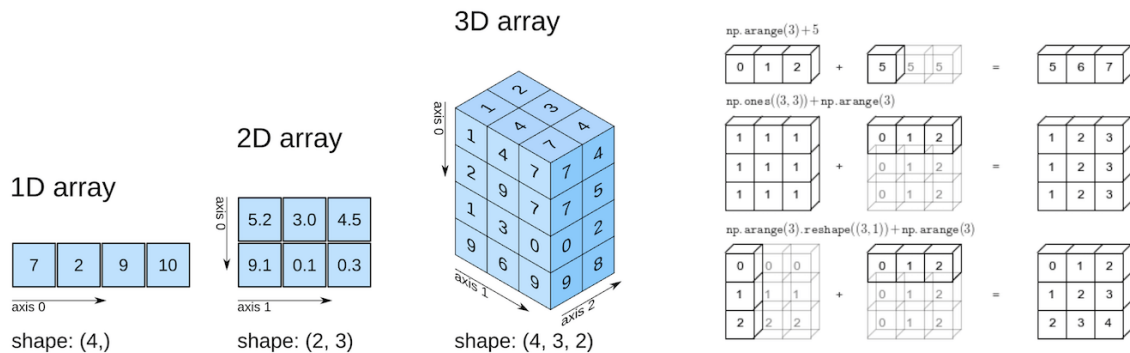
Target is better for: Barbara

Kroger is better for: Ben

No difference for: Beth

Total potential savings if everyone shops optimally: \$3.50

## 9 NumPy Fundamentals



**NumPy** is a foundational library in Python, providing support for large, multi-dimensional arrays and matrices, along with a variety of mathematical functions. It's a critical tool in data science and machine learning because it enables efficient numerical computations, data manipulation, and linear algebra operations. Many data science packages and machine learning algorithms rely on these operations to process data and perform complex calculations quickly. Moreover, popular libraries like **Pandas**, **SciPy**, and **TensorFlow** are built on top of NumPy, making it essential to understand for implementing and optimizing machine learning models.

### 9.1 Learning Objectives

By the end of this tutorial, you will be able to:

- **Create NumPy arrays** using various functions (`np.array()`, `np.zeros()`, `np.arange()`, etc.)
- **Interpret array attributes** (`shape`, `dtype`, `size`, `ndim`) and understand their roles in computation
- **Apply indexing and slicing** to extract and modify array elements

- Use **conditional selection** for advanced element filtering
- **Reshape and concatenate arrays efficiently** for flexible data manipulation

## 9.2 Getting Started with NumPy

```
Essential imports for numerical computing
import numpy as np
import pandas as pd
import time
import sys

Display NumPy version and configuration
print(f"NumPy version: {np.__version__}")
print(f"NumPy is installed at: {np.__file__}")
print("\n Ready to explore the world of numerical computing!")
```

NumPy version: 1.26.4

NumPy is installed at: c:\Users\lsi8012\AppData\Local\anaconda3\Lib\site-packages\numpy\\_\_in

Ready to explore the world of numerical computing!

If you encounter a `ModuleNotFoundError`, install NumPy using:

```
pip install numpy
```

## 9.3 Why NumPy?

Think of NumPy arrays as an upgrade to Python's built-in lists and tuples.

You can store data in them and perform the same kinds of computations—but with a big advantage:

- **Memory Savings:** Arrays use a fixed, compact layout in memory, reducing overhead.
- **Efficiency:** NumPy is optimized at the C level, making it far faster than pure Python structures.
- **Scalability:** Designed to handle large datasets smoothly without slowing down.

This is why NumPy is the backbone of scientific computing and data science in Python.

### 9.3.1 NumPy Arrays are Memory Efficient: Homogeneity & Contiguous Storage

A **NumPy array** stores elements of the **same data type** in **contiguous memory locations**. This contrasts with Python lists, which can hold mixed data types and store elements in scattered memory locations.

Because of this:

- NumPy arrays are **densely packed**, minimizing memory overhead.
- Operations can be executed much faster, since data is laid out predictably in memory.

The result: **lower memory consumption** and **higher performance** when working with large datasets.

The following example compares the memory usage of NumPy arrays with Python lists.

```
import sys

Create a NumPy array, Python list, and tuple with the same elements
array = np.arange(1000)
py_list = list(range(1000))
py_tuple = tuple(range(1000))

Calculate memory usage
array_memory = array.nbytes
list_memory = sys.getsizeof(py_list) + sum(sys.getsizeof(item) for item in py_list)
tuple_memory = sys.getsizeof(py_tuple) + sum(sys.getsizeof(item) for item in py_tuple)

Display the memory usage
memory_usage = {
 "NumPy Array (in bytes)": array_memory,
 "Python List (in bytes)": list_memory,
 "Python Tuple (in bytes)": tuple_memory
}

memory_usage
```

```
{'NumPy Array (in bytes)': 4000,
 'Python List (in bytes)': 36056,
 'Python Tuple (in bytes)': 36040}
```

```
each element in the array is a 64-bit integer
array.dtype
```

```
dtype('int32')
```

### 9.3.2 NumPy arrays are fast

NumPy arrays enable mathematical computations to run much faster than with native Python data structures. This speed advantage comes from three key factors:

- **Densely Packed & Homogeneous Data**

Since NumPy arrays store elements of the same type in contiguous memory, data retrieval is faster and computations can be executed more efficiently.

- **Vectorized Computations**

NumPy replaces slow Python `for` loops with **vectorized operations**, which are internally broken into optimized fragments and executed in parallel. This means calculations are applied to entire arrays at once, rather than element by element.

- **Integration with C/C++**

Under the hood, NumPy is implemented in **C and C++**, which are compiled languages with very low execution overhead compared to Python. This gives NumPy both the speed of C and the usability of Python.

We'll see the faster speed on NumPy computations in the example below.

```
def my_dot(a, b):
 """
 Compute the dot product of two vectors

 Args:
 a (ndarray (n,)): input vector
 b (ndarray (n,)): input vector with same dimension as a

 Returns:
 x (scalar):
 """
 x=0
 for i in range(a.shape[0]):
 x = x + a[i] * b[i]
 return x
```



```

np.random.seed(1)
a = np.random.rand(10000000) # very large arrays
b = np.random.rand(10000000)

tic = time.time() # capture start time
c = np.dot(a, b)
toc = time.time() # capture end time

print(f"np.dot(a, b) = {c:.4f}")
print(f"Vectorized version duration: {1000*(toc-tic):.4f} ms ")

tic = time.time() # capture start time
c = my_dot(a,b)
toc = time.time() # capture end time

print(f"my_dot(a, b) = {c:.4f}")
print(f"loop version duration: {1000*(toc-tic):.4f} ms ")

del(a);del(b)

```

```

np.dot(a, b) = 2501072.5817
Vectorized version duration: 34.6291 ms
my_dot(a, b) = 2501072.5817
loop version duration: 1979.4111 ms

```

Both methods produce the same result, but the vectorized `np.dot()` executes almost instantly, while the manual Python loop takes over a second.

This clearly demonstrates how NumPy's **vectorization** and **C backend** make computations orders of magnitude faster than pure Python loops.

## 9.4 Building Blocks: NumPy Arrays Fundamentals

### 9.4.1 Array Creation: Your Complete Toolkit

NumPy offers multiple ways to create arrays, each optimized for different scenarios.

#### 9.4.1.1 From Existing Data

| Method                     | Purpose                        | Example Use Case                 |
|----------------------------|--------------------------------|----------------------------------|
| <code>np.array()</code>    | Convert lists/tuples to arrays | Transform Python data structures |
| <code>df.to_numpy()</code> | Convert a Pandas DataFrame     | Bridge between Pandas and NumPy  |

```
1 Creating arrays from existing data
print("1 FROM EXISTING DATA")
print("=" * 30)

From Python lists
data_1d = [1, 2, 3, 4, 5]
arr_from_list = np.array(data_1d)
print(f"From list: {arr_from_list}")

From nested lists (2D array)
data_2d = [[1, 2, 3], [4, 5, 6]]
arr_2d = np.array(data_2d)
print(f"2D array:\n{arr_2d}")

From tuples
arr_from_tuple = np.array((10, 20, 30))
print(f"From tuple: {arr_from_tuple}")
```

```
1 FROM EXISTING DATA
=====
From list: [1 2 3 4 5]
2D array:
[[1 2 3]
 [4 5 6]]
From tuple: [10 20 30]
```

The `to_numpy()` method is used to convert a pandas DataFrame into a NumPy array.

```
data = {
 'A': [1, 2, 3],
 'B': [4, 5, 6],
 'C': [7, 8, 9]
}
df = pd.DataFrame(data)
print("DataFrame:")
```

```
print(df)

Convert DataFrame to NumPy array
array = df.to_numpy()
print("\nConverted NumPy array:")
print(array)
```

DataFrame:

```
 A B C
0 1 4 7
1 2 5 8
2 3 6 9
```

Converted NumPy array:

```
[[1 4 7]
 [2 5 8]
 [3 6 9]]
```

Notes:

- `.to_numpy()` returns a **NumPy** ndarray.
- The data type (`dtype`) is inferred automatically, but you can specify it if needed:

```
df.to_numpy(dtype=float)
```

```
array([[1., 4., 7.],
 [2., 5., 8.],
 [3., 6., 9.]])
```

- `.to_numpy()` is preferred over the older `.values` property.

This is often useful when you need to perform numerical operations with NumPy or machine learning libraries.

We will explore more examples and advanced operations in the **next chapter: NumPy Intermediate**.

#### 9.4.1.2 Specialized Constructors

| Method                           | Creates                   | When to Use                      |
|----------------------------------|---------------------------|----------------------------------|
| <code>np.zeros(shape)</code>     | Array of zeros            | Initialize arrays, placeholders  |
| <code>np.ones(shape)</code>      | Array of ones             | Mathematical operations, masks   |
| <code>np.full(shape, val)</code> | Array filled with a value | Default values, initialization   |
| <code>np.eye(n)</code>           | Identity matrix           | Linear algebra operations        |
| <code>np.empty(shape)</code>     | Uninitialized array       | Maximum speed (use with caution) |

```

print("\n2 SPECIALIZED CONSTRUCTORS")
print("=" * 30)
Arrays of zeros and ones
zeros_3x3 = np.zeros((3, 3))
ones_2x4 = np.ones((2, 4))
full_matrix = np.full((2, 3), 7) # Fill with custom value

print(f"Zeros (3x3):\n{zeros_3x3}")
print(f"Ones (2x4):\n{ones_2x4}")
print(f"Full of 7s:\n{full_matrix}")

Identity matrix
identity = np.eye(3)
print(f"Identity matrix:\n{identity}")

```

## 2 SPECIALIZED CONSTRUCTORS

=====

```

Zeros (3x3):
[[0. 0. 0.]
 [0. 0. 0.]
 [0. 0. 0.]]
Ones (2x4):
[[1. 1. 1. 1.]
 [1. 1. 1. 1.]]
Full of 7s:
[[7 7 7]
 [7 7 7]]
Identity matrix:
[[1. 0. 0.]
 [0. 1. 0.]
 [0. 0. 1.]]

```

### 9.4.1.3 Sequential & Mathematical Arrays

| Method                                     | Purpose                | Best For                               |
|--------------------------------------------|------------------------|----------------------------------------|
| <code>np.arange(start, stop, step)</code>  | Evenly spaced integers | Index generation, iteration            |
| <code>np.linspace(start, stop, num)</code> | Evenly spaced floats   | Plotting, function evaluation          |
| <code>np.logspace(start, stop, num)</code> | Logarithmically spaced | Scientific computing, exponential data |

```
print("\n3 SEQUENTIAL ARRAYS")
print("=" * 30)

Using arange (like Python's range)
sequence1 = np.arange(10) # 0 to 9
sequence2 = np.arange(5, 15) # 5 to 14
sequence3 = np.arange(0, 10, 2) # 0, 2, 4, 6, 8

print(f"arange(10): {sequence1}")
print(f"arange(5, 15): {sequence2}")
print(f"arange(0, 10, 2): {sequence3}")

Using linspace (evenly spaced floats)
linear = np.linspace(0, 1, 5) # 5 points between 0 and 1
print(f"linspace(0, 1, 5): {linear}")
```

```
3 SEQUENTIAL ARRAYS
=====
arange(10): [0 1 2 3 4 5 6 7 8 9]
arange(5, 15): [5 6 7 8 9 10 11 12 13 14]
arange(0, 10, 2): [0 2 4 6 8]
linspace(0, 1, 5): [0. 0.25 0.5 0.75 1.]
```

#### Pro Tip:

- Use `zeros()` for safe initialization
- Use `arange()` or `linspace()` for sequences

#### 9.4.1.4 Loading from Files

| Method                       | File Type                      | Features                                    |
|------------------------------|--------------------------------|---------------------------------------------|
| <code>np.load()</code>       | NumPy binary <code>.npy</code> | Fast saving/loading of arrays               |
| <code>np.loadtxt()</code>    | Simple text files              | Fast, lightweight parsing                   |
| <code>np.genfromtxt()</code> | Complex text files             | Handles missing values and mixed data types |

#### 9.4.2 Understanding Array Attributes

Let us define a NumPy array in order to access its attributes:

```
numpy_ex = np.array([[1,2,3],[4,5,6]])
numpy_ex
```

```
array([[1, 2, 3],
 [4, 5, 6]])
```

```
type(numpy_ex)
```

```
numpy.ndarray
```

It is an **ndarray** type.

You can explore the attributes and methods of `numpy_ex` by typing:

```
numpy_ex.
```

and then pressing the *tab* key.

Some of the basic attributes of a NumPy array are the following:

##### 9.4.2.1 `ndim`

Shows the number of dimensions (or axes) of the array.

```
numpy_ex.ndim
```

```
2
```

Numpy arrays can have any number of dimensions and different lengths along each dimension. We can inspect the length along each dimension using the `.shape` property of an array.

#### 9.4.2.2 `shape`

This is a tuple of integers indicating the size of the array in each dimension. For a matrix with  $n$  rows and  $m$  columns, the shape will be  $(n, m)$ . The length of the shape tuple is therefore the rank, or the number of dimensions, `ndim`.

```
numpy_ex.shape
```

```
(2, 3)
```

#### 9.4.2.3 `size`

This is the total number of elements of the array, which is the product of the elements of shape.

```
numpy_ex.size
```

```
6
```

#### 9.4.2.4 `dtype`

Unlike lists and tuples, NumPy arrays are designed to store elements of the same type, enabling more efficient memory usage and faster computations. The data type of the elements in a NumPy array can be accessed using the `.dtype` attribute

```
numpy_ex.dtype
```

```
dtype('int32')
```

## 9.5 Data Types and Memory Optimization

NumPy supports a wide range of data types, each with a defined memory size. These types map directly to **C data types**, since NumPy is implemented in C at its computational core. This design allows NumPy to handle array operations far more efficiently than native Python data structures.

### 9.5.1 Common NumPy Data Types

| Data Type      | Memory Size               |
|----------------|---------------------------|
| np.int8        | 1 byte                    |
| np.int16       | 2 bytes                   |
| np.int32       | 4 bytes                   |
| np.int64       | 8 bytes                   |
| np.uint8       | 1 byte                    |
| np.uint16      | 2 bytes                   |
| np.uint32      | 4 bytes                   |
| np.uint64      | 8 bytes                   |
| np.float16     | 2 bytes                   |
| np.float32     | 4 bytes                   |
| np.float64     | 8 bytes                   |
| np.complex64   | 8 bytes                   |
| np.complex128  | 16 bytes                  |
| np.bool_       | 1 byte                    |
| np.string_     | 1 byte per character      |
| np.unicode_    | 4 bytes per character     |
| np.object_     | Variable (Python objects) |
| np.datetime64  | 8 bytes                   |
| np.timedelta64 | 8 bytes                   |

**Tip:** Choosing the right data type is crucial for **memory efficiency** and **performance**. For large datasets, using smaller types (e.g., `int16` instead of `int64`) can save significant memory.

```
Add practical examples for data type selection
print(" CHOOSING THE RIGHT DATA TYPE")
print("=" * 40)

Memory efficiency example
large_numbers = np.array([1000, 2000, 3000], dtype=np.int16) # Efficient for small ranges
small_numbers = np.array([1, 2, 3], dtype=np.int8) # Very memory efficient

print(f"int16 array memory: {large_numbers.nbytes} bytes")
print(f"int8 array memory: {small_numbers.nbytes} bytes")

Precision example
high_precision = np.array([3.14159265359], dtype=np.float64)
low_precision = np.array([3.14159265359], dtype=np.float32)
```



```
print(f"float64 precision: {high_precision}")
print(f"float32 precision: {low_precision}")
```

#### CHOOSING THE RIGHT DATA TYPE

```
=====
int16 array memory: 6 bytes
int8 array memory: 3 bytes
float64 precision: [3.14159265]
float32 precision: [3.1415927]
```

### 9.5.2 Upcasting

When you create a NumPy array with elements of different data types, NumPy automatically **upcasts** them to a common type that can represent all values.

This process—also called **type coercion** or **type promotion**—follows a hierarchy of data types to prevent data loss.

Below are two common cases of upcasting, with examples:

**Numeric Upcasting:** If you mix integers and floats, NumPy will convert the entire array to floats.

```
arr = np.array([1, 2.5, 3])
print(arr.dtype)
```

float64

**String Upcasting:** If you mix numbers and strings, NumPy will upcast all elements to strings.

```
arr = np.array([1, 'hello', 3.5])
print(arr.dtype)
```

<U32

“<U32” means: a Unicode string array where each element can hold up to 32 characters, using little-endian byte order.

## 9.6 Array Indexing and Slicing: Accessing Your Data

### 9.6.1 Array Indexing

Similar to Python lists, NumPy uses zero-based indexing, meaning the first element of an array is accessed using index 0. You can use positive or negative indices to access elements

```
array = np.array([10, 20, 30, 40, 50])

print(array[0])
print(array[4])
print(array[-1])
print(array[-3])
```

```
10
50
50
30
```

In multi-dimensional arrays, indices are separated by commas. The first index refers to the row, and the second index refers to the column in a 2D array.

```
2D array (3 rows, 3 columns)
array_2d = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])
print(array_2d)
print(array_2d[0, 1])
print(array_2d[1, -1])
print(array_2d[-1, -1])
```

```
[[1 2 3]
 [4 5 6]
 [7 8 9]]
2
6
9
```

## 9.6.2 Array Slicing

Slicing is used to extract a sub-array from an existing array.

The Syntax for slicing is 'array[start:stop:step]

```
array = np.array([10, 20, 30, 40, 50])

print(array[1:4])
print(array[:3])
print(array[2:])
print(array[:2])
print(array[::-1])
```

```
[20 30 40]
[10 20 30]
[30 40 50]
[10 30 50]
[50 40 30 20 10]
```

For slicing in Multi-Dimensional Arrays, use commas to separate slicing for different dimensions

```
Indexing & Slicing Quick Reference

Extract a sub-array: elements from the first two rows and the first two columns
sub_array = array_2d[:2, :2]
print(sub_array)

Extract all rows for the second column
col = array_2d[:, 1]
print(col)

Extract the last two rows and last two columns
sub_array = array_2d[-2:, -2:]
print(sub_array)
```

```
[[1 2]
 [4 5]]
[[2 5 8]
 [5 6]
 [8 9]]
```

The `step` parameter can be used to select elements at regular intervals.

```
the step parameter in slicing
print(array[1:8:2])
print(array[::-2])
```

```
[20 40]
[50 30 10]
```

### 9.6.3 Combining Indexing and Slicing

You can combine indexing and slicing to extract specific elements or sub-arrays

```
Create a 3D array
array_3d = np.array([[[1, 2, 3], [4, 5, 6]], [[7, 8, 9], [10, 11, 12]]])

Select specific elements and slices
print(array_3d[0, :, 1]) # Output: [2 5] (second element from each row in the first sub-array)
print(array_3d[1, 1, :2]) # Output: [10 11] (first two elements in the last row of the second sub-array)
```

```
[2 5]
[10 11]
```

### 9.6.4 Boolean Mask: Conditional Selection

#### 9.6.4.1 Single Condition in NumPy

You can use **boolean arrays** to filter or select elements that meet a specific condition.

```
a = np.array([10, 20, 30, 40, 50])
mask = a > 25 # [False False True True True]
filtered = a[mask] # [30 40 50]
one-liner:
a[a > 25] # Output: [40 50]
```

```
array([30, 40, 50])
```

### 9.6.4.2 Combining Multiple Conditions in NumPy

Like in **pandas**, you can use **bitwise operators** with parentheses to combine multiple conditions:

& (AND), | (OR), ~ (NOT), ^ (XOR)

For example:

```
print(a[(a > 15) & (a < 45)])
print(a[(a < 20) | (a > 40)])
print(a[~(a % 20 == 0)])
```

```
[20 30 40]
```

```
[10 50]
```

```
[10 30 50]
```

### 9.6.5 Indexing & Slicing Quick Reference

| Operation         | Syntax                            | Example                      | Result             |
|-------------------|-----------------------------------|------------------------------|--------------------|
| Single element    | <code>arr[i]</code>               | <code>arr[2]</code>          | Element at index 2 |
| Negative indexing | <code>arr[-i]</code>              | <code>arr[-1]</code>         | Last element       |
| Basic slice       | <code>arr[start:stop]</code>      | <code>arr[1:4]</code>        | Elements 1, 2, 3   |
| Step slice        | <code>arr[start:stop:step]</code> | <code>arr[:,2]</code>        | Every 2nd element  |
| 2D indexing       | <code>arr[row, col]</code>        | <code>arr[0, 1]</code>       | Row 0, Column 1    |
| Boolean mask      | <code>arr[condition]</code>       | <code>arr[arr &gt; 5]</code> | Elements > 5       |

## 9.7 Advanced Selection Methods

### 9.7.1 Conditional Selection with `np.where()` and `np.select()`

Use these NumPy tools for fast, vectorized **if/else** logic on arrays—perfect for data cleaning, feature engineering, and conditional transforms.

**What you can do**

- **Replace values** based on a condition
- **Choose between arrays** element-wise

- Apply multi-way logic (if/elif/else) without loops

#### When to use which

- `np.where(condition, x, y)`: simple **two-way** branching (or `np.where(condition)` to get indices).
- `np.select([cond1, cond2, ...], [choice1, choice2, ...], default=...)`: clean **multi-branch** logic (3+ cases).

### 9.7.2 `np.where()`

- Two forms
  - `np.where(condition)` → returns the **indices** where `condition` is True.
  - `np.where(condition, x, y)` → **element-wise choose**: pick from `x` where `condition` is True, else from `y`.

```
Let's create sample data for our examples
ages = np.array([22, 25, 19, 35, 28, 24, 31, 26, 23, 29])
print(f"Student ages: {ages}")

scores = np.array([95, 87, 76, 63, 89, 92, 45, 78, 85, 91])
print(f"Student scores: {scores}")
```

```
Student ages: [22 25 19 35 28 24 31 26 23 29]
Student scores: [95 87 76 63 89 92 45 78 85 91]
```

```
print("=" * 50)
print(" USING np.where()")
print("=" * 50)

1) Simple conditional replacement: scores < 70 -> 0
pass_fail = np.where(scores >= 70, scores, 0)
print("\n1 Pass/Fail (70+ to pass):")
print(f"Original: {scores}")
print(f"Result: {pass_fail}")

2) Get indices (and values) of high scorers
high_idx = np.where(scores > 85)[0]
high_vals = scores[high_idx]
print("\n2 High Scorers (>85):")
print(f"Indices: {high_idx}")
print(f"Values : {high_vals}")
```

```
=====
USING np.where()
=====
```

```
1 Pass/Fail (70+ to pass):
Original: [95 87 76 63 89 92 45 78 85 91]
Result: [95 87 76 0 89 92 0 78 85 91]
```

```
2 High Scorers (>85):
Indices: [0 1 4 5 9]
Values : [95 87 89 92 91]
```

- **Chainable**

- You can **nest** multiple `np.where()` calls, but prefer `np.select` for **3+ branches**.

```
Conditional replacement with different values (letter grades)
letter_grades = np.where(scores >= 90, 'A',
 np.where(scores >= 80, 'B',
 np.where(scores >= 70, 'C',
 np.where(scores >= 60, 'D', 'F')))))

Example 3: Using np.where to find indices
print("\n3 Indices of A's and F's:")
a_idx = np.where(letter_grades == 'A')[0]
f_idx = np.where(letter_grades == 'F')[0]
print(f"A indices: {a_idx}")
print(f"F indices: {f_idx}")
```

```
3 Indices of A's and F's:
A indices: [0 5 9]
F indices: [6]
```

While you can nest multiple `np.where()` calls for complex conditions, `np.select()` is preferred for **3+ branches** because it's more readable and maintainable.

## 9.7.3 `np.select`: Multi-way Conditional Logic

### 9.7.3.1 Syntax

```
np.select(conditions, choices, default=...)
```

- **conditions**: list of boolean arrays (must be same shape or broadcastable)
- **choices**: list of result values/arrays (same length as **conditions**)
- **default**: value used where no condition is True (optional)

### 9.7.3.2 Example 1 — Letter grades (cleaner than nested `np.where()`)

```
scores = np.array([95, 87, 76, 63, 89, 92, 45, 78, 85, 91])
conds = [scores >= 90, scores >= 80, scores >= 70, scores >= 60]
choices = ['A', 'B', 'C', 'D']
grades = np.select(conds, choices, default='F')
print("\n4 Letter Grades with np.select():", grades)
print(f"Original Scores: {scores}")
```

```
4 Letter Grades with np.select(): ['A' 'B' 'C' 'D' 'B' 'A' 'F' 'C' 'B' 'A']
Original Scores: [95 87 76 63 89 92 45 78 85 91]
```

### 9.7.3.3 Example 2 — Numeric binning (labels)

```
x = np.array([5, 12, 20, 33, 47])
conds = [x < 10, (x >= 10) & (x < 30), x >= 30]
choices = ['low_precision', 'mid_precision', 'high_precision']
buckets = np.select(conds, choices, default='unknown') # ← Add this!

print(buckets)
```

```
['low_precision' 'mid_precision' 'mid_precision' 'high_precision'
 'high_precision']
```

#### `np.select` and the default dtype gotcha

If you omit `default=...` in `np.select`, NumPy uses `0` as the fallback. When your **choices** are **strings**, mixing them with the integer `0` forces a common dtype—strings and integers have no shared dtype—so NumPy raises a `TypeError`.

**Example (raises error)**



```
import numpy as np

x = np.array([5, 12, 30])
conds = [x < 10, (x >= 10) & (x < 20)]
choices = ['low', 'mid']

np.select(conds, choices) # default is 0 -> TypeError (mixed str + int)
```

### Best Practices:

- Use `np.where()` for simple binary conditions
- Use `np.select()` for 3+ conditions or complex logic
- **Always provide a default** to handle edge cases
- **Test conditions** to ensure they're mutually exclusive when needed

### 9.7.4 Finding Extremes with `np.argmin()` and `np.argmax()`

Use these to get the **index position(s)** of the minimum/maximum value. Specify an **axis** for row/column results; omit it for the global position.

```
a = np.array([4, 1, 9, 7])
i_min = np.argmin(a) # 1
i_max = np.argmax(a) # 2

print("min @ index", i_min, "value:", a[i_min])
print("max @ index", i_max, "value:", a[i_max])
```

```
min @ index 1 value: 1
max @ index 2 value: 9
```

```
Create a 2D array (e.g., student test scores)
scores = np.array([
 [85, 92, 78, 95], # Student 0
 [88, 76, 91, 82], # Student 1
 [95, 89, 84, 90], # Student 2
 [72, 85, 79, 88] # Student 3
])

print(" Student Test Scores (4 students × 4 tests):")
print(scores)
```

```

print()

=====
1 FLATTENED (no axis) - Returns single index
=====
print("=" * 60)
print("1 FLATTENED VIEW (axis=None) - Single Index")
print("=" * 60)

min_idx = np.argmin(scores)
max_idx = np.argmax(scores)

print(f"Lowest score index (flattened): {min_idx}")
print(f"Highest score index (flattened): {max_idx}")

Convert flat index to 2D coordinates
min_row, min_col = np.unravel_index(min_idx, scores.shape)
max_row, max_col = np.unravel_index(max_idx, scores.shape)

print(f"\n Minimum: {scores[min_row, min_col]} at position ({min_row}, {min_col})")
print(f" → Student {min_row}, Test {min_col}")
print(f" Maximum: {scores[max_row, max_col]} at position ({max_row}, {max_col})")
print(f" → Student {max_row}, Test {max_col}")
print()

```

```

Student Test Scores (4 students × 4 tests):
[[85 92 78 95]
 [88 76 91 82]
 [95 89 84 90]
 [72 85 79 88]]

```

```

=====
1 FLATTENED VIEW (axis=None) - Single Index
=====

```

```

Lowest score index (flattened): 12
Highest score index (flattened): 3

```

```

Minimum: 72 at position (3, 0)
 → Student 3, Test 0
Maximum: 95 at position (0, 3)
 → Student 0, Test 3

```

```

=====
2 AXIS=0 (down columns) - Compare students
=====
print("=" * 60)
print("2 AXIS=0 (down columns) - Which STUDENT performed best/worst?")
print("=" * 60)

min_student_per_test = np.argmin(scores, axis=0)
max_student_per_test = np.argmax(scores, axis=0)

print(f"Lowest scoring student per test: {min_student_per_test}")
print(f"Highest scoring student per test: {max_student_per_test}")

print("\nDetailed breakdown:")
for test_num in range(scores.shape[1]):
 worst_student = min_student_per_test[test_num]
 best_student = max_student_per_test[test_num]
 print(f" Test {test_num}: "
 f"Worst = Student {worst_student} ({scores[worst_student, test_num]}), "
 f"Best = Student {best_student} ({scores[best_student, test_num]})")
print()
=====
3 AXIS=1 (across rows) - Compare tests
=====
print("=" * 60)
print("3 AXIS=1 (across rows) - Which TEST was easiest/hardest?")
print("=" * 60)

min_test_per_student = np.argmin(scores, axis=1)
max_test_per_student = np.argmax(scores, axis=1)

print(f"Worst test per student: {min_test_per_student}")
print(f"Best test per student: {max_test_per_student}")

print("\nDetailed breakdown:")
for student_num in range(scores.shape[0]):
 worst_test = min_test_per_student[student_num]
 best_test = max_test_per_student[student_num]
 print(f" Student {student_num}: "
 f"Worst = Test {worst_test} ({scores[student_num, worst_test]}), "
 f"Best = Test {best_test} ({scores[student_num, best_test]})")
print()

```

```
=====
2 AXIS=0 (down columns) - Which STUDENT performed best/worst?
=====
```

```
Lowest scoring student per test: [3 1 0 1]
```

```
Highest scoring student per test: [2 0 1 0]
```

Detailed breakdown:

```
Test 0: Worst = Student 3 (72), Best = Student 2 (95)
```

```
Test 1: Worst = Student 1 (76), Best = Student 0 (92)
```

```
Test 2: Worst = Student 0 (78), Best = Student 1 (91)
```

```
Test 3: Worst = Student 1 (82), Best = Student 0 (95)
```

```
=====
3 AXIS=1 (across rows) - Which TEST was easiest/hardest?
=====
```

```
Worst test per student: [2 1 2 0]
```

```
Best test per student: [3 2 0 3]
```

Detailed breakdown:

```
Student 0: Worst = Test 2 (78), Best = Test 3 (95)
```

```
Student 1: Worst = Test 1 (76), Best = Test 2 (91)
```

```
Student 2: Worst = Test 2 (84), Best = Test 0 (95)
```

```
Student 3: Worst = Test 0 (72), Best = Test 3 (88)
```

## Tips

- `argmin/argmax` return **indices**, not values (use them to index back into the array).
- **Axis behavior**
  - `axis=None` (default): flattens the entire array and returns the **global** index.
  - `axis=0` (2D): compares **down the rows** within each column → returns **row indices per column**.
  - `axis=1` (2D): compares **across columns** within each row → returns **column indices per row**.
  - General ND: reduces along the specified axis; the **result shape equals the input shape with that axis removed**.
- For values at those positions:
  - Column-wise: `i = np.argmax(M, axis=0); vals = M[i, np.arange(M.shape[1])]`
  - Row-wise: `j = np.argmax(M, axis=1); vals = M[np.arange(M.shape[0]), j]`
- Convert a flat index to coordinates with `np.unravel_index(idx, arr.shape)`.

### 9.7.5 Finding the Top- $n$ with `np.argsort()`

`np.argsort()` returns **indices in ascending order** by default.

It has **no built-in descending** option (no `order/ascending/reverse` params).

You can still get the indices (and values) of the largest/smallest  $n$  elements:

#### 9.7.5.1 Descending order (two common workarounds)

- Method 1: Reverse the result (most common) `python` `indices_desc = np.argsort(arr)[::-1]`
- Method 2: Negate the array (for numeric data) `python` `indices_desc = np.argsort(-arr)`

#### 9.7.5.2 1D arrays

```
a = np.array([3, 10, 7, 10])
k = 2

Indices that would sort ascending
idx_asc = np.argsort(a) # e.g., [0, 2, 1, 3]
print("Indices to sort ascending:", idx_asc)

top=k indices (ascending by value)
topk_idx_asc = idx_asc[:k] # e.g., [0, 2]
print(f"Top-{k} indices (ascending by value):", topk_idx_asc)

the values corresponding to the top-k indices (ascending)
topk_vals_asc = a[topk_idx_asc]
print(f"Top-{k} values (ascending by value):", topk_vals_asc)

Top-k indices (descending by value)
topk_idx_des = idx_asc[-k:][::-1] # e.g., [3, 1]
print(f"Top-{k} indices (descending by value):", topk_idx_des)

the values corresponding to the top-k indices (descending)
topk_vals_des = a[topk_idx_des]
print(f"Top-{k} values (descending by value):", topk_vals_des)
```

Indices to sort ascending: [0 2 1 3]  
Top-2 indices (ascending by value): [0 2]  
Top-2 values (ascending by value): [3 7]  
Top-2 indices (descending by value): [3 1]  
Top-2 values (descending by value): [10 10]

Shortcut (numeric arrays):

```
topk_idx = np.argsort(-a)[:k] # also descending indices
topk_vals = a[topk_idx]
print(topk_idx)
print(topk_vals)
```

```
[1 3]
[10 10]
```

### 9.7.5.3 2D arrays (row-wise or column-wise)

```
A = np.array([[3, 7, 2],
 [5, 1, 9]])
k = 2

Row-wise: get the top-k column indices for each row
row_idx_sorted = np.argsort(A, axis=1) # shape (n_rows, n_cols)
topk_cols_per_row = row_idx_sorted[:, -k:][:, ::-1]

Column-wise: get the top-k row indices for each column
col_idx_sorted = np.argsort(A, axis=0)
topk_rows_per_col = col_idx_sorted[-k:, :][::-1, :]
```

To retrieve the values:

```
Row-wise values (gather with advanced indexing)
rows = np.arange(A.shape[0])[:, None] # column vector of row indices
topk_vals_per_row = A[rows, topk_cols_per_row]
print("Top-k values per row:\n", topk_vals_per_row)

Column-wise values
cols = np.arange(A.shape[1])[None, :]
topk_vals_per_col = A[topk_rows_per_col, cols]
print("Top-k values per column:\n", topk_vals_per_col)
```

Top-k values per row:

```
[[7 3]
 [9 5]]
```

Top-k values per column:

```
[[5 7 9]
 [3 1 2]]
```

## 9.8 Array Operations: Mathematical Power at Scale

NumPy arrays excel at **vectorized operations** that process entire arrays efficiently. These operations are the foundation of scientific computing and data analysis.

### 9.8.1 Arithmetic Operations

NumPy arrays support all standard arithmetic operators (+, -, \*, /, \*\*, %) and apply them **element-wise**. You can perform operations between:

- **Array and Array** → Operands must have **compatible shapes** (same shape or broadcastable)
- **Array and Vector** → The vector is **broadcast across rows or columns** depending on its shape, enabling **element-wise operations**
- **Array and Scalar** → The scalar is applied to **every element** of the array (**broadcasting**)

**Key Advantage:** These operations are **vectorized** - they execute at C speed, not Python loop speed!

#### 9.8.1.1 Array and Array

Let's explore **arithmetic operations between two arrays** through practical examples:

```
Sample Data: Sales figures for 3 stores × 4 quarters
store_sales_q1_q4 = np.array([[120, 150, 180, 200], # Store A
 [100, 130, 160, 190], # Store B
 [140, 110, 150, 170]]) # Store C

Bonus amounts for each store and quarter
bonus_amounts = np.array([[10, 15, 20, 25],
 [12, 18, 22, 28],
```

```

 [8, 12, 18, 22]])

print(" Quarterly Sales (in thousands):")
print("Store Q1 Q2 Q3 Q4")
print("A ", store_sales_q1_q4[0])
print("B ", store_sales_q1_q4[1])
print("C ", store_sales_q1_q4[2])

print("\n Quarterly Bonuses (in thousands):")
print("Store Q1 Q2 Q3 Q4")
print("A ", bonus_amounts[0])
print("B ", bonus_amounts[1])
print("C ", bonus_amounts[2])

For compatibility with existing examples
arr1, arr2 = store_sales_q1_q4, bonus_amounts

```

```

Quarterly Sales (in thousands):
Store Q1 Q2 Q3 Q4
A [120 150 180 200]
B [100 130 160 190]
C [140 110 150 170]

```

```

Quarterly Bonuses (in thousands):
Store Q1 Q2 Q3 Q4
A [10 15 20 25]
B [12 18 22 28]
C [8 12 18 22]

```

```

#Element-wise summation of arrays
arr1 + arr2

```

```

array([[130, 165, 200, 225],
 [112, 148, 182, 218],
 [148, 122, 168, 192]])

```

```

Element-wise subtraction
arr2 - arr1

```

```

array([[-110, -135, -160, -175],
 [-88, -112, -138, -162],
 [-132, -98, -132, -148]])

```



```
Element-wise multiplication
arr1 * arr2
```

```
array([[1200, 2250, 3600, 5000],
 [1200, 2340, 3520, 5320],
 [1120, 1320, 2700, 3740]])
```

### 9.8.1.2 Broadcasting: The Heart of NumPy Vectorization

Besides supporting arithmetic operations between arrays, **broadcasting** is NumPy's powerful mechanism that enables operations between arrays of different shapes **without explicitly reshaping them**.

- **Array and Vector** → The vector is **broadcast across rows or columns** depending on its shape, enabling **element-wise operations**
- **Array and Scalar** → The scalar is applied to **every element** of the array (**broadcasting**)

Broadcasting is the foundation that makes **vectorization** in NumPy so flexible and intuitive.

#### 9.8.1.2.1 Broadcasting Rules

NumPy broadcasting follows these rules:

1. **Start from the trailing dimension** and work backwards
2. **Dimensions are compatible** if:
  - They are equal, OR
  - One of them is 1, OR
  - One of them doesn't exist (missing dimension)
3. **Missing dimensions** are assumed to be size 1

```
Broadcasting Rules Demonstration

def check_broadcast_compatibility(shape1, shape2):
 """
 Check if two shapes are compatible for broadcasting
 """
 # Pad shorter shape with 1s on the left
```

```

 ndim1, ndim2 = len(shape1), len(shape2)
 max_ndim = max(ndim1, ndim2)

 shape1_padded = [1] * (max_ndim - ndim1) + list(shape1)
 shape2_padded = [1] * (max_ndim - ndim2) + list(shape2)

 result_shape = []
 compatible = True

 for i in range(max_ndim):
 dim1, dim2 = shape1_padded[i], shape2_padded[i]
 if dim1 == dim2:
 result_shape.append(dim1)
 elif dim1 == 1:
 result_shape.append(dim2)
 elif dim2 == 1:
 result_shape.append(dim1)
 else:
 compatible = False
 break

 return compatible, tuple(result_shape) if compatible else None

Test different shape combinations
test_cases = [
 ((3, 4), (4,)), # Compatible
 ((3, 4), (3, 1)), # Compatible
 ((3, 4), (2,)), # Incompatible
 ((2, 3, 4), (4,)), # Compatible
 ((2, 3, 4), (3, 4)), # Compatible
 ((2, 3, 4), (2, 1, 4)), # Compatible
 ((3, 4), (2, 3)), # Incompatible
]

print("Broadcasting Compatibility Check:")
print("-" * 50)
for shape1, shape2 in test_cases:
 compatible, result_shape = check_broadcast_compatibility(shape1, shape2)
 status = " Compatible" if compatible else " Incompatible"
 if compatible:
 print(f"{str(shape1):>10} + {str(shape2):>10} → {str(result_shape):>12} {status}")
 else:

```

```
print(f"{str(shape1):>10} + {str(shape2):>10} → {'N/A':>12} {status}")
```

#### Broadcasting Compatibility Check:

```

(3, 4) + (4,) → (3, 4) Compatible
(3, 4) + (3, 1) → (3, 4) Compatible
(3, 4) + (2,) → N/A Incompatible
(2, 3, 4) + (4,) → (2, 3, 4) Compatible
(2, 3, 4) + (3, 4) → (2, 3, 4) Compatible
(2, 3, 4) + (2, 1, 4) → (2, 3, 4) Compatible
(3, 4) + (2, 3) → N/A Incompatible
```

#### Comprehensive Broadcasting Examples:

```
print("=== BROADCASTING EXAMPLES ===\n")

Example 1: 2D array + 1D array (most common case)
arr_2d = np.array([[1, 2, 3, 4],
 [5, 6, 7, 8],
 [9, 10, 11, 12]])

arr_1d = np.array([10, 20, 30, 40])

print("Example 1: (3,4) + (4,) broadcasting")
print(f"2D array shape: {arr_2d.shape}")
print(f"1D array shape: {arr_1d.shape}")
result_1 = arr_2d + arr_1d
print(f"Result shape: {result_1.shape}")
print("Result:")
print(result_1)
print()

Example 2: Broadcasting with different orientations
arr_col = np.array([[100],
 [200],
 [300]]) # Column vector (3,1)

print("Example 2: (3,4) + (3,1) broadcasting")
print(f"2D array shape: {arr_2d.shape}")
print(f"Column vector shape: {arr_col.shape}")
result_2 = arr_2d + arr_col
```

```

print(f"Result shape: {result_2.shape}")
print("Result:")
print(result_2)
print()

Example 3: Scalar broadcasting (always works)
scalar = 1000
print("Example 3: (3,4) + scalar broadcasting")
result_3 = arr_2d + scalar
print(f"Result shape: {result_3.shape}")
print("Result:")
print(result_3)
print()

Example 4: 3D broadcasting
arr_3d = np.random.randint(1, 10, (2, 3, 4))
arr_2d_broadcast = np.array([[1, 2, 3, 4]]) # (1, 4)

print("Example 4: (2,3,4) + (1,4) broadcasting")
print(f"3D array shape: {arr_3d.shape}")
print(f"2D array shape: {arr_2d_broadcast.shape}")
result_4 = arr_3d + arr_2d_broadcast
print(f"Result shape: {result_4.shape}")
print("Broadcasting successful!")
print()

```

=== BROADCASTING EXAMPLES ===

Example 1: (3,4) + (4,) broadcasting

2D array shape: (3, 4)

1D array shape: (4,)

Result shape: (3, 4)

Result:

```

[[11 22 33 44]
 [15 26 37 48]
 [19 30 41 52]]

```

Example 2: (3,4) + (3,1) broadcasting

2D array shape: (3, 4)

Column vector shape: (3, 1)

Result shape: (3, 4)

Result:

```
[[101 102 103 104]
 [205 206 207 208]
 [309 310 311 312]]
```

Example 3: (3,4) + scalar broadcasting

Result shape: (3, 4)

Result:

```
[[1001 1002 1003 1004]
 [1005 1006 1007 1008]
 [1009 1010 1011 1012]]
```

Example 4: (2,3,4) + (1,4) broadcasting

3D array shape: (2, 3, 4)

2D array shape: (1, 4)

Result shape: (2, 3, 4)

Broadcasting successful!

**Invalid operations** (will raise ValueError)

```
Invalid operations (will raise ValueError)
try:
 a = np.ones((3, 4))
 b = np.ones((3, 3))
 result = a + b
except ValueError as e:
 print(f"Error: {e}")

try:
 a = np.ones((2, 3))
 b = np.ones((2, 1, 1))
 result = a + b
except ValueError as e:
 print(f"Error: {e}")
```

Error: operands could not be broadcast together with shapes (3,4) (3,3)

In the above example, `a + b` cannot be broadcast together and evaluated successfully. See the [broadcasting documentation](#) to learn more about it.

## 9.8.2 Aggregate Functions: Statistical Summaries

Aggregate functions reduce array dimensions by applying statistical operations. The `axis` parameter controls the direction of aggregation.

### 9.8.2.1 Global Aggregation (All Elements)

| Function                  | Purpose                 | Example                                    |
|---------------------------|-------------------------|--------------------------------------------|
| <code>np.sum(arr)</code>  | Sum all elements        | <code>np.sum([[1,2],[3,4]])</code> → 10    |
| <code>np.mean(arr)</code> | Average of all elements | <code>np.mean([[1,2],[3,4]])</code> → 2.5  |
| <code>np.min(arr)</code>  | Minimum value           | <code>np.min([[1,2],[3,4]])</code> → 1     |
| <code>np.max(arr)</code>  | Maximum value           | <code>np.max([[1,2],[3,4]])</code> → 4     |
| <code>np.std(arr)</code>  | Standard deviation      | <code>np.std([[1,2],[3,4]])</code> → 1.118 |

### 9.8.2.2 Axis-Specific Aggregation

Understanding Axes in 2D Arrays:

- **axis=0: Down the rows** (column-wise aggregation) → Result has shape (n\_cols,)
- **axis=1: Across the columns** (row-wise aggregation) → Result has shape (n\_rows,)

**Memory Tip:** “Axis 0 goes down, Axis 1 goes across”

```
Student grades: 3 students × 4 subjects (Math, Science, English, History)
Create sample data for demonstration
array = np.array([[4, 7, 1, 3],
 [5, 8, 2, 6],
 [9, 3, 5, 2]])

Display the original array
print("Original Array:\n", array)

Calculate the sum, mean, minimum, and maximum for the entire array
total_sum = np.sum(array)
mean_value = np.mean(array)
```

```

min_value = np.min(array)
max_value = np.max(array)

print(f"\nSum of all elements: {total_sum}")
print(f"Mean of all elements: {mean_value}")
print(f"Minimum value in the array: {min_value}")
print(f"Maximum value in the array: {max_value}")

Calculate the sum, mean, minimum, and maximum along each row (axis=1)
row_sum = np.sum(array, axis=1)
row_mean = np.mean(array, axis=1)
row_min = np.min(array, axis=1)
row_max = np.max(array, axis=1)

print("\nSum along each row:", row_sum)
print("Mean along each row:", row_mean)
print("Minimum value along each row:", row_min)
print("Maximum value along each row:", row_max)

Calculate the sum, mean, minimum, and maximum along each column (axis=0)
col_sum = np.sum(array, axis=0)
col_mean = np.mean(array, axis=0)
col_min = np.min(array, axis=0)
col_max = np.max(array, axis=0)

print("\nSum along each column:", col_sum)
print("Mean along each column:", col_mean)
print("Minimum value along each column:", col_min)
print("Maximum value along each column:", col_max)

```

Original Array:

```

[[4 7 1 3]
 [5 8 2 6]
 [9 3 5 2]]

```

Sum of all elements: 55

Mean of all elements: 4.583333333333333

Minimum value in the array: 1

Maximum value in the array: 9

Sum along each row: [15 21 19]

Mean along each row: [3.75 5.25 4.75]

```

Minimum value along each row: [1 2 2]
Maximum value along each row: [7 8 9]

Sum along each column: [18 18 8 11]
Mean along each column: [6. 6. 2.66666667 3.66666667]
Minimum value along each column: [4 3 1 2]
Maximum value along each column: [9 8 5 6]

```

## 9.9 Array Reshaping: Transforming Data Dimensions

**Array reshaping** is essential for preparing data for different computational tasks. Many operations require specific array shapes:

- **Machine Learning:** Models often expect specific input dimensions (e.g., 2D for tabular data, 4D for images)
- **Matrix Operations:** Linear algebra operations require compatible shapes for multiplication
- **Data Analysis:** Different analysis techniques may need data in specific formats
- **Broadcasting:** Reshaping enables efficient element-wise operations between arrays

**Key Insight:** Reshaping **never changes the data**—only how it’s organized in memory!

### 9.9.1 Core Reshaping Methods

#### 9.9.1.1 `reshape()`: Intelligent Dimension Control

The **`reshape()`** method creates a **new view** of the array with different dimensions. The total number of elements must remain the same.

**Syntax:** `array.reshape(new_shape)` or `np.reshape(array, new_shape)`

```

print(" RESHAPE() DEMONSTRATIONS")
print("=" * 40)

Original 1D array (12 elements)
original = np.arange(1, 13)
print(f"Original (1D): {original}")
print(f"Original shape: {original.shape}")

```



```

Reshape to 3D array (2 × 2 × 3)
array_3d = original.reshape(2, 2, 3)
print("\n3D Array (2×2×3):")
print(array_3d)
print(f"Shape: {array_3d.shape}")

Reshape to 2D matrices
matrix_3x4 = original.reshape(3, 4)
print("\nMatrix (3×4):")
print(matrix_3x4)
print(f"Shape: {matrix_3x4.shape}")

matrix_4x3 = original.reshape(4, 3)
print("\nMatrix (4×3):")
print(matrix_4x3)
print(f"Shape: {matrix_4x3.shape}")

matrix_2x6 = original.reshape(2, 6)
print("\nMatrix (2×6):")
print(matrix_2x6)
print(f"Shape: {matrix_2x6.shape}")

```

#### RESHAPE() DEMONSTRATIONS

```

=====
Original (1D): [1 2 3 4 5 6 7 8 9 10 11 12]
Original shape: (12,)

3D Array (2×2×3):
[[[1 2 3]
 [4 5 6]]

 [[7 8 9]
 [10 11 12]]]
Shape: (2, 2, 3)

Matrix (3×4):
[[1 2 3 4]
 [5 6 7 8]
 [9 10 11 12]]
Shape: (3, 4)

Matrix (4×3):

```

```
[[1 2 3]
 [4 5 6]
 [7 8 9]
 [10 11 12]]
Shape: (4, 3)
```

```
Matrix (2×6):
[[1 2 3 4 5 6]
 [7 8 9 10 11 12]]
Shape: (2, 6)
```

### 9.9.1.2 Smart Dimension Inference with -1

Use -1 to let NumPy **automatically calculate** one dimension based on the array size and other specified dimensions.

```
Automatic Dimension Calculation (-1)

data = np.arange(24) # 24 elements
print(f"Original data: {data}")
print(f"Total elements: {data.size}")

Use -1 to let NumPy infer exactly one dimension (only one -1 per reshape)

4 rows, infer columns -> 4×6
result_1 = data.reshape(4, -1)

infer rows, 3 columns -> 8×3
result_2 = data.reshape(-1, 3)

2 × 3 × ? -> 2×3×4
result_3 = data.reshape(2, 3, -1)

print("\n4×? becomes: 4×{} = {}".format(result_1.shape[1], result_1.shape))
print(result_1)

print("\n?×3 becomes: {}×3 = {}".format(result_2.shape[0], result_2.shape))
print(result_2)

print("\n2×3×? becomes: 2×3×{} = {}".format(result_3.shape[2], result_3.shape))
print(result_3)
```

```
Original data: [0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23]
Total elements: 24
```

```
4×? becomes: 4×6 = (4, 6)
```

```
[[0 1 2 3 4 5]
 [6 7 8 9 10 11]
 [12 13 14 15 16 17]
 [18 19 20 21 22 23]]
```

```
?×3 becomes: 8×3 = (8, 3)
```

```
[[0 1 2]
 [3 4 5]
 [6 7 8]
 [9 10 11]
 [12 13 14]
 [15 16 17]
 [18 19 20]
 [21 22 23]]
```

```
2×3×? becomes: 2×3×4 = (2, 3, 4)
```

```
[[[0 1 2 3]
 [4 5 6 7]
 [8 9 10 11]]
 [[12 13 14 15]
 [16 17 18 19]
 [20 21 22 23]]]
```

**How -1 Works:** NumPy calculates the missing dimension using the formula:

```
missing_dimension = total_elements ÷ (product_of_known_dimensions)
```

**Important:** You can only use -1 for **one dimension** per reshape operation!

### 9.9.1.3 flatten() vs ravel(): Converting to 1D

Both convert an array to 1D, but they differ in **copy vs view**, **speed**, and **memory order** handling.

| Method                 | Returns       | Copy/View                                            | Speed<br>(typical) | When to Use                                                                       |
|------------------------|---------------|------------------------------------------------------|--------------------|-----------------------------------------------------------------------------------|
| <code>flatten()</code> | 1D<br>ndarray | <b>Always makes a copy</b>                           | Slower             | You want an independent array you can modify safely                               |
| <code>ravel()</code>   | 1D<br>ndarray | <b>View if possible</b> , else copy (not guaranteed) | Faster             | You want a 1D view for efficiency (and accept that edits may affect the original) |

**Reading order options** (both functions support `order=`): - **'C' (default)**: Row-major (left→right, top→bottom) - **'F'**: Column-major (top→bottom, left→right) - **'A'**: 'F' if the array is Fortran-contiguous, otherwise 'C' - **'K'**: As close as possible to the array's **in-memory layout** (preserves current strides, useful after slicing)

**Gotcha:** `ravel()` may return a **copy** if a view isn't possible (e.g., non-contiguous slices or mismatched `order`). If you must ensure independence, use `flatten()` or call `.copy()` on the result of `ravel()`.

```
print(" FLATTEN() vs RAVEL() COMPARISON")
print("=" * 45)

Original 2D array
original_2d = np.array([[1, 2, 3],
 [4, 5, 6],
 [7, 8, 9]])

print("Original array:")
print(original_2d)

Method 1: flatten() - always returns a COPY
flat_copy = original_2d.flatten()

Method 2: ravel() - returns a VIEW when possible (else a copy)
flat_view = original_2d.ravel()

print("\nResults:")
print(f"flatten() result: {flat_copy}")
print(f"ravel() result: {flat_view}")

print("\nMemory sharing checks:")
print(f"flatten() shares memory with original? {np.shares_memory(original_2d, flat_copy)}")
print(f"ravel() shares memory with original? {np.shares_memory(original_2d, flat_view)}")
```

## FLATTEN() vs RAVEL() COMPARISON

=====

Original array:

```
[[1 2 3]
 [4 5 6]
 [7 8 9]]
```

Results:

flatten() result: [1 2 3 4 5 6 7 8 9]

ravel() result: [1 2 3 4 5 6 7 8 9]

Memory sharing checks:

flatten() shares memory with original? False

ravel() shares memory with original? True

```
from time import perf_counter

print("\n READING ORDER DEMONSTRATIONS")
print("=" * 40)

Small example matrix
matrix = np.array([[1, 2, 3],
 [4, 5, 6]])
print(f"Matrix:\n{matrix}")

Reading order: C (row-major) vs F (column-major)
c_order = matrix.flatten(order='C') # left→right, top→bottom
f_order = matrix.flatten(order='F') # top→bottom, left→right

print(f"\nC order (row-major): {c_order}") # [1 2 3 4 5 6]
print(f"F order (col-major): {f_order}") # [1 4 2 5 3 6]

(Optional) ravel with order behaves similarly:
print(matrix.ravel(order='C'))
print(matrix.ravel(order='F'))

print("\n Performance Test (1000×1000 array):")
large_array = np.random.rand(1000, 1000)

Simple timing helper: take the best of a few runs to reduce noise
def best_time(fn, reps=5):
 times = []
```

```

 for _ in range(reps):
 t0 = perf_counter()
 _ = fn()
 times.append(perf_counter() - t0)
 return min(times)

flatten_time = best_time(lambda: large_array.flatten(order='K'))
ravel_time = best_time(lambda: large_array.ravel(order='K'))

print(f"flatten(): {flatten_time:.6f} seconds")
print(f"ravel() : {ravel_time:.6f} seconds")
if ravel_time > 0:
 print(f"ravel() is ~{flatten_time / ravel_time:.2f}× {'slower' if flatten_time > ravel_t")

```

#### READING ORDER DEMONSTRATIONS

=====

Matrix:

```
[[1 2 3]
 [4 5 6]]
```

C order (row-major): [1 2 3 4 5 6]

F order (col-major): [1 4 2 5 3 6]

Performance Test (1000×1000 array):

flatten(): 0.001275 seconds

ravel() : 0.000000 seconds

ravel() is ~6378.81× slower than flatten()

#### 9.9.1.4 `resize()` — Destructive Reshaping with Size Changes

`ndarray.resize(new_shape, refcheck=True)` changes an array **in place**. It's powerful, but risky:

- **Can change the total number of elements** (unlike `reshape`)
- **Shrinking truncates data**
- **Expanding fills with zeros** (for numeric dtypes)
- **In-place / destructive:** permanently modifies the original array
- **Safety check:** raises `ValueError` if the array **references/is referenced** (use `refcheck=False` to force — unsafe)

- Don't confuse with `np.resize(a, new_shape)` which **returns a new array** and **repeats elements** to fill

```
print(" RESIZE() OPERATIONS")
print("=" * 30)

Example 1: Expanding (pads with zeros)

array1 = np.array([1, 2, 3, 4], dtype=int)
original_id = id(array1)

print("\n[Example 1] Expand array in-place to shape (2, 4)")
print(f"Before resize: {array1} | shape={array1.shape} | id={original_id}")

array1.resize(2, 4) # 4 -> 8 elements; pads with zeros for numeric dtypes
print(f"After resize: \n{array1} | shape={array1.shape} | id={id(array1)}")
print(f"Same memory object? {id(array1) == original_id}") # True

Example 2: Shrinking (truncates data)

array2 = np.array([[10, 20, 30, 40],
 [50, 60, 70, 80]], dtype=int)

print("\n[Example 2] Shrink array in-place to shape (1, 3) - data is truncated")
print(f"Before shrinking:\n{array2} | shape={array2.shape}")

array2.resize(1, 3) # 8 -> 3 elements; truncates
print(f"After resize:\n{array2} | shape={array2.shape}")

Example 3: Safe alternative - reshape (non-destructive)

array3 = np.array([1, 2, 3, 4, 5, 6], dtype=int)
print("\n[Example 3] Using reshape (safe, returns a new view/copy without changing size)")
print(f"Original: {array3} | shape={array3.shape} | id={id(array3)}")

Incompatible reshape (different element count) -> raises ValueError
try:
 reshaped = array3.reshape(2, 4) # 6 -> 8 elements (invalid)
except ValueError as e:
 print(f"reshape(2, 4) error: {e}")
```

```

Compatible reshape
safe_reshape = array3.reshape(2, 3) # 6 -> 6 elements (valid)
print(f"Safe reshape (2, 3):\n{safe_reshape} | shape={safe_reshape.shape}")
print(f"Original unchanged: {array3} | shape={array3.shape} | id={id(array3)}")

Bonus: np.resize (returns NEW array, repeats elements if needed)

a = np.array([1, 2, 3])
print("\n[Bonus] np.resize returns a new array (does not modify the original)")
print(f"Original a: {a} | id={id(a)}")
b = np.resize(a, (2, 5)) # repeats [1,2,3,1,2] in each row to fill
print(f"np.resize(a, (2,5)):\n{b} | id={id(b)}")
print(f"Original a unchanged: {a}")

```

## RESIZE() OPERATIONS

=====

[Example 1] Expand array in-place to shape (2, 4)  
Before resize: [1 2 3 4] | shape=(4,) | id=2948081772336  
After resize:  
[[1 2 3 4]  
[0 0 0 0]] | shape=(2, 4) | id=2948081772336  
Same memory object? True

[Example 2] Shrink array in-place to shape (1, 3) - data is truncated  
Before shrinking:  
[[10 20 30 40]  
[50 60 70 80]] | shape=(2, 4)  
After resize:  
[[10 20 30]] | shape=(1, 3)

[Example 3] Using reshape (safe, returns a new view/copy without changing size)  
Original: [1 2 3 4 5 6] | shape=(6,) | id=2948081772528  
reshape(2, 4) error: cannot reshape array of size 6 into shape (2,4)  
Safe reshape (2, 3):  
[[1 2 3]  
[4 5 6]] | shape=(2, 3)  
Original unchanged: [1 2 3 4 5 6] | shape=(6,) | id=2948081772528

[Bonus] np.resize returns a new array (does not modify the original)



```
Original a: [1 2 3] | id=2948081773392
np.resize(a, (2,5)):
[[1 2 3 1 2]
 [3 1 2 3 1]] | id=2948081763024
Original a unchanged: [1 2 3]
```

### 9.9.1.5 transpose() and .T: Matrix Transposition

**Transposition** flips an array along its diagonal—**rows become columns** (and vice versa). It's fundamental for linear algebra and data manipulation.

#### Ways to transpose

- **.T**: Shorthand for transposing a 2D array. For N-D arrays, it **reverses the axis order**.
- **np.transpose(a, axes=None)**: Function form. With **axes**, you can specify any axis permutation.
- **a.transpose(\*axes)**: Method form (equivalent to **np.transpose(a, axes=...)**).

All of the above return a **view** (no copy) when possible.

#### Common applications

- **Matrix multiplication**: Make shapes compatible (e.g., use **A.T @ B** when A is (m, n) and B is (n, k)).
- **Data analysis**: Switch between row-major and column-major organization.
- **Neural networks**: Align weight tensors and activations.
- **Statistics**: Work with covariance/correlation matrices.

```
print(" TRANSPOSE OPERATIONS")
print("=" * 30)

2D Matrix transposition

matrix = np.array([[1, 2, 3, 4],
 [5, 6, 7, 8]])
print(f"Original (2x4):\n{matrix}")
print(f"Shape: {matrix.shape}")

Three equivalent ways to transpose
method1 = matrix.T
method2 = matrix.transpose()
method3 = np.transpose(matrix)
```

```

print("\nTransposed (4×2) - All methods identical:")
print(method1)
print(f"Shape: {method1.shape}")
print(f"All equal? {np.array_equal(method1, method2) and np.array_equal(method2, method3)}")

Matrix multiplication example

print("\n Matrix Multiplication Example:")
A = np.array([[1, 2],
 [3, 4]])
B = np.array([[5, 6],
 [7, 8]])

print(f"A (2×2): \n{A}")
print(f"B (2×2): \n{B}")
print(f"A × B:\n{A @ B}")
print(f"A × B.T:\n{A @ B.T}")
print(f"A.T × B:\n{A.T @ B}")

Real-world example: Data conversion (students subjects)

print("\n PRACTICAL EXAMPLE: Student Grades")

students = ["Alice", "Bob", "Charlie"]
subjects = ["Math", "Stats", "CS"]

rows = students, cols = subjects
students_data = np.array([
 [85, 92, 78], # Alice
 [88, 76, 90], # Bob
 [91, 89, 84], # Charlie
])

print("By Students (rows = students):")
for i, student in enumerate(students):
 print(f"{student}: {students_data[i]}")

Transpose to organize by subject (rows)
subjects_data = students_data.T

```

```

print("\nBy Subjects (rows = subjects):")
for i, subject in enumerate(subjects):
 print(f"{subject}: {subjects_data[i]}")

print("=" * 40)

```

#### TRANSPOSE OPERATIONS

=====

Original (2×4):

```

[[1 2 3 4]
 [5 6 7 8]]

```

Shape: (2, 4)

Transposed (4×2) - All methods identical:

```

[[1 5]
 [2 6]
 [3 7]
 [4 8]]

```

Shape: (4, 2)

All equal? True

#### Matrix Multiplication Example:

A (2×2):

```

[[1 2]
 [3 4]]

```

B (2×2):

```

[[5 6]
 [7 8]]

```

A × B:

```

[[19 22]
 [43 50]]

```

A × B.T:

```

[[17 23]
 [39 53]]

```

A.T × B:

```

[[26 30]
 [38 44]]

```

#### PRACTICAL EXAMPLE: Student Grades

By Students (rows = students):

Alice: [85 92 78]

Bob: [88 76 90]

Charlie: [91 89 84]

By Subjects (rows = subjects):

Math: [85 88 91]

Stats: [92 76 89]

CS: [78 90 84]

=====

## 9.9.2 Reshaping Quick Reference

| Method                                        | Purpose                  | Memory               | Size Change | In-Place |
|-----------------------------------------------|--------------------------|----------------------|-------------|----------|
| <code>reshape()</code>                        | Change dimensions        | View (when possible) | No          | No       |
| <code>flatten()</code>                        | Convert to 1D            | Copy                 | No          | No       |
| <code>ravel()</code>                          | Convert to 1D            | View (when possible) | No          | No       |
| <code>resize()</code>                         | Change dimensions & size | In-place             | Yes         | Yes      |
| <code>transpose()</code><br>/ <code>.T</code> | Flip dimensions          | View                 | No          | No       |

### Best Practices:

- Use **`reshape()`** for safe dimension changes
- Use **`ravel()`** for fast 1D conversion
- Use **`flatten()`** when you need an independent copy
- **Avoid `resize()`** unless you specifically need to change array size
- Use **`.T`** for simple 2D matrix transposition

```
Practice Exercise: Image Data Reshaping
print(" PRACTICE: IMAGE DATA RESHAPING")
print("=" * 40)

Simulate image data (height × width × channels)
image_data = np.random.randint(0, 256, (64, 64, 3), dtype=np.uint8)
print(f"Original image shape: {image_data.shape} (H×W×C)")
print(f"Total pixels: {image_data.shape[0] * image_data.shape[1]}")
print(f"Memory usage: {image_data.nbytes:,} bytes")
```

```

Common reshaping operations in computer vision
print(f"\n Common Computer Vision Reshaping:")

1. Flatten for machine learning (pixels as features)
flat_features = image_data.reshape(-1, 3) # (4096, 3) - each pixel as RGB triplet
print(f"ML features: {flat_features.shape} (pixels×RGB)")

2. Batch processing format (batch × height × width × channels)
batch_format = image_data.reshape(1, 64, 64, 3) # Add batch dimension
print(f"Batch format: {batch_format.shape} (N×H×W×C)")

3. Channel-first format (channels × height × width) - PyTorch style
channels_first = image_data.transpose(2, 0, 1) # (3, 64, 64)
print(f"Channels first: {channels_first.shape} (C×H×W)")

4. Grayscale conversion (average RGB channels)
grayscale = np.mean(image_data, axis=2, keepdims=True) # (64, 64, 1)
print(f"Grayscale: {grayscale.shape} (H×W×1)")

5. Thumbnail creation (downsample)
thumbnail = image_data[::8, ::8, :] # Take every 8th pixel
print(f"Thumbnail: {thumbnail.shape} (downsampled)")

print(f"\n Memory efficiency:")
print(f"Original: {image_data.nbytes:,} bytes")
print(f"Thumbnail: {thumbnail.nbytes:,} bytes")
print(f"Reduction: {image_data.nbytes / thumbnail.nbytes:.1f}× smaller")

```

#### PRACTICE: IMAGE DATA RESHAPING

```

=====
Original image shape: (64, 64, 3) (H×W×C)
Total pixels: 4096
Memory usage: 12,288 bytes

```

```

Common Computer Vision Reshaping:
ML features: (4096, 3) (pixels×RGB)
Batch format: (1, 64, 64, 3) (N×H×W×C)
Channels first: (3, 64, 64) (C×H×W)
Grayscale: (64, 64, 1) (H×W×1)
Thumbnail: (8, 8, 3) (downsampled)

```

```

Memory efficiency:

```

Original: 12,288 bytes  
Thumbnail: 192 bytes  
Reduction: 64.0× smaller

## 9.10 Array Concatenation: Joining Arrays Together

**Array concatenation** combines multiple arrays into a single array. This is essential for:

- **Data merging:** Combining datasets from different sources
- **Batch processing:** Joining results from parallel computations
- **Feature engineering:** Combining different feature sets
- **Time series:** Appending new data to existing sequences

**Key Principle:** Arrays must have **compatible shapes** along all axes except the concatenation axis.

### 9.10.1 Understanding Axes in Concatenation

Before diving into methods, let's understand how axes work:

- **axis=0:** Concatenate **vertically** (stack rows) → increases number of rows
- **axis=1:** Concatenate **horizontally** (stack columns) → increases number of columns
- **axis=2:** For 3D+ arrays, concatenate along depth dimension

**Memory Tip:** Think of axis numbers as the dimension that **grows** during concatenation.

### 9.10.2 np.concatenate(): The Universal Joiner

`np.concatenate()` is the **most flexible** concatenation function. It can join arrays along any axis with fine-grained control.

**Syntax:** `np.concatenate((array1, array2, ...), axis=0)`

**Requirements:**

- All arrays must have the **same number of dimensions**
- Shapes must match along **all axes except the concatenation axis**

**Advantages:**

- Works with **any number of arrays**

- Can specify **any axis** for concatenation
- **Most memory efficient** for large operations

### 9.10.2.1 Comprehensive Concatenation Examples

Let's explore concatenation with practical, real-world examples:

```
Real-world Example: Student Test Scores
print(" STUDENT SCORE CONCATENATION")
print("=" * 40)

Semester 1 scores (students × subjects)
semester1 = np.array([[85, 92, 78], # Alice: Math, Science, English
 [88, 76, 91], # Bob: Math, Science, English
 [95, 89, 84]]) # Carol: Math, Science, English

Semester 2 scores (same students, same subjects)
semester2 = np.array([[87, 89, 82], # Alice: Math, Science, English
 [91, 78, 89], # Bob: Math, Science, English
 [93, 92, 88]]) # Carol: Math, Science, English

students = ['Alice', 'Bob', 'Carol']
subjects = ['Math', 'Science', 'English']

print("Semester 1 Scores:")
for i, student in enumerate(students):
 print(f"{student:6}: {semester1[i]}")

print("\nSemester 2 Scores:")
for i, student in enumerate(students):
 print(f"{student:6}: {semester2[i]}")
```

```
STUDENT SCORE CONCATENATION
=====
Semester 1 Scores:
Alice : [85 92 78]
Bob : [88 76 91]
Carol : [95 89 84]

Semester 2 Scores:
Alice : [87 89 82]
```

```
Bob : [91 78 89]
Carol : [93 92 88]
```

```
AXIS=0: Vertical Concatenation (More Students)
print("\n" + "=" * 50)
print(" AXIS=0 CONCATENATION (Vertical Stacking)")
print("=" * 50)

Add new students to the class
new_students = np.array([[82, 85, 79], # David: Math, Science, English
 [90, 88, 93]]) # Emma: Math, Science, English

Concatenate along axis=0 (add rows = add students)
expanded_class = np.concatenate((semester1, new_students), axis=0)

print(f"Original class size: {semester1.shape[0]} students")
print(f"New students added: {new_students.shape[0]} students")
print(f"Total class size: {expanded_class.shape[0]} students")
print(f"\nExpanded class scores (5 students × 3 subjects):")
print(expanded_class)

all_students = students + ['David', 'Emma']
for i, student in enumerate(all_students):
 print(f"{student:6}: {expanded_class[i]}")
```

```
=====
 AXIS=0 CONCATENATION (Vertical Stacking)
=====

Original class size: 3 students
New students added: 2 students
Total class size: 5 students

Expanded class scores (5 students × 3 subjects):
[[85 92 78]
 [88 76 91]
 [95 89 84]
 [82 85 79]
 [90 88 93]]
Alice : [85 92 78]
Bob : [88 76 91]
Carol : [95 89 84]
```



David : [82 85 79]  
Emma : [90 88 93]

```
AXIS=1: Horizontal Concatenation (More Subjects)
print("\n" + "=" * 50)
print(" AXIS=1 CONCATENATION (Horizontal Stacking)")
print("=" * 50)

Add new subjects (History and Art scores)
new_subjects_scores = np.array([[80, 92], # Alice: History, Art
 [85, 88], # Bob: History, Art
 [91, 95]]) # Carol: History, Art

Concatenate along axis=1 (add columns = add subjects)
expanded_subjects = np.concatenate((semester1, new_subjects_scores), axis=1)

print(f"Original subjects: {semester1.shape[1]} subjects")
print(f"New subjects added: {new_subjects_scores.shape[1]} subjects")
print(f"Total subjects: {expanded_subjects.shape[1]} subjects")
print(f"\nExpanded scores (3 students × 5 subjects):")

all_subjects = subjects + ['History', 'Art']
print(f"Subjects: {all_subjects}")
print(expanded_subjects)

for i, student in enumerate(students):
 print(f"{student:6}: {expanded_subjects[i]}")
```

```
=====
 AXIS=1 CONCATENATION (Horizontal Stacking)
=====

Original subjects: 3 subjects
New subjects added: 2 subjects
Total subjects: 5 subjects

Expanded scores (3 students × 5 subjects):
Subjects: ['Math', 'Science', 'English', 'History', 'Art']
[[85 92 78 80 92]
 [88 76 91 85 88]
 [95 89 84 91 95]]
Alice : [85 92 78 80 92]
```

Bob : [88 76 91 85 88]  
Carol : [95 89 84 91 95]

### 9.10.2.2 Visual Understanding of Concatenation

Here's how axis=0 and axis=1 concatenation work visually:

AXIS=0 (Vertical - Stack Rows):

|         |   |         |   |         |
|---------|---|---------|---|---------|
| Array 1 | + | Array 2 | = | Array 1 |
| [1,2,3] |   | [7,8,9] |   | [1,2,3] |
| [4,5,6] |   |         |   | [4,5,6] |
|         |   |         |   | [7,8,9] |

AXIS=1 (Horizontal - Stack Columns):

|         |   |       |   |           |
|---------|---|-------|---|-----------|
|         | + |       | = |           |
| Array 1 |   | Array |   | Combined  |
| [1,2,3] |   | [7]   |   | [1,2,3,7] |
| [4,5,6] |   | [8]   |   | [4,5,6,8] |

**Rule of Thumb:**

- **axis=0** → **More rows** (vertical growth)
- **axis=1** → **More columns** (horizontal growth)

### 9.10.2.3 Common Concatenation Errors and Solutions

Let's explore what happens when shapes don't match and how to fix it:

```
SHAPE MISMATCH EXAMPLE
print(" CONCATENATION ERROR DEMONSTRATION")
print("=" * 45)

Original 2D array
original_2d = np.array([[1, 2, 3],
 [4, 5, 6]])

print(f"Original 2D array shape: {original_2d.shape}")
print(f"Original 2D array:\n{original_2d}")
```

```
Problematic 1D array
problem_1d = np.array([7, 8, 9])
print(f"\nProblematic 1D array shape: {problem_1d.shape}")
print(f"Problematic 1D array: {problem_1d}")

print(f"\n Why this fails:")
print(f" 2D array: {original_2d.shape} (2 dimensions)")
print(f" 1D array: {problem_1d.shape} (1 dimension)")
print(f" Different number of dimensions!")
```

#### CONCATENATION ERROR DEMONSTRATION

```
=====
```

Original 2D array shape: (2, 3)

Original 2D array:

```
[[1 2 3]
 [4 5 6]]
```

Problematic 1D array shape: (3,)

Problematic 1D array: [7 8 9]

Why this fails:

2D array: (2, 3) (2 dimensions)

1D array: (3,) (1 dimension)

Different number of dimensions!

```
Demonstrate the actual error
print("\n Attempting concatenation (this will fail):")
try:
 result = np.concatenate((original_2d, problem_1d), axis=0)
 print("Success!")
except ValueError as e:
 print(f" Error: {e}")
 print(" → Arrays have different numbers of dimensions!")
```

Attempting concatenation (this will fail):

Error: all the input arrays must have same number of dimensions, but the array at index 0 has

→ Arrays have different numbers of dimensions!

### 9.10.2.4 Solutions to Shape Mismatch Problems

When arrays don't have compatible shapes, here are the most common fixes:

```
SOLUTION 1: Reshape to match dimensions
print(" SOLUTION 1: Reshape 1D → 2D")
print("=" * 35)

print(f"Original problematic shape: {problem_1d.shape}")

Method 1: Add row dimension
solution_1 = problem_1d.reshape(1, 3) # Convert to (1, 3)
print(f"Reshaped to row: {solution_1.shape}")
print(f"Reshaped array:\n{solution_1}")

Method 2: Add column dimension
solution_2 = problem_1d.reshape(3, 1) # Convert to (3, 1)
print(f"\nReshaped to column: {solution_2.shape}")
print(f"Reshaped array:\n{solution_2}")

Method 3: Using newaxis (more explicit)
solution_3 = problem_1d[np.newaxis, :] # Same as reshape(1, 3)
solution_4 = problem_1d[:, np.newaxis] # Same as reshape(3, 1)
print(f"\nUsing newaxis:")
print(f"Row format: {solution_3.shape} → {solution_3}")
print(f"Col format: {solution_4.shape} → \n{solution_4}")
```

```
SOLUTION 1: Reshape 1D → 2D
=====
Original problematic shape: (3,)
Reshaped to row: (1, 3)
Reshaped array:
[[7 8 9]]

Reshaped to column: (3, 1)
Reshaped array:
[[7]
 [8]
 [9]]

Using newaxis:
Row format: (1, 3) → [[7 8 9]]
```

```
Col format: (3, 1) →
[[7]
 [8]
 [9]]
```

### 9.10.2.5 Successful Concatenation After Reshaping

```
Now concatenation works!
print(" SUCCESSFUL CONCATENATION")
print("=" * 30)

Using the reshaped array
fixed_array = problem_1d.reshape(1, 3)
print(f"Original 2D: {original_2d.shape}")
print(f"Fixed 1D→2D: {fixed_array.shape}")

Now concatenation works
success_result = np.concatenate((original_2d, fixed_array), axis=0)
print(f"\n Concatenation successful!")
print(f"Result shape: {success_result.shape}")
print(f"Result:\n{success_result}")

print(f"\n What happened:")
print(f" • Started with (2,3) + (3,) → Incompatible")
print(f" • Reshaped to (2,3) + (1,3) → Compatible")
print(f" • Final result: (3,3) array")
```

```
SUCCESSFUL CONCATENATION
=====
```

```
Original 2D: (2, 3)
Fixed 1D→2D: (1, 3)

Concatenation successful!
Result shape: (3, 3)
Result:
[[1 2 3]
 [4 5 6]
 [7 8 9]]
```

What happened:

- Started with (2,3) + (3,) → Incompatible
- Reshaped to (2,3) + (1,3) → Compatible
- Final result: (3,3) array

### 9.10.2.6 Multiple Array Concatenation

`np.concatenate()` can join more than two arrays at once:

```
MULTIPLE ARRAY CONCATENATION
print(" JOINING MULTIPLE ARRAYS")
print("=" * 30)

Create multiple arrays representing quarterly sales data
q1_sales = np.array([[100, 150], [120, 180]]) # 2 stores, 2 products
q2_sales = np.array([[110, 160], [130, 190]])
q3_sales = np.array([[105, 155], [125, 185]])
q4_sales = np.array([[115, 165], [135, 195]])

print("Quarterly Sales Data (Stores × Products):")
print(f"Q1: \n{q1_sales}")
print(f"Q2: \n{q2_sales}")
print(f"Q3: \n{q3_sales}")
print(f"Q4: \n{q4_sales}")

Method 1: Concatenate all quarters horizontally (timeline view)
yearly_timeline = np.concatenate((q1_sales, q2_sales, q3_sales, q4_sales), axis=1)
print(f"\nHorizontal Timeline (Stores × Q1-Q4 Products):")
print(f"Shape: {yearly_timeline.shape}")
print(yearly_timeline)

Method 2: Stack all quarters vertically (easier analysis)
yearly_stack = np.concatenate((q1_sales, q2_sales, q3_sales, q4_sales), axis=0)
print(f"\nVertical Stack (All Store-Quarters × Products):")
print(f"Shape: {yearly_stack.shape}")
print(yearly_stack)

print(f"\n Analysis:")
print(f"Timeline view: {yearly_timeline.shape[0]} stores × {yearly_timeline.shape[1]} data points")
print(f"Stacked view: {yearly_stack.shape[0]} observations × {yearly_stack.shape[1]} products")
```

JOINING MULTIPLE ARRAYS

```

=====
Quarterly Sales Data (Stores × Products):
Q1:
[[100 150]
 [120 180]]
Q2:
[[110 160]
 [130 190]]
Q3:
[[105 155]
 [125 185]]
Q4:
[[115 165]
 [135 195]]

Horizontal Timeline (Stores × Q1-Q4 Products):
Shape: (2, 8)
[[100 150 110 160 105 155 115 165]
 [120 180 130 190 125 185 135 195]]

Vertical Stack (All Store-Quarters × Products):
Shape: (8, 2)
[[100 150]
 [120 180]
 [110 160]
 [130 190]
 [105 155]
 [125 185]
 [115 165]
 [135 195]]

Analysis:
Timeline view: 2 stores × 8 data points
Stacked view: 8 observations × 2 products

```

### 9.10.3 Specialized Stacking Functions

While `np.concatenate()` is the most flexible, NumPy provides convenient shortcuts for common operations:

| Function                       | Equivalent                          | Use Case                     | Advantage                         |
|--------------------------------|-------------------------------------|------------------------------|-----------------------------------|
| <code>np.vstack()</code>       | <code>concatenate(axis=0)</code>    | Stack vertically (rows)      | Clearer intent, handles 1D arrays |
| <code>np.hstack()</code>       | <code>concatenate(axis=1)</code>    | Stack horizontally (columns) | Clearer intent, handles 1D arrays |
| <code>np.dstack()</code>       | <code>concatenate(axis=2)</code>    | Stack along depth (3D)       | Easy 3D operations                |
| <code>np.column_stack()</code> | Special case of <code>hstack</code> | Treat 1D as columns          | Great for combining vectors       |
| <code>np.row_stack()</code>    | Same as <code>vstack</code>         | Treat inputs as rows         | Alias for <code>vstack</code>     |

#### When to use each:

- **`vstack/hstack`:** When you want clearer, more readable code
- **`concatenate`:** When you need maximum flexibility or performance

```
COMPREHENSIVE STACKING COMPARISON
print(" STACKING FUNCTIONS COMPARISON")
print("=" * 40)

Sample arrays for demonstration
array_a = np.array([[1, 2], [3, 4]])
array_b = np.array([[5, 6], [7, 8]])

print(f"Array A:\n{array_a}")
print(f"Array B:\n{array_b}")

Vertical stacking (stack rows)
print(f"\n VERTICAL STACKING (More Rows)")
print("-" * 35)
vstack_result = np.vstack((array_a, array_b))
concat_v_result = np.concatenate((array_a, array_b), axis=0)

print(f"np.vstack():\n{vstack_result}")
print(f"Equivalent concatenate(axis=0):\n{concat_v_result}")
print(f"Results identical: {np.array_equal(vstack_result, concat_v_result)}")

Horizontal stacking (stack columns)
print(f"\n HORIZONTAL STACKING (More Columns)")
print("-" * 38)
hstack_result = np.hstack((array_a, array_b))
```



```

concat_h_result = np.concatenate((array_a, array_b), axis=1)

print(f"np.hstack():\n{hstack_result}")
print(f"Equivalent concatenate(axis=1):\n{concat_h_result}")
print(f"Results identical: {np.array_equal(hstack_result, concat_h_result)}")

Depth stacking (3D)
print(f"\n DEPTH STACKING (3D Arrays)")
print("-" * 25)
dstack_result = np.dstack((array_a, array_b))
concat_d_result = np.concatenate((array_a[..., np.newaxis], array_b[..., np.newaxis]), axis=-1)

print(f"np.dstack() shape: {dstack_result.shape}")
print(f"First 'slice':\n{dstack_result[:, :, 0]}")
print(f"Second 'slice':\n{dstack_result[:, :, 1]}")

Special case: 1D arrays
print(f"\n SPECIAL: 1D Array Handling")
print("-" * 30)
vec1 = np.array([1, 2, 3])
vec2 = np.array([4, 5, 6])

print(f"Vector 1: {vec1}")
print(f"Vector 2: {vec2}")

vstack treats 1D as rows
vstack_1d = np.vstack((vec1, vec2))
print(f"vstack (treats as rows):\n{vstack_1d}")

hstack treats 1D as elements
hstack_1d = np.hstack((vec1, vec2))
print(f"hstack (concatenates): {hstack_1d}")

column_stack treats 1D as columns
column_stack_1d = np.column_stack((vec1, vec2))
print(f"column_stack (treats as cols):\n{column_stack_1d}")

```

## STACKING FUNCTIONS COMPARISON

=====

Array A:

```
[[1 2]
 [3 4]]
```

Array B:

```
[[5 6]
 [7 8]]
```

#### VERTICAL STACKING (More Rows)

np.vstack():

```
[[1 2]
 [3 4]
 [5 6]
 [7 8]]
```

Equivalent concatenate(axis=0):

```
[[1 2]
 [3 4]
 [5 6]
 [7 8]]
```

Results identical: True

#### HORIZONTAL STACKING (More Columns)

np.hstack():

```
[[1 2 5 6]
 [3 4 7 8]]
```

Equivalent concatenate(axis=1):

```
[[1 2 5 6]
 [3 4 7 8]]
```

Results identical: True

#### DEPTH STACKING (3D Arrays)

np.dstack() shape: (2, 2, 2)

First 'slice':

```
[[1 2]
 [3 4]]
```

Second 'slice':

```
[[5 6]
 [7 8]]
```

#### SPECIAL: 1D Array Handling

Vector 1: [1 2 3]

Vector 2: [4 5 6]

vstack (treats as rows):

```

[[1 2 3]
 [4 5 6]]
hstack (concatenates): [1 2 3 4 5 6]
column_stack (treats as cols):
[[1 4]
 [2 5]
 [3 6]]

```

#### 9.10.4 Advanced Concatenation Techniques

```

ADVANCED TECHNIQUES
print(" PERFORMANCE & MEMORY OPTIMIZATION")
print("-" * 40)

Performance comparison for large arrays
large_arrays = [np.random.rand(1000, 100) for _ in range(10)]

import time

Method 1: Multiple concatenate calls (inefficient)
start_time = time.perf_counter()
result_slow = large_arrays[0]
for arr in large_arrays[1:]:
 result_slow = np.concatenate((result_slow, arr), axis=0)
slow_time = time.perf_counter() - start_time

Method 2: Single concatenate call (efficient)
start_time = time.perf_counter()
result_fast = np.concatenate(large_arrays, axis=0)
fast_time = time.perf_counter() - start_time

print(f" Sequential concatenation: {slow_time:.4f} seconds")
print(f" Batch concatenation: {fast_time:.4f} seconds")
print(f" Speedup: {slow_time/fast_time:.1f}x faster")
print(f" Result shape: {result_fast.shape}")
print(f" Memory usage: {result_fast.nbytes / 1e6:.1f} MB")

Memory efficiency demonstration
print(f"\n MEMORY EFFICIENCY")
print("-" * 25)

```

```

Pre-allocate vs concatenate
data_size = (5000, 100)
total_arrays = 20

Method 1: Concatenation (creates multiple copies)
start_time = time.perf_counter()
concat_list = []
for i in range(total_arrays):
 concat_list.append(np.random.rand(*data_size))
concat_result = np.concatenate(concat_list, axis=0)
concat_time = time.perf_counter() - start_time

Method 2: Pre-allocation (more memory efficient)
start_time = time.perf_counter()
preallocated = np.empty((total_arrays * data_size[0], data_size[1]))
for i in range(total_arrays):
 start_idx = i * data_size[0]
 end_idx = (i + 1) * data_size[0]
 preallocated[start_idx:end_idx] = np.random.rand(*data_size)
prealloc_time = time.perf_counter() - start_time

print(f"Concatenation method: {concat_time:.4f} seconds")
print(f"Pre-allocation method: {prealloc_time:.4f} seconds")
print(f"Pre-allocation is {concat_time/prealloc_time:.1f}x faster")

```

## PERFORMANCE & MEMORY OPTIMIZATION

```

=====
Sequential concatenation: 0.0099 seconds
Batch concatenation: 0.0017 seconds
Speedup: 6.0x faster
Result shape: (10000, 100)
Memory usage: 8.0 MB

```

## MEMORY EFFICIENCY

```

Concatenation method: 0.0772 seconds
Pre-allocation method: 0.0718 seconds
Pre-allocation is 1.1x faster

```

### 9.10.4.1 Real-world Application: Time Series Data

Let's see concatenation in action with a practical time series example:

```
TIME SERIES CONCATENATION EXAMPLE
print(" FINANCIAL DATA PROCESSING")
print("=" * 35)

Simulate daily stock prices for different months
np.random.seed(42)
jan_prices = np.random.uniform(100, 120, (31, 4)) # 31 days, 4 stocks
feb_prices = np.random.uniform(98, 118, (28, 4)) # 28 days, 4 stocks
mar_prices = np.random.uniform(102, 122, (31, 4)) # 31 days, 4 stocks

stock_names = ['AAPL', 'GOOGL', 'MSFT', 'TSLA']
print(f"Stock data shape - Days x Stocks:")
print(f"January: {jan_prices.shape} ({jan_prices.shape[0]} days)")
print(f"February: {feb_prices.shape} ({feb_prices.shape[0]} days)")
print(f"March: {mar_prices.shape} ({mar_prices.shape[0]} days)")

Combine quarterly data
q1_prices = np.concatenate((jan_prices, feb_prices, mar_prices), axis=0)
print(f"\nQ1 Combined: {q1_prices.shape} ({q1_prices.shape[0]} total days)")

Calculate monthly averages
monthly_avg = np.array([
 np.mean(jan_prices, axis=0),
 np.mean(feb_prices, axis=0),
 np.mean(mar_prices, axis=0)
])

print(f"\nMonthly averages shape: {monthly_avg.shape}")
print(f"Monthly averages:")
months = ['January', 'February', 'March']
for i, month in enumerate(months):
 print(f"{month:8}: " + " | ".join([f"{stock}: ${avg:.2f}"
 for stock, avg in zip(stock_names, monthly_avg[i])]))

Add new features (volume data) using horizontal concatenation
np.random.seed(42)
q1_volumes = np.random.randint(1000000, 5000000, (q1_prices.shape[0], 4))
q1_complete = np.hstack((q1_prices, q1_volumes))
```

```
print(f"\nWith volume data: {q1_complete.shape}")
print(f"Columns: {stock_names} (prices) + {stock_names} (volumes)")
print(f"Sample day 1: Prices={q1_complete[0, :4]}")
print(f" Volumes={q1_complete[0, 4:].astype(int)}")
```

#### FINANCIAL DATA PROCESSING

=====

Stock data shape - Days × Stocks:

January: (31, 4) (31 days)

February: (28, 4) (28 days)

March: (31, 4) (31 days)

Q1 Combined: (90, 4) (90 total days)

Monthly averages shape: (3, 4)

Monthly averages:

January : AAPL: \$109.57 | GOOGL: \$109.65 | MSFT: \$108.89 | TSLA: \$110.21

February: AAPL: \$106.62 | GOOGL: \$106.70 | MSFT: \$109.63 | TSLA: \$108.16

March : AAPL: \$113.36 | GOOGL: \$112.84 | MSFT: \$111.29 | TSLA: \$110.70

With volume data: (90, 8)

Columns: ['AAPL', 'GOOGL', 'MSFT', 'TSLA'] (prices) + ['AAPL', 'GOOGL', 'MSFT', 'TSLA'] (volumes)

Sample day 1: Prices=[107.49080238 119.01428613 114.63987884 111.97316968]

Volumes=[3219110 3768307 3229084 4511566]

### 9.10.5 Concatenation Quick Reference

| Operation        | Function       | Syntax                         | Result            |
|------------------|----------------|--------------------------------|-------------------|
| Vertical Stack   | vstack()       | np.vstack((A, B))              | More rows         |
| Horizontal Stack | hstack()       | np.hstack((A, B))              | More columns      |
| General Concat   | concatenate()  | np.concatenate((A, B), axis=n) | Custom axis       |
| 3D Stack         | dstack()       | np.dstack((A, B))              | Depth dimension   |
| Column Combine   | column_stack() | np.column_stack((v1, v2))      | Vectors → columns |

### 9.10.6 Common Pitfalls and Solutions

| Problem            | Cause                | Solution                                               |
|--------------------|----------------------|--------------------------------------------------------|
| shapes not aligned | Different dimensions | Use <code>reshape()</code> or <code>newaxis</code>     |
| axis out of bounds | Wrong axis number    | Check array dimensions with <code>.ndim</code>         |
| memory error       | Too many copies      | Use pre-allocation or batch operations                 |
| 1D array issues    | Mixed 1D/2D arrays   | Use <code>vstack/hstack</code> or reshape consistently |

## 9.11 Independent Practice

### 9.11.1 Capitals & Distance from Washington, D.C.

**Data:** `country-capital-lat-long-population.csv`

**Task:**

Use NumPy to print the **name** and **coordinates** of the capital city **closest** to the U.S. capital, **Washington, D.C.** (exclude Washington, D.C. itself).

**Notes**

1. The **Country Name** for the U.S. is “**United States of America**” in the dataset.
2. “Closeness” is measured by **Euclidean distance** computed on the latitude/longitude pairs (for this exercise).
3. **Exclude** the U.S. capital row to avoid the trivial zero distance.

**Hints**

1. Use `DataFrame.to_numpy()` to convert columns to NumPy arrays.
2. Use **broadcasting** to compute euclidean distances from Washington, D.C. to all other capitals.
3. Use `np.argmin` to get the index of the **minimum** distance (closest capital).
4. Use `np.argsort()` to get the **indices of the 10 smallest distances** (nearest capitals) and index back into the DataFrame for names/coordinates.

5. Use `np.argsort()` to get the **indices of the 10 largest distances** (farthest capitals) similarly.
6. Remember to **mask out** or drop the Washington, D.C. row before computing distances.

Closest capital city is: Ottawa-Gatineau

Coordinates of the closest capital city are: [ 45.4166 -75.698 ]

Top 10 closest capital cities to Washington DC are:

|     | Capital City       | Country                   |
|-----|--------------------|---------------------------|
| 36  | Ottawa-Gatineau    | Canada                    |
| 14  | Nassau             | Bahamas                   |
| 22  | Hamilton           | Bermuda                   |
| 55  | La Habana (Havana) | Cuba                      |
| 215 | Cockburn Town      | Turks and Caicos Islands  |
| 38  | George Town        | Cayman Islands            |
| 94  | Port-au-Prince     | Haiti                     |
| 107 | Kingston           | Jamaica                   |
| 64  | Santo Domingo      | Dominican Republic        |
| 177 | Saint-Pierre       | Saint Pierre and Miquelon |

Task 2: Print the name and coordinates of the capital city farthest from the U.S. capital, Washington, D.C.

|     | Country          | Capital City | Latitude | Longitude | Population | Capital Type | Distance   |
|-----|------------------|--------------|----------|-----------|------------|--------------|------------|
| 149 | New Zealand      | Wellington   | -41.2866 | 174.7756  | 411346     | Capital      | 264.269537 |
| 74  | Fiji             | Suva         | -18.1416 | 178.4415  | 178339     | Capital      | 261.767344 |
| 216 | Tuvalu           | Funafuti     | -8.5189  | 179.1991  | 7042       | Capital      | 260.585339 |
| 112 | Kiribati         | Tarawa       | 1.3272   | 172.9813  | 64011      | Capital      | 252.824440 |
| 226 | Vanuatu          | Port Vila    | -17.7338 | 168.3219  | 52690      | Capital      | 251.808514 |
| 148 | New Caledonia    | Nouméa       | -22.2763 | 166.4572  | 197787     | Capital      | 251.059900 |
| 130 | Marshall Islands | Majuro       | 7.0897   | 171.3803  | 30661      | Capital      | 250.444485 |
| 145 | Nauru            | Nauru        | -0.5308  | 166.9112  | 11312      | Capital      | 247.112997 |
| 191 | Solomon Islands  | Honiara      | -9.4333  | 159.9500  | 81801      | Capital      | 241.863987 |
| 11  | Australia        | Canberra     | -35.2835 | 149.1281  | 447692     | Capital      | 238.018583 |

### 9.11.2 Bonus Task: Top 10 Nearest & Farthest Capitals from Washington, D.C. (Haversine)

Using the **haversine** (great-circle) distance, find:



- The **top 10 closest** capital cities to **Washington, D.C.**
- The **top 10 farthest** capital cities from **Washington, D.C.**

#### Notes

- Use lat/lon in **degrees** (your haversine function should convert to **radians** internally).
- Exclude **Washington, D.C.** itself from the results.
- Handle ties deterministically (e.g., `kind="stable"` in `argsort` if needed).

Why haversine? It computes **real geodesic distance on a sphere**, which is more accurate than Euclidean distance on raw lat/lon.

# 10 NumPy Intermediate

<IPython.core.display.Image object>

## 10.1 Learning Objectives

At the end of this chapter, you will learn:

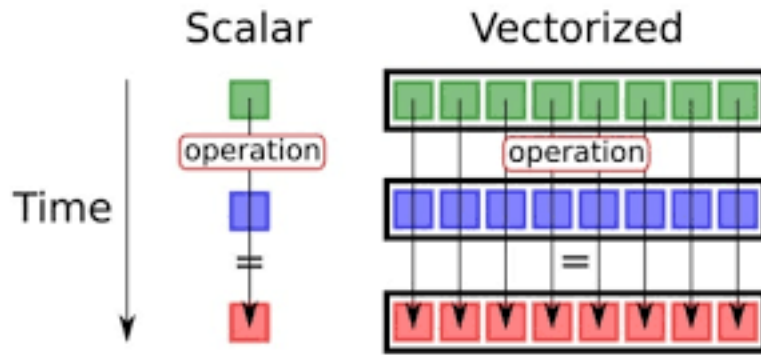
- **Optimize code performance** using vectorization instead of loops
- **Perform vectorized computations** that are 10–100x faster than Python loops
- **Generate random data** for simulations and statistical analysis
- **Convert between NumPy and Pandas** for optimal workflow efficiency

```
import numpy as np
import pandas as pd
import time
import warnings

Suppress warnings for cleaner output
warnings.filterwarnings('ignore')
```

## 10.2 Vectorization in NumPy

**Vectorization** applies operations to entire arrays simultaneously instead of looping through individual elements. This enables NumPy to leverage highly optimized C/Fortran libraries for dramatic performance gains.



### 10.2.1 Why Vectorization Matters

**Vectorized operations** require fewer lines of code and are easier to read compared to Python loops.

Moreover, since they rely on compiled **C** and **Fortran** libraries that leverage **CPU SIMD instructions** and **efficient cache utilization**, they can be **significantly faster and more scalable** than pure Python loops.

**Loop Approach:**

```
result = []
for i in range(len(array1)):
 for j in range(len(array2)):
 result.append(array1[i] * array2[j] + some_value)
```

**Vectorized Approach:**

```
result = array1[:, np.newaxis] * array2 + some_value
```

#### 10.2.1.1 Performance Comparison: NumPy Vectorization Vs. Python Loops

Understanding the performance benefits of vectorization is crucial for writing efficient scientific computing code. Let's compare different scenarios to see when and why vectorization matters.

### 10.2.1.1.1 Scenario 1: Basic Mathematical Operations

`np.dot` is a vectorized operation in NumPy that performs matrix multiplication or dot product between arrays. It efficiently computes the element-wise multiplications and then sums them up. To better understand its efficiency, let's first implement the dot product using for loops, and then compare its performance with `np.dot` to see the benefits of vectorization.

```
Function to calculate dot product using for loops
def dot_product_loops(arr1, arr2):
 result = 0
 for i in range(len(arr1)):
 result += arr1[i] * arr2[i]
 return result

Create sample arrays of different sizes for comparison
sizes = [1000, 10000, 100000, 1000000]

print("Performance Comparison: Loops vs NumPy Vectorization")
print("=" * 60)

for size in sizes:
 # Create random arrays
 arr1 = np.random.rand(size)
 arr2 = np.random.rand(size)

 # Measure time for the loop-based implementation
 start_time = time.time()
 loop_result = dot_product_loops(arr1, arr2)
 loop_time = time.time() - start_time

 # Measure time for np.dot
 start_time = time.time()
 numpy_result = np.dot(arr1, arr2)
 numpy_time = time.time() - start_time

 # Calculate speedup
 speedup = loop_time / numpy_time if numpy_time > 0 else float('inf')

 print(f"\nArray size: {size:,}")
 print(f"Loop result: {loop_result:.5f}, Time: {loop_time:.5f}s")
 print(f"NumPy result: {numpy_result:.5f}, Time: {numpy_time:.5f}s")
 print(f"Speedup: {speedup:.1f}x faster")
```

```

print(f"\n{'='*60}")
print("Key Observations:")
print("• NumPy becomes dramatically faster as array size increases")
print("• Speedup can reach over 100x for large arrays")
print("• NumPy overhead is minimal for small arrays but pays off quickly")

```

#### Performance Comparison: Loops vs NumPy Vectorization

=====

Array size: 1,000  
 Loop result: 240.51012, Time: 0.00100s  
 NumPy result: 240.51012, Time: 0.00000s  
 Speedup: infx faster

Array size: 10,000  
 Loop result: 2483.13248, Time: 0.00450s  
 NumPy result: 2483.13248, Time: 0.00000s  
 Speedup: infx faster

Array size: 100,000  
 Loop result: 24918.72212, Time: 0.02491s  
 NumPy result: 24918.72212, Time: 0.00000s  
 Speedup: infx faster

Array size: 1,000,000  
 Loop result: 250291.73101, Time: 0.18839s  
 NumPy result: 250291.73101, Time: 0.00098s  
 Speedup: 192.8x faster

=====

#### Key Observations:

- NumPy becomes dramatically faster as array size increases
- Speedup can reach over 100x for large arrays
- NumPy overhead is minimal for small arrays but pays off quickly

Array size: 1,000,000  
 Loop result: 250291.73101, Time: 0.18839s  
 NumPy result: 250291.73101, Time: 0.00098s  
 Speedup: 192.8x faster

=====

Key Observations:

- NumPy becomes dramatically faster as array size increases
- Speedup can reach over 100x for large arrays
- NumPy overhead is minimal for small arrays but pays off quickly

#### 10.2.1.1.2 Scenario 2: Complex Mathematical Operations

Let's compare more complex operations that show even greater performance differences:

```
Complex mathematical operation: polynomial evaluation
$f(x) = 3x^3 + 2x^2 - 5x + 1$

def polynomial_loops(x_values):
 """Evaluate polynomial using loops"""
 results = []
 for x in x_values:
 result = 3*x**3 + 2*x**2 - 5*x + 1
 results.append(result)
 return results

def polynomial_vectorized(x_values):
 """Evaluate polynomial using vectorized operations"""
 return 3*x_values**3 + 2*x_values**2 - 5*x_values + 1

Test with different sizes
size = 5000000
x_data = np.random.uniform(-10, 10, size)

print("Complex Operations: Polynomial Evaluation")
print("f(x) = 3x3 + 2x2 - 5x + 1")
print("-" * 40)

Loop version
start_time = time.time()
loop_results = polynomial_loops(x_data)
loop_time = time.time() - start_time

Vectorized version
start_time = time.time()
vector_results = polynomial_vectorized(x_data)
vector_time = time.time() - start_time
```

```

speedup = loop_time / vector_time

print(f"Array size: {size:,}")
print(f"Loop time: {loop_time:.4f}s")
print(f"Vectorized time: {vector_time:.4f}s")
print(f"Speedup: {speedup:.1f}x")

Verify results match
print(f"Results match: {np.allclose(loop_results, vector_results)}")

```

Complex Operations: Polynomial Evaluation

$f(x) = 3x^3 + 2x^2 - 5x + 1$

-----

```

Array size: 5,000,000
Loop time: 2.8250s
Vectorized time: 0.2053s
Speedup: 13.8x
Results match: True
Array size: 5,000,000
Loop time: 2.8250s
Vectorized time: 0.2053s
Speedup: 13.8x
Results match: True

```

The performance comparisons clearly demonstrate that **vectorized NumPy operations** significantly outperform equivalent Python loops

## 10.3 Vectorized Operations in NumPy

Pandas **vectorized operations** are **built on top of NumPy**, which provides the foundation for efficient computation.

### 10.3.1 Types of Vectorized Operations

NumPy supports several categories of vectorized operations:

### 10.3.1.1 Element-wise Operations (Universal Functions - ufuncs)

These operations apply a function to each element of an array:

```
Basic arithmetic operations (all vectorized)
arr = np.array([1, 4, 9, 16, 25])

print("Original array:", arr)
print("Square root:", np.sqrt(arr))
print("Natural log:", np.log(arr))
print("Sine:", np.sin(arr))
print("Power of 2:", np.power(arr, 2))

Comparison operations
print("\nComparison operations:")
print("Greater than 10:", arr > 10)
print("Equal to 9:", arr == 9)
print("Between 5 and 20:", (arr >= 5) & (arr <= 20))
```

```
Original array: [1 4 9 16 25]
Square root: [1. 2. 3. 4. 5.]
Natural log: [0. 1.38629436 2.19722458 2.77258872 3.21887582]
Sine: [0.84147098 -0.7568025 0.41211849 -0.28790332 -0.13235175]
Power of 2: [1 16 81 256 625]
```

```
Comparison operations:
Greater than 10: [False False False True True]
Equal to 9: [False False True False False]
Between 5 and 20: [False False True True False]
```

### 10.3.1.2 Aggregation Operations

These reduce arrays to scalar values or smaller arrays:

```
2D array for aggregation examples
matrix = np.array([[1, 2, 3, 4],
 [5, 6, 7, 8],
 [9, 10, 11, 12]])

print("Matrix:")
print(matrix)
```



```
Aggregations across entire array
print(f"\nSum of all elements: {np.sum(matrix)}")
print(f"Mean: {np.mean(matrix)}")
print(f"Standard deviation: {np.std(matrix)}")
print(f"Min: {np.min(matrix)}, Max: {np.max(matrix)}")

Aggregations along specific axes
print(f"\nSum along axis 0 (columns): {np.sum(matrix, axis=0)}")
print(f"Sum along axis 1 (rows): {np.sum(matrix, axis=1)}")
print(f"Mean along axis 0: {np.mean(matrix, axis=0)}")
print(f"Max along axis 1: {np.max(matrix, axis=1)}")
```

Matrix:

```
[[1 2 3 4]
 [5 6 7 8]
 [9 10 11 12]]
```

Sum of all elements: 78

Mean: 6.5

Standard deviation: 3.452052529534663

Min: 1, Max: 12

Sum along axis 0 (columns): [15 18 21 24]

Sum along axis 1 (rows): [10 26 42]

Mean along axis 0: [5. 6. 7. 8.]

Max along axis 1: [ 4 8 12]

### 10.3.1.3 Boolean Operations and Fancy Indexing

Vectorized selection and filtering operations:

```
Boolean indexing - vectorized filtering
data = np.array([1, 15, 8, 23, 4, 16, 42, 3, 19, 7])

Find elements meeting multiple conditions (vectorized)
mask = (data > 5) & (data < 20)
filtered_data = data[mask]
print(f"Original data: {data}")
print(f"Elements between 5 and 20: {filtered_data}")
```

```

Using np.where for conditional replacement (vectorized)
Replace values > 15 with -1, others with original value
result = np.where(data > 15, -1, data)
print(f"After conditional replacement: {result}")

Fancy indexing - select multiple elements at once
indices = [0, 2, 4, 7]
selected = data[indices]
print(f"Elements at indices {indices}: {selected}")

Count how many elements meet condition (vectorized)
count_large = np.sum(data > 10)
print(f"Count of elements > 10: {count_large}")

```

```

Original data: [1 15 8 23 4 16 42 3 19 7]
Elements between 5 and 20: [15 8 16 19 7]
After conditional replacement: [1 15 8 -1 4 -1 -1 3 -1 7]
Elements at indices [0, 2, 4, 7]: [1 8 4 3]
Count of elements > 10: 5

```

### 10.3.2 Matrix Multiplication in NumPy with Vectorization

Matrix multiplication is one of the most common and computationally intensive operations in numerical computing and deep learning. NumPy offers efficient and highly optimized methods for performing matrix multiplication, which leverage vectorization to handle large matrices quickly and accurately.

**Note:** NumPy matrix operations follow the standard rules of linear algebra, so it's important to ensure that the shapes of the matrices are compatible. If they are not, consider reshaping the matrices before performing multiplication.

There are two common methods for matrix multiplication:

#### 10.3.2.1 Method 1: Matrix Multiplication Using `np.dot()`

```

Define two 2D arrays (matrices)
matrix1 = np.array([[1, 2, 3],
 [4, 5, 6]])
matrix2 = np.array([[7, 8],
 [9, 10],

```

```

 [11, 12]])

Matrix multiplication using np.dot
result_dot = np.dot(matrix1, matrix2)
print("Matrix Multiplication using np.dot:\n", result_dot)

Another way to perform np.dot for matrix multiplication
result_dot2 = matrix1.dot(matrix2)
print("\nMatrix Multiplication using dot method:\n", result_dot2)

```

Matrix Multiplication using np.dot:

```

[[58 64]
 [139 154]]

```

Matrix Multiplication using dot method:

```

[[58 64]
 [139 154]]

```

### 10.3.2.2 Method 2: Matrix Multiplication using np.matmul() or @

```

Matrix multiplication using np.matmul or @ operator
result_matmul = np.matmul(matrix1, matrix2)
result_operator = matrix1 @ matrix2
print("\nMatrix Multiplication using np.matmul:\n", result_matmul)
print("\nMatrix Multiplication using @ operator:\n", result_operator)

```

Matrix Multiplication using np.matmul:

```

[[58 64]
 [139 154]]

```

Matrix Multiplication using @ operator:

```

[[58 64]
 [139 154]]

```

**Important:** Note that the \* operator in NumPy performs element-wise multiplication, not matrix multiplication.

```
Using * operator for element-wise multiplication will cause an error if shapes don't match
Uncomment the code below to see the error:
element_wise = matrix1 * matrix2
print("\nElement-wise Multiplication:\n", element_wise)

reshape the array for element-wise multiplication
matrix2_resaped = matrix2.reshape(2, 3)
element_wise = matrix1 * matrix2_resaped
print("\nElement-wise Multiplication after reshaping:\n", element_wise)
```

Element-wise Multiplication after reshaping:

```
[[7 16 27]
 [40 55 72]]
```

## 10.4 Converting Between NumPy Arrays and Pandas DataFrames

**Seamless data conversion** between NumPy and pandas is essential for efficient data science workflows. Each library excels in different areas, and knowing when and how to convert between them maximizes your analytical power.

### Typical Data Science Workflow

1. LOAD DATA → Use Pandas (CSV, Excel, databases)
2. CLEAN & PREPARE → Use Pandas (filtering, grouping, handling missing data)
3. COMPUTE → Convert to NumPy (ML algorithms, linear algebra, heavy math)
4. ANALYZE & VISUALIZE → Convert back to Pandas (results interpretation)
5. EXPORT → Use Pandas (CSV, Parquet, SQL)

NumPy and pandas complement each other perfectly:

| Aspect              | NumPy Arrays                              | Pandas DataFrames                |
|---------------------|-------------------------------------------|----------------------------------|
| <b>Strength</b>     | Mathematical computations                 | Data manipulation & analysis     |
| <b>Speed</b>        | Faster numerical operations               | Easier data exploration          |
| <b>Memory</b>       | More memory efficient                     | Rich metadata (labels, dtypes)   |
| <b>Use Cases</b>    | Linear algebra, statistics, ML algorithms | Data cleaning, grouping, merging |
| <b>Indexing</b>     | Integer-based only                        | Label-based + integer-based      |
| <b>Missing Data</b> | Limited NaN support                       | Robust NaN handling              |

**The key question:** *When should you stay in pandas vs convert to NumPy?*

## 10.4.1 Decision Framework: When to Use Which Library

### 10.4.1.1 Stay in Pandas for Vectorized Operations

Vectorized operations in Pandas are built on top of **NumPy** and are **often fast enough** for most tasks!

These operations include, but are not limited to:

#### 1. Standard Mathematical Operations

- Arithmetic: `df['total'] = df['price'] * df['quantity']`
- Normalization: `df_norm = (df - df.mean()) / df.std()`
- Scaling: `df['scaled'] = df['value'] / df['value'].max()`

#### 2. Statistical Computations

- Aggregations: `df.mean()`, `df.sum()`, `df.std()`
- Correlations: `df.corr()`, `df['A'].corr(df['B'])`
- Rolling windows: `df['MA'] = df['price'].rolling(7).mean()`

#### 3. Element-wise Transformations

- Vectorized operations: `df['log_val'] = np.log(df['value'])`
- Conditional logic: `df['flag'] = df['amount'] > 100`
- String operations: `df['upper'] = df['name'].str.upper()`

Let's see this in action:

```
print(" STAYING IN PANDAS: When Conversion Isn't Needed")
print("=" * 60)

Example: Normalize data WITHOUT converting to NumPy
df_data = pd.DataFrame({
 "Sales_Q1": [10000, 20000, 30000, 40000],
 "Sales_Q2": [12000, 22000, 28000, 38000],
 "Sales_Q3": [15000, 25000, 35000, 45000]
}, index=["Product_A", "Product_B", "Product_C", "Product_D"])

print("Original Sales Data:")
print(df_data)
print(f"\nDtypes: {df_data.dtypes.unique()}")

METHOD 1: Using NumPy (requires metadata handling)
print("\n" + "="*60)
print(" Method 1: NumPy Approach (more complex)")
```

```

print("="*60)

Save metadata
saved_index = df_data.index
saved_columns = df_data.columns

Convert to NumPy
arr = df_data.to_numpy()
print(f"1. Converted to NumPy array (shape: {arr.shape})")

Normalize
arr_norm = (arr - arr.mean(axis=0)) / arr.std(axis=0)
print(f"2. Normalized using NumPy")

Convert back and restore metadata
df_norm_numpy = pd.DataFrame(arr_norm, index=saved_index, columns=saved_columns)
print(f"3. Converted back to DataFrame with manual metadata restoration")
print(f"\nResult:")
print(df_norm_numpy.round(3))

METHOD 2: Pure Pandas (simpler and automatic!)
print("\n" + "="*60)
print(" Method 2: Pandas Approach (simpler!)")
print("="*60)

df_norm_pandas = (df_data - df_data.mean()) / df_data.std()
print(f"Result (metadata preserved automatically):")
print(df_norm_pandas.round(3))

Verify they're the same
print(f"\n Results identical? {np.allclose(df_norm_numpy.values, df_norm_pandas.values)}")
print(f" Metadata preserved? {df_norm_pandas.index.equals(df_data.index) and df_norm_pandas.

print("\n Takeaway: For standard operations, pandas is simpler and just as fast!")

```

#### STAYING IN PANDAS: When Conversion Isn't Needed

=====

Original Sales Data:

|           | Sales_Q1 | Sales_Q2 | Sales_Q3 |
|-----------|----------|----------|----------|
| Product_A | 10000    | 12000    | 15000    |
| Product_B | 20000    | 22000    | 25000    |
| Product_C | 30000    | 28000    | 35000    |

```
Product_D 40000 38000 45000
```

```
Dtypes: [dtype('int64')]
```

```
=====
Method 1: NumPy Approach (more complex)
=====
```

1. Converted to NumPy array (shape: (4, 3))
2. Normalized using NumPy
3. Converted back to DataFrame with manual metadata restoration

Result:

|           | Sales_Q1 | Sales_Q2 | Sales_Q3 |
|-----------|----------|----------|----------|
| Product_A | -1.342   | -1.378   | -1.342   |
| Product_B | -0.447   | -0.318   | -0.447   |
| Product_C | 0.447    | 0.318    | 0.447    |
| Product_D | 1.342    | 1.378    | 1.342    |

```
=====
Method 2: Pandas Approach (simpler!)
=====
```

Result (metadata preserved automatically):

|           | Sales_Q1 | Sales_Q2 | Sales_Q3 |
|-----------|----------|----------|----------|
| Product_A | -1.162   | -1.193   | -1.162   |
| Product_B | -0.387   | -0.275   | -0.387   |
| Product_C | 0.387    | 0.275    | 0.387    |
| Product_D | 1.162    | 1.193    | 1.162    |

Results identical? False

Metadata preserved? True

Takeaway: For standard operations, pandas is simpler and just as fast!

### 10.4.1.2 Convert to NumPy When You Must

Convert your DataFrame to a **NumPy array** only in specific scenarios where Pandas alone isn't enough:

#### 1. Specialized NumPy Functions

- Linear algebra: `np.linalg.inv()`, `np.linalg.eig()`, matrix decomposition
- FFT/signal processing: `np.fft.fft()`, `np.convolve()`
- Advanced random sampling: `np.random.multivariate_normal()`

- Custom mathematical operations not available in pandas

## 2. ML Library Integration

- scikit-learn: `model.fit(X_train, y_train)` expects NumPy arrays
- TensorFlow/PyTorch: Neural networks require tensor/array inputs
- SciPy: Scientific functions expect NumPy arrays

## 3. Performance-Critical Code

- Nested loops that can be vectorized with NumPy broadcasting
- Custom algorithms requiring low-level array manipulation
- Very large datasets (> 10 million rows) where memory matters

## 4. Complex Conditional Logic

- Multi-condition selections with `np.where()`, `np.select()`
- Better performance than chained pandas operations

Let's see when NumPy is actually necessary:

```
print(" WHEN NUMPY IS NECESSARY: Real Use Cases")
print("=" * 60)

Create a realistic dataset
np.random.seed(42)
n_rows = 50000

performance_data = pd.DataFrame({
 'customer_id': range(1, n_rows + 1),
 'purchase_amount': np.random.uniform(10, 1000, n_rows),
 'discount_pct': np.random.choice([0, 5, 10, 15, 20], n_rows),
 'category': np.random.choice(['A', 'B', 'C', 'D'], n_rows),
 'rating': np.random.randint(1, 6, n_rows),
 'is_premium': np.random.choice([True, False], n_rows)
})

print(f" Dataset: {performance_data.shape[0]:,} rows × {performance_data.shape[1]} columns")
print(f"First 3 rows:")
print(performance_data.head(3))
```

```
WHEN NUMPY IS NECESSARY: Real Use Cases
```

```
=====
Dataset: 50,000 rows × 6 columns
```



First 3 rows:

|   | customer_id | purchase_amount | discount_pct | category | rating | is_premium |
|---|-------------|-----------------|--------------|----------|--------|------------|
| 0 | 1           | 380.794718      | 0            | C        | 1      | True       |
| 1 | 2           | 951.207163      | 15           | D        | 1      | False      |
| 2 | 3           | 734.674002      | 5            | D        | 1      | True       |

```
print("\n" + "="*60)
print("USE CASE 1: Complex Multi-Condition Logic with np.select()")
print("="*60)

def customer_segment_function(row):
 """Complex business logic - slow with apply()"""
 if row['is_premium'] and row['purchase_amount'] > 500:
 return 'VIP'
 elif row['is_premium'] and row['rating'] >= 4:
 return 'Premium+'
 elif row['purchase_amount'] > 200 and row['rating'] >= 4:
 return 'High Value'
 elif row['purchase_amount'] > 100:
 return 'Standard'
 else:
 return 'Basic'

Method 1: APPLY (Easy to write but slow)
print("\n Method 1: Using apply() with function")
start_time = time.time()
performance_data['segment_apply'] = performance_data.apply(customer_segment_function, axis=1)
apply_time = time.time() - start_time
print(f"Time: {apply_time:.4f}s")

Method 2: VECTORIZED with np.select() (Faster!)
print("\n Method 2: NumPy np.select() - vectorized")
start_time = time.time()

Define conditions using NumPy/pandas vectorized operations
conditions = [
 (performance_data['is_premium'] & (performance_data['purchase_amount'] > 500)),
 (performance_data['is_premium'] & (performance_data['rating'] >= 4)),
 ((performance_data['purchase_amount'] > 200) & (performance_data['rating'] >= 4)),
 (performance_data['purchase_amount'] > 100)
]
choices = ['VIP', 'Premium+', 'High Value', 'Standard']
```

```

np.select() evaluates conditions vectorized and picks corresponding choice
performance_data['segment_vectorized'] = np.select(conditions, choices, default='Basic')
vectorized_time = time.time() - start_time
print(f"Time: {vectorized_time:.4f}s")

Performance comparison
speedup = apply_time / vectorized_time if vectorized_time > 0 else float('inf')
print(f"\n Speedup: {speedup:.1f}x faster with np.select()!")
print(f"Time saved: {((apply_time - vectorized_time) / apply_time * 100):.1f}%")

Verify results match
results_match = (performance_data['segment_apply'] == performance_data['segment_vectorized'])
print(f" Results identical: {results_match}")

Show distribution
print(f"\n Customer Segment Distribution:")
segment_counts = performance_data['segment_apply'].value_counts().sort_index()
for segment, count in segment_counts.items():
 print(f" {segment}: {count:,} ({count/len(performance_data)*100:.1f}%")

Clean up
performance_data.drop(['segment_vectorized'], axis=1, inplace=True)

```

```

=====
USE CASE 1: Complex Multi-Condition Logic with np.select()
=====

```

Method 1: Using apply() with function  
Time: 0.3499s

Method 2: NumPy np.select() - vectorized  
Time: 0.0124s

Speedup: 28.2x faster with np.select()!  
Time saved: 96.5%  
Results identical: True

Customer Segment Distribution:  
Basic: 3,648 (7.3%)  
High Value: 8,116 (16.2%)

```

Premium+: 4,967 (9.9%)
Standard: 20,606 (41.2%)
VIP: 12,663 (25.3%)
Time: 0.3499s

Method 2: NumPy np.select() - vectorized
Time: 0.0124s

Speedup: 28.2x faster with np.select()!
Time saved: 96.5%
Results identical: True

Customer Segment Distribution:
Basic: 3,648 (7.3%)
High Value: 8,116 (16.2%)
Premium+: 4,967 (9.9%)
Standard: 20,606 (41.2%)
VIP: 12,663 (25.3%)

```

### 10.4.1.3 Quick Decision Guide

#### Summary Table

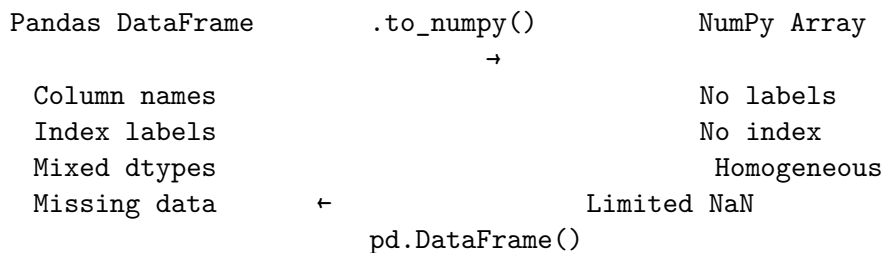
| Operation Type              | Pandas   | NumPy                              | Recommendation        |
|-----------------------------|----------|------------------------------------|-----------------------|
| Arithmetic operations       | Fast     | Faster                             | Use Pandas (simpler)  |
| Aggregations (sum, mean)    | Fast     | Faster                             | Use Pandas (metadata) |
| String operations           | Built-in | Manual                             | Use Pandas            |
| DateTime operations         | Built-in | Manual                             | Use Pandas            |
| Linear algebra              | Limited  |                                    | Use NumPy             |
| ML library input            | Convert  | Comprehensive<br>Native            | Convert to NumPy      |
| Complex conditionals        | Slow     | Fast<br>( <code>np.select</code> ) | Use NumPy             |
| Large datasets (> 10M rows) | Slower   | Faster                             | Use NumPy             |

Now let's learn the actual conversion methods!

## 10.5 Conversion Methods: Pandas ↔ NumPy

Now that you know **when** to convert, let's learn **how** to convert between pandas and NumPy.

### 10.5.1 The Two-Way Street



**Challenge:** Converting a DataFrame to a NumPy array **drops all metadata**:

- Column names are lost
- Index labels are removed
- Data type information is simplified or lost
- Categorical mappings are gone

**Why this matters:** After performing NumPy computations, you often need to convert results back to a DataFrame with the original structure for analysis, visualization, or export.

**Solution:** Save & Restore Metadata Manually

### 10.5.2 Save & Restore Metadata Manually

The most straightforward approach: **save metadata before conversion**, then **restore it after computation**.

```
print(" Manual Metadata Preservation")
print("=" * 50)

Create a DataFrame with rich metadata
df_original = pd.DataFrame(
 {
 "Temperature": [72.5, 68.3, 75.1, 71.9],
```

```

 "Humidity": [45, 52, 48, 50],
 "WindSpeed": [12.3, 8.7, 15.2, 10.1]
 },
 index=["Monday", "Tuesday", "Wednesday", "Thursday"]
)

print("Original DataFrame:")
print(df_original)
print(f"\nIndex: {df_original.index.tolist()}")
print(f"Columns: {df_original.columns.tolist()}")
print(f"Dtypes:\n{df_original.dtypes}")

STEP 1: Save metadata BEFORE converting to NumPy
saved_index = df_original.index.copy()
saved_columns = df_original.columns.copy()
saved_dtypes = df_original.dtypes.to_dict()

print("\n Metadata saved!")

STEP 2: Convert to NumPy for computation
arr = df_original.to_numpy()

print(f"\nNumPy array (metadata lost):")
print(arr)
print(f"Shape: {arr.shape}, dtype: {arr.dtype}")

STEP 3: Perform computations (example: normalize values)
Subtract mean and divide by std for each column
arr_normalized = (arr - arr.mean(axis=0)) / arr.std(axis=0)

print(f"\nNormalized array (still no metadata):")
print(arr_normalized)

STEP 4: Restore metadata when converting back to DataFrame
df_result = pd.DataFrame(
 arr_normalized,
 index=saved_index,
 columns=saved_columns
)

print(f"\n Reconstructed DataFrame with metadata:")
print(df_result)

```

```
print(f"\nIndex restored: {df_result.index.tolist()}")
print(f"Columns restored: {df_result.columns.tolist()}")
print(f"Dtypes restored (may differ due to normalization):\n{df_result.dtypes}")
```

#### STRATEGY 1: Manual Metadata Preservation

Original DataFrame:

|           | Temperature | Humidity | WindSpeed |
|-----------|-------------|----------|-----------|
| Monday    | 72.5        | 45       | 12.3      |
| Tuesday   | 68.3        | 52       | 8.7       |
| Wednesday | 75.1        | 48       | 15.2      |
| Thursday  | 71.9        | 50       | 10.1      |

Index: ['Monday', 'Tuesday', 'Wednesday', 'Thursday']

Columns: ['Temperature', 'Humidity', 'WindSpeed']

Dtypes:

Temperature float64

Humidity int64

WindSpeed float64

dtype: object

Metadata saved!

NumPy array (metadata lost):

```
[[72.5 45. 12.3]
 [68.3 52. 8.7]
 [75.1 48. 15.2]
 [71.9 50. 10.1]]
```

Shape: (4, 3), dtype: float64

Normalized array (still no metadata):

```
[[0.22667166 -1.45010473 0.29531936]
 [-1.50427558 1.25675744 -1.171094]
 [1.29821043 -0.29002095 1.47659679]
 [-0.02060651 0.48336824 -0.60082214]]
```

Reconstructed DataFrame with metadata:

|           | Temperature | Humidity  | WindSpeed |
|-----------|-------------|-----------|-----------|
| Monday    | 0.226672    | -1.450105 | 0.295319  |
| Tuesday   | -1.504276   | 1.256757  | -1.171094 |
| Wednesday | 1.298210    | -0.290021 | 1.476597  |
| Thursday  | -0.020607   | 0.483368  | -0.600822 |

```
Index restored: ['Monday', 'Tuesday', 'Wednesday', 'Thursday']
Columns restored: ['Temperature', 'Humidity', 'WindSpeed']
Dtypes restored (may differ due to normalization):
Temperature float64
Humidity float64
WindSpeed float64
dtype: object
```

### 10.5.3 Pandas DataFrame → NumPy Array

**When to convert:** You need NumPy's computational speed and mathematical functions.

**Common scenarios:**

- Performance-critical computations (linear algebra, statistics)
- Integration with scientific computing libraries (SciPy, scikit-learn)
- Machine learning algorithms that expect NumPy arrays
- Mathematical operations not available in pandas

#### 10.5.3.1 Conversion Methods & Performance

Let's create a pandas DataFrame to demonstrate the conversion:

```
print(" PANDAS TO NUMPY CONVERSION")
print("=" * 40)

Start with a DataFrame of tech stock prices
stock_data = pd.DataFrame(
 {
 "AAPL": [150.0, 152.5, 148.2, 155.1],
 "GOOGL": [2800.0, 2750.0, 2825.0, 2900.0],
 "MSFT": [300.0, 305.0, 298.5, 310.0],
 "TSLA": [800.0, 795.0, 820.0, 815.0],
 },
 index=["Day 1", "Day 2", "Day 3", "Day 4"]
)

print(" Original DataFrame:")
print(stock_data)
print(f"\nShape: {stock_data.shape}")
print("Dtypes:")
```

```
print(stock_data.dtypes)
print(f"Memory usage (deep): {stock_data.memory_usage(deep=True).sum()} bytes")
```

#### PANDAS TO NUMPY CONVERSION

```
=====
```

Original DataFrame:

|       | AAPL  | GOOGL  | MSFT  | TSLA  |
|-------|-------|--------|-------|-------|
| Day 1 | 150.0 | 2800.0 | 300.0 | 800.0 |
| Day 2 | 152.5 | 2750.0 | 305.0 | 795.0 |
| Day 3 | 148.2 | 2825.0 | 298.5 | 820.0 |
| Day 4 | 155.1 | 2900.0 | 310.0 | 815.0 |

Shape: (4, 4)

Dtypes:

AAPL float64

GOOGL float64

MSFT float64

TSLA float64

dtype: object

Memory usage (deep): 344 bytes

##### 10.5.3.1.1 Method 1: `.to_numpy()` (Recommended )

The **preferred modern approach** for converting DataFrames to NumPy arrays. This method:

- Converts DataFrame data into a NumPy array
- **Discards** index and column labels (keeps only values)
- Preserves the **same row and column order** as the original DataFrame
- Returns a clean 2D array ready for numerical computations

```
Method 1: .to_numpy() - The modern, recommended approach
print("\n Method 1 - .to_numpy() (recommended):")
array_modern = stock_data.to_numpy()

print("Converted NumPy array:")
print(array_modern)
print(f"\nShape: {array_modern.shape}")
print(f"Data type: {array_modern.dtype}")
print(f"\n Note: Row indices (Day 1, Day 2...) and column names (AAPL, GOOGL...) are lost!")
```



```
Method 1 - .to_numpy() (recommended):
Shape: (4, 4)
Data type: float64
```

```
array([[150. , 2800. , 300. , 800.],
 [152.5, 2750. , 305. , 795.],
 [148.2, 2825. , 298.5, 820.],
 [155.1, 2900. , 310. , 815.]])
```

**Key Point:** Notice how the resulting array contains **only the numerical values**. All the metadata (index labels like “Day 1”, column names like “AAPL”) is removed. This is what makes NumPy arrays fast but less descriptive than DataFrames.

#### 10.5.3.1.2 Method 2: .values (legacy but still works)

An **older method** that still works but is **not recommended** for new code. It behaves similarly to `.to_numpy()` but can produce unexpected results with certain pandas dtypes.

```
Method 2: .values - Legacy approach (avoid in new code)
print("\n Method 2 - .values (legacy):")
array_legacy = stock_data.values

print("Converted array using .values:")
print(array_legacy)
print(f"\nShape: {array_legacy.shape}")
print(f"Data type: {array_legacy.dtype}")

Verify both methods produce same result (in this case)
print(f"\nSame result as .to_numpy()? {np.array_equal(array_modern, array_legacy)}")
print(" For simple numeric DataFrames, both methods work identically")
```

```
Method 2 - .values (legacy):
[[150. 2800. 300. 800.]
 [152.5 2750. 305. 795.]
 [148.2 2825. 298.5 820.]
 [155.1 2900. 310. 815.]]
Shape: (4, 4)
Same result as .to_numpy()? True
```

### Why avoid `.values`?

- Less clear intent (what does “values” mean?)
- Can behave unpredictably with pandas extension dtypes (nullable integers, strings, etc.)
- Not officially recommended in pandas documentation
- `.to_numpy()` is more explicit and future-proof

#### 10.5.3.1.3 Method 3: Force a specific dtype (e.g., `int32`)

```
Method 3: Force a specific dtype during conversion
print("\n Method 3 - Force data type (int32):")
array_int = stock_data.to_numpy(dtype=np.int32)

print("Array with forced int32 dtype:")
print(array_int)
print(f"\nOriginal dtype: {stock_data.to_numpy().dtype}")
print(f"Forced dtype: {array_int.dtype}")
print(f"\n Warning: Float values were truncated (not rounded) to integers!")
print(f"Example: 150.0 → {array_int[0, 0]}, but 152.5 → {array_int[1, 0]} (loses 0.5)")
```

```
Method 3 - Force data type (int32):
[[150 2800 300 800]
 [152 2750 305 795]
 [148 2825 298 820]
 [155 2900 310 815]]
Shape: (4, 4)
Data type: int32
Preview (first 2 rows):
[[150 2800 300 800]
 [152 2750 305 795]]
```

### When to use `dtype` parameter:

#### Good use cases:

- Ensuring consistent data types for mathematical operations
- Reducing memory usage (e.g., `float64` → `float32`)
- Preparing data for ML libraries that require specific types
- Converting to integer types when you know there are no decimals

#### Be careful:

- Data loss when converting float to int (truncation, not rounding)
- May raise errors if conversion is impossible (e.g., strings to numbers)
- Check for NaN values before converting to integer types

#### 10.5.3.1.4 Method 4: Convert specific columns only

Why select specific columns?

- **Performance:** Process only needed data
- **Memory:** Smaller arrays use less RAM
- **Clarity:** Makes your intent explicit
- **Debugging:** Easier to track which data is being used

```
Method 4: Select and convert specific columns
print("\n Method 4 - Specific columns only (AAPL, MSFT):")

Select only AAPL and MSFT columns, then convert
tech_stocks = stock_data[["AAPL", "MSFT"]].to_numpy()

print("Selected columns converted to array:")
print(tech_stocks)
print(f"\nOriginal DataFrame shape: {stock_data.shape} (4 rows × 4 columns)")
print(f"Selected array shape: {tech_stocks.shape} (4 rows × 2 columns)")
print(f>Data type: {tech_stocks.dtype}")
print(f"\n Memory saved: {(stock_data.shape[1] - tech_stocks.shape[1]) / stock_data.shape[1]}
```

```
Method 4 - Specific columns only (AAPL, MSFT):
[[150. 300.]
 [152.5 305.]
 [148.2 298.5]
 [155.1 310.]]
Shape: (4, 2)
Data type: float64
```

Summary table

| Method                   | Recommendation | Pros                                   | Cons                                              |
|--------------------------|----------------|----------------------------------------|---------------------------------------------------|
| <code>.to_numpy()</code> | Use by default | Clear intent, stable API, future-proof | May copy data depending on dtypes/backing storage |

| Method                            | Recommendation               | Pros                                            | Cons                                                    |
|-----------------------------------|------------------------------|-------------------------------------------------|---------------------------------------------------------|
| <code>.to_numpy(dtype=...)</code> | <b>When you need a dtype</b> | Explicit, consistent numeric types; easier math | Casting may be lossy or fail; extra param to manage     |
| <code>.values</code>              | <b>Legacy / avoid</b>        | Works on old pandas; quick                      | Ambiguous with mixed/extension dtypes; not future-proof |

### Best practices

- **Convert only needed columns** to reduce memory and copying.
- **Specify dtype** when downstream computations require consistent numeric types.
- **Check for missing values** (NaN, NaT) before casting to integer dtypes.
- **Prefer `.to_numpy()` over `.values`** for clarity and forward compatibility.

## 10.5.4 NumPy Array → Pandas DataFrame

Let's create an NumPy array to demonstrate the conversion:

```
COMPREHENSIVE ARRAY → DATAFRAME CONVERSION
print(" NUMPY TO PANDAS CONVERSION")
print("=" * 40)

Original NumPy array
scores = np.array([
 [85, 92, 78, 95],
 [88, 76, 91, 82],
 [95, 89, 84, 90]
])

students = ['Alice', 'Bob', 'Carol']
subjects = ['Math', 'Science', 'English', 'History']

print("Original NumPy array:")
print(scores)
print(f"Shape: {scores.shape}")
```

```
NUMPY TO PANDAS CONVERSION
=====
Original NumPy array:
[[85 92 78 95]
```

```
[88 76 91 82]
[95 89 84 90]]
Shape: (3, 4)
```

#### 10.5.4.1 Method 1: Basic Conversion

Create a DataFrame directly from a NumPy array using `pd.DataFrame()`. By default, pandas will auto-generate integer labels (0, 1, 2, ...) for both rows and columns.

```
Basic conversion - auto-generated index and column labels
df_basic = pd.DataFrame(scores)
print("\n Basic DataFrame (auto index/columns):")
df_basic
```

Basic DataFrame (auto index/columns):

|   | 0  | 1  | 2  | 3  |
|---|----|----|----|----|
| 0 | 85 | 92 | 78 | 95 |
| 1 | 88 | 76 | 91 | 82 |
| 2 | 95 | 89 | 84 | 90 |

Notice that the row indices are [0, 1, 2] and column names are [0, 1, 2, 3] — these are automatically generated when no labels are specified.

#### 10.5.4.2 Method 2: Custom Row and Column Labels

For better readability and data analysis, you can specify meaningful names for rows (`index`) and columns when creating the DataFrame.

```
Custom labels for better data interpretation
df_labeled = pd.DataFrame(scores, index=students, columns=subjects)
print("\n Labeled DataFrame (custom index & columns):")
df_labeled
```

Labeled DataFrame (custom index & columns):

|       | Math | Science | English | History |
|-------|------|---------|---------|---------|
| Alice | 85   | 92      | 78      | 95      |
| Bob   | 88   | 76      | 91      | 82      |
| Carol | 95   | 89      | 84      | 90      |

Now the DataFrame has meaningful labels: - **Row indices:** Student names (Alice, Bob, Carol)  
- **Column names:** Subject names (Math, Science, English, History)

This makes the data much more interpretable and easier to work with!

### 10.5.4.3 Key Parameters & Options

When converting NumPy arrays to DataFrames, you can customize the output using these parameters:

| Parameter | Purpose                | Example                              | Default                     |
|-----------|------------------------|--------------------------------------|-----------------------------|
| data      | NumPy array to convert | <code>np.array([[1,2],[3,4]])</code> | Required                    |
| columns   | Column labels          | <code>['A', 'B']</code>              | <code>[0, 1, 2, ...]</code> |
| index     | Row labels             | <code>['row1', 'row2']</code>        | <code>[0, 1, 2, ...]</code> |

#### Best Practices:

- **Always specify meaningful column names** for better code readability and maintenance
- **Use descriptive index names** when your rows represent specific entities (people, dates, etc.)
- **Check data types and index** after conversion with `df.dtypes` to ensure correct interpretation

## 10.6 Random Number Generation in NumPy

Random number generation is a **fundamental tool** in data science, used for:

- **Simulations:** Monte Carlo methods, risk analysis, game theory
- **Statistical Sampling:** Bootstrap, cross-validation, hypothesis testing
- **Machine Learning:** Data augmentation, weight initialization, dropout
- **Scientific Computing:** Stochastic modeling, numerical experiments

NumPy's `random` module provides **fast, vectorized** random number generation that's **orders of magnitude faster** than Python's built-in `random` module.

### 10.6.1 Why NumPy Random Over Python Random?

| Aspect               | Python <code>random</code> | NumPy <code>random</code> |
|----------------------|----------------------------|---------------------------|
| <b>Speed</b>         | Slow (one at a time)       | Fast (vectorized)         |
| <b>Output</b>        | Single values              | Arrays of any shape       |
| <b>Distributions</b> | Limited                    | 40+ distributions         |
| <b>Use Case</b>      | Simple scripts             | Scientific computing      |
| <b>Performance</b>   | ~0.1M numbers/sec          | ~10M+ numbers/sec         |

**Rule:** Use NumPy `random` for data science; use Python `random` for simple scripts.

### 10.6.2 Core Random Number Functions

NumPy provides functions for various probability distributions. Let's explore the most commonly used ones with practical examples.

#### 10.6.2.1 Comparison Table: Which Function to Use?

| Function                | Distribution                    | Parameters           | Use Case                           |
|-------------------------|---------------------------------|----------------------|------------------------------------|
| <code>rand()</code>     | Uniform [0, 1)                  | shape                | Quick random arrays, probabilities |
| <code>randn()</code>    | Normal ( $\mu=0$ , $\sigma=1$ ) | shape                | Standard normal samples            |
| <code>randint()</code>  | Discrete uniform                | low, high, size      | Random integers, sampling          |
| <code>choice()</code>   | Sample from array               | array, size, replace | Random selection, bootstrapping    |
| <code>uniform()</code>  | Uniform [a, b]                  | low, high, size      | Custom range uniform               |
| <code>normal()</code>   | Normal ( $\mu$ , $\sigma$ )     | mean, std, size      | Real-world measurements            |
| <code>binomial()</code> | Binomial                        | n, p, size           | Coin flips, success/failure        |

| Function               | Distribution | Parameters                              | Use Case               |
|------------------------|--------------|-----------------------------------------|------------------------|
| <code>poisson()</code> | Poisson      | <code>lambda</code> , <code>size</code> | Event counts, arrivals |

Let's see each in action with real examples!

### 10.6.3 Uniform Distribution: `np.random.rand()` and `np.random.uniform()`

**Uniform distribution:** All values in a range are equally likely.

#### 10.6.3.1 `np.random.rand()` - Quick Uniform [0, 1)

```
print(" UNIFORM DISTRIBUTION: np.random.rand()")
print("=" * 60)

Generate a 2x3 array of random values between 0 and 1
rand_array = np.random.rand(2, 3)
print("2x3 array of uniform random numbers [0, 1):")
print(rand_array)
print(f"\nShape: {rand_array.shape}")
print(f"Min: {rand_array.min():.4f}, Max: {rand_array.max():.4f}")
print(f"Mean: {rand_array.mean():.4f} (expected ~0.5)")

Practical use: Generate random probabilities
print("\n Practical Use: Simulate coin flip probabilities")
probabilities = np.random.rand(10)
results = ['Heads' if p > 0.5 else 'Tails' for p in probabilities]
print(f"Probabilities: {probabilities.round(3)}")
print(f"Results: {results}")
```

#### 10.6.3.2 `np.random.uniform()` - Custom Range Uniform [a, b]

More flexible - specify your own range.

```
print("\n UNIFORM DISTRIBUTION: np.random.uniform()")
print("=" * 60)

Generate random numbers in a custom range
```



```

print("Example 1: Random temperatures between 60°F and 90°F")
temperatures = np.random.uniform(60, 90, size=7)
days = ['Mon', 'Tue', 'Wed', 'Thu', 'Fri', 'Sat', 'Sun']
for day, temp in zip(days, temperatures):
 print(f" {day}: {temp:.1f}°F")

2D array example
print("\nExample 2: 3×5 matrix of random prices between $10 and $100")
prices = np.random.uniform(10, 100, size=(3, 5))
print(prices.round(2))
print(f"\nAverage price: ${prices.mean():.2f}")

```

```

[[0.5376299 0.42818625 -0.2799933]
 [-1.27903074 0.51529432 -0.83428181]
 [2.18409673 0.57050721 -0.5806483]]

```

#### 10.6.4 Normal (Gaussian) Distribution: `np.random.randn()` and `np.random.normal()`

**Normal distribution:** Bell-shaped curve, most values cluster around the mean.  
Used for: heights, weights, test scores, measurement errors, etc.

##### 10.6.4.1 `np.random.randn()` - Standard Normal ( $\mu=0$ , $\sigma=1$ )

```

print(" NORMAL DISTRIBUTION: np.random.randn()")
print("=" * 60)

Generate standard normal distribution
normal_array = np.random.randn(1000)

print(f"Generated {len(normal_array)} samples from standard normal distribution")
print(f"Mean: {normal_array.mean():.4f} (expected: 0)")
print(f"Std Dev: {normal_array.std():.4f} (expected: 1)")
print(f"Min: {normal_array.min():.2f}, Max: {normal_array.max():.2f}")

Show distribution statistics
print("\n Distribution check (68-95-99.7 rule):")
within_1std = np.sum(np.abs(normal_array) <= 1) / len(normal_array) * 100
within_2std = np.sum(np.abs(normal_array) <= 2) / len(normal_array) * 100

```

```

within_3std = np.sum(np.abs(normal_array) <= 3) / len(normal_array) * 100

print(f" Within ±1 : {within_1std:.1f}% (expected: ~68%)")
print(f" Within ±2 : {within_2std:.1f}% (expected: ~95%)")
print(f" Within ±3 : {within_3std:.1f}% (expected: ~99.7%)")

```

#### 10.6.4.2 np.random.normal() - Custom Normal ( , )

Specify your own mean and standard deviation.

```

print("\n NORMAL DISTRIBUTION: np.random.normal()")
print("=" * 60)

Example 1: Student exam scores (mean=75, std=10)
print("Example 1: Simulate exam scores (=75, =10)")
exam_scores = np.random.normal(75, 10, size=20)
print(f"Scores: {exam_scores.round(1)}")
print(f"Class average: {exam_scores.mean():.1f}")
print(f"Passing (60): {np.sum(exam_scores >= 60)}/{len(exam_scores)}")

Example 2: Heights in inches (mean=68, std=3)
print("\nExample 2: Adult heights in inches (=68\", =3\")")
heights = np.random.normal(68, 3, size=100)
print(f"Generated {len(heights)} height measurements")
print(f"Average height: {heights.mean():.2f}\")")
print(f"Tallest: {heights.max():.1f}\", Shortest: {heights.min():.1f}\")")
print(f"Between 65\" and 71\": {np.sum((heights >= 65) & (heights <= 71))}")

```

#### 10.6.5 Integer Random Numbers: np.random.randint()

Generate random **integers** within a specific range.

**Use cases:** Dice rolls, random IDs, sampling indices, game mechanics

```

print(" RANDOM INTEGERS: np.random.randint()")
print("=" * 60)

Example 1: Dice rolls
print("Example 1: Roll a six-sided die 20 times")
dice_rolls = np.random.randint(1, 7, size=20) # 1 to 6 inclusive

```

```

print(f"Rolls: {dice_rolls}")
print(f"Distribution: {dict(zip(*np.unique(dice_rolls, return_counts=True)))}")

Example 2: Random matrix of integers
print("\nExample 2: 4x4 matrix of random integers [10, 20)")
int_matrix = np.random.randint(10, 20, size=(4, 4))
print(int_matrix)
print(f"Sum: {int_matrix.sum()}, Average: {int_matrix.mean():.2f}")

Example 3: Random customer IDs
print("\nExample 3: Generate 10 random customer IDs [1000, 9999]")
customer_ids = np.random.randint(1000, 10000, size=10)
print(f"IDs: {customer_ids}")

```

### 10.6.6 Random Selection: `np.random.choice()`

**Randomly select** elements from an array with or without replacement.

**Use cases:** Bootstrapping, random sampling, A/B testing, lottery

```

print(" RANDOM SELECTION: np.random.choice()")
print("=" * 60)

Example 1: Random selection WITH replacement
print("Example 1: Select 5 numbers WITH replacement [1-5]")
choice_array = np.random.choice([1, 2, 3, 4, 5], size=10, replace=True)
print(f"Selected: {choice_array}")
print(f"Notice: Numbers can repeat!")

Example 2: Random selection WITHOUT replacement
print("\nExample 2: Select 3 winners from 10 contestants (no duplicates)")
contestants = np.array(['Alice', 'Bob', 'Carol', 'David', 'Eve',
 'Frank', 'Grace', 'Henry', 'Iris', 'Jack'])
winners = np.random.choice(contestants, size=3, replace=False)
print(f"Winners: {winners}")

Example 3: Weighted random choice (probabilities)
print("\nExample 3: Biased coin flip (70% Heads, 30% Tails)")
outcomes = ['Heads', 'Tails']
probabilities = [0.7, 0.3]
flips = np.random.choice(outcomes, size=100, p=probabilities)
unique, counts = np.unique(flips, return_counts=True)

```

```

for outcome, count in zip(unique, counts):
 print(f" {outcome}: {count}/100 ({count}%)")

Example 4: Bootstrap sampling
print("\nExample 4: Bootstrap sampling (sampling with replacement)")
data = np.array([23, 45, 67, 34, 89, 12, 56])
bootstrap_sample = np.random.choice(data, size=len(data), replace=True)
print(f"Original: {data}")
print(f"Bootstrap: {bootstrap_sample}")
print(f"Bootstrap mean: {bootstrap_sample.mean():.2f}, Original mean: {data.mean():.2f}")

```

## 10.6.7 Other Useful Distributions

NumPy provides many more distributions for specialized use cases.

```

print(" OTHER USEFUL DISTRIBUTIONS")
print("=" * 60)

1. Binomial: Number of successes in n trials
print("\n1. BINOMIAL: Coin flips (10 flips, 50% probability)")
coin_flips = np.random.binomial(n=10, p=0.5, size=1000)
print(f" Average heads in 10 flips: {coin_flips.mean():.2f} (expected: 5)")

2. Poisson: Count of events in fixed interval
print("\n2. POISSON: Customer arrivals (=3 per hour)")
arrivals = np.random.poisson(lam=3, size=24) # 24 hours
print(f" Arrivals per hour: {arrivals}")
print(f" Total customers: {arrivals.sum()}, Average: {arrivals.mean():.2f}")

3. Exponential: Time between events
print("\n3. EXPONENTIAL: Time between customer arrivals (scale=5 min)")
wait_times = np.random.exponential(scale=5, size=10)
print(f" Wait times (minutes): {wait_times.round(1)}")
print(f" Average wait: {wait_times.mean():.2f} min")

4. Beta: Probabilities and proportions
print("\n4. BETA: Probability distribution (=2, =5)")
probabilities = np.random.beta(2, 5, size=1000)
print(f" Mean probability: {probabilities.mean():.3f}")
print(f" Range: [{probabilities.min():.3f}, {probabilities.max():.3f}]")

```

### 10.6.8 Reproducibility: Random Seeds

**Problem:** Random numbers are different every time you run your code!

**Solution:** Set a **seed** for reproducible results.

### 10.6.9 Performance: NumPy vs Python Random

Let's prove that NumPy random is **dramatically faster** than Python's built-in random.

```
import random as py_random

print(" PERFORMANCE COMPARISON: NumPy vs Python random")
print("=" * 60)

n_numbers = 1_000_000

Python random (slow)
print(f"\n Python random module (generating {n_numbers:,} numbers):")
start_time = time.time()
python_randoms = [py_random.random() for _ in range(n_numbers)]
python_time = time.time() - start_time
print(f" Time: {python_time:.4f} seconds")

NumPy random (fast)
print(f"\n NumPy random module (generating {n_numbers:,} numbers):")
start_time = time.time()
numpy_randoms = np.random.rand(n_numbers)
numpy_time = time.time() - start_time
print(f" Time: {numpy_time:.4f} seconds")

Comparison
speedup = python_time / numpy_time if numpy_time > 0 else float('inf')
print(f"\n RESULT:")
print(f" NumPy is {speedup:.1f}x faster!")
print(f" Time saved: {(python_time - numpy_time):.4f} seconds ({(python_time - numpy_time):.1%})")

print("\n Key Takeaway:")
print(" Use NumPy random for any serious data science work!")
```

```
PERFORMANCE COMPARISON: NumPy vs Python random
=====
```

Python random module (generating 1,000,000 numbers):  
Time: 0.1046 seconds

NumPy random module (generating 1,000,000 numbers):  
Time: 0.0061 seconds

RESULT:  
NumPy is 17.3x faster!  
Time saved: 0.0986 seconds (94.2%)

Key Takeaway:  
Use NumPy random for any serious data science work!

### 10.6.10 Quick Reference Guide

```
Uniform [0, 1)
np.random.rand(5) # 1D array of 5 numbers
np.random.rand(3, 4) # 3x4 matrix

Uniform [a, b]
np.random.uniform(10, 20, size=10) # 10 numbers between 10 and 20

Normal (Gaussian)
np.random.randn(100) # Standard normal (=0, =1)
np.random.normal(50, 10, size=100) # Custom =50, =10

Integers
np.random.randint(1, 100, size=50) # 50 random integers [1, 100)

Random selection
np.random.choice([1,2,3,4,5], size=3, replace=False) # No duplicates
np.random.choice(['A','B','C'], size=100, p=[0.5, 0.3, 0.2]) # Weighted

Shuffling
arr = np.array([1, 2, 3, 4, 5])
np.random.shuffle(arr) # Shuffles in-place

Permutation
np.random.permutation(10) # Random permutation of 0-9
```

## 10.7 Independent Study

This is where you apply everything you've learned about NumPy vectorization to solve real-world problems. Each exercise progressively builds your skills in:

- **Vectorized computations** for performance optimization
- **Matrix multiplication** for multi-dimensional operations
- **Random number generation** for simulations
- **Statistical methods** like bootstrapping

### 10.7.1 Practice Exercise 1: Shopping Optimization with Vectorization

**Problem Statement:** Three shoppers (Ben, Barbara, and Beth) need to buy groceries (rolls, buns, cakes, and bread). Two stores (Target and Kroger) have different prices. **Which store should each person choose to minimize their total cost?**

**Data Files:** - `food_quantity.csv`: How many of each item each person needs - `price.csv`: Price of each item at each store

**Strategy:** We'll solve this in **3 progressive steps** to understand the power of vectorization:

1. **Step 1:** Calculate Ben's cost at Target (simplest) 2. **Step 2:** Calculate Ben's cost at both stores (intermediate) 3. **Step 3:** Calculate everyone's cost at all stores (complete solution)

#### 10.7.1.1 Step 1: Calculate Ben's Cost at Target (Simplest Case)

**Goal:** Find how much Ben will spend if he buys everything at Target.

**Key Insight:** Ben will spend **\$50** at Target. Vectorized operations are cleaner and faster!

#### 10.7.1.2 Step 2: Calculate Ben's Cost at BOTH Stores (Intermediate)

**Goal:** Find Ben's cost at Target AND Kroger to determine which is cheaper.

**Key Insight:** Ben spends **\$50 at Target** vs **\$49 at Kroger** → **Kroger saves \$1!**

### 10.7.1.3 Step 3: Calculate EVERYONE'S Cost at ALL Stores (Complete Solution)

**Goal:** Find the best store for Ben, Barbara, AND Beth.

Method 1: using pandas dataframe nested loops

Method 2: Using numpy matrix multiplication

This is where vectorization truly shines!

### 10.7.2 Practice Exercise 2: Movie Rating Analysis with Matrix Multiplication

#### Problem Statement:

You have a dataset of movies with: - **Ratings:** IMDB Rating and Rotten Tomatoes Rating - **Genres:** Comedy, Action, Drama, Horror (binary flags: 1 = movie is in genre, 0 = not)

**Dataset:** movies\_cleaned.csv

**Questions to Answer:** 1. What is the **average IMDB rating** for each genre? 2. What is the **average Rotten Tomatoes rating** for each genre? 3. Which genre is **most preferred** by IMDB users? 4. Which genre is **least preferred** by Rotten Tomatoes critics?

#### 10.7.2.1 Step 0: Load movies dataset

#### 10.7.2.2 Step 1: Create Rating Matrix ( $N \text{ movies} \times 2 \text{ ratings}$ )

#### 10.7.2.3 Step 2: Create Genre Matrix ( $N \text{ movies} \times 4 \text{ genres}$ )

#### 10.7.2.4 Step 3: Matrix Multiplication for Total Ratings

**Goal:** Find total IMDB and Rotten Tomatoes ratings for each genre.

**Matrix Operation:** Ratings.T @ Genres

**Dimension Check:** - Ratings matrix: ( $N \times 2$ ) - Genres matrix: ( $N \times 4$ ) - For multiplication, we need:  $(2 \times N) @ (N \times 4) = (2 \times 4)$  - **Solution:** Transpose the ratings matrix!

#### 10.7.2.5 Step 4: Count Movies per Genre

To get **averages**, we need to divide total ratings by the number of movies in each genre.



### 10.7.2.6 Step 5: Compute the average rating per Genre

#### Key Findings:

IMDB users **prefer DRAMA** (highest average rating), and are **least amused by COMEDY** movies on average.

Rotten Tomatoes critics **prefer drama** over HORROR movies on average.

### 10.7.3 Practice Exercise 3: Simulation Study with Random Number Generation

#### Problem Statement:

Two food carts serve 500 customers each, every day for 30 days. The waiting times follow different distributions:

- **Food Cart 1:** Uniform distribution [5, 25] minutes (unpredictable service)
- **Food Cart 2:** Normal distribution with  $\mu=8$  min,  $\sigma=3$  min (consistent service)

#### Assumptions:

- Waiting times are measured **simultaneously** (paired observations)
- Same 500 people visit daily over 30 days

#### Questions to Answer:

1. On how many days is the **average waiting time** at Food Cart 2 higher than Food Cart 1?
2. What percentage of individual waiting times at Food Cart 2 exceed Food Cart 1?
3. How much **faster** is vectorized random generation vs loops?

#### Strategy:

- **Simulation size:** 500 people  $\times$  30 days = 15,000 observations per cart
- **Method 1:** Nested loops (slow but explicit)
- **Method 2:** Vectorized NumPy (fast and elegant)

## 10.7.4 Practice Exercise 4: Bootstrapping for Confidence Intervals

### Problem Statement:

Find the **95% confidence interval** for the mean profit of **Action** movies using bootstrapping.

### What is Bootstrapping?

Bootstrapping is a **non-parametric statistical method** that estimates the sampling distribution of a statistic by resampling from the observed data. It's used when: - Sample size is small - Distribution is unknown or non-normal - Theoretical formulas are complex or unavailable

### Bootstrap Algorithm:

1. **Extract** profit data for all Action movies (N movies)
2. **Resample** N values **WITH** replacement from the profit data
3. **Calculate** the mean of the resampled data
4. **Repeat** steps 2-3 M=1000 times
5. **Find** the 2.5th and 97.5th percentiles of the 1000 means

**Result:** [2.5th percentile, 97.5th percentile] = 95% Confidence Interval

**Dataset:** movies\_cleaned.csv

Hints: Use `np.random.choice()` for sampling with replacement

```
=====
BOOTSTRAPPING: 95% CI for Action Movie Profits
=====
```

|   | Title               | IMDB Rating | Rotten Tomatoes Rating | Running Time min | Release Date | US C |
|---|---------------------|-------------|------------------------|------------------|--------------|------|
| 0 | Broken Arrow        | 5.8         | 55                     | 108              | Feb 09 1996  | 7064 |
| 1 | Brazil              | 8.0         | 98                     | 136              | Dec 18 1985  | 9929 |
| 2 | The Cable Guy       | 5.8         | 52                     | 95               | Jun 14 1996  | 6024 |
| 3 | Chain Reaction      | 5.2         | 13                     | 106              | Aug 02 1996  | 2122 |
| 4 | Clash of the Titans | 5.9         | 65                     | 108              | Jun 12 1981  | 3000 |

Confidence interval = [\$133.5 million, \$181.08 million]

# 11 Data Visualization Fundamentals

<IPython.core.display.Image object>

*“One picture is worth a thousand words”* - Fred R. Barnard

Visual perception offers the highest bandwidth channel, as we acquire much more information through visual perception than with all of the other channels combined, as billions of our neurons are dedicated to this task. Moreover, the processing of visual information is, at its first stages, a highly parallel process. Thus, it is generally easier for humans to comprehend information with plots, diagrams and pictures, rather than with text and numbers. This makes data visualizations a vital part of data science. Some of the key purposes of data visualization are:

1. Data visualization is the first step towards exploratory data analysis (EDA), which reveals trends, patterns, insights, or even irregularities in data.
2. Data visualization can help explain the workings of complex mathematical models.
3. Data visualization are an elegant way to summarise the findings of a data analysis project.
4. Data visualizations (especially interactive ones such as those on Tableau) may be the end-product of data analytics project, where the stakeholders make decisions based on the visualizations.

## 11.1 Learning Objectives

By the end of this chapter, you will be able to:

- **Understand** the fundamental principles of effective data visualization
- **Classify** data types and choose appropriate visualization methods
- **Create** basic plots using Pandas’ built-in plotting capabilities
- **Customize** sophisticated visualizations with Matplotlib’s pyplot interface
- **Design** publication-quality statistical plots using Seaborn
- **Apply** best practices for visual storytelling with data
- **Compare** different visualization libraries and their use cases

## 11.2 The Art of Visualization: Choosing the Right Plot Type

There are various types of plots available, and selecting the appropriate one is crucial for successful data visualization. The choice primarily depends on two factors:

- The type of data you are working with, and
- The role of visualization in your data analysis

### 11.2.1 Data Classification for Visualization

Data visualization is commonly used to plot data in a pandas DataFrame. The data can be classified into two categories:

- **Numeric Data:** This type of data represents quantities and can take any value within a range. Common examples include age, height, temperature, etc.
- **Categorical Data:** This type of data represents distinct categories or groups. It can be nominal (no inherent order, like colors or names) or ordinal (with a defined order, like ratings).

### 11.2.2 The Role of Visualization in Data Analysis

Data visualization is essential for effectively communicating insights derived from data analysis. By using various visualization techniques, we can **uncover patterns, and understand relationships**. Below, we discuss different types of data exploration and the relevant visualizations used for each.

#### 11.2.2.1 Univariate Exploration

**Purpose:** Univariate exploration analyzes a single variable to understand its distribution, central tendency, and spread.

##### 11.2.2.1.1 Common Visualizations:

- **Histograms:** Display the frequency distribution of a numeric variable, helping to identify the shape of the data (e.g., normal, skewed).
- **Box Plots:** Summarize key statistics of a variable, including median, quartiles, and potential outliers.
- **Bar Plots:** Show the count or proportion of categorical variables, revealing the frequency of each category.

- **Line Plots:** Used to display trends in numeric data over time, helping to visualize changes in a variable.

#### 11.2.2.1.2 Insights Gained:

- Identify outliers and anomalies.
- Understand the range and distribution of values.
- Determine central tendency (mean, median, mode).

#### 11.2.2.2 Bivariate Analysis

**Purpose:** Bivariate analysis examines the relationship between two variables, helping to understand how changes in one variable might affect another.

##### 11.2.2.2.1 Common Visualizations:

- **Scatter Plots:** Illustrate the relationship between two numeric variables, highlighting trends and correlations.
- **Grouped Bar Plots:** Compare categorical variables against a numeric variable, revealing trends across categories.
- **Heatmaps:** Represent correlation coefficients between pairs of variables, allowing easy identification of strong correlations.

##### 11.2.2.2.2 Insights Gained:

- Assess the strength and direction of relationships (positive, negative, or no correlation).
- Identify potential predictive relationships for further analysis.
- Discover patterns that may indicate causal relationships.

#### 11.2.2.3 Multivariate Analysis

**Purpose:** Multivariate analysis investigates more than two variables simultaneously, providing a comprehensive view of complex relationships and interactions.

#### 11.2.2.3.1 Common Visualizations:

- **Pair Plots:** Show pairwise relationships in a dataset, facilitating quick insights into correlations among multiple variables.
- **3D Scatter Plots:** Visualize the interaction between three numeric variables in a three-dimensional space.
- **Facet Grids:** Display multiple plots for different subsets of data, enabling comparisons across categories.

#### 11.2.2.3.2 Insights Gained:

- Understand interactions and dependencies among multiple variables.
- Identify clusters or groups within the data.
- Enhance predictive modeling by considering multiple influences.

### 11.2.3 Quick Reference: Data Types and Visualization Mapping

| Data Type                    | Examples                         | Best Visualizations                |
|------------------------------|----------------------------------|------------------------------------|
| <b>Single Numeric</b>        | Age, Temperature, Price          | Histogram, Box Plot, Density Plot  |
| <b>Single Categorical</b>    | Gender, Region, Product Type     | Bar Chart, Pie Chart, Count Plot   |
| <b>Time Series</b>           | Stock prices, Daily sales        | Line Plot, Area Chart              |
| <b>Two Numeric</b>           | Height vs Weight, Price vs Sales | Scatter Plot, Regression Plot      |
| <b>Numeric + Categorical</b> | Sales by Region, Age by Gender   | Grouped Bar, Box Plot, Violin Plot |
| <b>Two Categorical</b>       | Gender vs Product Preference     | Stacked Bar, Grouped Bar, Heatmap  |
| <b>Three+ Variables</b>      | Multiple dimensions              | Pair Plot, Facet Grid, 3D Scatter  |
| <b>Correlations</b>          | Feature relationships            | Correlation Matrix, Heatmap        |

Choosing the appropriate plot depends on the data type and the specific analysis purpose. Numeric data typically requires plots that can handle continuous data (like line plots or histograms), while categorical data often benefits from comparisons (like bar plots or pie charts). Always consider what story you want to tell with your data and select your visualization method accordingly.

## 11.3 Visualization Tools: The Python Ecosystem

Python offers a rich ecosystem of visualization libraries, each designed for specific purposes. In this course, we'll focus on three fundamental libraries that form the foundation of data visualization in Python.

### 11.3.1 The Three-Library Approach

1. **Pandas** - Quick Exploratory Plots - Built-in `.plot()` method for DataFrames - Perfect for rapid data exploration - Minimal code required
2. **Matplotlib** - Complete Control - Low-level plotting library - Maximum customization capability - Foundation for other libraries
3. **Seaborn** - Statistical Beauty - High-level interface built on Matplotlib - Specialized for statistical visualization - Beautiful defaults out of the box

### 11.3.2 Why These Three?

| Feature                     | Benefit                                               |
|-----------------------------|-------------------------------------------------------|
| <b>Complementary</b>        | Each library excels at different tasks                |
| <b>Integrated</b>           | They work seamlessly together                         |
| <b>Industry Standard</b>    | Most widely used in data science                      |
| <b>Well-Documented</b>      | Extensive resources and community support             |
| <b>Progressive Learning</b> | Start simple (Pandas) → Advanced (Matplotlib/Seaborn) |

### 11.3.3 Library Comparison at a Glance

Pandas Plot

- Pros: Fastest to write, DataFrame-native
- Cons: Limited customization, basic styling

Matplotlib

- Pros: Complete control, publication-quality
- Cons: Verbose syntax, requires more code

Seaborn

- Pros: Beautiful defaults, statistical functions
- Cons: Less control than Matplotlib, opinionated

### 11.3.4 Step 1: Import Libraries

Let's start by importing all three libraries with their standard aliases:

```
import pandas as pd
import matplotlib
import matplotlib.pyplot as plt
import seaborn as sns

We'll also import numpy for numerical operations
import numpy as np

print(" Visualization libraries imported and configured successfully!")
print(f" - Pandas version: {pd.__version__}")
print(f" - Matplotlib version: {matplotlib.__version__}")
print(f" - Seaborn version: {sns.__version__}")
print(f" - NumPy version: {np.__version__}")
```

```
Visualization libraries imported and configured successfully!
- Pandas version: 2.2.2
- Matplotlib version: 3.8.4
- Seaborn version: 0.13.2
- NumPy version: 1.26.4
```

### 11.3.5 Step 2: Understanding the Imports

Let's understand what each import does:

**import pandas as pd** - Imports the pandas library for data manipulation - Provides the `.plot()` method on DataFrames - **pd** is the universally recognized alias

**import matplotlib.pyplot as plt** - Imports matplotlib's pyplot module for plotting - pyplot provides MATLAB-like interface for creating plots - **plt** is the standard alias used in all documentation

**import seaborn as sns** - Imports seaborn for statistical visualizations - Built on top of matplotlib with enhanced styling - **sns** alias comes from the TV show "The West Wing"

**import numpy as np** - Not a visualization library, but essential for: - Generating sample data for demonstrations - Performing numerical calculations - Creating arrays for plotting



### 11.3.6 Step 3: Configure Your Environment

Now let's set up optimal defaults for our visualization environment:

```
Configure matplotlib for better display in Jupyter
%matplotlib inline

Set default figure size for all plots (width, height in inches)
import matplotlib
matplotlib.rcParams['figure.figsize'] = (8, 5) # Larger default size
matplotlib.rcParams['figure.dpi'] = 100 # Resolution
matplotlib.rcParams['font.size'] = 11 # Default font size

Configure Seaborn style
sns.set_style("whitegrid") # Clean grid background
sns.set_palette("deep") # Colorblind-friendly palette
sns.set_context("notebook") # Appropriate scaling
```

### 11.3.7 Step 4: Understanding the Configuration

Let's break down what each configuration does:

#### 11.3.7.1 Matplotlib Configuration

**%matplotlib inline** (Jupyter Magic Command)

- Displays plots directly in the notebook
- Without this, plots may not appear or open in separate windows
- Only needed once per notebook session

**figure.figsize** - Default Plot Dimensions

- Format: (width, height) in inches
- Default is (6.4, 4.8) - often too small
- We set (10, 6) for better visibility
- Can be overridden for individual plots

**figure.dpi** - Dots Per Inch (Resolution)

- Controls image quality and sharpness
- Default: 72 DPI (screen display)
- We set 100 DPI for crisper display

- Use 300 DPI for publication-quality exports

**font.size** - Text Size

- Default: 10 points
- We set 11 for better readability
- Affects all text: labels, titles, legends

### 11.3.7.2 Seaborn Configuration

**set\_style("whitegrid")** - Visual Theme

- Options: `darkgrid`, `whitegrid`, `dark`, `white`, `ticks`
- `whitegrid`: Clean white background with subtle grid
- Professional appearance suitable for presentations

**set\_palette("deep")** - Color Scheme

- Options: `deep`, `muted`, `pastel`, `colorblind`, etc.
- `deep`: Saturated colors with good contrast
- Automatically applied to Seaborn plots

**set\_context("notebook")** - Scaling Preset

- Options: `paper`, `notebook`, `talk`, `poster`
- Controls relative size of elements
- `notebook`: Optimal for Jupyter display
- Use `talk` for presentations, `poster` for large displays

### 11.3.8 Step 5: Verify Your Setup

Let's create a quick test to confirm everything is working correctly:

```
Quick test plot to verify configuration
fig, axes = plt.subplots(1, 3, figsize=(15, 4))

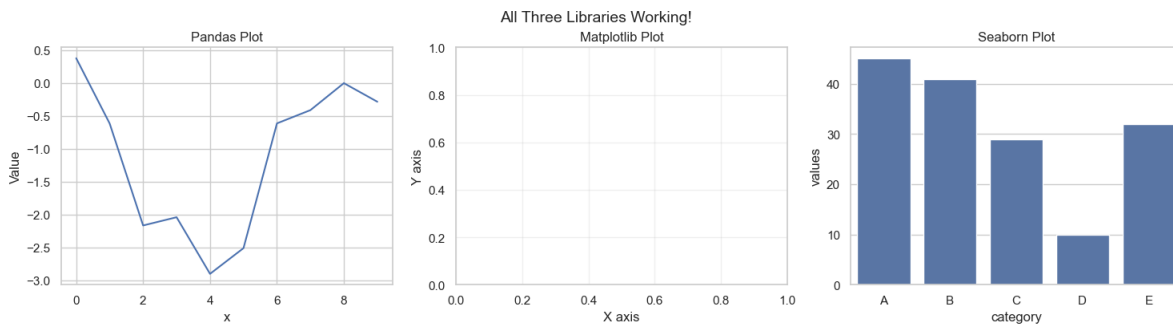
Test 1: Pandas plot
test_data = pd.DataFrame({
 'x': range(10),
 'y': np.random.randn(10).cumsum()
})
test_data.plot(x='x', y='y', ax=axes[0], title='Pandas Plot', legend=False)
axes[0].set_ylabel('Value')
```

```
Test 2: Matplotlib plot
axes[1].set_title('Matplotlib Plot')
axes[1].set_xlabel('X axis')
axes[1].set_ylabel('Y axis')
axes[1].grid(True, alpha=0.3)

Test 3: Seaborn plot
test_df = pd.DataFrame({
 'category': ['A', 'B', 'C', 'D', 'E'],
 'values': np.random.randint(10, 50, 5)
})
sns.barplot(data=test_df, x='category', y='values', ax=axes[2])
axes[2].set_title('Seaborn Plot')

plt.tight_layout()
plt.suptitle('All Three Libraries Working!', fontsize=14, y=1.02)
plt.show()

print("\n All libraries are working correctly!")
print(" You're ready to start creating visualizations!")
```



All libraries are working correctly!  
 You're ready to start creating visualizations!

### 11.3.9 Step 6: Pro Tips for Working with These Libraries

Now that everything is set up, here are some professional tips for effective visualization:

### 11.3.9.1 Layer Your Learning

Start simple and progressively add complexity:

- **Begin:** Use Pandas `.plot()` for quick data exploration
- **Enhance:** Add Matplotlib customization for specific needs
- **Refine:** Use Seaborn for statistical visualizations
- **Combine:** Mix all three in complex projects

### 11.3.9.2 Common Workflow Pattern

Here's a typical pattern that combines all three libraries:

```
Step 1: Create base plot with Seaborn (easy syntax + beautiful defaults)
sns.scatterplot(data=df, x='var1', y='var2', hue='category')

Step 2: Customize with Matplotlib (fine-grained control)
plt.title('My Custom Title', fontsize=14, fontweight='bold')
plt.xlabel('Custom X Label')
plt.axhline(y=0, color='red', linestyle='--', alpha=0.5)

Step 3: Display
plt.show()
```

### 11.3.9.3 Troubleshooting Quick Reference

| Problem                    | Solution                                                                              |
|----------------------------|---------------------------------------------------------------------------------------|
| Plots not showing          | Add <code>plt.show()</code> or verify <code>%matplotlib inline</code> is set          |
| Plots overlapping          | Use <code>plt.figure()</code> before each new plot or <code>plt.clf()</code> to clear |
| Text too small             | Increase <code>font.size</code> in rcParams or use <code>fontsize</code> parameter    |
| Poor image quality         | Save with higher DPI:<br><code>plt.savefig('plot.png', dpi=300)</code>                |
| Colors hard to distinguish | Use colorblind-friendly palette:<br><code>sns.set_palette("colorblind")</code>        |
| Legend outside plot area   | Adjust with <code>bbox_inches='tight'</code> when saving                              |

#### 11.3.9.4 Performance Tips for Large Datasets

When working with datasets >100K points:

- **Sampling:** Plot a representative random subset
- **Aggregation:** Group/bin data before plotting
- **Rasterization:** Use `rasterized=True` in scatter plots
- **Alternative libraries:** Consider Plotly or Bokeh for interactive visualizations

#### 11.3.9.5 Saving High-Quality Plots

Professional way to save your visualizations:

```
plt.savefig('my_plot.png',
 dpi=300, # Publication quality (300 DPI)
 bbox_inches='tight', # Remove extra whitespace
 facecolor='white', # Ensure white background
 edgecolor='none', # No border
 transparent=False) # Solid background
```

#### 11.3.10 You're Ready!

Your visualization environment is now fully configured and tested. In the following sections, we'll explore each library in detail, starting with Pandas for quick exploratory analysis.

### 11.4 Basic Plotting with Pandas

In previous chapters, we focused on using pandas for data reading and analysis. In addition to its powerful data manipulation capabilities, **pandas also provides built-in plotting tools** that make it especially valuable for:

- **Rapid Exploration** - Create plots with minimal code
- **Seamless Integration** - No data conversion needed
- **Quick Insights** - Perfect for initial data investigation
- **Iterative Analysis** - Easy to modify and regenerate

**The Power of `.plot()`:** Pandas' `.plot()` method is your Swiss Army knife for quick visualizations. It's built on Matplotlib but provides a simpler, DataFrame-centric interface.

### 11.4.1 Dataset: COVID-19 Cases

In this section, we'll use real COVID-19 data to demonstrate practical plotting techniques. This dataset contains time series information about new cases and deaths, making it ideal for learning data visualization.

```
covid_df = pd.read_csv('./Datasets/covid.csv')
covid_df.head(5)
```

|   | date       | new_cases | total_cases | new_deaths | total_deaths | new_tests | total_tests | cases_per_ |
|---|------------|-----------|-------------|------------|--------------|-----------|-------------|------------|
| 0 | 2019-12-31 | 0.0       | 0.0         | 0.0        | 0.0          | NaN       | NaN         | 0.0        |
| 1 | 2020-01-01 | 0.0       | 0.0         | 0.0        | 0.0          | NaN       | NaN         | 0.0        |
| 2 | 2020-01-02 | 0.0       | 0.0         | 0.0        | 0.0          | NaN       | NaN         | 0.0        |
| 3 | 2020-01-03 | 0.0       | 0.0         | 0.0        | 0.0          | NaN       | NaN         | 0.0        |
| 4 | 2020-01-04 | 0.0       | 0.0         | 0.0        | 0.0          | NaN       | NaN         | 0.0        |

### 11.4.2 Step 1: Your First Pandas Plot

Let's start with the simplest possible plot - visualizing a single variable.

**Concept: Line Plots for Time Series** Line plots are ideal for showing changes over continuous data, such as the progression of new cases over a series of dates.

**The Simplest Plot:** With pandas, creating a plot is as easy as calling `.plot()` on a Series or DataFrame:

```
covid_df.new_cases.plot()
```



### Problem Identified:

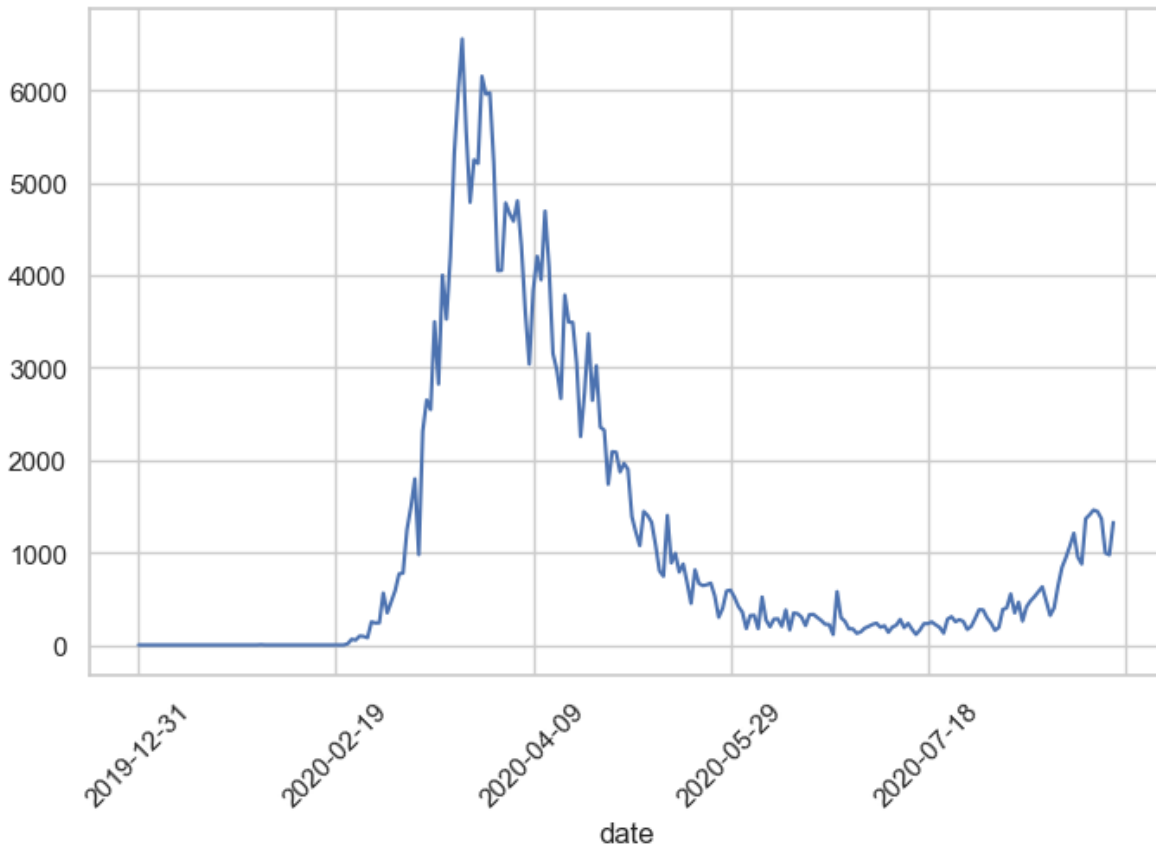
While this plot shows the overall trend, it's hard to tell *when* the peak occurred - the X-axis shows indices (0, 1, 2...) instead of dates!

### Solution: Set the Date as Index

Since this is a time series dataset, we can use the date column as the DataFrame index. This tells pandas to use actual dates on the X-axis:

```
covid_df.set_index('date', inplace=True)
```

```
covid_df.new_cases.plot(rot=45);
```



### Much Better!

Now we can clearly see that the peak occurred around March 2020. The `rot=45` parameter rotates the X-axis labels 45 degrees for better readability.

**Key Insight:** When working with time series data, always set the date/time column as the index for automatic, proper time-axis formatting.

### 11.4.3 Step 2: Plotting Multiple Variables

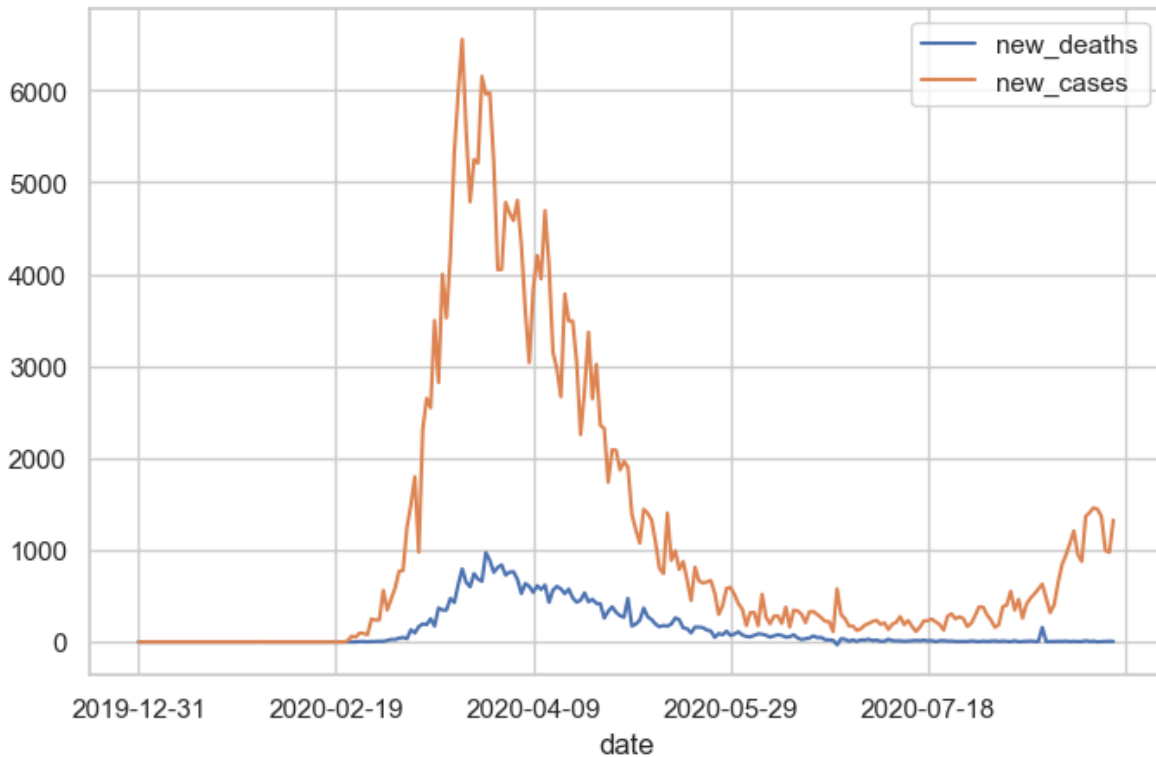
Now that we have one variable plotted, let's compare multiple variables on the same plot to see relationships.

**Goal:** Compare new cases and new deaths trends over time

**How:** Simply pass a list of column names to the DataFrame:

```
covid_df[['new_deaths', 'new_cases']].plot();
```





## What Happened?

Pandas automatically:

- Created two lines (one per column)
- Assigned different colors to each line
- Generated a legend with column names
- Used the index (dates) for the X-axis

### 11.4.4 Step 3: Customizing Your Plots

By default, pandas generates line plots using the `.plot()` method. However, you can adjust several parameters to enhance the appearance:

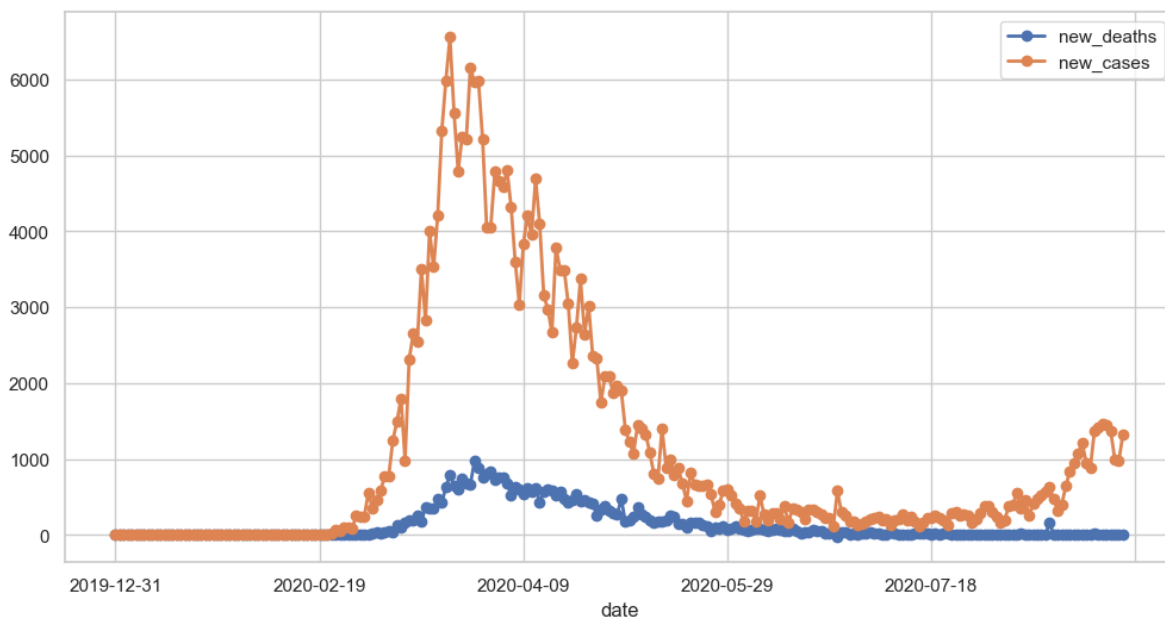
#### Common Customization Parameters:

| Parameter                                 | Purpose                                         | Example                      |
|-------------------------------------------|-------------------------------------------------|------------------------------|
| <code>figsize</code>                      | Set figure dimensions (width, height) in inches | <code>figsize=(12, 6)</code> |
| <code>linewidth</code> or <code>lw</code> | Control line thickness                          | <code>linewidth=2</code>     |
| <code>marker</code>                       | Add data point markers                          | <code>marker='o'</code>      |

| Parameter | Purpose                                       | Example                                 |
|-----------|-----------------------------------------------|-----------------------------------------|
| color     | Set line color(s)                             | color='red' or<br>color=['red', 'blue'] |
| alpha     | Set transparency<br>(0=transparent, 1=opaque) | alpha=0.7                               |
| rot       | Rotate X-axis labels                          | rot=45                                  |
| grid      | Show/hide gridlines                           | grid=True                               |
| title     | Add plot title                                | title='My Plot'                         |

Let's enhance our plot:

```
covid_df[['new_deaths', 'new_cases']].plot(figsize=(12, 6), linewidth=2, marker='o');
```



**Result:** The plot is now larger, has thicker lines, and shows markers at each data point - much easier to read!

#### 11.4.5 Step 4: Different Plot Types

So far, we've only created line plots (the default). However, pandas supports **11 different plot types** through the `kind` parameter:

| kind Value         | Plot Type               | Best For                                |
|--------------------|-------------------------|-----------------------------------------|
| 'line'             | Line plot (default)     | Trends over time, continuous data       |
| 'scatter'          | Scatter plot            | Relationships between two variables     |
| 'bar'              | Vertical bar chart      | Comparing categories                    |
| 'barh'             | Horizontal bar chart    | Comparing categories (long labels)      |
| 'hist'             | Histogram               | Distribution of a single variable       |
| 'box'              | Box plot                | Distribution summary, outlier detection |
| 'kde' or 'density' | Kernel Density Estimate | Smooth distribution visualization       |
| 'area'             | Area plot               | Cumulative trends                       |
| 'pie'              | Pie chart               | Part-to-whole relationships             |
| 'hexbin'           | Hexagonal bin plot      | Dense scatter plot patterns             |

Let's explore the most common plot types:

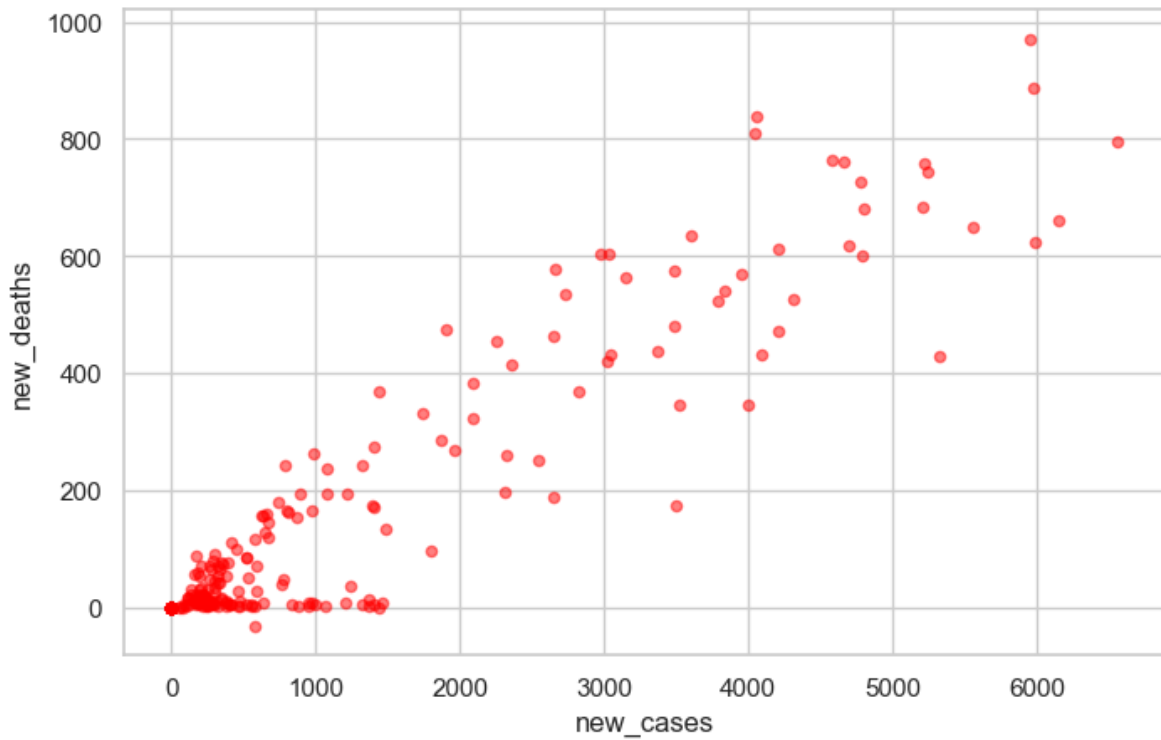
#### 11.4.5.1 Scatter Plot: Finding Relationships

**Question:** Is there a correlation between new cases and new deaths?

**Best Tool:** Scatter plot - shows the relationship between two numerical variables

Let's next create a scatter plot to visualize the relationship between new cases and new deaths, and explore whether there's a correlation between them.

```
covid_df.plot(kind='scatter', x='new_cases', y='new_deaths', color='r', alpha=0.5);
```



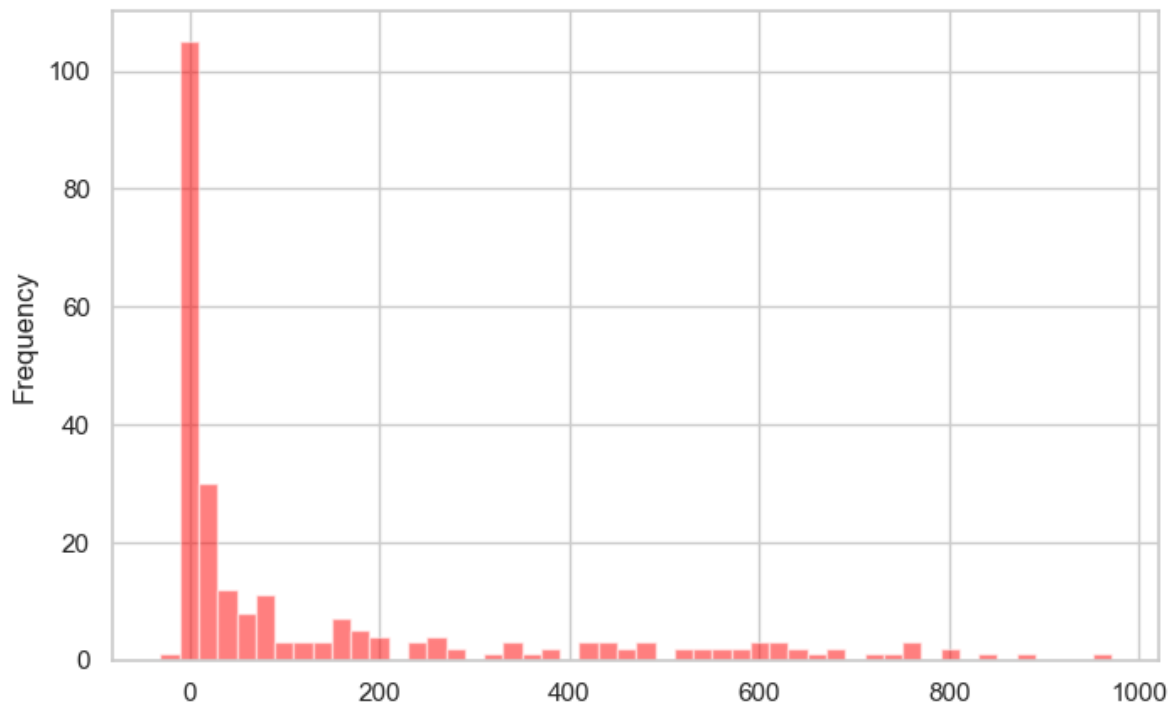
**Observation:** There appears to be a positive correlation - as new cases increase, new deaths tend to increase as well. The `alpha=0.5` parameter makes points semi-transparent, helping us see overlapping data points.

#### 11.4.5.2 Histogram: Understanding Distribution

**Question:** What's the typical range of daily deaths? Are there outliers?

**Best Tool:** Histogram - shows the frequency distribution of a single variable

```
covid_df.new_deaths.plot(kind='hist', color='r', alpha=0.5, bins=50);
```



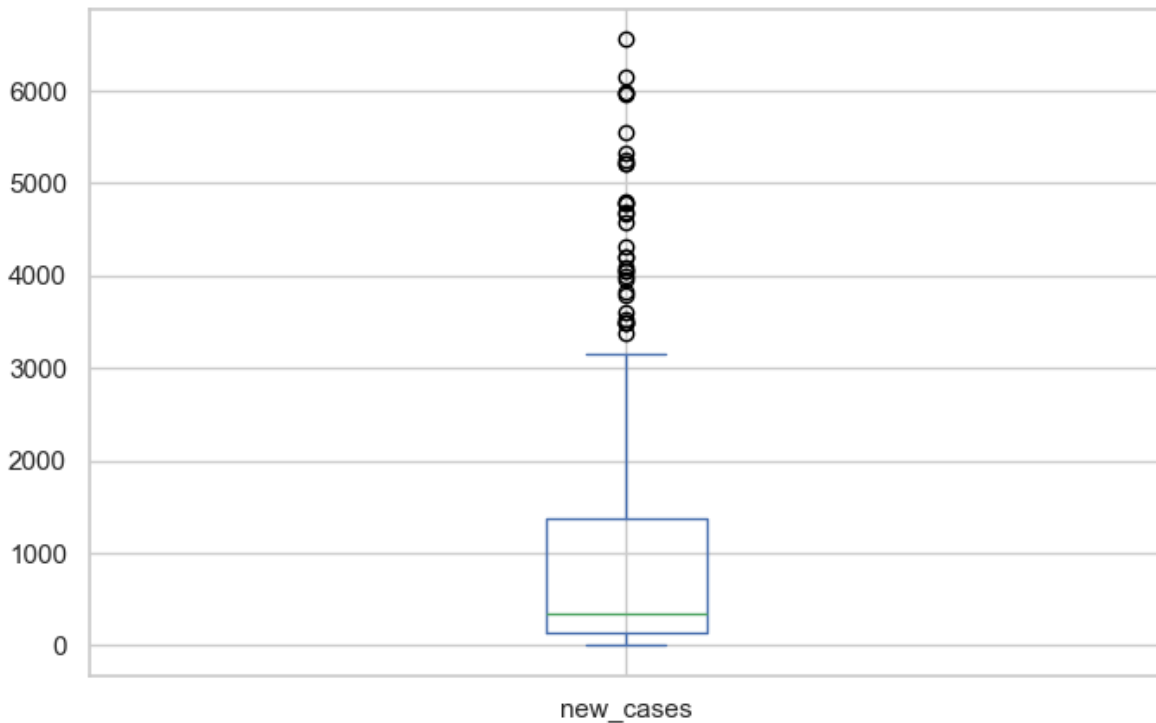
**Observation:** Most days have relatively few deaths (left-skewed distribution), with occasional days having much higher counts. The `bins=50` parameter creates 50 bins for finer detail.

#### 11.4.5.3 Box Plot: Spotting Outliers

**Question:** What's the median, quartiles, and outliers for new cases?

**Best Tool:** Box plot - summarizes distribution and highlights outliers

```
covid_df.new_cases.plot(kind='box');
```



**Observation:** The box shows the median (orange line), 25th-75th percentiles (box), and outliers (circles). We can see several high-outlier days with exceptionally high case counts.

### 11.4.6 Step 5: Choosing the Right Plot Type

Not sure which plot to use? Follow this decision guide:

WANT TO SHOW TRENDS OVER TIME?

- > Use: `kind='line'` (default)
- > Example: Stock prices, temperature changes

WANT TO SHOW RELATIONSHIPS BETWEEN TWO VARIABLES?

- > Use: `kind='scatter'`
- > Example: Height vs. weight, study time vs. grades

WANT TO SHOW DISTRIBUTION OF ONE VARIABLE?

- > Use: `kind='hist'` or `kind='box'`
- > Histogram: See the shape of distribution

> Box plot: See median, quartiles, and outliers

WANT TO COMPARE CATEGORIES?

> Use: `kind='bar'` or `kind='barh'`

> Example: Sales by region, counts by category

WANT TO SHOW PARTS OF A WHOLE?

> Use: `kind='pie'`

> Warning: Use sparingly - often harder to read than bar charts!

### 11.4.7 Additional Resources

For more plot types and detailed information, refer to the official pandas documentation:

- [Series.plot](#) - Plotting methods for Series
- [DataFrame.plot](#) - Plotting methods for DataFrames

### 11.4.8 Summary: Pandas Plotting Strengths & Limitations

#### 11.4.8.1 Strengths (When to Use Pandas):

1. **Speed** - Create plots in 1-2 lines of code
2. **Integration** - Works directly with DataFrames (no conversion)
3. **Exploration** - Perfect for quick data checks
4. **Learning Curve** - Easiest to learn for beginners
5. **Iteration** - Rapidly test different visualizations

**Best Use Cases:** - Rapid data exploration during analysis - Quick sanity checks during data cleaning - Initial pattern recognition - Jupyter notebook investigations

#### 11.4.8.2 Limitations (When to Move Beyond Pandas):

1. **Limited Aesthetics** - Basic styling, not publication-ready
2. **Simple Layouts** - Hard to create multi-panel figures
3. **Customization** - Fine-grained control requires Matplotlib
4. **Statistical Plots** - No built-in statistical visualizations
5. **Plot Variety** - Missing advanced plot types

**When to Switch Libraries:**

- → **Matplotlib:** Need fine-grained control, custom layouts, publication quality
- → **Seaborn:** Need statistical plots, beautiful defaults, correlation matrices

- → **Plotly**: Need interactive plots, dashboards, 3D visualizations

### 11.4.9 Practice Exercise: Apply Your Skills

Now it's your turn! Try creating these plots with the COVID dataset:

#### Exercise 1: Basic Line Plot

```
Plot new_cases with:
- Figure size of (14, 6)
- Green color
- Title "COVID-19 New Cases Over Time"
```

#### Exercise 2: Comparison Plot

```
Plot new_cases and new_deaths together with:
- Different line styles (solid and dashed)
- Add a grid
```

#### Exercise 3: Distribution Analysis

```
Create a histogram of new_cases with:
- 30 bins
- Blue color with 50% transparency
```

#### Exercise 4: Relationship Analysis

```
Create a scatter plot showing new_cases vs new_deaths
Add appropriate axis labels
```

**Tip:** Refer to the [DataFrame.plot documentation](#) to find the parameters you need!

## 11.5 Data Plotting with Matplotlib pyplot Interface

You've just learned to create plots with pandas' `.plot()` method. But here's an important insight: **Pandas plotting is built on top of Matplotlib!**

When you call `df.plot()`, pandas is actually calling Matplotlib functions behind the scenes. Think of it like this:



|                                    |                                     |
|------------------------------------|-------------------------------------|
| Pandas <code>.plot()</code>        | ← High-level, easy, limited control |
| Matplotlib ( <code>pyplot</code> ) | ← Low-level, powerful, full control |

## Why Learn Matplotlib Directly?

While Pandas is great for quick plots, Matplotlib gives you:

- **Maximum Control** - Customize every element of your plots
- **Complex Layouts** - Create multi-panel figures and subplots
- **Publication Quality** - Professional scientific visualizations
- **Foundation Knowledge** - Understanding how visualization really works in Python
- **Flexibility** - Work with any data structure (lists, arrays, DataFrames)

### 11.5.1 What is Matplotlib?

Matplotlib is:

- A comprehensive library for creating static, animated, and interactive visualizations in Python
- Designed to emulate MATLAB's plotting interface (hence the name)
- The foundation for many other visualization libraries (Pandas, Seaborn, etc.)
- Compatible with Python scripts, IPython shells, Jupyter notebooks, and web servers
- Mostly written in Python, with some segments in C, Objective-C, and JavaScript for platform compatibility

### 11.5.2 Step 1: Understanding pyplot

What is pyplot?

`pyplot` is a module within Matplotlib that provides a state-based interface for creating plots. It's the most commonly used part of Matplotlib.

**Key Concepts:**

- **Matplotlib** = The entire plotting library/package
- **pyplot** = A specific module within Matplotlib that makes plotting easier
- **plt** = The conventional alias used when importing pyplot

### The State-Based Interface:

pyplot maintains an internal “current” figure and axes. When you call functions like `plt.plot()`, `plt.xlabel()`, etc., they automatically apply to the current figure. This makes it easy to build plots step by step without explicitly managing figure objects.

### Import Convention:

The standard way to import pyplot is:

```
import matplotlib.pyplot as plt
```

### Data Sources:

pyplot can work with various data types: - Python lists - NumPy arrays - Pandas Series and DataFrames

**Note:** Internally, all sequences are converted to NumPy arrays for processing.

## 11.5.3 Step 2: Your First pyplot Plot

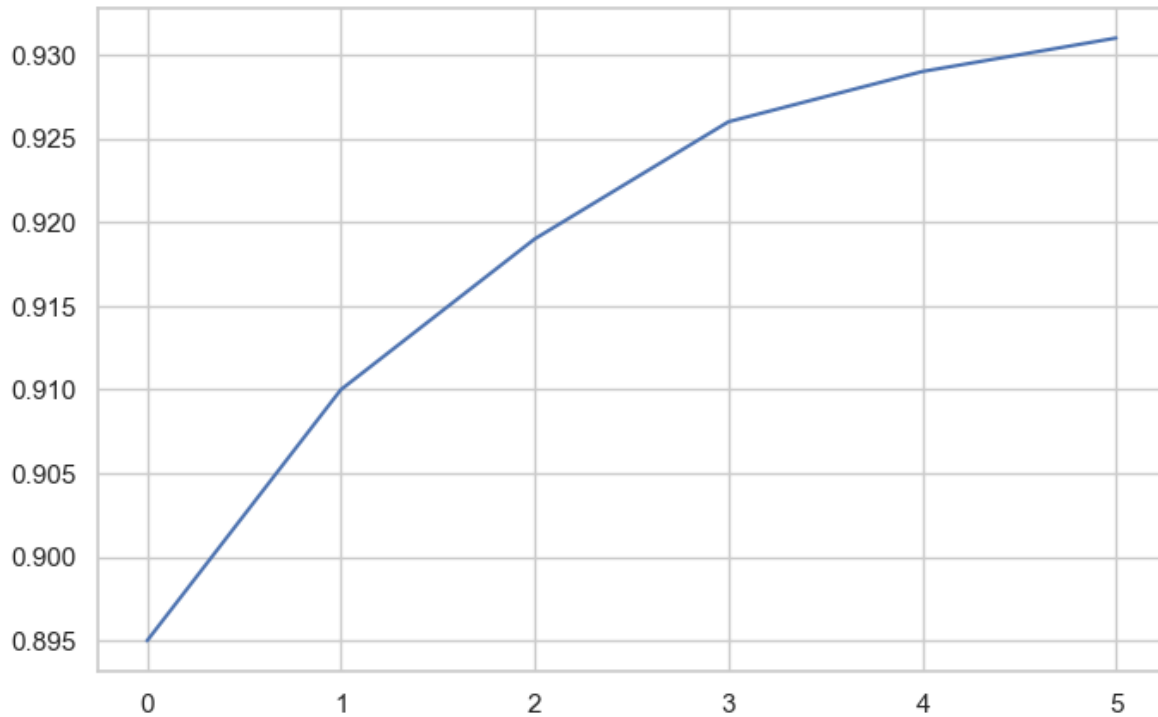
Let’s start with the simplest possible plot using Python lists to illustrate basic plotting with Matplotlib pyplot:

```
yield_apples = [0.895, 0.91, 0.919, 0.926, 0.929, 0.931]
```

### Create the Plot:

Now let’s visualize this data with a simple line plot:

```
plt.plot(yield_apples);
```



### What Just Happened?

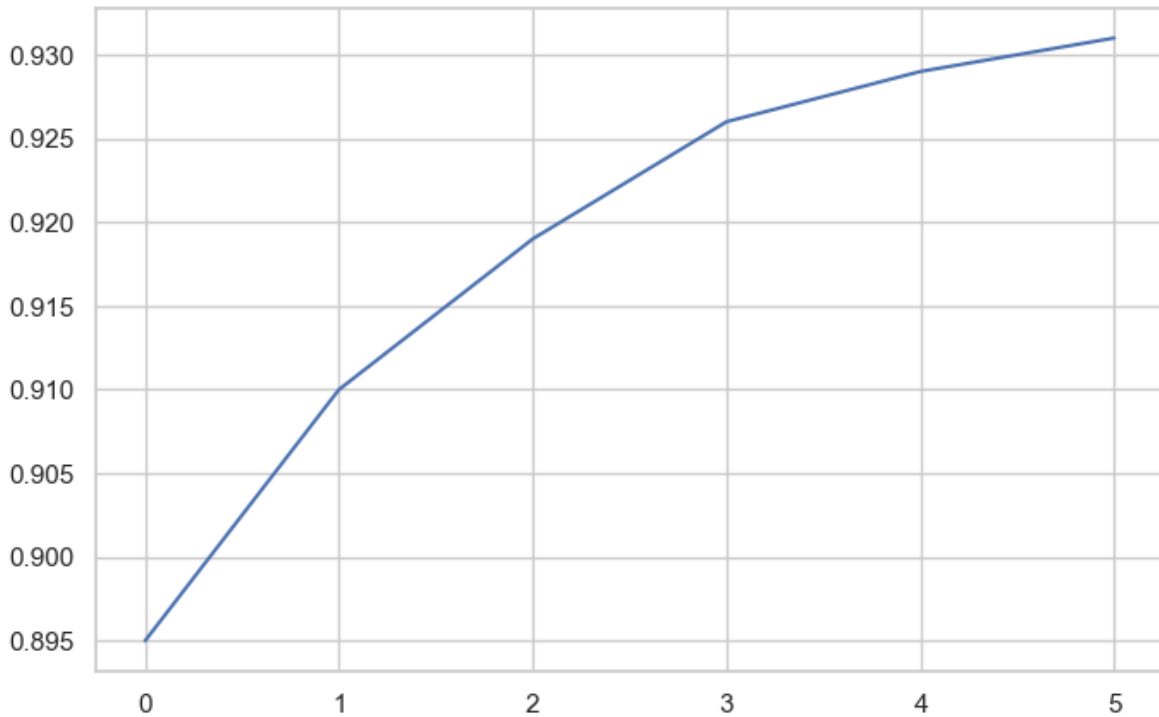
Calling `plt.plot()` creates a line chart showing the trend in apple yields.

### The Semicolon Trick:

You might notice the semicolon (;) at the end of the command. Without it, Matplotlib returns a text representation like `[<matplotlib.lines.Line2D at 0x2194b571df0>]` before displaying the plot. The semicolon suppresses this output, showing only the graph.

### With semicolon (cleaner output):

```
plt.plot(yield_apples);
```



**Problem:** The X-axis shows list indices (0, 1, 2, 3, 4, 5) instead of meaningful values like years.

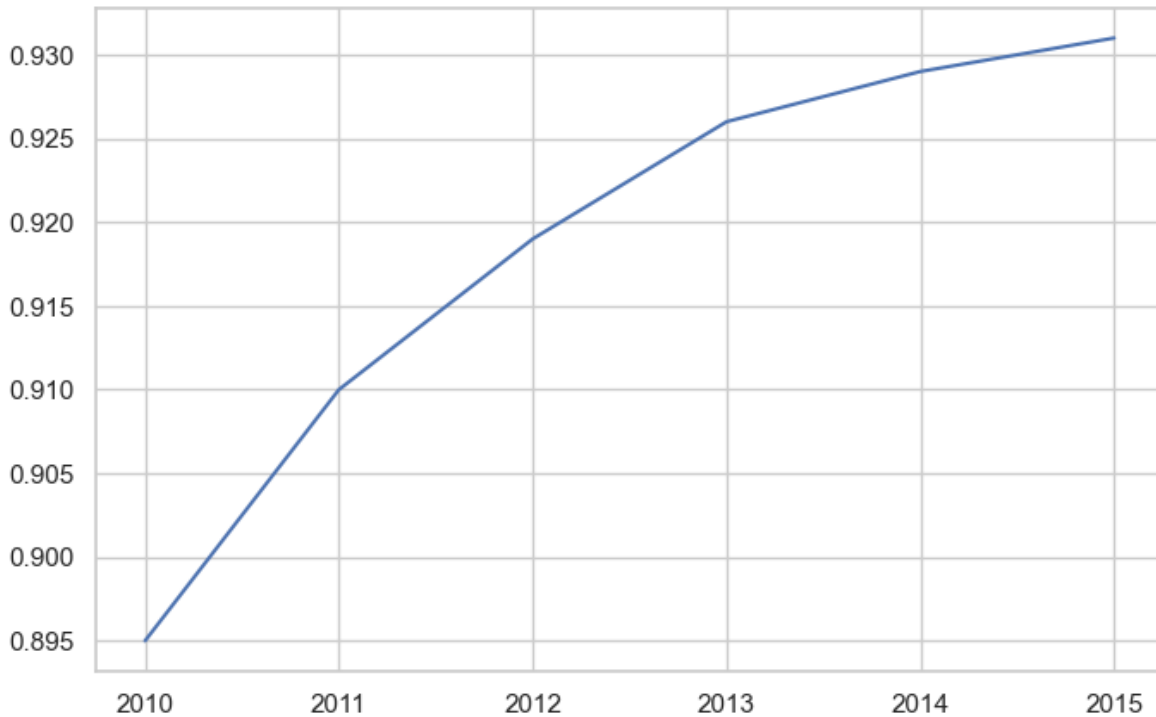
**Solution:** Provide both X and Y data to `plt.plot()`.

### 11.5.4 Step 3: Customizing Axes

Let's make our plot more informative by specifying custom X-axis values. We'll use years instead of indices:

```
years = [2010, 2011, 2012, 2013, 2014, 2015]
yield_apples = [0.895, 0.91, 0.919, 0.926, 0.929, 0.931]
```

```
plt.plot(years, yield_apples);
```



**Much Better!** Now the X-axis shows actual years, making the plot meaningful.

**Syntax:** `plt.plot(x_values, y_values)`

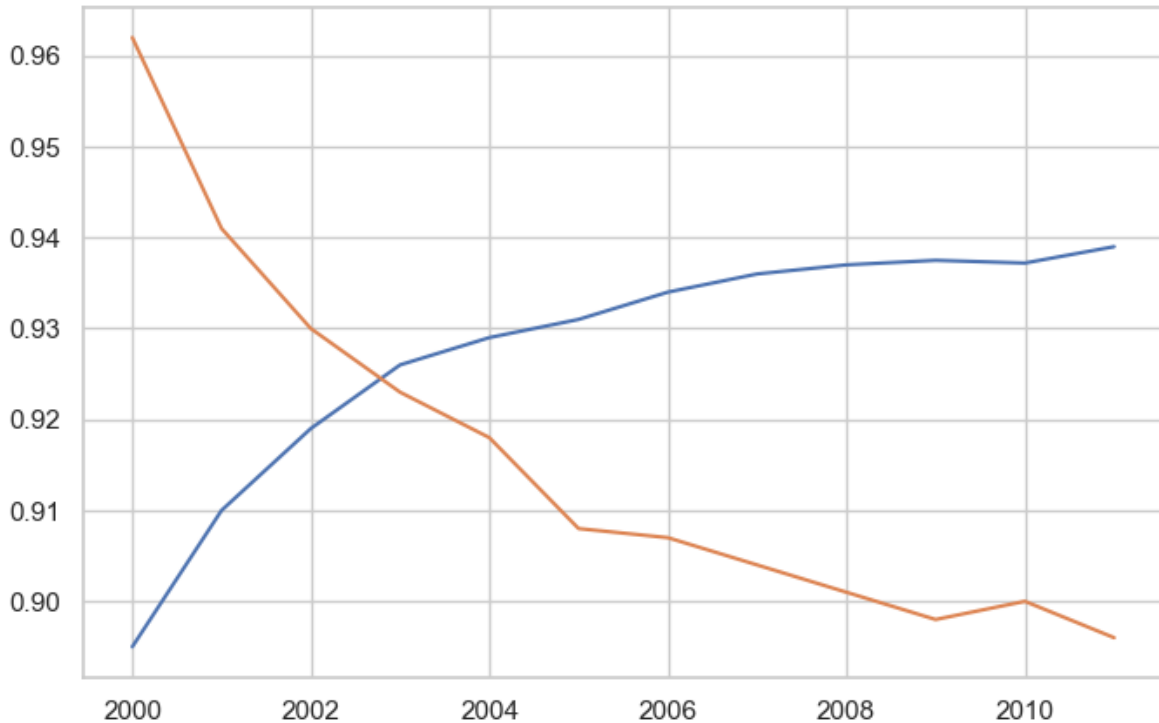
### 11.5.5 Step 4: Plotting Multiple Lines

You can create multiple lines on the same plot by calling `plt.plot()` multiple times. Each call adds another line to the current figure.

**Use Case:** Let's compare the yields of apples vs. oranges over time.

```
years = range(2000, 2012)
apples = [0.895, 0.91, 0.919, 0.926, 0.929, 0.931, 0.934, 0.936, 0.937, 0.9375, 0.9372, 0.938]
oranges = [0.962, 0.941, 0.930, 0.923, 0.918, 0.908, 0.907, 0.904, 0.901, 0.898, 0.9, 0.896]
```

```
plt.plot(years, apples)
plt.plot(years, oranges);
```



**Result:** Matplotlib automatically assigns different colors to each line and displays both on the same axes.

### Default Settings:

When `plt.plot()` is called without formatting parameters, pyplot uses these defaults:

| Property          | Default Value         |
|-------------------|-----------------------|
| Figure size       | 6.4 × 4.8 inches      |
| Line style        | Solid line            |
| Line width        | 1.5                   |
| First line color  | Blue (#1f77b4)        |
| Subsequent colors | Automatic color cycle |

### Customizing Global Defaults:

You can change default settings for all future plots using `matplotlib.rcParams`:

```
import matplotlib
matplotlib.rcParams['font.size'] = 14
matplotlib.rcParams['figure.figsize'] = (7, 4)
matplotlib.rcParams['figure.facecolor'] = '#00000000'
```

**Note:** While we're focusing on the pyplot interface in this section, Matplotlib also offers an object-oriented interface that provides even more control. We'll explore that in the next chapter.

For more on customizing defaults, see: [Customizing Matplotlib with rcParams](#)

### 11.5.6 Step 5: Adding Labels, Titles, and Legends

A good plot needs clear labels so readers understand what they're looking at. Let's enhance our plot with descriptive text elements.

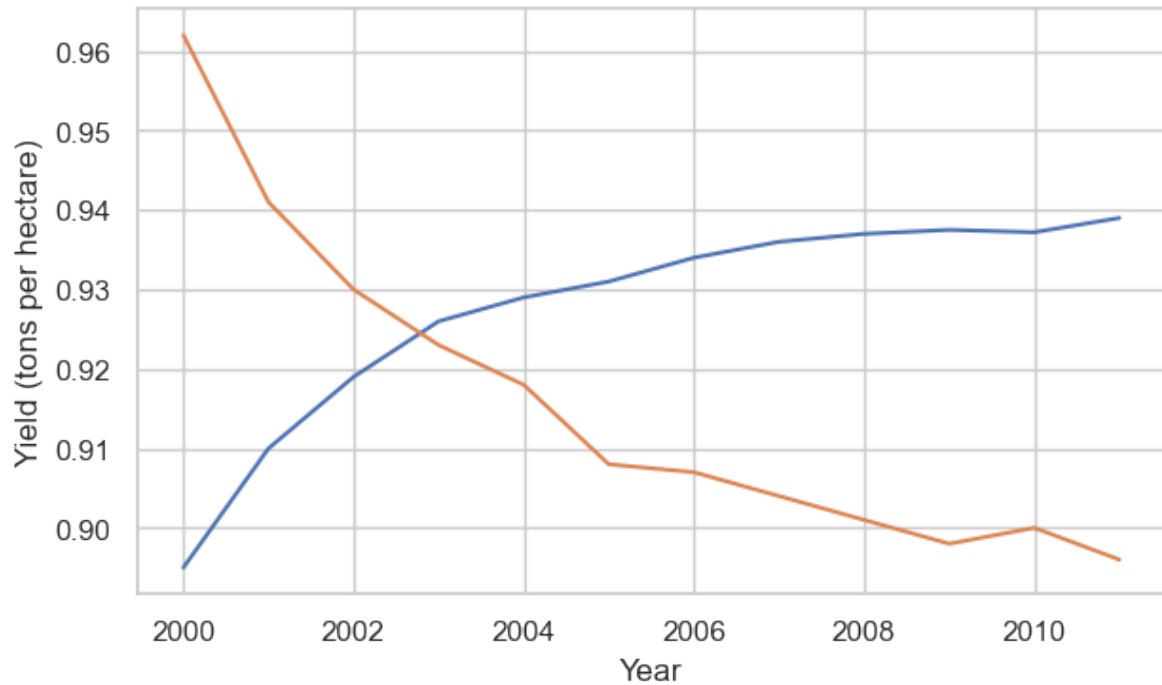
**Every Complete Plot Should Have:**

- **Title** - What the plot shows
- **Axis Labels** - What each axis represents (with units!)
- **Legend** - Which line is which (when plotting multiple series)

#### 11.5.6.1 Adding Axis Labels

Let's start by adding labels to our axes using `plt.xlabel()` and `plt.ylabel()`:

```
plt.plot(years, apples)
plt.plot(years, oranges)
plt.xlabel('Year')
plt.ylabel('Yield (tons per hectare)');
```



**Good!** But which line is which? Let's add a title and legend.

#### 11.5.6.2 Adding Title and Legend

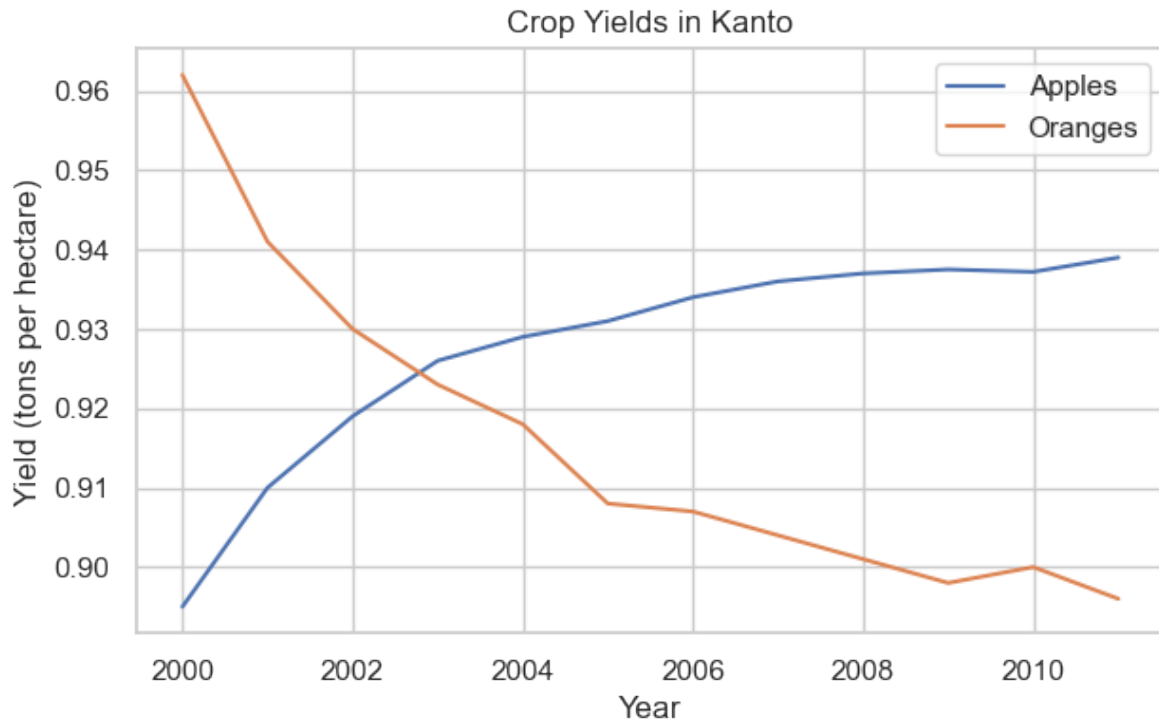
Use `plt.title()` for the title and `plt.legend()` to identify each line:

```
plt.plot(years, apples)
plt.plot(years, oranges)

plt.xlabel('Year')
plt.ylabel('Yield (tons per hectare)')

plt.title("Crop Yields in Kanto")
plt.legend(['Apples', 'Oranges']);
```





**Perfect!** Now our plot is fully labeled and easy to understand.

### 11.5.7 Step 6: Styling Lines and Markers

Matplotlib offers extensive customization for how lines and markers appear. Let's explore the options.

#### 11.5.7.1 Adding Markers

Show data points explicitly using the `marker` parameter. Matplotlib provides many marker styles:

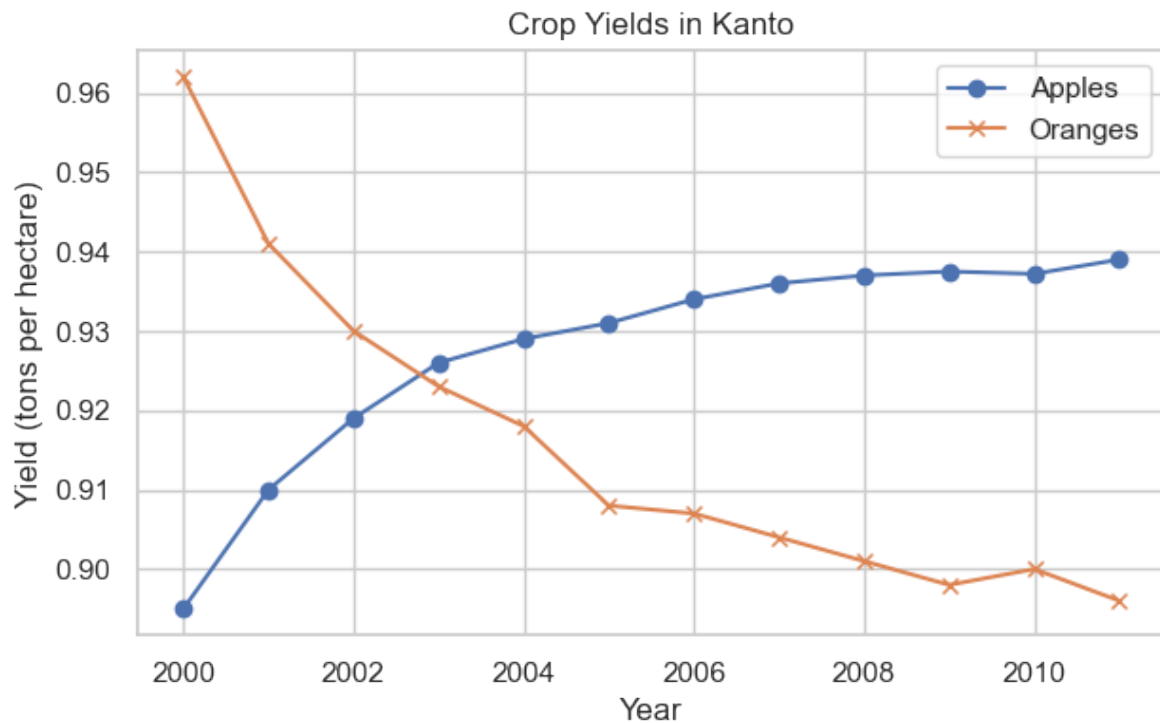
- 'o' = circle
- 'x' = X mark
- 's' = square
- '^' = triangle up
- '\*' = star
- '+' = plus sign

See the [full list of markers](#).

```
plt.plot(years, apples, marker='o')
plt.plot(years, oranges, marker='x')

plt.xlabel('Year')
plt.ylabel('Yield (tons per hectare)')

plt.title("Crop Yields in Kanto")
plt.legend(['Apples', 'Oranges']);
```



**Great!** The markers make it easier to see individual data points.

### 11.5.7.2 Complete Styling Options

The `plt.plot()` function supports many styling arguments:

| Parameter       | Alias | Purpose                                    | Example                       |
|-----------------|-------|--------------------------------------------|-------------------------------|
| color           | c     | Set line color                             | color='red' or<br>c='#FF0000' |
| linestyle       | ls    | Line style (solid,<br>dashed, etc.)        | linestyle='--' or<br>ls=':'   |
| linewidth       | lw    | Line thickness                             | linewidth=2 or lw=3           |
| marker          | -     | Marker style                               | marker='o'                    |
| markersize      | ms    | Marker size                                | markersize=10 or<br>ms=8      |
| markeredgecolor | mec   | Marker edge color                          | mec='navy'                    |
| markeredgewidth | mew   | Marker edge<br>thickness                   | mew=2                         |
| markerfacecolor | mfc   | Marker fill color                          | mfc='lightblue'               |
| alpha           | -     | Transparency<br>(0=invisible,<br>1=opaque) | alpha=0.7                     |

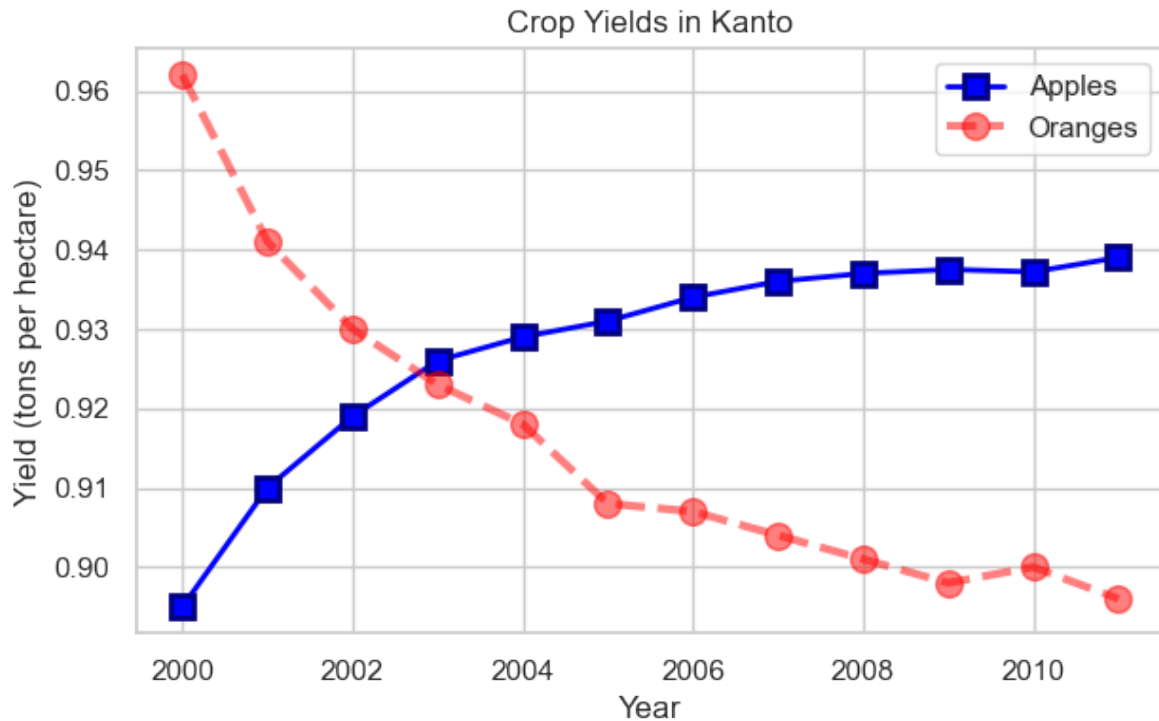
**Documentation:** [plt.plot\(\) reference](#)

**Example with Multiple Styling Parameters:**

```
plt.plot(years, apples, marker='s', c='b', ls='-', lw=2, ms=8, mew=2, mec='navy')
plt.plot(years, oranges, marker='o', c='r', ls='--', lw=3, ms=10, alpha=.5)

plt.xlabel('Year')
plt.ylabel('Yield (tons per hectare)')

plt.title("Crop Yields in Kanto")
plt.legend(['Apples', 'Oranges']);
```



**Impressive!** We've created a highly customized plot with thick lines, custom colors, different markers, and transparency.

### 11.5.7.3 The `fmt` Shorthand

For quick styling, Matplotlib provides the `fmt` string format as a shorthand:

**Syntax:** `fmt = '[marker][line][color]'`

**Common Examples:**

- `'o-r'` = circle markers, solid line, red color
- `'x--b'` = X markers, dashed line, blue color
- `'^:g'` = triangle markers, dotted line, green color
- `'sb'` = square markers, no line, blue color

**Color Codes:**

- `'b'` = blue, `'g'` = green, `'r'` = red, `'c'` = cyan
- `'m'` = magenta, `'y'` = yellow, `'k'` = black, `'w'` = white

**Line Styles:**

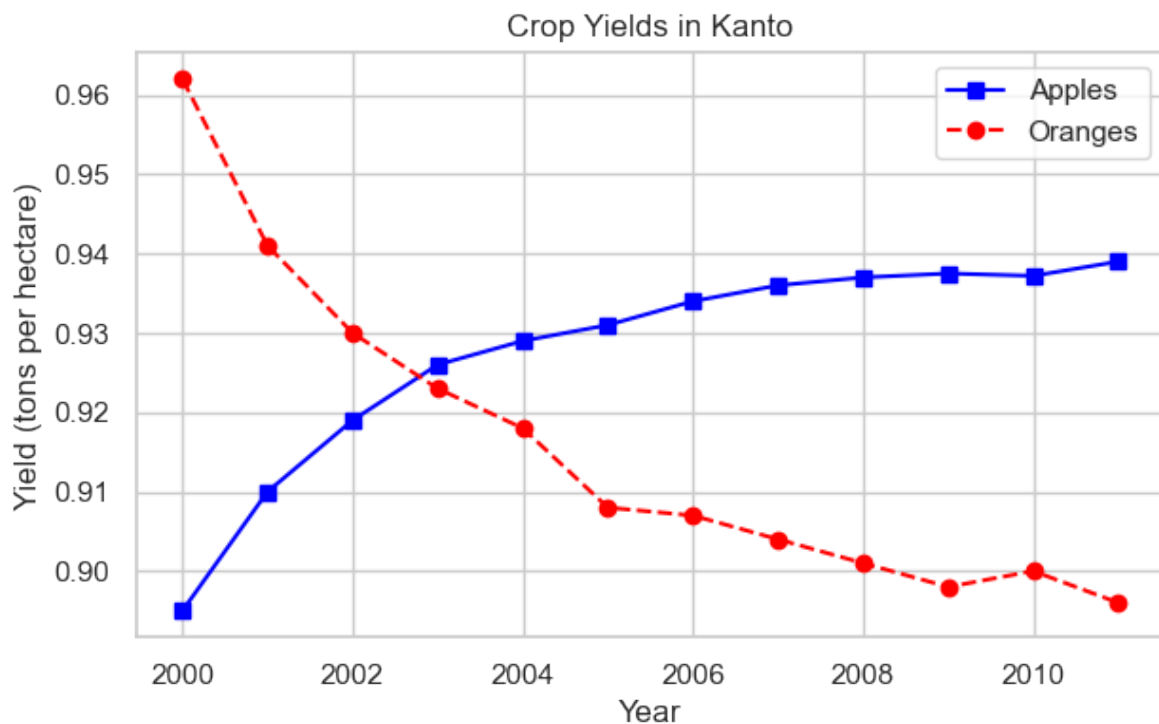
- '-' = solid, '--' = dashed, ':' = dotted, '-.' = dash-dot

**Example:**

```
plt.plot(years, apples, 's-b')
plt.plot(years, oranges, 'o--r')

plt.xlabel('Year')
plt.ylabel('Yield (tons per hectare)')

plt.title("Crop Yields in Kanto")
plt.legend(['Apples', 'Oranges']);
```

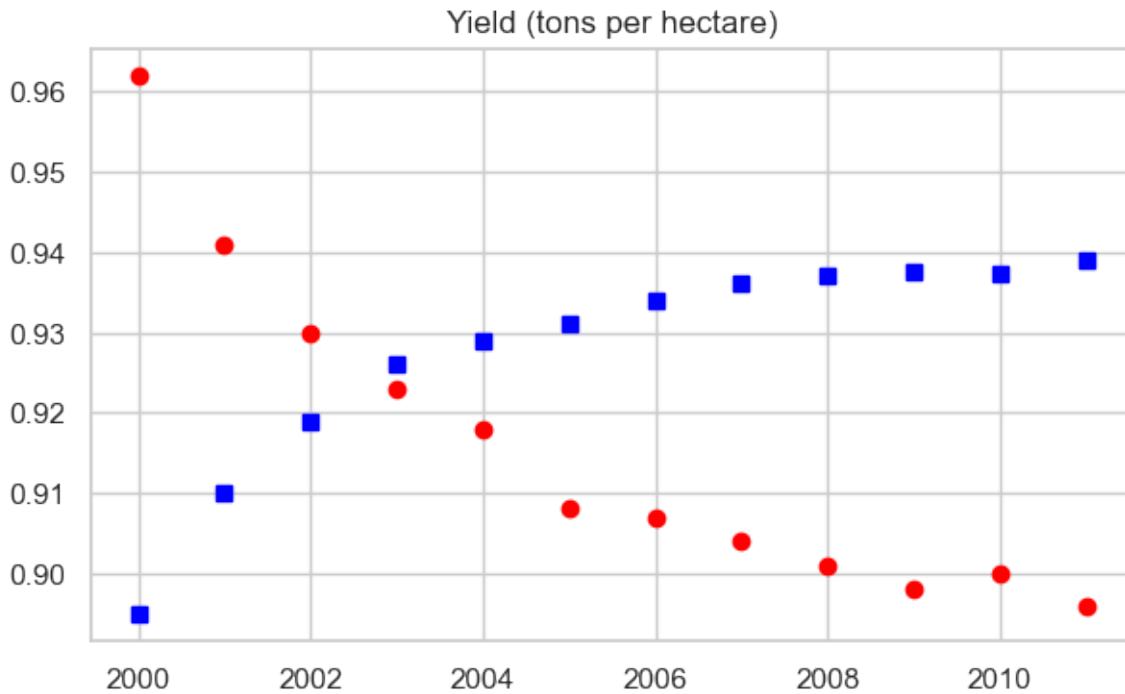


**Much simpler code!** The fmt string 's-b' means “square markers, solid line, blue” and 'o--r' means “circle markers, dashed line, red”.

#### 11.5.7.4 Markers Without Lines

If you don't specify a line style in `fmt`, only markers are drawn (no connecting lines):

```
plt.plot(years, apples, 'sb')
plt.plot(years, oranges, 'or')
plt.title("Yield (tons per hectare)");
```



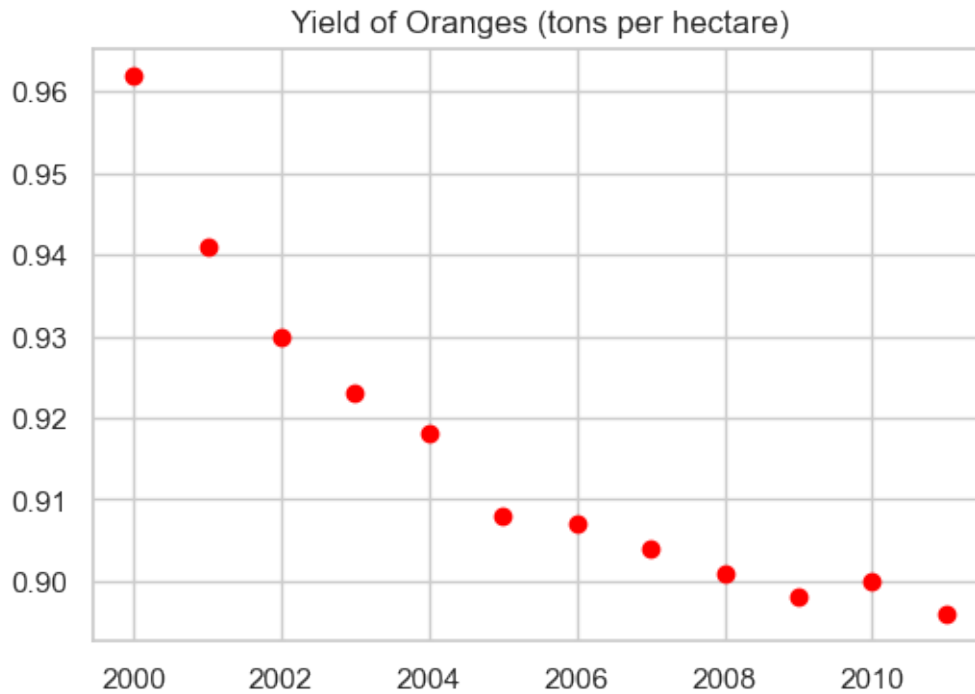
**Result:** Only markers, no lines. Perfect for scatter-like visualizations!

### 11.5.8 Step 7: Controlling Figure Size

By default, Matplotlib creates figures that are  $6.4 \times 4.8$  inches. You can change this using `plt.figure(figsize=(width, height))`:

```
plt.figure(figsize=(6, 4))

plt.plot(years, oranges, 'or')
plt.title("Yield of Oranges (tons per hectare)");
```



**Important:** Call `plt.figure()` BEFORE `plt.plot()` to set the size for that specific figure.

### 11.5.9 Step 8: Other Plot Types with pyplot

So far we've focused on line plots, but pyplot supports many other visualization types. Let's explore them using a different dataset.

#### New Dataset: FIFA Player Data

Let's load FIFA player statistics to demonstrate various plot types:

```
fifa = pd.read_csv('./Datasets/fifa_data.csv')
fifa.head(5)
```

|   | Unnamed: 0 | ID     | Name              | Age | Photo                                                                                                       | Nation |
|---|------------|--------|-------------------|-----|-------------------------------------------------------------------------------------------------------------|--------|
| 0 | 0          | 158023 | L. Messi          | 31  | <a href="https://cdn.sofifa.org/players/4/19/158023.png">https://cdn.sofifa.org/players/4/19/158023.png</a> | Argen  |
| 1 | 1          | 20801  | Cristiano Ronaldo | 33  | <a href="https://cdn.sofifa.org/players/4/19/20801.png">https://cdn.sofifa.org/players/4/19/20801.png</a>   | Portu  |
| 2 | 2          | 190871 | Neymar Jr         | 26  | <a href="https://cdn.sofifa.org/players/4/19/190871.png">https://cdn.sofifa.org/players/4/19/190871.png</a> | Brazil |
| 3 | 3          | 193080 | De Gea            | 27  | <a href="https://cdn.sofifa.org/players/4/19/193080.png">https://cdn.sofifa.org/players/4/19/193080.png</a> | Spain  |
| 4 | 4          | 192985 | K. De Bruyne      | 27  | <a href="https://cdn.sofifa.org/players/4/19/192985.png">https://cdn.sofifa.org/players/4/19/192985.png</a> | Belgiu |

### 11.5.9.1 Histogram: Distribution of Values

**Use Case:** Understanding the distribution of player skill levels

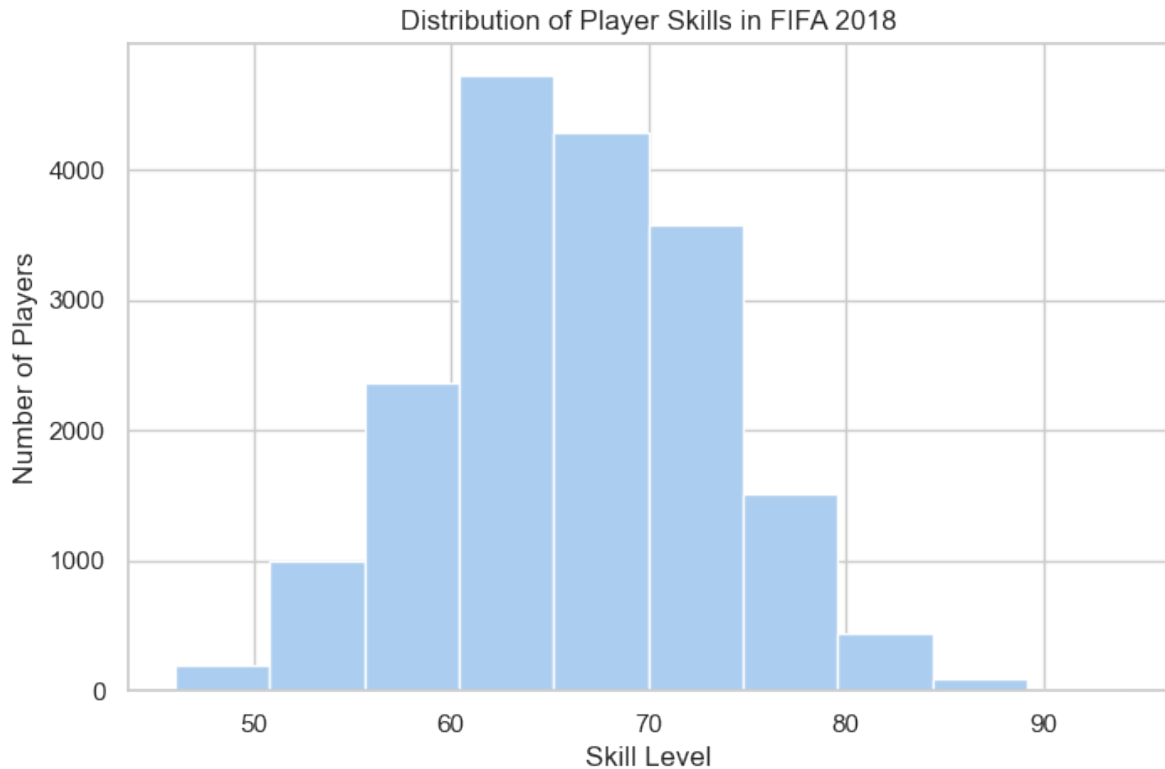
**Function:** `plt.hist(data, bins=number, color='color')`

```
plt.figure(figsize=(8,5))

plt.hist(fifa.Overall, color='#abcdef')

plt.ylabel('Number of Players')
plt.xlabel('Skill Level')
plt.title('Distribution of Player Skills in FIFA 2018')
```

`Text(0.5, 1.0, 'Distribution of Player Skills in FIFA 2018')`



**Observation:** Player skills follow a roughly normal distribution, with most players having moderate skills (60-70 range).



### 11.5.9.2 Bar Chart: Comparing Categories

**Use Case:** Comparing counts between categories

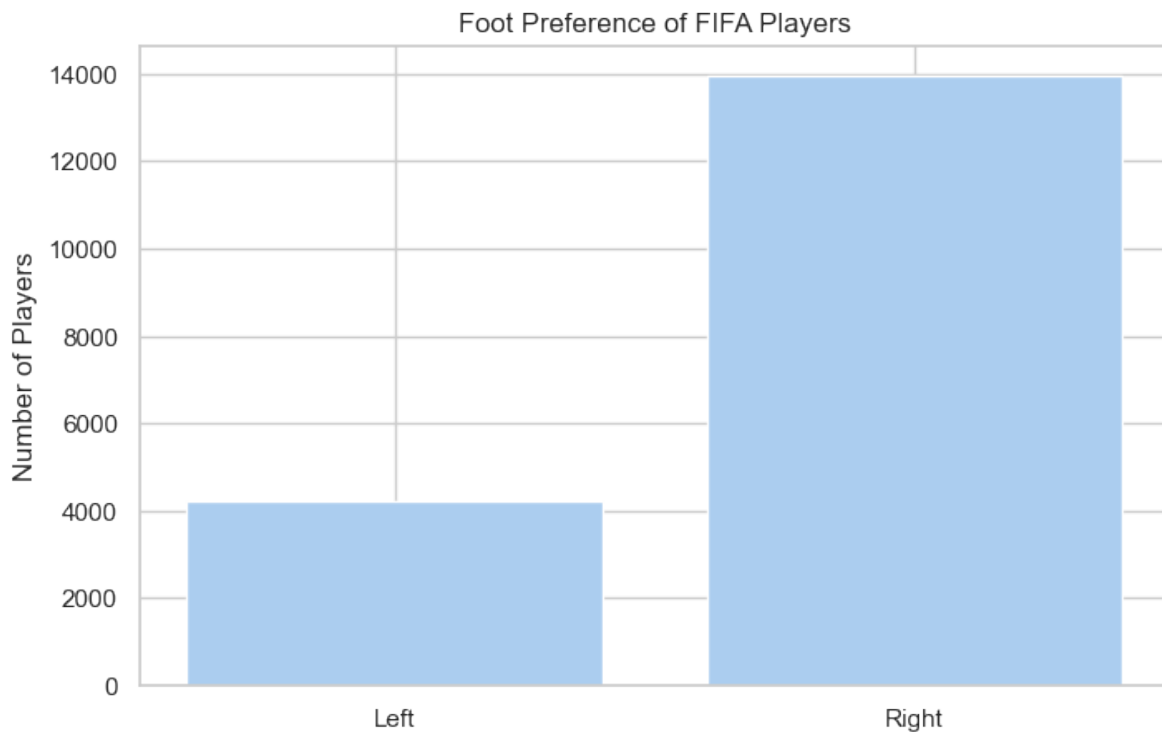
**Function:** `plt.bar(categories, values, color='color')`

```
plotting bar chart for the best players
plt.figure(figsize=(8,5))

foot_preference = fifa['Preferred Foot'].value_counts()

plt.bar(['Left', 'Right'], [foot_preference.iloc[1], foot_preference.iloc[0]], color='#abcde')

plt.ylabel('Number of Players')
plt.title('Foot Preference of FIFA Players');
```



**Observation:** More FIFA players prefer their right foot over their left.

### 11.5.9.3 Pie Chart: Parts of a Whole

**Use Case:** Showing proportions/percentages

**Function:** `plt.pie(values, labels=labels, autopct='format')`

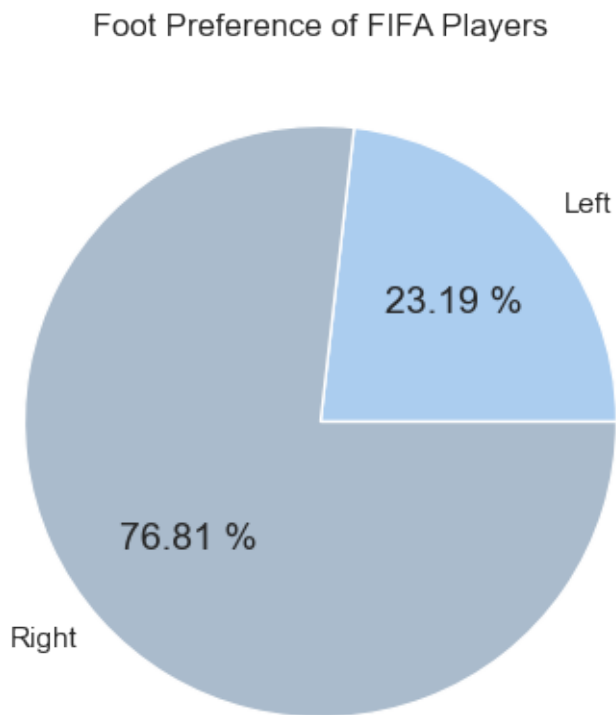
```
left = fifa.loc[fifa['Preferred Foot'] == 'Left'].count().iloc[0]
right = fifa.loc[fifa['Preferred Foot'] == 'Right'].count().iloc[0]

plt.figure(figsize=(8,5))

labels = ['Left', 'Right']
colors = ['#abcdef', '#aabbcc']

plt.pie([left, right], labels = labels, colors=colors, autopct='%.2f %%')

plt.title('Foot Preference of FIFA Players');
```



**Tip:** The `autopct='%.2f %'` parameter automatically calculates and displays percentages.

#### 11.5.9.4 Advanced Pie Chart with Explode

You can “explode” (pull out) specific slices to emphasize them:

```

plt.figure(figsize=(8,5), dpi=100)

plt.style.use('ggplot')

fifa.Weight = [int(x.strip('lbs')) if type(x)==str else x for x in fifa.Weight]

light = fifa.loc[fifa.Weight < 125].count().iloc[0]
light_medium = fifa[(fifa.Weight >= 125) & (fifa.Weight < 150)].count().iloc[0]
medium = fifa[(fifa.Weight >= 150) & (fifa.Weight < 175)].count().iloc[0]
medium_heavy = fifa[(fifa.Weight >= 175) & (fifa.Weight < 200)].count().iloc[0]
heavy = fifa[fifa.Weight >= 200].count().iloc[0]

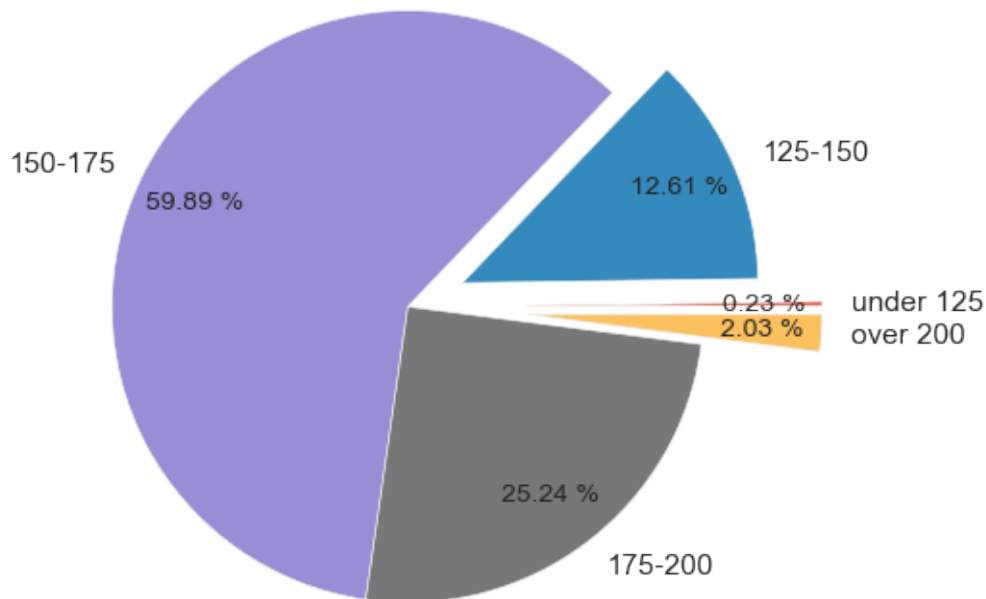
weights = [light, light_medium, medium, medium_heavy, heavy]
label = ['under 125', '125-150', '150-175', '175-200', 'over 200']
explode = (.4, .2, 0, 0, .4)

plt.title('Weight of Professional Soccer Players (lbs)')

plt.pie(weights, labels=label, explode=explode, pctdistance=0.8, autopct='% .2f %%');

```

## Weight of Professional Soccer Players (lbs)



### Advanced Features:

- `explode` parameter pulls out slices (0 = normal, 0.4 = pulled out significantly)
- `plt.style.use('ggplot')` applies a different visual style
- `pctdistance` controls where percentages are positioned

### 11.5.9.5 Box Plot: Distribution Summary

**Use Case:** Comparing distributions across groups, identifying outliers

**Function:** `plt.boxplot(data_list, tick_labels=labels)`

```
plt.figure(figsize=(5,8), dpi=100)

plt.style.use('default')

barcelona = fifa.loc[fifa.Club == "FC Barcelona"]['Overall']
madrid = fifa.loc[fifa.Club == "Real Madrid"]['Overall']
revs = fifa.loc[fifa.Club == "New England Revolution"]['Overall']
```

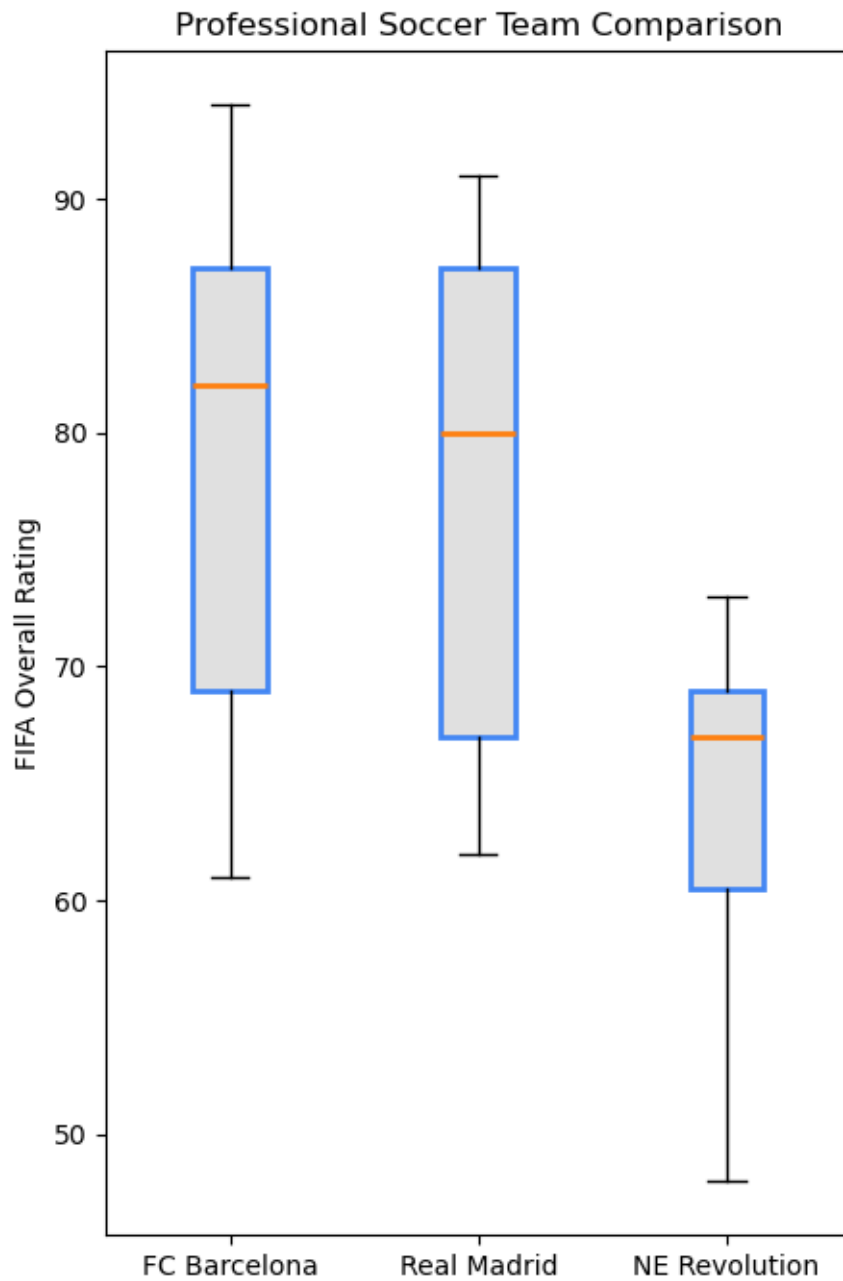
```

bp = plt.boxplot([barcelona, madrid, revs], labels=['a','b','c'], boxprops=dict(facecolor=
bp = plt.boxplot([barcelona, madrid, revs], labels=['FC Barcelona','Real Madrid','NE Revolut

plt.title('Professional Soccer Team Comparison')
plt.ylabel('FIFA Overall Rating')

for box in bp['boxes']:
 # change outline color
 box.set(color='#4286f4', linewidth=2)
 # change fill color
 box.set(facecolor = '#e0e0e0')
 # change hatch
 #box.set(hatch = '/')

```



#### Box Plot Elements:

- **Box** = 25th to 75th percentile (middle 50% of data)
- **Orange line** = Median
- **Whiskers** = Extend to show data range
- **Circles** = Outliers

**Observation:** FC Barcelona and Real Madrid have higher overall player ratings compared to New England Revolution.

#### 11.5.9.6 Strengths of Matplotlib pyplot

- **Maximum Control** - Customize every element
- **Publication Quality** - Professional output suitable for papers
- **Extensive Plot Types** - Line, scatter, bar, histogram, pie, box, and many more
- **Well Documented** - Large community and comprehensive documentation
- **Foundation** - Base for Pandas, Seaborn, and other libraries

#### 11.5.9.7 Essential Best Practices

##### 1. Set Figure Size Early

```
plt.figure(figsize=(width, height)) # Before plt.plot()
```

##### 2. Always Label Your Axes

```
plt.xlabel('X-axis label (with units!)\nplt.ylabel('Y-axis label (with units!)\n
```

##### 3. Add Descriptive Titles

```
plt.title('Clear description of what the plot shows')
```

##### 4. Include Legends for Multiple Lines

```
plt.legend(['Series 1', 'Series 2'])
```

##### 5. Use Semicolons in Jupyter

```
plt.plot(x, y); # Suppresses unwanted text output
```

##### 6. Save High-Quality Figures

```
plt.savefig('filename.png', dpi=300, bbox_inches='tight')
```

#### 11.5.9.8 When to Use pyplot

##### Best For:

- Creating custom visualizations with specific requirements
- Building complex multi-panel layouts (covered in next chapter)
- Fine-tuning every visual element

- Generating publication-ready figures
- When you need maximum control

### 11.5.10 What's Next?

You've mastered the **pyplot interface** - the state-based, MATLAB-style approach to plotting.

In the next chapter, we'll explore Matplotlib's **Object-Oriented Interface**, which gives you even more control and is better for:

- Creating complex multi-panel figures
- Writing reusable plotting functions
- Managing multiple figures simultaneously
- Professional data visualization workflows

#### Preview of OO Interface:

```
pyplot style (what we just learned)
plt.plot(x, y)
plt.xlabel('X Label')

Object-oriented style (next chapter)
fig, ax = plt.subplots()
ax.plot(x, y)
ax.set_xlabel('X Label')
```

The OO interface is the professional standard for complex visualizations, but everything you learned about styling, colors, and plot types applies equally to both interfaces!

## 11.6 Plotting with Seaborn

**Seaborn** is a powerful data visualization library built on top of Matplotlib, designed to make statistical plots easier and more attractive. It provides:

- **High-level interface** for drawing attractive statistical graphics
- **Beautiful default styles** that are publication-ready out of the box
- **Seamless Pandas integration** with direct DataFrame support
- **Built-in statistical computations** (confidence intervals, regression lines, etc.)
- **Specialized plot types** for statistical analysis

Think of it this way:



|                             |                              |
|-----------------------------|------------------------------|
| Pandas <code>.plot()</code> | ← Quickest, least control    |
| Seaborn                     | ← Beautiful + Statistical    |
| Matplotlib                  | ← Most control, most verbose |

### 11.6.1 Step 1: Why Choose Seaborn Over Matplotlib?

Seaborn addresses several pain points of using Matplotlib directly:

#### 11.6.1.1 Problem 1: Default Aesthetics

**Matplotlib Challenge:** - Default styles can look dated - Requires manual styling for professional appearance - Grid, colors, and fonts need explicit configuration

**Seaborn Solution:** - Modern, publication-ready styles out of the box - Professional appearance with zero configuration - Multiple preset themes for different contexts

#### 11.6.1.2 Problem 2: Statistical Visualizations

**Matplotlib Challenge:** - Requires extensive code for statistical plots - Manual calculation of confidence intervals - Complex code for regression lines and error bars

**Seaborn Solution:** - Built-in statistical functions - Automatic confidence interval computation - One-line regression plots with `regplot()`

#### 11.6.1.3 Problem 3: DataFrame Integration

**Matplotlib Challenge:** - Works primarily with arrays and lists - Requires extracting columns manually - No automatic label generation from column names

**Seaborn Solution:** - Native DataFrame support via `data` parameter - Column names automatically become labels - Natural, intuitive syntax: `x='column_name'`

#### 11.6.1.4 Problem 4: Complex Multi-Panel Plots

**Matplotlib Challenge:** - Subplots require significant boilerplate code - Manual management of figure and axes objects - Difficult to create consistent multi-plot layouts

**Seaborn Solution:** - FacetGrid for automatic multi-panel layouts - PairPlot for pairwise relationship matrices - Simplified API for complex visualizations

#### 11.6.2 Step 2: Setup and Aesthetic Customization

Before creating plots, let's learn how to configure Seaborn's appearance.

First, let's import Seaborn and explore its built-in datasets:

##### Seaborn Datasets:

Seaborn comes with 17 built-in datasets, perfect for learning and practice. This means you can focus on visualization techniques without spending time finding and cleaning data.

```
import seaborn as sns
get names of the builtin dataset
sns.get_dataset_names()
```

```
['anagrams',
 'anscombe',
 'attention',
 'brain_networks',
 'car_crashes',
 'diamonds',
 'dots',
 'dowjones',
 'exercise',
 'flights',
 'fmri',
 'geyser',
 'glue',
 'healthexp',
 'iris',
 'mpg',
 'penguins',
 'planets',
 'seaice',
 'taxis',
```

```
'tips',
'titanic']
```

**Great!** Now you can use any of these datasets with `sns.load_dataset('dataset_name')`.

### 11.6.2.1 Customizing Plot Aesthetics

Seaborn provides powerful functions to customize the visual appearance of plots:

#### Style Options:

Seaborn offers five built-in styles via `sns.set_style()`:

| Style       | Description                      | Best For                                       |
|-------------|----------------------------------|------------------------------------------------|
| "whitegrid" | White background with grid lines | Most presentations and reports (recommended)   |
| "darkgrid"  | Dark background with grid lines  | Data with many points or intricate details     |
| "white"     | Simple white background, no grid | Minimalist aesthetic, emphasis on data         |
| "dark"      | Dark background without grid     | Emphasizing data points, reducing distractions |
| "ticks"     | White background with axis ticks | Adding detail for precise reference            |

**Let's apply a style:**

```
sns.set_style("whitegrid")
```

Perfect! Now all our Seaborn plots will have a clean, professional appearance with white background and gridlines.

### 11.6.3 Step 3: Distribution Plots

Distribution plots help you understand how your data is spread out. Let's explore using the famous Iris flower dataset.

**Load the Dataset:**

```
Load data into a Pandas dataframe
flowers_df = sns.load_dataset("iris")
flowers_df.head()
```

|   | sepal_length | sepal_width | petal_length | petal_width | species |
|---|--------------|-------------|--------------|-------------|---------|
| 0 | 5.1          | 3.5         | 1.4          | 0.2         | setosa  |
| 1 | 4.9          | 3.0         | 1.4          | 0.2         | setosa  |
| 2 | 4.7          | 3.2         | 1.3          | 0.2         | setosa  |
| 3 | 4.6          | 3.1         | 1.5          | 0.2         | setosa  |
| 4 | 5.0          | 3.6         | 1.4          | 0.2         | setosa  |

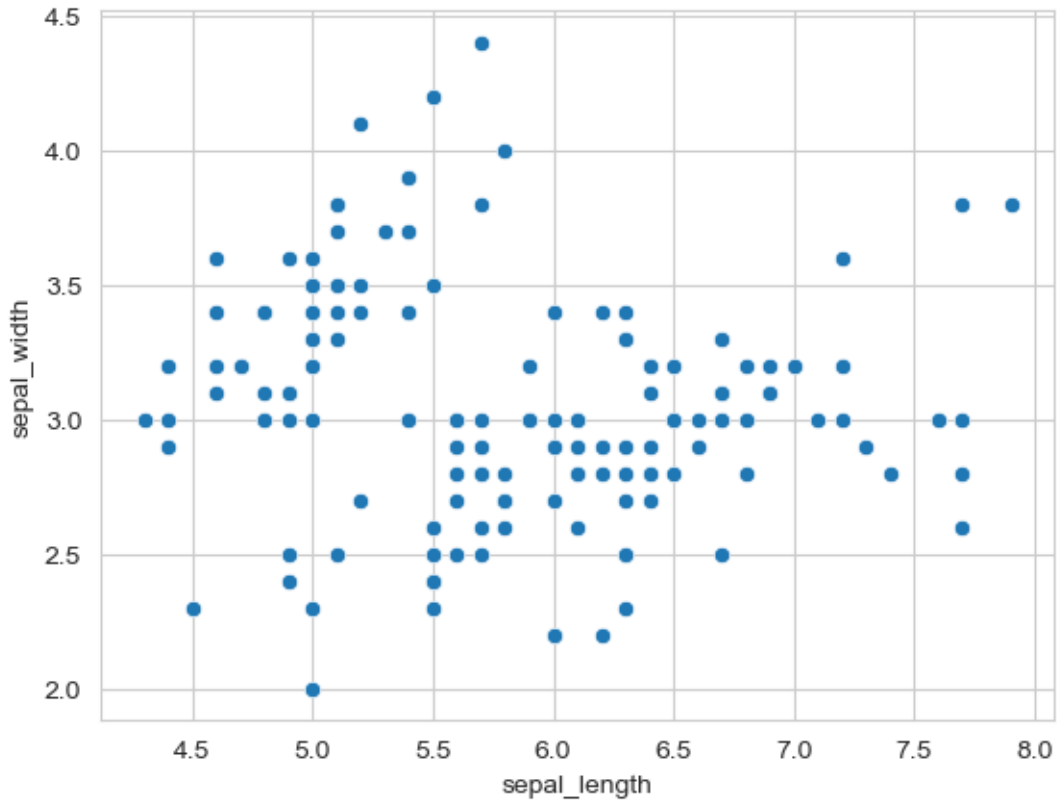
**Dataset:** The Iris dataset contains measurements of 150 iris flowers from three different species. Let's visualize the relationships between features.

#### 11.6.4 Step 4: Relational Plots - Scatter Plots

Scatter plots show relationships between two numerical variables. Seaborn's `scatterplot()` makes this easy.

##### 11.6.4.1 Basic Scatter Plot

```
sns.scatterplot(x=flowers_df.sepal_length, y=flowers_df.sepal_width);
```



**Observation:** The plot shows a general pattern, but we can see distinct clusters. This suggests different groups might exist in the data.

#### 11.6.4.2 Adding a Third Dimension with Hue

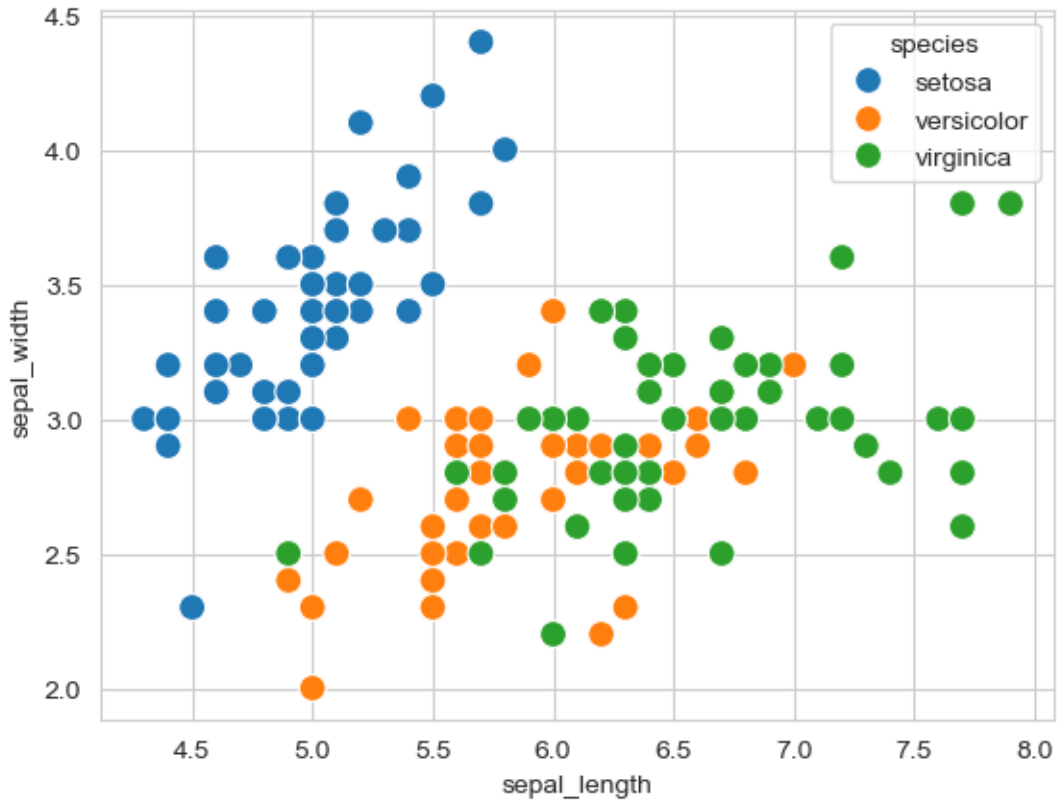
The real power of Seaborn comes from easily adding additional dimensions to your plots. The `hue` parameter colors points based on a categorical variable.

**First, let's check what species we have:**

```
flowers_df.species.unique()
```

```
array(['setosa', 'versicolor', 'virginica'], dtype=object)
```

```
sns.scatterplot(x=flowers_df.sepal_length, y=flowers_df.sepal_width, hue=flowers_df.species,
```



**Much more informative!** Now we can clearly see:

- **Setosa** flowers have smaller sepal length but larger sepal width
- **Virginica** flowers show the opposite pattern (longer, narrower sepals)
- **Versicolor** falls in between
- The three species are clearly separable based on these measurements

**Key Insight:** The `hue` parameter is one of Seaborn's most powerful features - it adds a third dimension to 2D plots with automatic color coding and legend generation.

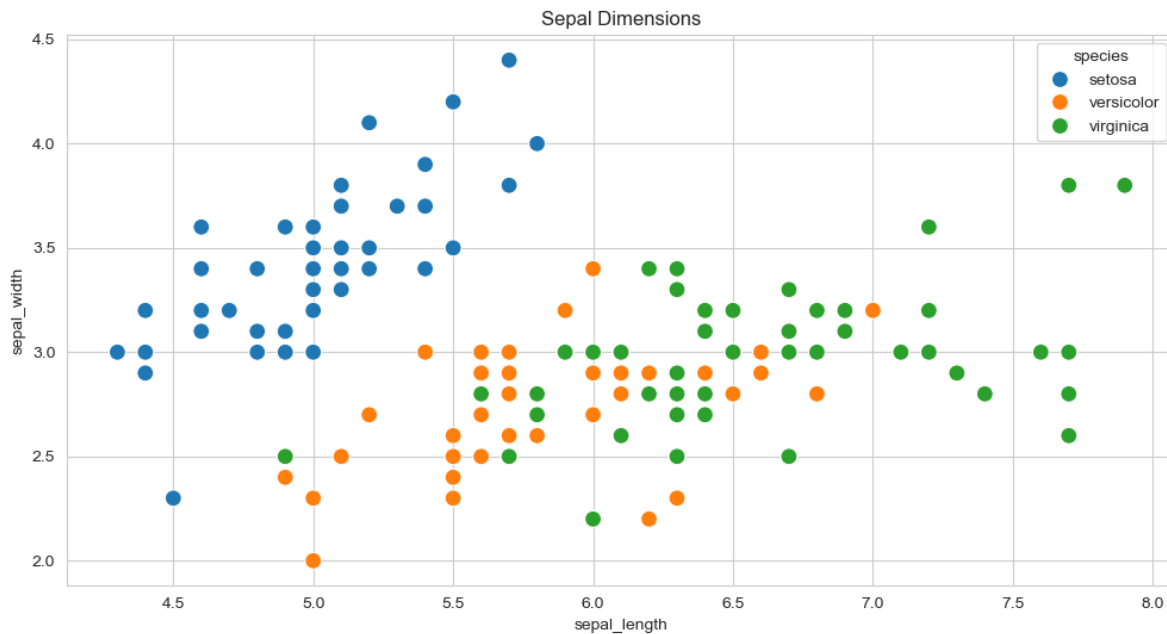
#### 11.6.4.3 Integrating Seaborn with Matplotlib

Since Seaborn is built on Matplotlib, you can use Matplotlib functions to customize Seaborn plots further:

```
plt.figure(figsize=(12, 6))
plt.title('Sepal Dimensions')

sns.scatterplot(x=flowers_df.sepal_length,
```

```
y=flowers_df.sepal_width,
hue=flowers_df.species,
s=100);
```

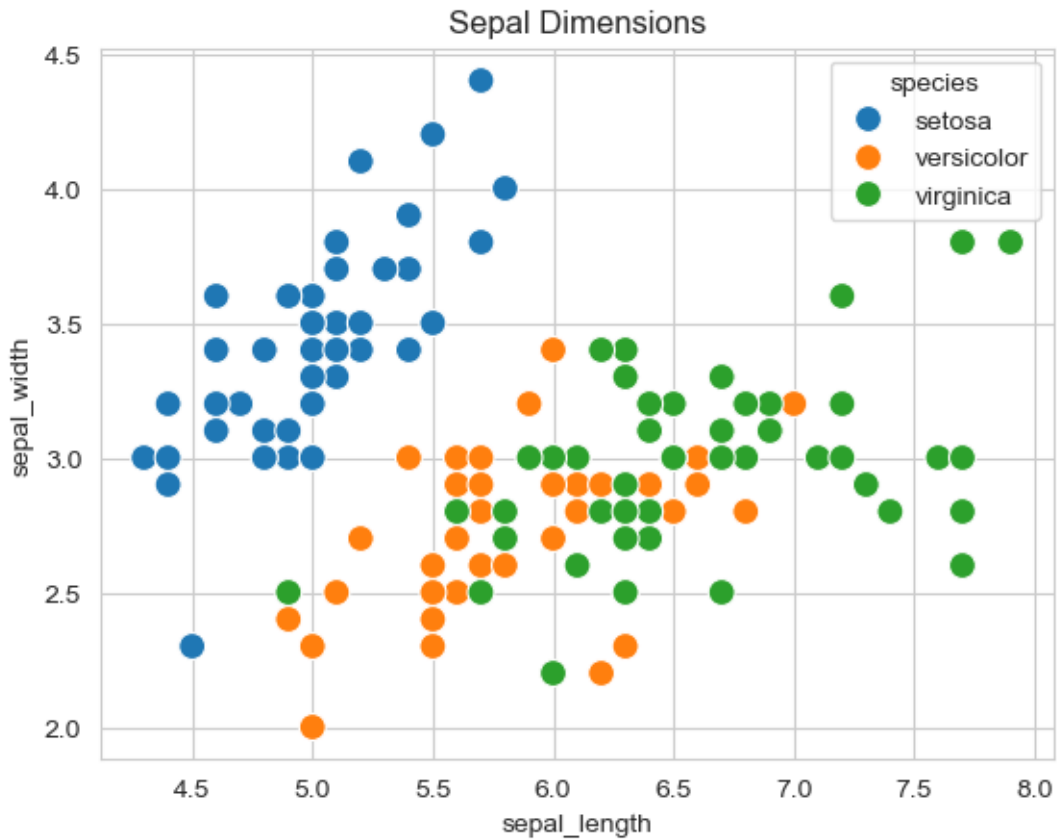


**Perfect combination!** Seaborn creates beautiful plots quickly, and Matplotlib adds fine-tuning.

#### 11.6.4.4 DataFrame Integration - The Recommended Approach

Instead of passing Series objects, Seaborn works best with the `data` parameter and column names:

```
plt.title('Sepal Dimensions')
sns.scatterplot(x='sepal_length',
 y='sepal_width',
 hue='species',
 s=100,
 data=flowers_df);
```



#### Benefits:

- Cleaner, more readable code
- Easier to modify (just change column names)
- Consistent with Seaborn's design philosophy

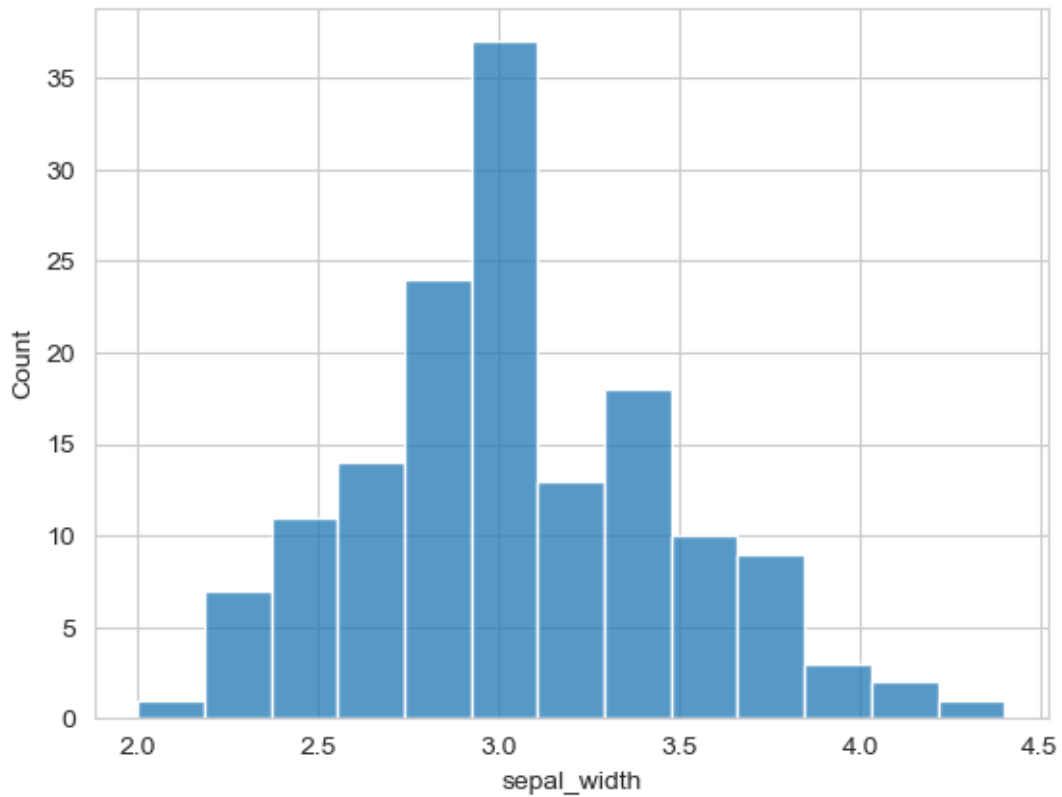
### 11.6.5 Step 5: Distribution Plots - Histograms

Histograms show the frequency distribution of a single variable. Let's visualize sepal width distribution.

#### 11.6.5.1 Basic Histogram

```
sns.histplot(data=flowers_df, x='sepal_width');
```



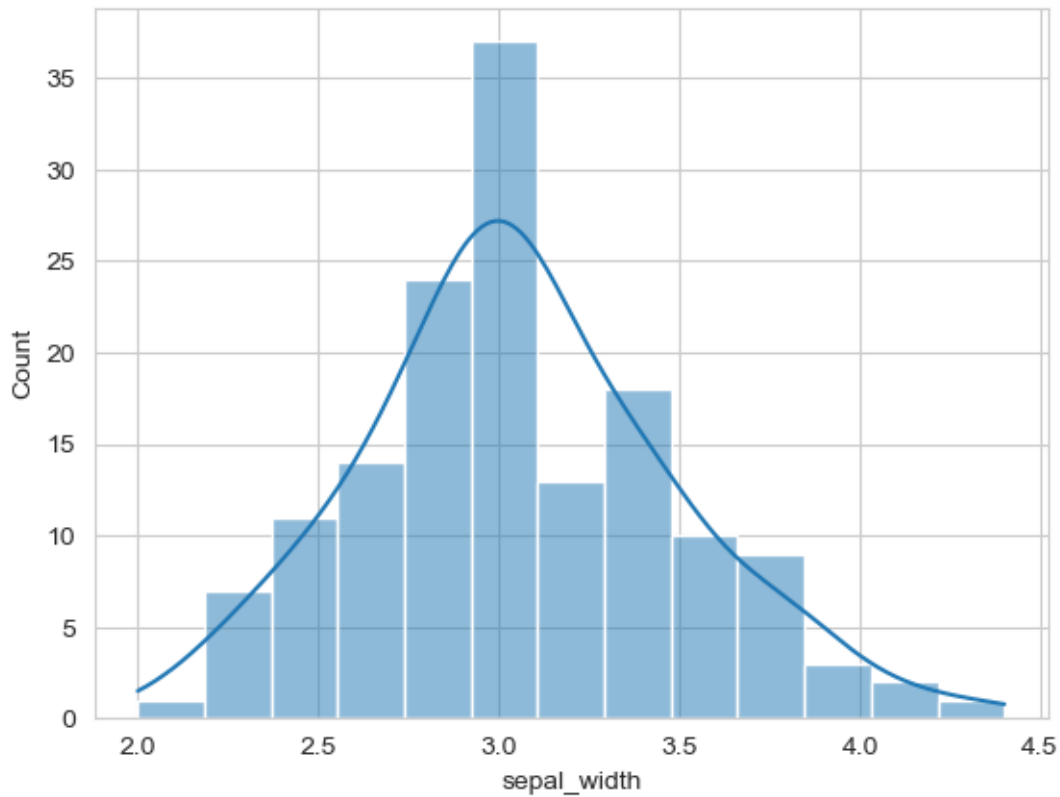


**Observation:** The distribution appears roughly bell-shaped (normal), with most values around 3.0.

#### 11.6.5.2 Adding KDE (Kernel Density Estimate)

KDE creates a smooth curve showing the distribution's shape:

```
sns.histplot(data=flowers_df, x='sepal_width', kde=True);
```

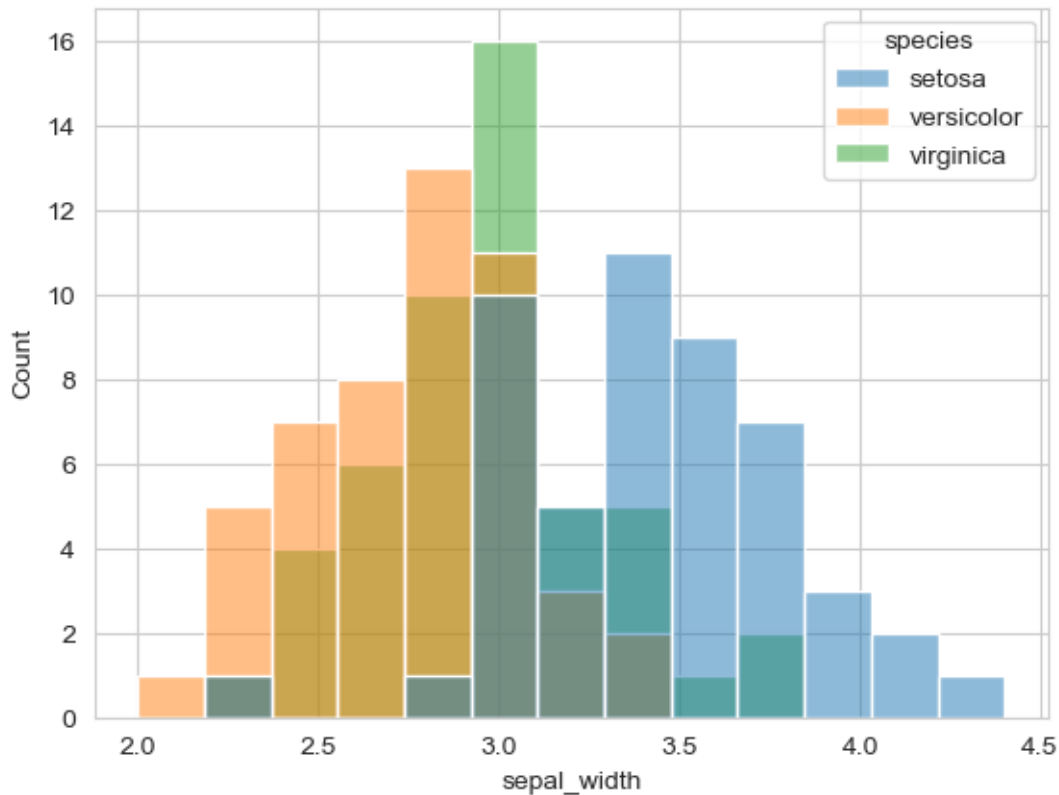


**Enhanced!** The smooth KDE curve helps visualize the overall distribution shape.

### 11.6.5.3 Comparing Distributions with Hue

Use hue to compare distributions across categories:

```
adding hue
sns.histplot(data=flowers_df, x="sepal_width", hue="species");
```



**Insight:** Different species have different sepal width distributions:

- Setosa tends to have wider sepals (peak around 3.4)
- Versicolor and Virginica have narrower sepals (peaks around 2.8-3.0)

## 11.6.6 Step 6: Categorical Plots - Bar Plots

Bar plots show aggregate statistics (like mean) for different categories. Let's use the Tips dataset.

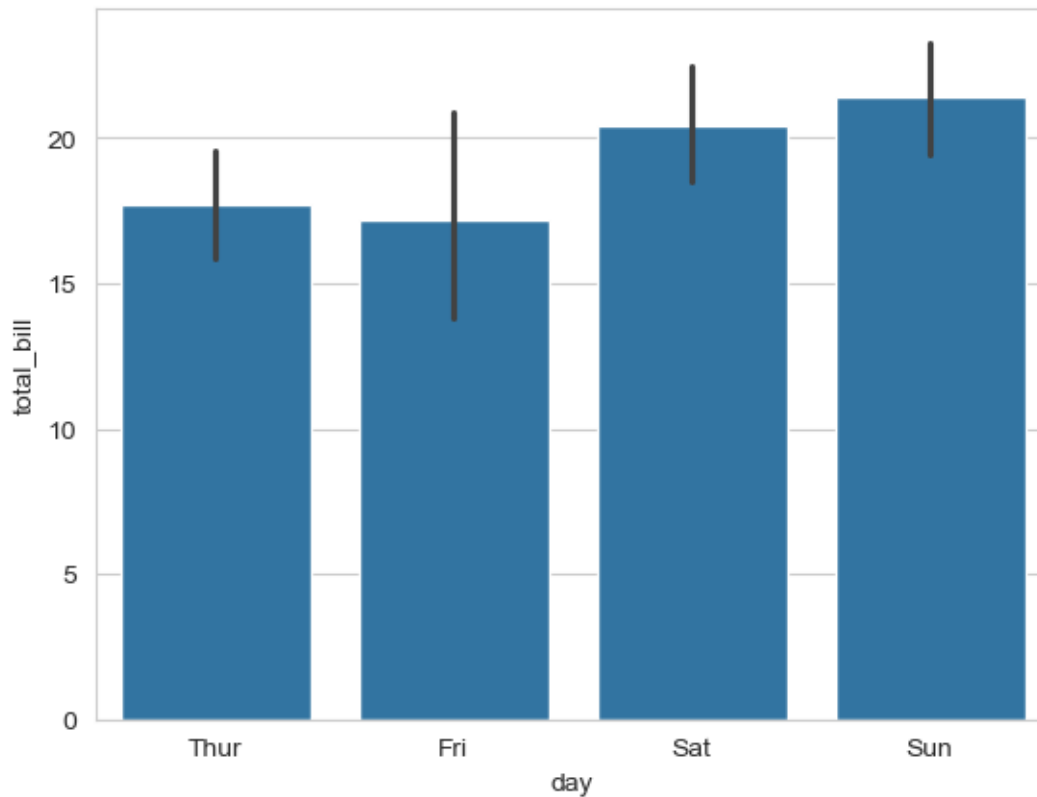
**Load Tips Dataset:**

**Dataset:** Tips received by restaurant servers, including day, time, party size, and whether the customer smoked.

### 11.6.6.1 Basic Bar Plot

By default, `barplot()` shows the mean value with confidence interval error bars:

```
tips_df = sns.load_dataset("tips")
sns.barplot(x='day', y='total_bill', data=tips_df);
```



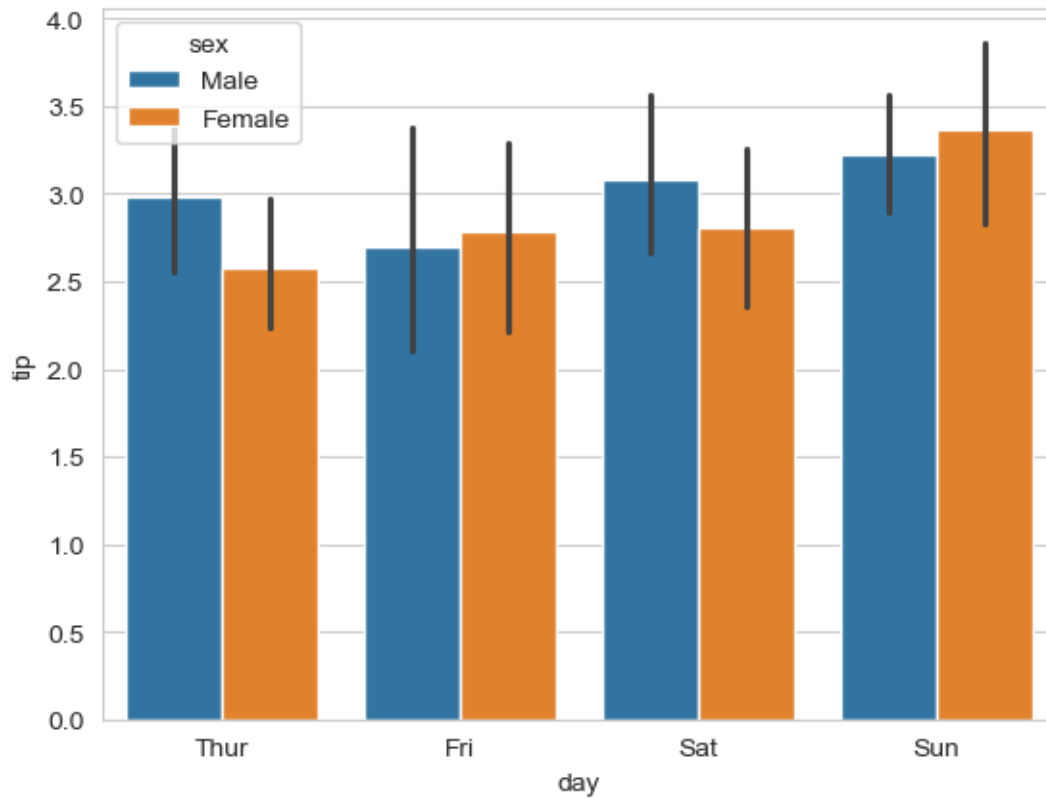
**Observation:** Weekend (Saturday/Sunday) bills tend to be higher on average than weekday bills.

**Note:** The black lines are confidence intervals showing the uncertainty in the mean estimate.

#### 11.6.6.2 Adding Hue for Comparison

Compare tips by gender across different days:

```
sns.barplot(x='day', y='tip', hue='sex', data=tips_df);
```

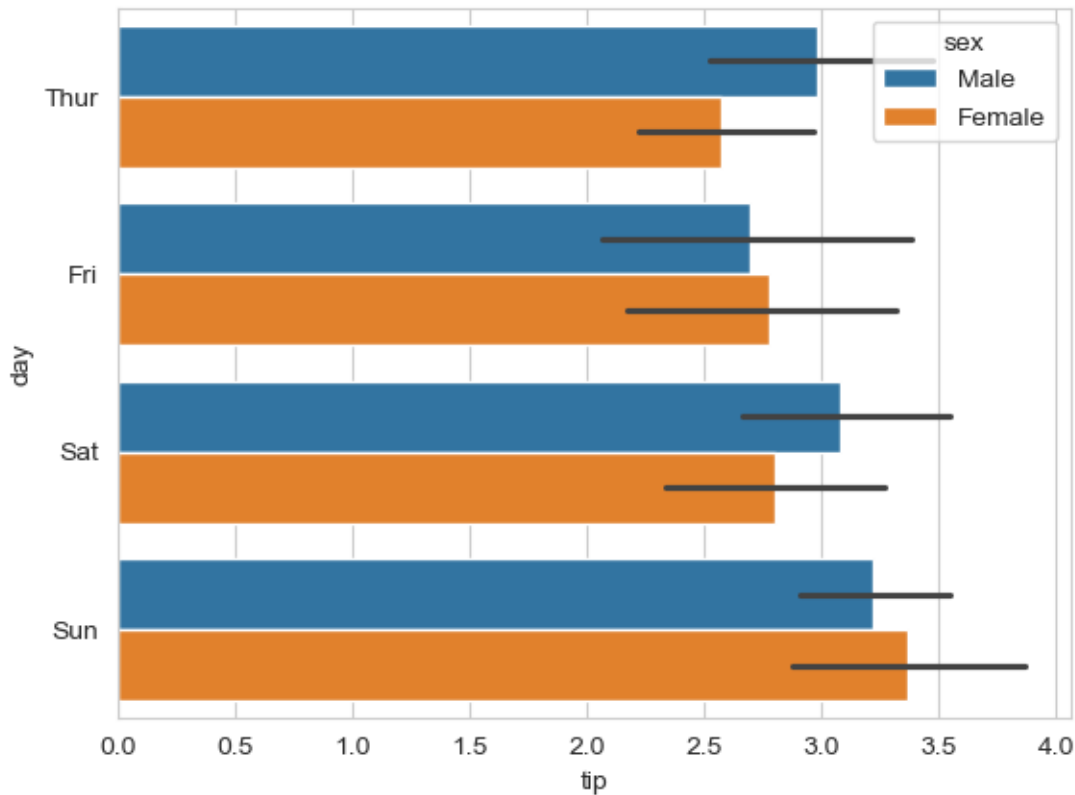


**Insight:** Males tend to give slightly higher tips across most days.

### 11.6.6.3 Horizontal Bar Charts

Simply swap the x and y parameters to make bars horizontal (useful for long category names):

```
make the bars horizontal simply by switching the axes
sns.barplot(x='tip', y='day', hue='sex', data=tips_df);
```



Horizontal bars are easier to read when you have many categories or long labels.

### 11.6.7 Step 7: Box Plots - Distribution Summary

Box plots provide a statistical summary of distributions and are excellent for comparing groups and identifying outliers.

#### 11.6.7.1 Understanding Box Plots

Box plots display five key statistics that describe a distribution:

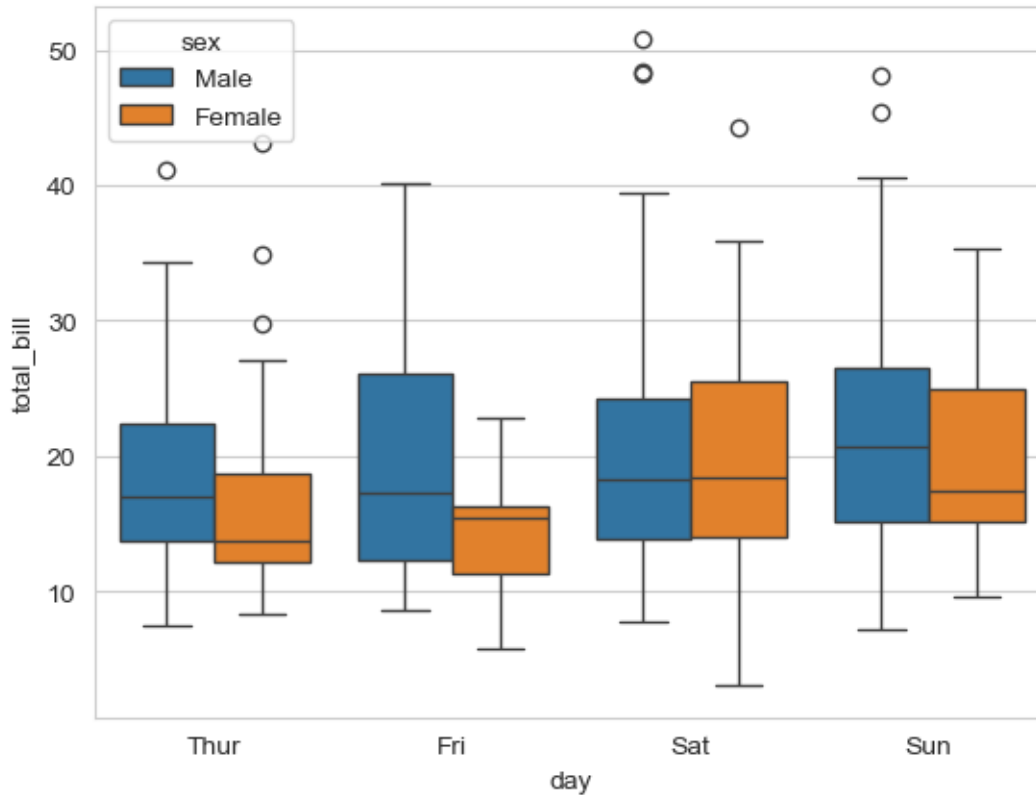
<IPython.core.display.Image object>

**Box Plot Elements:** - **Box** = 25th to 75th percentile (middle 50% of data - the “interquartile range”) - **Line inside box** = Median (50th percentile) - **Whiskers** = Extend to show the data range (typically  $1.5 \times \text{IQR}$ ) - **Individual points** = Outliers (unusually high or low values)

### 11.6.7.2 Creating a Box Plot

Compare total bills across days, separated by gender:

```
sns.boxplot(data=tips_df, y='total_bill', x='day', hue='sex');
```



What can we observe from this box plot?

**Key Insights from the Box Plot:**

**1. Weekend vs Weekday Patterns:**

- Saturday and Sunday show higher median total bills (thicker middle line)
- Thursday has the lowest median bills

**2. Gender Differences:**

- Males generally have higher median total bills than females across all days
- The difference is most pronounced on weekends

**3. Variability:**

- Saturday shows the highest variability (tallest box and longest whiskers)
- Thursday shows the most consistent bills (shortest box)

#### 4. Outliers:

- Several outlier points visible on most days (individual dots above whiskers)
- These represent unusually large bills
- Outliers are more common on weekends

#### 5. Distribution Shape:

- Most distributions are slightly right-skewed (median closer to bottom of box)
- This suggests occasional very high bills pull the mean up

## 11.6.8 Step 8: Matrix Visualizations - Heatmaps

Heatmaps are excellent for visualizing 2D data matrices, especially for showing patterns in tabular data.

### 11.6.8.1 Understanding Heatmaps

Heatmaps represent 2-dimensional data (like a matrix or table) using colors. Darker/brighter colors indicate higher or lower values.

**Use Case:** Let's visualize airline passenger data to see patterns over time.

**Dataset Structure:** Rows represent months, columns represent years, values show passenger counts (in thousands).

**Note:** The `pivot()` function restructures data for heatmap visualization. You'll learn more about pivoting in later chapters.

### 11.6.8.2 Creating a Basic Heatmap

```
flights_df = sns.load_dataset("flights").pivot(index="month", columns="year", values="passengers")
flights_df
you will learn pivot in the later chapters
```

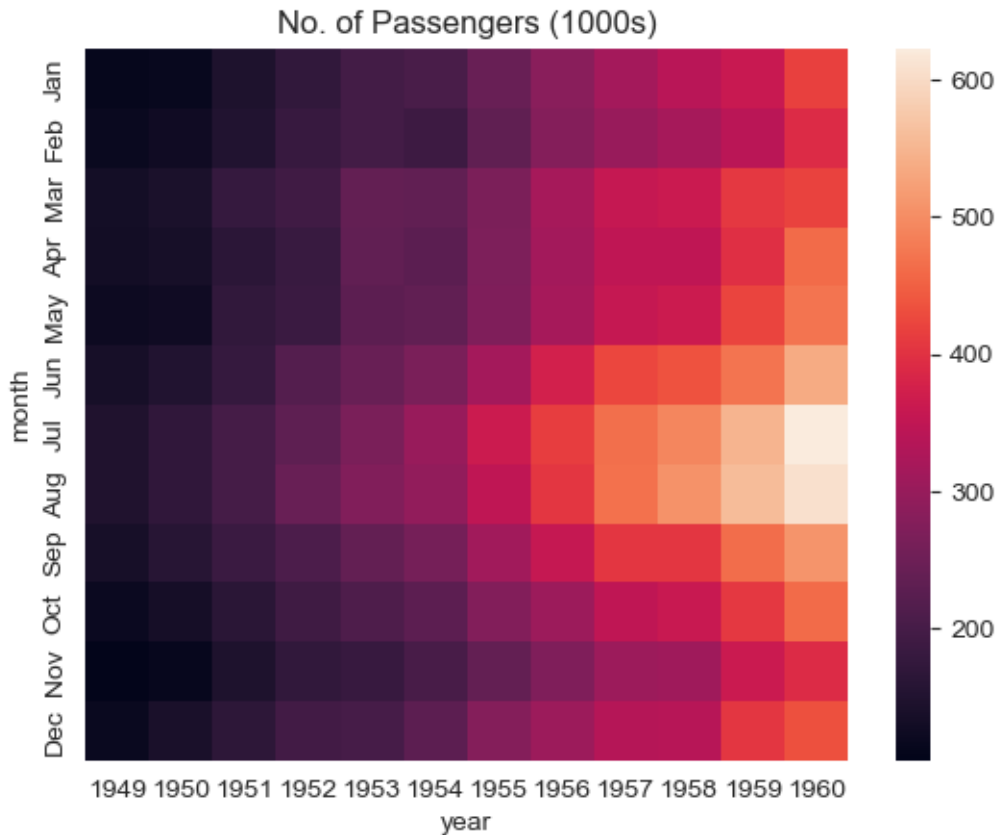
|       | 1949 | 1950 | 1951 | 1952 | 1953 | 1954 | 1955 | 1956 | 1957 | 1958 | 1959 | 1960 |
|-------|------|------|------|------|------|------|------|------|------|------|------|------|
| month |      |      |      |      |      |      |      |      |      |      |      |      |
| Jan   | 112  | 115  | 145  | 171  | 196  | 204  | 242  | 284  | 315  | 340  | 360  | 417  |



| year<br>month | 1949 | 1950 | 1951 | 1952 | 1953 | 1954 | 1955 | 1956 | 1957 | 1958 | 1959 | 1960 |
|---------------|------|------|------|------|------|------|------|------|------|------|------|------|
| Feb           | 118  | 126  | 150  | 180  | 196  | 188  | 233  | 277  | 301  | 318  | 342  | 391  |
| Mar           | 132  | 141  | 178  | 193  | 236  | 235  | 267  | 317  | 356  | 362  | 406  | 419  |
| Apr           | 129  | 135  | 163  | 181  | 235  | 227  | 269  | 313  | 348  | 348  | 396  | 461  |
| May           | 121  | 125  | 172  | 183  | 229  | 234  | 270  | 318  | 355  | 363  | 420  | 472  |
| Jun           | 135  | 149  | 178  | 218  | 243  | 264  | 315  | 374  | 422  | 435  | 472  | 535  |
| Jul           | 148  | 170  | 199  | 230  | 264  | 302  | 364  | 413  | 465  | 491  | 548  | 622  |
| Aug           | 148  | 170  | 199  | 242  | 272  | 293  | 347  | 405  | 467  | 505  | 559  | 606  |
| Sep           | 136  | 158  | 184  | 209  | 237  | 259  | 312  | 355  | 404  | 404  | 463  | 508  |
| Oct           | 119  | 133  | 162  | 191  | 211  | 229  | 274  | 306  | 347  | 359  | 407  | 461  |
| Nov           | 104  | 114  | 146  | 172  | 180  | 203  | 237  | 271  | 305  | 310  | 362  | 390  |
| Dec           | 118  | 140  | 166  | 194  | 201  | 229  | 278  | 306  | 336  | 337  | 405  | 432  |

`flights_df` is a matrix with one row for each month and one column for each year. The values show the number of passengers (in thousands) that visited the airport in a specific month of a year. We can use the `sns.heatmap` function to visualize the footfall at the airport.

```
plt.title("No. of Passengers (1000s)")
sns.heatmap(flights_df);
```



### Reading the Heatmap:

Brighter colors indicate higher passenger counts. From this visualization we can see:

### Key Patterns Observed:

#### 1. Seasonal Pattern (Vertical):

- Footfall is highest in July & August (summer months) - brightest colors
- Lowest in winter months (November-February) - darker colors

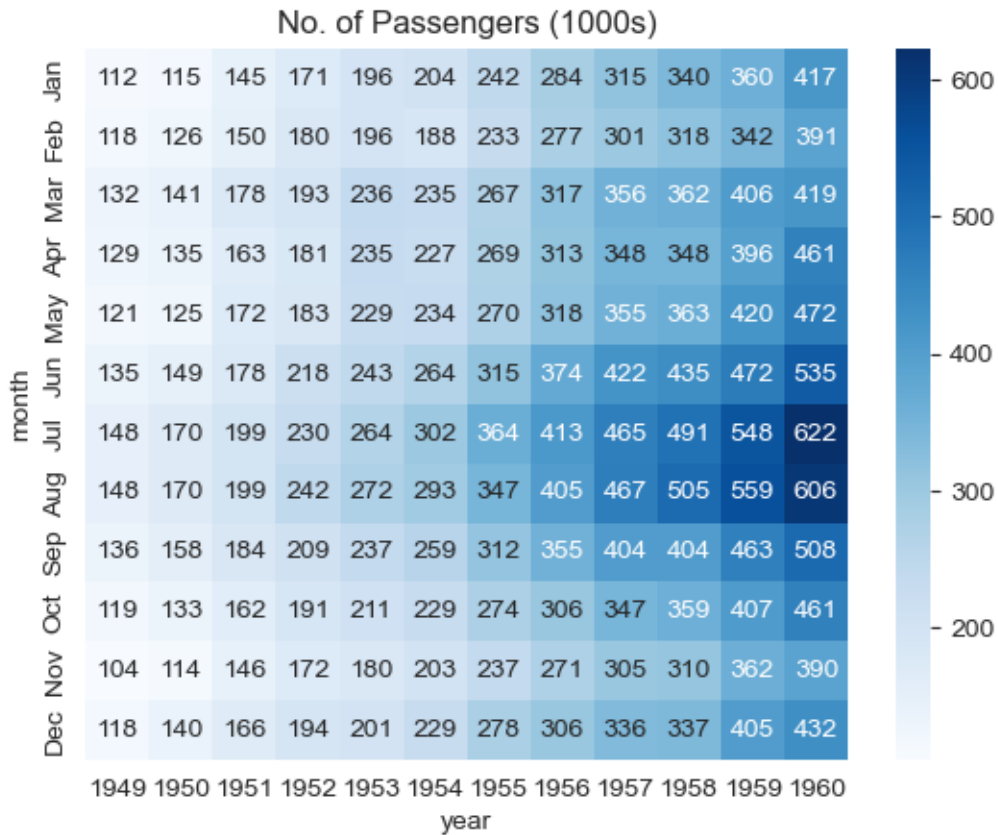
#### 2. Growth Trend (Horizontal):

- Passenger numbers increase year over year
- Each year shows generally brighter colors than the previous

### 11.6.8.3 Enhanced Heatmap with Annotations

Add actual numbers and customize colors for clarity:

```
fmt = "d" decimal integer. output are the number in base 10
plt.title("No. of Passengers (1000s)")
sns.heatmap(flights_df, fmt="d", annot=True, cmap='Blues');
```



#### Enhancements:

- `annot=True` displays actual values in each cell
- `fmt="d"` formats numbers as integers (no decimals)
- `cmap='Blues'` uses a blue color scheme (darker = more passengers)

**Much clearer!** Now we can see exact passenger counts while still benefiting from the color visualization.

### 11.6.9 Step 9: Correlation Matrices

One of the most powerful uses of heatmaps is visualizing correlation matrices.

### 11.6.9.1 What is a Correlation Matrix?

A **correlation matrix** is a special heatmap where:

- Values represent correlation coefficients between pairs of variables
- Range from -1 (perfect negative correlation) to +1 (perfect positive correlation)
- Shows strength and direction of linear relationships

**Visual Guide:**

- **Dark red/positive** = Variables increase together
- **Dark blue/negative** = One increases as other decreases
- **Light colors near 0** = Little to no linear relationship

**Important Note:** In this course, “correlation” always refers to **Pearson’s correlation coefficient**, which measures **linear** association between variables.

**Critical Caveat:** Correlation does NOT imply causation! A strong correlation means variables move together, but doesn’t tell us if one causes the other.

### 11.6.9.2 Computing Correlations with Pandas

Pandas provides built-in functions for correlation analysis:

**Pandas Correlation Functions:**

- **.corr()** - Computes pairwise correlation between all numeric columns of a DataFrame
- **.corrwith()** - Computes correlation of DataFrame columns with another DataFrame or Series

**Let’s load a survey dataset to explore correlations:**

```
#Pairwise correlation amongst all columns
survey_data = pd.read_csv('./Datasets/survey_data_clean.csv')

survey_data.head()
```

|   | Timestamp                    | fav_alcohol             | parties_per_month | smoke | weed         |
|---|------------------------------|-------------------------|-------------------|-------|--------------|
| 0 | 2022/09/13 1:43:34 pm GMT-5  | I don't drink           | 1.0               | No    | Occasionally |
| 1 | 2022/09/13 5:28:17 pm GMT-5  | Hard liquor/Mixed drink | 3.0               | No    | Occasionally |
| 2 | 2022/09/13 7:56:38 pm GMT-5  | Hard liquor/Mixed drink | 3.0               | No    | Yes          |
| 3 | 2022/09/13 10:34:37 pm GMT-5 | Hard liquor/Mixed drink | 12.0              | No    | No           |

|   | Timestamp                   | fav_alcohol   | parties_per_month | smoke | weed |
|---|-----------------------------|---------------|-------------------|-------|------|
| 4 | 2022/09/14 4:46:19 pm GMT-5 | I don't drink | 1.0               | No    | No   |

**Compute the pairwise correlation matrix:**

```
#Pairwise correlation amongst all columns
survey_data.select_dtypes(include='number').corr()
```

|                               | parties_per_month | love_first_sight | num_insta_followers | expected_marriage_age |
|-------------------------------|-------------------|------------------|---------------------|-----------------------|
| parties_per_month             | 1.000000          | 0.096129         | 0.239705            | -0.064079             |
| love_first_sight              | 0.096129          | 1.000000         | -0.024010           | -0.084406             |
| num_insta_followers           | 0.239705          | -0.024010        | 1.000000            | -0.130157             |
| expected_marriage_age         | -0.064079         | -0.084406        | -0.130157           | 1.000000              |
| expected_starting_salary      | 0.114881          | 0.080138         | 0.127226            | -0.014881             |
| minutes_ex_per_week           | 0.195561          | 0.099244         | 0.099341            | -0.088073             |
| sleep_hours_per_day           | -0.052542         | -0.025378        | -0.042421           | 0.182009              |
| farthest_distance_travelled   | -0.017081         | -0.075539        | 0.011308            | -0.024038             |
| fav_number                    | -0.050139         | 0.105095         | -0.124763           | -0.008924             |
| internet_hours_per_day        | 0.087390          | -0.007652        | -0.028427           | -0.029772             |
| only_child                    | -0.142519         | 0.124345         | -0.152184           | -0.043141             |
| num_majors_minors             | -0.073127         | 0.108730         | 0.050431            | -0.055280             |
| high_school_GPA               | 0.295646          | 0.069288         | 0.147402            | 0.017052              |
| NU_GPA                        | -0.080548         | -0.114041        | 0.004702            | 0.011925              |
| age                           | -0.032771         | 0.142384         | -0.230698           | 0.060416              |
| height                        | -0.005405         | 0.216072         | 0.009318            | 0.044577              |
| height_father                 | 0.126741          | 0.029419         | 0.179684            | 0.026949              |
| height_mother                 | 0.079121          | 0.082684         | 0.129716            | 0.075316              |
| procrastinator                | -0.056871         | 0.033951         | -0.089871           | -0.020012             |
| num_clubs                     | -0.010514         | 0.083342         | 0.265958            | -0.137069             |
| student_athlete               | 0.290830          | 0.014595         | 0.044807            | -0.036122             |
| AP_stats                      | -0.013222         | -0.062992        | 0.005947            | 0.010447              |
| used_python_before            | -0.040033         | -0.034692        | -0.016201           | 0.052727              |
| childhood_in_US               | 0.081905          | -0.118260        | 0.072622            | 0.053759              |
| cant_change_math_ability      | -0.052912         | 0.005254         | -0.150658           | -0.072163             |
| can_change_math_ability       | 0.055575          | 0.020758         | 0.130774            | 0.087633              |
| math_is_genetic               | -0.013374         | -0.003710        | -0.018411           | -0.086898             |
| much_effort_is_lack_of_talent | -0.029838         | 0.013376         | -0.165899           | 0.052813              |

**The matrix is hard to read!** Let's find which features correlate most with NU\_GPA:

```
survey_data.select_dtypes(include='number').corrwith(survey_data.NU_GPA).sort_values(ascending=True)
```

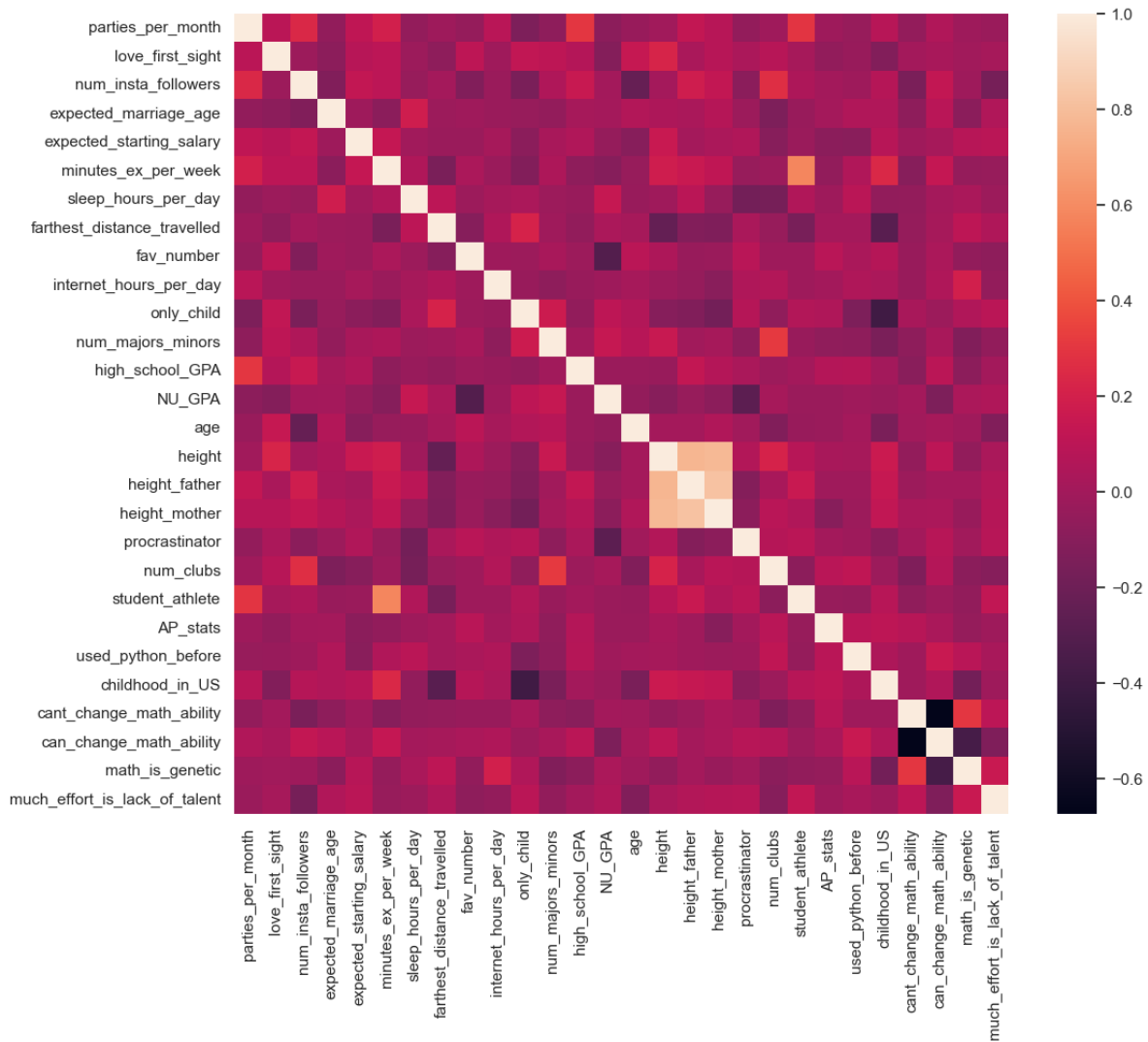
|                               |           |
|-------------------------------|-----------|
| NU_GPA                        | 1.000000  |
| sleep_hours_per_day           | 0.143997  |
| num_majors_minors             | 0.141988  |
| only_child                    | 0.106440  |
| much_effort_is_lack_of_talent | 0.047840  |
| farthest_distance_travelled   | 0.038238  |
| math_is_genetic               | 0.036731  |
| num_clubs                     | 0.016724  |
| expected_marriage_age         | 0.011925  |
| num_insta_followers           | 0.004702  |
| cant_change_math_ability      | 0.002094  |
| used_python_before            | -0.008536 |
| internet_hours_per_day        | -0.014531 |
| AP_stats                      | -0.026544 |
| student_athlete               | -0.027378 |
| childhood_in_US               | -0.028968 |
| high_school_GPA               | -0.030883 |
| height_father                 | -0.040120 |
| expected_starting_salary      | -0.048069 |
| age                           | -0.052039 |
| height_mother                 | -0.079276 |
| parties_per_month             | -0.080548 |
| height                        | -0.099082 |
| minutes_ex_per_week           | -0.108177 |
| love_first_sight              | -0.114041 |
| can_change_math_ability       | -0.137330 |
| procrastinator                | -0.269552 |
| fav_number                    | -0.307656 |
| dtype: float64                |           |

**Better!** Now we can see correlations with NU\_GPA sorted from strongest to weakest.

### 11.6.9.3 Visualizing the Correlation Matrix

Now let's create a heatmap to see all correlations at once:

```
sns.set(rc={'figure.figsize':(12,10)})
sns.heatmap(survey_data.select_dtypes(include='number').corr());
```



### Key Findings from the Correlation Heatmap:

#### 1. Strong Positive Correlation:

- `student_athlete` is strongly positively correlated with `minutes_ex_per_week`
- This makes sense: student athletes exercise more

#### 2. Strong Negative Correlation:

- `procrastinator` is strongly negatively correlated with `NU_GPA`
- Students who procrastinate tend to have lower GPAs

#### 3. Diagonal:

- All 1.0 values (variables perfectly correlate with themselves)

#### 4. Symmetry:

- Matrix is symmetric:  $\text{correlation}(A, B) = \text{correlation}(B, A)$

### 11.6.10 Interpreting Correlation Coefficients

Understanding what correlation values mean:

| Coefficient Range | Interpretation       | Relationship Strength               |
|-------------------|----------------------|-------------------------------------|
| 0.9 to 1.0        | Very strong positive | Nearly perfect linear relationship  |
| 0.7 to 0.9        | Strong positive      | Clear linear pattern                |
| 0.5 to 0.7        | Moderate positive    | Noticeable but imperfect pattern    |
| 0.3 to 0.5        | Weak positive        | Slight tendency                     |
| -0.3 to 0.3       | Little to none       | No linear relationship              |
| -0.5 to -0.3      | Weak negative        | Slight inverse tendency             |
| -0.7 to -0.5      | Moderate negative    | Noticeable inverse pattern          |
| -0.9 to -0.7      | Strong negative      | Clear inverse linear pattern        |
| -1.0 to -0.9      | Very strong negative | Nearly perfect inverse relationship |

#### Critical Caveats:

##### 1. Correlation Causation

- Strong correlation doesn't mean one variable causes changes in the other
- Example: Ice cream sales and drowning deaths correlate (both happen in summer), but ice cream doesn't cause drowning!

##### 2. Linear Only

- Pearson correlation only captures linear relationships
- Variables might have strong non-linear relationships with correlation near 0

##### 3. Outliers Matter

- A few extreme values can heavily influence correlation coefficients
- Always visualize your data, don't just look at numbers

##### 4. Hidden Variables

- Third variables (confounders) might explain apparent correlations
- Consider lurking variables before drawing conclusions



### 11.6.11 Summary: Seaborn Best Practices

Now that you've learned Seaborn, here are essential best practices:

#### 11.6.11.1 When to Use Seaborn

##### Best For:

- Statistical visualizations (distributions, relationships, comparisons)
- Quick, beautiful plots with minimal code
- Exploring relationships in DataFrames
- Creating publication-ready figures with default settings
- Plots with categorical and numerical data together

##### Not Ideal For:

- Highly customized, non-standard plot types (use Matplotlib)
- Interactive visualizations (use Plotly or Bokeh)
- 3D plots (use Matplotlib's mplot3d or Plotly)
- Very simple exploratory plots (Pandas might be faster)

#### 11.6.11.2 Essential Seaborn Workflow

##### 1. Set Aesthetics First:

```
sns.set_style("whitegrid") # Professional appearance
sns.set_context("notebook") # Appropriate sizing
sns.set_palette("colorblind") # Accessible colors
```

##### 2. Use the data Parameter:

```
Recommended: Clear and readable
sns.scatterplot(data=df, x='col1', y='col2', hue='col3')

Avoid: Harder to read and modify
sns.scatterplot(x=df['col1'], y=df['col2'], hue=df['col3'])
```

##### 3. Leverage hue for Multi-Dimensional Plots:

- Adds a third dimension via color
- Works across most plot types
- Automatically generates legends

#### 4. Combine with Matplotlib for Fine-Tuning:

```
sns.boxplot(data=df, x='category', y='value')
plt.title('Custom Title') # Matplotlib customization
plt.ylabel('Custom Y Label')
plt.xticks(rotation=45)
```

#### 5. Choose the Right Plot Type:

| Goal                         | Plot Type           | Seaborn Function           |
|------------------------------|---------------------|----------------------------|
| Single variable distribution | Histogram           | <code>histplot()</code>    |
|                              | Smooth distribution | <code>kdeplot()</code>     |
| Compare categories           | Bar chart           | <code>barplot()</code>     |
|                              | Box plot            | <code>boxplot()</code>     |
| Two numeric variables        | Scatter             | <code>scatterplot()</code> |
| Correlations                 | Heatmap             | <code>heatmap()</code>     |
| All pairwise relationships   | Pair plot           | <code>pairplot()</code>    |

#### 11.6.11.3 Color Palette Guide

##### For Categorical Data (no order):

- "deep" - default, good contrast
- "colorblind" - accessible (highly recommended!)
- "Set2", "Set3" - soft, professional

##### For Sequential Data (low to high):

- "Blues", "Greens", "Reds" - single hue
- "viridis", "plasma", "cividis" - perceptually uniform

##### For Diverging Data (meaningful midpoint):

- "coolwarm" - blue to red through white
- "RdBu" - red to blue
- "vlag" - light to dark to light

### 11.6.12 Practice Exercises

Now apply what you've learned!

#### Exercise 1: Distribution Analysis

```
Using the iris dataset:
1. Create a histogram of petal_length with KDE
2. Add hue by species
3. Add a title and customize figure size
```

#### Exercise 2: Categorical Comparison

```
Using the tips dataset:
1. Create a box plot comparing total_bill across different times (Lunch vs Dinner)
2. Add hue by sex
3. Make it horizontal
```

#### Exercise 3: Correlation Analysis

```
Using the iris dataset:
1. Compute the correlation matrix for all numeric columns
2. Create a heatmap with annotations
3. Use the 'coolwarm' color palette
4. What are the two most correlated features?
```

#### Exercise 4: Multi-Dimensional Scatter

```
Using the tips dataset:
1. Create a scatter plot of total_bill vs tip
2. Use hue for time (Lunch/Dinner)
3. Use size for the party size
4. Add appropriate labels and title
```

**Hint:** Refer back to the examples in this section and the [Seaborn documentation](#) for parameter details!

## 11.7 Chapter Summary

Congratulations! You've completed a comprehensive journey through data visualization in Python. Let's recap what you've accomplished.

## 11.7.1 Visualization Fundamentals

### Data Understanding:

- Distinguishing numeric vs. categorical data
- Univariate, bivariate, and multivariate analysis
- Matching data types to visualization types

## 11.7.2 Complete Plot Type Reference

Here's a master reference of all plot types you've learned:

| Plot Type           | Purpose                            | Pandas                               | Matplotlib                 | Seaborn                        |
|---------------------|------------------------------------|--------------------------------------|----------------------------|--------------------------------|
| <b>Line Plot</b>    | Trends over time/continuous data   | <code>df.plot()</code>               | <code>plt.plot()</code>    | <code>sns.lineplot()</code>    |
| <b>Scatter Plot</b> | Relationship between two variables | <code>df.plot(kind='scatter')</code> | <code>plt.scatter()</code> | <code>sns.scatterplot()</code> |
| <b>Histogram</b>    | Distribution of single variable    | <code>df.plot(kind='hist')</code>    | <code>plt.hist()</code>    | <code>sns.histplot()</code>    |
| <b>Bar Chart</b>    | Compare categories                 | <code>df.plot(kind='bar')</code>     | <code>plt.bar()</code>     | <code>sns.barplot()</code>     |
| <b>Box Plot</b>     | Distribution summary + outliers    | <code>df.plot(kind='box')</code>     | <code>plt.boxplot()</code> | <code>sns.boxplot()</code>     |
| <b>Pie Chart</b>    | Part-to-whole relationships        | <code>df.plot(kind='pie')</code>     | <code>plt.pie()</code>     | N/A                            |
| <b>Heatmap</b>      | 2D data matrix/correlations        | N/A                                  | <code>plt.imshow()</code>  | <code>sns.heatmap()</code>     |
| <b>KDE Plot</b>     | Smooth distribution                | <code>df.plot(kind='kde')</code>     | N/A                        | <code>sns.kdeplot()</code>     |

### 11.7.2.1 The Three-Library Ecosystem

You've mastered Python's three-tier approach to visualization:

Pandas `.plot()` ← Quick & Easy

|                   |                           |
|-------------------|---------------------------|
| Seaborn           | ← Beautiful & Statistical |
| Matplotlib pyplot | ← Complete Control        |

### 11.7.3 When to Use Each Library

Is this exploratory analysis?

Yes → Use Pandas `.plot()` for speed

No

Need statistical visualization?

Yes → Use Seaborn

No → Continue below

Need custom/complex plot?

Yes → Use Matplotlib

No → Use Seaborn (easier syntax)

### 11.7.4 Essential Best Practices

#### 11.7.4.1 Universal Guidelines

##### 1. Always Label Your Axes

- Include units: “Temperature (°C)” not just “Temperature”
- Make labels descriptive and self-explanatory

##### 2. Add Informative Titles

- Describe what the plot shows
- Answer: “What am I looking at?”

##### 3. Choose Appropriate Colors

- Use colorblind-friendly palettes (“colorblind” in Seaborn)
- Avoid red-green combinations
- Ensure sufficient contrast

##### 4. Control Figure Size

```
Pandas
df.plot(figsize=(12, 6))

Matplotlib
plt.figure(figsize=(12, 6))

Seaborn (use Matplotlib)
plt.figure(figsize=(12, 6))
sns.scatterplot(data=df, x='x', y='y')
```

## 5. Save High-Quality Figures

```
plt.savefig('figure.png', dpi=300, bbox_inches='tight')
```

### 11.7.4.2 Library-Specific Tips

#### Pandas:

- Set time column as index for time series plots
- Use `rot=45` to rotate axis labels
- Chain `.plot()` with other DataFrame operations

#### Matplotlib:

- Use semicolons (;) in Jupyter to suppress unwanted output
- Call `plt.figure()` before creating a new plot
- Use `fmt` shorthand for quick styling: 'o-r' = circles, solid line, red

#### Seaborn:

- Always set style first: `sns.set_style("whitegrid")`
- Use `data` parameter with column names (not Series)
- Leverage `hue` for multi-dimensional visualization
- Combine with Matplotlib for fine-tuning

### 11.7.5 Pro Tip: Combine Libraries!

The most powerful approach is mixing libraries in a single plot:

```
Create beautiful plot with Seaborn
sns.scatterplot(data=df, x='x', y='y', hue='category')

Fine-tune with Matplotlib
plt.title('Custom Title', fontsize=16, fontweight='bold')
plt.axhline(y=0, color='red', linestyle='--', alpha=0.5)
plt.tight_layout()

Works seamlessly!
```

This gives you:

- Seaborn's easy syntax and beautiful defaults
- Matplotlib's precise customization
- Best of both worlds!

## 11.8 Independent Study

### 11.8.1 Practice exercise 1

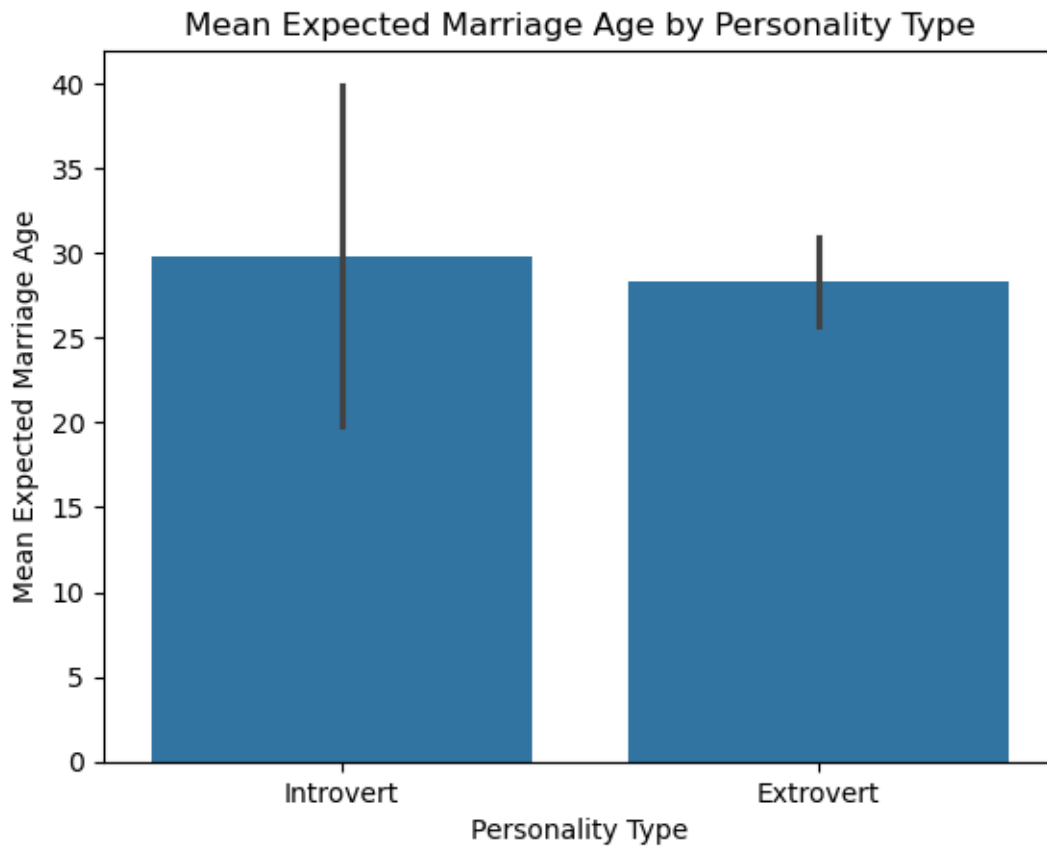
Read *survey\_data\_clean.csv*

|   | Timestamp                    | fav_alcohol             | parties_per_month | smoke | weed         |
|---|------------------------------|-------------------------|-------------------|-------|--------------|
| 0 | 2022/09/13 1:43:34 pm GMT-5  | I don't drink           | 1.0               | No    | Occasionally |
| 1 | 2022/09/13 5:28:17 pm GMT-5  | Hard liquor/Mixed drink | 3.0               | No    | Occasionally |
| 2 | 2022/09/13 7:56:38 pm GMT-5  | Hard liquor/Mixed drink | 3.0               | No    | Yes          |
| 3 | 2022/09/13 10:34:37 pm GMT-5 | Hard liquor/Mixed drink | 12.0              | No    | No           |
| 4 | 2022/09/14 4:46:19 pm GMT-5  | I don't drink           | 1.0               | No    | No           |

How does the expected marriage age of the people of STAT303-1 depend on their characteristics? We'll use visualizations to answer this question.

#### 11.8.1.1

Make a visualization that compares the mean `expected_marriage_age` of introverts and extroverts (use the variable *introvert\_extrovert*). What insights do you obtain?

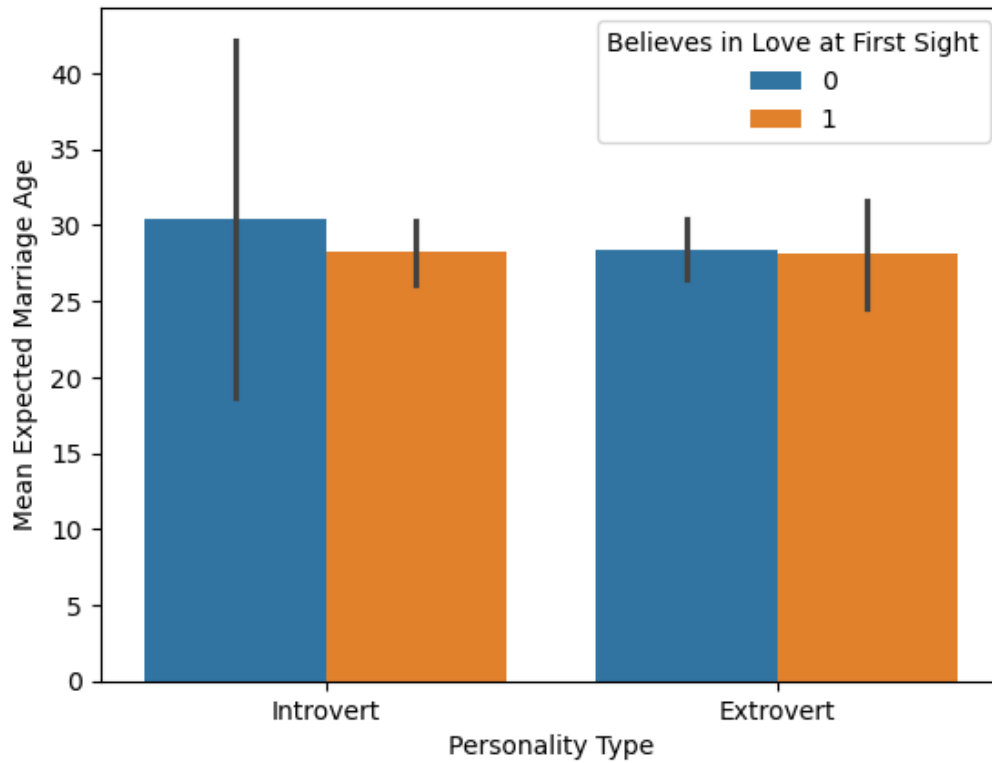


#### 11.8.1.2

Does the mean `expected_marriage_age` of introverts and extroverts depend on whether they believe in love in first sight (*variable name: `love_first_sight`*)? Update the previous visualization to answer the question.

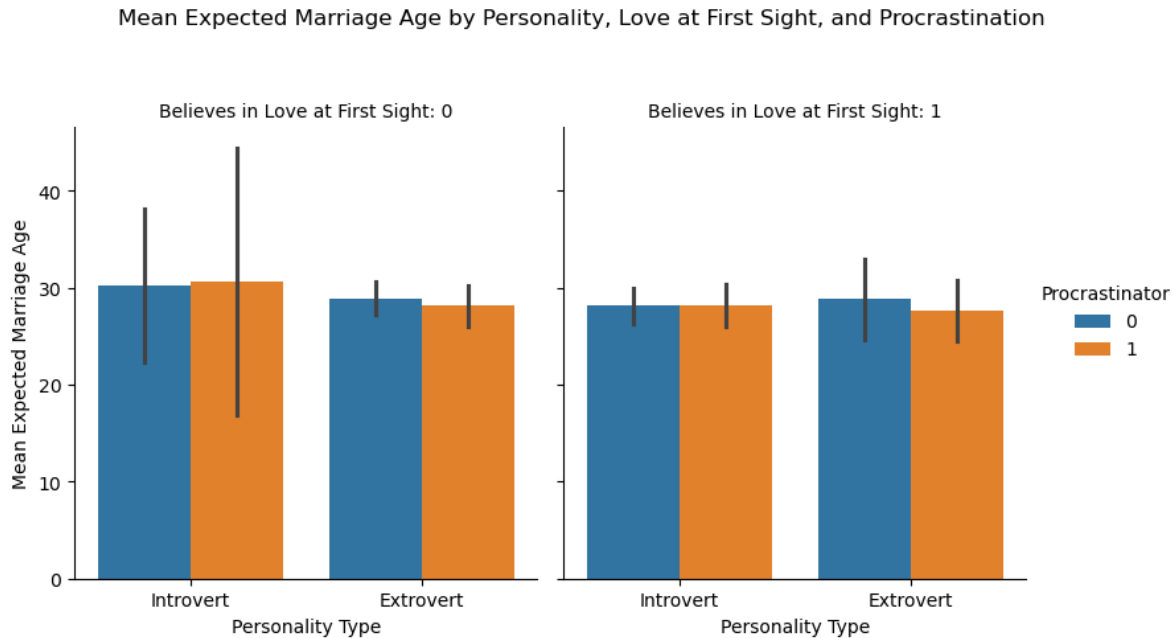


Mean Expected Marriage Age by Personality and Belief in Love at First Sight



### 11.8.1.3

In addition to `love_first_sight`, does the mean `expected_marriage_age` of introverts and extroverts depend on whether they are a procrastinator (*variable name: `procrastinator`*)? Update the previous visualization to answer the question.



#### 11.8.1.4

Is there any critical information missing in the above visualizations that, if revealed, may cast doubts on the patterns observed in them?

### 11.8.2 Practice exercise 2

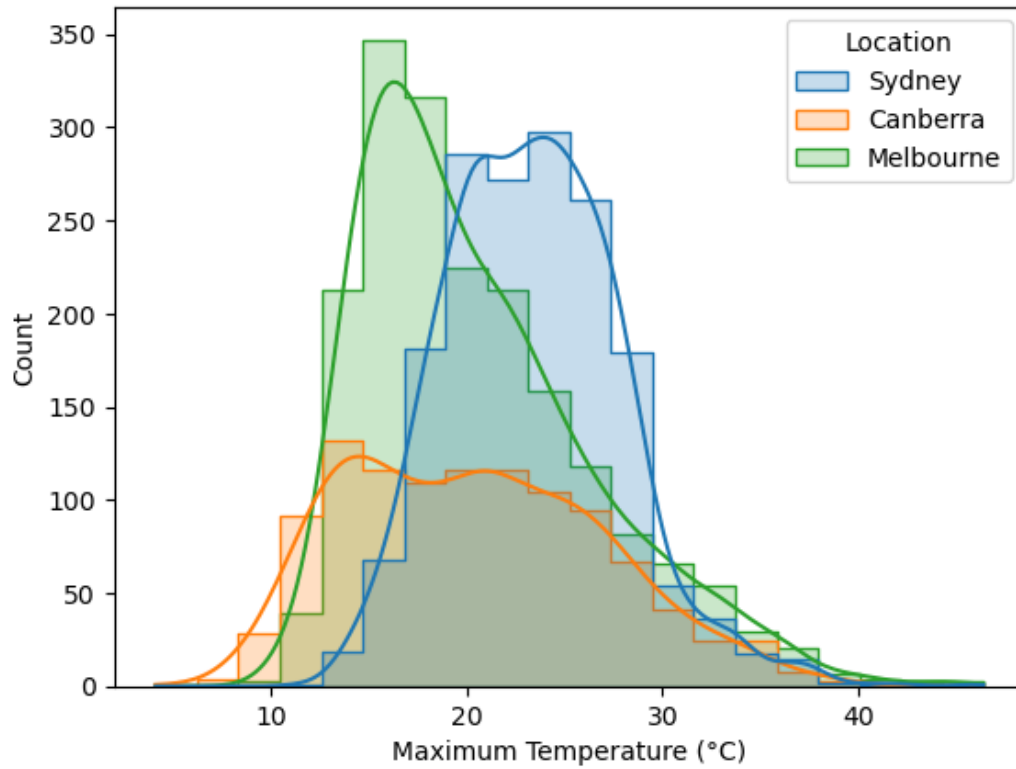
Read *Australia\_weather.csv*,

|   | Date       | Location | MinTemp | MaxTemp | Rainfall | Evaporation | Sunshine | WindGustDir | Wind |
|---|------------|----------|---------|---------|----------|-------------|----------|-------------|------|
| 0 | 10/20/2010 | Sydney   | 12.9    | 20.3    | 0.2      | 3.0         | 10.9     | ENE         | 37   |
| 1 | 10/21/2010 | Sydney   | 13.3    | 21.5    | 0.0      | 6.6         | 11.0     | ENE         | 41   |
| 2 | 10/22/2010 | Sydney   | 15.3    | 23.0    | 0.0      | 5.6         | 11.0     | NNE         | 41   |
| 3 | 10/26/2010 | Sydney   | 12.9    | 26.7    | 0.2      | 3.8         | 12.1     | NE          | 33   |
| 4 | 10/27/2010 | Sydney   | 14.8    | 23.8    | 0.0      | 6.8         | 9.6      | SSE         | 54   |

#### 11.8.2.1

Create a histogram showing the distributions of maximum temperature in Sydney, Canberra and Melbourne.

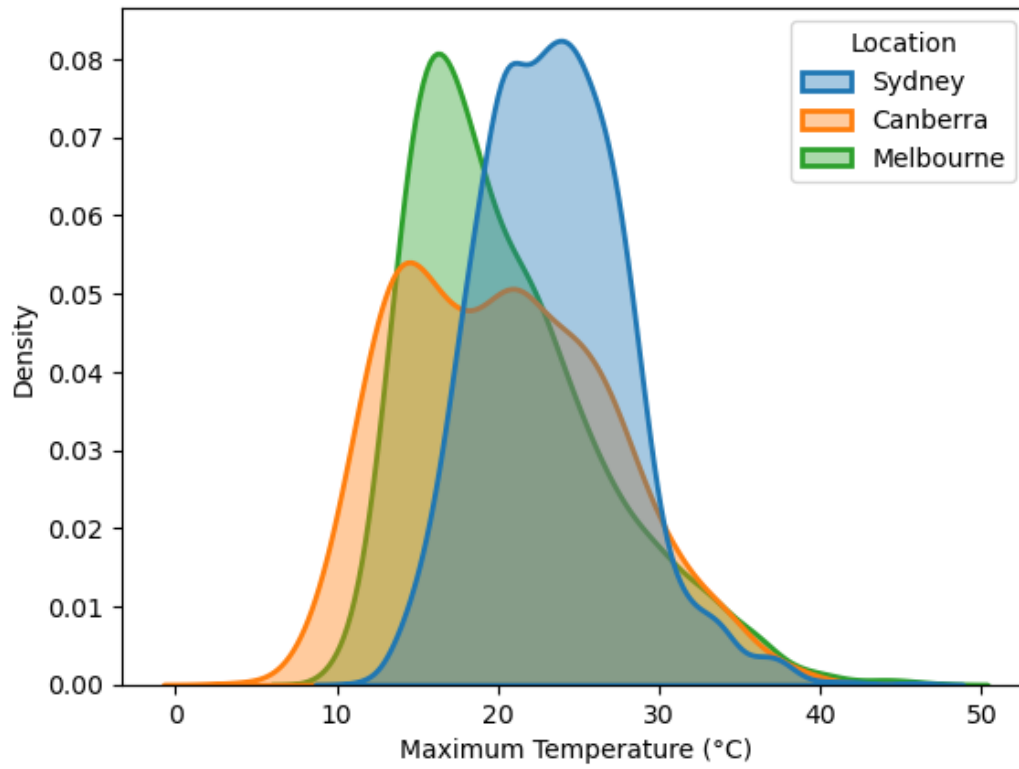
Distribution of Maximum Temperature in Sydney, Canberra, and Melbourne



#### 11.8.2.2

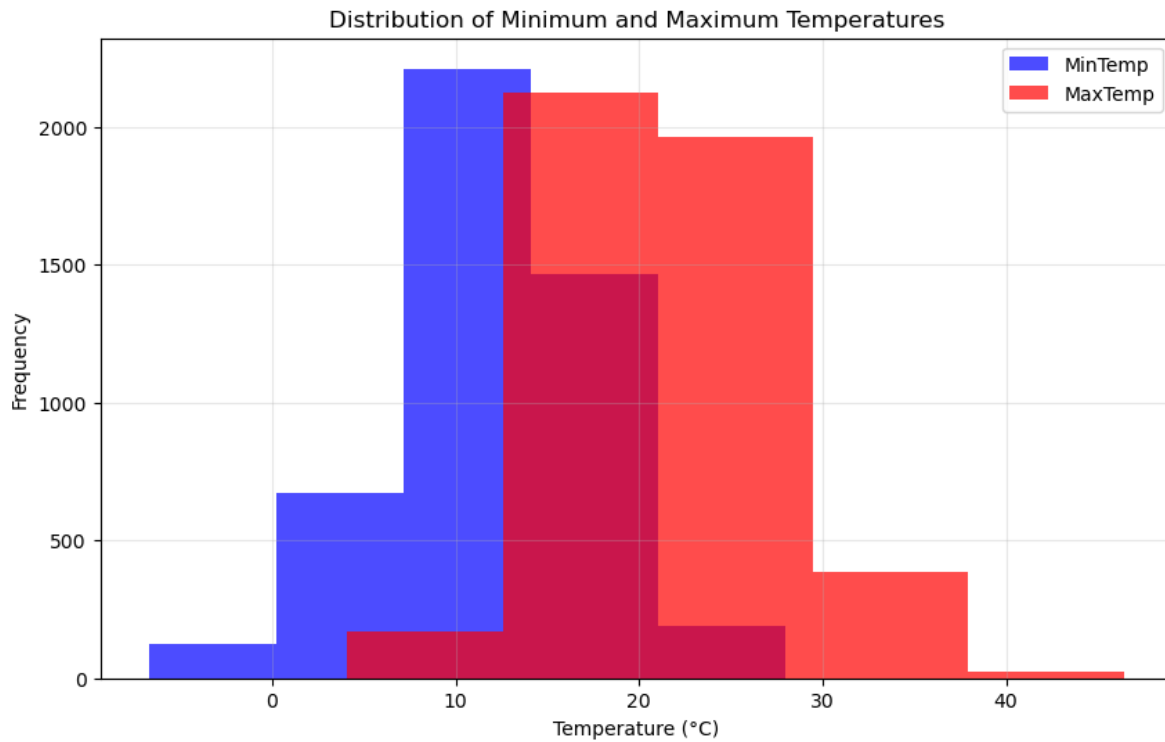
Make a density plot showing the distributions of maximum temperature in Sydney, Canberra and Melbourne.

Density Plot of Maximum Temperature in Sydney, Canberra, and Melbourne



### 11.8.2.3

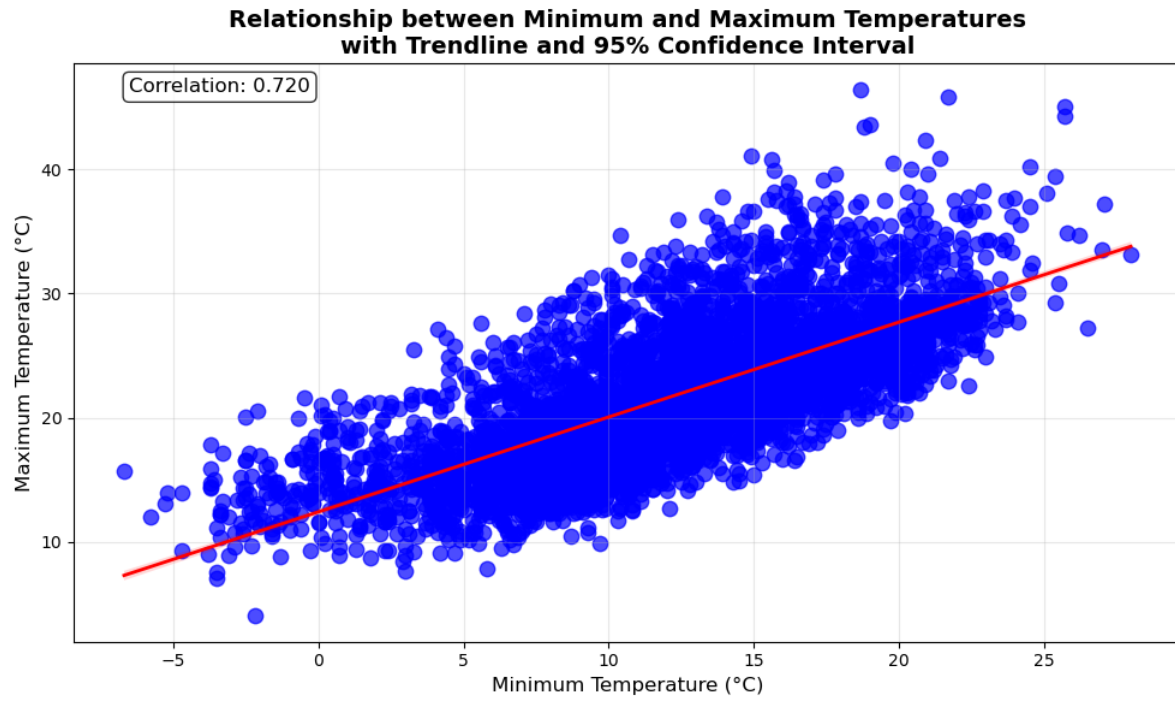
Show the distributions of the maximum and minimum temperatures across all locations in a single plot.



#### 11.8.2.4

Create a scatter plot with a trendline for `MinTemp` and `MaxTemp`, including a confidence interval.

**Hint:** Using Seaborn, the `regplot()` function enables us to overlay a trendline on the scatter plot, complete with a 95% confidence interval for the trendline



# 12 Data Visualization Intermediate

<IPython.core.display.Image object>

## 12.1 Chapter Overview

In the previous chapter, you mastered the fundamentals of creating plots using pandas, matplotlib's pyplot interface, and seaborn. You learned to create individual visualizations like scatter plots, line plots, bar plots, and statistical plots.

This chapter advances your visualization skills by focusing on:

- **Matplotlib's object-oriented interface** for fine-grained control
- **Complex multi-panel layouts** with sophisticated subplots
- **Advanced seaborn techniques** for multi-dimensional data exploration
- **Geospatial visualization** for mapping and location-based data

By the end of this chapter, you'll be able to create publication-quality, complex visualizations that combine multiple data views and handle specialized data types like geographic information.

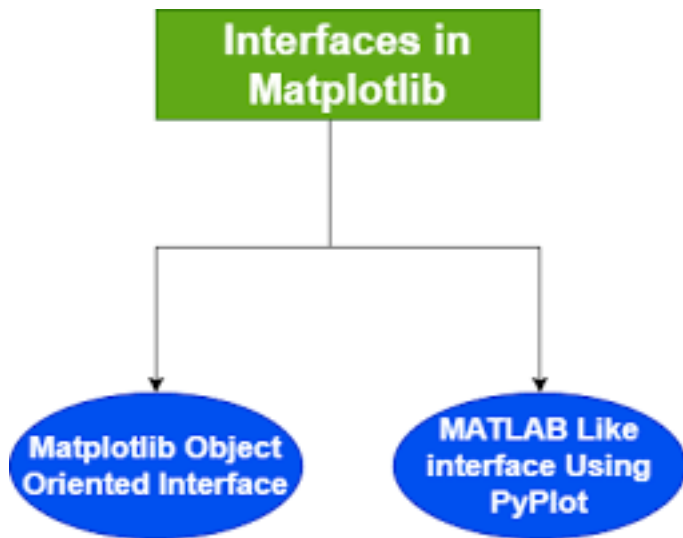
To get started, let's import necessary libraries.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

%matplotlib inline
```

## 12.2 Matplotlib's Two Interfaces

Matplotlib provides **two main interfaces** for creating visualizations: the **pyplot interface** and the **object-oriented (OOP) interface**.



In the previous chapter, we primarily used the **pyplot interface** — a convenient, MATLAB-style approach where functions such as `plt.plot()` and `plt.xlabel()` operate on an implicit “*current*” figure and axes.

The **object-oriented (OOP) interface**, on the other hand, offers **explicit control** over every element of your plot. It’s the preferred approach for building **complex layouts**, **multiple subplots**, or **publication-quality figures** where precision and flexibility matter.

### 12.2.1 Compare Pyplot vs Object-Oriented Interfaces

Let’s create the same plot using both interfaces to see the difference in syntax and approach.

We’ll use a dataset containing GDP and life expectancy data for countries.

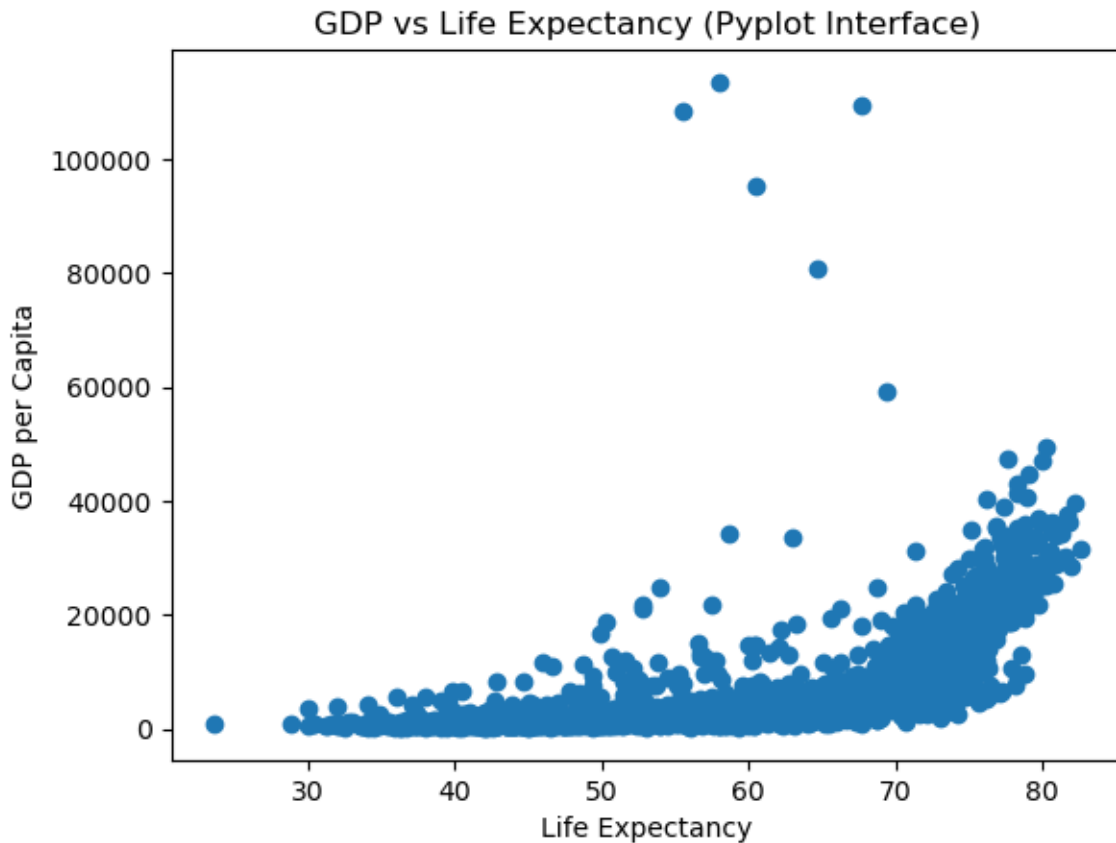
```
Load the dataset
gdp_data = pd.read_csv('datasets/gdp_lifeExpectancy.csv')
gdp_data.head()
```

|   | country     | continent | year | lifeExp | pop      | gdpPercap  |
|---|-------------|-----------|------|---------|----------|------------|
| 0 | Afghanistan | Asia      | 1952 | 28.801  | 8425333  | 779.445314 |
| 1 | Afghanistan | Asia      | 1957 | 30.332  | 9240934  | 820.853030 |
| 2 | Afghanistan | Asia      | 1962 | 31.997  | 10267083 | 853.100710 |
| 3 | Afghanistan | Asia      | 1967 | 34.020  | 11537966 | 836.197138 |
| 4 | Afghanistan | Asia      | 1972 | 36.088  | 13079460 | 739.981106 |



### 12.2.1.1 Pyplot Interface (Implicit)

```
Pyplot interface - operates on the "current" figure/axes
plt.scatter(gdp_data.lifeExp, gdp_data.gdpPercap)
plt.xlabel('Life Expectancy')
plt.ylabel('GDP per Capita')
plt.title('GDP vs Life Expectancy (Pyplot Interface)');
```

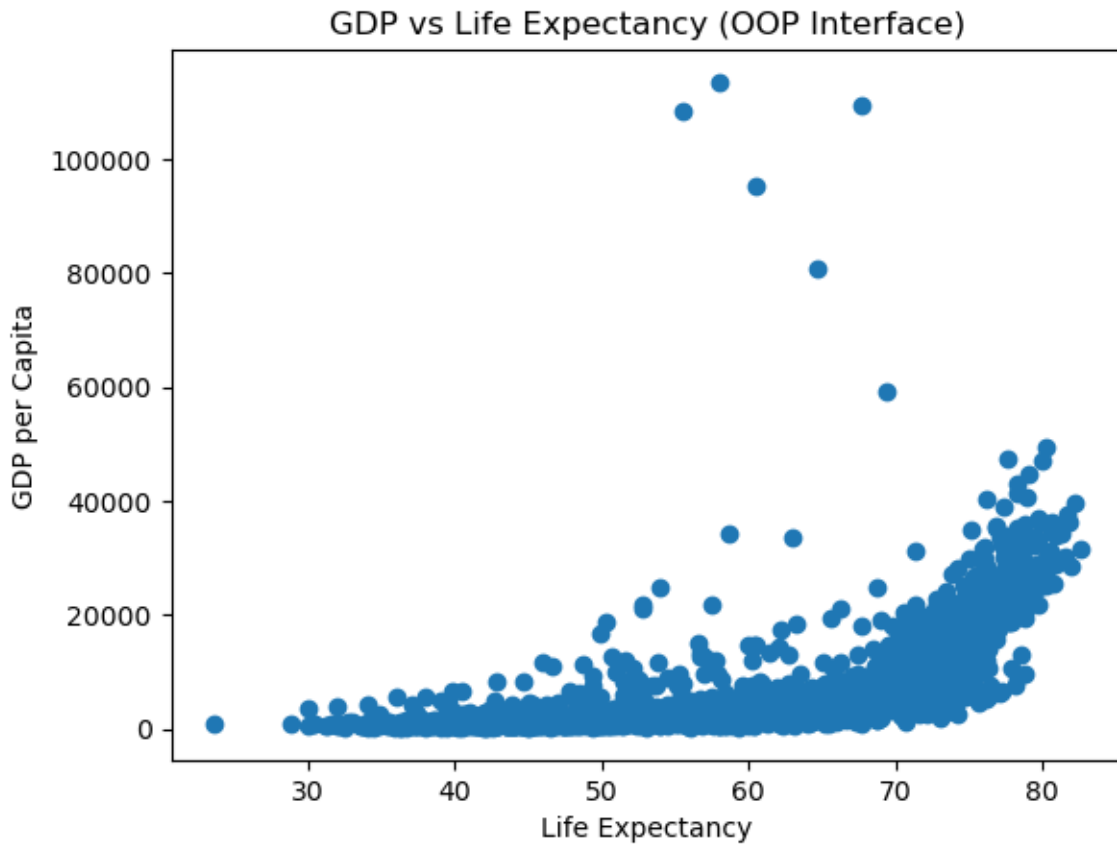


### 12.2.1.2 Object-Oriented Interface (Explicit)

```
Object-oriented interface
fig, ax = plt.subplots() # Create a figure and an axes explicitly

Plot on the specific axes object
```

```
ax.scatter(gdp_data.lifeExp, gdp_data.gdpPercap)
ax.set_xlabel('Life Expectancy')
ax.set_ylabel('GDP per Capita')
ax.set_title('GDP vs Life Expectancy (OOP Interface)');
```



### Key Observations:

Both approaches produce identical plots, but notice the differences:

- **Pyplot Interface:**
  - Simpler syntax: `plt.scatter()`, `plt.xlabel()`
  - Operates on implicit “current” figure
  - Great for quick, simple plots
  - Less control when working with multiple subplots
- **Object-Oriented Interface:**
  - Explicit syntax: `ax.scatter()`, `ax.set_xlabel()`

- Direct control over specific axes object
- Essential for complex layouts
- Can pass axes to pandas/seaborn functions
- More verbose but more flexible

### 12.2.1.3 Pyplot: A Convenience Wrapper

The `pyplot` interface (e.g., `plt.plot()`) is a *convenience layer* built on top of Matplotlib’s **object-oriented (OOP)** interface.

Behind the scenes, it:

- Automatically creates a **Figure** and **Axes** if they don’t already exist
- Keeps track of the “**current**” figure and axes
- Routes plotting commands to that current axes object

This makes `pyplot` extremely convenient for **quick, one-off plots** or **exploratory analysis**, but it can quickly become **confusing or limiting** when working with **multiple figures or subplots**.

### 12.2.1.4 When the OOP Interface Becomes Essential

While `pyplot` is great for simplicity, the **OOP interface** (`fig, ax = plt.subplots()`) is the right choice for:

- Creating **multiple subplots** within a single figure
- Building **complex layouts** or **publication-quality plots**
- Integrating with **pandas** or **seaborn**, which both accept an `ax` parameter — allowing you to specify **exactly where** a plot should appear (something `pyplot` alone cannot do)

## 12.2.2 Understand the Matplotlib Object Hierarchy

To work effectively with matplotlib’s OOP interface, you need to understand how plot components are organized.

### 12.2.2.1 The Hierarchical Structure

Matplotlib plots follow a **hierarchical structure**, with two core components:

- **Figure:** The overall container for one or more plots (the entire window/canvas)
- **Axes:** The individual plot area where data is visualized (what you think of as “a plot”)

Each Axes contains further elements such as:

- **Axis** (x-axis and y-axis) - the number lines with ticks and labels
- **Title, Labels, Ticks** - text and markers
- **Drawable objects** like Lines, Text, and Patches (collectively called **Artists**)

Here’s the hierarchy visualized:

```
Figure (the entire canvas)
 Axes (1 or more plot areas)
 XAxis, YAxis (the number lines)
 Axis Labels
 Tick Marks
 Tick Labels
 Title
 Legend
 Artist objects (Lines, Patches, Text, Collections, etc.)
```

#### Important Terminology:

- **Figure** = The entire window/image (can contain multiple plots)
- **Axes** = A single plot area (confusingly NOT the axis lines!)
- **Axis** = The x-axis or y-axis number line

Understanding this structure is key to using Matplotlib’s object-oriented interface, as it allows you to **access and customize each component directly**.

### 12.2.2.2 Figure: The Top-Level Container

The **Figure** is an instance of `matplotlib.figure.Figure`. It is the **top level container** for all plot elements - think of it as the canvas or paper on which you draw.

#### Key points about Figure:

- It’s the final image that may contain one or more Axes
- It keeps track of all the Axes, titles, legends, etc.
- You cannot plot data directly on a Figure (only on Axes)

- It controls the overall image size, DPI, background color

Let's create an empty Figure to see what it looks like:

```
Create an empty figure - just the container, no plots yet
fig = plt.figure(figsize=(8, 6))
print(f"Figure object: {fig}")
print(f"Number of axes in figure: {len(fig.axes)}")
```

```
Figure object: Figure(800x600)
Number of axes in figure: 0
```

```
<Figure size 800x600 with 0 Axes>
```

**Observation:** The Figure is just a blank canvas. Creating a figure with `plt.figure()` does NOT automatically create an axes - that's why you see an empty rectangle.

### 12.2.2.3 Axes: The Plot Area

An Axes is an instance of `matplotlib.axes.Axes`. **This is where data are plotted** - it's what you typically think of as “a plot” or “a graph.”

#### Key points about Axes:

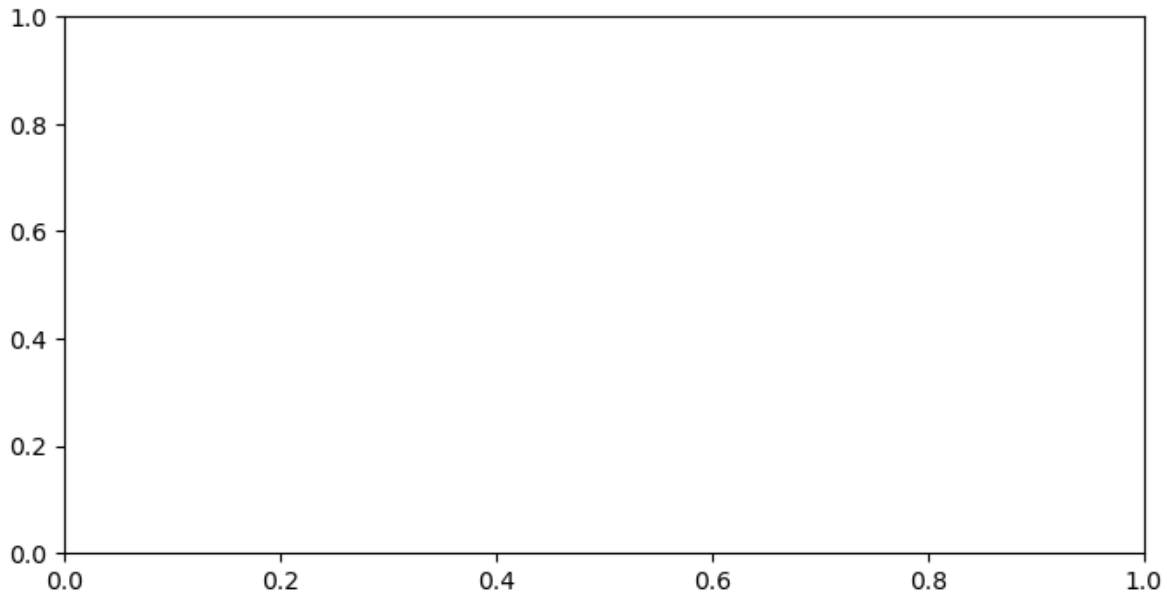
- A Figure can contain many Axes (multiple plots)
- But each Axes belongs to only one Figure
- This is where you call plotting methods like `.plot()`, `.scatter()`, `.bar()`
- Each Axes has its own x-axis, y-axis, title, labels, etc.

#### Creating Axes:

There are two common ways to add Axes to a Figure:

```
Method 1: Create figure first, then add axes
fig = plt.figure(figsize=(8, 4))
ax = fig.add_subplot(1, 1, 1) # Add subplot: (rows, cols, position)
print(f"Created axes: {ax}")
print(f"Figure now has {len(fig.axes)} axes")
```

```
Created axes: Axes(0.125,0.11;0.775x0.77)
Figure now has 1 axes
```



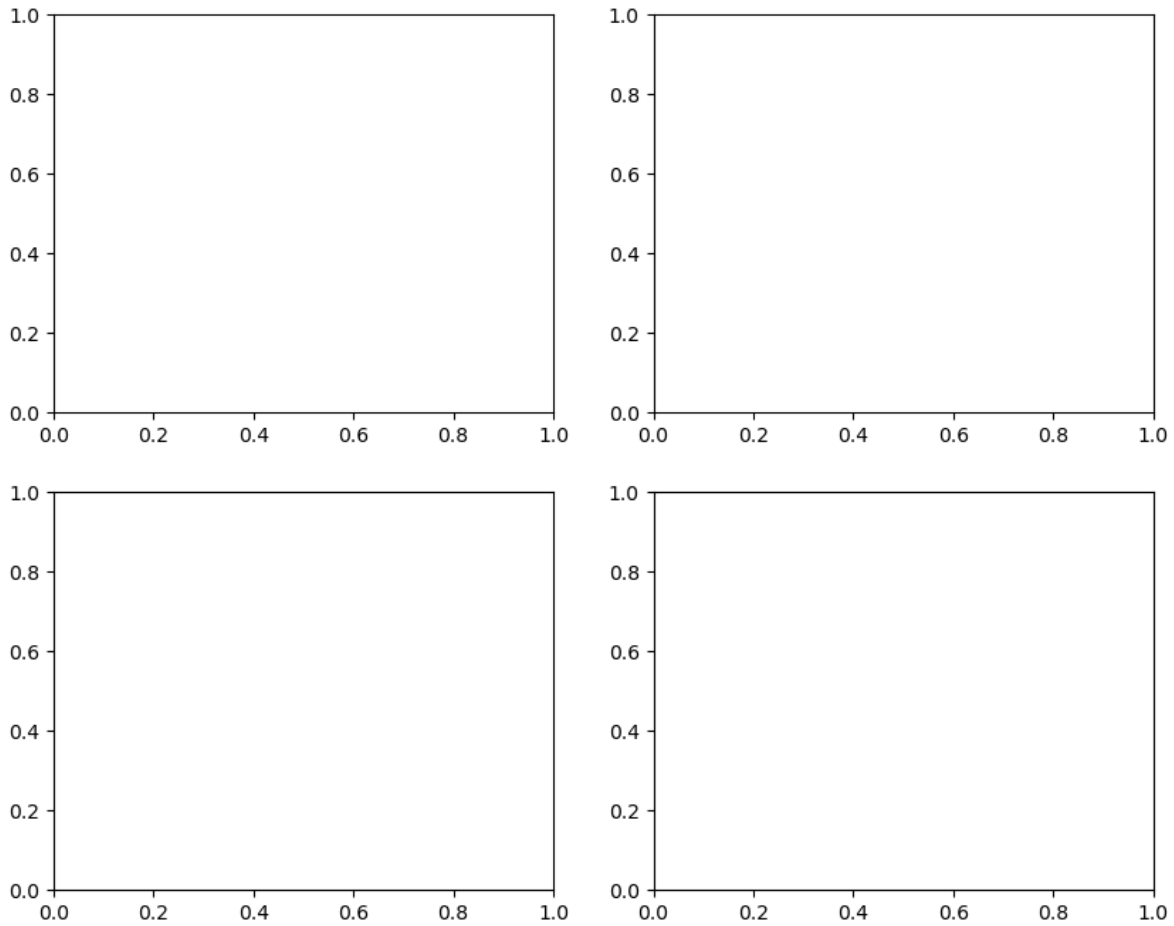
**Better approach:** Use `plt.subplots()` to create Figure and Axes together:

```
Method 2: Create figure and axes in one step (RECOMMENDED)
fig, axes = plt.subplots(2, 2, figsize=(10, 8))
print(f"Created figure with {len(fig.axes)} axes")
print(f"Axes array shape: {axes.shape}")
print(f"Access top-left axes: {axes[0, 0]}")
```

Created figure with 4 axes

Axes array shape: (2, 2)

Access top-left axes: Axes(0.125,0.53;0.352273x0.35)



**Note:** When you create a 2x2 grid with `subplots(2, 2)`, you get a 2D array of Axes objects. You access them using indexing: `axes[row, col]`.

### 12.2.3 Create and Customize Plots with the OOP Interface

Now that we understand the hierarchy, let's create a complete plot using the OOP interface, customizing every component.

#### 12.2.3.1 Components of an Axes Object

The `Axes` object contains several elements that make up the plot:

- **Data plotting area:** Contains the actual data visualizations (lines, bars, points, etc.)
- **X-axis and Y-axis:** Controls axis limits, labels, and ticks
- **Title and Labels:** The overall title and labels for each axis

- **Gridlines:** Optional lines to help align data visually
- **Spines:** The borders around the plot
- **Legend:** Explains what different data series represent
- **Annotations:** Text or arrows highlighting points of interest

Let's create a fully customized plot demonstrating all these components:

```
Create sample data
x = np.arange(10)
y = x**2

Step 1: Create figure and axes
fig, ax = plt.subplots(1, 1, figsize=(10, 6))

Step 2: Plot the data
ax.plot(x, y, color='blue', linewidth=2, marker='o', markersize=8, label='y = x2')

Step 3: Set title and labels
ax.set_title("Exponential Growth Pattern", fontsize=16, fontweight='bold')
ax.set_xlabel("Age (years)", fontsize=12)
ax.set_ylabel("Cell Growth (count)", fontsize=12)

Step 4: Set axis limits
ax.set_xlim([0, 10])
ax.set_ylim([0, 100])

Step 5: Customize ticks
ax.set_xticks(range(0, 11, 2))
ax.set_yticks(range(0, 101, 20))

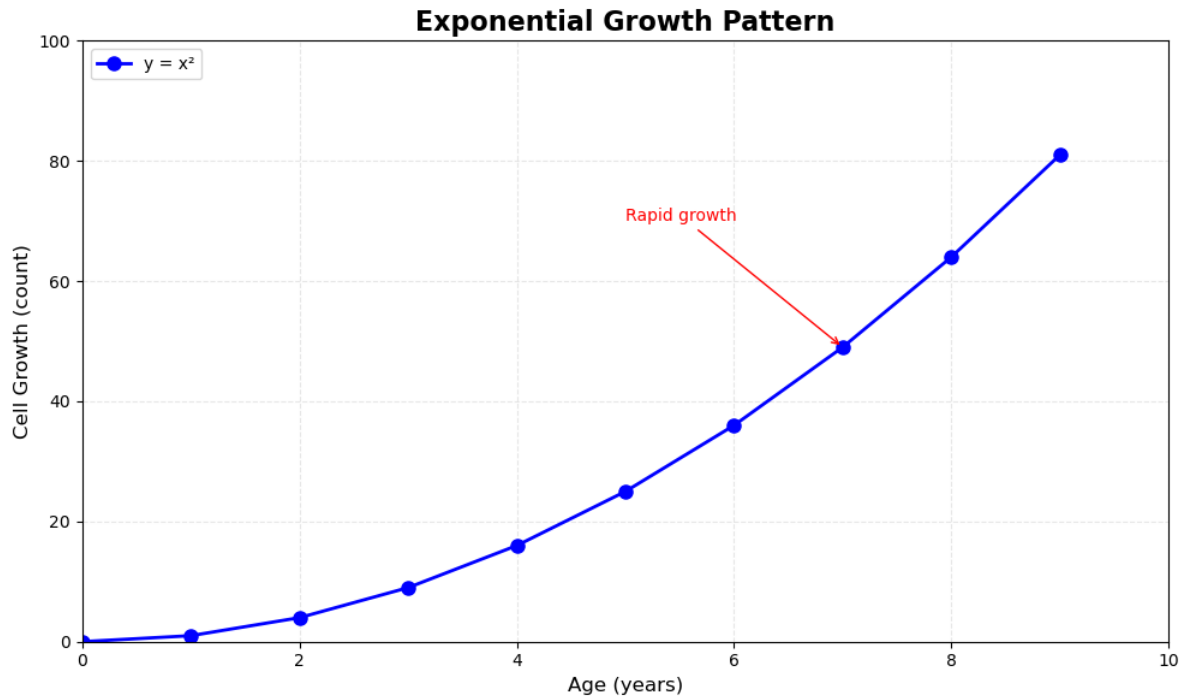
Step 6: Add grid for readability
ax.grid(True, alpha=0.3, linestyle='--')

Step 7: Add legend
ax.legend(loc="upper left", frameon=True)

Step 8: Add annotation
ax.annotate('Rapid growth', xy=(7, 49), xytext=(5, 70),
 arrowprops=dict(arrowstyle='->', color='red'),
 fontsize=10, color='red')

plt.tight_layout()
plt.show()
```





### Key Observations:

Notice the pattern in the OOP interface - every customization method starts with `ax.set_*`() or acts on the axes object:

- `ax.set_title()` - sets title
- `ax.set_xlabel()` / `ax.set_ylabel()` - sets labels
- `ax.set_xlim()` / `ax.set_ylim()` - sets axis limits
- `ax.set_xticks()` / `ax.set_yticks()` - sets tick positions
- `ax.grid()` - adds gridlines
- `ax.legend()` - adds legend
- `ax.annotate()` - adds annotations

This explicit control is what makes the OOP interface powerful for complex visualizations.

## 12.3 Mastering Subplots

Now that you understand matplotlib's architecture, let's explore how to create sophisticated multi-panel layouts. Subplots allow you to display multiple related visualizations in a single figure, making comparisons easier and creating comprehensive data stories.

The OOP interface truly shines when you need to work with multiple plots or integrate with other libraries. Here's why it's essential:

### 12.3.1 Types of Subplot Layouts

We'll cover two main scenarios:

1. **Non-overlapping subplots** - Multiple plots in a grid (most common)
2. **Nested subplots** - Plots inside other plots (for insets and zoomed views)

### 12.3.2 Create Simple Subplot Grids

The most common use case is arranging multiple plots in a grid pattern. The `plt.subplots()` function makes this straightforward.

**Syntax:**

```
fig, axes = plt.subplots(nrows, ncols, figsize=(width, height))
```

Let's create a 2x2 grid and understand how to access individual subplots:

```
Create a simple 2x2 grid of subplots
fig, axes = plt.subplots(2, 2, figsize=(12, 10))

Create sample data
x = np.linspace(0, 10, 100)

Plot in each subplot using array indexing
axes[0, 0].plot(x, np.sin(x))
axes[0, 0].set_title('Sine Wave')

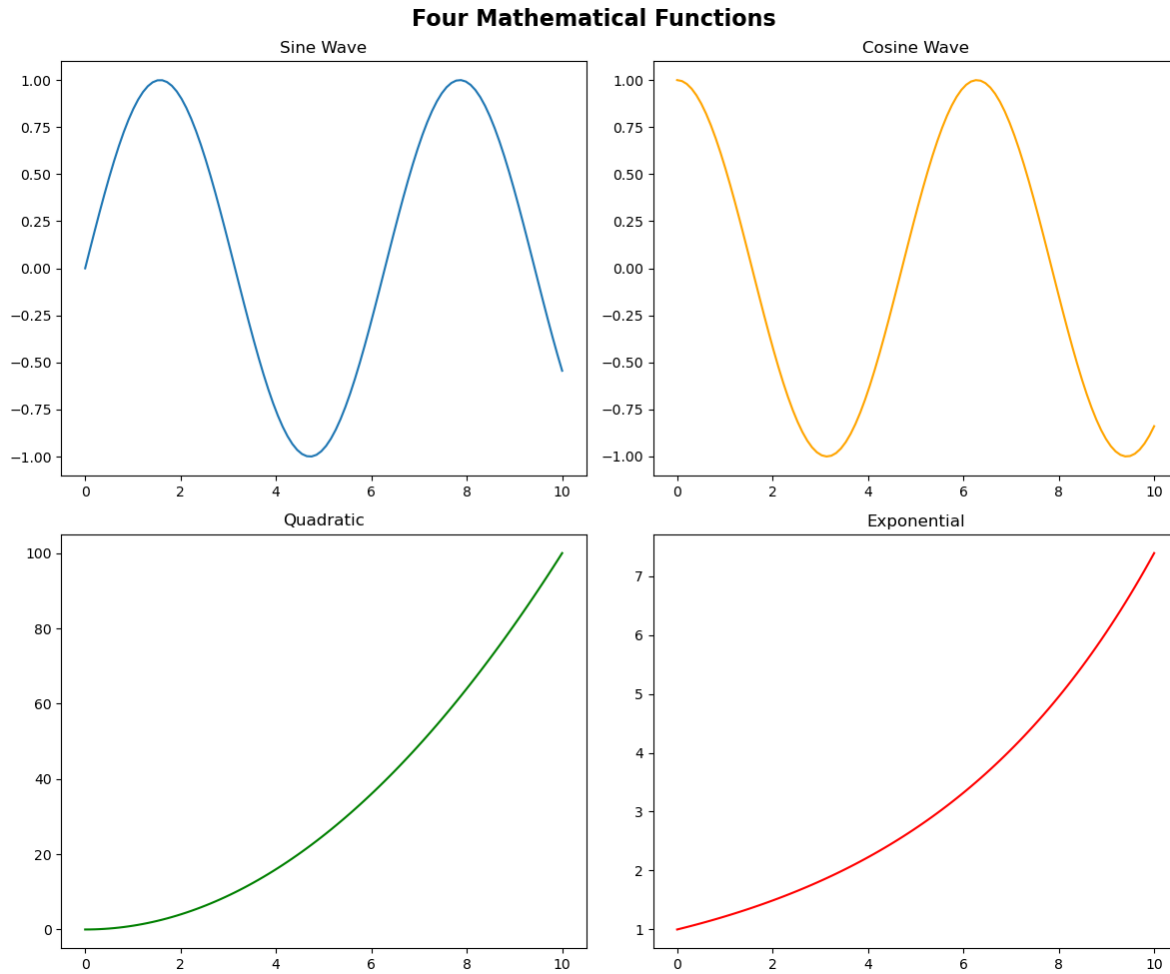
axes[0, 1].plot(x, np.cos(x), color='orange')
axes[0, 1].set_title('Cosine Wave')

axes[1, 0].plot(x, x**2, color='green')
axes[1, 0].set_title('Quadratic')

axes[1, 1].plot(x, np.exp(x/5), color='red')
axes[1, 1].set_title('Exponential')

Add a main title for the entire figure
fig.suptitle('Four Mathematical Functions', fontsize=16, fontweight='bold')

plt.tight_layout()
plt.show()
```



#### Key Points:

- `axes` is a 2D NumPy array with shape `(2, 2)`
- Access subplots using `axes[row, col]` (0-indexed)
- Each axes is independent - customize titles, labels, colors separately
- `fig.suptitle()` adds a title for the entire figure (not just one subplot)
- `plt.tight_layout()` automatically adjusts spacing to prevent overlaps

### 12.3.3 Control Subplot Layout and Spacing

Proper spacing between subplots is crucial for readability. Let's explore layout control options.

#### Layout Control Methods:

- `plt.tight_layout()` - Automatic spacing adjustment (recommended)
- `plt.tight_layout(pad=value)` - Control padding between subplots
- `plt.subplots_adjust()` - Manual control over spacing
- `figsize` parameter - Control overall figure dimensions

### 12.3.4 Integrate Matplotlib with Pandas and Seaborn

One of the most powerful features of the OOP interface is the ability to pass axes objects to pandas and seaborn plotting functions. This gives you precise control over where each plot appears.

#### The `ax` Parameter:

Both pandas and seaborn plotting functions accept an `ax` parameter:

- `df.plot(..., ax=axes[0, 0])` - pandas plots
- `sns.scatterplot(..., ax=axes[0, 1])` - seaborn plots

This allows you to:

- Combine different plot types in one figure
- Mix pandas, seaborn, and matplotlib plotting
- Create complex dashboards with precise layout control

Let's create a comprehensive visualization using multiple data sources and libraries:

```
Load datasets
flowers_df = sns.load_dataset('iris')
tips_df = sns.load_dataset('tips')
flights_df = sns.load_dataset("flights").pivot(index="month", columns="year", values="passengers")

Create a 2x2 grid
fig, axes = plt.subplots(2, 2, figsize=(14, 10))

Subplot 1: Seaborn scatter plot
axes[0, 0].set_title('Sepal Length vs Width by Species', fontsize=12, fontweight='bold')
sns.scatterplot(
 data=flowers_df,
 x='sepal_length',
 y='sepal_width',
 hue='species',
 s=100,
 ax=axes[0, 0]
)
```

```

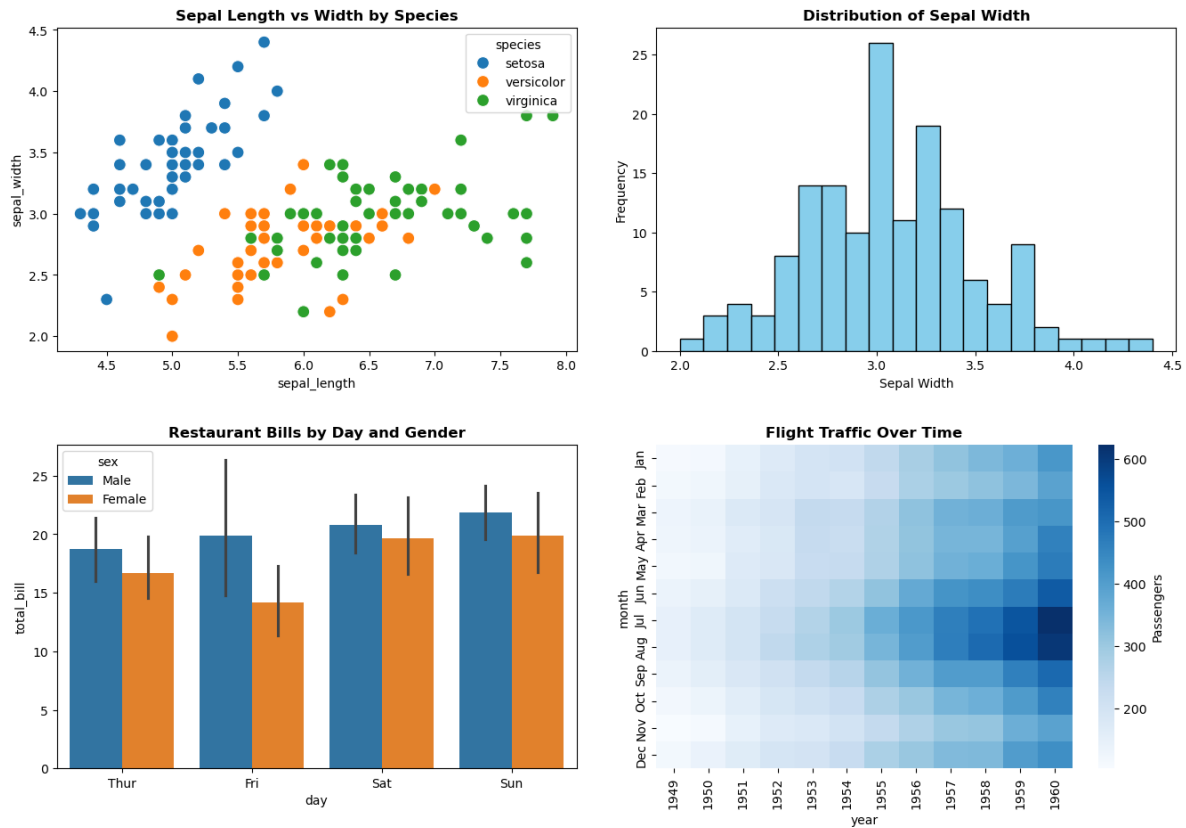
Subplot 2: Pandas histogram
axes[0, 1].set_title('Distribution of Sepal Width', fontsize=12, fontweight='bold')
flowers_df['sepal_width'].plot.hist(bins=20, ax=axes[0, 1], color='skyblue', edgecolor='black')
axes[0, 1].set_xlabel('Sepal Width')
axes[0, 1].set_ylabel('Frequency')

Subplot 3: Seaborn bar plot
axes[1, 0].set_title('Restaurant Bills by Day and Gender', fontsize=12, fontweight='bold')
sns.barplot(
 data=tips_df,
 x='day',
 y='total_bill',
 hue='sex',
 ax=axes[1, 0]
)

Subplot 4: Seaborn heatmap
axes[1, 1].set_title('Flight Traffic Over Time', fontsize=12, fontweight='bold')
sns.heatmap(flights_df, cmap='Blues', ax=axes[1, 1], cbar_kws={'label': 'Passengers'})

Adjust layout with custom padding
plt.tight_layout(pad=3)
plt.show()

```



## Key Observations:

1. **Mixed Libraries:** We seamlessly combined seaborn scatter plots, pandas histograms, seaborn bar plots, and seaborn heatmaps
2. **The `ax` Parameter:** Every plotting function received a specific axes object (`ax=axes[row, col]`)
3. **Independent Customization:** Each subplot has its own title, labels, and styling
4. **Layout Control:** `tight_layout(pad=3)` adds extra padding for readability

## Important Notes:

- Seaborn and pandas are wrappers around matplotlib
- They create plots ON the axes you specify via the `ax` parameter
- Without the `ax` parameter, they would create their own figure/axes
- This is why the OOP interface is essential for complex layouts

### 12.3.5 Create Nested Subplots (Insets)

Sometimes you want to show a detailed view of a specific region within a larger plot. This is called an **inset** or **nested subplot**.

**Use Cases for Insets:**

- Zooming into a specific region of interest
- Showing a histogram or distribution alongside the main plot
- Displaying summary statistics or related analysis
- Creating “picture-in-picture” visualizations

You can create nested subplots using `fig.add_axes()` or `inset_axes()`.

#### 12.3.5.1 Using `add_axes()` for Precise Control

**Syntax:**

```
ax = fig.add_axes([left, bottom, width, height])
```

**Parameters (all are fractions from 0 to 1):**

- **left:** Horizontal starting position (0=left edge, 1=right edge)
- **bottom:** Vertical starting position (0=bottom edge, 1=top edge)
- **width:** Width as fraction of figure width
- **height:** Height as fraction of figure height

Let’s create a plot with two inset plots showing related information:

```
Set random seed for reproducibility
np.random.seed(19680801)

Create colored noise data
dt = 0.001
t = np.arange(0.0, 10.0, dt)
r = np.exp(-t[:1000] / 0.05) # Impulse response
x = np.random.randn(len(t))
s = np.convolve(x, r)[:len(x)] * dt # Colored noise

Create the main figure and axes
fig, main_ax = plt.subplots(figsize=(12, 6))

Plot main data
```

```

main_ax.plot(t, s, linewidth=0.5)
main_ax.set_xlim(0, 1)
main_ax.set_ylim(1.1 * np.min(s), 2 * np.max(s))
main_ax.set_xlabel('Time (s)', fontsize=12)
main_ax.set_ylabel('Current (nA)', fontsize=12)
main_ax.set_title('Gaussian Colored Noise with Inset Analysis', fontsize=14, fontweight='bold')

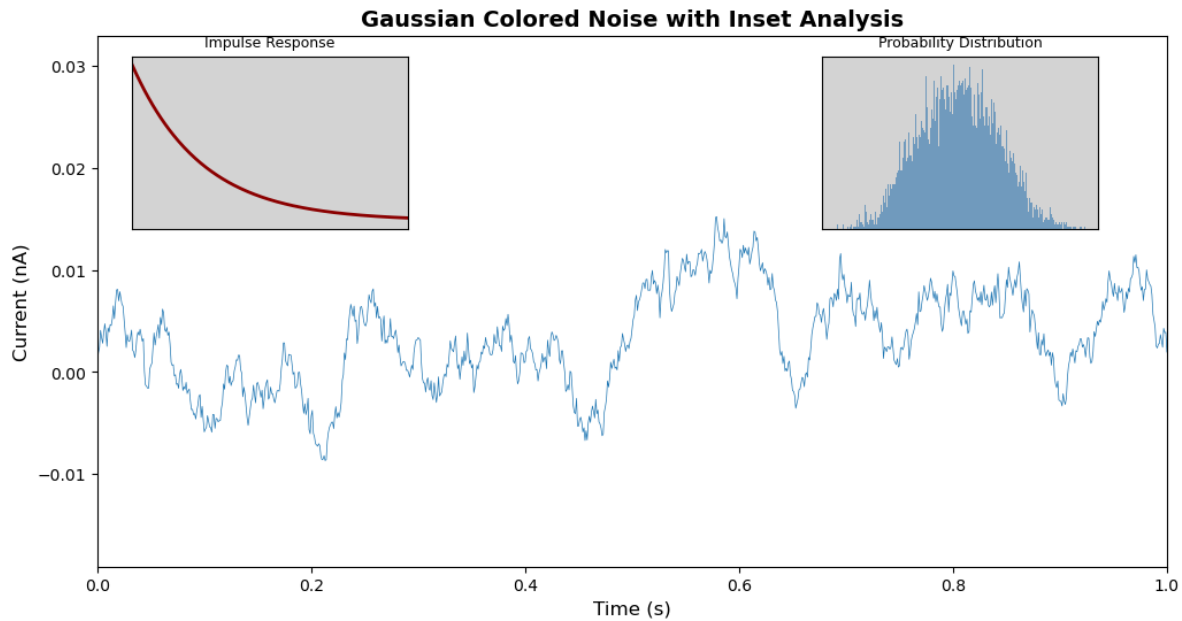
Create right inset showing probability distribution
[left, bottom, width, height] as fractions of figure
right_inset_ax = fig.add_axes([0.65, 0.6, 0.2, 0.25], facecolor='lightgray')
right_inset_ax.hist(s, 400, density=True, color='steelblue', alpha=0.7)
right_inset_ax.set_title('Probability Distribution', fontsize=9)
right_inset_ax.set_xticks([])
right_inset_ax.set_yticks([])

Create left inset showing impulse response
left_inset_ax = fig.add_axes([0.15, 0.6, 0.2, 0.25], facecolor='lightgray')
left_inset_ax.plot(t[:len(r)], r, color='darkred', linewidth=2)
left_inset_ax.set_title('Impulse Response', fontsize=9)
left_inset_ax.set_xlim(0, 0.2)
left_inset_ax.set_xticks([])
left_inset_ax.set_yticks([])

plt.show()

```





### Understanding the Inset Positions:

1. **Right Inset** [0.65, 0.6, 0.2, 0.25]:

- Starts 65% from left edge
- Starts 60% from bottom edge
- Width is 20% of figure width
- Height is 25% of figure height

2. **Left Inset** [0.15, 0.6, 0.2, 0.25]:

- Starts 15% from left edge
- Same vertical position and size as right inset

### Tips for Insets:

- Use `facecolor` to distinguish insets from main plot
- Remove ticks (`set_xticks([])`) to reduce clutter
- Keep inset titles short and informative
- Position insets where they don't obscure important data
- Consider using `inset_axes()` from `mpl_toolkits.axes_grid1` for more flexible positioning

## 12.3.6 Advanced Formatting with Custom Tick Formatters

When creating professional visualizations, you often need to format axis labels for readability. This is especially important for:

- Large numbers (adding commas or using K/M notation)
- Currency values (adding \$ or other symbols)
- Percentages
- Dates and times
- Scientific notation

The OOP interface provides powerful formatting capabilities through the `yaxis` and `xaxis` objects.

### 12.3.6.1 Example: Formatting Large Numbers with Commas

Let's visualize noise complaint data with properly formatted axis labels:

```
Load noise complaint data
nyc_party_complaints = pd.read_csv('datasets/party_nyc.csv')
nyc_party_complaints.head()
```

	Created Date	Closed Date	Location Type	Incident Zip	City	Borough
0	12/31/2015 0:01	12/31/2015 3:48	Store/Commercial	10034.0	NEW YORK	MANHAT
1	12/31/2015 0:02	12/31/2015 4:36	Store/Commercial	10040.0	NEW YORK	MANHAT
2	12/31/2015 0:03	12/31/2015 0:40	Residential Building/House	10026.0	NEW YORK	MANHAT
3	12/31/2015 0:03	12/31/2015 1:53	Residential Building/House	11231.0	BROOKLYN	BROOKLYN
4	12/31/2015 0:05	12/31/2015 3:49	Residential Building/House	10033.0	NEW YORK	MANHAT

Now let's create a bar plot showing complaint locations, with properly formatted y-axis labels:

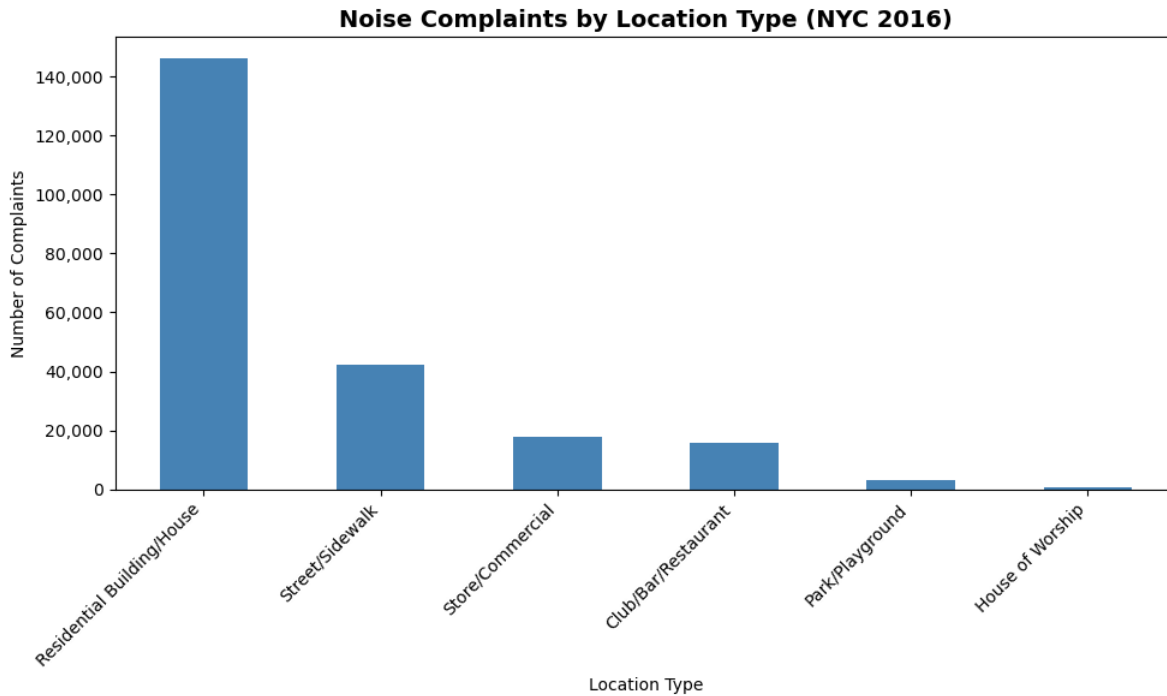
```
Create bar plot using pandas (returns axes object)
ax = nyc_party_complaints['Location Type'].value_counts().plot.bar(
 ylabel='Number of Complaints',
 xlabel='Location Type',
 figsize=(10, 6),
 color='steelblue'
)

Format y-axis to add commas to large numbers
ax.yaxis.set_major_formatter('{x:,.0f}')

Customize the plot
ax.set_title('Noise Complaints by Location Type (NYC 2016)', fontsize=14, fontweight='bold')
```

```
ax.set_xticklabels(ax.get_xticklabels(), rotation=45, ha='right')

plt.tight_layout()
plt.show()
```



### Key Technique - Custom Axis Formatting:

```
ax.yaxis.set_major_formatter('{x:,.0f}')
```

This uses Python’s format specification mini-language:

- `{x:,.0f}` - Format as float with comma separators and 0 decimal places
- `{x:,.2f}` - Two decimal places with commas
- `{x:.2%}` - Percentage with 2 decimal places
- `${x:,.0f}` - Currency format

### Observations:

- Most complaints come from residential buildings and houses (as expected for party/music noise)
- The y-axis now shows “1,000” instead of “1000” - much easier to read!
- Pandas `.plot.bar()` returns an axes object that we can further customize

- We rotated x-axis labels 45° for better readability using `set_xticklabels()`

This demonstrates how pandas plotting (convenience) and matplotlib OOP (control) work together seamlessly.

## 12.4 Creating Subplots with Seaborn

We previously demonstrated how Seaborn integrates seamlessly with Matplotlib's object-oriented interface, allowing you to pass the `ax` argument to any Seaborn function, thereby directing the plot to a specific axis within a subplot grid.

Additionally, Seaborn offers a more convenient and simplified approach to creating subplots, thanks to its high-level functions and built-in integration with Matplotlib. Here's how Seaborn makes working with subplots easier:

### 12.4.1 Using Facetgrid

Seaborn's `FacetGrid` is a powerful tool for creating **small multiples** - grids of plots where each subplot shows a subset of your data based on categorical variables.

#### Why Use FacetGrid?

- Automatically creates subplot grids based on data categories
- Much easier than manually creating subplots and filtering data
- Ideal for comparing patterns across different groups
- Enables exploration of multi-dimensional relationships

#### Key Parameters:

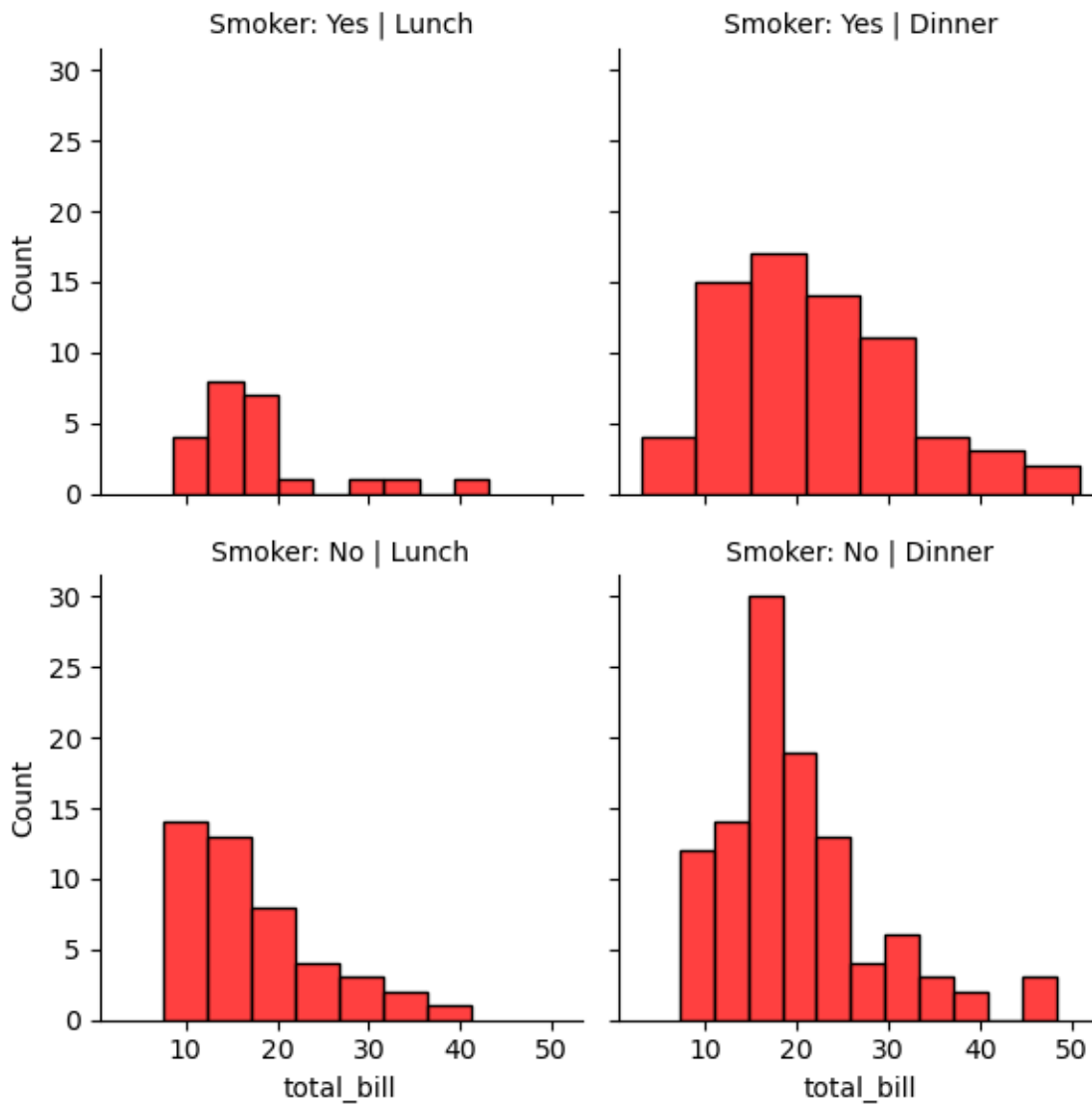
- `data`: DataFrame to visualize
- `col`: Variable to create column-wise subplots
- `row`: Variable to create row-wise subplots
- `hue`: Variable for color-coding within each subplot
- `col_wrap`: Wrap columns after this many plots

Let's explore with the tips dataset:

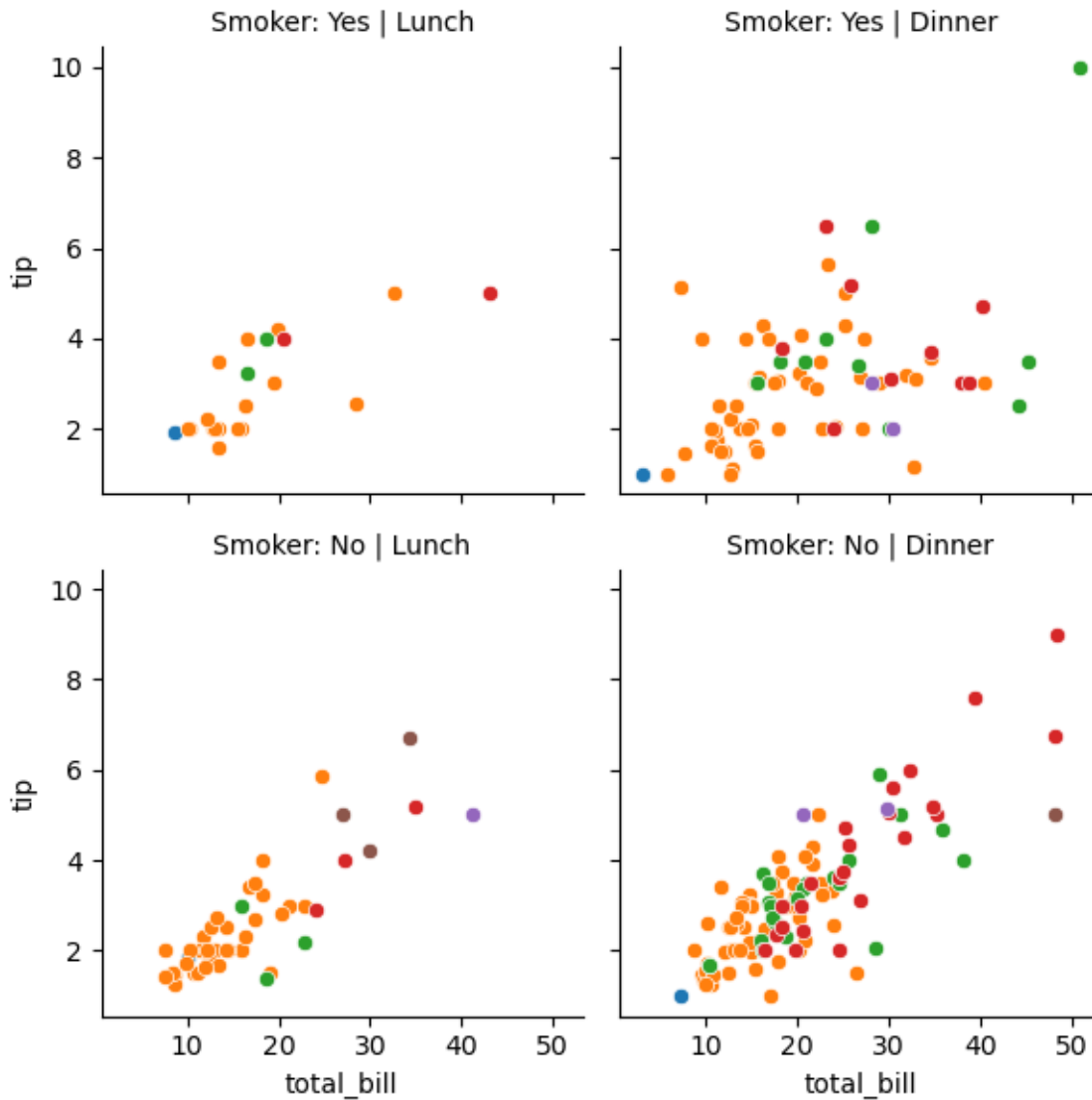
```
Seaborn Example using FacetGrid:
tips_df = sns.load_dataset("tips")
tips_df.head()
```

	total_bill	tip	sex	smoker	day	time	size
0	16.99	1.01	Female	No	Sun	Dinner	2
1	10.34	1.66	Male	No	Sun	Dinner	3
2	21.01	3.50	Male	No	Sun	Dinner	3
3	23.68	3.31	Male	No	Sun	Dinner	2
4	24.59	3.61	Female	No	Sun	Dinner	4

```
g = sns.FacetGrid(tips_df, col='time', row='smoker')
g.map(sns.histplot, 'total_bill', color='r')
g.set_titles(col_template="{col_name}", row_template="Smoker: {row_name}");
```



```
adding hue to the FacetGrid
g = sns.FacetGrid(tips_df, col='time', row='smoker', hue='size')
Plot a scatterplot of the total bill and tip for each combination of time and smoker
g.map(sns.scatterplot, 'total_bill', 'tip')
g.set_titles(col_template="{col_name}", row_template="Smoker: {row_name}");
```

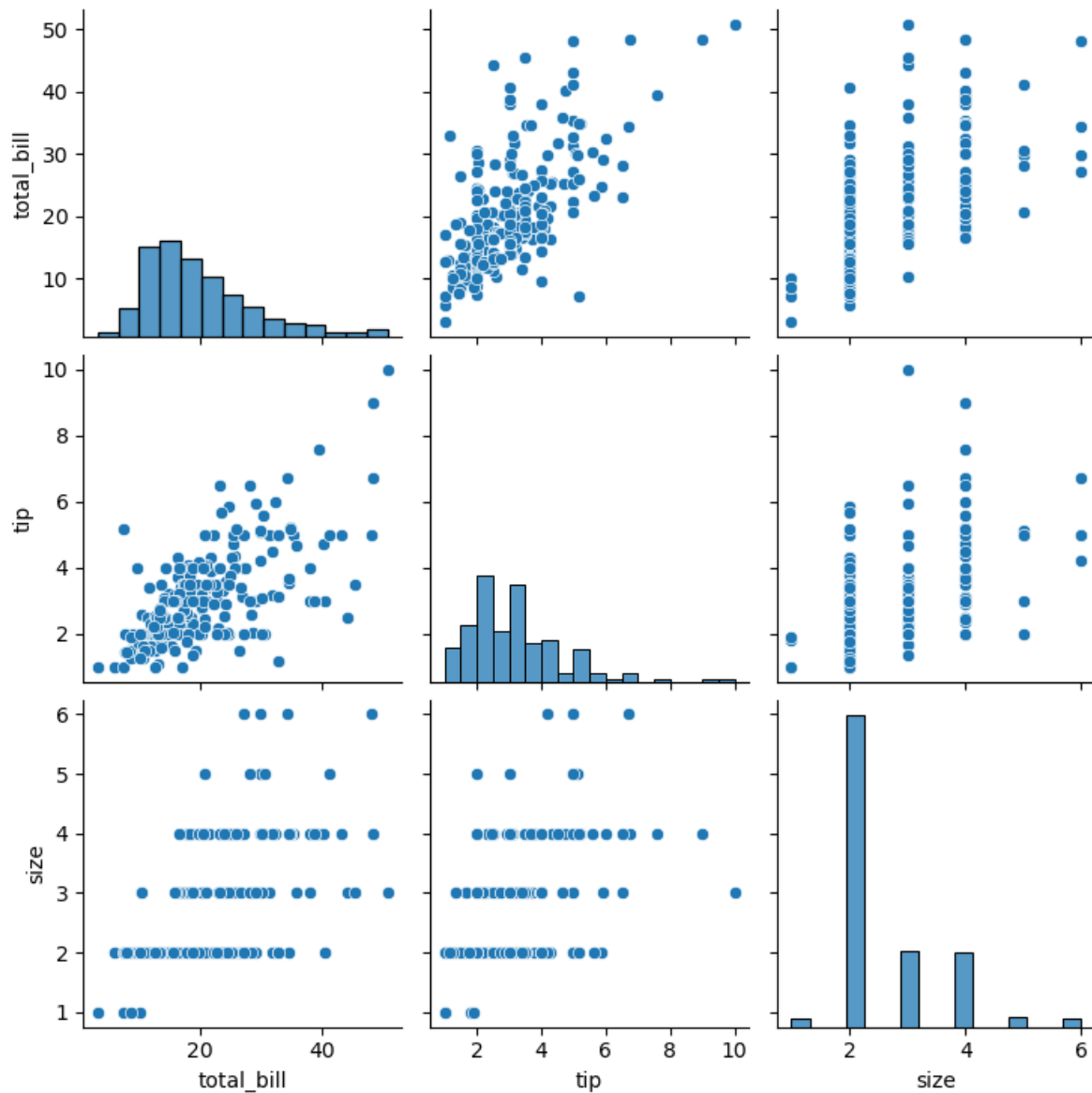


### 12.4.2 Using Pairplot

Pairplots are used to visualize the association between all variable-pairs in the data. In other words, pairplots simultaneously visualize the scatterplots between all variable-pairs.

Let us visualize the pair-wise association of tips variables in the tips dataset

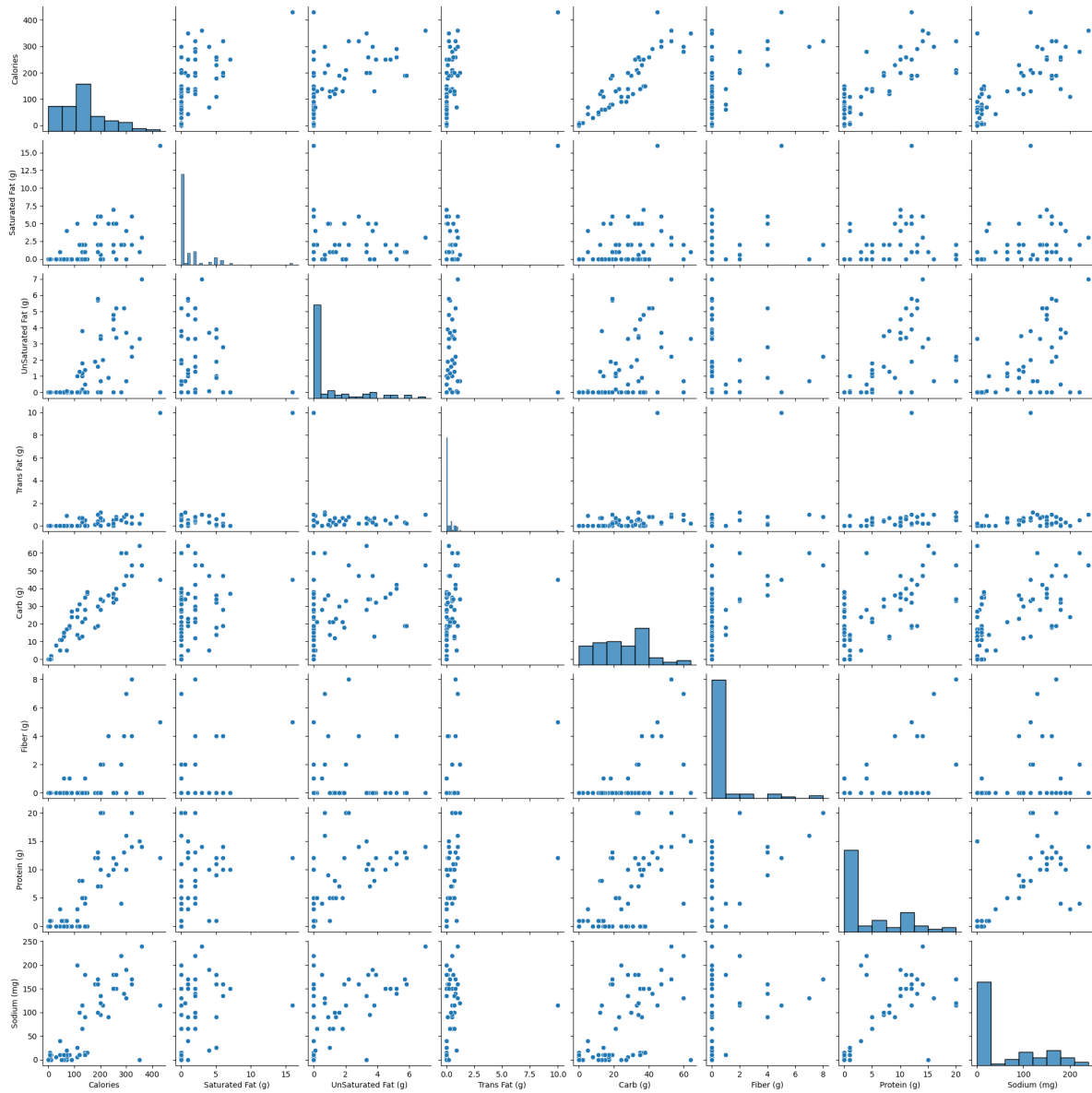
```
sns.pairplot(tips_df);
```



Let us visualize the pair-wise association of nutrition variables in the starbucks drinks data.

```
starbucks_drinks = pd.read_csv('datasets/starbucks-menu-nutrition-drinks.csv')
sns.pairplot(starbucks_drinks);
```





In the above pairplot, note that:

- The histograms on the diagonal of the grid show the distribution of each of the variables.
- Instead of a histogram, we can visualize the density plot with the argument `kde = True`.
- The scatterplots in the rest of the grid are the pair-wise plots of all the variables.

## 12.5 Geospatial Plotting

There are several widely used Python packages specifically designed for working with geospatial datasets. In this lesson, we will cover:

- GeoPandas
- Folium

Let's import them

```
import warnings

Suppress all non-critical warnings
warnings.filterwarnings("ignore")
import geopandas as gpd
import geopandas
import folium
import geodatasets
```

### 12.5.1 Static Plots with GeoPandas

A **shapefile** is a widely-used format for storing geographic information system (GIS) data, specifically vector data. It contains geometries (like points, lines, and polygons) that represent features on the earth's surface, along with associated attributes for each feature, such as names, populations, or other data relevant to the feature.

#### 12.5.1.1 Components of a Shapefile

A shapefile isn't a single file but a collection of files with the same name and different extensions, which work together to store geographic and attribute data:

- **.shp**: Stores the geometry (shapes of features, like points, lines, polygons).
- **.shx**: Contains an index to quickly access geometries in the **.shp** file.
- **.dbf**: A table storing attributes associated with each feature.

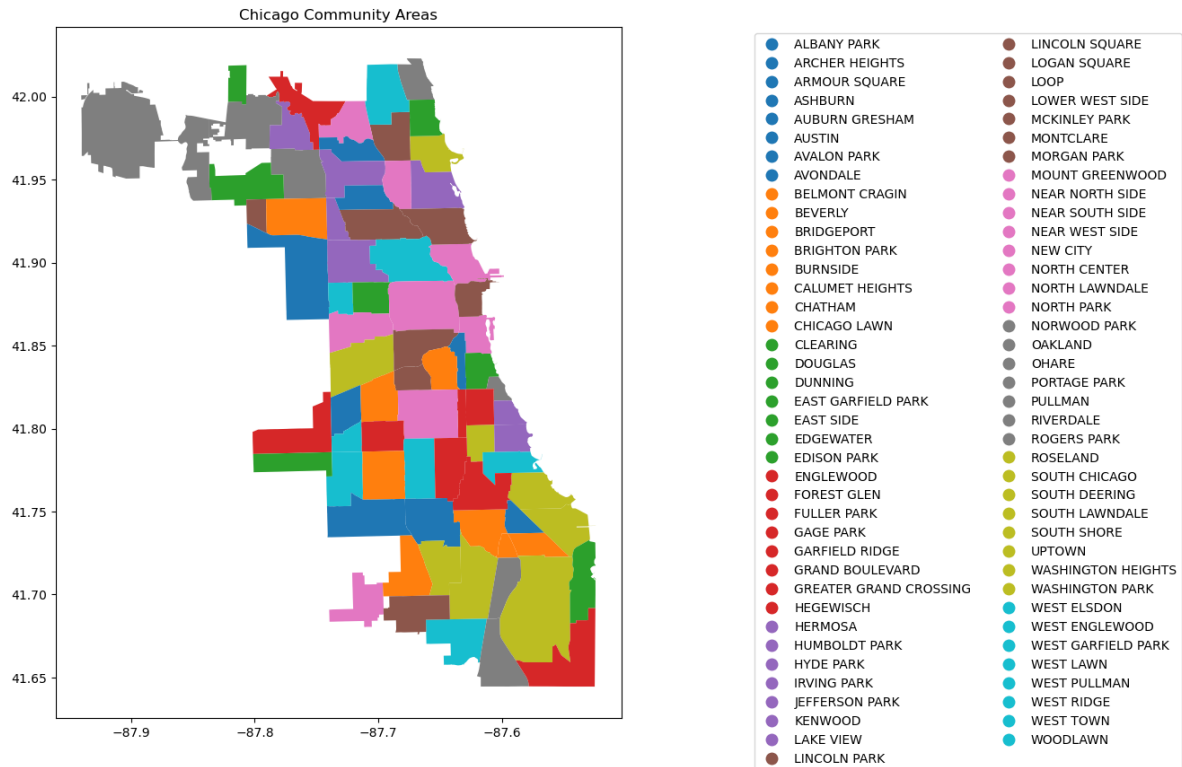
There may also be other optional files (e.g., **.prj** for projection information).

```
Create figure and axis
fig, ax = plt.subplots(figsize=(15, 10))

Plot your GeoDataFrame
chicago = gpd.read_file(r'datasets/chicago_boundaries\geo_export_26bce2f2-c163-42a9-9329-9ca
```

```
chicago.plot(column='community', ax=ax, legend=True, legend_kwds={'ncol': 2, 'bbox_to_anchor': (10, 10)})

Add title (optional)
plt.title('Chicago Community Areas');
```



Let's print out the information in the shapefile

```
chicago.head()
```

	area	area_num_1	area_numbe	comarea	comarea_id	community	perimeter	shape
0	0.0	35	35	0.0	0.0	DOUGLAS	0.0	4.6004
1	0.0	36	36	0.0	0.0	OAKLAND	0.0	1.6913
2	0.0	37	37	0.0	0.0	FULLER PARK	0.0	1.9916
3	0.0	38	38	0.0	0.0	GRAND BOULEVARD	0.0	4.8492
4	0.0	39	39	0.0	0.0	KENWOOD	0.0	2.9071

```
chicago['geometry'].head()
```

```
0 POLYGON ((-87.60914 41.84469, -87.60915 41.844...
1 POLYGON ((-87.59215 41.81693, -87.59231 41.816...
2 POLYGON ((-87.6288 41.80189, -87.62879 41.8017...
3 POLYGON ((-87.60671 41.81681, -87.6067 41.8165...
4 POLYGON ((-87.59215 41.81693, -87.59215 41.816...
Name: geometry, dtype: geometry
```

```
Check the column names to see available data fields
print("Columns in the shapefile:", chicago.columns)

Check the data types of each column
print("Data types:", chicago.dtypes)

View the spatial extent (bounding box) of the shapes
print("Bounding box:", chicago.total_bounds)

Check the coordinate reference system (CRS)
print("CRS:", chicago.crs)
```

```
Columns in the shapefile: Index(['area', 'area_num_1', 'area_numbe', 'comarea', 'comarea_id',
 'community', 'perimeter', 'shape_area', 'shape_len', 'geometry'],
 dtype='object')
Data types: area float64
area_num_1 object
area_numbe object
comarea float64
comarea_id float64
community object
perimeter float64
shape_area float64
shape_len float64
geometry geometry
dtype: object
Bounding box: [-87.94011408 41.64454312 -87.5241371 42.02303859]
CRS: EPSG:4326
```

To enhance the geospatial plot, we'll use the shapefile as a background to provide context for Chicago's community areas. On top of that, we'll layer points of interest, such as restaurants,

and shops, to illustrate the city's amenities. This approach will make the map more informative and visually engaging, with community boundaries as the foundation and key locations overlaid to highlight areas of interest.

Next, we will add the Divvy bicycle stations on top of the chicago shapefile

### 12.5.2 Dataset: Bicycle Sharing in Chicago

Divvy is Chicagoland's bike share system (in collaboration with Chicago Department of Transportation), with 6,000 bikes available at 570+ stations across Chicago and Evanston. Divvy provides residents and visitors with a convenient, fun and affordable transportation option for getting around and exploring Chicago.

Divvy, like other bike share systems, consists of a fleet of specially designed, sturdy and durable bikes that are locked into a network of docking stations throughout the region. The bikes can be unlocked from one station and returned to any other station in the system. People use bike share to explore Chicago, commute to work or school, run errands, get to appointments or social engagements, and more.

Divvy is available for use 24 hours/day, 7 days/week, 365 days/year, and riders have access to all bikes and stations across the system.

We will be using divvy trips in the year of 2013

```
read the csv file 'divvy_2013.csv' into pandas dataframe
data = pd.read_csv('datasets/divvy_2013.csv')
data.head()
```

	trip_id	usertype	gender	starttime	stoptime	tripduration	from_station_id
0	3940	Subscriber	Male	2013-06-27 01:06:00	2013-06-27 09:46:00	31177	91
1	4095	Subscriber	Male	2013-06-27 12:06:00	2013-06-27 12:11:00	301	85
2	4113	Subscriber	Male	2013-06-27 11:09:00	2013-06-27 11:11:00	140	88
3	4118	Customer	NaN	2013-06-27 12:11:00	2013-06-27 12:16:00	316	85
4	4119	Subscriber	Male	2013-06-27 11:12:00	2013-06-27 11:13:00	87	88

In the Divvy dataset, each trip record includes the latitude and longitude coordinates of both the pickup and drop-off locations, which correspond to Divvy bike stations. These coordinates allow us to map the precise locations of each station, making it possible to visually display the network of Divvy stations across the city. By plotting these stations on a map, we can better understand the geographic distribution and accessibility of Divvy's bike-sharing network.

Below are the basic data cleaning steps to extract the coordinates of the Divvy stations.

```
drop the duplicates in the column 'to_station_id', 'to_station_name', 'latitude_end', 'long
data_station_same = data[['from_station_id', 'from_station_name', 'latitude_start', 'longi
data_station_same.shape
```

### 12.5.3 Adding the divvy station to the plot

Once the coordinates are prepared, we'll add them as scatter plots on top of the Chicago shapefile

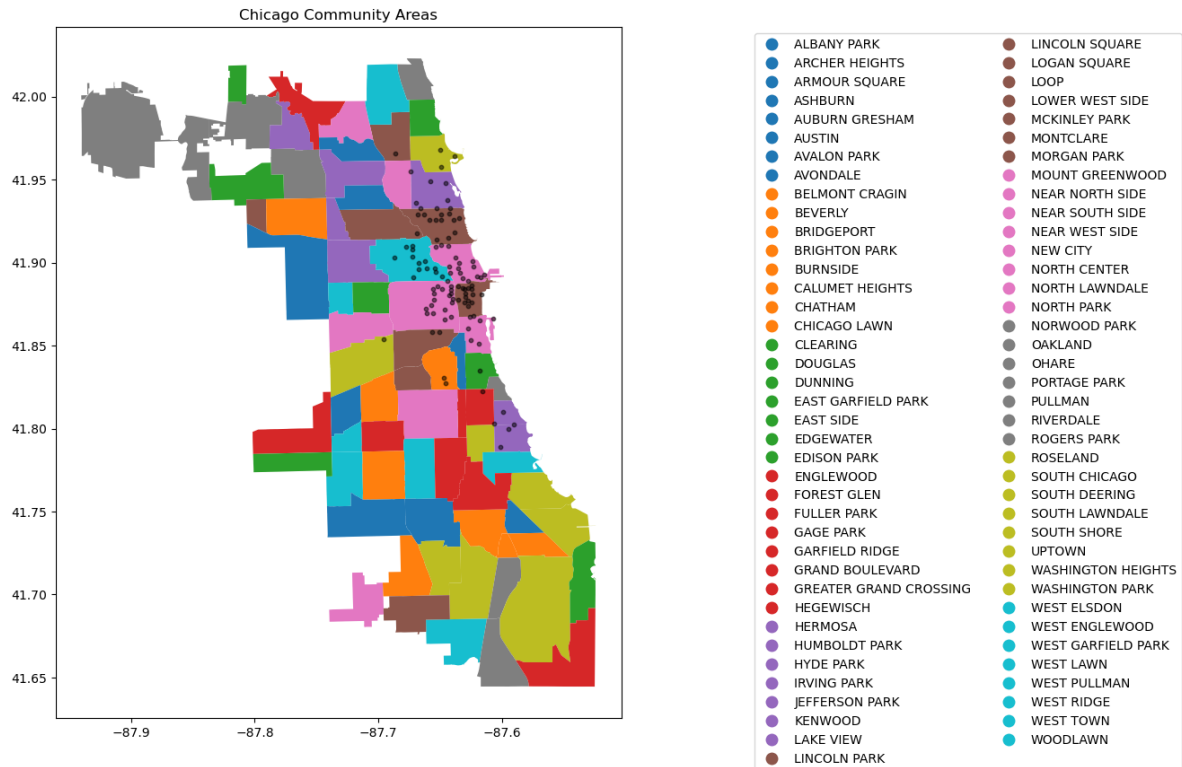
```
Adding the stations to the plot
fig, ax = plt.subplots(figsize=(15, 10))

chicago = gpd.read_file(r'datasets/chicago_boundaries\geo_export_26bce2f2-c163-42a9-9329-9ca
chicago.plot(column='community', ax=ax, legend=True, legend_kwds={'ncol': 2, 'bbox_to_anchor

Plot the stations
longlat_df = data[['latitude_start', 'longitude_start']].drop_duplicates()

plt.scatter(longlat_df['longitude_start'], longlat_df['latitude_start'], s=10, alpha=0.5, co

Add title (optional)
plt.title('Chicago Community Areas');
```

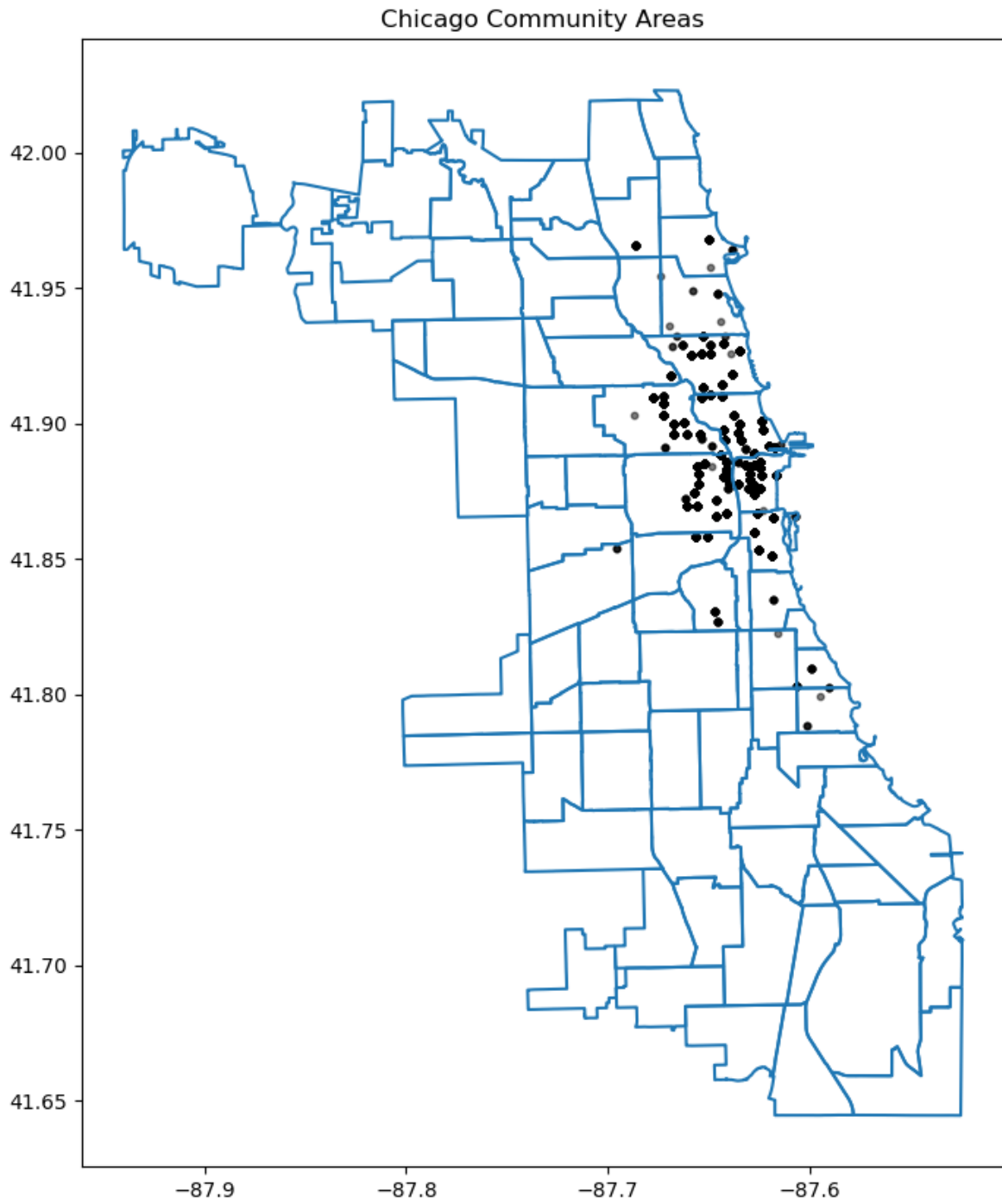


### 12.5.4 Change the chicago shapefile

Using a different Chicago shapefile from GeoDa is a great way to observe how geographic boundaries or data details may vary

```
chicago = gpd.read_file(geodatasets.get_path("geoda.chicago_commpop"))

Plot the data
fig, ax = plt.subplots(figsize=(15, 10))
chicago.boundary.plot(ax=ax)
plt.scatter(data['longitude_start'], data['latitude_start'], s=10, alpha=0.5, color='black',
plt.title('Chicago Community Areas');
```





## 12.5.5 Interactive Plotting

Alongside static plots, **geopandas** can create interactive maps based on the [folium](#) library.

Creating maps for interactive exploration mirrors the API of static [plots](#) in an `explore()` method of a `GeoSeries` or `GeoDataFrame`.

Here's an explanation of how `explore()` works and its key features:

Key Features of `explore()`:

### 1. Interactive Map Display:

- When you call `explore()` on a `Geodataframe` (`gdf`), it launches an interactive map widget directly within your Jupyter notebook.
- This map allows you to pan, zoom, and interact with the geometries (points, lines, polygons) in your `Geodataframe`.

### 2. Layer Control:

- `explore()` automatically adds the geometries from your `Geodataframe` as layers on the map.
- Each geometry type (points, lines, polygons) is displayed with appropriate styling and markers.

### 3. Tooltip Information:

- When you hover over a geometry in the map, `explore()` displays tooltip information that typically includes attribute data associated with that geometry.
- This feature is useful for inspecting specific details or properties of individual features in your geospatial dataset.

### 4. Search and Filter:

- `explore()` provides basic search and filter functionalities directly on the map.
- You can search for specific attribute values or filter the displayed features based on attribute criteria defined in your `Geodataframe`.

### 5. Customization:

- Although `explore()` provides default styling and interaction behaviors, you can customize the map further using parameters or by manipulating the `Geodataframe` before calling `explore()`.

```
use the geopandas explore default settings
chicago = gpd.read_file(geodatasets.get_path("geoda.chicago_commpop"))
m = chicago.explore()
display(m)
```

<folium.folium.Map at 0x27bd76af6b0>

Adding the population layer

```
import os
os.environ["OMP_NUM_THREADS"] = "1"
Customize the explore settings
chicago = gpd.read_file(geodatasets.get_path("geoda.chicago_commpop"))

m = chicago.explore(
 column="POP2010", # make choropleth based on "POP2010" column
 scheme="naturalbreaks", # use mapclassify's natural breaks scheme
 legend=True, # show legend
 k=10, # use 10 bins
 tooltip=False, # hide tooltip
 popup=["POP2010", "POP2000"], # show popup (on-click)
 legend_kwds=dict(colorbar=False), # do not use colorbar
 name="chicago", # name of the layer in the map
)

m
```

<folium.folium.Map at 0x27bd20a2240>

The `explore()` method returns a `folium.Map` object, which can also be passed directly (as you do with `ax` in `plot()`). You can then use folium functionality directly on the resulting map. Next, let's add the divvy station plot.

```
type(m)
```

`folium.folium.Map`

## 12.5.6 Adding the divvy station on the interactive Folium.Map

We need to extract the station information from the trip dataset and add description to the station. You can skip this part

```
Helper function for adding the description to the station
def row_to_html(row):
 row_df = pd.DataFrame(row).T
 row_df.columns = [col.capitalize() for col in row_df.columns]
 return row_df.to_html(index=False)

Extracting the latitude, longitude, and station name for plotting, and also counting the number of trips
grouped_df = data.groupby(['from_station_name', 'latitude_start', 'longitude_start'])['trip_id'].count()
display(grouped_df.sort_values('trip_id', ascending=False).head())
grouped_df.rename(columns={'from_station_name': 'title', 'latitude_start': 'latitude', 'longitude_start': 'longitude'})
grouped_df['description'] = grouped_df.apply(lambda row: row_to_html(row), axis=1)
geometry = gpd.points_from_xy(grouped_df['longitude'], grouped_df['latitude'])
geo_df = gpd.GeoDataFrame(grouped_df, geometry=geometry)
Optional: Assign Coordinate Reference System (CRS)
geo_df.crs = "EPSG:4326" # WGS84 coordinate system
geo_df.head()
```

	from_station_name	latitude_start	longitude_start	trip_id
75	Millennium Park	41.881032	-87.624084	207
54	Lake Shore Dr & Monroe St	41.881050	-87.616970	191
72	Michigan Ave & Oak St	41.900960	-87.623777	186
68	McClurg Ct & Illinois St	41.891020	-87.617300	177
73	Michigan Ave & Pearson St	41.897660	-87.623510	127

	title	latitude	longitude	count	description
0	Aberdeen St & Jackson Blvd	41.877726	-87.654787	28	<table border="1" class="dataframe">\
1	Aberdeen St & Madison St	41.881487	-87.654752	28	<table border="1" class="dataframe">\
2	Adler Planetarium	41.866095	-87.607267	6	<table border="1" class="dataframe">\
3	Ashland Ave & Armitage Ave	41.917859	-87.668919	20	<table border="1" class="dataframe">\
4	Ashland Ave & Augusta Blvd	41.899643	-87.667700	27	<table border="1" class="dataframe">\

We can add a hover tooltip (sometimes referred to as a tooltip or tooltip popup) for each point on your Folium map. This tooltip will appear when you hover over the markers on the map, providing additional information without needing to click on them. Here's how you can modify your existing code to include hover tooltips:

```

chicago = gpd.read_file(geodatasets.get_path("geoda.chicago_commpop"))

m = chicago.explore(
 column="POP2010", # make choropleth based on "POP2010" column
 scheme="naturalbreaks", # use mapclassify's natural breaks scheme
 legend=True, # show legend
 k=10, # use 10 bins
 tooltip=False, # hide tooltip
 popup=["POP2010", "POP2000"], # show popup (on-click)
 legend_kwds=dict(colorbar=False), # do not use colorbar
 name="chicago", # name of the layer in the map
)

geo_df.explore(
 m=m, # pass the map object
 color="red", # use red color on all points
 marker_kwds=dict(radius=5, fill=True), # make marker radius 10px with fill
 tooltip="description", # show "name" column in the tooltip
 tooltip_kwds=dict(labels=False), # do not show column label in the tooltip
 name="divstation", # name of the layer in the map
)

m

```

<folium.folium.Map at 0x27bda336690>

## 12.6 Independent Study

### 12.6.1 Practice exercise 1

Read *survey\_data\_clean.csv*

#### 12.6.1.1

Is `NU_GPA` associated with `parties_per_month`? Analyze the association separately for *Sophomores*, *Juniors*, and *Seniors* (categories of the variable `school_year`). Make scatterplots of `NU_GPA` vs `parties_per_month` in a 1 x 3 grid, where each grid is for a distinct `school_year`. Plot the trendline as well for each scatterplot. Use the file *survey\_data\_clean.csv*.

### 12.6.1.2

Capping the the values of `parties_per_month` to 30, and make the visualizations again.

## **Part IV**

# **From messy to insight**

# 13 Data Cleaning

<IPython.core.display.Image object>

Real-world data is rarely clean and ready for analysis. Data scientists often spend **60-80% of their time** on data preprocessing, with data cleaning being a critical first step that directly impacts the quality of insights and model performance.

This chapter focuses on essential **data cleaning techniques** - identifying and addressing issues in raw data quality that can compromise your analysis.

## 13.1 Common Data Quality Issues

Issue Type	Example	Impact	Solution
Missing values	NaN in “income” column	Reduces sample size, biases results	Impute or drop strategically
Duplicates	Multiple identical rows	Inflates counts, skews statistics	Identify and remove
Inconsistent types	'1', 1.0, 1 for same value	Causes errors in operations	Standardize data types
Outliers	Salary = \$1,000,000,000	Distorts means, regression lines	Cap, transform, or remove

**Note:** We previously covered handling inconsistent data types (e.g., converting strings to datetime for time-based feature engineering). This chapter focuses on the other three major issues.

Let’s first import necessary libraries

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import sklearn as sk
import seaborn as sns
```

```
sns.set(font_scale=1.5)

%matplotlib inline
```

## 13.2 Handling Missing Data

Missing values in a dataset can occur for several reasons:

- **Equipment failure** - Breakdown of measuring equipment
- **Human error** - Accidental deletion or failure to record observations
- **Non-response** - Survey respondents skip questions
- **Data entry mistakes** - Errors made by researchers or data collectors

### 13.2.1 Dataset Introduction

For this chapter, we'll use two related datasets:

- **GDP\_missing\_data.csv** - Contains artificially introduced missing values
- **GDP\_complete\_data.csv** - The complete dataset without missing values

We'll use the complete dataset later to evaluate the accuracy of our imputation methods by comparing imputed values against actual values.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import sklearn as sk
import seaborn as sns

sns.set(font_scale=1.5)

%matplotlib inline
```

```
gdp_missing_values_data = pd.read_csv('./Datasets/GDP_missing_data.csv')
gdp_complete_data = pd.read_csv('./Datasets/GDP_complete_data.csv')
```

```
gdp_missing_values_data.head()
```



	economicActivityFemale	country	lifeMale	infantMortality	gdpPerCapita	economicActivityMale
0	7.2	Afghanistan	45.0	154.0	2474.0	87.5
1	7.8	Algeria	67.5	44.0	11433.0	76.4
2	41.3	Argentina	69.6	22.0	NaN	76.2
3	52.0	Armenia	67.2	25.0	13638.0	65.0
4	53.8	Australia	NaN	6.0	54891.0	NaN

Observe that the `gdp_missing_values_data` dataset consists of some missing values shown as NaN (Not a Number).

## 13.2.2 Identifying missing values in a dataframe

There are multiple ways to identify missing values in a dataframe

### 13.2.2.1 `describe()` Method

Note that the descriptive statistics methods associated with Pandas objects ignore missing values by default. Consider the summary statistics of `gdp_missing_values_data`:

```
gdp_missing_values_data.describe()
```

	economicActivityFemale	lifeMale	infantMortality	gdpPerCapita	economicActivityMale	illit
count	145.000000	145.000000	145.000000	145.000000	145.000000	145.000000
mean	45.935172	65.491724	37.158621	24193.482759	76.563448	13.3
std	16.875922	9.099256	34.465699	22748.764444	7.854730	16.4
min	1.900000	36.000000	3.000000	772.000000	51.200000	0.00
25%	35.500000	62.900000	10.000000	6837.000000	72.000000	1.00
50%	47.600000	67.800000	24.000000	15184.000000	77.300000	6.60
75%	55.900000	72.400000	54.000000	35957.000000	81.600000	19.5
max	90.600000	77.400000	169.000000	122740.000000	93.000000	70.5

Observe that the `count` statistics report the number of non-missing values of each column in the data, as the number of rows in the data (see code below) is more than the number of non-missing values of all the variables in the above table. Similarly, for the rest of the statistics, such as `mean`, `std`, etc., the missing values are ignored.

```
#The dataset gdp_missing_values_data has 155 rows
gdp_missing_values_data.shape[0]
```

155

### 13.2.2.2 info() Method

Shows the count of non-null entries in each column, helping you quickly identify columns with missing values.

```
gdp_missing_values_data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 155 entries, 0 to 154
Data columns (total 12 columns):
 # Column Non-Null Count Dtype
--- -
 0 economicActivityFemale 145 non-null float64
 1 country 155 non-null object
 2 lifeMale 145 non-null float64
 3 infantMortality 145 non-null float64
 4 gdpPerCapita 145 non-null float64
 5 economicActivityMale 145 non-null float64
 6 illiteracyMale 145 non-null float64
 7 illiteracyFemale 145 non-null float64
 8 lifeFemale 145 non-null float64
 9 geographic_location 155 non-null object
10 contraception 84 non-null float64
11 continent 155 non-null object
dtypes: float64(9), object(3)
memory usage: 14.7+ KB
```

### 13.2.2.3 isnull() Method

This is one of the most direct methods. Using `df.isnull()` returns a DataFrame of Boolean values where `True` indicates a missing value. To get a summary, you can use `df.isnull().sum()` to see the count of missing values in each column.

For finding the number of missing values in each column of `gdp_missing_values_data`, we will sum up the missing values in each column of the dataset:

```
gdp_missing_values_data.isnull().sum()
```

```
economicActivityFemale 10
country 0
lifeMale 10
infantMortality 10
gdpPerCapita 10
economicActivityMale 10
illiteracyMale 10
illiteracyFemale 10
lifeFemale 10
geographic_location 0
contraception 71
continent 0
dtype: int64
```

### 13.2.3 Types of Missing Values

In data science, missing values typically fall into three main types, each requiring different handling strategies:

#### 13.2.3.1 Missing Completely at Random (MCAR)

- **Definition:** Missing values are entirely independent of any variables in the dataset.
- **Example:** A respondent accidentally skips a question on a survey.
- **Impact:** MCAR data can usually be ignored or imputed without biasing the analysis.
- **Handling:** Simple imputation methods, like filling with mean or median values, are often appropriate.

#### 13.2.3.2 Missing at Random (MAR)

- **Definition:** The likelihood of a value being missing is related to other observed variables but not to the missing data itself.
- **Example:** People with higher incomes may be less likely to report their spending, but income data itself is not missing.
- **Impact:** Ignoring MAR values may bias results, so careful imputation based on related variables is recommended.
- **Handling:** More complex imputation methods, like conditional mean imputation or predictive modeling, are suitable.

### 13.2.3.3 Missing Not at Random (MNAR)

- **Definition:** The probability of missingness is related to the missing data itself, meaning the value is systematically missing.
- **Example:** Patients with severe health conditions might be less likely to report their health status, or students with low scores may be less likely to submit their grades.
- **Impact:** MNAR is the most challenging type, as missing values may introduce significant bias.
- **Handling:** Solutions often include sensitivity analysis, data augmentation, or modeling techniques that account for the missing mechanism, though sometimes domain-specific approaches are necessary.

Understanding the type of missing data helps in selecting the right imputation method and mitigating potential biases in the analysis.

### 13.2.3.4 Questions

#### 13.2.3.4.1

Why can we ignore observations with missing values without risking skewing the analysis or trends in the case of **Missing Completely at Random (MCAR)**?

#### 13.2.3.4.2

Why could ignoring missing values lead to biased results for **Missing at Random (MAR)** and **Missing Not at Random (MNAR)** data?

#### 13.2.3.4.3

For the dataset consisting of GDP per capita, think of hypothetical scenarios in which the missing values of GDP per capita can correspond to **MCAR** / **MAR** / **MNAR**.

## 13.2.4 Methods for Handling Missing Values

### 13.2.4.1 Removing Missing Values

- **Row/Column Removal** — use `df.dropna()`
  - **When to use:** If the missing values are few or the affected rows/columns are not critical.
  - **Risk:** May reduce the dataset's size, potentially removing useful information.

Let's drop all rows that contain any missing value from `gdp_missing_values_data`.

```
gdp_no_missing_data = gdp_missing_values_data.dropna()
```

By default, `df.dropna()` will drop any row that contains at least one missing value, keeping only the rows that are completely free of missing values.

```
#Shape of gdp_no_missing_data
gdp_no_missing_data.shape
```

```
(42, 12)
```

#### 13.2.4.2 Limitations of Using `dropna()` Directly

Using `df.dropna()` to remove rows with missing values can sometimes lead to a **significant reduction in data**, which can be problematic if much of the data is valuable and non-missing.

##### 13.2.4.2.1 Example

- **Impact of Default Behavior:**

Dropping rows with even a single missing value reduced the number of rows from **155 to 42!**

This drastic reduction occurs because, by default, `dropna()` removes any row containing at least one missing value, keeping only rows that are completely complete.

- **Loss of Non-Missing Data:**

Even if some columns have only a few missing values (e.g., at most 10), using `dropna()` without modification causes all non-missing data in those rows to be lost.

This is usually a poor choice—valuable, non-missing data may be removed unnecessarily.

#### 13.2.4.3 Adjusting `dropna()` Behavior

To avoid losing too much data, you can modify how `dropna()` operates using its parameters:

- **how parameter**

- Controls the criteria for dropping rows or columns.
- `how='any'` (*default*): Drops a row or column if **any** value is missing.
- `how='all'`: Drops a row or column **only if all** values are missing.

- **thresh parameter**

- Specifies the minimum number of **non-missing** values required to retain a row or column.
- Useful when you want to keep rows or columns that are mostly complete, even if a few values are missing.

If only a few values in a column are missing, we can often estimate them using the remaining data to **maximize the information** retained in the dataset. However, if most of a column's values are missing, it becomes much more difficult to reliably estimate or impute those values.

```
Calculate percentage of missing values per column
missing_percent = gdp_missing_values_data.isna().mean() * 100

Display results sorted by percentage (optional)
missing_percent.sort_values(ascending=False)
```

```
contraception 45.806452
economicActivityFemale 6.451613
lifeMale 6.451613
infantMortality 6.451613
gdpPerCapita 6.451613
economicActivityMale 6.451613
illiteracyMale 6.451613
illiteracyFemale 6.451613
lifeFemale 6.451613
country 0.000000
geographic_location 0.000000
continent 0.000000
dtype: float64
```

In this dataset, about **50% of the values** in the `contraception` column are missing. Therefore, we'll drop this column, since imputing it would be unreliable given the limited number of non-missing observations.

```
#Deleting column with missing values in almost half of the observations
gdp_missing_values_data.drop(['contraception'],axis=1,inplace=True)
gdp_missing_values_data.head()
```

	economicActivityFemale	country	lifeMale	infantMortality	gdpPerCapita	economicActivityMale
0	7.2	Afghanistan	45.0	154.0	2474.0	87.5
1	7.8	Algeria	67.5	44.0	11433.0	76.4
2	41.3	Argentina	69.6	22.0	NaN	76.2
3	52.0	Armenia	67.2	25.0	13638.0	65.0
4	53.8	Australia	NaN	6.0	54891.0	NaN

For the remaining columns, only about **6% of the values** are missing. In this case, instead of dropping rows or columns, it's often better to **impute** (i.e., fill in) the missing values using the available data.

Let's explore several methods for imputing missing values and how they can help us preserve as much information as possible from the dataset.

### 13.2.5 Imputing Missing Values

There are many ways to impute missing values, ranging from simple replacements to complex model-based techniques. Some common approaches are described in the [Pandas documentation](#).

The **best imputation strategy** depends on the **mechanism of missingness**:

- **MCAR (Missing Completely at Random)** — Simple methods (e.g., mean or median imputation) are generally acceptable.
- **MAR (Missing at Random)** — Imputation should account for relationships among variables, such as using conditional means or regression-based methods.
- **MNAR (Missing Not at Random)** — Imputation is challenging and often requires domain expertise to avoid introducing bias.

#### 13.2.5.1 Evaluation Approach

We will apply several imputation techniques to `gdp_missing_values_data` and evaluate their effectiveness by:

1. Comparing imputed `gdpPerCapita` values against actual values from `gdp_complete_data`
2. Computing the **Root Mean Square Error (RMSE)** for each method
3. Visualizing the relationship between imputed and actual values

First, let's define a helper function to visualize and evaluate our imputation results:

```
Index of rows with missing values for GDP per capita
null_ind_gdpPC = gdp_missing_values_data.index[gdp_missing_values_data.gdpPerCapita.isnull()]

Define a function to plot and evaluate imputed values vs actual values
def plot_actual_vs_predicted(y, title_suffix=""):
 """
 Plot imputed vs actual GDP per capita values and display RMSE.

 Parameters:
 - y: DataFrame with imputed gdpPerCapita values
 - title_suffix: Optional string to add to the plot (e.g., method name)
 """
 fig, ax = plt.subplots(figsize=(8, 6))

 # Extract actual and imputed values
 x = gdp_complete_data.loc[null_ind_gdpPC, 'gdpPerCapita']
 y_imputed = y.loc[null_ind_gdpPC, 'gdpPerCapita']

 # Create scatter plot
 ax.scatter(x, y_imputed, alpha=0.6, s=50)

 # Add perfect prediction line (45-degree line)
 ax.plot(x, x, color='orange', linewidth=2, label='Perfect imputation')

 # Labels and formatting
 ax.set_xlabel('Actual GDP per capita', fontsize=14)
 ax.set_ylabel('Imputed GDP per capita', fontsize=14)
 ax.xaxis.set_major_formatter('${x:,.0f}')
 ax.yaxis.set_major_formatter('${x:,.0f}')
 ax.tick_params(labelsize=12)
 ax.grid(True, alpha=0.3)
 ax.legend(fontsize=11)

 # Calculate and display RMSE
 rmse = np.sqrt(np.mean((y_imputed - x)**2))

 # Position text dynamically based on data range
 x_pos = x.min() + (x.max() - x.min()) * 0.05
 y_pos = y_imputed.max() - (y_imputed.max() - y_imputed.min()) * 0.1
 ax.text(x_pos, y_pos, f'RMSE = ${rmse:,.2f}',
 fontsize=13, bbox=dict(boxstyle='round', facecolor='wheat', alpha=0.5))
```



```
plt.tight_layout()
plt.show()

return rmse
```

### 13.2.5.2 Method 1: Mean/Median Imputation

**Simple imputation** replaces missing values with a statistical measure of central tendency from the non-missing values in the same column.

- **Mean imputation** — Best for normally distributed data without outliers (MCAR scenarios)
- **Median imputation** — Better for skewed data or when outliers are present
- **Mode imputation** — Appropriate for categorical variables

**Advantages:** - Simple and fast to implement - Works well for MCAR data

**Disadvantages:** - Reduces variance in the data - Ignores relationships between variables - May introduce bias for MAR or MNAR data

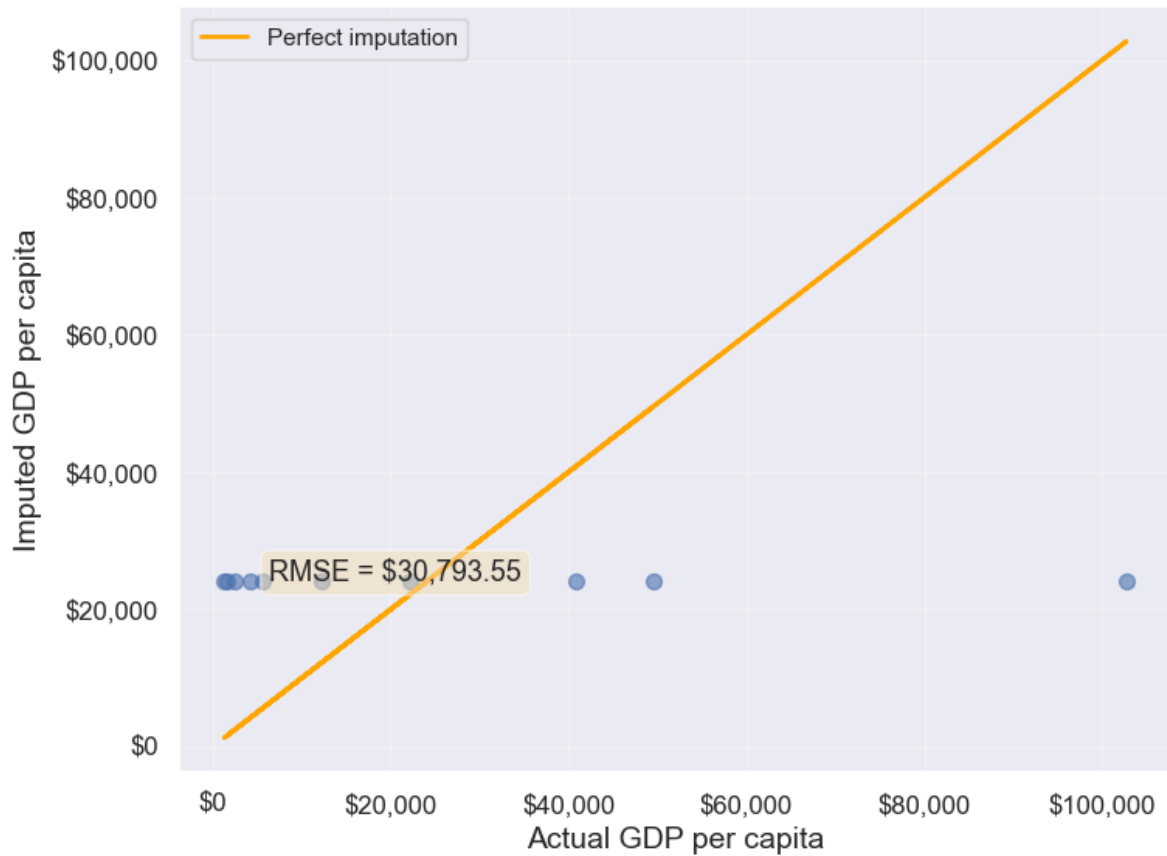
Let's impute missing `gdpPerCapita` values using the mean of non-missing values:

```
Extracting the columns with missing values
columns_with_missing = [col for col in gdp_missing_values_data.columns if gdp_missing_values_data[col].isnull().any()]
columns_with_missing
```

```
['economicActivityFemale',
 'lifeMale',
 'infantMortality',
 'gdpPerCapita',
 'economicActivityMale',
 'illiteracyMale',
 'illiteracyFemale',
 'lifeFemale']
```

```
Imputing missing values using the mean
gdp_imputed_data_mean = gdp_missing_values_data[columns_with_missing].fillna(gdp_missing_values_data[columns_with_missing].mean())
```

```
plot_actual_vs_predicted(gdp_imputed_data_mean)
```



30793.549983587087

**Observation:** The RMSE value indicates how well the mean imputation performed.

**Note on data types:** - **Numerical data:** Use mean or median - **Categorical data:** Use mode (the most frequently occurring value)

Since all columns with missing values in this dataset are numerical, mean imputation is appropriate. For categorical columns, you would use `fillna(df['column'].mode()[0])`.

### 13.2.5.3 Method 2: Conditional Imputation

**Conditional imputation** uses relationships between variables to predict missing values more accurately than simple mean/median imputation. This approach is better suited for **MAR** (Missing at Random) data.

We'll explore three conditional imputation techniques:

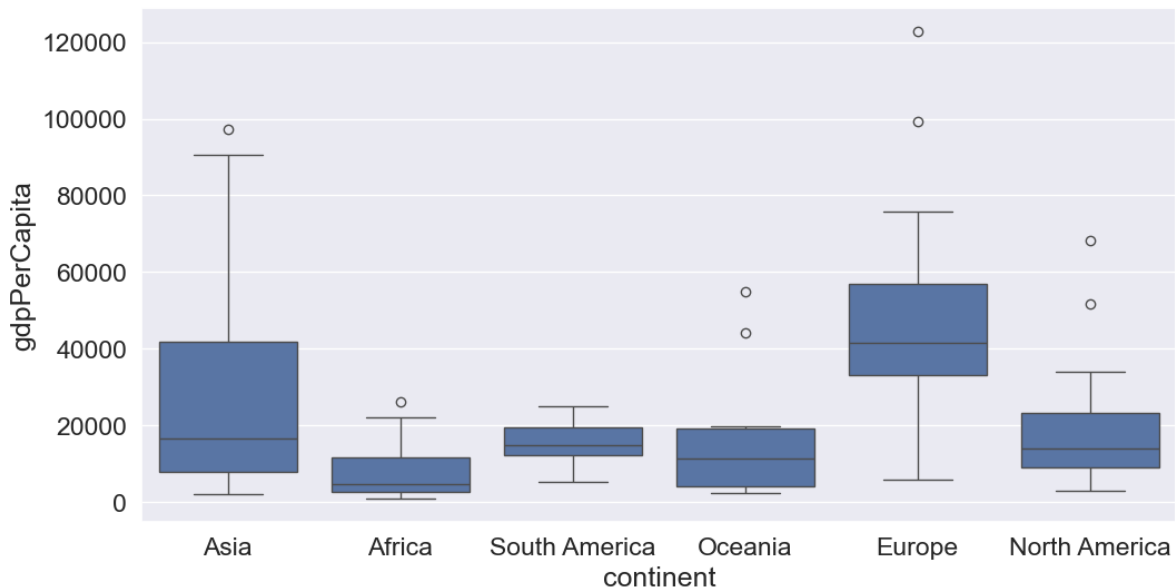
### 13.2.5.3.1 A. Group-based Imputation (Conditional Mean)

**Idea:** Impute missing values using the mean of a **subgroup** rather than the overall mean.

If categorical variables show distinct patterns (e.g., GDP varies significantly by continent), we can impute missing values using the mean of the corresponding group. This is more accurate than using the global mean because it accounts for systematic differences between groups.

Let us visualize the distribution of GDP per capita for different continents.

```
plt.rcParams["figure.figsize"] = (12,6)
sns.boxplot(x = 'continent',y='gdpPerCapita',data = gdp_missing_values_data);
```



We observe that there is a distinct difference between the GDPs per capita of some of the continents. Let us impute the missing GDP per capita of a country as the mean GDP per capita of the corresponding continent. This imputation should be better than imputing the missing GDP per capita as the mean of all the non-missing values, as the GDP per capita of a country is likely to be closer to the mean GDP per capita of the continent, rather than the mean GDP per capita of the whole world.

```
#Finding the mean GDP per capita of the continent - please defer the understanding of this code
avg_gdpPerCapita = gdp_missing_values_data['gdpPerCapita'].groupby(gdp_missing_values_data['continent']).agg('mean')
avg_gdpPerCapita
```

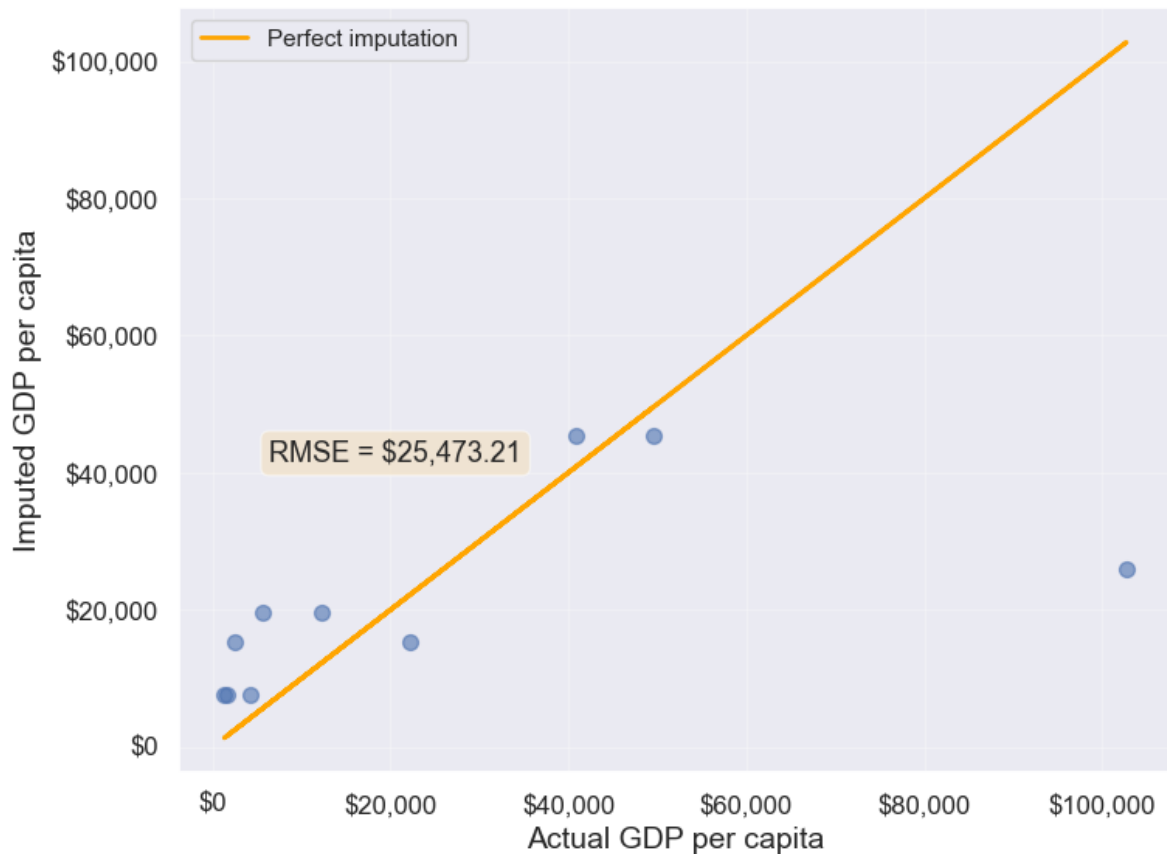
```
continent
Africa 7638.178571
```

```
Asia 25922.750000
Europe 45455.303030
North America 19625.210526
Oceania 15385.857143
South America 15360.909091
Name: gdpPerCapita, dtype: float64
```

```
#Creating a copy of missing data to impute missing values
gdp_imputed_data_group_mean = gdp_missing_values_data.copy()
```

```
#Replacing missing GDP per capita with the mean GDP per capita for the corresponding continent
```

```
gdp_imputed_data_group_mean.gdpPerCapita = \
gdp_imputed_data_group_mean['gdpPerCapita'].fillna(gdp_imputed_data_group_mean['continent'].r
plot_actual_vs_predicted(gdp_imputed_data_group_mean)
```



```
25473.20645170116
```

Note that the imputed values are closer to the actual values, and the RMSE has further reduced as expected.

Suppose we wish to impute the missing values of each numeric column with the average of the non-missing values of the respective column corresponding to the continent of the observation. The above logic can be extended to each column as shown in the code below.

```
all_columns_imputed_data = gdp_missing_values_data.iloc[:, [0,2,3,4,5,6,7,8]].apply(lambda x:
x.fillna(gdp_imputed_data_group_mean['continent']).map(x.groupby(gdp_missing_values_data['con
```

### 13.2.5.3.2 B. Regression-based Imputation

**Idea:** Use **linear regression** to predict missing values based on their relationship with other variables.

If a variable is highly correlated with another variable in the dataset, we can build a regression model to predict missing values. This method leverages the statistical relationship between variables.

```
#Let us identify the variable highly correlated with GDP per capita.
gdp_missing_values_data.select_dtypes(include='number').corrwith(gdp_missing_values_data.gdpPerCapita)
```

```
economicActivityFemale 0.078332
lifeMale 0.579850
infantMortality -0.572201
gdpPerCapita 1.000000
economicActivityMale -0.134108
illiteracyMale -0.479143
illiteracyFemale -0.448273
lifeFemale 0.615954
dtype: float64
```

```
#The variable *lifeFemale* has the strongest correlation with GDP per capita. Let us use it to
```

```
Extract the variables lifeFemale and GDP per capita
x = gdp_missing_values_data.lifeFemale
y = gdp_missing_values_data.gdpPerCapita
```

```
Identify non-missing indices
idx_non_missing = np.isfinite(x) & np.isfinite(y)
```

```
Fit a linear regression model (degree=1) to predict GDP per capita given lifeFemale
```

```

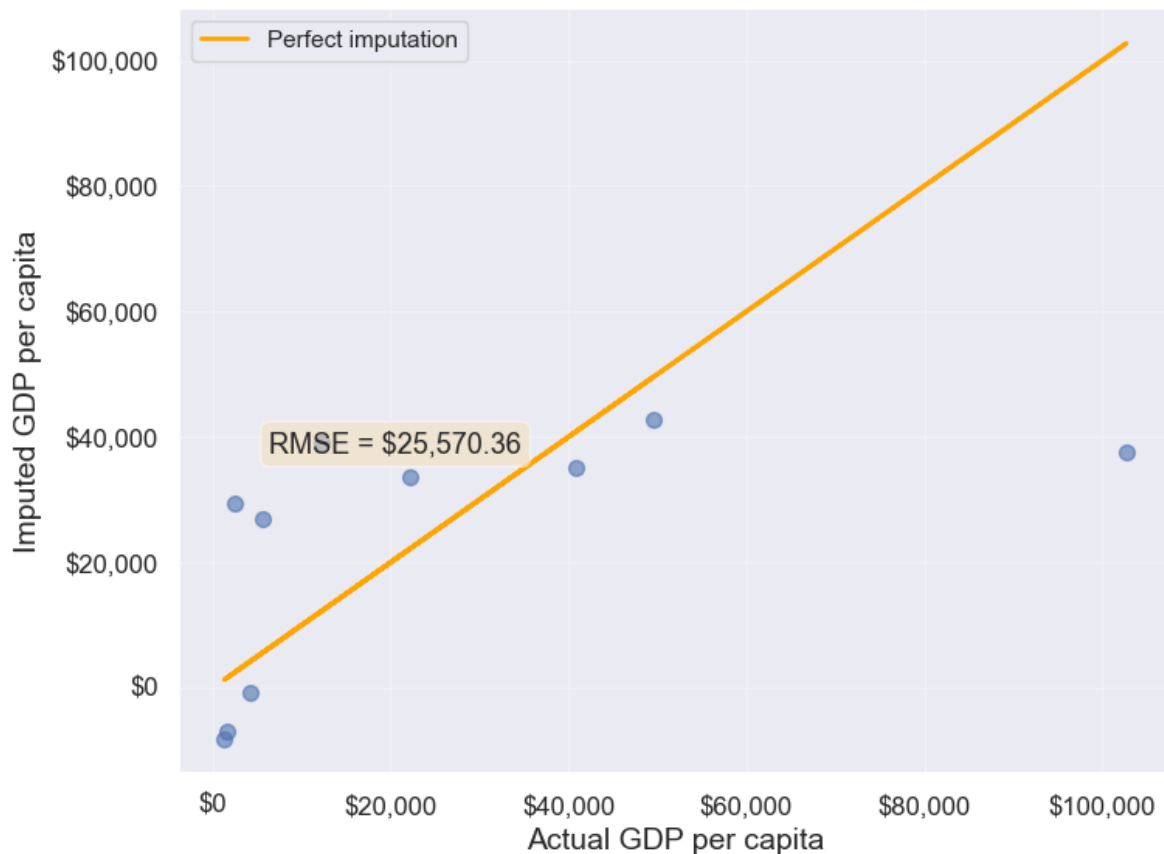
slope_intercept_trendline = np.polyfit(x[idx_non_missing],y[idx_non_missing],1) #Finding the slope and intercept
compute_y_given_x = np.poly1d(slope_intercept_trendline)

#Creating a copy of missing data to impute missing values
gdp_imputed_data_lr = gdp_missing_values_data.copy()

#Imputing missing values of GDP per capita using the linear regression model
gdp_imputed_data_lr.loc[null_ind_gdpPC,'gdpPerCapita']=compute_y_given_x(gdp_missing_values_data.loc[null_ind_gdpPC,'ActualGDPPerCapita'])

plot_actual_vs_predicted(gdp_imputed_data_lr)

```



25570.361516956993

#### 13.2.5.3.3 C. K-Nearest Neighbors (KNN) Imputation

**Idea:** Impute missing values using the **mean of K most similar observations** (nearest neighbors).

KNN finds observations that are “similar” based on Euclidean distance calculated from other variables. This method is more sophisticated and often produces better results than simple mean imputation because it considers the overall similarity between observations.

In this method, we’ll impute the missing value of the variable as the mean value of the  $K$ -nearest neighbors having non-missing values for that variable. The neighbors to a data-point are identified based on their Euclidean distance to the point in terms of the standardized values of rest of the variables in the data.

Let’s consider a toy example to understand missing value imputation by KNN. Suppose we have to impute missing values in a toy dataset, named as `toy_data` having 4 observations and 3 variables.

```
#Toy example - A 4x3 array with missing values
nan = np.nan
toy_data = np.array([[1, 2, nan], [3, 4, 3], [nan, 6, 5], [8, 8, 7]])
toy_data
```

```
array([[1., 2., nan],
 [3., 4., 3.],
 [nan, 6., 5.],
 [8., 8., 7.]])
```

We’ll use some functions from the *sklearn* library to perform the KNN imputation. It is much easier to directly use the algorithm from *sklearn*, instead of coding it from scratch.

```
#Library to compute pair-wise Euclidean distance between all observations in the data
from sklearn import metrics

#Library to impute missing values with the KNN algorithm
from sklearn import impute
```

We’ll use the *sklearn* function `nan_euclidean_distances()` to compute the Euclidean distance between all pairs of observations in the data.

```
#This is the distance matrix containing the distance of the ith observation from the jth obs
metrics.pairwise.nan_euclidean_distances(toy_data,toy_data)
```

```
array([[0. , 3.46410162, 6.92820323, 11.29158979],
 [3.46410162, 0. , 3.46410162, 7.54983444],
 [6.92820323, 3.46410162, 0. , 3.46410162],
 [11.29158979, 7.54983444, 3.46410162, 0.]])
```

Note that the size of the above matrix is 4x4. This is because the  $(i, j)^{th}$  element of the matrix is the distance of the  $i^{th}$  observation from the  $j^{th}$  observation. The matrix is symmetric because the distance of  $i^{th}$  observation to the  $j^{th}$  observation is the same as the distance of the  $j^{th}$  observation to the  $i^{th}$  observation.

We'll use the *sklearn* function `KNNImputer()` to impute the missing value of a column in `toy_data` as the mean of the values of the  $K$  nearest neighbors to the observation that have non-missing values for that column.

Let us impute the missing values in `toy_data` using the values of  $K = 2$  nearest neighbors from the corresponding observation.

```
#imputing missing values with 2 nearest neighbors, where the neighbors have equal weights

#Define an object of type KNNImputer
imputer = impute.KNNImputer(n_neighbors=2)

#Use the object method 'fit_transform' to impute missing values
imputer.fit_transform(toy_data)
```

```
array([[1. , 2. , 4.],
 [3. , 4. , 3.],
 [5.5, 6. , 5.],
 [8. , 8. , 7.]])
```

The third observation was the closest to the 2nd and 4th observations based on the Euclidean distance matrix. Thus, the missing value in the 3rd row of the `toy_data` has been imputed as the mean of the values in the 2nd and 4th observations for the corresponding column. Similarly, the 1st observation is the closest to the 2nd and 3rd observations. Thus the missing value in the 1st row of `toy_data` has been imputed as the mean of the values in the 1st and 2nd observations for the corresponding column.

Let us use KNN to impute the missing values of `gdpPerCapita` in `gdp_missing_values_data`. We'll use only the numeric columns of the data in imputing the missing values. Also, we'll ignore `contraception` as it has a lot of missing values, and thus may not be useful.



```
#Considering numeric columns in the data to use KNN
num_cols = list(range(0,1))+list(range(2,9))
num_cols
```

```
[0, 2, 3, 4, 5, 6, 7, 8]
```

Before computing the pair-wise Euclidean distance of observations, we must standardize the data so that all columns are at the same scale. This will avoid columns with a higher magnitude of values having a higher weight in determining the Euclidean distance. Unless there is a reason to give a higher weight to a column, we assume all columns to have the same weight in the Euclidean distance computation.

We can use the code below to scale the data. However, after imputing the missing values, the data is to be scaled back to the original scale, so that each variable is in the same units as in the original dataset. However, if the code below is used, we'll lose the original scale of each of the columns.

```
#Scaling data to compute equally weighted distances from the 'k' nearest neighbors
scaled_data = gdp_missing_values_data.iloc[:,num_cols].apply(lambda x:(x-x.min())/(x.max()-x
```

To alleviate the problem of losing the original scale of the data, we'll use the [MinMaxScaler](#) object of the *sklearn* library. The object will store the original scale of the data, which will help transform the data back to the original scale once the missing values have been imputed in the standardized data.

```
Scaling data - using sklearn

#Create an object of type MinMaxScaler
scaler = sk.preprocessing.MinMaxScaler()

#Use the object method 'fit_transform' to scale the values to a standard uniform distribution
scaled_data = pd.DataFrame(scaler.fit_transform(gdp_missing_values_data.iloc[:,num_cols]))
```

```
#Imputing missing values with KNNImputer

#Define an object of type KNNImputer
imputer = impute.KNNImputer(n_neighbors=3, weights="uniform")

#Use the object method 'fit_transform' to impute missing values
imputed_arr = imputer.fit_transform(scaled_data)
```

```
#Scaling back the scaled array to obtain the data at the original scale
```

```
#Use the object method 'inverse_transform' to scale back the values to the original scale of
unscaled_data = scaler.inverse_transform(imputed_arr)
```

```
#Note the method imputes the missing value of all the columns
```

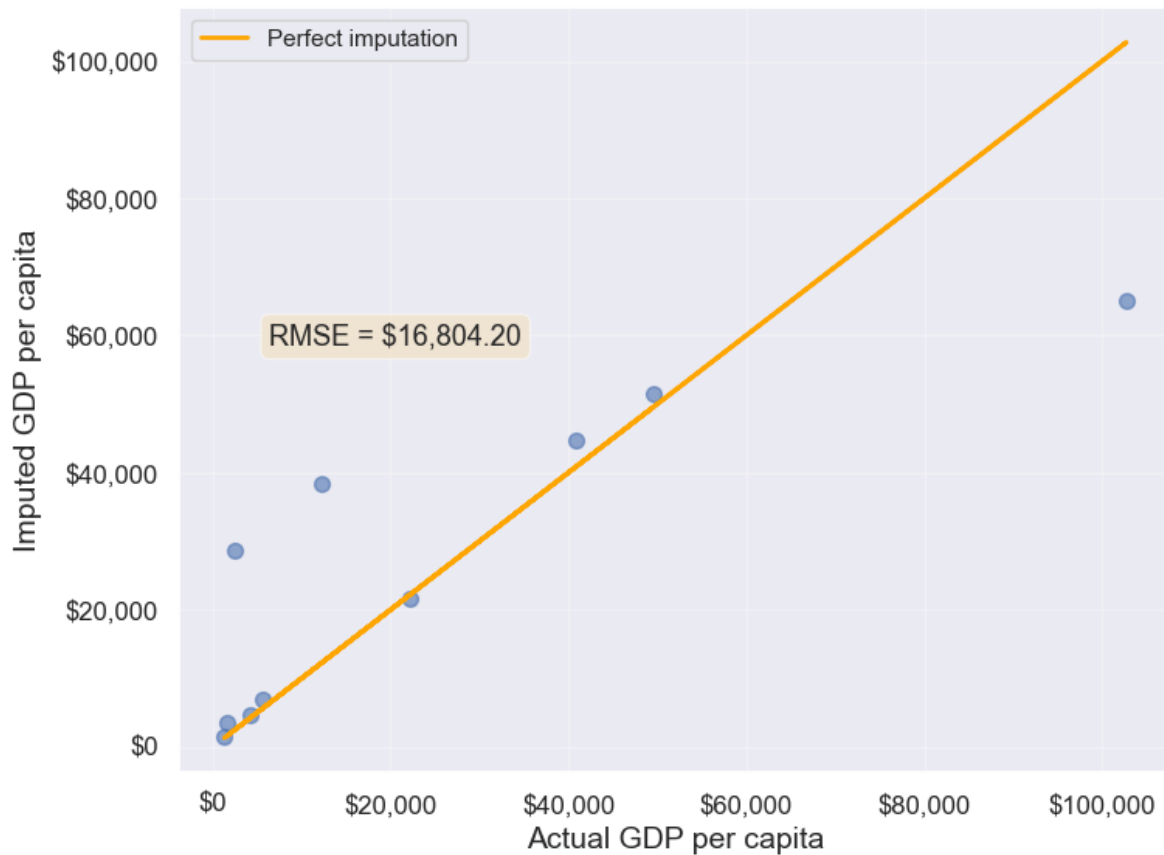
```
#However, we are interested in imputing the missing values of only the 'gdpPerCapita' column
```

```
gdp_imputed_data_knn = gdp_missing_values_data.copy()
```

```
gdp_imputed_data_knn.loc[:, 'gdpPerCapita'] = unscaled_data[:, 3]
```

```
#Visualizing the accuracy of missing value imputation with KNN
```

```
plot_actual_vs_predicted(gdp_imputed_data_knn)
```



16804.195967740387

Note that the RMSE is the lowest in this method. It is because this method imputes missing values as the average of the values of “similar” observations, which is smarter and more robust than the previous methods.

We chose  $K = 3$  in the missing value imputation for GDP per capita. However, the value of  $K$  is typically chosen using a method known as cross validation. We’ll learn about cross-validation in the next course of the sequence.

#### 13.2.5.4 Method 3: Forward Fill / Backward Fill

**Idea:** Fill missing values by propagating the **previous** (forward fill) or **next** (backward fill) non-missing value.

**When to use:** - Time series data where values change gradually - Data sorted in a meaningful order - Assumes temporal or sequential dependency

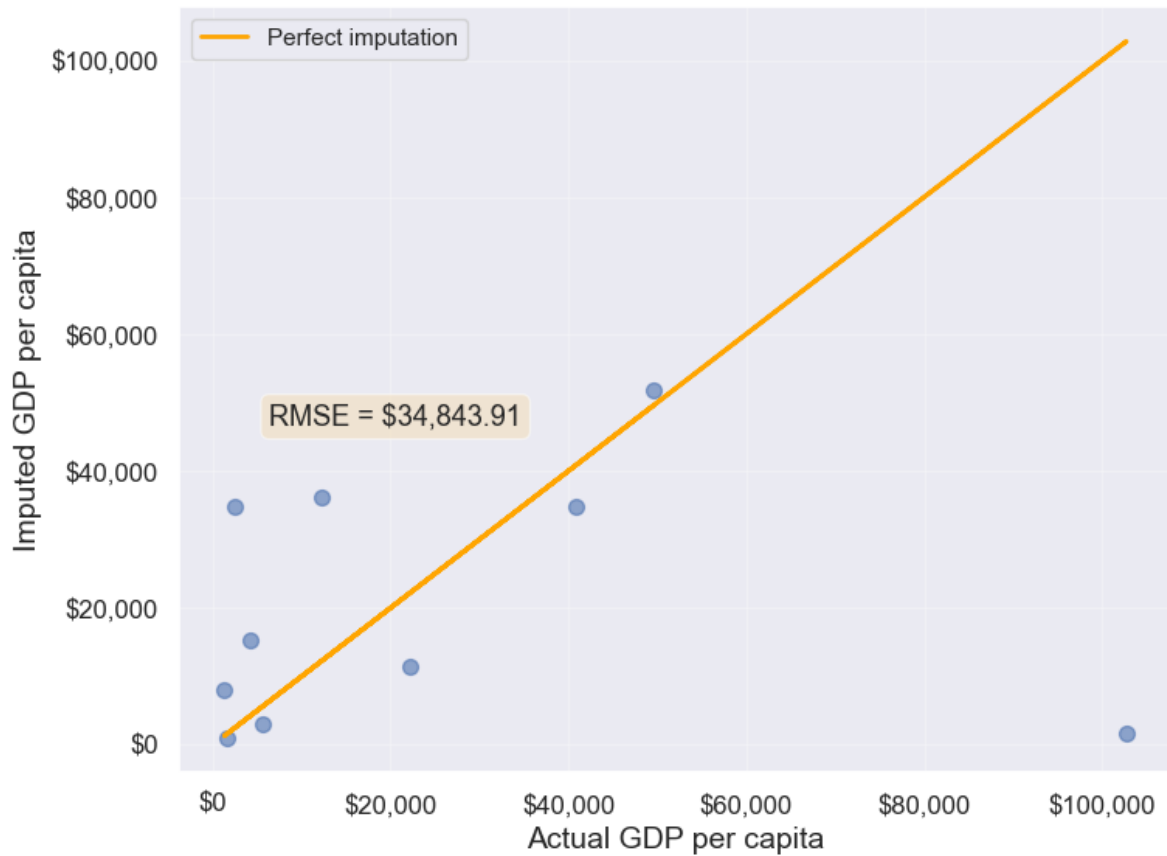
**Limitations:** - Inappropriate for cross-sectional data (like our GDP dataset) - Assumes the missing value should be similar to adjacent observations - Can introduce bias if data order is arbitrary

Let’s demonstrate forward fill on our dataset (though it’s not ideal for this use case):

```
#Filling missing values: Method 1- Naive way
gdp_imputed_data_ffill = gdp_missing_values_data.ffill()
```

Let us next check how good is this method in imputing missing values.

```
plot_actual_vs_predicted(gdp_imputed_data_ffill)
```



34843.91091137732

We observe that the accuracy of imputation is poor as GDP per capita can vary a lot across countries, and the data is not sorted by GDP per capita. There is no reason why the GDP per capita of a country should be close to the GDP per capita of the country in the observation above it.

```
#Checking if any missing values are remaining
gdp_imputed_data_ffill.isnull().sum()
```

```
economicActivityFemale 0
country 0
lifeMale 0
infantMortality 0
gdpPerCapita 0
economicActivityMale 0
```

```
illiteracyMale 1
illiteracyFemale 0
lifeFemale 0
geographic_location 0
continent 0
dtype: int64
```

After imputing missing values, note there is still one missing value for *illiteracyMale*. Can you guess why one missing value remained?

## 13.3 Dealing with Duplicate Records

Duplicate records can occur in datasets for various reasons: - Data entry errors (same record entered multiple times) - Merging datasets from different sources - System errors during data collection - Intentional duplicates that need investigation

Fortunately, detecting and removing duplicates is relatively straightforward in pandas. Let's explore the methods for identifying and handling duplicate records.

### 13.3.1 Creating a Sample Dataset with Duplicates

First, let's manually create a dataset that contains duplicate records to demonstrate the various duplicate detection and removal techniques.

```
Create a sample dataframe with duplicate records
student_data = pd.DataFrame({
 'StudentID': [101, 102, 103, 101, 104, 102, 105, 103],
 'Name': ['Alice', 'Bob', 'Charlie', 'Alice', 'David', 'Bob', 'Eve', 'Charlie'],
 'Major': ['CS', 'Math', 'CS', 'CS', 'Physics', 'Math', 'Biology', 'CS'],
 'GPA': [3.8, 3.5, 3.9, 3.8, 3.7, 3.5, 3.6, 3.9]
})

print("Original dataset with duplicates:")
student_data
```

Original dataset with duplicates:

	StudentID	Name	Major	GPA
0	101	Alice	CS	3.8
1	102	Bob	Math	3.5
2	103	Charlie	CS	3.9
3	101	Alice	CS	3.8
4	104	David	Physics	3.7
5	102	Bob	Math	3.5
6	105	Eve	Biology	3.6
7	103	Charlie	CS	3.9

## 13.3.2 Identifying Duplicate Records

There are several ways to identify duplicates in a DataFrame:

### 13.3.2.1 Check How Many Duplicate Rows Exist

The `duplicated()` method returns a boolean Series indicating whether each row is a duplicate. We can sum this to count the total number of duplicate rows.

```
Count the number of duplicate rows
num_duplicates = student_data.duplicated().sum()
print(f"Number of duplicate rows: {num_duplicates}")

Check which rows are duplicates (returns boolean Series)
print("\nDuplicate flags for each row:")
student_data.duplicated()
```

Number of duplicate rows: 3

Duplicate flags for each row:

```
0 False
1 False
2 False
3 True
4 False
5 True
6 False
7 True
dtype: bool
```

**Note:** By default, `duplicated()` marks all duplicate rows as `True` **except the first occurrence**. So if a row appears 3 times, the first occurrence is marked `False` and the next 2 are marked `True`.

### 13.3.2.2 Display the Duplicated Rows

To see the actual duplicate records, we can filter the DataFrame using the boolean mask returned by `duplicated()`.

```
Display the duplicate rows (excluding the first occurrence)
print("Duplicate rows:")
student_data[student_data.duplicated()]
```

Duplicate rows:

	StudentID	Name	Major	GPA
3	101	Alice	CS	3.8
5	102	Bob	Math	3.5
7	103	Charlie	CS	3.9

To see **all occurrences** of duplicate rows (including the first occurrence), use `keep=False`:

```
Display ALL occurrences of duplicate rows (including first occurrence)
print("All occurrences of duplicate rows:")
student_data[student_data.duplicated(keep=False)]
```

All occurrences of duplicate rows:

	StudentID	Name	Major	GPA
0	101	Alice	CS	3.8
1	102	Bob	Math	3.5
2	103	Charlie	CS	3.9
3	101	Alice	CS	3.8
5	102	Bob	Math	3.5
7	103	Charlie	CS	3.9

### 13.3.2.3 Check Duplicates Based on Specific Columns

Sometimes you want to identify duplicates based on only certain columns, not all columns. For example, you might consider two students duplicates if they have the same `StudentID`, even if other details differ.

```
Check duplicates based on StudentID only
print("Duplicates based on StudentID:")
print(f"Number of duplicate StudentIDs: {student_data.duplicated(subset=['StudentID']).sum()}")

print("\nRows with duplicate StudentID:")
student_data[student_data.duplicated(subset=['StudentID'], keep=False)]
```

Duplicates based on StudentID:

Number of duplicate StudentIDs: 3

Rows with duplicate StudentID:

	StudentID	Name	Major	GPA
0	101	Alice	CS	3.8
1	102	Bob	Math	3.5
2	103	Charlie	CS	3.9
3	101	Alice	CS	3.8
5	102	Bob	Math	3.5
7	103	Charlie	CS	3.9

```
Check duplicates based on multiple columns (Name and Major)
print("Duplicates based on Name and Major:")
print(f"Number of duplicate Name-Major combinations: {student_data.duplicated(subset=['Name', 'Major']).sum()}")

print("\nRows with duplicate Name-Major combinations:")
student_data[student_data.duplicated(subset=['Name', 'Major'], keep=False)]
```

Duplicates based on Name and Major:

Number of duplicate Name-Major combinations: 3

Rows with duplicate Name-Major combinations:



	StudentID	Name	Major	GPA
0	101	Alice	CS	3.8
1	102	Bob	Math	3.5
2	103	Charlie	CS	3.9
3	101	Alice	CS	3.8
5	102	Bob	Math	3.5
7	103	Charlie	CS	3.9

### 13.3.3 Removing Duplicate Records

Once duplicates are identified, we can remove them using the `drop_duplicates()` method.

#### 13.3.3.1 Remove All Duplicated Rows

By default, `drop_duplicates()` keeps the **first occurrence** and removes subsequent duplicates.

```
Remove all duplicate rows (keeps first occurrence)
student_data_cleaned = student_data.drop_duplicates()

print("Dataset after removing duplicates:")
print(student_data_cleaned)

print(f"\nOriginal dataset size: {len(student_data)} rows")
print(f"Cleaned dataset size: {len(student_data_cleaned)} rows")
print(f"Removed: {len(student_data) - len(student_data_cleaned)} duplicate rows")
```

#### 13.3.3.2 Remove Duplicates Based on Specific Columns

You can specify which columns to consider when identifying duplicates using the `subset` parameter.

```
Remove duplicates based on StudentID only
student_data_unique_id = student_data.drop_duplicates(subset=['StudentID'])

print("Dataset after removing duplicates based on StudentID:")
print(student_data_unique_id)
```

```

print(f"\nOriginal dataset: {len(student_data)} rows")
print(f"After removing duplicate StudentIDs: {len(student_data_unique_id)} rows")

Remove duplicates based on Name and Major
student_data_unique_name_major = student_data.drop_duplicates(subset=['Name', 'Major'])

print("Dataset after removing duplicates based on Name and Major:")
print(student_data_unique_name_major)

print(f"\nOriginal dataset: {len(student_data)} rows")
print(f"After removing duplicate Name-Major combinations: {len(student_data_unique_name_major)} rows")

```

### 13.3.3.3 Keep First or Last Occurrence

The `keep` parameter controls which occurrence to keep: - `keep='first'` (*default*): Keep the first occurrence, remove subsequent duplicates - `keep='last'`: Keep the last occurrence, remove earlier duplicates - `keep=False`: Remove all duplicates (including all occurrences)

```

Keep the first occurrence (default behavior)
keep_first = student_data.drop_duplicates(keep='first')
print("Keep first occurrence:")
print(keep_first)
print()

```

```

Keep the last occurrence
keep_last = student_data.drop_duplicates(keep='last')
print("Keep last occurrence:")
print(keep_last)
print()

```

```

Remove ALL occurrences of duplicates (keep none)
keep_none = student_data.drop_duplicates(keep=False)
print("Remove all occurrences of duplicates:")
print(keep_none)

print(f"\nOriginal dataset: {len(student_data)} rows")
print(f"After removing ALL duplicate occurrences: {len(keep_none)} rows")
print("Note: Only rows that appear exactly once are kept!")

```

### 13.3.4 Applying to Real Data

Now let's check for duplicates in our GDP dataset:

```
Check for duplicate records in GDP dataset
num_duplicates_gdp = gdp_imputed_data_ffill.duplicated().sum()
print(f"Number of duplicate rows in GDP dataset: {num_duplicates_gdp}")

if num_duplicates_gdp > 0:
 print("\nDuplicate rows:")
 print(gdp_imputed_data_ffill[gdp_imputed_data_ffill.duplicated(keep=False)])
else:
 print("\nNo duplicate records found in the GDP dataset!")
```

Number of duplicate rows in GDP dataset: 0

No duplicate records found in the GDP dataset!

If the dataset had duplicates, we could remove them using:

```
Remove duplicate records (if they existed)
gdp_imputed_data_ffill = gdp_imputed_data_ffill.drop_duplicates()
```

**Summary:** - Use `duplicated()` to identify duplicate rows - Use `drop_duplicates()` to remove them - The `subset` parameter lets you specify which columns to check - The `keep` parameter controls which occurrence to keep ('first', 'last', or False to remove all)

## 13.4 Outlier detection and handling

An outlier is an observation that is significantly different from the rest of the data. Outlier detection and handling are important in data science because outliers can significantly impact the quality, accuracy, and interpretability of data analysis and model performance. Here are the main reasons why it's crucial to address outliers:

- **Preventing Skewed Results:** Outliers can distort statistical measures, such as the mean and standard deviation, leading to misleading interpretations of the data. For example, a few extreme values can inflate the mean, making the data seem larger than it is in reality.

- **Improving Model Accuracy:** Many machine learning algorithms are sensitive to outliers and can perform poorly if outliers are present. For instance, in linear regression, outliers can disproportionately affect the model by pulling the line of best fit toward them, reducing predictive accuracy.
- **Enhancing Robustness of Models:** Identifying and handling outliers can make models more robust and stable. By minimizing the influence of extreme values, models become less sensitive to noise, which improves generalization and reduces overfitting.

### 13.4.1 Outlier detection

Outlier detection is crucial for identifying unusual values in your data that may need to be investigated, removed, or transformed. Here are several popular methods for detecting outliers:

We will use *College.csv* that contains information about US universities as our dataset in this section. The description of variables of the dataset can be found on page 65 of this [book](#).

```
Load the College dataset
college = pd.read_csv('./Datasets/College.csv')
college.head()
```

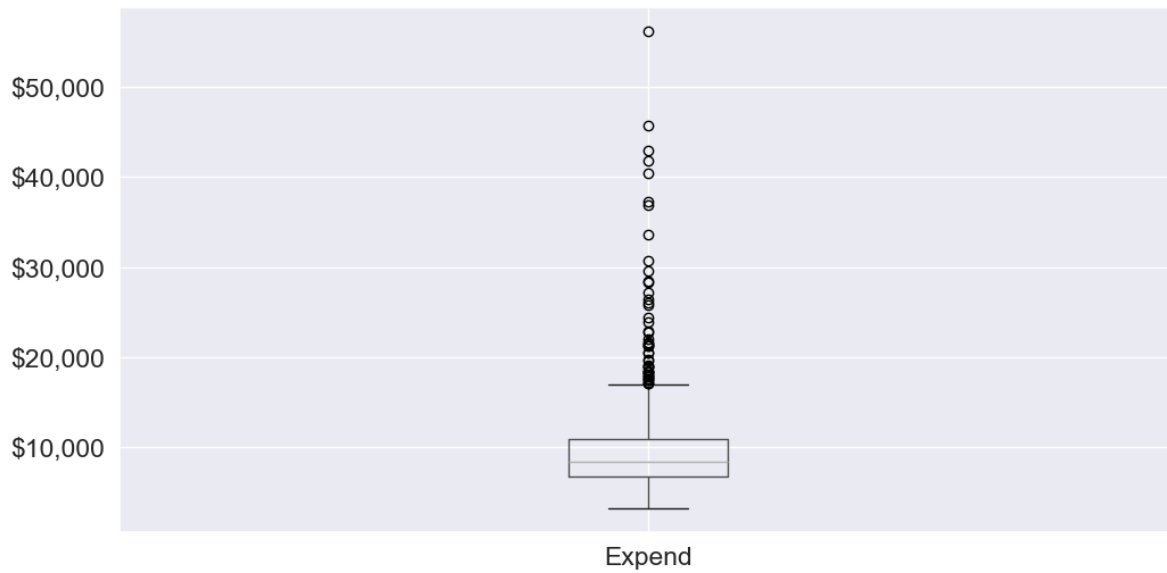
	Unnamed: 0	Private	Apps	Accept	Enroll	Top10perc	Top25perc	F.Undergrad
0	Abilene Christian University	Yes	1660	1232	721	23	52	2885
1	Adelphi University	Yes	2186	1924	512	16	29	2683
2	Adrian College	Yes	1428	1097	336	22	50	1036
3	Agnes Scott College	Yes	417	349	137	60	89	510
4	Alaska Pacific University	Yes	193	146	55	16	44	249

#### 13.4.1.1 Visualization Methods

- **Box Plot:** A box plot is a quick and visual way to detect outliers, where points outside the “whiskers” are considered outliers.

Let us visualize outliers in average instructional expenditure per student given by the variable *Expend*.

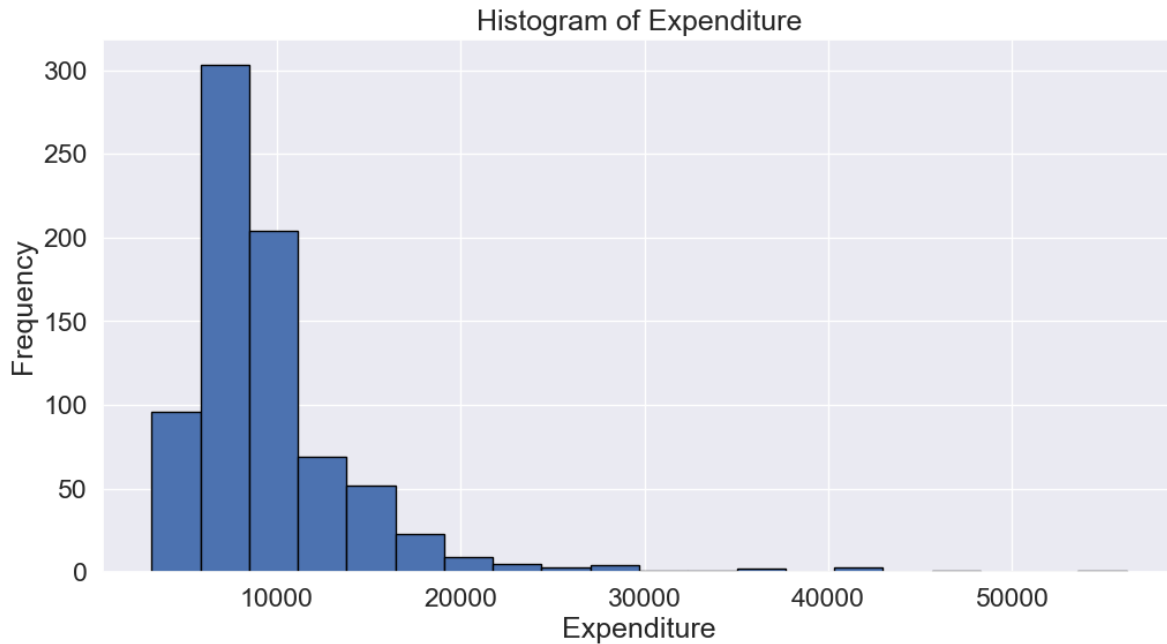
```
ax=college.boxplot(column = 'Expend')
ax.yaxis.set_major_formatter('${x:,.0f}')
```



There are several outliers (shown as circles in the above boxplot), which correspond to high values of average instructional expenditure per student.

- **Histogram:** Histograms can reveal unusual values as isolated bars, helping to identify outliers in a single feature.

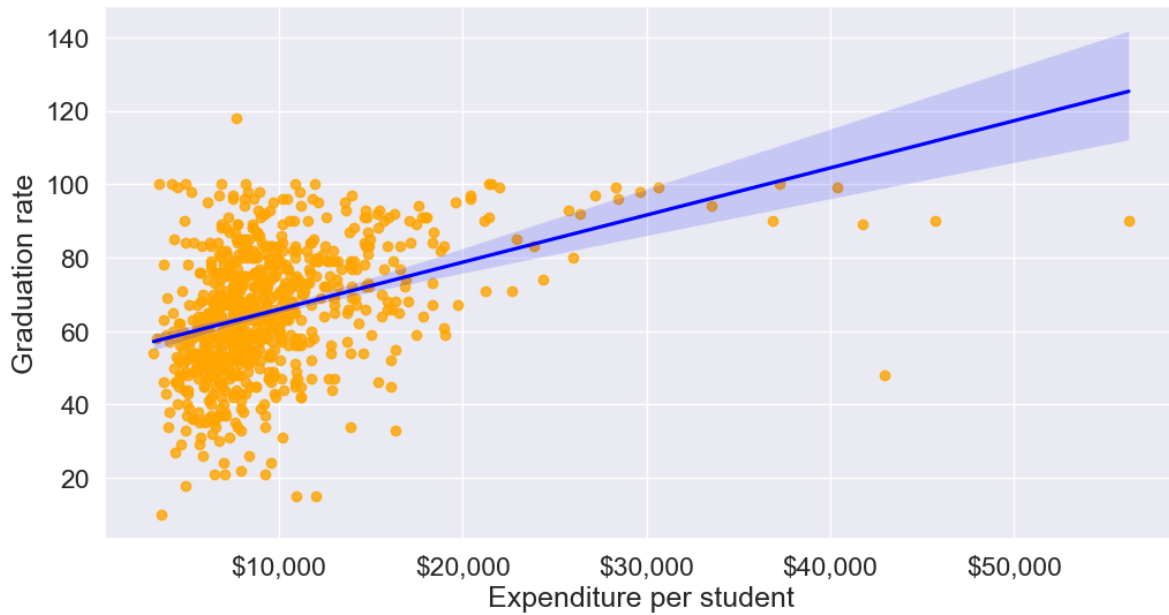
```
create a histogram of expend
college['Expend'].plot(kind='hist', edgecolor='black', bins=20)
plt.xlabel('Expenditure')
plt.ylabel('Frequency')
plt.title('Histogram of Expenditure')
plt.show()
```



- **Scatter Plot:** A scatter plot can reveal outliers, especially in two-dimensional data. Outliers will often appear as points separated from the main cluster.

Let's make a scatterplot of 'Grad.Rate' vs 'Expend' with a trendline, to visualize potential outliers

```
sns.set(font_scale=1.5)
ax=sns.regplot(data = college, x = "Expend", y = "Grad.Rate",scatter_kws={"color": "orange"})
ax.xaxis.set_major_formatter('${x:,.0f}')
ax.set_xlabel('Expenditure per student')
ax.set_ylabel('Graduation rate')
plt.show()
```



The trendline indicates a positive correlation between **Expend** and **Grad.Rate**. However, there seems to be a lot of noise and presence of outliers in the data.

#### 13.4.1.2 Statistical Methods

Visualization helps us identify potential outliers. To address them effectively, we need to extract these outlier instances using a defined threshold. Two commonly used statistical methods for this purpose are the Z-Score method and the Interquartile Range (IQR) (Tukey's fences).

##### 13.4.1.2.1 Z-Score (Standard Score)

Calculates how many standard deviations a data point is from the mean. Commonly, data points with a Z-score greater than 3 (or less than -3) are considered outliers.

```
from scipy import stats

Calculate Z-scores and identify outliers
college['z_score'] = stats.zscore(college['Expend'])
college['is_z_score_outlier'] = college['z_score'].abs() > 3

Filter to show only the outliers
z_score_outliers = college[college['is_z_score_outlier']]
print(z_score_outliers.shape)
z_score_outliers.head()
```

(16, 21)

	Unnamed: 0	Private	Apps	Accept	Enroll	Top10perc	Top25perc	F.Undergrad	P.Un
20	Antioch University	Yes	713	661	252	25	44	712	23
144	Columbia University	Yes	6756	1930	871	78	96	3376	55
158	Dartmouth College	Yes	8587	2273	1087	87	99	3918	32
174	Duke University	Yes	13789	3893	1583	90	98	6188	53
191	Emory University	Yes	8506	4168	1236	76	97	5544	192

Boxplot identifies outliers based on the [Tukey's fences](#) criterion:

#### 13.4.1.2.2 IQR method (Tukey's fences)

John Tukey proposed that observations outside the range  $[Q1 - 1.5(Q3 - Q1), Q3 + 1.5(Q3 - Q1)]$  are outliers, where  $Q1$  and  $Q3$  are the lower (25%) and upper (75%) quartiles respectively. Let us detect outliers based on this threshold.

```
Calculate the first and third quartiles, and the IQR
Q1 = np.percentile(college['Expend'], 25)
Q3 = np.percentile(college['Expend'], 75)
IQR = Q3 - Q1

Define the lower and upper bounds for outliers
lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR

Add a new column to indicate IQR outliers
college['is_IQR_outlier'] = (college['Expend'] < lower_bound) | (college['Expend'] > upper_bound)

Filter to show only the IQR outliers
iqr_outliers = college[college['is_IQR_outlier']]
print(iqr_outliers.shape)
iqr_outliers.head()
```

(48, 22)

	Unnamed: 0	Private	Apps	Accept	Enroll	Top10perc	Top25perc	F.Undergrad	P.Unde
3	Agnes Scott College	Yes	417	349	137	60	89	510	63



	Unnamed: 0	Private	Apps	Accept	Enroll	Top10perc	Top25perc	F.Undergrad	P.Undergrad
16	Amherst College	Yes	4302	992	418	83	96	1593	5
20	Antioch University	Yes	713	661	252	25	44	712	23
60	Bowdoin College	Yes	3356	1019	418	76	100	1490	8
64	Brandeis University	Yes	4186	2743	740	48	77	2819	62

Using the IQR method, more instances in the college dataset are marked as outliers.

#### 13.4.1.2.3 When to use each:

- Use IQR when:
  - Data is skewed
  - You don't want to assume any distribution
  - You want a more sensitive detector
- Use Z-score when:
  - Data is approximately normal
  - You want a more conservative approach
  - You need a method that's easily interpretable

The actual number of outliers marked by each method will depend on your specific dataset's distribution and characteristics.

### 13.4.2 Common Methods for Handling outliers

Once we identify outlier instances using statistical methods, the next step is to handle them. Below are some commonly used approaches:

- **Removing Outliers:** Discarding extreme values if they are likely due to errors or are irrelevant for the analysis.
- **Winsorizing:** Capping/flooring outliers to a specific percentile to reduce their influence without removing them completely.
- **Replacing with the median:** Replacing outlier values with the median of the remaining data. The median is less affected by extreme values than the mean, making it an ideal choice for imputation when dealing with outliers
- **Transforming Data:** Applying transformations (e.g., log, square root) to reduce the impact of outliers.

```
def method_removal():
 """Method 1: Remove outliers"""
 data_cleaned = college[~college['is_outlier']]['Expend']
 return data_cleaned
```

```
def method_capping():
 """Method 2: Capping (Winsorization)"""
 data_capped = college['Expend'].copy()
 data_capped[data_capped < lower_bound] = lower_bound
 data_capped[data_capped > upper_bound] = upper_bound
 return data_capped
```

```
def method_mean_replacement():
 """Method 3: Replace with median"""
 data_median = college['Expend'].copy()
 median_value = college[~college['is_outlier']]['Expend'].median()
 data_median[college['is_outlier']] = median_value
 return data_median
```

```
def method_log_transformation():
 """Method 4: Log transformation"""
 data_log = np.log1p(college['Expend'])
 return data_log
```

**Your Practice:** Try each method and compare their results to see how they differ in handling outliers.

In real-world applications, handling outliers should be approached on a case-by-case basis. However, here are some general recommendations:

#### Recommendations for Different Scenarios:

- Data Removal:
  - Use when: You have a large dataset and can afford to lose data
  - Pros: Simple, removes all outlier influence
  - Cons: Loss of data, potential loss of important information
- Capping (Winsorization):
  - Use when: You want to preserve data count while limiting extreme values
  - Pros: Preserves data size, reduces impact of outliers
  - Cons: Artificial boundary creation, potential loss of genuine extreme events

- Median Replacement:
  - Use when: The data is normally distributed, and outliers are verified as errors or anomalies not representing true values
  - Pros: Maintains data size, simple to implement
  - Cons: Reduces variance, may not represent the true distribution
- Log Transformation:
  - Use when: Data has a right-skewed distribution, and you want to reduce the impact of large outliers.
  - Pros: Reduces skewness, minimizes the effect of extreme values, can make data more normal-like.
  - Cons: Only applicable to positive values, may not be effective for extremely high outliers.

## 13.5 Independent Study

### 13.5.1 Exercise 1: Missing Value Imputation Comparison

Using the *GDP\_missing\_data.csv* dataset:

1. Compare the RMSE for imputing missing `lifeFemale` values using:
  - Mean imputation
  - Median imputation
  - Regression with the most correlated variable
  - KNN imputation (k=3)
2. Which method performs best? Why do you think this is the case?
3. How does the choice of imputation method affect the distribution of the `lifeFemale` variable? Create histograms to visualize the differences.

### 13.5.2 Exercise 2: Outlier Detection and Impact Analysis

Using the *College.csv* dataset:

1. Identify outliers in the `Grad.Rate` column using both Z-score and IQR methods.
2. How many outliers are identified by each method? Which method is more conservative?
3. Apply three different outlier handling methods (removal, capping, median replacement) to the `Grad.Rate` column.

4. For each method, calculate:
  - The mean graduation rate before and after handling outliers
  - The standard deviation before and after
  - Create box plots to visualize the differences
5. Which outlier handling method would you recommend for this dataset and why? Consider the context of graduation rates when making your decision.

### 13.5.3 Exercise 3: Combined Data Cleaning Pipeline

Using the *GDP\_missing\_data.csv* dataset:

1. Create a comprehensive data cleaning pipeline that:
  - Removes columns with more than 40% missing values
  - Identifies the type of missing data (MCAR, MAR, or MNAR) for each remaining column with missing values
  - Applies appropriate imputation methods based on the missing data type
  - Detects outliers in all numeric columns using the IQR method
  - Handles outliers using an appropriate strategy
2. Document your decisions at each step with justifications.
3. Compare key statistics (mean, median, standard deviation) before and after your cleaning pipeline.
4. Create a summary report showing:
  - Number of missing values handled
  - Number of outliers detected and handled
  - RMSE for your imputation methods (where applicable)
  - Visual comparisons (scatter plots or box plots) showing the impact of your cleaning steps

# 14 Data Preparation

<IPython.core.display.Image object>

After data cleaning removes quality issues (missing values, outliers, duplicates), **data preparation** transforms the cleaned data into a format optimized for modeling and analysis. While data cleaning ensures data quality, data preparation ensures data is in the **right structure, scale, and format** for algorithms to work effectively.

Data scientists often spend **20-30% of their time** on data preparation after cleaning, as most machine learning algorithms have specific requirements:

- **Numerical inputs only** (requiring encoding of categorical variables)
- **Similar feature scales** (preventing some features from dominating others)
- **Meaningful representations** (creating derived features that capture important patterns)

## 14.1 Why Data Preparation Matters

Challenge	Issue	Solution
Information loss	Continuous data may hide useful patterns	Bin values into meaningful groups
Categorical data	Algorithms require numbers, not text	Encode categories as dummy variables
Scale differences	Income (\$50K) vs. Age (30) have different magnitudes	Normalize or standardize features
Model performance	Poorly prepared data leads to biased or inaccurate models	Apply appropriate transformations

**Note:** This chapter focuses on fundamental preparation techniques. Advanced feature engineering (creating interaction terms, polynomial features, domain-specific transformations) is typically covered in machine learning courses.

Let's first import necessary libraries

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import sklearn as sk
import seaborn as sns

sns.set(font_scale=1.5)

%matplotlib inline
```

## 14.2 Dataset Introduction

Throughout this chapter, we'll use the **College dataset** (`College.csv`) that contains information about US universities as our dataset in this section. The description of variables of the dataset can be found on page 65 of this [book](#).

We'll demonstrate how binning and encoding can reveal meaningful patterns and support actionable recommendations for universities. Finally, we'll apply feature scaling to prepare the data for predictive modeling.

Let's read the dataset into pandas dataframe

```
college = pd.read_csv('./datasets/College.csv', index_col=0)
college.head()
```

	Private	Apps	Accept	Enroll	Top10perc	Top25perc	F.Undergrad	P.Undergrad
Abilene Christian University	Yes	1660	1232	721	23	52	2885	537
Adelphi University	Yes	2186	1924	512	16	29	2683	122
Adrian College	Yes	1428	1097	336	22	50	1036	99
Agnes Scott College	Yes	417	349	137	60	89	510	63
Alaska Pacific University	Yes	193	146	55	16	44	249	869

## 14.3 Data Binning

Data binning is a practical technique in data analysis where continuous numerical values are grouped into discrete intervals, or “bins.” Think of it as sorting a long list of numbers into a few meaningful categories—like putting marbles of different colors into separate jars. Binning

helps us simplify complex data, spot patterns, and make comparisons that would be hard to see otherwise.

### Why do we use binning?

- **Easier interpretation:** Instead of analyzing hundreds of unique values, we can summarize data into a handful of groups. For example, rather than looking at every possible age, we might use age groups like ‘under 20’, ‘20-30’, ‘30-40’, and ‘40 and over’.
- **Better recommendations:** Grouping data helps us make targeted decisions. For instance, a doctor might recommend different treatments for children, adults, and seniors—each defined by an age bin.
- **Noise reduction:** Binning can smooth out random fluctuations in data, making trends clearer. For example, daily sales numbers might be noisy, but weekly or monthly bins reveal the bigger picture.

### Real-world examples:

- **Tax brackets:** Income is grouped into ranges (bins) to determine tax rates, such as ‘up to \$11,000’, ‘\$11,001 to \$44,725’, etc.
- **Credit card marketing:** Customers are binned as ‘High spenders’, ‘Medium spenders’, or ‘Low spenders’ to tailor marketing strategies.
- **Medical guidelines:** Vaccine recommendations often depend on age bins (e.g., under 12, 12-18, 18-65, over 65).

In this section, we’ll see how binning can help us interpret and communicate insights from data, using real datasets and practical code examples. We’ll also compare different binning methods and discuss when each is most useful.

<IPython.core.display.HTML object>

Now, let’s see how binning can help us uncover meaningful insights from our college dataset. We’ll focus on two key variables: instructional expenditure per student (**Expend**) and graduation rate (**Grad.Rate**) for U.S. universities.

By grouping universities into expenditure bins, we can more clearly observe how spending levels relate to graduation outcomes. This approach not only simplifies the analysis but also enables us to make practical, data-driven recommendations for different types of institutions.

Before we apply binning, let’s revisit the scatterplot of ‘Grad.Rate’ versus ‘Expend’ with a trendline. This plot shows the overall relationship, but as we’ll see, binning can reveal patterns and actionable insights that might be hidden in the raw data.

```
scatter plot of Expend vs Grad.Rate
ax=sns.regplot(data = college, x = "Expend", y = "Grad.Rate",scatter_kws={"color": "orange"})
ax.xaxis.set_major_formatter('${x:,.0f}')
ax.set_xlabel('Expenditure per student')
ax.set_ylabel('Graduation rate')
plt.show()
```



The trendline indicates a positive correlation between `Expend` and `Grad.Rate`. However, there seems to be a lot of noise and presence of outliers in the data, which makes it hard to interpret the overall trend.

### Common Data Binning Methods

There are several ways to create bins, each with its own advantages depending on your data and analysis goals:

- **Equal-width binning:** Divides the range of a variable into intervals of the same size. This is simple and works well when the data is uniformly distributed, but can result in bins with very different numbers of observations if the data is skewed or has outliers.

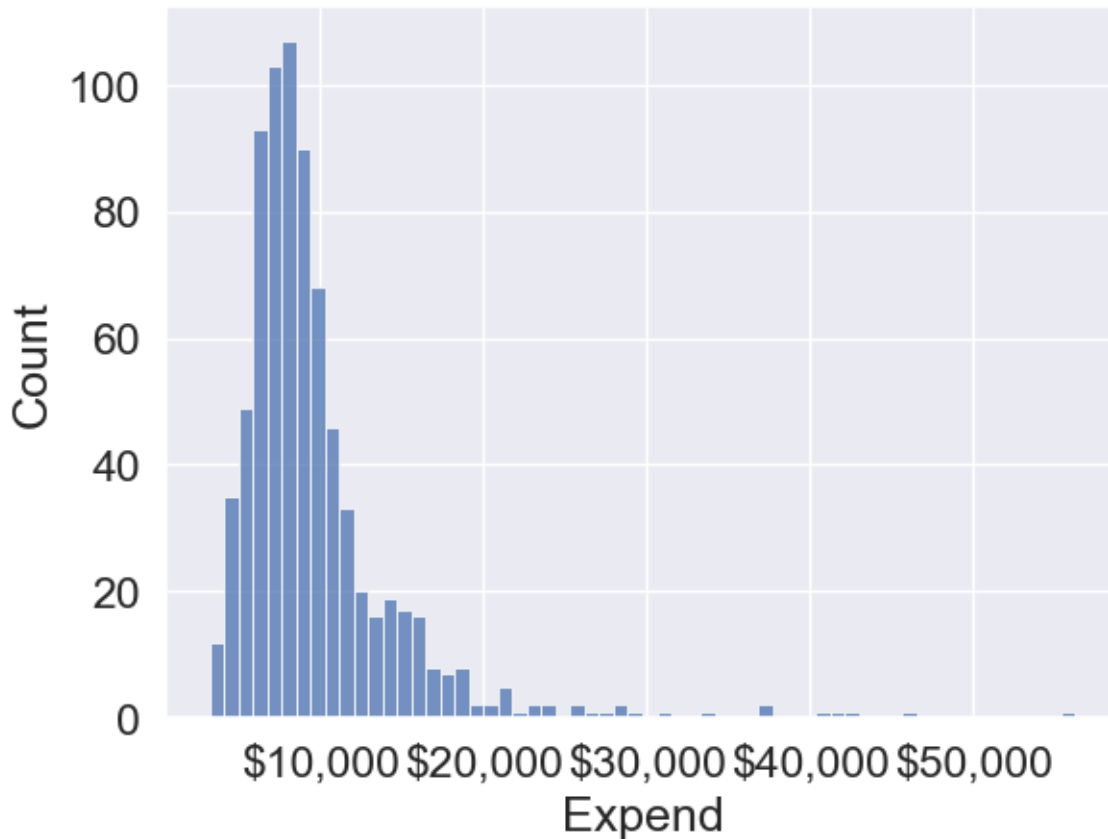


- *Example:* Splitting ages 0–100 into five bins of 20 years each: 0–20, 21–40, 41–60, 61–80, 81–100.
- **Equal-size (quantile) binning:** Each bin contains (roughly) the same number of observations, regardless of the actual range of values. This is useful for comparing groups of similar size and for handling skewed data.
  - *Example:* Dividing 300 students into three bins so that each bin has 100 students, even if the score ranges are different.
- **Custom binning:** Bins are defined based on domain knowledge or specific cutoffs relevant to the problem. This approach is flexible and can make results more interpretable for your audience.
  - *Example:* Income groups defined as ‘Low’ (<\$30,000), ‘Middle’ (\$30,000–\$70,000), and ‘High’ (>\$70,000) based on policy or business needs.

In practice, the choice of binning method should be guided by the data distribution and the questions you want to answer. We’ll demonstrate each of these methods in the following examples.

We’ll bin `Expend` to see if we can better analyze its association with `Grad.Rate`. However, let us first visualize the distribution of `Expend`.

```
#Visualizing the distribution of expend
ax=sns.histplot(data = college, x= 'Expend')
ax.xaxis.set_major_formatter('${x:,.0f}')
```



The distribution of **Expend** is right skewed with potentially some extremely high outlying values.

### 14.3.1 Equal-width Binning

Equal-width binning divides the range of **Expend** into intervals of the same size. This is useful for evenly spaced data, but can result in bins with very different numbers of universities if the data is skewed.

We'll use the Pandas function `cut()` to create three equal-width bins for instructional expenditure.

```
#Using the cut() function in Pandas to bin "Expend"
Binned_expend = pd.cut(college['Expend'],3,retbins = True)
Binned_expend
```

```
(Abilene Christian University (3132.953, 20868.333]
```

```

Adelphi University (3132.953, 20868.333]
Adrian College (3132.953, 20868.333]
Agnes Scott College (3132.953, 20868.333]
Alaska Pacific University (3132.953, 20868.333]
...
Worcester State College (3132.953, 20868.333]
Xavier University (3132.953, 20868.333]
Xavier University of Louisiana (3132.953, 20868.333]
Yale University (38550.667, 56233.0]
York College of Pennsylvania (3132.953, 20868.333]
Name: Expend, Length: 777, dtype: category
Categories (3, interval[float64, right]): [(3132.953, 20868.333] < (20868.333, 38550.667] <
array([3132.953 , 20868.33333333, 38550.66666667, 56233.]))

```

The `cut()` function returns a tuple of length 2. The first element of the tuple are the bins, while the second element is an array containing the cut-off values for the bins.

```
type(Binned_expend)
```

```
tuple
```

```
len(Binned_expend)
```

```
2
```

Once the bins are obtained, we'll add a column in the dataset that indicates the bin for Expend.

```

#Creating a categorical variable to store the level of expenditure on a student
college['Expend_bin'] = Binned_expend[0]
college.head()

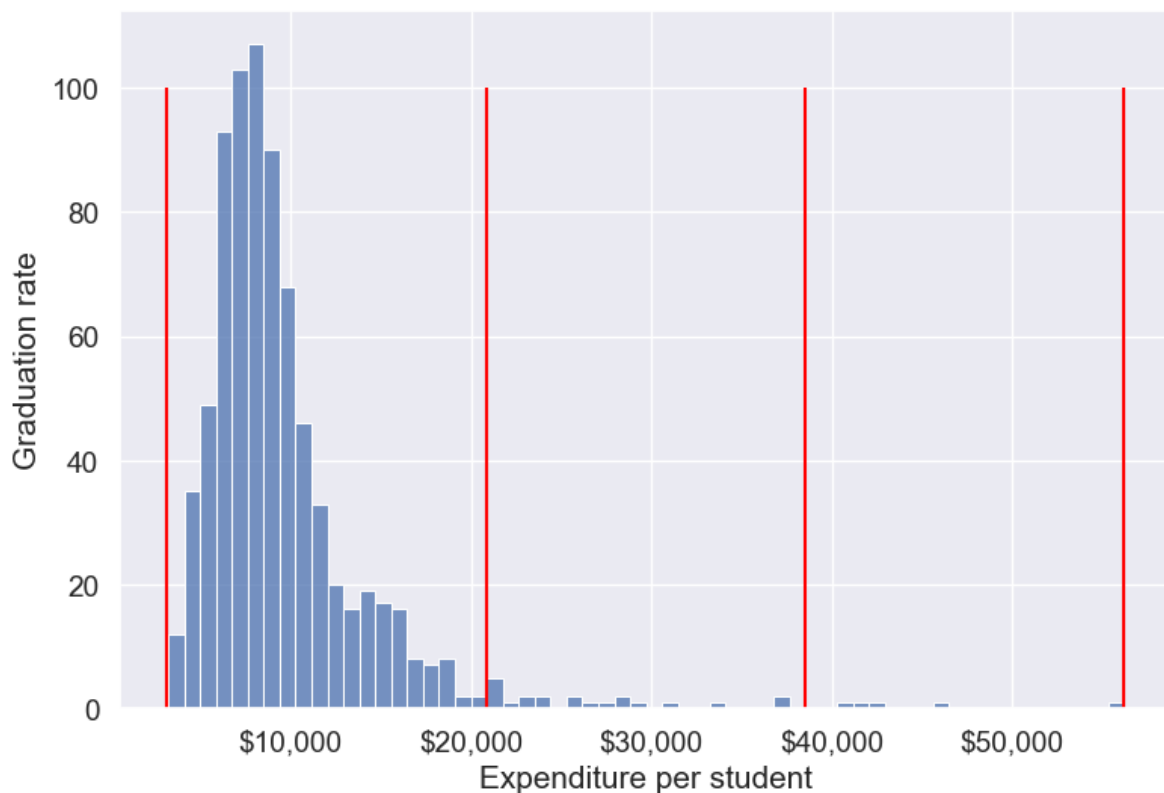
```

	Private	Apps	Accept	Enroll	Top10perc	Top25perc	F.Undergrad	P.U
Abilene Christian University	Yes	1660	1232	721	23	52	2885	537
Adelphi University	Yes	2186	1924	512	16	29	2683	122
Adrian College	Yes	1428	1097	336	22	50	1036	99
Agnes Scott College	Yes	417	349	137	60	89	510	63
Alaska Pacific University	Yes	193	146	55	16	44	249	869

See the variable `Expend_bin` in the above dataset.

Let us visualize the `Expend` bins over the distribution of the `Expend` variable.

```
#Visualizing the bins for instructional expenditure on a student
sns.set(font_scale=1.25)
plt.rcParams["figure.figsize"] = (9,6)
ax=sns.histplot(data = college, x= 'Expend')
plt.vlines(Binned_expend[1], 0,100,color='red')
plt.xlabel('Expenditure per student');
plt.ylabel('Graduation rate');
ax.xaxis.set_major_formatter('${x:,.0f}')
```



By default, the bins created have equal width. They are created by dividing the range between the maximum and minimum value of `Expend` into the desired number of equal-width intervals. We can label the bins as well as follows.

```
college['Expend_bin'] = pd.cut(college['Expend'],3,labels = ['Low expend','Med expend','High expend'])
college['Expend_bin']
```

```

Abilene Christian University Low expend
Adelphi University Low expend
Adrian College Low expend
Agnes Scott College Low expend
Alaska Pacific University Low expend
...
Worcester State College Low expend
Xavier University Low expend
Xavier University of Louisiana Low expend
Yale University High expend
York College of Pennsylvania Low expend
Name: Expend_bin, Length: 777, dtype: category
Categories (3, object): ['Low expend' < 'Med expend' < 'High expend']

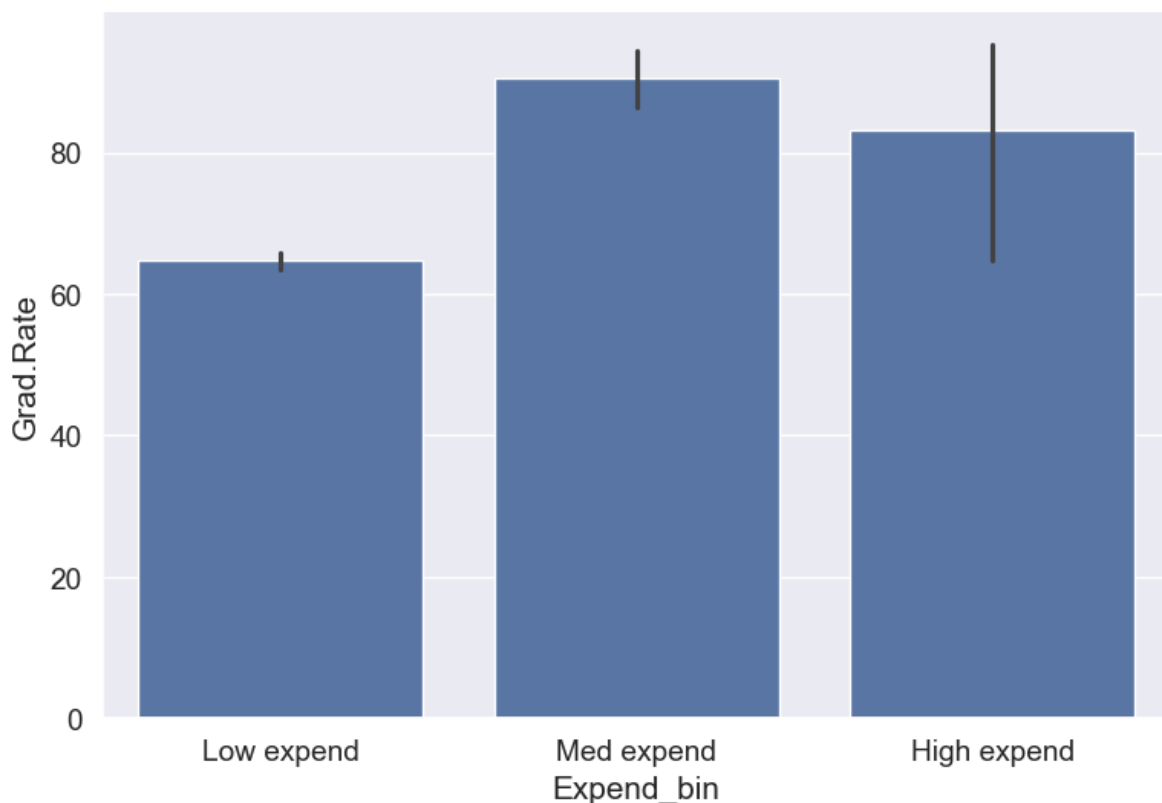
```

Now that we have binned the variable `Expend`, let us see if we can better visualize the association of graduation rate with expenditure per student using `Expend_bin`.

```

#Visualizing average graduation rate vs categories of instructional expenditure per student
sns.barplot(x = 'Expend_bin', y = 'Grad.Rate', data = college)

```



It seems that the graduation rate is the highest for universities with medium level of expenditure per student. This is different from the trend we saw earlier in the scatter plot. Let us investigate.

Let us find the number of universities in each bin.

```
college['Expend_bin'].value_counts()
```

```
Expend_bin
Low expend 751
Med expend 21
High expend 5
Name: count, dtype: int64
```

The bin **High expend** consists of only 5 universities, or 0.6% of all the universities in the dataset. These universities may be outliers that are skewing the trend (as also evident in the histogram above).

Let us see if we get the correct trend with the outliers removed from the data.

#### 14.3.1.1 Removing outliers in Expend (setup for fair comparisons)

Before we compare graduation rates across expenditure levels, we'll remove extreme outliers in instructional expenditure (**Expend**) so they don't distort averages. We'll use the IQR rule ( $Q1 - 1.5 \times IQR$ ,  $Q3 + 1.5 \times IQR$ ) to define lower and upper bounds and filter the dataset. This also defines `lower_bound` and `upper_bound` used later.

```
#Data without outliers

Q1 = college['Expend'].quantile(0.25)
Q3 = college['Expend'].quantile(0.75)
IQR = Q3 - Q1
lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR

college_data_without_outliers = college[((college.Expend>=lower_bound) & (college.Expend<=upper_bound))]
```

Let's visualize 'Expend' before and after outlier removal and show bounds

```

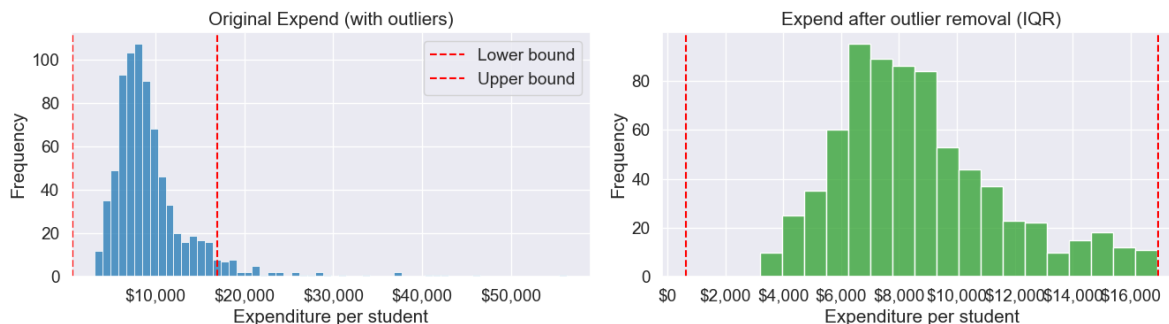
fig, axes = plt.subplots(1, 2, figsize=(14, 4))
sns.histplot(college['Expend'], ax=axes[0], color='tab:blue')
axes[0].axvline(lower_bound, color='red', linestyle='--', label='Lower bound')
axes[0].axvline(upper_bound, color='red', linestyle='--', label='Upper bound')
axes[0].set_title('Original Expend (with outliers)')
axes[0].set_xlabel('Expenditure per student')
axes[0].set_ylabel('Frequency')
axes[0].xaxis.set_major_formatter('${x:,.0f}')
axes[0].legend()

sns.histplot(college_data_without_outliers['Expend'], ax=axes[1], color='tab:green')
axes[1].axvline(lower_bound, color='red', linestyle='--')
axes[1].axvline(upper_bound, color='red', linestyle='--')
axes[1].set_title('Expend after outlier removal (IQR)')
axes[1].set_xlabel('Expenditure per student')
axes[1].set_ylabel('Frequency')
axes[1].xaxis.set_major_formatter('${x:,.0f}')

plt.tight_layout()
plt.show()

print(f"Q1 = {Q1:,.0f}, Q3 = {Q3:,.0f}")
print(f"IQR = {IQR:,.0f}")
print(f"Lower bound = {lower_bound:,.0f}, Upper bound = {upper_bound:,.0f}")

```



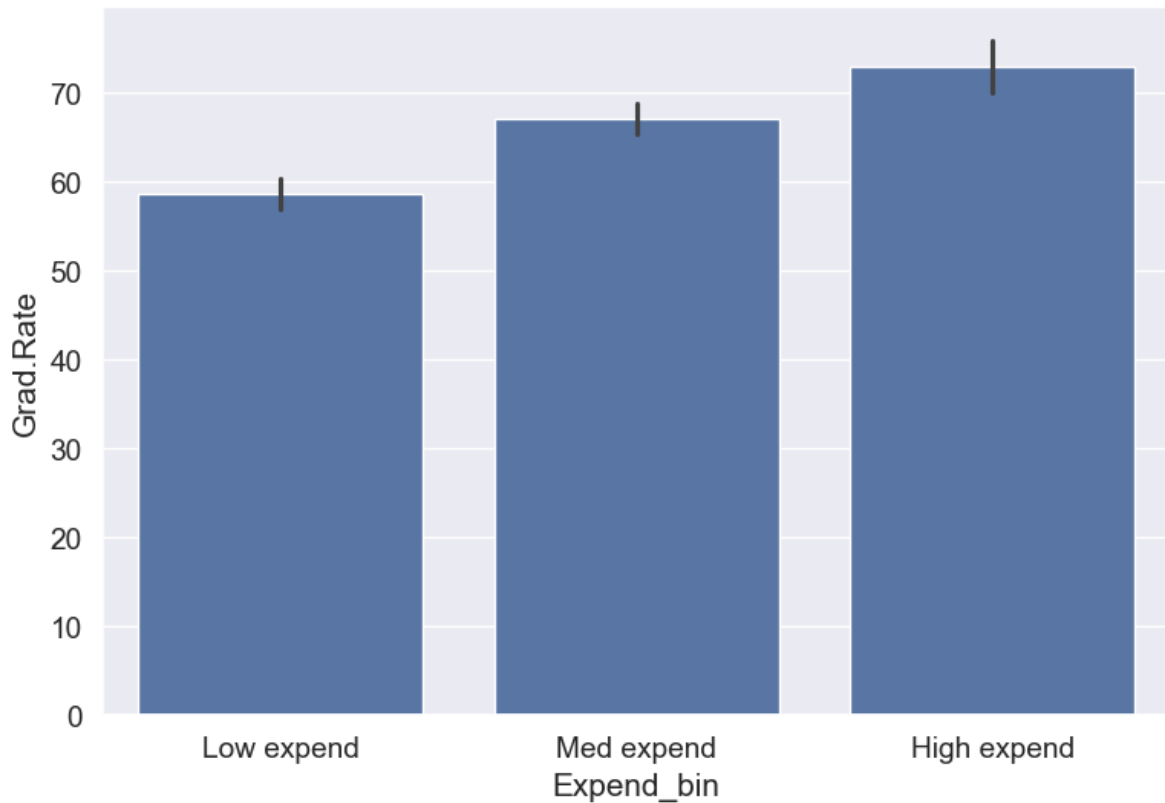
Q1 = 6,751, Q3 = 10,830  
 IQR = 4,079  
 Lower bound = 632, Upper bound = 16,948

Note that the right tail of the histogram has disappeared since we removed outliers.

Let's recreate the bins with equal size and visualize the relationship.

```
Binned_data = pd.cut(college_data_without_outliers['Expend'],3,labels = ['Low expend','Med e
college_data_without_outliers.loc[:, 'Expend_bin'] = Binned_data[0]
```

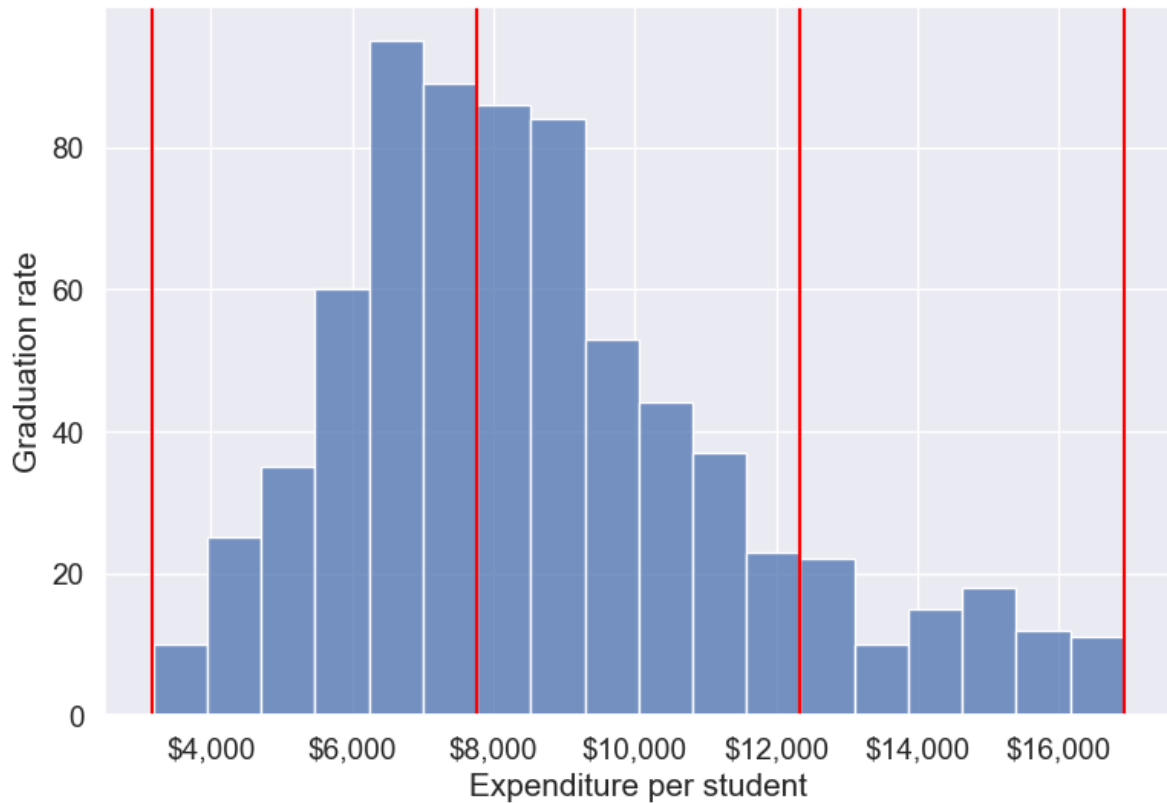
```
sns.barplot(x = 'Expend_bin', y = 'Grad.Rate', data = college_data_without_outliers)
```



With the outliers removed, we obtain the correct overall trend, even in the case of equal-width bins. Note that these bins have unequal number of observations as shown below.

```
ax=sns.histplot(data = college_data_without_outliers, x= 'Expend')
for i in range(4):
 plt.axvline(Binned_data[1][i], 0,100,color='red')
plt.xlabel('Expenditure per student');
plt.ylabel('Graduation rate');
ax.xaxis.set_major_formatter('${x:,.0f}')
```





```
college_data_without_outliers['Expend_bin'].value_counts()
```

```
Expend_bin
Med expend 327
Low expend 314
High expend 88
Name: count, dtype: int64
```

Instead of removing outliers, we can address the issue by binning observations into equal-sized bins, ensuring each bin has the same number of observations. Let's explore this approach next.

### 14.3.2 Equal-size (quantile) Binning

Equal-sized (quantile) binning splits a variable so that each bin contains approximately the same number of observations. This is especially helpful for skewed distributions and makes

comparisons across groups fair. We'll use `pd.qcut()` to divide `Expend` into three equally sized groups (tertiles).

Let us bin the variable `Expend` such that each bin consists of the same number of observations.

```
#Using the Pandas function qcut() to create bins with the same number of observations
Binned_expend = pd.qcut(college['Expend'],3,retbins = True)
college['Expend_bin'] = Binned_expend[0]
```

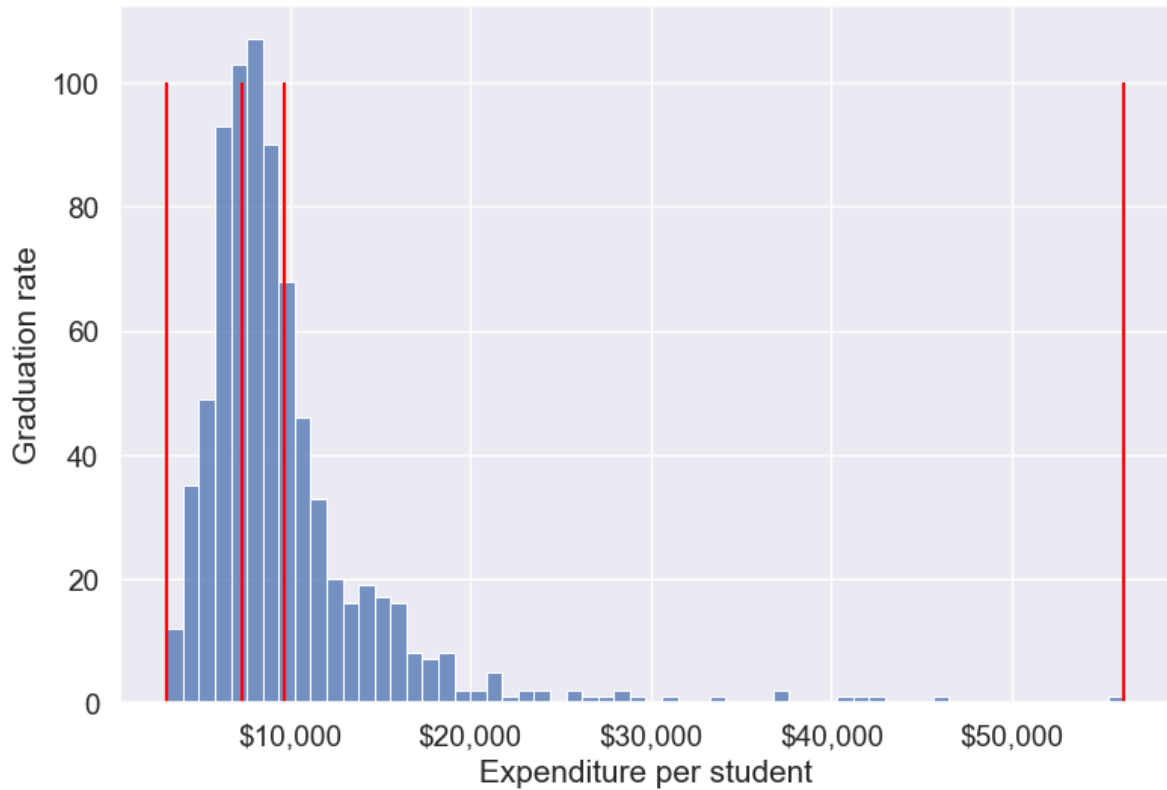
Let's check the number of observations in each bin

```
college['Expend_bin'].value_counts()
```

```
Expend_bin
(3185.999, 7334.333] 259
(7334.333, 9682.333] 259
(9682.333, 56233.0] 259
Name: count, dtype: int64
```

Let us visualize the `Expend` bins over the distribution of the `Expend` variable.

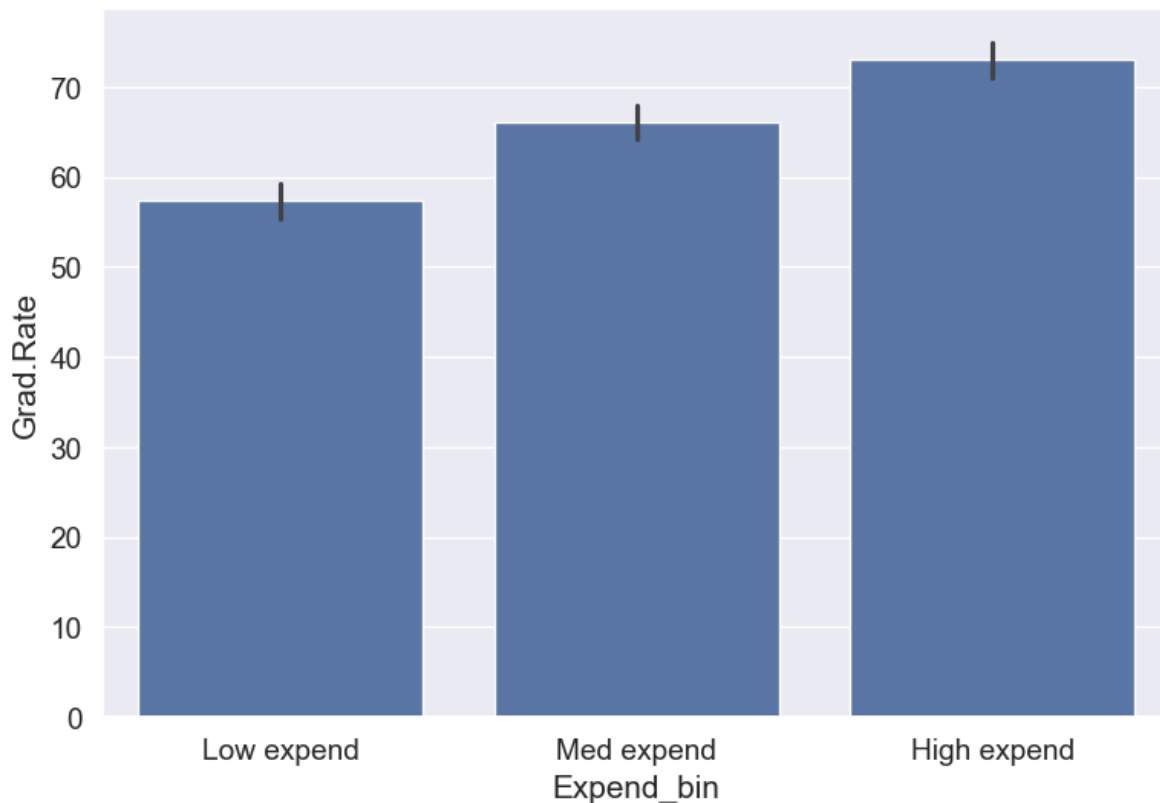
```
#Visualizing the bins for instructional expenditure on a student
sns.set(font_scale=1.25)
plt.rcParams["figure.figsize"] = (9,6)
ax=sns.histplot(data = college, x= 'Expend')
plt.vlines(Binned_expend[1], 0,100,color='red')
plt.xlabel('Expenditure per student');
plt.ylabel('Graduation rate');
ax.xaxis.set_major_formatter('${x:,.0f}')
```



Note that the bin-widths have been adjusted to have the same number of observations in each bin. The bins are narrower in domains of high density, and wider in domains of sparse density.

Let us again make the barplot visualizing the average graduate rate with level of instructional expenditure per student.

```
college['Expend_bin'] = pd.qcut(college['Expend'],3,labels = ['Low expend','Med expend','High expend'])
a=sns.barplot(x = 'Expend_bin', y = 'Grad.Rate', data = college)
```



Now we see the same trend that we saw in the scatterplot, but without the noise. We have smoothed the data. Note that making equal-sized bins helps reduce the effect of outliers in the overall trend.

Suppose this analysis was done to provide recommendations to universities for increasing their graduation rate. With binning, we can provide one recommendation to ‘*Low expend*’ universities, and another one to ‘*Med expend*’ universities. For example, the recommendations can be:

1. ‘*Low expend*’ universities can expect an increase of 9 percentage points in **Grad.Rate**, if they migrate to the ‘*Med expend*’ category.
2. ‘*Med expend*’ universities can expect an increase of 7 percentage points in **Grad.Rate**, if they migrate to the ‘*High expend*’ category.

The numbers in the above recommendations are based on the table below.

```
college.groupby(college.Expend_bin, observed=False)['Grad.Rate'].mean()
```

Expend_bin	
Low expend	57.343629

```
Med expend 66.057915
High expend 72.988417
Name: Grad.Rate, dtype: float64
```

We can also provide recommendations based on the confidence intervals of the mean graduation rate (Grad.Rate). The confidence intervals, calculated below, are derived using a method known as bootstrapping. For a detailed explanation of bootstrapping, please refer to [this article on Wikipedia](#).

```
Bootstrapping to find 95% confidence intervals of Graduation Rate of US universities based
confidence_intervals = {}

Loop through each expenditure bin
for expend_bin, data_sub in college.groupby('Expend_bin', observed=False):
 # Generate bootstrap samples for the graduation rate
 samples = np.random.choice(data_sub['Grad.Rate'], size=(10000, len(data_sub)), replace=True)

 # Calculate the mean of each sample
 sample_means = samples.mean(axis=1)

 # Calculate the 95% confidence interval
 lower_bound = np.percentile(sample_means, 2.5)
 upper_bound = np.percentile(sample_means, 97.5)

 # Store the result in the dictionary
 confidence_intervals[expend_bin] = (round(lower_bound, 2), round(upper_bound, 2))

Print the results
for expend_bin, ci in confidence_intervals.items():
 print(f"95% Confidence interval of Grad.Rate for {expend_bin} universities = [{ci[0]}, {ci[1]}]")
```

```
95% Confidence interval of Grad.Rate for Low expend universities = [55.36, 59.36]
95% Confidence interval of Grad.Rate for Med expend universities = [64.15, 67.95]
95% Confidence interval of Grad.Rate for High expend universities = [71.03, 74.92]
```

Apart from equal-width and equal-sized bins, custom bins can be created using the *bins* argument. Suppose, bins are to be created for **Expend** with cutoffs \$10,000, \$20,000, \$30,000...\$60,000. Then, we can use the *bins* argument as in the code below:

### 14.3.3 Custom Binning

Sometimes your analysis calls for bins that match business rules or domain thresholds rather than equal-width or equal-size splits. With `pd.cut()`, you can supply exact edges and readable labels to get bins that make sense for your audience.

Practical tips:

- Choose edges based on policy or domain knowledge (e.g., <\$10K, \$10K–\$40K, >\$40K).
- Use `include_lowest=True` to ensure the minimum value is captured by the first bin, and `right` to control which side of the interval is closed (default is right-closed).
- To avoid leaving values outside your bins, use open-ended edges with `-np.inf` and `np.inf`.
- Provide human-friendly `labels`; the result is an ordered categorical, so plots and tables keep the intended order.

We'll demonstrate two patterns:

- 1) Fixed-width \$10K bins (for a quick grid-like view)
- 2) Domain-based bins (Low/Medium/High) with open-ended edges

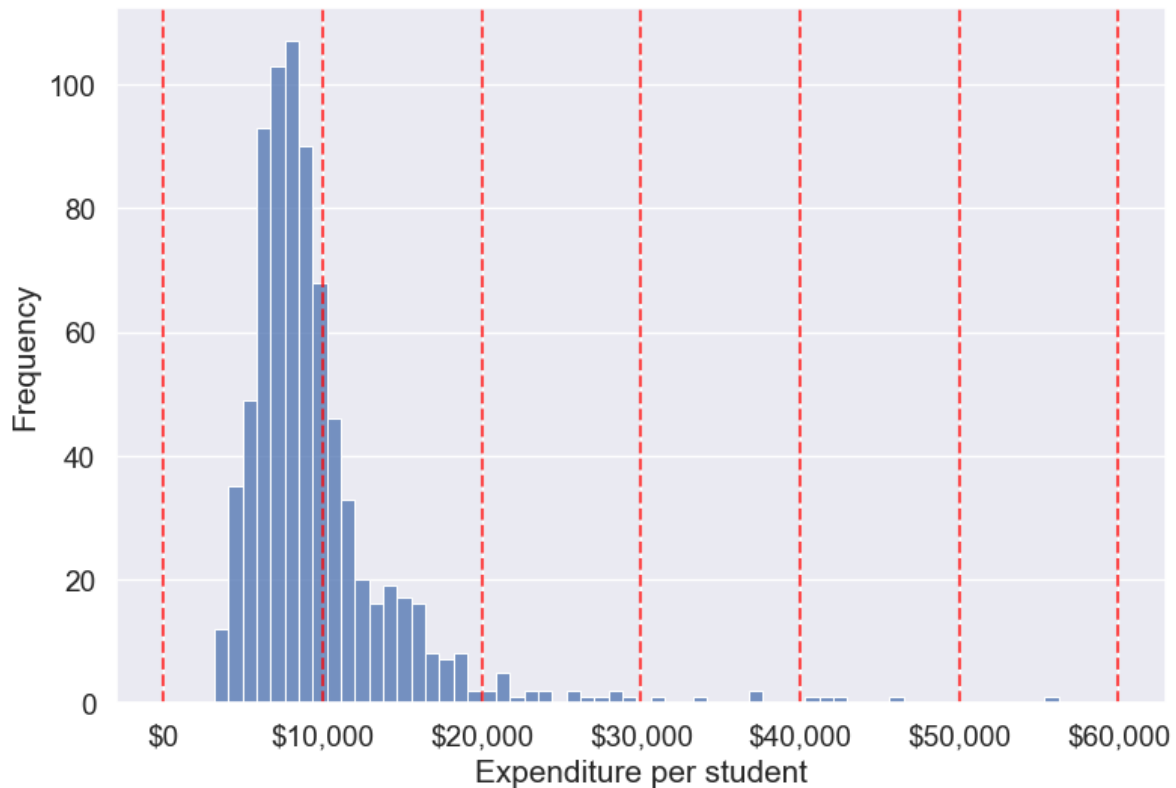
```
Example 1: fixed-width $10K bins up to $60K
edges_10k = list(range(0, 70000, 10000))
Expend_bins_10k = pd.cut(
 college['Expend'],
 bins=edges_10k,
 include_lowest=True,
 right=False, # intervals are [left, right)
 ordered=True,
 retbins=True
)

Save the categorical bins as a new column
college['Expend_bin_custom_10k'] = Expend_bins_10k[0]

Show counts and proportions per bin
counts_10k = college['Expend_bin_custom_10k'].value_counts().sort_index()
summary_10k = pd.DataFrame({
 'count': counts_10k,
 'proportion': (counts_10k / len(college)).round(3)
})
summary_10k
```

	count	proportion
Expend_bin_custom_10k		
[0, 10000)	535	0.689
[10000, 20000)	214	0.275
[20000, 30000)	19	0.024
[30000, 40000)	4	0.005
[40000, 50000)	4	0.005
[50000, 60000)	1	0.001

```
Visualizing the fixed $10K bins over the Expend distribution
sns.set(font_scale=1.25)
plt.rcParams["figure.figsize"] = (9,6)
ax = sns.histplot(data=college, x='Expend')
for edge in edges_10k:
 ax.axvline(edge, color='red', linestyle='--', alpha=0.7)
plt.xlabel('Expenditure per student')
plt.ylabel('Frequency')
ax.xaxis.set_major_formatter('${x:,.0f}')
plt.show()
```



Next, let's create a domain-based set of custom bins with unequal widths using open-ended edges. This pattern ensures every value is assigned to a bin (no NaNs from falling outside the range) and keeps the labels interpretable for stakeholders.

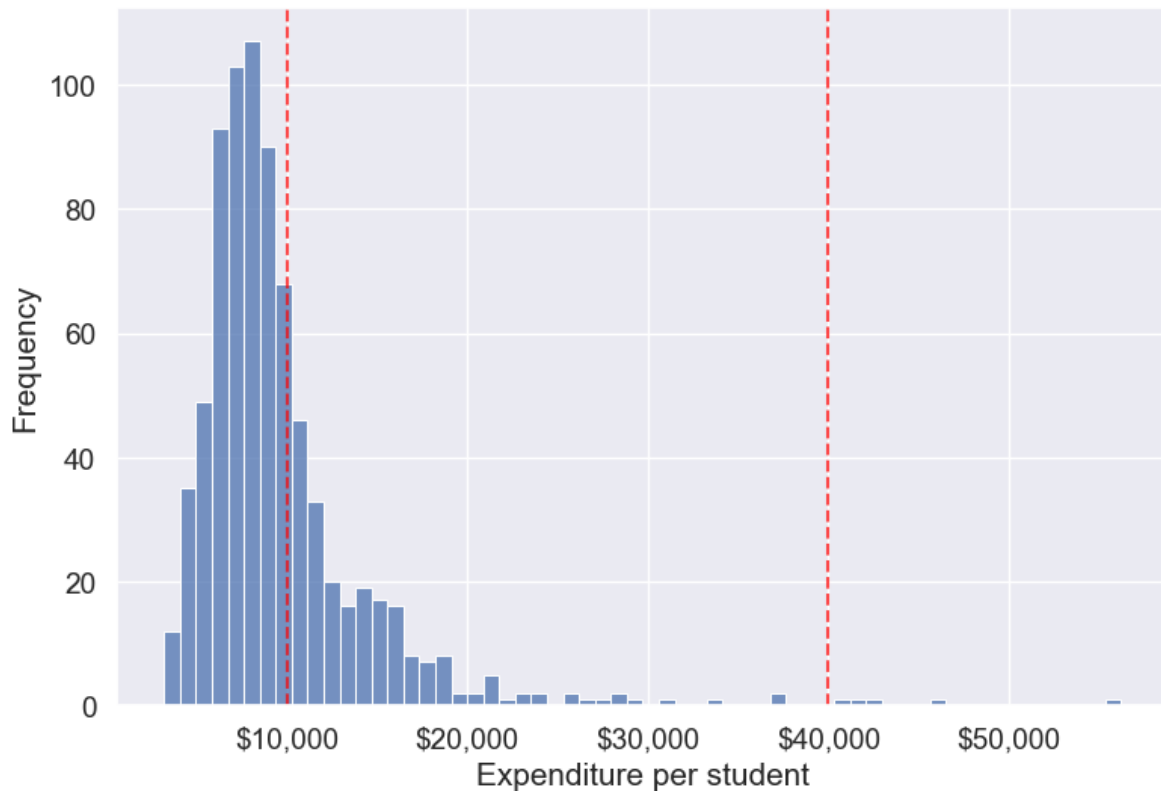
```
Example 2: domain-based custom bins with open-ended edges
bins = [-np.inf, 10000, 40000, np.inf]
labels = ['Low Expenditure', 'Medium Expenditure', 'High Expenditure']

Binning the 'Expend' column (ordered categorical)
college['Expend_bin_custom'] = pd.cut(
 college['Expend'], bins=bins, labels=labels, ordered=True, include_lowest=True
)

Visualizing the bins using a histogram
ax = sns.histplot(data=college, x='Expend', kde=False)
Add vertical lines for the finite bin edges
for edge in bins:
 if np.isfinite(edge):
 ax.axvline(edge, color='red', linestyle='--', alpha=0.7)
```



```
Labels and formatting
plt.xlabel('Expenditure per student')
plt.ylabel('Frequency')
ax.xaxis.set_major_formatter('${x:,.0f}')
plt.show()
```



```
Check counts and proportions per custom (domain-based) bin
counts_custom = college['Expend_bin_custom'].value_counts().sort_index()
summary_custom = pd.DataFrame({
 'count': counts_custom,
 'proportion': (counts_custom / len(college)).round(3)
})
summary_custom
```

	count	proportion
Expend_bin_custom		
Low Expenditure	535	0.689

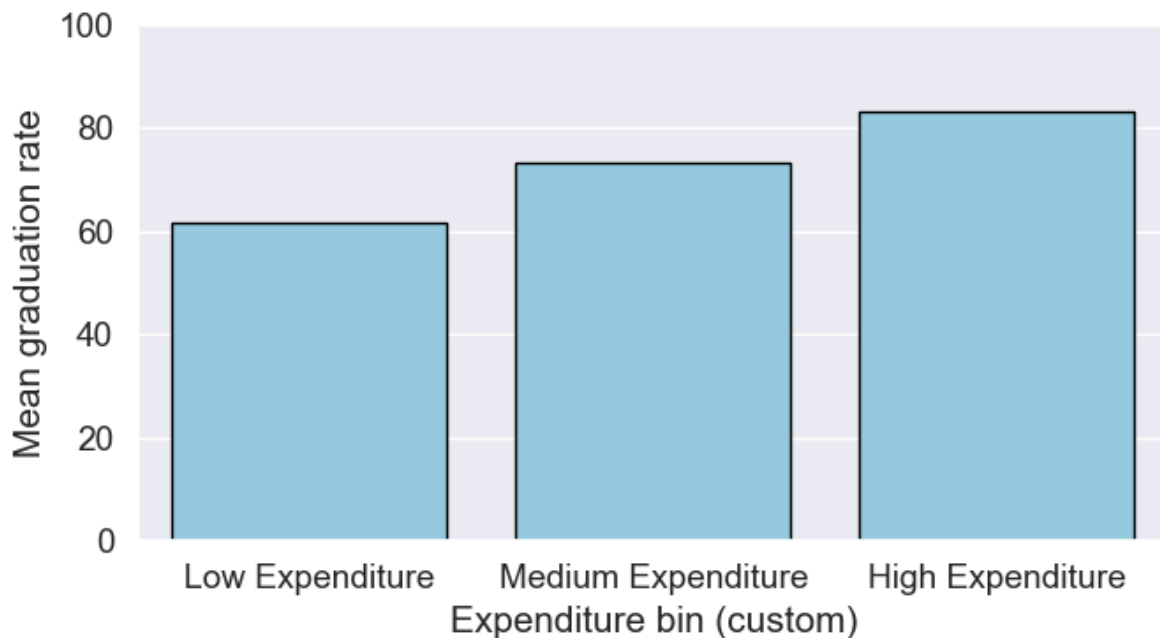
	count	proportion
Expend_bin_custom		
Medium Expenditure	237	0.305
High Expenditure	5	0.006

Compute mean graduation rate by these custom bins without using groupby (we'll use a pivot table, which preserves the categorical bin order).

```
college.groupby(college.Expend_bin_custom, observed=False)['Grad.Rate'].mean().round(2)
```

```
Expend_bin_custom
Low Expenditure 61.79
Medium Expenditure 73.37
High Expenditure 83.20
Name: Grad.Rate, dtype: float64
```

```
Draw bar plot using the groupby result
means_by_custom = college.groupby(college.Expend_bin_custom, observed=False)['Grad.Rate'].mean()
plt.figure(figsize=(7,4))
sns.barplot(x=means_by_custom.index.astype(str), y=means_by_custom.values, color='skyblue', c
plt.xlabel('Expenditure bin (custom)')
plt.ylabel('Mean graduation rate')
plt.ylim(0, 100)
plt.tight_layout()
plt.show()
```



As custom bin-cutoffs can be specified with the `cut()` function, custom bin quantiles can be specified with the `qcut()` function.

## 14.4 Encoding Categorical Variables

Many machine learning algorithms require numerical input, so categorical variables (text labels or categories) must be converted to numbers. There are two main approaches: label encoding and one-hot encoding. The choice depends on the nature of the variable and the model you plan to use.

### 14.4.1 Label Encoding

Label encoding assigns each unique category in a variable a unique integer value. This is simple and memory-efficient, but it can introduce unintended ordinal relationships (e.g.,  $0 < 1 < 2$ ) that may mislead some models.

**When to use:**

- For ordinal variables (where categories have a natural order, e.g., 'Low', 'Medium', 'High').
- For target variables in classification tasks, where labels must be numeric.

**When to avoid:** - For nominal variables (no order), especially with linear models, as the numeric codes may be misinterpreted as ordered.

Let's use the 'Private' column (Yes/No) in the college dataset as an example

```
Example: Label encoding a categorical column
from sklearn.preprocessing import LabelEncoder
le = LabelEncoder()
college['Private_label'] = le.fit_transform(college['Private'])
college[['Private', 'Private_label']].head()
```

	Private	Private_label
Abilene Christian University	Yes	1
Adelphi University	Yes	1
Adrian College	Yes	1
Agnes Scott College	Yes	1
Alaska Pacific University	Yes	1

## 14.4.2 One-Hot Encoding

One-hot encoding creates a new binary column for each category, with a 1 indicating the presence of that category and 0 otherwise. This avoids introducing any ordinal relationship between categories.

**When to use:** - For nominal variables (no natural order), e.g., 'State', 'Major', 'Color'. - For most linear models and neural networks.

**When to avoid:** - When the variable has many unique categories (can create too many columns, known as the "curse of dimensionality").

### 14.4.2.1 How to Create Dummy variables

The pandas library in Python has a built-in function, `pd.get_dummies()`, to generate dummy variables. If a column in a DataFrame has  $k$  distinct values, we will get a DataFrame with  $k$  columns containing 0s and 1s

Let us make dummy variables with the equal-sized bins we created for the average instruction expenditure per student.

```
#Using the Pandas function qcut() to create bins with the same number of observations
Binned_expend = pd.qcut(college['Expend'],3,retbins = True,labels = ['Low_expend','Med_expend','High_expend'])
college['Expend_bin'] = Binned_expend[0]
```

```
#Making dummy variables based on the levels (categories) of the 'Expend_bin' variable
dummy_Expend = pd.get_dummies(college['Expend_bin'], dtype=int)
```

The dummy data `dummy_Expend` has a value of 1 if the observation corresponds to the category referenced by the column name.

```
dummy_Expend.head()
```

	Low_expend	Med_expend	High_expend
0	1	0	0
1	0	0	1
2	0	1	0
3	0	0	1
4	0	0	1

Adding the dummy columns back to the original dataframe

```
#Concatenating the dummy variables to the original data
college = pd.concat([college,dummy_Expend],axis=1)
print(college.shape)
college.head()
```

(777, 26)

	Unnamed: 0	Private	Apps	Accept	Enroll	Top10perc	Top25perc	F.Undergrad
0	Abilene Christian University	Yes	1660	1232	721	23	52	2885
1	Adelphi University	Yes	2186	1924	512	16	29	2683
2	Adrian College	Yes	1428	1097	336	22	50	1036
3	Agnes Scott College	Yes	417	349	137	60	89	510
4	Alaska Pacific University	Yes	193	146	55	16	44	249

We can find the correlation between the dummy variables and graduation rate to identify if any of the dummy variables will be useful to estimate graduation rate (`Grad.Rate`).

```
#Finding if dummy variables will be useful to estimate 'Grad.Rate'
dummy_Expend.corrwith(college['Grad.Rate'])
```

```
Low_expend -0.334456
Med_expend 0.024492
High_expend 0.309964
dtype: float64
```

The dummy variables `Low_expend` and `High_expend` may contribute in explaining `Grad.Rate` in a regression model.

#### 14.4.2.2 Using `drop_first` to Avoid Multicollinearity

In cases like linear regression, you may want to drop one dummy variable to avoid multicollinearity (perfect correlation between variables).

```
create dummy variables, dropping the first to avoid multicollinearity
dummy_Expend = pd.get_dummies(college['Expend_bin'], dtype=int, drop_first=True)

add the dummy variables to the original dataset
college = pd.concat([college, dummy_Expend], axis=1)

college.head()
```

	Unnamed: 0	Private	Apps	Accept	Enroll	Top10perc	Top25perc	F.Undergrad
0	Abilene Christian University	Yes	1660	1232	721	23	52	2885
1	Adelphi University	Yes	2186	1924	512	16	29	2683
2	Adrian College	Yes	1428	1097	336	22	50	1036
3	Agnes Scott College	Yes	417	349	137	60	89	510
4	Alaska Pacific University	Yes	193	146	55	16	44	249

## 14.5 Feature Scaling

Feature scaling is a crucial step in data preparation, especially for machine learning algorithms that are sensitive to the scale of input features. Scaling ensures that numerical features contribute equally to model training and prevents features with larger ranges from dominating those with smaller ranges.

### Why scale features?

- Many algorithms (e.g., k-nearest neighbors, k-means, principal component analysis, gradient descent-based models) compute distances or rely on feature magnitude.

- Features with larger scales can unduly influence the model if not scaled.

#### Common scaling methods:

- **Min-Max Scaling:** Rescales features to a fixed range, usually  $[0, 1]$ .
- **Standardization (Z-score):** Centers features by removing the mean and scaling to unit variance.

**Note:** Only continuous numerical features should be scaled. **Do not scale encoded categorical variables (such as label-encoded or one-hot encoded columns).** Scaling these can distort their meaning and introduce unintended relationships. For example, one-hot encoded columns are already binary (0/1), and scaling them would break their interpretation as category indicators.

### 14.5.1 How to Scale Features in Practice

Let's demonstrate how to scale the continuous numerical features in the college dataset. We'll use both Min-Max scaling and Standardization (Z-score). We'll **exclude all categorical and encoded categorical variables** from scaling, including:

- The original 'Private' column (categorical)
- Any label-encoded columns (e.g., 'Private\_label')
- Any one-hot/dummy variables (e.g., columns created by `pd.get_dummies`)

Scaling these columns would destroy their categorical meaning. For example, one-hot encoded columns are meant to be 0 or 1, representing category membership. Scaling would turn them into non-binary values, which is not meaningful for models.

```
Select only continuous numerical columns for scaling
Exclude categorical, label-encoded, and dummy variables
from sklearn.preprocessing import MinMaxScaler, StandardScaler

Identify columns to exclude (categorical and encoded)
categorical_cols = ['Private', 'Private_label']
dummy_cols = [col for col in college.columns if col in ['Low_expend', 'Med_expend', 'High_expend']]

Get all numeric columns
numeric_cols = college.select_dtypes(include=[float, int]).columns.tolist()

Remove encoded categorical columns from scaling
cols_to_scale = [col for col in numeric_cols if col not in categorical_cols + dummy_cols]

Min-Max Scaling
```

```

minmax_scaler = MinMaxScaler()
college_minmax_scaled = college.copy()
college_minmax_scaled[cols_to_scale] = minmax_scaler.fit_transform(college[cols_to_scale])

Standardization (Z-score)
standard_scaler = StandardScaler()
college_standard_scaled = college.copy()
college_standard_scaled[cols_to_scale] = standard_scaler.fit_transform(college[cols_to_scale])

Show the first few rows of the scaled data
college_minmax_scaled[cols_to_scale].head()

```

	Apps	Accept	Enroll	Top10perc	Top25perc	F.Undergrad	P.Un
Abilene Christian University	0.032887	0.044177	0.107913	0.231579	0.472527	0.087164	0.024
Adelphi University	0.043842	0.070531	0.075035	0.157895	0.219780	0.080752	0.056
Adrian College	0.028055	0.039036	0.047349	0.221053	0.450549	0.028473	0.004
Agnes Scott College	0.006998	0.010549	0.016045	0.621053	0.879121	0.011776	0.002
Alaska Pacific University	0.002333	0.002818	0.003146	0.157895	0.384615	0.003492	0.039

```

Show the first few rows of the standardized (Z-score) scaled data
college_standard_scaled[cols_to_scale].head()

```

	Apps	Accept	Enroll	Top10perc	Top25perc	F.Undergrad	P.U
Abilene Christian University	-0.346882	-0.321205	-0.063509	-0.258583	-0.191827	-0.168116	-0.3
Adelphi University	-0.210884	-0.038703	-0.288584	-0.655656	-1.353911	-0.209788	0.2
Adrian College	-0.406866	-0.376318	-0.478121	-0.315307	-0.292878	-0.549565	-0.4
Agnes Scott College	-0.668261	-0.681682	-0.692427	1.840231	1.677612	-0.658079	-0.3
Alaska Pacific University	-0.726176	-0.764555	-0.780735	-0.655656	-0.596031	-0.711924	0.0

## Summary: When and What to Scale

- **Always scale continuous numerical features** before using algorithms sensitive to feature magnitude (e.g., k-means, k-NN, PCA, neural networks).
- **Never scale categorical variables or their encoded versions** (label-encoded or one-hot/dummy variables). Scaling these destroys their categorical meaning and can confuse models.
- For tree-based models (e.g., decision trees, random forests), scaling is not required, as these models are insensitive to feature scale.



## 14.6 Independent Study

### 14.6.1 Practice exercise 1

Read *survey\_data\_clean.csv*. Split the columns of the dataset, such that all columns with categorical values transform into dummy variables with each category corresponding to a column of 0s and 1s. Leave the *Timestamp* column.

As all categorical columns are transformed to dummy variables, all columns have numeric values.

What is the total number of columns in the transformed data? What is the total number of columns of the original data?

Find the:

1. Top 5 variables having the highest positive correlation with `NU_GPA`.
2. Top 5 variables having the highest negative correlation with `NU_GPA`.

### 14.6.2 Practice exercise 2

Consider the dataset *survey\_data\_clean.csv*. Find the number of outliers in each column of the dataset based on both the Z-score and IQR criteria. Do not use a `for` loop.

Which column(s) have the maximum number of outliers using Z-score?

Which column(s) have the maximum number of outliers using IQR?

Do you think the outlying observations identified for those columns(s) should be considered as outliers? If not, then which type of columns should be considered when finding outliers?

# 15 Data Grouping and Aggregation

<IPython.core.display.Image object>

## 15.1 Why Grouping and Aggregation Matter in Data Science

Grouping and aggregation are foundational techniques in data analysis that transform raw data into actionable insights. Here's why they're essential:

### Data Summarization

Grouping allows you to condense large amounts of data into concise summaries. Aggregation provides statistical summaries (e.g., mean, sum, count), making it easier to understand data trends and characteristics without needing to examine every detail.

### Insights Across Categories

Grouping by categories—like regions, demographics, or time periods—lets you analyze patterns within each group. For example:

- Aggregating sales data by region can reveal which areas are performing better.
- Grouping customer data by age group can highlight consumer trends.
- Comparing product performance across different market segments.

### Time Series Analysis

In time-series data, grouping by time intervals (e.g., day, month, quarter) allows you to observe trends over time, which is crucial for forecasting, seasonal analysis, and trend detection.

### Reducing Data Complexity

By summarizing data at a higher level, you reduce complexity, making it easier to visualize and interpret—especially in large datasets where working with raw data could be overwhelming.

In this chapter, we'll explore grouping and aggregating using pandas. These methods will help you group and summarize data, making complex analysis comparatively easy.

Throughout this chapter, we'll use the **GDP and Life Expectancy dataset** (`gdp_lifeExpectancy.csv`), which contains information about countries' GDP per capita and life expectancy over time. This dataset is ideal for demonstrating how grouping can reveal patterns across countries, continents, and years.

Let's start by importing the necessary libraries and loading the data.

```
Import necessary libraries
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt

Set visualization defaults
sns.set(font_scale=1.5)
%matplotlib inline
```

```
Load the GDP and Life Expectancy dataset
gdp_lifeExp_data = pd.read_csv('./Datasets/gdp_lifeExpectancy.csv')

Display the first few rows to understand the structure
gdp_lifeExp_data.head()
```

	country	continent	year	lifeExp	pop	gdpPercap
0	Afghanistan	Asia	1952	28.801	8425333	779.445314
1	Afghanistan	Asia	1957	30.332	9240934	820.853030
2	Afghanistan	Asia	1962	31.997	10267083	853.100710
3	Afghanistan	Asia	1967	34.020	11537966	836.197138
4	Afghanistan	Asia	1972	36.088	13079460	739.981106

## 15.2 Categorical Aggregation

Before diving into advanced grouping operations, it's important to understand how to aggregate **categorical variables**. Unlike numerical aggregation (mean, sum, etc.), categorical aggregation focuses on **counting frequencies** and understanding the distribution of categories.

Categorical aggregation helps answer questions like:

- How many observations belong to each category?
- What's the distribution across different categories?
- How do two categorical variables relate to each other?

### 15.2.1 One-way Aggregation: `value_counts()`

The `value_counts()` method is the simplest way to count occurrences of each unique value in a categorical column. It returns a Series with categories as the index and their counts as values.

**Example:** Let's count how many observations (country-year combinations) we have for each continent.

```
Count the number of observations for each continent
continent_counts = gdp_lifeExp_data['continent'].value_counts()
continent_counts
```

```
continent
Africa 624
Asia 396
Europe 360
Americas 300
Oceania 24
Name: count, dtype: int64
```

**Interpretation:** Africa has the most observations (624), while Oceania has the fewest (24). This tells us the dataset has unequal representation across continents.

#### 15.2.1.1 Useful `value_counts()` Parameters

The `value_counts()` method has several useful parameters:

- **`normalize=True`** - Returns proportions instead of counts
- **`sort=False`** - Returns results in the order they appear (not sorted by frequency)
- **`dropna=False`** - Includes counts of missing values

Let's see the **proportion** of observations in each continent:

```
Get proportions instead of counts
continent_proportions = gdp_lifeExp_data['continent'].value_counts(normalize=True)
print("Proportions of observations by continent:")
print(continent_proportions)
print(f"\nAfrica represents {continent_proportions['Africa']:.1%} of all observations")
```

Proportions of observations by continent:

```
continent
Africa 0.366197
Asia 0.232394
Europe 0.211268
Americas 0.176056
Oceania 0.014085
Name: proportion, dtype: float64
```

Africa represents 36.6% of all observations

## 15.2.2 Two-way Aggregation: `crosstab()`

While `value_counts()` works for a single categorical variable, `crosstab()` is used to examine the relationship between **two categorical variables**. It creates a frequency table (also called a contingency table) showing counts for each combination of categories.

**Use case:** `crosstab()` is ideal for:

- Understanding how categories from two variables overlap
- Checking data balance across multiple dimensions
- Creating frequency tables for statistical analysis

The `crosstab()` method is a special case of a pivot table specifically designed for computing group frequencies (or the size of each group).

### 15.2.2.1 Basic `crosstab()` Example

Let's start with a simple one-way frequency count using `crosstab()`. While `value_counts()` is more direct for this purpose, `crosstab()` can do it too:

```
Create a basic crosstab to count observations by continent
The 'columns' parameter is just a label for the column header
pd.crosstab(gdp_lifeExp_data['continent'], columns='count')
```

col_0	count
continent	
Africa	624
Americas	300
Asia	396
Europe	360

col_0	count
continent	
Oceania	24

### 15.2.2.2 Two-way Frequency Table

Now let's create a **two-way frequency table** to see how observations are distributed across both `continent` and `year`. This helps us understand the temporal coverage for each continent.

#### Adding Margins:

Use the `margins=True` argument to add row and column totals. The `All` row and column provide the sum across each dimension.

```
Create a two-way frequency table for continent and year
continent_year_table = pd.crosstab(
 gdp_lifeExp_data['continent'],
 gdp_lifeExp_data['year'],
 margins=True
)

Display the table
continent_year_table
```

year	1952	1957	1962	1967	1972	1977	1982	1987	1992	1997	2002	2007	All
continent													
Africa	52	52	52	52	52	52	52	52	52	52	52	52	624
Americas	25	25	25	25	25	25	25	25	25	25	25	25	300
Asia	33	33	33	33	33	33	33	33	33	33	33	33	396
Europe	30	30	30	30	30	30	30	30	30	30	30	30	360
Oceania	2	2	2	2	2	2	2	2	2	2	2	2	24
All	142	142	142	142	142	142	142	142	142	142	142	142	1704

#### Interpretation:

- Each cell shows the count of country-year observations for that continent-year combination
- The `All` column shows total observations per continent (same as `value_counts()`)
- The `All` row shows total observations per year
- The bottom-right cell (1704) is the grand total of all observations

This table is useful for:

- Checking if data is representative across all groups
- Identifying any missing years for specific continents
- Understanding the temporal balance of your dataset

### 15.2.2.3 Using `crosstab()` with Aggregation Functions

So far, we've used `crosstab()` to count frequencies. However, you can also use it to **aggregate numerical values** for each category combination by specifying:

- **values** - The numerical column to aggregate
- **aggfunc** - The aggregation function to apply (e.g., 'mean', 'sum', 'median')

This makes `crosstab()` a powerful tool that combines categorical grouping with numerical aggregation.

**Example:** Calculate the mean life expectancy for each continent-year combination.

This helps us see how life expectancy has evolved over time across different continents.

```
Calculate mean life expectancy for each continent-year combination
mean_lifeExp_table = pd.crosstab(
 gdp_lifeExp_data['continent'],
 gdp_lifeExp_data['year'],
 values=gdp_lifeExp_data['lifeExp'],
 aggfunc='mean'
)

Round to 1 decimal place for better readability
mean_lifeExp_table = mean_lifeExp_table.round(1)

Display the table
mean_lifeExp_table
```

year	1952	1957	1962	1967	1972	1977	1982	1987	1992	1997	2002	2007
continent												
Africa	39.1	41.3	43.3	45.3	47.5	49.6	51.6	53.3	53.6	53.6	53.3	54.8
Americas	53.3	56.0	58.4	60.4	62.4	64.4	66.2	68.1	69.6	71.2	72.4	73.6
Asia	46.3	49.3	51.6	54.7	57.3	59.6	62.6	64.9	66.5	68.0	69.2	70.7
Europe	64.4	66.7	68.5	69.7	70.8	71.9	72.8	73.6	74.4	75.5	76.7	77.6
Oceania	69.3	70.3	71.1	71.3	71.9	72.9	74.3	75.3	76.9	78.2	79.7	80.7

### Key Insights:

- Life expectancy has generally increased over time across all continents
- Oceania consistently has the highest life expectancy
- Africa has the lowest life expectancy, though it's improving
- The gap between continents is gradually narrowing over time

While `crosstab()` is excellent for two-way categorical analysis, `groupby()` offers significantly more flexibility:

Think of it as a progression:

- `value_counts()` → Count frequencies for one variable
- `crosstab()` → Cross-tabulate two categorical variables with optional aggregation
- `groupby()` → The most flexible and powerful grouping tool for all scenarios

In the next section, we'll explore `groupby()` in depth, starting with single-column grouping and gradually building up to more complex operations.

## 15.3 `groupby()`: Grouping by a Single Column

The `groupby()` method allows you to split a `DataFrame` based on the values in one or more columns, creating groups that can be analyzed separately. Unlike `crosstab()`, which is optimized for creating cross-tabulation tables, `groupby()` gives you complete control over how to aggregate, transform, and filter your data.

### 15.3.1 Syntax of `groupby()`

```
DataFrame.groupby(by="column_name")
```

This creates a **GroupBy** object that represents the grouped data.

### 15.3.2 Example: Grouping by Continent

Let's group the life expectancy data by `continent`. This will allow us to analyze statistics separately for each continent.



```
Create a GroupBy object by grouping data by 'continent'
grouped = gdp_lifeExp_data.groupby('continent')

This logically splits the data into groups based on unique values of 'continent'
However, the data is not physically split into separate DataFrames
grouped
```

```
<pandas.core.groupby.generic.DataFrameGroupBy object at 0x0000014936493FE0>
```

### 15.3.3 Understanding the GroupBy Object

The `groupby()` method returns a **GroupBy object**, not a DataFrame. This object contains information about how the data is grouped, but doesn't display the groups directly.

```
Check the type of the grouped object
type(grouped)
```

```
pandas.core.groupby.generic.DataFrameGroupBy
```

**Key Point:** The GroupBy object `grouped` contains the information about how observations are distributed across groups. Each observation has been assigned to a specific group based on the value of `continent` for that observation. However, the dataset is **not physically split** into different DataFrames—all observations remain in the same DataFrame `gdp_lifeExp_data` until you apply an aggregation or transformation.

### 15.3.4 Exploring GroupBy Objects: Attributes and Methods

The [GroupBy object](#) has several useful attributes and methods that help you understand and work with your grouped data. Let's explore the most important ones.

#### 15.3.4.1 keys - Identifying the Grouping Column(s)

The column(s) used to group the data are called **keys**. You can view the grouping key(s) using the `keys` attribute.

```
View the grouping key(s)
grouped.keys
```

```
'continent'
```



```

1670, 1671, 1672, 1673, 1674, 1675, 1676, 1677, 1678, 1679],
dtype='int64', length=396), Index([12, 13, 14, 15, 16, 17, 18, 19, 20,
...
1598, 1599, 1600, 1601, 1602, 1603, 1604, 1605, 1606, 1607],
dtype='int64', length=360), Index([60, 61, 62, 63, 64, 65, 66, 67, 68,
1092, 1093, 1094, 1095, 1096, 1097, 1098, 1099, 1100, 1101, 1102, 1103],
dtype='int64']])

```

#### 15.3.4.4 size() - Counting Observations per Group

The `size()` method returns the number of observations in each group. This is useful for understanding the distribution of data across groups.

```

Count the number of observations in each continent group
grouped.size()

```

```

continent
Africa 624
Americas 300
Asia 396
Europe 360
Oceania 24
dtype: int64

```

#### 15.3.4.5 first() - Viewing the First Element of Each Group

The `first()` method returns the first non-missing element from each group. This can be useful for quickly inspecting what each group looks like.

```

View the first non-missing observation from each continent group
grouped.first()

```

	country	year	lifeExp	pop	gdpPercap
continent					
Africa	Algeria	1952	43.077	9279525	2449.008185
Americas	Argentina	1952	62.485	17876956	5911.315053
Asia	Afghanistan	1952	28.801	8425333	779.445314
Europe	Albania	1952	55.230	1282697	1601.056136
Oceania	Australia	1952	69.120	8691212	10039.595640

#### 15.3.4.6 get\_group() - Extracting Data for a Specific Group

The `get_group()` method returns all observations belonging to a particular group. This is useful when you want to analyze or visualize a specific group in detail.

```
Extract all observations for the 'Asia' continent
grouped.get_group('Asia').head(10)
```

	country	continent	year	lifeExp	pop	gdpPercap
0	Afghanistan	Asia	1952	28.801	8425333	779.445314
1	Afghanistan	Asia	1957	30.332	9240934	820.853030
2	Afghanistan	Asia	1962	31.997	10267083	853.100710
3	Afghanistan	Asia	1967	34.020	11537966	836.197138
4	Afghanistan	Asia	1972	36.088	13079460	739.981106
5	Afghanistan	Asia	1977	38.438	14880372	786.113360
6	Afghanistan	Asia	1982	39.854	12881816	978.011439
7	Afghanistan	Asia	1987	40.822	13867957	852.395945
8	Afghanistan	Asia	1992	41.674	16317921	649.341395
9	Afghanistan	Asia	1997	41.763	22227415	635.341351

## 15.4 Data Aggregation Within Groups

Once you've created a GroupBy object, the real power comes from applying **aggregation functions** to summarize the data within each group. Aggregation functions compute summary statistics that help you understand patterns and trends across different groups.

### 15.4.1 Common Aggregation Functions

Below are the most commonly used aggregation functions when working with grouped data in pandas:

Function	Description	Example Use Case
<code>mean()</code>	Calculates the <b>average</b> value	Average life expectancy per continent
<code>sum()</code>	Computes the <b>total</b> by summing all values	Total population across countries
<code>min()</code>	Finds the <b>minimum</b> value	Lowest GDP per capita in each region
<code>max()</code>	Finds the <b>maximum</b> value	Highest life expectancy in each year

Function	Description	Example Use Case
<code>count()</code>	Counts the <b>number of non-null entries</b>	Number of countries per continent
<code>median()</code>	Finds the <b>middle value</b> when sorted	Median income across categories
<code>std()</code>	Calculates the <b>standard deviation</b>	Variability in GDP within continents

Each of these functions helps summarize different aspects of your grouped data, making it easier to identify trends, outliers, and patterns.

### 15.4.2 Example: Calculating Mean Statistics by Country

Let's find the mean life expectancy, population, and GDP per capita for each country during the period 1952-2007. This will give us a single summary statistic for each country across all years in the dataset.

#### Approach:

1. Remove columns we don't want to aggregate (`continent` and `year`)
2. Group by `country`
3. Calculate the mean for all remaining numeric columns

```
Group the data by 'country' and calculate mean statistics
First, drop columns we don't want to aggregate
grouped_country = gdp_lifeExp_data.drop(['continent', 'year'], axis=1).groupby('country')

Calculate the mean for all numeric columns (lifeExp, pop, gdpPercap)
country_means = grouped_country.mean()

Display the results
country_means.head(10)
```

	lifeExp	pop	gdpPercap
country			
Afghanistan	37.478833	1.582372e+07	802.674598
Albania	68.432917	2.580249e+06	3255.366633
Algeria	59.030167	1.987541e+07	4426.025973
Angola	37.883500	7.309390e+06	3607.100529
Argentina	69.060417	2.860224e+07	8955.553783

	lifeExp	pop	gdpPercap
country			
Australia	74.662917	1.464931e+07	19980.595634
Austria	73.103250	7.583298e+06	20411.916279
Bahrain	65.605667	3.739132e+05	18077.663945
Bangladesh	49.834083	9.075540e+07	817.558818
Belgium	73.641750	9.725119e+06	19900.758072

### 15.4.3 Calculating Other Statistics

We can apply different aggregation functions to understand the variability and spread of data within each group. Let's calculate the standard deviation to see how much variation exists within each country over time.

```
Calculate the standard deviation for each country
country_std = grouped_country.std()

Display the results
country_std.head(10)
```

	lifeExp	pop	gdpPercap
country			
Afghanistan	5.098646	7.114583e+06	108.202929
Albania	6.322911	8.285855e+05	1192.351513
Algeria	10.340069	8.613355e+06	1310.337656
Angola	4.005276	2.672281e+06	1165.900251
Argentina	4.186470	7.546609e+06	1862.583151
Australia	4.147774	3.915203e+06	7815.405220
Austria	4.379838	4.376600e+05	9655.281488
Bahrain	8.571871	2.108933e+05	5415.413364
Bangladesh	9.028335	3.471166e+07	235.079648
Belgium	3.779658	5.206359e+05	8391.186269

**Interpretation:** Countries with higher standard deviations in `lifeExp` experienced greater changes in life expectancy over the years, while those with lower standard deviations remained more stable.

You can apply any of the aggregation functions mentioned earlier (`min()`, `max()`, `count()`, `median()`, etc.) in the same way to compute different statistics for your groups.

## 15.5 Multiple and Custom Aggregations Using `agg()`

While applying single aggregation functions like `mean()` or `std()` is useful, you'll often need to:

1. Apply **multiple aggregation functions** simultaneously
2. Use **custom aggregation functions** that aren't built into pandas

The `agg()` method of a `GroupBy` object provides this flexibility.

### 15.5.1 Multiple Aggregations on a Single Column

When you need to compute several statistics for the same column, pass a **list of function names** to `agg()`.

**Example:** Calculate both the mean and standard deviation of GDP per capita for each country.

```
Apply multiple aggregation functions to 'gdpPercap' column
gdp_stats = grouped_country['gdpPercap'].agg(['mean', 'std'])

Sort by mean GDP per capita (descending) and show top 10
gdp_stats.sort_values(by='mean', ascending=False).head(10)
```

	mean	std
country		
Kuwait	65332.910472	33882.139536
Switzerland	27074.334405	6886.463308
Norway	26747.306554	13421.947245
United States	26261.151347	9695.058103
Canada	22410.746340	8210.112789
Netherlands	21748.852208	8918.866411
Denmark	21671.824888	8305.077866
Germany	20556.684433	8076.261913
Iceland	20531.422272	9373.245893
Austria	20411.916279	9655.281488

## 15.5.2 Custom Aggregation Functions

In addition to built-in functions, you can create your own custom aggregation functions. This is useful when you need a statistic that isn't available in pandas.

**Example:** Calculate the **range** (max - min) of GDP per capita for each country using a lambda function.

```
Calculate the range (max - min) of gdpPercap for each country using a lambda function
gdp_range = grouped_country['gdpPercap'].agg(lambda x: x.max() - x.min())

Display the results sorted by range (descending)
gdp_range.sort_values(ascending=False).head(10)
```

```
country
Kuwait 85404.702920
Singapore 44828.041413
Norway 39261.768450
Hong Kong, China 36670.557461
Ireland 35465.716022
Austria 29989.416208
United States 28961.171010
Iceland 28913.100762
Japan 28439.111713
Netherlands 27856.361462
Name: gdpPercap, dtype: float64
```

```
Alternatively, define a named function for better readability
def range_func(x):
 """Calculate the range (max - min) of a series"""
 return x.max() - x.min()
```

```
Combine built-in functions with custom functions
gdp_complete_stats = grouped_country['gdpPercap'].agg(['mean', 'std', range_func])

Sort by range and display top 10 countries
gdp_complete_stats.sort_values(by='range_func', ascending=False).head(10)
```



	mean	std	range_func
country			
Kuwait	65332.910472	33882.139536	85404.702920
Singapore	17425.382267	14926.147774	44828.041413
Norway	26747.306554	13421.947245	39261.768450
Hong Kong, China	16228.700865	12207.329731	36670.557461
Ireland	15758.606238	11573.311022	35465.716022
Austria	20411.916279	9655.281488	29989.416208
United States	26261.151347	9695.058103	28961.171010
Iceland	20531.422272	9373.245893	28913.100762
Japan	17750.869984	10131.612545	28439.111713
Netherlands	21748.852208	8918.866411	27856.361462

### 15.5.3 Renaming Aggregated Columns

For better readability and professional reporting, you can rename the columns resulting from aggregation. Pass a **list of tuples** to `agg()`, where each tuple contains:

1. The new column name (string)
2. The aggregation function (string or callable)

**Example:** Create a summary table with custom column names and include a 90th percentile calculation.

```
Apply multiple aggregations with custom column names
gdp_summary = grouped_country['gdpPercap'].agg(
 [('Average', 'mean'),
 ('Standard Deviation', 'std'),
 ('90th Percentile', lambda x: x.quantile(0.9))
])

Sort by 90th percentile and display top 10
gdp_summary.sort_values(by='90th Percentile', ascending=False).head(10)
```

	Average	Standard Deviation	90th Percentile
country			
Kuwait	65332.910472	33882.139536	109251.315590
Norway	26747.306554	13421.947245	44343.894158
United States	26261.151347	9695.058103	38764.132898
Singapore	17425.382267	14926.147774	35772.742520

	Average	Standard Deviation	90th Percentile
country			
Switzerland	27074.334405	6886.463308	34246.394240
Netherlands	21748.852208	8918.866411	33376.895065
Ireland	15758.606238	11573.311022	33121.539164
Canada	22410.746340	8210.112789	32891.561152
Saudi Arabia	20261.743635	8754.387440	32808.019502
Austria	20411.916279	9655.281488	32085.438987

## 15.6 Multiple Aggregations on Multiple Columns

Often, you'll want to apply the same aggregation functions to several columns simultaneously. This is straightforward with `agg()`.

**Example:** Calculate the mean and standard deviation of both life expectancy (`lifeExp`) and population (`pop`) for each country.

```
Apply the same aggregations to multiple columns
multi_column_stats = grouped_country[['lifeExp', 'pop']].agg(['mean', 'std'])

Sort by mean life expectancy (descending)
multi_column_stats.sort_values(by=('lifeExp', 'mean'), ascending=False).head(10)
```

	lifeExp		pop	
	mean	std	mean	std
country				
Iceland	76.511417	3.026593	2.269781e+05	4.854168e+04
Sweden	76.177000	3.003990	8.220029e+06	6.365660e+05
Norway	75.843000	2.423994	4.031441e+06	4.107955e+05
Netherlands	75.648500	2.486363	1.378680e+07	2.005631e+06
Switzerland	75.565083	4.011572	6.384293e+06	8.582009e+05
Canada	74.902750	3.952871	2.446297e+07	5.940793e+06
Japan	74.826917	6.494629	1.117588e+08	1.488988e+07
Australia	74.662917	4.147774	1.464931e+07	3.915203e+06
Denmark	74.370167	2.220111	4.994187e+06	3.520599e+05
France	74.348917	4.304761	5.295256e+07	6.086809e+06

## 15.7 Different Aggregations for Different Columns

Sometimes you need to apply **different sets of functions to different columns**. For this, pass a **dictionary** to `agg()` where:

- **Keys** are column names
- **Values** are lists of aggregation functions (as strings or callables)

This approach gives you complete control over which statistics are computed for each column.

```
Apply different aggregation functions to different columns using a dictionary
varied_stats = grouped_country.agg({
 'gdpPercap': ['mean', 'std'], # Mean and std for GDP per capita
 'lifeExp': ['median', 'std'], # Median and std for life expectancy
 'pop': ['max', 'min'] # Max and min for population
})

Display the first 10 rows
varied_stats.head(10)
```

	gdpPercap		lifeExp		pop	
	mean	std	median	std	max	min
country						
Afghanistan	802.674598	108.202929	39.1460	5.098646	31889923	8425333
Albania	3255.366633	1192.351513	69.6750	6.322911	3600523	1282697
Algeria	4426.025973	1310.337656	59.6910	10.340069	33333216	9279525
Angola	3607.100529	1165.900251	39.6945	4.005276	12420476	4232095
Argentina	8955.553783	1862.583151	69.2115	4.186470	40301927	17876956
Australia	19980.595634	7815.405220	74.1150	4.147774	20434176	8691212
Austria	20411.916279	9655.281488	72.6750	4.379838	8199783	6927772
Bahrain	18077.663945	5415.413364	67.3225	8.571871	708573	120447
Bangladesh	817.558818	235.079648	48.4660	9.028335	150448339	46886859
Belgium	19900.758072	8391.186269	73.3650	3.779658	10392226	8730405

### 15.7.1 Combining Dictionary Syntax with Custom Column Names

You can also use the dictionary approach with custom column names by passing tuples as the aggregation values.

**Example:** For each country, calculate:

- Mean and standard deviation of life expectancy (with custom names)

- Min and max values of GDP per capita

```
Apply different functions to different columns with custom names
custom_varied_stats = grouped_country.agg({
 'lifeExp': [('Average', 'mean'),
 ('Standard deviation', 'std')],
 'gdpPercap': ['min', 'max']
})

Display the first 10 rows
custom_varied_stats.head(10)
```

	lifeExp		gdpPercap	
	Average	Standard deviation	min	max
country				
Afghanistan	37.478833	5.098646	635.341351	978.011439
Albania	68.432917	6.322911	1601.056136	5937.029526
Algeria	59.030167	10.340069	2449.008185	6223.367465
Angola	37.883500	4.005276	2277.140884	5522.776375
Argentina	69.060417	4.186470	5911.315053	12779.379640
Australia	74.662917	4.147774	10039.595640	34435.367440
Austria	73.103250	4.379838	6137.076492	36126.492700
Bahrain	65.605667	8.571871	9867.084765	29796.048340
Bangladesh	49.834083	9.028335	630.233627	1391.253792
Belgium	73.641750	3.779658	8343.105127	33692.605080

## 15.8 Grouping by Multiple Columns

Above, we demonstrated grouping by a single column, which is useful for summarizing data based on one categorical variable. However, in many cases, we need to group by multiple columns. Grouping by multiple columns allows us to create more detailed summaries by accounting for multiple categorical variables. This approach enables us to analyze data at a finer granularity, revealing insights that might be missed with single-column grouping alone.

### 15.8.1 Basic Syntax for Grouping by Multiple Columns

Use `groupby()` with a list of column names to group data by multiple columns.

- `DataFrame.groupby(by=["col1", "col2"])`

Consider the life expectancy dataset, we can group by both country and continent to analyze `gdpPercap`, `lifeExp`, and `pop` trends for each country within each continent, providing a more comprehensive view of the data.

```
#Grouping by multiple columns
grouped_continent_contry = gdp_lifeExp_data.groupby(['continent', 'country'])["lifeExp"].agg
```

```
grouped_continent_contry
```

continent	country
Europe	Iceland
	Sweden
	Norway
	Netherlands
	Switzerland
...	...
Africa	Mozambique
	Guinea-Bissau
	Angola
Asia	Afghanistan
Africa	Sierra Leone

## 15.8.2 Understanding Hierarchical (Multi-Level) Indexing

- Grouping by multiple columns creates a hierarchical index (also called a multi-level index).
- This index allows each level (e.g., continent, country) to act as an independent category that can be accessed individually.

In the above output, `continent` and `country` form a two-level hierarchical index, allowing us to drill down from continent-level to country-level summaries.

```
grouped_continent_contry.index.nlevels
```

```
2
```

```
get the first level of the index
grouped_continent_contry.index.levels[0]
```

```
Index(['Africa', 'Americas', 'Asia', 'Europe', 'Oceania'], dtype='object', name='continent')
```

```
get the second level of the index
grouped_continent_contry.index.levels[1]
```

```
Index(['Afghanistan', 'Albania', 'Algeria', 'Angola', 'Argentina', 'Australia',
 'Austria', 'Bahrain', 'Bangladesh', 'Belgium',
 ...,
 'Uganda', 'United Kingdom', 'United States', 'Uruguay', 'Venezuela',
 'Vietnam', 'West Bank and Gaza', 'Yemen, Rep.', 'Zambia', 'Zimbabwe'],
 dtype='object', name='country', length=142)
```

### 15.8.3 Subsetting Data in a Hierarchical Index

`grouped_continent_country` is still a `DataFrame` with hierarchical indexing. You can use `.loc[]` for subsetting, just as you would with a single-level index.

```
get the observations for the 'Americas' continent
grouped_continent_contry.loc['Americas'].head()
```

	mean	std	max	min
country				
Canada	74.902750	3.952871	80.653	68.750
United States	73.478500	3.343781	78.242	68.440
Puerto Rico	72.739333	3.984267	78.746	64.280
Cuba	71.045083	6.022798	78.273	59.421
Uruguay	70.781583	3.342937	76.384	66.071

```
get the mean life expectancy for the 'Americas' continent
grouped_continent_contry.loc['Americas']['mean'].head()
```

```
country
Canada 74.902750
United States 73.478500
Puerto Rico 72.739333
Cuba 71.045083
Uruguay 70.781583
Name: mean, dtype: float64
```

```
another way to get the mean life expectancy for the 'Americas' continent
grouped_continent_contry.loc['Americas', 'mean'].head()
```

```
country
Canada 74.902750
United States 73.478500
Puerto Rico 72.739333
Cuba 71.045083
Uruguay 70.781583
Name: mean, dtype: float64
```

You can use a tuple to access data for specific levels in a multi-level index.

```
get the observations for the 'United States' country
grouped_continent_contry.loc[('Americas', 'United States')]
```

```
mean 73.478500
std 3.343781
max 78.242000
min 68.440000
Name: (Americas, United States), dtype: float64
```

```
grouped_continent_contry.loc[('Americas', 'United States'), ['mean', 'std']]
```

```
mean 73.478500
std 3.343781
Name: (Americas, United States), dtype: float64
```

```
gdp_lifeExp_data.columns
```

```
Index(['country', 'continent', 'year', 'lifeExp', 'pop', 'gdpPercap'], dtype='object')
```

Finally, you can use `reset_index()` to convert the hierarchical index into a regular index, allowing you to apply the standard subsetting and filtering methods covered in previous chapters

```
grouped_continent_contry.reset_index().head()
```

	continent	country	mean	std	max	min
0	Europe	Iceland	76.511417	3.026593	81.757	72.49
1	Europe	Sweden	76.177000	3.003990	80.884	71.86
2	Europe	Norway	75.843000	2.423994	80.196	72.67
3	Europe	Netherlands	75.648500	2.486363	79.762	72.13
4	Europe	Switzerland	75.565083	4.011572	81.701	69.62

### 15.8.4 Grouping by multiple columns and aggregating multiple variables

```
#Grouping by multiple columns
grouped_continent_contry_multi = gdp_lifeExp_data.groupby(['continent', 'country', 'year'])[
grouped_continent_contry_multi
```

continent	country
Africa	Algeria
...	...
Oceania	New Zealand

#### Breaking Down Grouping and Aggregation

- **Grouping by Multiple Columns:**  
In this example, we are grouping the data by three columns: `continent`, `country`, and `year`. This creates groups based on unique combinations of these columns.
- **Aggregating Multiple Variables:**  
We apply multiple aggregation functions (`mean`, `std`, `max`, and `min`) to multiple variables (`lifeExp`, `pop`, and `gdpPercap`).



This type of operation is commonly referred to as “**multi-column grouping with multiple aggregations**” in pandas. It’s a powerful approach because it allows us to obtain a detailed statistical summary for each combination of grouping columns across several variables.

```
its columns are also two levels deep
grouped_continent_contry_multi.columns.nlevels
```

2

```
pass a tuple to the loc() method to access the values of the multi-level columns with a mu.
grouped_continent_contry_multi.loc[('Americas','United States'), ('lifeExp', 'mean')]
```

```
year
1952 68.440
1957 69.490
1962 70.210
1967 70.760
1972 71.340
1977 73.380
1982 74.650
1987 75.020
1992 76.090
1997 76.810
2002 77.310
2007 78.242
Name: (lifeExp, mean), dtype: float64
```

## 15.9 Advanced Operations within groups: `apply()`, `transform()`, and `filter()`

### 15.9.1 Using `apply()` on groups

The `apply()` function applies a custom function to each group, allowing for flexible operations. The function can return either a scalar, Series, or DataFrame.

**Example:** Consider the life expectancy dataset, find the top 3 life expectancy values for each continent

We’ll first define a function that sorts a dataset by decreasing life expectancy and returns the top 3 rows. Then, we’ll apply this function on each group using the `apply()` method of the *GroupBy* object.

```
Define a function to get the top 3 rows based on life expectancy for each group
def top_3_life_expectancy(group):
 return group.nlargest(3, 'lifeExp')
```

```
#Defining the groups in the data
grouped_gdpcapital_data = gdp_lifeExp_data.groupby('continent')
```

Now we'll use the `apply()` method to apply the `top_3_life_expectancy()` function on each group of the object `grouped_gdpcapital_data`.

```
Apply the function to each continent group
top_life_expectancy = gdp_lifeExp_data.groupby('continent')[['continent', 'country', 'year',
Display the result
top_life_expectancy.head()
```

	continent	country	year	lifeExp	gdpPercap
0	Africa	Reunion	2007	76.442	7670.122558
1	Africa	Reunion	2002	75.744	6316.165200
2	Africa	Reunion	1997	74.772	6071.941411
3	Americas	Canada	2007	80.653	36319.235010
4	Americas	Canada	2002	79.770	33328.965070

The `top_3_life_expectancy()` function is applied to each group, and the results are concatenated internally with the `concat()` function. The output therefore has a hierarchical index whose outer level indices are the group keys.

We can also use a lambda function instead of separately defining the function `top_3_life_expectancy()`:

```
Use a lambda function to get the top 3 life expectancy values for each continent
top_life_expectancy = (
 gdp_lifeExp_data
 .groupby('continent')[['continent', 'country', 'year', 'lifeExp', 'gdpPercap']] # Avoid
 .apply(lambda x: x.nlargest(3, 'lifeExp'))
 .reset_index(drop=True)
)

Display the result
top_life_expectancy.head()
```

	continent	country	year	lifeExp	gdpPercap
0	Africa	Reunion	2007	76.442	7670.122558
1	Africa	Reunion	2002	75.744	6316.165200
2	Africa	Reunion	1997	74.772	6071.941411
3	Americas	Canada	2007	80.653	36319.235010
4	Americas	Canada	2002	79.770	33328.965070

## 15.9.2 Using `transform()` on Groups

The `transform()` function applies a function to each group and returns a Series aligned with the original DataFrame's index. This makes it suitable for adding or modifying columns based on group-level calculations.

Recall that in the data cleaning and preparation chapter, we imputed missing values based on correlated variables in the dataset.

In the example provided, some countries had missing values for GDP per capita. To handle this, we imputed the missing GDP per capita for each country using the average GDP per capita of its corresponding continent.

Now, we'll explore an alternative approach using `groupby()` and `transform()` to perform this imputation.

Let us read the datasets and the function that makes a visualization to compare the imputed values with the actual values.

```
#Importing data with missing values
gdp_missing_data = pd.read_csv('./Datasets/GDP_missing_data.csv')

#Importing data with all values
gdp_complete_data = pd.read_csv('./Datasets/GDP_complete_data.csv')

Index of rows with missing values for GDP per capita
null_ind_gdpPC = gdp_missing_data.index[gdp_missing_data.gdpPerCapita.isnull()]

Define a function to plot and evaluate imputed values vs actual values
def plot_actual_vs_predicted(y, title_suffix=""):
 """
 Plot imputed vs actual GDP per capita values and display RMSE.

 Parameters:
 - y: DataFrame with imputed gdpPerCapita values
```

```

- title_suffix: Optional string to add to the plot (e.g., method name)
"""
fig, ax = plt.subplots(figsize=(8, 6))

Extract actual and imputed values
x = gdp_complete_data.loc[null_ind_gdpPC, 'gdpPerCapita']
y_imputed = y.loc[null_ind_gdpPC, 'gdpPerCapita']

Create scatter plot
ax.scatter(x, y_imputed, alpha=0.6, s=50)

Add perfect prediction line (45-degree line)
ax.plot(x, x, color='orange', linewidth=2, label='Perfect imputation')

Labels and formatting
ax.set_xlabel('Actual GDP per capita', fontsize=14)
ax.set_ylabel('Imputed GDP per capita', fontsize=14)
ax.xaxis.set_major_formatter('${x:,.0f}')
ax.yaxis.set_major_formatter('${x:,.0f}')
ax.tick_params(labelsize=12)
ax.grid(True, alpha=0.3)
ax.legend(fontsize=11)

Calculate and display RMSE
rmse = np.sqrt(np.mean((y_imputed - x)**2))

Position text dynamically based on data range
x_pos = x.min() + (x.max() - x.min()) * 0.05
y_pos = y_imputed.max() - (y_imputed.max() - y_imputed.min()) * 0.1
ax.text(x_pos, y_pos, f'RMSE = ${rmse:,.2f}',
 fontsize=13, bbox=dict(boxstyle='round', facecolor='wheat', alpha=0.5))

plt.tight_layout()
plt.show()

return rmse

```

### Approach 1: Using the ‘loc’

```

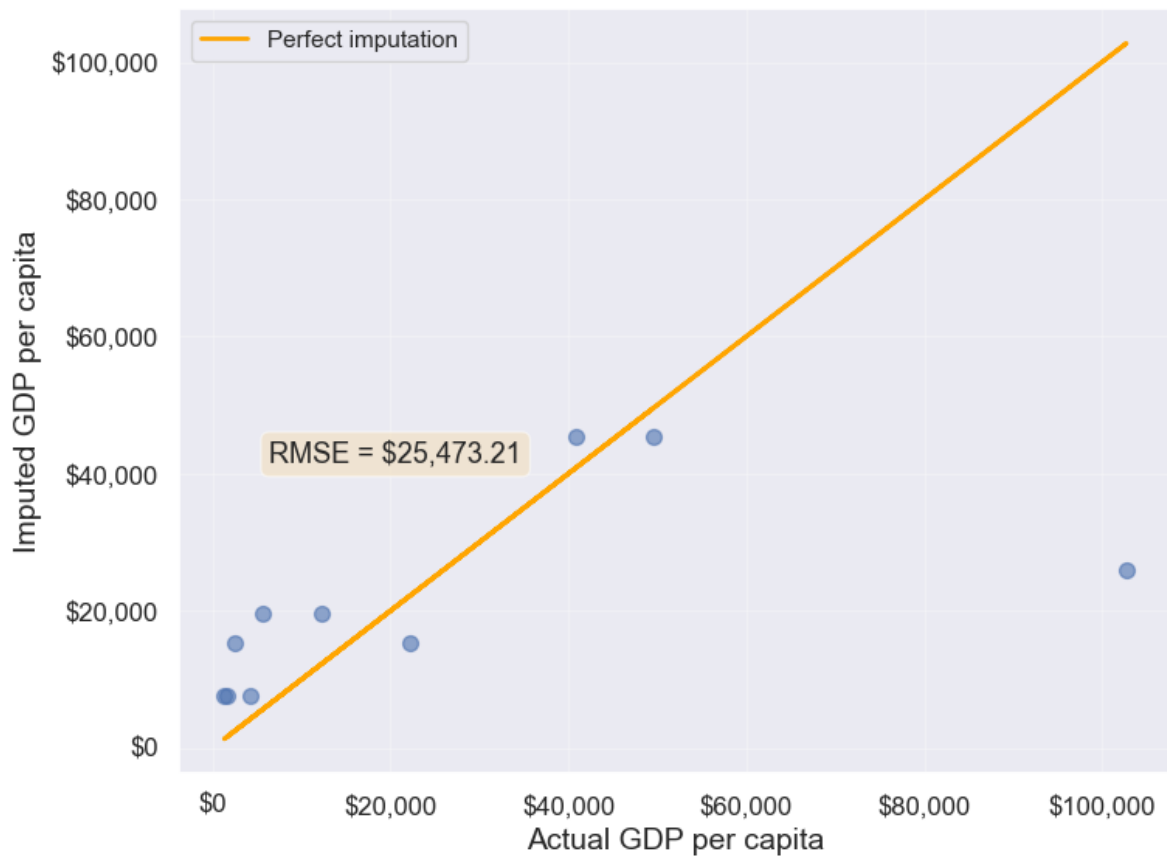
Finding the mean GDP per capita of the continent
avg_gdpPerCapita = gdp_missing_data['gdpPerCapita'].groupby(gdp_missing_data['continent']).m

```

```
Creating a copy of missing data to impute missing values
gdp_imputed_data = gdp_missing_data.copy()

Replacing missing GDP per capita with the mean GDP per capita for the corresponding continent
for cont in avg_gdpPerCapita.index:
 gdp_imputed_data.loc[(gdp_imputed_data.continent==cont) & (gdp_imputed_data.gdpPerCapita.isnull())
]['gdpPerCapita'] = avg_gdpPerCapita[cont]

Plot the actual vs predicted values
plot_actual_vs_predicted(gdp_imputed_data)
```



25473.20645170116

**Approach 2:** Using the `groupby()` and `transform()` methods.

The `transform()` function is a powerful tool for filling missing values in grouped data. It allows us to apply a function across each group and align the result back to the original DataFrame, making it perfect for filling missing values based on group statistics.

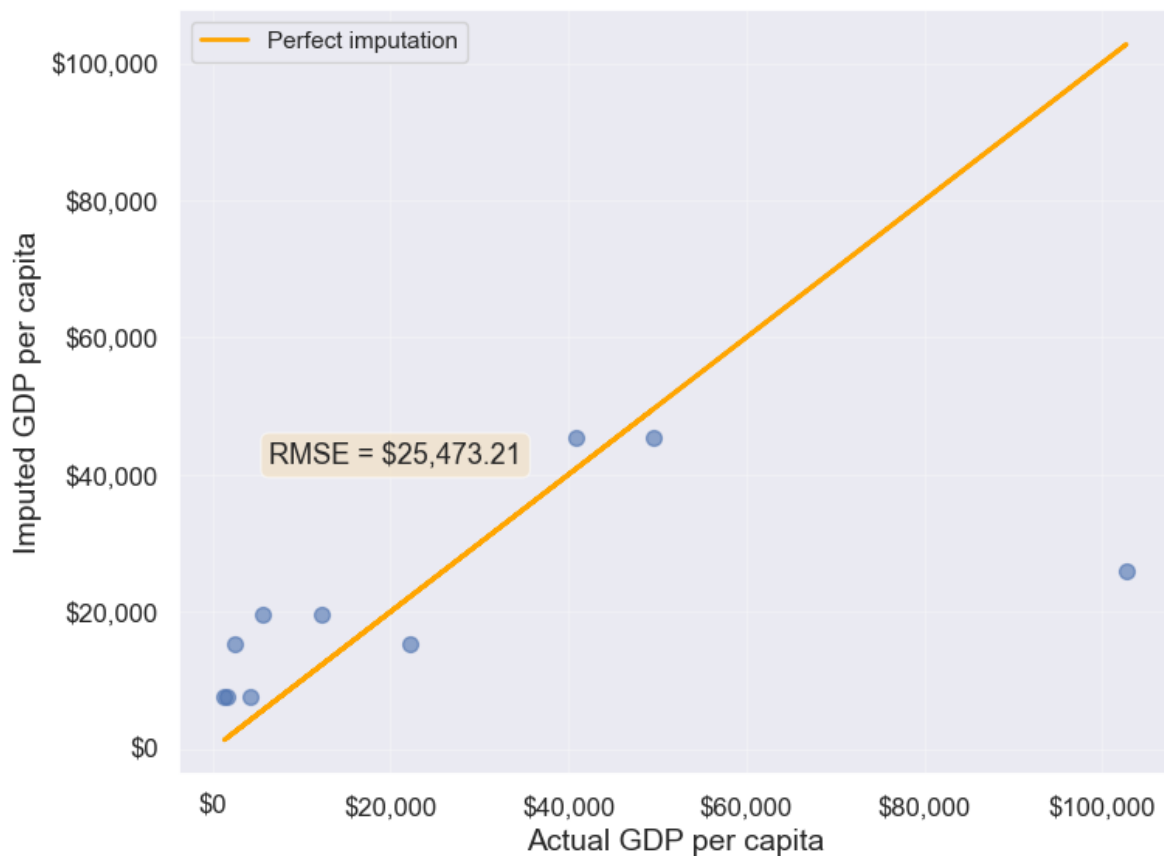
In this example, we use `transform()` to impute missing values in the `gdpPerCapita` column by filling them with the mean `gdpPerCapita` of each continent:

```
Creating a copy of missing data to impute missing values
gdp_imputed_data = gdp_missing_data.copy()

Grouping data by continent
grouped = gdp_missing_data.groupby('continent')

Imputing missing values with the mean GDP per capita of the continent
gdp_imputed_data['gdpPerCapita'] = grouped['gdpPerCapita'].transform(lambda x: x.fillna(x.mean()))

Plot the actual vs predicted values
plot_actual_vs_predicted(gdp_imputed_data)
```



25473.20645170116

Using the `transform()` function, missing values in `gdpPerCapita` for each group are filled with the group's mean `gdpPerCapita`. This approach is not only more convenient to write but also faster compared to using for loops. While a for loop imputes missing values one group at a time, `transform()` performs built-in operations (like mean, sum, etc.) in a way that is optimized internally, making it more efficient.

Let's use `apply()` instead of `transform()` with `groupby()`

Please copy the code below and run it in your notebook:

```
#Creating a copy of missing data to impute missing values
gdp_imputed_data = gdp_missing_data.copy()

#Grouping data by continent
grouped = gdp_missing_data.groupby('continent')

#Applying the lambda function on the 'gdpPerCapita' column of the groups
gdp_imputed_data['gdpPerCapita'] = grouped['gdpPerCapita'].apply(lambda x: x.fillna(x.mean()))

plot_actual_vs_predicted()
```

Why we ran into this error? and `apply()` doesn't work?

Here's a deeper look at why `apply()` doesn't work as expected here:

#### 15.9.2.1 Behavior of `groupby().apply()` vs. `groupby().transform()`

- **`groupby().apply()`:** This method applies a function to each group and returns the result with a hierarchical (multi-level) index by default. This hierarchical index can make it difficult to align the result back to a single column in the original DataFrame.
- **`groupby().transform()`:** In contrast, `transform()` is specifically designed to apply a function to each group and return a Series that is aligned with the original DataFrame's index. This alignment makes it directly compatible for assignment to a new or existing column in the original DataFrame.

### 15.9.2.2 Why transform() Works for Imputation

When using `transform()` to fill missing values, it applies the function (e.g., `fillna(x.mean())`) based on each group's statistics, such as the mean, while keeping the result aligned with the original DataFrame's index. This allows for smooth assignment to a column in the DataFrame without any index mismatch issues.

Additionally, `transform()` applies the function to each element in a group independently and returns a result that has the same shape as the original data, making it ideal for adding or modifying columns.

### 15.9.3 Using filter() on Groups

The `filter()` function filters entire groups based on a condition. It evaluates each group and keeps only those that meet the specified criteria.

Example: Keep only the countries where the mean life expectancy is greater than 70

```
keep only the continent where the mean life expectancy is greater than 74
gdp_lifeExp_data.groupby('continent').filter(lambda x: x['lifeExp'].mean() > 74)['continent']
```

```
array(['Oceania'], dtype=object)
```

```
keep only the country where the mean life expectancy is greater than 74
gdp_lifeExp_data.groupby('country').filter(lambda x: x['lifeExp'].mean() > 74)['country'].un
```

```
array(['Australia', 'Canada', 'Denmark', 'France', 'Iceland', 'Italy',
 'Japan', 'Netherlands', 'Norway', 'Spain', 'Sweden', 'Switzerland'],
 dtype=object)
```

Using `.nunique()` get the number of countries that satisfy this condition

```
gdp_lifeExp_data.groupby('country').filter(lambda x: x['lifeExp'].mean() > 74)['country'].nu
```

12



## 15.10 Sampling data by group

If a dataset contains a large number of observations, operating on it can be computationally expensive. Instead, working on a sample of entire observations is a more efficient alternative. The `groupby()` method combined with `apply()` can be used for stratified random sampling from a large dataset.

Before taking the random sample, let us find the number of countries in each continent.

```
gdp_lifeExp_data.continent.value_counts()
```

```
continent
Africa 624
Asia 396
Europe 360
Americas 300
Oceania 24
Name: count, dtype: int64
```

Let us take a random sample of 650 observations from the entire dataset.

```
sample_lifeExp_data = gdp_lifeExp_data.sample(650)
```

Now, let us see the number of countries of each continent in our sample.

```
sample_lifeExp_data.continent.value_counts()
```

```
continent
Africa 236
Asia 154
Europe 136
Americas 113
Oceania 11
Name: count, dtype: int64
```

Some of the continent have a very low representation in the data. To rectify this, we can take a random sample of 130 observations from each of the 5 continents. In other words, we can take a random sample from each of the continent-based groups.

```
evenly_sampled_lifeExp_data = gdp_lifeExp_data.groupby('continent').apply(lambda x:x.sample(
group_sizes = evenly_sampled_lifeExp_data.groupby(level=0).size()
print(group_sizes)
```

```
continent
Africa 130
Americas 130
Asia 130
Europe 130
Oceania 130
dtype: int64
```

The above stratified random sample equally represents all the continent.

## 15.11 `corr()`: Correlation by group

The `corr()` method of the *GroupBy* object returns the correlation between all pairs of columns within each group.

**Example:** Find the correlation between `lifeExp` and `gdpPerCap` for each continent-country level combination.

```
gdp_lifeExp_data.groupby(['continent','country']).apply(lambda x:x['lifeExp'].corr(x['gdpPer
```

```
continent country
Africa Algeria 0.904471
 Angola -0.301079
 Benin 0.843949
 Botswana 0.005597
 Burkina Faso 0.881677
 ...
Europe Switzerland 0.980715
 Turkey 0.954455
 United Kingdom 0.989893
Oceania Australia 0.986446
 New Zealand 0.974493
Length: 142, dtype: float64
```

Life expectancy is closely associated with GDP per capita across most continent-country combinations.

## 15.12 Independent Study

### 15.12.1 Practice exercise 1

Read the spotify dataset from `spotify_data.csv` that contains information about tracks and artists

#### 15.12.1.1

Find the mean and standard deviation of the track popularity for each genre.

#### 15.12.1.2

Create a new categorical column, `energy_lvl`, with two levels – ‘Low energy’ and ‘High energy’ – using equal-sized bins based on the track’s energy level. Then, calculate the mean, standard deviation, and 90th percentile of track popularity for each genre and energy level combination

#### 15.12.1.3

Find the mean and standard deviation of track popularity and danceability for each genre and energy level. What insights you can gain from the generated table

#### 15.12.1.4

For each genre and energy level, find the mean and standard deviation of the track popularity, and the minimum and maximum values of loudness.

#### 15.12.1.5

Find the most popular artist from each genre.

#### 15.12.1.6

Filter the first 4 columns of the spotify dataset. Drop duplicate observations in the resulting dataset using the Pandas DataFrame method `drop_duplicates()`. Find the top 3 most popular artists for each genre.

#### 15.12.1.7

The spotify dataset has more than 200k observations. It may be expensive to operate with so many observations. Take a random sample of 650 observations to analyze spotify data, such that all genres are equally represented.

#### 15.12.1.8

Find the correlation between `danceability` and `track popularity` for each genre-energy level combination.

#### 15.12.1.9

Find the number of observations in each group, where each groups corresponds to a distinct genre-energy lvl combination

#### 15.12.1.10

Find the percentage of observations in each group of the above table.

#### 15.12.1.11

What percentage of unique tracks are contributed by the top 5 artists of each genre?

**Hint:** Find the top 5 artists based on `artist_popularity` for each genre. Count the total number of unique tracks (`track_name`) contributed by these artists. Divide this number by the total number of unique tracks in the data. The `nunique()` function will be useful.

# 16 Data Reshaping

<IPython.core.display.HTML object>

## 16.1 Introduction to Data Reshaping

Data reshaping involves transforming the **structure** of your data without changing the underlying information. Think of it like rearranging furniture in a room—the furniture stays the same, but its organization changes to suit different purposes.

In data analysis, the same dataset can be organized in different ways:

- **Wide format** spreads variables across multiple columns
- **Long format** stacks variables into fewer columns with more rows

Neither format is inherently “better”—the right format depends on what you’re trying to accomplish.

### 16.1.1 Why Data Reshaping Matters

#### 1. Different Tools Require Different Formats

- **Visualization libraries** (like seaborn and matplotlib) often prefer long format for grouped plotting
- **Statistical models** may require wide format with one row per subject
- **Machine learning algorithms** typically need wide format where each row is an observation and columns are features

#### 2. Facilitates Specific Analyses

- **Time series comparisons** are easier in wide format (comparing 2007 vs 1957 life expectancy)
- **Grouped aggregations** work naturally with long format
- **Cross-sectional analysis** benefits from wide format

#### 3. Simplifies Data Operations

- Some calculations are trivial in one format but complex in another
- Merging datasets often requires matching formats
- Creating summary tables may need pivoting

### 16.1.2 Understanding Wide vs. Long Format

Let's use a concrete example to understand these formats. Imagine we have life expectancy data for three countries across three years.

```
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt

%matplotlib inline

Create a simple example dataset in LONG format
long_data = pd.DataFrame({
 'Country': ['USA', 'USA', 'USA', 'Canada', 'Canada', 'Canada', 'Mexico', 'Mexico', 'Mexico'],
 'Year': [2000, 2005, 2010, 2000, 2005, 2010, 2000, 2005, 2010],
 'LifeExp': [76.8, 77.5, 78.5, 79.2, 80.1, 81.0, 74.1, 75.2, 76.5]
})

print("LONG FORMAT (Tidy Data):")
print("="*50)
print(long_data)
print("\nShape:", long_data.shape)
print("Characteristics: More rows, fewer columns, each observation is one row")
```

LONG FORMAT (Tidy Data):

```
=====
```

	Country	Year	LifeExp
0	USA	2000	76.8
1	USA	2005	77.5
2	USA	2010	78.5
3	Canada	2000	79.2
4	Canada	2005	80.1
5	Canada	2010	81.0
6	Mexico	2000	74.1
7	Mexico	2005	75.2
8	Mexico	2010	76.5

Shape: (9, 3)

Characteristics: More rows, fewer columns, each observation is one row

```
Convert to WIDE format
wide_data = long_data.pivot(index='Country', columns='Year', values='LifeExp')

print("\nWIDE FORMAT:")
print("="*50)
print(wide_data)
print("\nShape:", wide_data.shape)
print("Characteristics: Fewer rows, more columns, years spread across columns")
```

WIDE FORMAT:

```
=====
Year 2000 2005 2010
Country
Canada 79.2 80.1 81.0
Mexico 74.1 75.2 76.5
USA 76.8 77.5 78.5
```

Shape: (3, 3)

Characteristics: Fewer rows, more columns, years spread across columns

### Key Observations:

Format	Rows	Columns	Use Case
<b>Long</b>	9	3	Better for plotting with seaborn, groupby operations, statistical modeling
<b>Wide</b>	3	3 (+index)	Better for comparing across years, reading tables, some statistical tests

Same data, different organization!

### 16.1.3 Reshaping Tools in Pandas

Pandas provides several functions to reshape data between wide and long formats:

Function	Direction	Primary Use
<code>pivot_table()</code>	Long → Wide	Create summary tables with aggregation
<code>melt()</code>	Wide → Long	Unpivot columns into rows
<code>stack()</code>	Wide → Long	Move column index to row index (works with MultiIndex)
<code>unstack()</code>	Long → Wide	Move row index to column index (works with MultiIndex)

In this chapter, we'll explore each method with practical examples using the *gdp\_lifeExpectancy.csv* dataset.

Let's start by reading the CSV file into a pandas DataFrame.

```
Load the data
gdp_lifeExp_data = pd.read_csv('./Datasets/gdp_lifeExpectancy.csv')
gdp_lifeExp_data.head()
```

	country	continent	year	lifeExp	pop	gdpPercap
0	Afghanistan	Asia	1952	28.801	8425333	779.445314
1	Afghanistan	Asia	1957	30.332	9240934	820.853030
2	Afghanistan	Asia	1962	31.997	10267083	853.100710
3	Afghanistan	Asia	1967	34.020	11537966	836.197138
4	Afghanistan	Asia	1972	36.088	13079460	739.981106

## 16.2 Pivoting from Long to Wide: `pivot_table()`

The `pivot_table()` function is one of pandas' most versatile tools—it serves a dual purpose as both an **aggregation tool** and a **data reshaping tool**. While you may have seen pivot tables in Excel for summarizing data, pandas' `pivot_table()` is even more powerful.



### 16.2.1 What is Pivoting?

**Pivoting** transforms long format data into wide format by:

- Taking values from rows and spreading them **across columns**
- Creating a **cross-tabulation** structure that's easy to read
- Automatically **aggregating** when there are multiple values per group

Think of it as “pivoting” your data 90 degrees—what was vertical becomes horizontal.

Let's start with a simple example before moving to more complex scenarios.

### 16.2.2 Example 1: Simple Pivot with Single Index

Let's start by pivoting life expectancy data to compare specific years. We'll create a table where continents are rows and years are columns.

```
Simple pivot: continents as rows, years as columns
simple_pivot = gdp_lifeExp_data.pivot_table(
 index='continent', # Rows
 columns='year', # Columns
 values='lifeExp', # Values to display
 aggfunc='mean' # How to aggregate (mean life expectancy)
)

print("Mean Life Expectancy by Continent and Year:")
simple_pivot
```

Mean Life Expectancy by Continent and Year:

year	1952	1957	1962	1967	1972	1977	1982	1987	1992
continent									
Africa	39.135500	41.266346	43.319442	45.334538	47.450942	49.580423	51.592865	53.344788	55.354678
Americas	53.279840	55.960280	58.398760	60.410920	62.394920	64.391560	66.228840	68.090720	69.851680
Asia	46.314394	49.318544	51.563223	54.663640	57.319269	59.610556	62.617939	64.851182	66.981122
Europe	64.408500	66.703067	68.539233	69.737600	70.775033	71.937767	72.806400	73.642167	74.425467
Oceania	69.255000	70.295000	71.085000	71.310000	71.910000	72.855000	74.290000	75.320000	76.140000

**Understanding the Parameters:**

- **index='continent'** - Values from this column become row labels

- `columns='year'` - Unique values from this column become column headers
- `values='lifeExp'` - The data to display in the table cells
- `aggfunc='mean'` - Since multiple countries exist per continent-year, we average them

**Result:** Each cell shows the mean life expectancy for a continent in a specific year. This wide format makes it easy to compare across years (reading horizontally across a row).

### 16.2.3 Example 2: Multi-Level Index (Hierarchical Pivot)

Now let's create a more detailed pivot table with both continent and country as row indices. This creates a hierarchical structure that allows drill-down analysis.

```
Multi-level pivot: both continent and country as row indices
gdp_lifeExp_data_pivot = gdp_lifeExp_data.pivot_table(
 index=['continent', 'country'], # Two-level row index
 columns='year', # Years as columns
 values='lifeExp', # Life expectancy values
 aggfunc='mean' # Aggregation function
)

Display first 10 rows
gdp_lifeExp_data_pivot.head(10)
```

continent	year country
Africa	Algeria
	Angola
	Benin
	Botswana
	Burkina Faso
	Burundi
	Cameroon
	Central African
	Chad
	Comoros

#### Understanding Multi-Level Indices:

When you pass a list to the `index` parameter, pandas creates a **hierarchical (multi-level) index**:

- **Level 0** (outermost): `continent` - Groups countries by continent
- **Level 1** (inner): `country` - Individual countries within each continent

#### Benefits of this structure:

- Easy to see all years for a country in one row
- Natural grouping by continent
- Facilitates comparisons across time periods

### 16.2.4 Practical Use: Temporal Comparisons

With years as columns, comparing life expectancy across different time periods becomes trivial. Let's visualize how life expectancy changed over 50 years (1957 vs 2007).

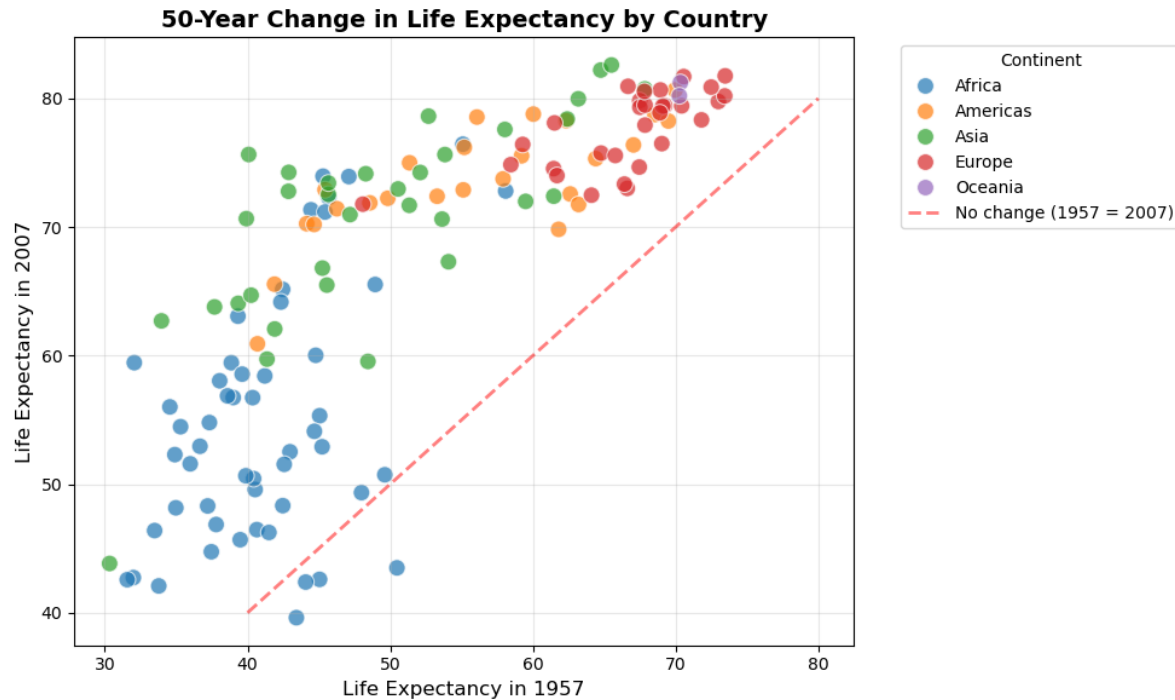
```
Create a scatter plot comparing 1957 vs 2007 life expectancy
plt.figure(figsize=(10, 6))

Add continent information for coloring
Reset index to access 'continent' column for hue parameter
plot_data = gdp_lifeExp_data_pivot.reset_index()

Create scatter plot
sns.scatterplot(data=plot_data, x=1957, y=2007, hue='continent', s=100, alpha=0.7)

Add diagonal reference line (y = x) showing where 1957 = 2007
plt.plot([40, 80], [40, 80], 'r--', linewidth=2, label='No change (1957 = 2007)', alpha=0.5)

Labels and formatting
plt.xlabel('Life Expectancy in 1957', fontsize=12)
plt.ylabel('Life Expectancy in 2007', fontsize=12)
plt.title('50-Year Change in Life Expectancy by Country', fontsize=14, fontweight='bold')
plt.legend(title='Continent', bbox_to_anchor=(1.05, 1), loc='upper left')
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```



### Interpreting the Visualization:

- **Points above the red line:** Countries where life expectancy **increased** from 1957 to 2007 (most countries)
- **Points below the red line:** Countries where life expectancy **decreased** (concerning trend worth investigating)
- **Distance from the line:** Magnitude of change over the 50-year period

**Key Insight:** A few African countries show decreased life expectancy after 50 years, likely due to factors like HIV/AIDS epidemic, conflicts, or healthcare system challenges. This pattern would be harder to spot without the wide format pivot table!

## 16.2.5 Visualizing Pivot Tables with Heatmaps

Another powerful way to visualize pivot tables is using **heatmaps**, which represent the table as a color-coded matrix. This makes it easy to spot patterns, trends, and outliers at a glance.

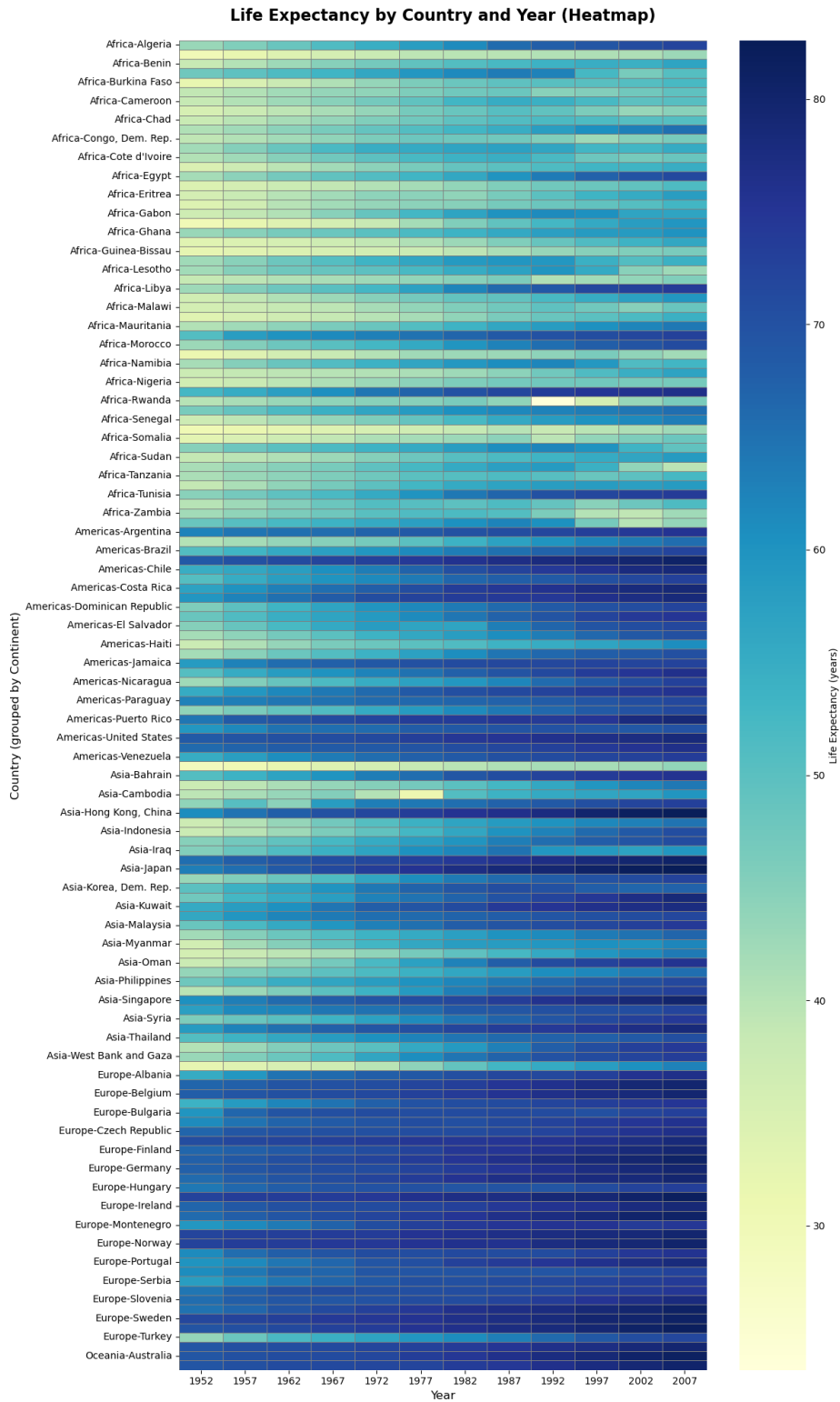
```
Create a heatmap of the pivot table
plt.figure(figsize=(12, 20))
plt.title("Life Expectancy by Country and Year (Heatmap)", fontsize=16, fontweight='bold', p
Create heatmap with better formatting
```

```

sns.heatmap(
 gdp_lifeExp_data_pivot,
 cmap='YlGnBu', # Yellow-Green-Blue color scheme
 cbar_kws={'label': 'Life Expectancy (years)'},
 linewidths=0.5, # Add grid lines
 linecolor='gray',
 fmt='.1f'
)

plt.xlabel('Year', fontsize=12)
plt.ylabel('Country (grouped by Continent)', fontsize=12)
plt.tight_layout()
plt.show()

```



## Reading the Heatmap:

- **Darker (blue) colors** indicate higher life expectancy
- **Lighter (yellow) colors** indicate lower life expectancy
- **Horizontal patterns** show trends over time for individual countries
- **Vertical patterns** show disparities across countries in a specific year

## Patterns to Notice:

1. General darkening from left to right = life expectancy increasing over time globally
2. Top rows (Oceania, Europe) consistently darker = higher life expectancy
3. Bottom rows (Africa) show more yellow = lower life expectancy, though improving

## Common Aggregation Functions in `pivot_table()`

You can use various aggregation functions within `pivot_table()` to summarize data:

- **mean** – Calculates the average of values within each group.
- **sum** – Computes the total of values within each group.
- **count** – Counts the number of non-null entries within each group.
- **min** and **max** – Finds the minimum and maximum values within each group.

We can also define our own custom aggregation function and pass it to the `aggfunc` parameter of the `pivot_table()` function

**For Example:** Find the 90<sup>th</sup> percentile of life expectancy for each country and year combination

```
pd.pivot_table(data = gdp_lifeExp_data, values = 'lifeExp', index = 'country', columns = 'year'
```

year country	1952	1957	1962	1967	1972	1977	1982	1987	1992	1997
Afghanistan	28.801	30.332	31.997	34.020	36.088	38.438	39.854	40.822	41.674	41.763
Albania	55.230	59.280	64.820	66.220	67.690	68.930	70.420	72.000	71.581	72.950
Algeria	43.077	45.685	48.303	51.407	54.518	58.014	61.368	65.799	67.744	69.152
Angola	30.015	31.999	34.000	35.985	37.928	39.483	39.942	39.906	40.647	40.963
Argentina	62.485	64.399	65.142	65.634	67.065	68.481	69.942	70.774	71.868	73.275
...	...	...	...	...	...	...	...	...	...	...
Vietnam	40.412	42.887	45.363	47.838	50.254	55.764	58.816	62.820	67.662	70.672
West Bank and Gaza	43.160	45.671	48.127	51.631	56.532	60.765	64.406	67.046	69.718	71.096
Yemen, Rep.	32.548	33.970	35.180	36.984	39.848	44.175	49.113	52.922	55.599	58.020
Zambia	42.038	44.077	46.023	47.768	50.107	51.386	51.821	50.821	46.100	40.238
Zimbabwe	48.451	50.469	52.358	53.995	55.635	57.674	60.363	62.351	60.377	46.809

Example: Consider the life expectancy dataset, calculate the average life expectancy for each country and year combination

```
pd.pivot_table(data = gdp_lifeExp_data, values = 'lifeExp', index = 'country', columns = 'year'
```

year country	1952	1957	1962	1967	1972	1977	1982	1987
Afghanistan	28.80100	30.332000	31.997000	34.02000	36.088000	38.438000	39.854000	40.854000
Albania	55.23000	59.280000	64.820000	66.22000	67.690000	68.930000	70.420000	72.000000
Algeria	43.07700	45.685000	48.303000	51.40700	54.518000	58.014000	61.368000	65.714000
Angola	30.01500	31.999000	34.000000	35.98500	37.928000	39.483000	39.942000	39.942000
Argentina	62.48500	64.399000	65.142000	65.63400	67.065000	68.481000	69.942000	70.714000
...	...	...	...	...	...	...	...	...
West Bank and Gaza	43.16000	45.671000	48.127000	51.63100	56.532000	60.765000	64.406000	67.000000
Yemen, Rep.	32.54800	33.970000	35.180000	36.98400	39.848000	44.175000	49.113000	52.942000
Zambia	42.03800	44.077000	46.023000	47.76800	50.107000	51.386000	51.821000	50.857000
Zimbabwe	48.45100	50.469000	52.358000	53.99500	55.635000	57.674000	60.363000	62.357000
All	49.05762	51.507401	53.609249	55.67829	57.647386	59.570157	61.533197	63.500000

## 16.3 Melting from Wide to Long: melt()

`melt()` is the inverse operation of pivoting—it transforms **wide format** data into **long format** by “unpivoting” columns into rows. This is also called “melting” because you’re “melting” wide columns down into a taller, narrower structure.

### 16.3.1 Why Melt Your Data?

#### Visualization Libraries Prefer Long Format:

- Seaborn and many plotting tools expect data in long format
- Makes it easy to create grouped plots with `hue`, `style`, or `col` parameters

#### Better for Statistical Analysis:

- Long format is “tidy data”—each variable in one column, each observation in one row
- Required for many statistical models and tests

#### More Flexible for groupby() Operations:

- Easier to filter and aggregate when variables are in rows rather than column names

Let’s see this in action!



### 16.3.2 Example 1: Basic Melt

Let's take our pivoted data (wide format) and melt it back to long format. First, let's look at what we're starting with:

```
Let's look at our wide format data (first 5 rows)
print(f"\nShape: {gdp_lifeExp_data_pivot.shape}")
print("Notice: Years are columns (wide), each country is one row")

print("WIDE FORMAT (pivoted data):")
print("="*60)
gdp_lifeExp_data_pivot.head()
```

Shape: (142, 12)

Notice: Years are columns (wide), each country is one row

WIDE FORMAT (pivoted data):

=====

continent	year	country
Africa		Algeria
		Angola
		Benin
		Botswana
		Burkina Faso

```
Melt the wide format back to long format
melted_data = gdp_lifeExp_data_pivot.melt(
 var_name='year', # Name for the column that will hold year values
 value_name='lifeExp', # Name for the column that will hold life expectancy values
 ignore_index=False # Keep the original index (continent, country)
)

print(f"\nShape: {melted_data.shape}")
print("Notice: Years are now rows (long), multiple rows per country")

print("\nLONG FORMAT (after melting):")
print("="*60)
melted_data.head(15) # Show more rows to see the pattern
```

Shape: (1704, 2)

Notice: Years are now rows (long), multiple rows per country

LONG FORMAT (after melting):

=====

continent	country
Africa	Algeria
	Angola
	Benin
	Botswana
	Burkina Faso
	Burundi
	Cameroon
	Central African Republic
	Chad
	Comoros
	Congo, Dem. Rep.
	Congo, Rep.
	Cote d'Ivoire
	Djibouti
	Egypt

## What Just Happened?

### 1. Wide Format (before melt):

- 142 rows  $\times$  12 columns (one column per year)
- Each country had one row with year values spread across columns

### 2. Long Format (after melt):

- 1,704 rows  $\times$  1 column (142 countries  $\times$  12 years = 1,704 rows)
- Each country-year combination gets its own row
- The `year` column now contains what were previously column headers
- The `lifeExp` column contains the actual values

## Key Parameters Used:

- `var_name='year'` - Column header values (1952, 1957, etc.) go into a new column called 'year'

- `value_name='lifeExp'` - Cell values go into a new column called 'lifeExp'
- `ignore_index=False` - Preserve the multi-level index (continent, country)

This is now in **tidy format**, perfect for plotting and analysis!

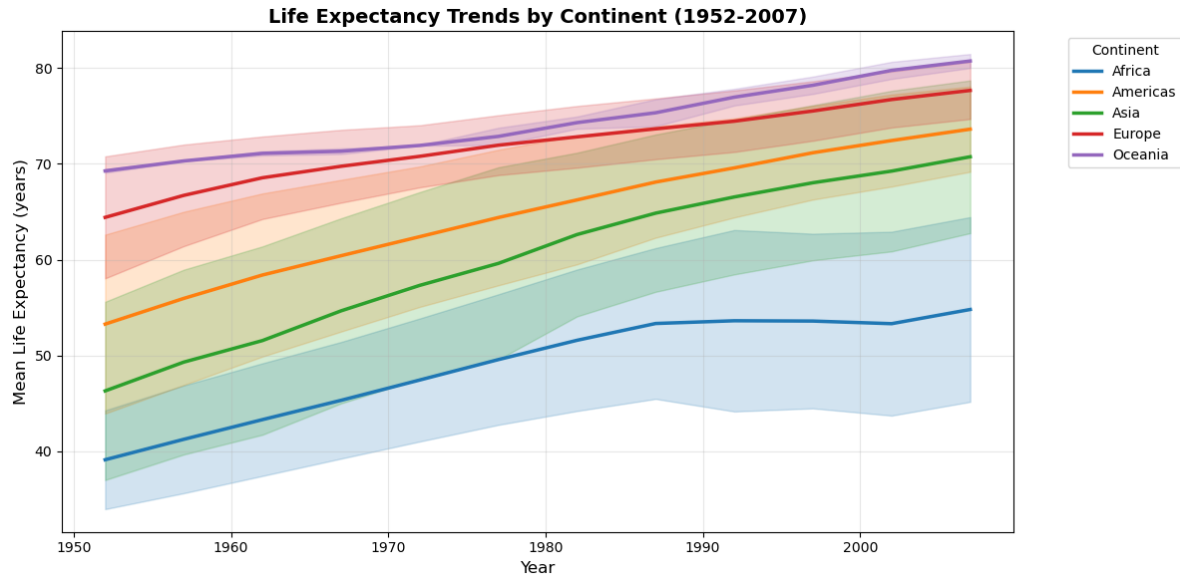
### 16.3.3 Example 2: Using Melted Data for Visualization

Now that we have data in long format, we can easily create time series plots grouped by continent. This would be much harder with wide format data!

```
Prepare data for plotting by resetting index to access continent column
plot_data = melted_data.reset_index()

Create a line plot showing life expectancy trends over time
plt.figure(figsize=(12, 6))
sns.lineplot(
 data=plot_data,
 x='year',
 y='lifeExp',
 hue='continent',
 estimator='mean', # Average across countries in each continent
 errorbar='sd', # Show standard deviation as error bars
 linewidth=2.5
)

plt.title('Life Expectancy Trends by Continent (1952-2007)', fontsize=14, fontweight='bold')
plt.xlabel('Year', fontsize=12)
plt.ylabel('Mean Life Expectancy (years)', fontsize=12)
plt.legend(title='Continent', bbox_to_anchor=(1.05, 1), loc='upper left')
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```



### Why This Works So Well:

This visualization would be extremely difficult with wide format data! With long format:

- Seaborn automatically groups by continent using the `hue` parameter
- The `x='year'` and `y='lifeExp'` mappings are straightforward
- Error bars show variation across countries within each continent

### Key Insights from the Plot:

- All continents show upward trends (global improvement in healthcare)
- Oceania maintains the highest life expectancy throughout
- Africa starts lowest but shows improvement
- The gap between continents has narrowed over time (error bars converge)

This is the power of having data in the right format!

### 16.3.4 Pivot Melt: The Full Circle

Let's verify that `pivot_table()` and `melt()` are truly inverse operations by starting with long format, pivoting to wide, then melting back to long:

```
Step 1: Start with original long format data
print("Step 1 - ORIGINAL (Long Format):")
print(f"Shape: {gdp_lifeExp_data.shape}")
print(gdp_lifeExp_data.head(3))
```

```

Step 2: Pivot to wide format
wide = gdp_lifeExp_data.pivot_table(
 index=['continent', 'country'],
 columns='year',
 values='lifeExp'
)
print("\n" + "="*60)
print("Step 2 - PIVOTED (Wide Format):")
print(f"Shape: {wide.shape}")
print(wide.head(3))

Step 3: Melt back to long format
long_again = wide.melt(var_name='year', value_name='lifeExp', ignore_index=False).reset_index()
print("\n" + "="*60)
print("Step 3 - MELTED BACK (Long Format):")
print(f"Shape: {long_again.shape}")
print(long_again.head(3))

print("\n We're back where we started! pivot_table() and melt() are inverses.")

```

Step 1 - ORIGINAL (Long Format):

Shape: (1704, 6)

	country	continent	year	lifeExp	pop	gdpPercap
0	Afghanistan	Asia	1952	28.801	8425333	779.445314
1	Afghanistan	Asia	1957	30.332	9240934	820.853030
2	Afghanistan	Asia	1962	31.997	10267083	853.100710

=====

Step 2 - PIVOTED (Wide Format):

Shape: (142, 12)

year			1952	1957	1962	1967	1972	1977	1982	\
continent	country									
Africa	Algeria		43.077	45.685	48.303	51.407	54.518	58.014	61.368	
	Angola		30.015	31.999	34.000	35.985	37.928	39.483	39.942	
	Benin		38.223	40.358	42.618	44.885	47.014	49.190	50.904	
year			1987	1992	1997	2002	2007			
continent	country									
Africa	Algeria		65.799	67.744	69.152	70.994	72.301			
	Angola		39.906	40.647	40.963	41.003	42.731			
	Benin		52.337	53.919	54.777	54.406	56.728			

=====

Step 3 - MELTED BACK (Long Format):

Shape: (1704, 4)

```
 continent country year lifeExp
0 Africa Algeria 1952 43.077
1 Africa Angola 1952 30.015
2 Africa Benin 1952 38.223
```

We're back where we started! `pivot_table()` and `melt()` are inverses.

## 16.4 Stacking and Unstacking using `stack()` and `unstack()`

Stacking and unstacking are powerful techniques used to reshape DataFrames by moving data between rows and columns. These operations are particularly useful when working with **multi-level index** or **multi-level columns**, where the data is structured in a hierarchical format.

- `stack()`: Converts columns into rows, which means it “stacks” the data into a long format.
- `unstack()`: Converts rows into columns, reversing the effect of `stack()`.

Let's use the GDP per capita dataset and create a DataFrame with a multi-level index (e.g., continent, country) and multi-level columns (e.g., lifeExp, gdpPercap, and pop).

```
calculate the average and standard deviation of life expectancy, population, and GDP per c
gdp_lifeExp_multilevel_data = gdp_lifeExp_data.groupby(['continent', 'country'])[['lifeExp',
gdp_lifeExp_multilevel_data
```

---

continent	country
Africa	Algeria
	Angola
	Benin
	Botswana
	Burkina Faso
...	...
Europe	Switzerland
	Turkey

continent	country
Oceania	United Kingdom
	Australia
	New Zealand

The resulting DataFrame has two levels of index and two levels of columns, as shown below:

```
check the level of row and column index
gdp_lifeExp_multilevel_data.index.nlevels
```

2

```
gdp_lifeExp_multilevel_data.columns.nlevels
```

2

if you want to see the data along with the index and columns (in a more accessible way), you can directly print the `.index` and `.columns` attributes.

```
gdp_lifeExp_multilevel_data.index
```

```
MultiIndex([('Africa', 'Algeria'),
 ('Africa', 'Angola'),
 ('Africa', 'Benin'),
 ('Africa', 'Botswana'),
 ('Africa', 'Burkina Faso'),
 ('Africa', 'Burundi'),
 ('Africa', 'Cameroon'),
 ('Africa', 'Central African Republic'),
 ('Africa', 'Chad'),
 ('Africa', 'Comoros'),
 ...
 ('Europe', 'Serbia'),
 ('Europe', 'Slovak Republic'),
 ('Europe', 'Slovenia'),
 ('Europe', 'Spain'),
 ('Europe', 'Sweden'),
```

```
('Europe', 'Switzerland'),
('Europe', 'Turkey'),
('Europe', 'United Kingdom'),
('Oceania', 'Australia'),
('Oceania', 'New Zealand')],
names=['continent', 'country'], length=142)
```

```
gdp_lifeExp_multilevel_data.columns
```

```
MultiIndex([('lifeExp', 'mean'),
 ('lifeExp', 'std'),
 ('pop', 'mean'),
 ('pop', 'std'),
 ('gdpPercap', 'mean'),
 ('gdpPercap', 'std')],
)
```

As seen, **the index and columns each contain tuples**. This is the reason that when performing data subsetting on a DataFrame with multi-level index and columns, you need to supply a tuple to specify the exact location of the data.

```
gdp_lifeExp_multilevel_data.loc[('Americas', 'United States'), ('lifeExp', 'mean')]
```

```
73.4785
```

```
gdp_lifeExp_multilevel_data.columns.values
```

```
array([('lifeExp', 'mean'), ('lifeExp', 'std'), ('pop', 'mean'),
 ('pop', 'std'), ('gdpPercap', 'mean'), ('gdpPercap', 'std')],
 dtype=object)
```

### 16.4.1 Understanding the level in multi-level index and columns

The `gdp_lifeExp_multilevel_data` dataframe have two levels of index and two level of columns, You can reference these levels in various operations, such as grouping, subsetting, or performing aggregations. The `level` parameter helps identify which part of the multi-level structure you're working with.



```
gdp_lifeExp_multilevel_data.index.get_level_values(0)
```

```
Index(['Africa', 'Africa', 'Africa', 'Africa', 'Africa', 'Africa', 'Africa',
 'Africa', 'Africa', 'Africa',
 ...
 'Europe', 'Europe', 'Europe', 'Europe', 'Europe', 'Europe', 'Europe',
 'Europe', 'Oceania', 'Oceania'],
 dtype='object', name='continent', length=142)
```

```
gdp_lifeExp_multilevel_data.index.get_level_values(1)
```

```
Index(['Algeria', 'Angola', 'Benin', 'Botswana', 'Burkina Faso', 'Burundi',
 'Cameroon', 'Central African Republic', 'Chad', 'Comoros',
 ...
 'Serbia', 'Slovak Republic', 'Slovenia', 'Spain', 'Sweden',
 'Switzerland', 'Turkey', 'United Kingdom', 'Australia', 'New Zealand'],
 dtype='object', name='country', length=142)
```

```
gdp_lifeExp_multilevel_data.columns.get_level_values(0)
```

```
Index(['lifeExp', 'lifeExp', 'pop', 'pop', 'gdpPercap', 'gdpPercap'], dtype='object')
```

```
gdp_lifeExp_multilevel_data.columns.get_level_values(1)
```

```
Index(['mean', 'std', 'mean', 'std', 'mean', 'std'], dtype='object')
```

```
gdp_lifeExp_multilevel_data.columns.get_level_values(-1)
```

```
Index(['mean', 'std', 'mean', 'std', 'mean', 'std'], dtype='object')
```

As seen above, the outermost level corresponds to `level=0`, and it increases as we move inward to the inner levels. The innermost level can be referred to as `-1`.

Now, let's use the `droplevel()` method to see how it works. The syntax for this method is as follows:

```
DataFrame.droplevel(level, axis=0)
```

```
drop the first level of row index
gdp_lifeExp_multilevel_data.droplevel(0, axis=0)
```

country	lifeExp mean	std	pop mean	std	gdpPercap mean	std
Algeria	59.030167	10.340069	1.987541e+07	8.613355e+06	4426.025973	1310.337656
Angola	37.883500	4.005276	7.309390e+06	2.672281e+06	3607.100529	1165.900251
Benin	48.779917	6.128681	4.017497e+06	2.105002e+06	1155.395107	159.741306
Botswana	54.597500	5.929476	9.711862e+05	4.710965e+05	5031.503557	4178.136987
Burkina Faso	44.694000	6.845792	7.548677e+06	3.247808e+06	843.990665	183.430087
...	...	...	...	...	...	...
Switzerland	75.565083	4.011572	6.384293e+06	8.582009e+05	27074.334405	6886.463308
Turkey	59.696417	9.030591	4.590901e+07	1.667768e+07	4469.453380	2049.665102
United Kingdom	73.922583	3.378943	5.608780e+07	3.174339e+06	19380.472986	7388.189399
Australia	74.662917	4.147774	1.464931e+07	3.915203e+06	19980.595634	7815.405220
New Zealand	73.989500	3.559724	3.100032e+06	6.547108e+05	17262.622813	4409.009167

```
drop the first level of column index
gdp_lifeExp_multilevel_data.droplevel(0, axis=1)
```

continent	country
Africa	Algeria
	Angola
	Benin
	Botswana
	Burkina Faso
...	...
Europe	Switzerland
	Turkey
	United Kingdom
Oceania	Australia
	New Zealand

## 16.4.2 Stacking (Columns to Rows ): `stack()`

The `stack()` function pivots the columns into rows. You can specify which level you want to stack using the `level` parameter. By default, it will stack the innermost level of the columns.

```
gdp_lifeExp_multilevel_data.stack(future_stack=True)
```

---

continent	country
Africa	Algeria
	Angola
	Benin
...	...
Europe	United Kingdom
Oceania	Australia
	New Zealand

---

Let's change the default setting by specifying the `level=0`(the outmost level)

```
gdp_lifeExp_multilevel_data.stack(level=0, future_stack=True)
```

---

continent	country
Africa	Algeria
	Angola
	...
...	Australia
Oceania	New Zealand

---

### 16.4.3 Unstacking (Rows to Columns) : `unstack()`

The `unstack()` function pivots the rows back into columns. By default, it will unstack the innermost level of the index.

Let's reverse the previous dataframe to its original shape using `unstack()`.

```
gdp_lifeExp_multilevel_data.stack(level=0, future_stack=True).unstack()
```

continent	country
Africa	Algeria
	Angola
	Benin
	Botswana
	Burkina Faso
...	...
Europe	Switzerland
	Turkey
	United Kingdom
Oceania	Australia
	New Zealand

#### 16.4.4 Transposing using `.T` attribute

If you simply want to swap rows and columns, you can use the `.T` attribute.

```
gdp_lifeExp_multilevel_data.T
```

	continent	Africa
	country	Algeria
lifeExp	mean	5.90
	std	1.03
pop	mean	1.90
	std	8.60
gdpPercap	mean	4.40
	std	1.30

### 16.5 Summary of Common Reshaping Functions

Function	Description	Example
<code>pivot()</code>	Reshape data by creating new columns from existing ones.	Pivot data on index/columns.
<code>melt()</code>	Unpivot data, turning columns into rows.	Convert wide format to long.
<code>stack()</code>	Convert columns to rows.	Move column data to rows.
<code>unstack()</code>	Convert rows back to columns.	Move row data to columns.
<code>.T</code>	Transpose the DataFrame (swap rows and columns).	Transpose the entire DataFrame.

### Practical Use Cases:

- **Pivoting:** Useful for time series analysis or when you want to group and summarize data by specific categories.
- **Melting:** Helpful when you need to reshape wide data for easier analysis or when preparing data for machine learning algorithms.
- **Stacking/Unstacking:** Useful when you are working with hierarchical index DataFrames.
- **Transpose:** Helpful for reorienting data for analysis or visualization.

## 16.6 Converting from Multi-Level Index to Single-Level Index DataFrame

If you're uncomfortable working with DataFrames that have multi-level indexes and columns, you can convert them into single-level DataFrames using the methods you learned in the previous chapter:

- `reset_index()`
- Flattening the multi-level columns

```
reset the index to convert the multi-level index to a single-level index
gdp_lifeExp_multilevel_data.reset_index()
```

	continent	country	lifeExp mean	std	pop mean	std	gdpPercap mean	s
0	Africa	Algeria	59.030167	10.340069	1.987541e+07	8.613355e+06	4426.025973	1
1	Africa	Angola	37.883500	4.005276	7.309390e+06	2.672281e+06	3607.100529	1
2	Africa	Benin	48.779917	6.128681	4.017497e+06	2.105002e+06	1155.395107	1
3	Africa	Botswana	54.597500	5.929476	9.711862e+05	4.710965e+05	5031.503557	4
4	Africa	Burkina Faso	44.694000	6.845792	7.548677e+06	3.247808e+06	843.990665	1

	continent	country	lifeExp mean	std	pop mean	std	gdpPercap mean	s
...	...	...	...	...	...	...	...	...
137	Europe	Switzerland	75.565083	4.011572	6.384293e+06	8.582009e+05	27074.334405	6
138	Europe	Turkey	59.696417	9.030591	4.590901e+07	1.667768e+07	4469.453380	2
139	Europe	United Kingdom	73.922583	3.378943	5.608780e+07	3.174339e+06	19380.472986	7
140	Oceania	Australia	74.662917	4.147774	1.464931e+07	3.915203e+06	19980.595634	7
141	Oceania	New Zealand	73.989500	3.559724	3.100032e+06	6.547108e+05	17262.622813	4

```
flatten the multi-level column
gdp_lifeExp_multilevel_data.columns = ['_'.join(col).strip() for col in gdp_lifeExp_multilevel_data.columns]
gdp_lifeExp_multilevel_data
```

continent	country
Africa	Algeria
	Angola
	Benin
	Botswana
	Burkina Faso
...	...
Europe	Switzerland
	Turkey
	United Kingdom
Oceania	Australia
	New Zealand

## 16.7 Independent Study

### 16.7.1 Preparing GDP per capita data

Read the GDP per capita data from [https://en.wikipedia.org/wiki/List\\_of\\_countries\\_by\\_GDP\\_\(nominal\)\\_per\\_capita](https://en.wikipedia.org/wiki/List_of_countries_by_GDP_(nominal)_per_capita)

Drop all the **Year** columns. Use the `drop()` method with the `columns`, `level` and `inplace` arguments. Print the first 2 rows of the updated DataFrame. If the first row of the DataFrame has missing values for all columns, drop that row as well.

### 16.7.1.1

Drop the inner level of column labels. Use the `droplevel()` method. Print the first 2 rows of the updated DataFrame.

### 16.7.1.2

Convert the columns consisting of GDP per capita by *IMF*, *World Bank*, and the *United Nations* to numeric. Apply a lambda function on these columns to convert them to numeric. Print the number of missing values in each column of the updated DataFrame.

*Note: Do not apply the function 3 times. Apply it once on a DataFrame consisting of these 3 columns.*

### 16.7.1.3

Apply the lambda function below on all the column names of the dataset obtained in the previous question to clean the column names.

```
import re

column_name_cleaner = lambda x:re.split(r'\[/', x)[0]
```

*Note: You will need to edit the parameter of the function, i.e.,  $x$  in the above function to make sure it is applied on column names and not columns.*

Print the first 2 rows of the updated DataFrame.

### 16.7.1.4

Create a new column `GDP_per_capita` that copies the GDP per capita values of the **United Nations**. If the GDP per capita is missing in the **United Nations** column, then copy it from the **World Bank** column. If the GDP per capita is missing both in the **United Nations** and the **World Bank** columns, then copy it from the **IMF** column.

Print the number of missing values in the `GDP_per_capita` column.

### 16.7.1.5

Drop all the columns except **Country** and `GDP_per_capita`. Print the first 2 rows of the updated DataFrame.

### 16.7.1.6

The country names contain some special characters (*characters other than letters*) and need to be cleaned. The following function can help clean country names:

```
import re

country_names_clean_gdp_data = lambda x: re.sub(r'[^w\s]', '', x).strip()
```

Apply the above lambda function on the country column to clean country names. Save the cleaned dataset as `gdp_per_capita_data`. Print the first 2 rows of the updated DataFrame.

### 16.7.2 Preparing population data

Read the population data from [https://en.wikipedia.org/wiki/List\\_of\\_countries\\_by\\_population\\_\(United\\_Nations\)](https://en.wikipedia.org/wiki/List_of_countries_by_population_(United_Nations))

- Drop all columns except Country, UN Continental Region[1], and Population (1 July 2023).
- Drop the first row since it is the total population in the world

#### 16.7.2.1

Apply the lambda function below on all the column names of the dataset obtained in the previous question to clean the column names.

```
import re

column_name_cleaner = lambda x: re.split(r'[/|\\(| |', x.name)[0]
```

*Note: You will need to edit the parameter of the function, i.e., `x` in the above function to make sure it is applied on column names and not columns.*

Print the first 2 rows of the updated DataFrame.

#### 16.7.2.2

The country names contain some special characters (*characters other than letters*) and need to be cleaned. The following function can help clean country names:

```
import re

country_names_clean_population_data = lambda x: re.sub("[\\(\\[\\.*?\\]\\]", "", x).strip()
```

Apply the above lambda function on the country column to clean country names. Save the cleaned dataset as `population_data`.



# 17 Combining Datasets

<IPython.core.display.HTML object>

## 17.1 Introduction: Why Combine Datasets?

In real-world data analysis, information is rarely contained in a single dataset. You'll often need to **combine multiple datasets** from different sources to get a complete picture. This process is fundamental to data analysis and goes by several names:

- **Merging** - joining datasets based on common keys
- **Joining** - combining datasets using indices
- **Concatenating** - stacking datasets vertically or horizontally

### 17.1.1 Common Scenarios Requiring Data Combination

#### 1. Distributed Information

- Customer demographics in one table, purchase history in another
- Product details separate from sales transactions
- Survey responses split across multiple files

#### 2. Data from Multiple Sources

- Internal company data + external market data
- Multiple databases (HR, Sales, Inventory)
- Different time periods stored in separate files

#### 3. Analytical Requirements

- Enriching existing data with additional attributes
- Comparing data across different groups or time periods
- Building comprehensive datasets for machine learning

### 17.1.2 The Three Main Combination Methods

Pandas provides three powerful functions for combining datasets, each suited to different scenarios:

Method	Function	Use Case	Key Parameter
<b>Merge</b>	<code>merge()</code>	Combine datasets with common columns (like SQL JOIN)	<code>on=</code> (column name)
<b>Join</b>	<code>join()</code>	Combine datasets using their indices	<code>lsuffix=</code> , <code>rsuffix=</code>
<b>Concatenate</b>	<code>concat()</code>	Stack datasets vertically or horizontally	<code>axis=</code> (0 or 1)

In this chapter, we'll explore each method with practical examples, understand when to use which approach, and learn how to handle common challenges like missing values and duplicate columns.

### 17.1.3 Setup: Loading Required Libraries

Let's start by importing the necessary libraries and loading our example datasets.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

Display settings for better output
pd.set_option('display.max_columns', None)
pd.set_option('display.width', None)
```

## 17.2 Method 1: Merging with `merge()`

The `merge()` function is pandas' most versatile tool for combining datasets. It works similarly to **SQL JOIN operations**, combining DataFrames based on one or more **common columns** (called keys).

### 17.2.1 Basic Syntax

```
pd.merge(left_df, right_df, on='key_column', how='inner')
```

Or using the DataFrame method:

```
left_df.merge(right_df, on='key_column', how='inner')
```

Let's see this in action!

### 17.2.2 Example Datasets: Lord of the Rings Fellowship

Throughout this chapter, we'll use multiple datasets that contain complementary information, we'll combine using different methods.

Let's first load the **primary dataset** containing basic fellowship member information:

```
Load the primary dataset
df_existing = pd.read_csv('./datasets/LOTR.csv')

print(f"Dataset shape: {df_existing.shape}")
print(f"Columns: {list(df_existing.columns)}\n")

df_existing.head()
```

```
Dataset shape: (4, 3)
Columns: ['FellowshipID', 'FirstName', 'Skills']
```

	FellowshipID	FirstName	Skills
0	1001	Frodo	Hiding
1	1002	Samwise	Gardening
2	1003	Gandalf	Spells
3	1004	Pippin	Fireworks

Now let's load the **external dataset** containing additional information about the fellowship members:

```
Load the external dataset
df_external = pd.read_csv("./datasets/LOTR 2.csv")

print(f"Dataset shape: {df_external.shape}")
print(f"Columns: {list(df_external.columns)}\n")

df_external.head()
```

Dataset shape: (5, 3)  
Columns: ['FellowshipID', 'FirstName', 'Age']

	FellowshipID	FirstName	Age
0	1001	Frodo	50
1	1002	Samwise	39
2	1006	Legolas	2931
3	1007	Elrond	6520
4	1008	Barromir	51

**Observation:** Both datasets contain information about fellowship members, but they may have:

- **Common columns** that can serve as keys for merging (like FellowshipID or Name)
- **Different sets of attributes** that complement each other
- **Potentially different numbers of rows** (not all members may appear in both datasets)

Our goal is to combine these datasets to create a comprehensive view of the fellowship.

### 17.2.3 Example 1: Basic Merge with Automatic Key Detection

When both DataFrames have columns with the same name, `merge()` automatically uses them as keys:

```
Merge using automatic key detection
df_merged = pd.merge(df_existing, df_external)

print(f"Original datasets: {df_existing.shape} + {df_external.shape}")
print(f"Merged dataset: {df_merged.shape}")
print(f"\nMerged columns: {list(df_merged.columns)}\n")
```

```
df_merged.head()
```

Original datasets: (4, 3) + (5, 3)

Merged dataset: (2, 4)

Merged columns: ['FellowshipID', 'FirstName', 'Skills', 'Age']

	FellowshipID	FirstName	Skills	Age
0	1001	Frodo	Hiding	50
1	1002	Samwise	Gardening	39

**Alternative Syntax:** You can also use the DataFrame method, which is more readable in method chains:

```
Using DataFrame.merge() method (equivalent to above)
df_merged_method = df_existing.merge(df_external)

print(f"Results are identical: {df_merged.equals(df_merged_method)}")
print(f"Merged dataset shape: {df_merged_method.shape}\n")

df_merged_method.head()
```

Results are identical: True

Merged dataset shape: (2, 4)

	FellowshipID	FirstName	Skills	Age
0	1001	Frodo	Hiding	50
1	1002	Samwise	Gardening	39

#### 17.2.4 Example 2: Merging with Different Column Names

What if the key columns have **different names** in each DataFrame? Use the `left_on` and `right_on` parameters to specify which columns to match.

```

Example: Suppose the external dataset uses 'ID' instead of 'FellowshipID'
Let's create a version with a different column name to demonstrate
df_external_renamed = df_external.rename(columns={'FellowshipID': 'ID'})

print("External dataset columns after renaming:")
print(list(df_external_renamed.columns))
print()

Now merge using left_on and right_on
df_merged_diff = pd.merge(
 df_existing,
 df_external_renamed,
 left_on='FellowshipID', # Column name in left DataFrame
 right_on='ID' # Column name in right DataFrame
)

print(f"Merged dataset shape: {df_merged_diff.shape}")
print(f"Merged columns: {list(df_merged_diff.columns)}\n")

df_merged_diff.head()

```

#### Important observations:

- Notice that **both** key columns (FellowshipID and ID) appear in the result
- They contain identical values (the matched keys)
- You can drop one of them after merging: `df_merged_diff.drop('ID', axis=1)`

**Common use case:** Merging data from different sources where column naming conventions differ (e.g., `customer_id` vs `cust_id`, `date` vs `timestamp`).

### 17.2.5 Example 3: Merging on Multiple Columns

Sometimes you need to match rows based on **multiple columns** together (composite keys). This is common when a single column isn't unique.

```

Example: Merge on both Name AND Race to ensure exact matches
df_multi_key = pd.merge(
 df_existing,
 df_external,
 on=['Name', 'Race'] # Both columns must match
)

```

```
print(f"Merged on multiple keys: {df_multi_key.shape}")
print("\nMerging on multiple columns ensures rows match on ALL specified keys\n")

df_multi_key.head()
```

### Real-world examples of composite keys:

- **Sales transactions:** Match on both `Store_ID` AND `Date` (same store can have sales on different dates)
- **Student grades:** Match on `Student_ID` AND `Course_ID` (same student takes multiple courses)
- **Time series data:** Match on `Location` AND `Timestamp` (multiple locations measured over time)

### Syntax variations:

```
Using on= (when column names are the same)
pd.merge(df1, df2, on=['col1', 'col2'])

Using left_on= and right_on= (when column names differ)
pd.merge(df1, df2, left_on=['id', 'date'], right_on=['ID', 'timestamp'])
```

## 17.2.6 Understanding Join Types: The `how` Parameter

By default, `merge()` performs an **inner join**, which keeps only rows where the key exists in **both** DataFrames. However, you can control this behavior using the `how` parameter.

### The Four Join Types:

Join Type	<code>how=</code>	Keeps Rows From	Use Case
<b>Inner</b>	<code>'inner'</code>	Both (intersection)	Only matching records
<b>Left</b>	<code>'left'</code>	All from left + matches from right	Keep all primary data
<b>Right</b>	<code>'right'</code>	All from right + matches from left	Keep all secondary data
<b>Outer</b>	<code>'outer'</code>	Both (union)	Keep everything

Let's explore each type with examples.

### 17.2.6.1 Inner Join (Default): Only Matching Rows

An **inner join** returns only the rows where the key exists in both DataFrames. This is the default behavior.

```
Inner join - explicitly specified (same as default)
df_inner = pd.merge(df_existing, df_external, how='inner')

print(f"Inner join result: {df_inner.shape}")
print("Only rows with matching keys in BOTH datasets are kept\n")

df_inner.head()
```

Inner join result: (2, 4)

Only rows with matching keys in BOTH datasets are kept

	FellowshipID	FirstName	Skills	Age
0	1001	Frodo	Hiding	50
1	1002	Samwise	Gardening	39

#### When to use inner join:

Use inner join when you need **complete information** from both datasets for your analysis.

#### Real-world examples:

##### 1. Analyzing vaccination impact on COVID infection rates

- Dataset 1: COVID infection rates by country
- Dataset 2: Vaccination counts by country
- **Why inner join?** You can't analyze the relationship unless you have BOTH metrics for a country. Countries missing either value can't contribute to the analysis.

##### 2. Customer purchase analysis

- Dataset 1: Customer demographics
- Dataset 2: Purchase transactions
- **Why inner join?** To analyze how demographics relate to purchases, you need both demographic info AND purchase history.

##### 3. A/B testing results

- Dataset 1: User group assignments (control/treatment)



- Dataset 2: Conversion outcomes
- **Why inner join?** Only users with both group assignment AND outcome data can be included in the test analysis.

### 17.2.6.2 Left Join: Keep All Left DataFrame Rows

A **left join** returns **all rows from the left DataFrame** and matching rows from the right DataFrame. Non-matching rows from the right will have NaN values.

```
Left join - keep all rows from left (existing) dataset
df_left = pd.merge(df_existing, df_external, how='left')

print(f"Left DataFrame shape: {df_existing.shape}")
print(f"Right DataFrame shape: {df_external.shape}")
print(f"Left join result: {df_left.shape}")
print("\nAll rows from LEFT dataset are kept, even if no match in RIGHT\n")

Show rows with missing values from the right dataset
missing_count = df_left.isnull().any(axis=1).sum()
print(f"Rows with NaN values (no match in right dataset): {missing_count}\n")

df_left.head(10)
```

```
Left DataFrame shape: (4, 3)
Right DataFrame shape: (5, 3)
Left join result: (4, 4)
```

All rows from LEFT dataset are kept, even if no match in RIGHT

Rows with NaN values (no match in right dataset): 2

	FellowshipID	FirstName	Skills	Age
0	1001	Frodo	Hiding	50.0
1	1002	Samwise	Gardening	39.0
2	1003	Gandalf	Spells	NaN
3	1004	Pippin	Fireworks	NaN

**When to use left join:**

Use left join when the **left dataset contains your primary data** that must be retained, while the right dataset provides supplementary information that may be missing for some records.

### Real-world examples:

#### 1. COVID infection rates + government effectiveness scores

- Left: COVID infection rates (critical, hard to impute)
- Right: Government effectiveness scores (can be estimated from GDP, crime rate, etc.)
- **Why left join?** Keep all countries with infection rate data. Missing government scores can be imputed later using correlated variables.

#### 2. Customer demographics + credit card spending

- Left: Customer demographics (all customers)
- Right: Credit card transactions (only customers who made purchases)
- **Why left join?** Keep all customers, even those with no purchases. Missing spend can be filled with 0 (indicating no transactions).

#### 3. Employee records + performance reviews

- Left: All employee records
- Right: Recent performance reviews (not all employees reviewed yet)
- **Why left join?** Maintain complete employee roster. Missing reviews can be handled separately (e.g., “Pending” status).

**Key principle:** Use left join when you can’t afford to lose the left dataset’s records, and missing right-side values can be reasonably handled (imputed, filled with defaults, or marked as missing).

### 17.2.6.3 Right Join: Keep All Right DataFrame Rows

A **right join** is the mirror image of a left join—it returns **all rows from the right DataFrame** and matching rows from the left DataFrame.

```
Right join - keep all rows from right (external) dataset
df_right = pd.merge(df_existing, df_external, how='right')

print(f"Left DataFrame shape: {df_existing.shape}")
print(f"Right DataFrame shape: {df_external.shape}")
print(f"Right join result: {df_right.shape}")
print("\nAll rows from RIGHT dataset are kept, even if no match in LEFT\n")
```

```
df_right.head()
```

Left DataFrame shape: (4, 3)  
Right DataFrame shape: (5, 3)  
Right join result: (5, 4)

All rows from RIGHT dataset are kept, even if no match in LEFT

	FellowshipID	FirstName	Skills	Age
0	1001	Frodo	Hiding	50
1	1002	Samwise	Gardening	39
2	1006	Legolas	NaN	2931
3	1007	Elrond	NaN	6520
4	1008	Barromir	NaN	51

### When to use right join:

Right join serves the same purpose as left join, just with the DataFrames swapped. In practice:

- **Most analysts prefer left join** because it's more intuitive (reads left-to-right)
- **Right join is rarely used** - you can always swap the DataFrames and use left join instead

### Example equivalence:

```
These produce the same result:
df_right = pd.merge(df_A, df_B, how='right') # Right join
df_left = pd.merge(df_B, df_A, how='left') # Equivalent left join
```

### When you might see right join:

- Legacy code or specific coding style preferences
- SQL background where RIGHT JOIN might be more familiar
- Method chaining where the order is determined by workflow

**Best practice:** Stick with left join and arrange your DataFrames accordingly—it makes code more readable and maintainable.

#### 17.2.6.4 Outer Join: Keep All Rows from Both DataFrames

An **outer join** (also called a **full outer join**) returns **all rows from both DataFrames**, with NaN values where matches don't exist. This is the most inclusive join type—no data is lost.

```
Outer join - keep all rows from BOTH datasets
df_outer = pd.merge(df_existing, df_external, how='outer')

print(f"Left DataFrame shape: {df_existing.shape}")
print(f"Right DataFrame shape: {df_external.shape}")
print(f"Outer join result: {df_outer.shape}")
print("\nAll rows from BOTH datasets are kept, with NaN for non-matches\n")

Check for missing values
print("Missing values per column:")
print(df_outer.isnull().sum())
print()

df_outer
```

```
Left DataFrame shape: (4, 3)
Right DataFrame shape: (5, 3)
Outer join result: (7, 4)
```

All rows from BOTH datasets are kept, with NaN for non-matches

```
Missing values per column:
FellowshipID 0
FirstName 0
Skills 3
Age 2
dtype: int64
```

	FellowshipID	FirstName	Skills	Age
0	1001	Frodo	Hiding	50.0
1	1002	Samwise	Gardening	39.0
2	1003	Gandalf	Spells	NaN
3	1004	Pippin	Fireworks	NaN
4	1006	Legolas	NaN	2931.0

	FellowshipID	FirstName	Skills	Age
5	1007	Elrond	NaN	6520.0
6	1008	Barromir	NaN	51.0

### When to use outer join:

Use outer join when you **cannot afford to lose any data** from either dataset, even if it means having incomplete records.

### Real-world examples:

#### 1. Course surveys from multiple time points

- Dataset 1: Mid-semester survey responses
- Dataset 2: End-of-semester survey responses
- **Why outer join?** Students who responded to either survey provide valuable feedback. Keep all responses, even from students who only completed one survey. Analyze response patterns and sentiment across all participants.

#### 2. Multi-source customer data integration

- Dataset 1: Online purchase records
- Dataset 2: In-store purchase records
- **Why outer join?** Customers may shop through one channel only. Keep all customers to understand total customer base and channel preferences.

#### 3. Scientific study with partial data

- Dataset 1: Lab test results
- Dataset 2: Clinical observations
- **Why outer join?** Some subjects may have only lab results or only clinical observations due to scheduling, dropouts, or data collection issues. Keep all subjects to maximize sample size and identify patterns in data availability.

#### 4. Data quality auditing

- Dataset 1: Expected records (should exist)
- Dataset 2: Actual records (what we have)
- **Why outer join?** Identify both missing expected records (in left but not right) and unexpected extra records (in right but not left) for data quality assessment.

**Key principle:** Use outer join when completeness trumps having matched pairs, and you'll handle missing values appropriately in subsequent analysis.

## 17.3 Method 2: Joining with join()

While `merge()` combines DataFrames based on **column values**, the `join()` method combines them based on their **indices**. This is particularly useful when your DataFrames are already indexed by a meaningful key (like customer ID, timestamp, etc.).

### 17.3.1 Key Difference: `merge()` vs `join()`

Aspect	<code>merge()</code>	<code>join()</code>
<b>Joins on</b>	Column values	Index values
<b>Default join</b>	Inner	Left
<b>Syntax</b>	More explicit	More concise
<b>Use when</b>	Keys are in columns	Keys are in index

### 17.3.2 Important: Handling Overlapping Columns

Unlike `merge()`, `join()` requires you to specify **suffixes** for overlapping column names (it has no default). This is done with the `lsuffix` and `rsuffix` parameters.

Let's see what happens without suffixes:

The code above will raise an error because both DataFrames have overlapping column names, and `join()` doesn't know how to handle them without suffixes.

**Let's fix this by adding suffixes:**

```
Join with suffixes to handle overlapping column names
df_joined = df_existing.join(
 df_external,
 lsuffix='_existing', # Suffix for left DataFrame columns
 rsuffix='_external' # Suffix for right DataFrame columns
)

print(f"Joined dataset shape: {df_joined.shape}")
print(f"\nColumns with suffixes:")
print([col for col in df_joined.columns if '_existing' in col or '_external' in col])
print()

df_joined.head()
```

Joined dataset shape: (4, 6)

Columns with suffixes:

['FellowshipID\_existing', 'FirstName\_existing', 'FellowshipID\_external', 'FirstName\_external']

	FellowshipID_existing	FirstName_existing	Skills	FellowshipID_external	FirstName_external
0	1001	Frodo	Hiding	1001	Frodo
1	1002	Samwise	Gardening	1002	Samwise
2	1003	Gandalf	Spells	1006	Legolas
3	1004	Pippin	Fireworks	1007	Elrond

### Key Observations:

1. **Default behavior:** `join()` performs a **left join** by default (keeps all rows from the left DataFrame)
2. **Index-based:** Rows are matched using the DataFrame indices (0, 1, 2, ...)
3. **Suffixes required:** Overlapping column names get the specified suffixes

This works, but notice we're joining on the **default integer index** (0, 1, 2, ...), which may not be meaningful. Let's make this more useful by setting a proper index.

### 17.3.3 Setting a Meaningful Index for Joining

For `join()` to be truly useful, we should set a **meaningful column as the index** (like FellowshipID). This way, rows are matched based on actual business logic, not arbitrary row numbers.

```
Set 'FellowshipID' as the index for both DataFrames
df_existing_indexed = df_existing.set_index('FellowshipID')
df_external_indexed = df_external.set_index('FellowshipID')

print("Left DataFrame (indexed by Fellowship ID):")
print(df_existing_indexed.head())
print("\nRight DataFrame (indexed by FellowshipID):")
print(df_external_indexed.head())
```

Left DataFrame (indexed by Fellowship ID):

	FirstName	Skills
FellowshipID		
1001	Frodo	Hiding

1002	Samwise	Gardening
1003	Gandalf	Spells
1004	Pippin	Fireworks

Right DataFrame (indexed by FellowshipID):

	FirstName	Age
FellowshipID		
1001	Frodo	50
1002	Samwise	39
1006	Legolas	2931
1007	Elrond	6520
1008	Barromir	51

```
Now join using the FellowshipID index
df_joined_indexed = df_existing_indexed.join(
 df_external_indexed,
 lsuffix='_existing',
 rsuffix='_external'
)

print(f"Joined dataset shape: {df_joined_indexed.shape}")
print(f"\nIndex name: {df_joined_indexed.index.name}")
print("\nNow rows are matched by FellowshipID, not arbitrary row numbers!\n")

df_joined_indexed
```

Joined dataset shape: (4, 4)

Index name: FellowshipID

Now rows are matched by FellowshipID, not arbitrary row numbers!

	FirstName_existing	Skills	FirstName_external	Age
FellowshipID				
1001	Frodo	Hiding	Frodo	50.0
1002	Samwise	Gardening	Samwise	39.0
1003	Gandalf	Spells	NaN	NaN
1004	Pippin	Fireworks	NaN	NaN

**Perfect!** Now the join is meaningful:



- Rows are matched by `FellowshipID` (a business key)
- All rows from the left DataFrame are kept (default left join)
- `NaN` values appear where `FellowshipID` doesn't exist in the right DataFrame
- The index preserves the key for easy lookup

### 17.3.4 Changing Join Type in `join()`

Just like `merge()`, you can specify the join type using the `how` parameter:

```
Examples of different join types with join()
print("Inner join:")
print(df_existing_indexed.join(df_external_indexed, how='inner', lsuffix='_L', rsuffix='_R')

print("\nOuter join:")
print(df_existing_indexed.join(df_external_indexed, how='outer', lsuffix='_L', rsuffix='_R')

print("\nRight join:")
print(df_existing_indexed.join(df_external_indexed, how='right', lsuffix='_L', rsuffix='_R')
```

Inner join:  
(2, 4)

Outer join:  
(7, 4)

Right join:  
(5, 4)

## 17.4 Method 3: Concatenating with `concat()`

While `merge()` and `join()` combine DataFrames by matching keys or indices, `concat()` simply **stacks DataFrames together** either vertically (one on top of another) or horizontally (side by side). Think of it as physically gluing DataFrames together.

### 17.4.1 The Two Directions: `axis` Parameter

axis=	Direction	Effect	Use Case
0 (default)	Vertical	Rows on top of rows	Combining data from multiple time periods
1	Horizontal	Columns side by side	Adding new features/attributes

## 17.4.2 Concatenating Along Rows (Vertical Stacking)

The default behavior (axis=0) stacks DataFrames vertically—adding more rows.

### 17.4.2.1 Simple Example with Fellowship Data

Let's start with a simple example using our LOTR datasets to understand the basics:

```
Split our LOTR data into two groups to demonstrate concatenation
lotr_df1 = df_existing.head(5) # First 5 fellowship members
lotr_df2 = df_existing.tail(4) # Last 4 fellowship members

print("First group:")
print(lotr_df1)
print("\nSecond group:")
print(lotr_df2)
```

```
Concatenate vertically (default: axis=0) - stack rows
lotr_combined = pd.concat([lotr_df1, lotr_df2])

print(f"First group: {lotr_df1.shape}")
print(f"Second group: {lotr_df2.shape}")
print(f"Combined: {lotr_combined.shape}")
print("\nRows are simply stacked on top of each other:\n")

lotr_combined
```

#### Key points about vertical concatenation:

- Rows are simply stacked (group 1 rows, then group 2 rows)
- Index values may be duplicated (notice indices 0-4 appear, then 5-8)
- All columns from both DataFrames are kept
- No key matching occurs—just physical stacking

Use `ignore_index=True` to create a new sequential index:

```
Reset index to create sequential numbering
lotr_combined_reset = pd.concat([lotr_df1, lotr_df2], ignore_index=True)

print("With ignore_index=True, we get a clean sequential index:\n")
lotr_combined_reset
```

#### 17.4.2.2 Real-World Example: Combining Continental Data

Now let's see a practical real-world use case: combining GDP and life expectancy data that's been split across multiple files by continent. This is a common scenario when data is stored in separate files for organizational purposes.

```
data_asia = pd.read_csv('./Datasets/gdp_lifeExpec_Asia.csv')
data_europe = pd.read_csv('./Datasets/gdp_lifeExpec_Europe.csv')
data_africa = pd.read_csv('./Datasets/gdp_lifeExpec_Africa.csv')
data_oceania = pd.read_csv('./Datasets/gdp_lifeExpec_Oceania.csv')
data_americas = pd.read_csv('./Datasets/gdp_lifeExpec_Americas.csv')
```

```
#Appending all the data files, i.e., stacking them on top of each other
data_all_continents = pd.concat([data_asia,data_europe,data_africa,data_oceania,data_americas],
data_all_continents
```

		country
Asia	0	Afghanistan
	1	Afghanistan
	2	Afghanistan
	3	Afghanistan
	4	Afghanistan
Americas	...	...
	295	Venezuela
	296	Venezuela
	297	Venezuela
	298	Venezuela
	299	Venezuela

Let's have the continent as a column as we need to use that in the visualization.

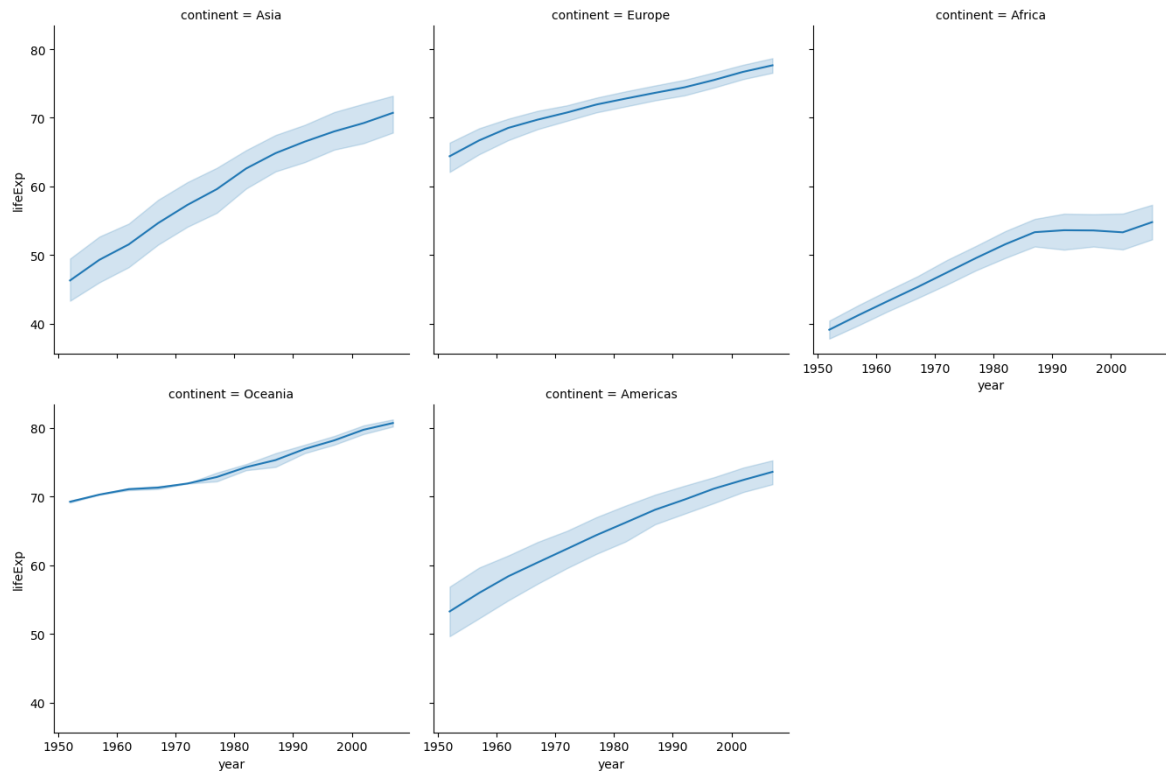
```
data_all_continents.reset_index(inplace = True)
data_all_continents.head()
```

	level_0	level_1	country	year	lifeExp	pop	gdpPercap
0	Asia	0	Afghanistan	1952	28.801	8425333	779.445314
1	Asia	1	Afghanistan	1957	30.332	9240934	820.853030
2	Asia	2	Afghanistan	1962	31.997	10267083	853.100710
3	Asia	3	Afghanistan	1967	34.020	11537966	836.197138
4	Asia	4	Afghanistan	1972	36.088	13079460	739.981106

```
data_all_continents.drop(columns = 'level_1',inplace = True)
data_all_continents.rename(columns = {'level_0':'continent'},inplace = True)
data_all_continents.head()
```

	continent	country	year	lifeExp	pop	gdpPercap
0	Asia	Afghanistan	1952	28.801	8425333	779.445314
1	Asia	Afghanistan	1957	30.332	9240934	820.853030
2	Asia	Afghanistan	1962	31.997	10267083	853.100710
3	Asia	Afghanistan	1967	34.020	11537966	836.197138
4	Asia	Afghanistan	1972	36.088	13079460	739.981106

```
#change of life expectancy over time for different continents
a = sns.FacetGrid(data_all_continents,col = 'continent',col_wrap = 3,height = 4.5,aspect = 1)
a.map(sns.lineplot,'year','lifeExp')
a.add_legend();
```



In the above example, datasets were appended (or stacked on top of each other).

Datasets can also be concatenated side-by-side (by providing the argument `axis = 1` with the `concat()` function) as we saw with the `merge` function.

### 17.4.3 Horizontal Concatenation (Combining Columns)

Setting `axis=1` combines DataFrames side-by-side by aligning their **indices**.

```
Concatenate horizontally (axis=1) - combine columns
result = pd.concat([df_existing, df_external], axis=1)
result
```

	FellowshipID	FirstName	Skills	FellowshipID	FirstName	Age
0	1001.0	Frodo	Hiding	1001	Frodo	50
1	1002.0	Samwise	Gardening	1002	Samwise	39
2	1003.0	Gandalf	Spells	1006	Legolas	2931
3	1004.0	Pippin	Fireworks	1007	Elrond	6520
4	NaN	NaN	NaN	1008	Barromir	51

### Notice:

- DataFrames are placed side-by-side
- **Index values are used for alignment** (matching FellowshipIDs are aligned)
- Non-matching indices result in NaN values
- This is similar to an outer join on the index

### When to use horizontal concat:

- Adding new features/columns to existing data
- Combining results from different analyses on the same observations
- Building feature matrices for machine learning (each DataFrame contains different feature sets)

#### 17.4.3.1 Handling Duplicate Column Names

When concatenating horizontally with duplicate column names, pandas keeps both columns:

```
See column names when both DataFrames have 'Name'
result.columns
```

```
Index(['FellowshipID', 'FirstName', 'Skills', 'FellowshipID', 'FirstName',
 'Age'],
 dtype='object')
```

Both `Name` columns are kept, which can be confusing. You can use the `keys` parameter to create hierarchical column names:

```
Use keys to label which DataFrame each column came from
result_labeled = pd.concat([df_existing, df_external], axis=1, keys=['Fellowship', 'Extended'])
result_labeled
```

	Fellowship			Extended		
	FellowshipID	FirstName	Skills	FellowshipID	FirstName	Age
0	1001.0	Frodo	Hiding	1001	Frodo	50
1	1002.0	Samwise	Gardening	1002	Samwise	39
2	1003.0	Gandalf	Spells	1006	Legolas	2931
3	1004.0	Pippin	Fireworks	1007	Elrond	6520
4	NaN	NaN	NaN	1008	Barromir	51

Now you can clearly see which DataFrame each column came from. This creates a **MultiIndex** for columns (we'll cover this in detail in future chapters).

## 17.5 Summary: Choosing the Right Method

Now that we've covered all three methods for combining DataFrames, here's how to choose:

Method	Best For	Key Feature	Common Pitfall
<code>merge()</code>	Combining on <b>shared column(s)</b>	Flexible join types (inner/left/right/outer)	Forgetting to specify join type (defaults to inner)
<code>join()</code>	Combining on <b>index</b> values	Quick syntax for index-based joins	Index must be set up properly beforehand
<code>concat()</code>	Stacking DataFrames with <b>same structure</b>	Simple vertical/horizontal stacking	Doesn't match on keys (just stacks/aligns)

### 17.5.1 Decision Tree:

1. Do you need to match rows based on values in columns?
  - Yes → Use `merge()` (specify join type and `on=` parameter)
2. Do you need to match rows based on index values?
  - Yes → Use `join()` (ensure indices are meaningful)
3. Do you just need to stack DataFrames?
  - Vertically (add more rows) → Use `concat(axis=0)`
  - Horizontally (add more columns) → Use `concat(axis=1)` (aligns on index)

## 17.6 Handling Missing Values After Combining Datasets

When you combine datasets using `merge()`, `join()`, or `concat()`, you often introduce **missing values** (NaN) for unmatched entries. Understanding *why* these appear and *how* to handle them is crucial for maintaining data quality.

### 17.6.1 Why Missing Values Occur

Missing values appear for different reasons depending on the join type:

Join Type	When NaN Appears	Example
<b>Left Join</b>	Right DataFrame has no match for left keys	Customer exists but has no purchases → purchase columns = NaN
<b>Right Join</b>	Left DataFrame has no match for right keys	Purchase exists but customer deleted → customer columns = NaN
<b>Outer Join</b>	Either DataFrame has no match	Missing data from both sides
<b>concat (axis=1)</b>	Indices don't align	Different row indices → NaN where no alignment

**Inner joins never produce NaN** from the join operation itself (only matched rows are kept).

## 17.6.2 Strategies for Handling Missing Values

The right strategy depends on **why the data is missing** and **what you're analyzing**.

### 17.6.2.1 Strategy 1: Keep the Missing Values

**When to use:** Missing values are meaningful and represent “absence” of something.

**Example:** Customer purchase analysis with left join

```
All customers + their purchases (if any)
df_customer_purchases = customers.merge(purchases, on='CustomerID', how='left')
```

**Interpretation:**

- NaN in purchase columns = Customer hasn't made a purchase yet
- This is **informative** - these customers might be new or inactive
- Analysis: “What percentage of customers have never purchased?” requires keeping NaN

**Action:** Keep NaN and analyze it as a category (e.g., `df['Purchase'].isna().sum()`)



### 17.6.2.2 Strategy 2: Fill with Default Values

**When to use:** Missing values should be interpreted as zero or a default state.

**Example:** Sales data where no transaction = zero sales

```
Fill missing sales with 0
df_merged['TotalSales'] = df_merged['TotalSales'].fillna(0)
df_merged['TransactionCount'] = df_merged['TransactionCount'].fillna(0)
```

**Common fill values:**

- 0 - for counts, amounts, quantities
- 'None' or 'Unknown' - for categorical data
- False - for boolean flags
- Mean/median - for numerical features (use with caution!)

**Caution:** Only fill when you're certain what the missing value *means*.

### 17.6.2.3 Strategy 3: Drop Rows with Missing Values

**When to use:** Analysis requires complete information from both datasets.

**Example:** Analyzing relationship between two variables (need both)

```
Drop rows where either GDP or Population is missing
df_complete = df_merged.dropna(subset=['GDP', 'Population'])
```

**Options:**

- dropna() - drop rows with *any* NaN
- dropna(subset=['col1', 'col2']) - drop only if specific columns have NaN
- dropna(thresh=5) - drop only if fewer than 5 non-NaN values

**Caution:** You're losing data! Document how many rows were dropped and why.

### 17.6.2.4 Strategy 4: Use a Different Join Type

**When to use:** You're getting NaN because you used the wrong join type!

**Example:** Used outer join but only need matching rows

```
Wrong: Creates many NaN values
df_outer = pd.merge(df1, df2, how='outer') # 1000 rows, 50% NaN

Better: Use inner join to keep only complete matches
df_inner = pd.merge(df1, df2, how='inner') # 600 rows, 0% NaN
```

**Before handling missing values, ask:** 1. Did I use the right join type for my analysis question? 2. Should I actually be using inner join instead of left/outer? 3. Are the missing values telling me about data quality issues?

### 17.6.3 Practical Example: Handling Missing Values

Let's see these strategies in action with our fellowship data:

```
Create an outer join to see missing values
df_outer_example = pd.merge(df_existing, df_external, how='outer')

print("Outer join result:")
print(f"Total rows: {len(df_outer_example)}")
print(f"\nMissing values per column:")
print(df_outer_example.isnull().sum())
print(f"\nRows with ANY missing value: {df_outer_example.isnull().any(axis=1).sum()}")

df_outer_example
```

Now let's apply different strategies:

```
Strategy 1: Identify WHERE the missing values are
print("Rows with missing 'Race' (from left dataset):")
print(df_outer_example[df_outer_example['Race'].isna()][['Name', 'Race', 'Birthplace']])
print()

print("Rows with missing 'Birthplace' (from right dataset):")
print(df_outer_example[df_outer_example['Birthplace'].isna()][['Name', 'Race', 'Birthplace']])

Strategy 2: Drop incomplete rows (for complete-case analysis)
df_complete = df_outer_example.dropna()

print(f"Original rows: {len(df_outer_example)}")
print(f"Complete rows: {len(df_complete)}")
```

```

print(f"Rows dropped: {len(df_outer_example) - len(df_complete)}")
print()
df_complete

Strategy 3: Fill with meaningful defaults
df_filled = df_outer_example.copy()
df_filled['Birthplace'] = df_filled['Birthplace'].fillna('Unknown')

print("After filling missing 'Birthplace' with 'Unknown':")
print(df_filled[['Name', 'Birthplace']])
print(f"\nMissing 'Birthplace' values: {df_filled['Birthplace'].isna().sum()}")

```

## 17.6.4 Best Practices for Missing Value Handling

### 1. Investigate First

```

Always check WHERE and WHY values are missing
df.isnull().sum() # Count per column
df[df['column'].isna()] # See the actual rows

```

### 2. Document Your Decisions

```

Good: Document what you did and why
Filled missing GDP with 0 because these are countries with no economic data reported
df['GDP'] = df['GDP'].fillna(0)

Also good: Create a flag for missing values before filling
df['GDP_was_missing'] = df['GDP'].isna()
df['GDP'] = df['GDP'].fillna(df['GDP'].median())

```

### 3. Consider the Impact

- **Dropping rows:** Reduces sample size, may introduce bias
- **Filling with defaults:** May distort statistics (mean, correlations)
- **Keeping NaN:** Requires careful handling in analysis/visualization

### 4. Choose Join Type Carefully

The best way to handle missing values is often to **prevent them** by using the right join type for your analysis question!

**Remember:** Missing values after combining datasets are **expected and normal**. The key is understanding what they mean and handling them appropriately for your specific analysis goals.

## 17.7 Independent Study

### 17.7.1 Merging GDP per capita and population datasets

In this independent study on **data reshaping**, we have provided two datasets:

- `gdp_per_capita_data`
- `population_data`

This exercise builds on the results from the previous section.

**Task:**

Merge `gdp_per_capita_data` with `population_data` to combine each country's **population** and **GDP per capita** into a single DataFrame.

Finally, print the **first two rows** of the merged DataFrame.

Assume that:

1. We want to keep the GDP per capita of all countries in the merged dataset, even if their population is unavailable in the population dataset. For countries whose population is unavailable, their `Population` column will show `NA`.
2. We want to discard an observation of a country if its GDP per capita is unavailable.

#### 17.7.1.1

For how many countries in `gdp_per_capita_data` does the population seem to be unavailable in `population_data`? Note that you don't need to clean country names any further than cleaned by the functions provided.

Print the observations of `gdp_per_capita_data` with missing `Population`.

### 17.7.2 Merging datasets with *similar* values in the *key* column

We suspect that population of more countries may be available in `population_data`. However, due to unclean country names, the observations could not merge. For example, the country *Guinea Bissau* is mentioned as *GuineaBissau* in `gdp_per_capita_data` and *Guinea-Bissau* in `population_data`. To resolve this issue, we'll use a different approach to merge datasets. We'll merge the population of a country to an observation in the GDP per capita dataset, whose name in `population_data` is the most '*similar*' to the name of the country in `gdp_per_capita_data`.

### 17.7.2.1

Proceed as follows:

1. For each country in `gdp_per_capita_data`, find the country with the most ‘*similar*’ name in `population_data`, based on the similarity score. Use the lambda function provided below to compute the similarity score between two strings (*The higher the score, the more similar are the strings. The similarity score is 1.0 if two strings are exactly the same*).
2. Merge the population of the most ‘*similar*’ country to the country in `gdp_per_capita_data`. The merged dataset must include 5 columns - the country name as it appears in `gdp_per_capita_data`, the GDP per capita, the country name of the most ‘*similar*’ country as it appears in `population_data`, the population of that country, and the similarity score between the country names.
3. After creating the merged dataset, **print** the rows of the dataset that have similarity scores less than 1.

Use the function below to compute the similarity score between the Country names of the two datasets:

```
from difflib import SequenceMatcher

similar = lambda a,b: SequenceMatcher(None, a, b).ratio()
```

**Note:** You may use one for loop only for this particular question. However, it would be perfect if don’t use a for loop

**Hint:**

1. Define a function that computes the index of the observation having the most ‘*similar*’ country name in `population_data` for an observation in `gdp_per_capita_data`. The function returns a Series consisting of the most ‘*similar*’ country name, its population, and its similarity score (*This function can be written with only one line in its body, excluding the return statement and the definition statement. However, you may use as many lines as you wish*).
2. Apply the function on the Country column of `gdp_per_capita_data`. A DataFrame will be obtained.
3. Concatenate the DataFrame obtained in (2) with `gdp_per_capita_data` with the pandas `concat()` function.

### 17.7.2.2

In the dataset obtained in the previous question, for all observations where similarity score is less than 0.8, replace the population with Nan.

Print the observations of the dataset having missing values of population.

# A Assignment A

Python Iterables and Data Structures

## Instructions

1. You may talk to a friend, discuss the questions and potential directions for solving them. However, you need to write your own solutions and code separately, and not as a group activity.
2. Write your code in the *Code* cells and your answer in the *Markdown* cells of the Jupyter notebook. Ensure that the solution is written neatly enough to understand and grade.
3. Use [Quarto](#) to print the *.ipynb* file as HTML. You will need to open the command prompt, navigate to the directory containing the file, and use the command: `quarto render filename.ipynb --to html`. Submit the HTML file.
4. The assignment is worth 100 points, and is due on **5th October 2025 at 11:59 pm**.
5. **Five points are properly formatting the assignment.** The breakdown is as follows:
  - Must be an HTML file rendered using Quarto (2 pts).
  - There aren't excessively long outputs of extraneous information (e.g. no printouts of entire data frames without good reason, there aren't long printouts of which iteration a loop is on, there aren't long sections of commented-out code, etc.) (1 pt)
  - Final answers of each question are written in Markdown cells (1 pt).
  - There is no piece of unnecessary / redundant code, and no unnecessary / redundant text (1 pt)

## A.1 GDP per Capita (35 pts)

### A.1.1 List Comprehension

USA's GDP per capita from 1960 to 2021 is given by the tuple `T` in the code cell below. The values are arranged in ascending order of the year, i.e., the first value is for 1960, the second value is for 1961, and so on.

```
T = (3007, 3067, 3244, 3375, 3574, 3828, 4146, 4336, 4696, 5032, 5234, 5609, 6094, 6726, 7226, 7801)
```

#### A.1.1.1

Use list comprehension to create a list of the gaps between consecutive entries in `T`, i.e, the increase in GDP per capita with respect to the previous year. The list with gaps should look like: `[60, 177, ...]`. Let the name of this list be `GDP_increase`.

*(4 points)*

#### A.1.1.2

Use `GDP_increase` to find the maximum gap size, i.e, the maximum increase in GDP per capita.

*(1 point)*

### A.1.2

Use list comprehension with `GDP_increase` to find the percentage of gaps that have size greater than \$1000.

*(3 points)*

#### A.1.2.1

Use list comprehension with `GDP_increase` to print the list of years in which the GDP per capita increase was more than \$2000.

**Hint:** The `enumerate()` function may help.

*(4 points)*



### A.1.2.2

Use list comprehension to:

1. Create a list that consists of the difference between the maximum and minimum GDP per capita values for each of the 5 year-periods starting from 1976, i.e., for the periods 1976-1980, 1981-1985, 1986-1990, ..., 2016-2020.
2. Find the five year period in which the difference (*between the maximum and minimum GDP per capita values*) was the least.

(4 + 2 points)

### A.1.3 Dictionary

The GDP per capita of USA for most years from 1960 to 2021 is given by the dictionary D given in the code cell below.

```
D = {'1960':3007, '1961':3067, '1962':3244, '1963':3375, '1964':3574, '1965':3828, '1966':4146, '1967':4414, '1968':4687, '1969':4960, '1970':5233, '1971':5506, '1972':5779, '1973':6052, '1974':6325, '1975':6598, '1976':6871, '1977':7144, '1978':7417, '1979':7690, '1980':7963, '1981':8236, '1982':8509, '1983':8782, '1984':9055, '1985':9328, '1986':9601, '1987':9874, '1988':10147, '1989':10420, '1990':10693, '1991':10966, '1992':11239, '1993':11512, '1994':11785, '1995':12058, '1996':12331, '1997':12604, '1998':12877, '1999':13150, '2000':13423, '2001':13696, '2002':13969, '2003':14242, '2004':14515, '2005':14788, '2006':15061, '2007':15334, '2008':15607, '2009':15880, '2010':16153, '2011':16426, '2012':16699, '2013':16972, '2014':17245, '2015':17518, '2016':17791, '2017':18064, '2018':18337, '2019':18610, '2020':18883, '2021':19156}
```

Find:

#### A.1.3.1

The GDP per capita in 2015

(2 points)

#### A.1.3.2

The GDP per capita of 2014 is missing. Update the dictionary to include the GDP per capita of 2014 as the average of the GDP per capita of 2013 and 2015.

(5 points)

#### A.1.3.3

Impute the GDP per capita of other missing years in the same manner as in (2), i.e., as the average GDP per capita of the previous year and the next year. Note that the GDP per capita is not missing for any two consecutive years.

(5 points)

#### A.1.3.4

Print the years and the imputed GDP per capita for the years having a missing value of GDP per capita in (3).

(5 points)

## A.2 Student Survey (40 pts)

### A.2.1 Marriage age responses

Below is a list consisting of responses to the question: “At what age do you think you will marry?” from students of the STAT303-1 Fall 2022 class.

```
exp_marriage_age=['24','30','28','29','30','27','26','28','30+','26','28','30','30','30','pr
```

Use list comprehension to:

#### A.2.1.1

Remove the elements that are not integers - such as ‘probably never’, ‘30+’, etc. What is the length of the new list?

**Hint:** The built-in python function of the `str` class - `isdigit()` may be useful to check if the string contains only digits.

(3 points)

#### A.2.1.2

Cap the values greater than 80 to 80, in the clean list obtained in (1). What is the mean age when people expect to marry in the new list?

(3 + 4 points)

#### A.2.1.3

Determine the percentage of people who expect to marry at an age of 30 or more.

(5 points)

### A.2.2 Majors/Minors: Nested lists of student majors.

Below is the list consisting of the majors / minors of students of the course STAT303-1 Fall 2023. This data is a list of lists, where each sub-list (*smaller list within the outer larger list*) consists of the majors / minors of a student. Most of the students have majors / minors in one or more of these four areas:

1. Math / Statistics / Computer Science
2. Humanities / Communication
3. Social Sciences / Education
4. Physical Sciences / Natural Sciences / Engineering

There are some students having majors / minors in other areas as well.

Use list comprehension for all the questions below.

```
majors_minors = majors_minors = [['Humanities / Communications', 'Math / Statistics / Comput
```

### A.2.2.1

How many students have major / minor in any three of the above mentioned four areas?

(1 point)

### A.2.2.2

How many students have *Math / Statistics / Computer Science* as an area of their major / minor?

**Hint:** Nested list comprehension

(4 points)

### A.2.2.3

How many students have *Math / Statistics / Computer Science* as the **only area** of their major / minor?

**Hint:** Nested list comprehension

(5 points)



#### **A.3.1.2**

If the object in (1) is a dictionary, what is the datatype of the values of the dictionary?

*(2 points)*

#### **A.3.1.3**

If the object in (1) is a dictionary, what is the datatype of the elements within the values of the dictionary?

*(2 points)*

#### **A.3.1.4**

How many calories are there in `Iced Coffee`?

*(4 points)*

#### **A.3.1.5**

Which drink(s) have the highest amount of protein in them, and what is that protein amount?

#### **A.3.1.6**

Which drink(s) have a fat content of more than 10g, and what is their fat content?

*(5 points)*

#### **A.3.1.7**

Use dictionary-comprehension to print the name and `carb` content of all drinks that have a `carb` content of more than 50 units.

**Hint:** It will be a nested dictionary comprehension.

*(5 points)*

## A.4 AI Use Disclosure (5 pts)

In 1–3 short paragraphs, clearly state whether **generative AI tools** were used to complete any part of this assignment.

- If **no AI was used**, write:  
*“I did not use generative AI tools on this assignment.”*
- If **AI was used**, you must specify:
  1. **Tool(s)** used (e.g., ChatGPT, GitHub Copilot, Claude, etc.)
  2. **How** they were used (e.g., idea generation, debugging, code suggestions, writing help)
  3. **Where** they influenced your work (which questions/sections)
  4. **Edits & verification** you made (how you checked correctness, what you modified)

### A.4.1 AI Use Template

- **Used AI?** Yes / No
- **Tool(s):**
- **Purpose / How used (1–3 sentences):**
- **Scope (which questions/sections):**
- **Edits & verification (what you changed and how you checked correctness):**

**Example (for illustration only):**

- Used AI? Yes
- Tools: ChatGPT (free), GitHub Copilot
- How used: Asked for a pandas snippet to impute missing values and for a seaborn example.
- Scope: Q1(b) imputation, Q2(a) first draft of bar chart code.
- Edits & verification: Rewrote code to use `groupby().transform('median')`; validated results by comparing summary stats; added axis labels manually.

**Grading (5 points):**

- **5 points** = complete, specific, and honest (tools named, usage described clearly)
- **3–4 points** = vague or missing some details

- **1–2 points** = minimal effort or very unclear
- **0 points** = missing or misleading disclosure

# B Assignment B

Pandas

## Instructions

1. You may talk to a friend, discuss the questions and potential directions for solving them. However, you need to write your own solutions and code separately, and not as a group activity.
2. Do not write your name on the assignment.
3. Write your code in the *Code* cells and your answer in the *Markdown* cells of the Jupyter notebook. Ensure that the solution is written neatly enough to understand and grade.
4. Use [Quarto](#) to print the *.ipynb* file as HTML. You will need to open the command prompt, navigate to the directory containing the file, and use the command: `quarto render filename.ipynb --to html`. Submit the HTML file.
5. The assignment is worth 100 points, and is due on **13th October 2025 at 11:59 pm**. There is an optional bonus question worth 10 points. You can score a maximum of 110 (out of 100) points.
6. You are **not allowed to use a for loop** or any other kind of loop in this assignment.

## B.1 GDP per capita & social indicators

Read the file *social\_indicator.txt* with python. Set the first column as the index when reading the file. How many observations and variables are there in the data?

(4 points)



### B.1.1

Which variables have the strongest and weakest correlations with GDP per capita? Note that `lifeFemale` and `lifeMale` are the female and male life expectancies respectively.

Note that only when the magnitude of the correlation is considered when judging a correlation as strong or weak.

(4 points)

### B.1.2

Does the male economic activity (*in the column `economicActivityMale`*) have a positive or negative correlation with GDP per capita? Did you expect the positive/negative correlation? If not, why do you think you are observing that correlation?

(4 points)

### B.1.3

What is the rank of the US amongst all countries in terms of GDP per capita? Which countries lie immediately above, and immediately below the US in the ranking in terms of GDP per capita? The country having the highest GDP per capita ranks 1.

Note that:

1. The US is mentioned as *United States* in the data.
2. The country with the highest GDP per capita will have rank 1, the country with the second highest GDP per capita will have rank 2, and so on.

**Hint:** `rank()`

(4 points)

### B.1.4

Which country or countries rank among the top 20 in terms of each of these social indicators - `economicActivityFemale`, `economicActivityMale`, `gdpPerCapita`, `lifeFemale`, `lifeMale`? For each of these social indicators, the country having the largest value ranks 1 for that indicator.

(6 points)

**Hint:**

1. Use `rank()`. Note that this method is different from the method given in the hint of the previous question. This method is of the `DataFrame` class, while the one in the previous question is of the `Series` class. *This part of the hint is just for your understanding. You don't need to write any code in this part.*
2. Using `rank()`, get the `DataFrame` consisting of the ranks of countries on each of the relevant social indicators (*one line of code*).
3. In the `DataFrame` obtained in (2), filter the rows for which the maximum rank is less than or equal to 20 (*one line of code*).

### B.1.5

On which social indicator among `economicActivityFemale`, `economicActivityMale`, `gdpPerCapita`, `lifeFemale`, `lifeMale`, `illiteracyFemale`, `illiteracyMale`, `infantMortality`, and `totalfertilityrate` does the US have its worst ranking, and what is the rank? Note that for `illiteracyFemale`, `illiteracyMale`, and `infantMortality`, the country having the lowest value will rank 1, in contrast to the other social indicators.

(8 points)

### B.1.6

Find all the countries that have a lower GDP per capita than the US, despite having lower illiteracy rates (for both genders), higher economic activity (for both genders), higher life expectancy (for both genders), and lower infant mortality rate than the US?

(6 points)

## B.2 GDP per capita vs social indicators

We'll use the same data as in the previous question. For the questions below, assume that all numeric columns, **except GDP per capita**, are social indicators.

### B.2.1

Use the column `geographic_location` to create a new column called `continent`. Merge the values of the `geographic_location` column appropriately to obtain 6 distinct values for the `continent` column – *Asia*, *Africa*, *North America*, *South America*, *Europe* and *Oceania*. Drop the column `geographic_location`. Print the first 5 observations of the updated `DataFrame`.

(8 points)

**Hint:**

1. Use `value_counts()` to see the values of `geographic_location`. The code `if 'Asia' in 'something'` will return `True` if `'something'` contains the string `'Asia'`, for example, if `'something'` is `'North Asia'`, the code will return `True`. *This part of the hint is just for your understanding. You don't need to write any code in this part.*
2. Apply a lambda function on the Series `geographic_location` to replace a string that contains `'Asia'` with `'Asia'`, replace a string that contains `'Europe'` with `'Europe'`, and replace a string that contains `'Africa'` with `'Africa'`. *This will be a single line of code.*
3. Rename the column `geographic_location` to `continent`.

### B.2.2

Sort the column labels lexicographically. Drop the columns `region` and `contraception`. Print the first 5 observations of the updated DataFrame.

**Hint:** `sort_index()`

*(4 points)*

### B.2.3

Find the percentage of the total countries in each continent.

**Hint:** One line of code with `value_counts()` and `shape`

*(4 points)*

### B.2.4

Which country has the highest GDP per capita? Let us call it country *G*.

*(4 points)*

### B.2.5

We need to find the African country that is the closest to country *G* with regard to social indicators. Perform the following steps:

### B.2.5.1

Standardize each of the social indicators to a standard normal distribution so that all of them are on the same scale (remember to exclude *GDP per capita* from social indicators).

**Hint:**

1. For scaling a random variable to standard normal, subtract the mean from each value of the variable, and divide by its standard deviation.
2. Use the apply method with a lambda function to scale all the social indicators to standard normal.
3. The above (1) and (2) together is a single line of code.

(6 points)

### B.2.5.2

Compute the [Manhattan distance](#) between country *G* and each of the African countries, based on the scaled social indicators.

**Hint:**

1. Broadcast a Series to a DataFrame
2. The Manhattan distance between two points  $(x_1, x_2, \dots, x_p)$  and  $(y_1, y_2, \dots, y_p)$  is  $|x_1 - y_1| + |x_2 - y_2| + \dots + |x_p - y_p|$ , where  $|\cdot|$  stands for absolute value (for example,  $|-2| = 2$ ;  $|3| = 3$ ).

(8 points)

### B.2.5.3

Identify the African country, say country *A*, with the least Manhattan distance to country *G*.

(8 points)

### B.2.6

Find the correlation between the Manhattan distance from country *G* and GDP per capita for African countries.

(6 points)

### B.2.7

Based on the correlation coefficient in 2(*f*), do you think African countries should try to emulate the social characteristics of country *G*? Justify your answer.

(4 points)

## B.3 Medical data

Read the data sets *conditions.csv* and *patients.csv*. Suppose we are interested in studying patients with prediabetes condition. Do not drop or compute any missing values. In *conditions.csv*, the patient IDs are stored in column `PATIENT`, and the medical conditions are stored in column `DESCRIPTION`. In *patient.csv*, the patient IDs are stored in column `Id`.

### B.3.1

Print the patient IDs of all the patients with prediabetes condition.

(4 points)

### B.3.2

Make a subset of the data with only prediabetes patients. How many prediabetes patients are there?

(4 points)

**Hint:** `.isin()`

### B.3.3

What proportion of the total `HEALTHCARE_EXPENSES` of all the patients correspond to the `HEALTHCARE_EXPENSES` of prediabetes patients.

(4 points)

## B.4 Bonus question

This is an optional question with no partial credit. You will get points only if your solution is completely correct. We advise you to attempt it only when you are done with the rest of the assignment.

(10 points)

Read the file *STAT303-1 survey for data analysis.csv*. In this question, we'll work to clean this data a bit. As with every question, you are **not** allowed to use a **for** loop or any other loop.

Execute the following code to read the data and clean the column names.

```
survey_data = pd.read_csv('STAT303-1 survey for data analysis.csv')
new_col_names = ['parties_per_month', 'smoke', 'weed', 'introvert_extrovert', 'love_first_sig']
survey_data.columns = list(survey_data.columns[0:2])+new_col_names
```

Check the datatype of the variables using the `dtypes` attribute of the Pandas DataFrame object. You will notice that only two variables are numeric. However, if you check the first few observations of the data with the function `head()` you will find there are several more variables that seem to have numeric values.

Write a function that accepts a Pandas Series (or a column of a Pandas DataFrame object) as argument, and if the datatype of the Series is non-numeric, does the following:

1. Checks if at least 10 values of the Series contain a digit in them.
2. If at least 10 values are found to contain a digit, then:
  - A. Eliminate the characters `~`, `+`, and `,` from all the values of the Series.
  - B. Convert the Series to numeric (with coercion if needed).
3. If at least 10 values are NOT found to contain a digit, then:
  - A. If the values of the Series are `'Yes'` and `'No'`, then replace `'Yes'` with `1` and `'No'` with `0`. The Series datatype must change to numeric as well.
  - B. If the values of the Series are `'Agree'` and `'Disagree'`, then replace `'Agree'` with `1` and `'Disagree'` with `0`. The Series datatype must change to numeric as well.

Apply the function to each column of `survey_data` using the `apply()` method. Save the updated DataFrame as `survey_data_clean`. Then, execute the following code.

```
survey_data_clean.describe().loc['mean',:]
```

The above code should print out the mean values of 28 numeric columns in the data `survey_data_clean`.

The variables that you should see as numeric in `survey_data_clean` are given in the list `numeric_columns` below (*this is just to check your work*):

```
numeric_columns = ['parties_per_month', 'love_first_sight', 'num_insta_followers',
 'expected_marriage_age', 'expected_starting_salary',
 'minutes_ex_per_week', 'sleep_hours_per_day',
 'farthest_distance_travelled', 'fav_number', 'internet_hours_per_day',
 'only_child', 'num_majors_minors', 'high_school_GPA', 'NU_GPA', 'age',
 'height', 'height_father', 'height_mother', 'procrastinator',
 'num_clubs', 'student_athlete', 'AP_stats', 'used_python_before',
 'childhood_in_US', 'cant_change_math_ability',
 'can_change_math_ability', 'math_is_genetic',
 'much_effort_is_lack_of_talent']
```

Note that your **function must be general**, i.e., it must work for any other dataset as well. This means, you cannot hard code a column name, or anything specific to `survey_data` in the function.

## B.5 AI Use Disclosure

In **1–3 short paragraphs**, clearly state whether **generative AI tools** were used to complete any part of this assignment.

- If **no AI was used**, write:  
*“I did not use generative AI tools on this assignment.”*
- If **AI was used**, you must specify:
  1. **Tool(s)** used (e.g., ChatGPT, GitHub Copilot, Claude, etc.)
  2. **How** they were used (e.g., idea generation, debugging, code suggestions, writing help)
  3. **Where** they influenced your work (which questions/sections)
  4. **Edits & verification** you made (how you checked correctness, what you modified)

### B.5.1 AI Use Template

- **Used AI?** Yes / No
- **Tool(s):**
- **Purpose / How used (1–3 sentences):**
- **Scope (which questions/sections):**
- **Edits & verification (what you changed and how you checked correctness):**

**Example (for illustration only):**

- Used AI? Yes
- Tools: ChatGPT (free), GitHub Copilot
- How used: Asked for a pandas snippet to impute missing values and for a seaborn example.
- Scope: Q1(b) imputation, Q2(a) first draft of bar chart code.
- Edits & verification: Rewrote code to use `groupby().transform('median')`; validated results by comparing summary stats; added axis labels manually.

**Grading (5 points):**

- **5 points** = complete, specific, and honest (tools named, usage described clearly)
- **3–4 points** = vague or missing some details
- **1–2 points** = minimal effort or very unclear
- **0 points** = missing or misleading disclosure



# C Assignment C

NumPy

## Instructions

1. You may talk to a friend, discuss the questions and potential directions for solving them. However, you need to write your own solutions and code separately, and not as a group activity.
2. Write your code in the *Code* cells and your answer in the *Markdown* cells of the Jupyter notebook. Ensure that the solution is written neatly enough to understand and grade.
3. Use [Quarto](#) to print the *.ipynb* file as HTML. You will need to open the command prompt, navigate to the directory containing the file, and use the command: `quarto render filename.ipynb --to html`. Submit the HTML file.
4. The assignment is worth 100 points, and is due on **25th October 2025 at 11:59 pm**.
5. **Three points are properly formatting the assignment.** The breakdown is as follows:
  - Must be an HTML file rendered using Quarto.
  - There aren't excessively long outputs of extraneous information (e.g. no printouts of entire data frames without good reason, there aren't long printouts of which iteration a loop is on, there aren't long sections of commented-out code, etc.) (1 pt)
  - Final answers of each question are written in Markdown cells (1 pt).
  - There is no piece of unnecessary / redundant code, and no unnecessary / redundant text (1 pt)

## C.1 Air quality sensors (30 pts)

### (An application of broadcasting NumPy arrays)

Air quality sensors are used to measure the amount of contaminants in air. This question will guide you in finding the location of installing 50 air quality sensors in the State of Colorado, such that they are as far away from each other as possible. The approach below is a greedy algorithm to find an approximate [Maximin design](#).

The file `colorado_coordinate_grid.txt` contains the coordinate-pairs (latitude and longitude) of potential locations for installing an air quality sensor.

#### C.1.1 Data

Read the file with NumPy. How many coordinate-pairs are there in the file?

Note that:

1. A coordinate-pair means a latitude-longitude pair.
2. ‘Air quality sensor’ will be referred as ‘sensor’ in the questions below for brevity.

(2 points)

#### C.1.2 First sensor

The first sensor is to be installed closest to Denver (*closest in terms of Euclidean distance*). Find the coordinate-pair of the location where the first sensor will be installed. The coordinate-pair of Denver is: [39.7392° N, 104.9903° W]

Note that the suffixes ° N and ° W are omitted in the file `colorado_coordinate_grid.txt`.

**Hint:** Broadcasting

(2 points)

#### C.1.3 Second sensor

Find the coordinate-pair of the installation of the next sensor, such that it is as far as possible from the first sensor installed near Denver.

**Hint:** Broadcasting

(4 points)

### C.1.4 First two sensors

Stack the coordinate-pairs of the first and second sensors vertically to obtain a 2 x 2 NumPy array. Name the array as `air_sensor_coordinates`.

Run the code below to check if your results seem correct. The coordinate-pairs of the two air quality sensors will be marked as blue dots.

(4 points)

```
import matplotlib.pyplot as plt
def sensor_viz():
 img = plt.imread("colorado.jpg")
 fig, ax = plt.subplots(figsize=(10, 100),dpi=80)
 fig.set_size_inches(10.5, 15)
 ax.imshow(img,extent=[-109, -102, 37, 41])
 plt.scatter(y = air_sensor_coordinates[:,0], x = -air_sensor_coordinates[:,1])
 plt.xlim(-109.05,-101.95)
 plt.ylim(36.95,41.05)
 plt.xlabel("Longitude")
 plt.ylabel("Latitude")
sensor_viz()
```

### C.1.5 Third sensor

Now you need to find the coordinate-pair for installing the third sensor such that it is far away from the two already-installed sensors. Proceed as follows:

1. Find the minimum distance of each coordinate-pair in *colorado\_coordinate\_grid.txt* from the two already installed sensors. For example, if a coordinate-pair is at a distance of 5 units from the first sensor, and 10 units from the second sensor, then its minimum distance from the sensors will be  $\min(5, 10) = 5$  units.
2. Select the coordinate-pair (from *colorado\_coordinate\_grid.txt*) whose minimum distance from the two already installed sensors is the maximum.
3. Stack the coordinate-pair of the third air quality sensor vertically on the array `air_sensor_coordinates`.

Call the function `sensor_viz()` to check if your results seem correct. The coordinate-pairs of the three air quality sensors will be marked as blue dots.

#### Hint:

For step (1) above:

1. Define a function which computes the distances of a coordinate-pair from all the coordinates of `air_sensor_coordinates`, and returns the minimum distance.
2. Apply the function on all the coordinate-pairs in `colorado_coordinate_grid.txt` using the NumPy function `apply_along_axis()`.

(12 points)

### C.1.6 All 50 sensors

You need to find 47 more coordinate-pairs to install air quality sensors well-spread across Colorado. We will generalize the steps in the previous question to proceed as follows:

1. Suppose you have already found the coordinate-pairs for the installation of  $i$  sensors.
2. Find the minimum distance of each coordinate in `colorado_coordinate_grid.txt` from the  $i$  already installed sensors. For example, if a coordinate-pair is at a distance of  $d_1$  from the first sensor,  $d_2$  from the second sensor,..., and  $d_i$  from the  $i^{th}$  sensor, then its minimum distance from the sensors will be  $\min(d_1, d_2, \dots, d_i)$ .
3. Select the  $i + 1^{th}$  coordinate-pair (from `colorado_coordinate_grid.txt`) as the one whose minimum distance from the  $i$  already installed sensors is the maximum.

Call the function `sensor_viz()` to check if your results seem correct. You should see 50 blue dots well spread across Colorado.

(6 points)

## C.2 Sales (16 pts)

### (An application of matrix multiplication with NumPy arrays)

When the monthly sales of a product are subject to seasonal fluctuations, a curve that approximates the sales formula might have the form:

$$y = a + b * x + c * \sin\left(2 * \pi * \frac{x}{12}\right),$$

where  $x$  is the time since the starting point in months and  $y$  is the monthly sales in USD (million). The term  $a + b * x$  gives the basic sales trend and the sin term reflects the seasonal changes in sales. Suppose the model parameters (i.e.,  $a$ ,  $b$ , and  $c$ ) are estimated and put on the list below for the sales of a certain brand of sunscreen starting June 1, 2017.

```
model_parameters = [2, 5, 18]
```

Then, the total monthly sales in June 2017 will be calculated by plugging 1 as  $x$  into the equation.

Using matrix multiplication with NumPy, we wish to estimate the total sales between June 1 2017 and March 1, 2020. *(So many models failed to predict sales after that - probably due to covid.)*

Proceed as follows.

### C.2.1 Create first array

Create a numpy array where the first column is all 1s, the second column is a range of numbers from 1 to the total number of months from June 1 2017 to March 1 2020 and the third column is  $\sin(2 * \pi * x/12)$  values with  $x$  values as plugged-in in the second column.

*(8 points)*

### C.2.2 Create second array

Create an array from the list `model_parameters`.

*(2 points)*

### C.2.3 Multiply arrays

Use matrix multiplication to get the monthly sales estimates for each month in the range: June 1 2017 and March 1, 2020.

*(4 points)*

### C.2.4 Sum array elements

Find the total sales between June 1 2017 and March 1, 2020.

*(2 points)*

## C.3 Starbucks restaurant (35 pts)

(An application of NumPy matrix operations and element-wise operations)

The dataset *starbucks-menu-nutrition-drinks.csv* consists of the nutrition information of each drink in Starbucks. The dataset *starbucks\_DrinkType\_USStates.csv* consists of the number of Starbucks drinks of each type consumed by each US State per 10,000 people.

In this section, you will use NumPy operations to analyze consumption patterns and nutritional content across different states.

### C.3.1 Most consumed drink type (name) and state with highest consumption (8 pts)

Using NumPy operations, answer the following questions:

1. Among all the drink types offered by Starbucks, which drink type is the most consumed across all states combined?
2. Among all the states, which state has the highest total consumption of Starbucks drinks (summed across all drink types)?

**Hint:** Use NumPy's `sum()` and `argmax()` functions to perform the calculations efficiently.

(4 points each)

### C.3.2 Highest total fat consumption drink type in Illinois (10 pts)

For this question, you need to calculate which drink type accounts for the highest total fat consumption in Illinois. Proceed as follows:

1. The `Total_Fat` of a drink is the sum of saturated fat, UnSaturated Fat, and Trans Fat.
2. For Illinois specifically, determine which drink type has the highest total fat consumption.

(10 points)

### C.3.3 States with lowest total fat and highest protein consumption (10 pts)

Using NumPy matrix operations, calculate the total nutrient consumption for each state:

1. Find the state with the **lowest** total fat consumption.
2. Find the state with the **highest** total protein consumption.

*(5 points each)*

### C.3.4 Counting states with protein $> 2 \times$ fat consumption (7 pts)

Using the total fat and protein consumption arrays calculated in the previous question, determine for how many states the protein consumption per 10,000 people is **more than twice** the total fat consumption per 10,000 people.

**Important:** You are **not allowed** to create new columns in your DataFrame. Use only NumPy array operations.

**Hint:** 1. Use NumPy's comparison operators to create a boolean array where  $\text{protein} > 2 \times \text{fat}$  2. Use `np.sum()` on the boolean array to count the number of `True` values

*(7 points)*

## C.4 Exercise minutes (16 pts)

(An application of parallel computation with NumPy)

This problem demonstrates the benefit of generating pseudo random number matrix with NumPy.

The list `exercise_minutes` below consists of exercise minutes per week of the students of STAT303-1 Fall 2022 class.

We wish to find the **95% confidence interval** of mean `exercise_minutes`, using [Bootstrapping](#).

Bootstrapping is a non-parametric method for obtaining confidence interval. The method is as follows.

- (a) Suppose the list `exercise_minutes` has  $N$  values.
- (b) Randomly sample  $N$  values with replacement from `exercise_minutes`
- (c) Find the mean of the  $N$  values obtained in (b)
- (d) Repeat steps (b) and (c) 10,000 times

- (e) The 95% Confidence interval is the range between the 2.5% and 97.5% percentile values of the 10,000 means obtained in (c)

```
exercise_minutes=[240, 180, 60, 300, 0, 360, 60, 140, 60, 0, 150, 60, 0, 6, 60, 300, 90, 100
```

Answer the following questions.

### C.4.1 Sequential computation without NumPy

Without using NumPy, compute the:

1. Confidence interval of mean `exercise_minutes`, and
2. Time taken to execute the code

#### Hints:

1. You may use the library `random`.
2. You may use the library `time` for computing the time taken to execute the code.

*(8 points)*

### C.4.2 Parallel computation with NumPy

Using NumPy, and without using loops, compute the:

1. Confidence interval of mean `exercise_minutes`, and
2. Time taken to execute the code

*(7 points)*

### C.4.3 Time saving with NumPy

Report the ratio of time taken to execute the code without NumPy to the time taken to execute the code with NumPy.

*(1 point)*



# D Assignment D

Data Visualization

## Instructions

1. You may talk to a friend, discuss the questions and potential directions for solving them. However, you need to write your own solutions and code separately, and not as a group activity.
2. Do not write your name on the assignment.
3. Write your code in the *Code* cells and your answer in the *Markdown* cells of the Jupyter notebook. Ensure that the solution is written neatly enough to understand and grade.
4. Use [Quarto](#) to print the *.ipynb* file as HTML. You will need to open the command prompt, navigate to the directory containing the file, and use the command: `quarto render filename.ipynb --to html`. Submit the HTML file.
5. The assignment is worth 100 points, and is due on **1st November 2025 at 11:59 pm**.
6. Some questions in this assignment do not have a single correct answer. As data visualization is subject to interpretation, any logically sound answer / explanation is acceptable.
7. There is a bonus question worth 20 points. However, there is no partial credit for the bonus question. You will get 20 or 0. If everything is correct, you can score 120 out of 100 in the assignment.

# Exploring factors associated with profitability of a movie

In this assignment we'll attempt to find the factors (or variables) that make a movie profitable.

Read the movies data from [here](#), in a Pandas DataFrame named as `movies_data`. The profit of movie is defined as:

$$profit = Worldwide\ Gross - Production\ Budget$$

## D.1 Time trend

Let us analyze if the profitability of a movie is associated with the time of its release.

### D.1.1 Month of release

#### D.1.1.1

Make an appropriate plot to visualize the mean profit of movies released each month.

**Hint:**

1. Use the Pandas function `to_datetime()` to convert *Release Date* to a `datetime` datatype.
2. Use the library `datetime` to extract the month from *Release Date*.

*(6 points)*

### D.1.1.2

Based on the plot, which seasons have been the most and least profitable (on an average) for a movie release. Don't worry about the exact start and end date of seasons. Don't perform any computations. Just make comments based on the plot. You can use seasons such as *early summer*, *late spring* etc.

(2 points)

## D.1.2 Month of release with number of movies in each genre

### D.1.2.1

Now that we know the most profitable season for releasing movies, let us visualize if some **genres** are more popular during certain seasons.

Use the code below to create a new column called **genre**.

```
#Combining Major Genre
movies_data['genre'] = movies_data['Major Genre'].apply(lambda x: 'Comedy' if x != None and 'Com' in x else 'Drama')
```

Make an appropriate plot to visualize the number of movies released for each **genre** in each calendar month.

(8 points)

#### Hint:

1. Use `barplot()` with *estimator* as `len`
2. Use the *hue* argument

### D.1.2.2

Based on the above plot, which **genre** is the most popular during the most profitable season of release? And which genre is the most popular during the least profitable season of release?

(2 points)

### D.1.3 Month of release with proportion of movies in each genre

#### D.1.3.1

Visualize the proportion of movies in each **genre** for each month of release.

Use the code below to re-arrange your data that will help with creating the visualization

```
genre_proportion_release_month = pd.crosstab(index=movies_data['release_month'],
 columns=movies_data['genre'],
 normalize="index")
genre_proportion_release_month.head()
```

**Hint:**

1. Make a 100% stacked barplot with the Pandas `plot()` function
2. Use the argument `bbox_to_anchor` with the Matplotlib function `legend()` to place the legend outside the plot area.

*(8 points)*

#### D.1.3.2

Which **genre** is the most popular during the month of May, and which one is the most popular during December?

*(2 points)*

### D.1.4 Year of release with genre

#### D.1.4.1

Make an appropriate figure to visualize the average profit of movies of each **genre** for each year. Consider only the movies released from 1991 to 2010. Also show the 95% confidence interval in the average profit.

**Hint:**

1. Use the library `datetime` to extract year from `Release Date`.
2. Use the Seaborn `Facetgrid()` object.
3. A figure can have multiple subplots. Put the figure for each genre in a separate subplot.

*(6 points)*

#### D.1.4.2

Based on the figure above, which **genre**'s profitability seems to be increasing over the years, and which **genre** has the least uncertainty in profit for most of the years.

*(2 points)*

## D.2 Associations

### D.2.1 Pairplot / heatmap

#### D.2.1.1

Make a pairplot and heatmap of all the continuous variables in the data.

*(8 points)*

#### D.2.1.2

Are there any trends that you can see in the pairplot, but not in the heatmap?

*(2 points)*

#### D.2.1.3

Based on the plots in 2(a)(i), which variables are associated with profit?

*(2 points)*

#### D.2.1.4

Among the variables listed in 2(a)(iii), select a subset of variables such that none of them are highly associated with each other. The rest of the variables identified in 2(a)(iii) are redundant with regard to association with profit.

*(2 points)*

## D.2.2 Nested associations

### D.2.2.1

Use the code below to create some new columns.

```
movies_data['screenplay'] = movies_data.Source.apply(lambda x: 'Non-original' if x != 'Original' else 'Original')
movies_data['rating'] = movies_data['MPAA Rating'].apply(lambda x: 'R rated' if x == 'R' else 'Not R rated')
movies_data['fiction'] = movies_data['Creative Type'].apply(lambda x: 'Contemporary' if x == 'Contemporary' else 'Not Contemporary')
```

Make an appropriate figure to visualize the association of the number of IMDB votes with profit for each genre (use the variable `genre`). Which genre has the highest association between profit and IMDB votes?

*(8 points)*

### D.2.2.2

Make an appropriate figure to visualize the association between the number of IMDB votes and profit, for each combination of the fiction type (use the variable `fiction`) and the movie rating (use the variable `rating`).

For which combination of `fiction` and `rating` categories do you observe the highest association between IMDB votes and profit?

*(8 points)*

**Hint:** Use `row` and `col` attributes of the Seaborn `Facetgrid()` object.

## D.2.3 Profit based on movie director

### D.2.3.1

Consider the directors who have directed more than 10 movies (based on the dataset). Make a horizontal barplot that shows the mean profit of the movies of these directors along with the 95% confidence interval. Sort the bars of the barplot such that the director with the highest mean profit is at the top.

If the dataset `director_with_more_than_10_movies` has only those movies that correspond to directors with more than 10 movies, then the following code will give you the order in which the names of the directors must appear in the barplot:

*(8 points)*

```
director_with_more_than_10_movies[['Director', 'profit']].groupby('Director').mean().sort_values(ascending=False).index.to_list()
```

### D.2.3.2

Based on the above plot, which director has the highest mean profitability, and which one has the highest variation in profitability?

*(2 points)*

## D.3 Distributions

### D.3.1 Distribution of profit based on genre (boxplots)

#### D.3.1.1

Make boxplots to visualize the distribution of `profit` based on `genre`. Based on the plot, which genre has the most profitable movies?

*(6 points)*

#### D.3.1.2

Which genre has the most variation in profit, and which one has the least?

*(2 points)*

### D.3.2 Distribution of profit based on genre (density plots)

#### D.3.2.1

Make density plots of `profit` based on `genre`. Adjust the limit on the horizontal axis, so that the plots are clearly visible.

*(6 points)*

#### **D.3.2.2**

What additional insight / trend can you see in the above plot that you cannot see with the boxplots?

*(2 points)*

### **D.4 Insights**

From all the visualizations above, describe the insights you get about the factors associated with the profitability of a movie.

Also, elaborate on the extent to which these trends can be generalized. For example, comment on whether these trends be generalized to the current time and all the Hollywood movies? If not, is there any time period or type of movie to which these trends can be applicable?

*(4+ 4 points)*



## Bonus question: Stock Market

This question is worth 20 points. However, there is no partial credit. It will be 20 or 0.

The stock market is made up of exchanges, such as the New York Stock Exchange and the Nasdaq. Stocks are listed on a specific exchange, which brings buyers and sellers together and acts as a market for the shares of those stocks. The exchange tracks the supply and demand — and directly related, the price — of each stock.

A market index tracks the performance of a group of stocks, which either represents the market as a whole or a specific sector of the market, like technology or retail companies. You're likely to hear most about the S&P 500, the Nasdaq composite and the Dow Jones Industrial Average; they are often used as proxies for the performance of the overall market.

There are two types of investment strategies: active and passive.

### Active investing

An active investment strategy involves using the information acquired by expert stock analysts to actively buy and sell stocks with specific characteristics. The goal is to beat the results of the indices and general stock market with higher returns and/or lower risk.

### Passive Investing

Passive investors have a buy-and-hold mentality that focuses on benefitting from the overall increase in market prices over time. One of the major benefits of passive investing is that it minimizes the mistakes investors can make when they react emotionally to every move of the stock market.

The easiest way to implement a passive approach is to buy and hold an index fund that follows one of the major indices like the S&P 500, Dow Jones, or Russell 2000 (small-cap stocks). These funds pool money from multiple investors to buy the individual stocks, bonds, or securities that make up their market index. When the index changes its components, the index funds that follow it also switch up their holdings to match.

## Tasks

In this exercise, we use S&P 500 index as an example to explore the gains/returns for passive investment.

## Return

The data is provided in a csv file, with dates roughly from 2000 to 2022. And we use the column **Close** as the **price** on a specific day.

If we buy the index on  $t_1$  and sell it on  $t_2$ , the **return** is defined as

$$r_{t_1, t_2} = (P_{t_2} - P_{t_1}) / P_{t_1}$$

Sometimes we are interested on the return on holding the index for a specific period  $T$ , the return is

$$r_{t, T} = (P_{t+T} - P_t) / P_t$$

where  $t$  is the date we buy the stock and  $T$  is the holding period. Since the stock is not traded on every day, when calculating  $t + T$  we simply skip the non-trading dates.

## Risk

If we take the return as a random variable, we could use standard deviation as its risk. A risk-averse investor expects a stable (low volatility) return.

## Sharpe Ratio

People like high return and low risk investment. But on the other hand, in the market high return always associates high risk (e.g. stock) and low risk means low return (e.g. treasury bonds).

The Sharpe ratio compares the excess return of an investment with its risk to make a single measure:

$$SR = (R - R_f) / \sigma$$

where  $R$  and  $\sigma$  are the expected return and stddev for the investment and  $R_f$  is the risk free rate.

Risk free rate is the rate of return offered by an investment that carries zero risk. In reality there is no truly risk free rate, but we usually take something like three-month U.S. Treasury bill as a proxy as risk free rate. To be simple, in this exercise we just **take risk free rate as 0**.

Read the data from `sp500.csv`. The dataset can be found in the Datasets section of the book.

## D.5 Return on investment

### D.5.1 Impatient investor (daily)

Suppose there is an investor who only holds the index for a single day (buy yesterday sell today).

Based on the data,

#### D.5.1.1

Show the histogram graph for all the possible returns.

#### D.5.1.2

What is the expected return, risk and sharpe ratio?

#### D.5.1.3

Is the return significantly greater than zero (a.k.a positive return) (use a threshold 0.01 for  $p$ -value) ?

**HINT:** use `scipy.stats.ttest_1samp` to do one-sided mean test. (We ignore the fact that  $T$ -test requires the data are sampled from a population of normal distribution, which might not be true in this exercise)

*(6 points)*

### D.5.2 Patient Investor (yearly)

Suppose there is an investor who will hold the index for a year (suppose there are 250 trading days in a year). Do the same analysis as the above:

#### D.5.2.1

Show the histogram graph for all the possible returns.

### D.5.2.2

What is the expected return, risk and sharpe ratio?

### D.5.2.3

Is the return significantly greater than zero (a.k.a positive return) (use a threshold 0.01 for  $p$ -value) ?

*(6 points)*

## D.5.3 From daily to yearly

Explore how the expected return/risk/shape ratio change as we increase our holding period from 1 day to 1 year(250 days).

Show/answer:

### D.5.3.1

At least how many days do you need to hold the index in order to make a significant positive return (threshold 0.01)?

### D.5.3.2

How are the returns associated with the risks for different investment strategies?

### D.5.3.3

Make a graph as shown below.

*(18 points)*

<IPython.core.display.Image object>

# E Assignment E

Data cleaning & preparation

## Instructions

1. You may talk to a friend, discuss the questions and potential directions for solving them. However, you need to write your own solutions and code separately, and not as a group activity.
2. Do not write your name on the assignment.
3. Write your code in the *Code* cells and your answer in the *Markdown* cells of the Jupyter notebook. Ensure that the solution is written neatly enough to understand and grade.
4. Use [Quarto](#) to print the *.ipynb* file as HTML. You will need to open the command prompt, navigate to the directory containing the file, and use the command: `quarto render filename.ipynb --to html`. Submit the HTML file.
5. The assignment is worth 100 points, and is due on **9th November 2025 at 11:59 pm**.
6. There is a bonus question worth 20 points, and an ultra bonus question worth 30 points. However, there is no partial credit for these questions. You will get 20 or 0 for the bonus question, and 30 or 0 for the ultra-bonus question. If everything is correct, you can score 150 out of 100 in the assignment.

## E.1 Missing value imputation

Read *Housing\_missing\_price.csv* as `housing_missing_price` and *Housing\_complete.csv* as `housing_complete`. The datasets consist of housing features, like number of bedrooms, bathrooms, etc., and the price. Both datasets are exactly the same, except that *Housing\_missing\_price.csv* has some missing values of price. In this question, you will try different methods to impute the missing values of house price in the file *Housing\_missing\_price.csv*. You will use the prices in *Housing\_complete.csv* to check the accuracy of your imputation.

Note that you **cannot** use *Housing\_complete.csv* to impute missing **price** in any of the questions. *Housing\_complete.csv* is just to check the accuracy of your imputation, after you have done the imputation. Before imputing the missing **price**, assume you do not have *Housing\_complete.csv*.

### E.1.1 Number of missing values

How many values of **price** are missing in *Housing\_missing\_price.csv*?

(3 points)

### E.1.2 Indices of missing values

Find the row labels, where the **price** is missing. Assign those row labels as an array or a list to `index_null_price`.

(4 points)

### E.1.3 Correlation of continuous variables with price

Find the continuous variable having the highest correlation with **price**. Let the variable be  $A_{cont}$ .

(2 points)

### E.1.4 Missing value imputation using correlated continuous variable

Make a scatterplot of **price** against  $A_{cont}$ , with a trendline. Using the trendline, impute the missing values of **price**.

Plot the imputed **price** vs true **price** (from *Housing\_complete.csv*) and print the RMSE (*Root mean squared error*). This is to be done only for the imputed values of **price**.

(10 points)

**Hint:**

1. Make the trendline using non-missing values of **price** and  $A_{cont}$ .
2. Impute the missing values of **price** only where they are missing, i.e., at row indices `index_null_price`.

3. You may use the function below to plot the imputed price vs true price (from *Housing\_complete.csv*) and print the RMSE. The function assumes that `housing_imputed_price` is the *Housing\_missing\_price.csv* dataset, with imputed values of `price`.

```
#Defining a function to plot the imputed values vs actual values
def plot_actual_vs_predicted():
 fig, ax = plt.subplots(figsize=(9, 6))
 plt.rc('xtick', labelsizes=15)
 plt.rc('ytick', labelsizes=15)
 x = housing_complete.loc[index_null_price, 'price']/1e3
 y = housing_imputed_price.loc[index_null_price, 'price']/1e3
 plt.scatter(x, y)
 z = np.polyfit(x, y, 1)
 p = np.poly1d(z)
 plt.plot(x, p(x), color='orange')
 plt.xlabel('Actual price', fontsize=20)
 plt.ylabel('Imputed price', fontsize=20)
 ax.xaxis.set_major_formatter('${x:,.0f}k')
 ax.yaxis.set_major_formatter('${x:,.0f}k')
 rmse = np.sqrt(((x-y).pow(2)).mean())
 print("RMSE= $" + str(np.round(rmse, 2)) + "k")
```

### E.1.5 Correlation of categorical variables with price

Split the categorical columns of the *Housing\_missing\_price.csv*, such that they transform into dummy variables with each category corresponding to a column of 0s and 1s. The continuous variables remain as they were in the original dataset. Name this dataset as `housing_dummy`.

Which categorical variable is the most highly correlated with `price`? Let that variable be  $V_{cat}$ .

(3 + 2 points)

**Hint:** `pd.get_dummies()`

### E.1.6 Missing value imputation using correlated categorical variable

Impute the missing value of the `price` of a house as the average price of all the houses that have the same value of  $V_{cat}$ . For example, if  $V_{cat}$  is `basement`, the missing price of a house that has a basement must be imputed as the average price of all the houses that have a basement, and the missing price of a house that lacks a basement must be imputed as the average price of all the houses that lack a basement.

Plot the imputed `price` vs true `price` (from *Housing\_complete.csv*) and print the RMSE (Root mean squared error). This is to be done only for the imputed values of `price`.

(10 points)

**Hint:** You may use the following code to get the mean house price for each level of the variable  $V_{cat}$ :

```
#Replace 'Vcat' by the variable that you found to be the most correlated with 'price'
price_Vcat = housing_missing_price['price'].groupby(housing_missing_price[Vcat]).mean()
price_Vcat
```

### E.1.7 Missing value imputation using correlated continuous variable within the categories of correlated categorical variable

Execute the following code. Note that the trendlines of `price` against `area` are different based on `airconditioning`.

For each house, select the appropriate trendline to impute the missing `price`.

Plot the imputed `price` vs true `price` (from *Housing\_complete.csv*) and print the RMSE (Root mean squared error). This is to be done only for the imputed values of `price`.

(15 points)

```
sns.set(font_scale=1.25)
a = sns.FacetGrid(housing_missing_price, hue = 'airconditioning',height=6, aspect=1.5)
a.map(sns.regplot,'area','price')
a.add_legend();
```

<IPython.core.display.Image object>

### E.1.8 Bonus question: Missing value imputation with KNN

(20 points)



### E.1.8.1 Identifying optimal $K$ by $k$ -fold cross validation

You need to impute the missing **price** using the KNN ( $K$ -nearest neighbor) algorithm. However, before implementing the algorithm, find the optimal value of  $K$ , using  $k$ -fold cross-validation. Use all the variables in **housing\_dummy**.

Note that you **cannot** use *Housing\_complete.csv* to find the optimal  $K$ .

Follow the  $k$ -fold cross validation procedure below to find the optimal  $K$ , i.e., the optimal number of nearest neighbors to consider when imputing missing values:

1. Remove observations with missing **price** from **housing\_dummy**. Let us call the resulting DataFrame as **housing\_missing\_removed**.
2. Split **housing\_missing\_removed** into  $k$ -folds. Take  $k = 10$ . Each fold will have one-tenth of the observations of **housing\_missing\_removed**.
3. Iterate over the  $K^{th}$  potential value of  $K$ , where  $K \in \{1, 2, \dots, 50\}$ .
  - A. Iterate over the  $i^{th}$  fold, where  $i \in \{1, 2, \dots, 10\}$ 
    - I. Assume that the all the **price** values of the  $i^{th}$  fold are missing. Impute the value of **price** for an observation in the  $i^{th}$  fold as the mean **price** of the  $K$ -nearest neighbors to the observation, where the neighbors are from among the observations in the remaining 9 folds.
    - II. Compute the RMSE (Root mean squared error) for the  $i^{th}$  fold. Let us denote the RMSE for  $i^{th}$  fold, and considering  $K$  nearest neighbors as  $RMSE_{iK}$ .
  - B. Find the average of the 10 RMSEs obtained in 3(A). Let us denote it as  $RMSE_K$ , i.e., RMSE for a given value of  $K$ . Then,

$$RMSE_K = \frac{1}{10} \sum_{i=1}^{i=10} RMSE_{iK}$$

4. The optimal value of  $K$  is the one for which  $RMSE_K$  is the minimum, i.e.,

$$K_{optimal} = \underset{K \in \{1, \dots, 50\}}{\operatorname{argmin}} RMSE_K$$

**Assumption to make it a bit simpler:** Ideally you should split the dataset randomly into  $k$ -folds. However, to make it simpler, you may assume that the data is already shuffled, and you may take the first  $1/10^{th}$  observations to be in the  $1^{st}$  fold, the next  $1/10^{th}$  to be in the  $2^{nd}$  fold and so on.

**More explanation** about  $k$ -fold cross validation and the optimal  $K$ :

You need to impute the missing price using the KNN (K-nearest neighbor) algorithm. However, before implementing the algorithm, find the optimal value of  $K$ , using  $k$ -fold cross-validation. This is an optimization method used for more advanced Machine Learning methods, such as KNN. In KNN, the number of neighbors,  $K$ , is called a hyperparameter, which cannot be optimized with a mathematical method. Therefore, it needs a more coding-based optimization method called cross-validation.

The idea of cross-validation is to split the dataset into subsets, called folds. After that a range of  $K$  values is picked. For each  $K$  value in the range, the KNN imputer is created and evaluated on each fold separately, returning an RMSE value for each fold. The average value of these RMSE values is the cross-validation RMSE value of that  $K$  value. Cross-validation is a robust method to find the best  $K$  in the KNN algorithm for the data at hand because it evaluates different parts of the data separately and takes the average of all results. It is called  $k$ -fold cross-validation, with  $k$  as the number of folds we want to use, usually 3, 5 or 10. In this problem, we will use 10-fold cross-validation. Note that you need nested for loops to iterate over both each  $K$  value and each fold for this algorithm.

### E.1.8.2 Implementing KNN with optimal $K$

Using the optimal value of  $K = K_{optimal}$  obtained in the previous question, impute the missing values of `price` in `housing_missing_price`. Use all the variables in `housing_dummy` for implementing the KNN algorithm. Plot the imputed price vs true price (from `Housing_complete.csv`) and print the RMSE (Root mean squared error). This is to be done only for the imputed values of `price`.

Answer check: The RMSE obtained with KNN should be lower than that obtained with any of the earlier methods. If not, then there may be some mistake in your KNN implementation.

### E.1.9 Ultra-bonus question

*(30 points)*

Develop an algorithm to impute the missing `price`, such that it reduces the imputation RMSE to below \$650k. Plot the imputed price vs true price (from `Housing_complete.csv`) and print the RMSE (Root mean squared error). This is to be done only for the imputed values of `price`.

*Note that we have not attempted to solve this question yet. We are not sure if a solution exists. However, if you find a solution, you will get 30 points.*

**Hint:** In the bonus question, all variables were given equal weights when imputing missing values with KNN. However, shouldn't some variables be given higher weight than others?

*If you think you are successful, email your solution to your instructor for grading.*

## E.2 Binning

Read *house\_features\_and\_price.csv*. We will bin a couple of variables to better analyze the trend of `house_price` with those variables.

### E.2.1 Trend with `house_age`

Make a scatterplot of `house_price` against `house_age`, along with the trendline. What is the trend indicated by the trendline?

*(4 points)*

### E.2.2 Trend with bins of `house_age`

#### E.2.2.1

Bin `house_age` into 5 approximately equal-sized bins.

*(3 points)*

#### E.2.2.2

After binning, plot the mean `house_price` against the house age bins.

*(3 points)*

#### E.2.2.3

Is the trend seen with this plot different from that seen with the trendline in the previous question? If yes, then is one of the trends incorrect? Why or why not?

*(4 points)*

#### E.2.2.4

Is one of the trends more informative? If yes, then which one and how?

*(4 points)*

### E.2.3 Trend with `number_convenience_stores`

Make a barplot to visualize the mean `house_price` against `number_convenience_stores`.

*(3 points)*

### E.2.4 Trend with bins of `number_convenience_stores`

Bin `number_convenience_stores` into an appropriate number of bins such that the non-linear trend of the variation of `house_price` with the bins of `number_convenience_stores` is retained, while minimizing the number of bins.

After binning, plot the mean `house_price` against the `number_convenience_stores` bins.

*(8 points)*

### E.2.5 Size of `number_convenience_stores` bins

Print the size of bins obtained in the previous question. Are the bins of approximately equal size? If not, is it reasonable to have bins of unequal sizes to visualize the trend of `house_price` with `number_convenience_stores`?

*(2 + 4 points)*

## E.3 Outliers

### E.3.1 Identifying outlying prices

Continue working with `house_features_and_price.csv`. Determine how many houses have outlying values in `house_price` using both the Z-score method and the IQR method.

- **Question:** Which method is more conservative for this dataset? *(6 + 2 points)*

### E.3.2 Quick EDA

How are these houses (identified in the previous question as outliers) different from the houses in the rest of the dataset, which might be making them extremely expensive / extremely cheap? Explore the data and mention your hypothesis.

*(8 points)*

# F Assignment F

Data Wrangling

## Instructions

1. You may talk to a friend, discuss the questions and potential directions for solving them. However, you need to write your own solutions and code separately, and not as a group activity.
2. Write your code in the *Code* cells and your answer in the *Markdown* cells of the Jupyter notebook. Ensure that the solution is written neatly enough to understand and grade.
3. Use [Quarto](#) to print the *.ipynb* file as HTML. You will need to open the command prompt, navigate to the directory containing the file, and use the command: `quarto render filename.ipynb --to html`. Submit the HTML file.
4. The assignment is worth 100 points, and is due on **17th Nov 2025 at 11:59 pm**. No extension is possible on this assignment due to tight grading deadlines.
5. You are **not allowed to use a for loop in this assignment**.
6. If you are updating a dataset (imputing missing values / creating new variables), then use the updated dataset in a subsequent question.
7. **Five points are properly formatting the assignment**. The breakdown is as follows:
  - Must be an HTML file rendered using Quarto (2 pts).
  - There aren't excessively long outputs of extraneous information (e.g. no printouts of entire data frames without good reason, there aren't long printouts of which iteration a loop is on, there aren't long sections of commented-out code, etc.) (1 pt)
  - Final answers of each question are written in Markdown cells (1 pt).
  - There is no piece of unnecessary / redundant code, and no unnecessary / redundant text (1 pt)

## F.1 Canadian Fish Biodiversity

Read data from the file *Canadian\_Fish\_Biodiversity.csv* on Canvas. Each row records a unique fishing event from a 2013 sample of fish populations in Ontario, Canada. To analyze the results of these fishing surveys, we need to understand the dynamics of projects, sites, and geographic locations.

### F.1.1 Top 3 projects

Each site (identified by the column `SITEID`) represents a time and place at which fishing events occurred. Sites are grouped into broader projects (identified by the column `Project Name`). We want to understand the scope of these projects.

Using `groupby()`, find the top three projects by number of unique sites.

*(4 points)*

### F.1.2 Missing value imputation with `groupby()`

#### F.1.2.1 Number of missing values

How many values are missing for the air temperature column (`Air Temperature (C)`)?

*(1 point)*

#### F.1.2.2 Missing value imputation: attempt 1

Using `groupby()`, impute the missing values of air temperature with the median air temperature of the corresponding water body (`Waterbody Name`) and `Month`.

*(4 points)*

#### F.1.2.3 Missing values remaining after attempt 1

How many missing values still remain for the air temperature column after the imputation in the previous question?

*(1 point)*

#### **F.1.2.4 Missing value imputation: attempt 2**

We will try to impute the remaining missing values for air temperature. Try to impute the remaining missing values of air temperature with the median air temperature of the corresponding project (`Project Name`) and `Month`.

*(4 points)*

#### **F.1.2.5 Missing values remaining after attempt 2**

How many missing values still remain for the air temperature column after the imputation in the previous question?

*(1 point)*

#### **F.1.2.6 Air-water temperatures correlation**

Find the correlation between air temperature and water temperature.

*(1 point)*

#### **F.1.2.7 Missing values remaining after hypothetical attempt 3**

As you found a high correlation between air temperature and water temperature, you can use water temperature to estimate the air temperature (*using the trendline, like you did in assignment 5*). Assuming you already did that, how many missing values will still remain for the air temperature column?

**Note:** Do not impute the missing values using the trendline, just assume you already did that.

*(3 points)*

#### **F.1.2.8 Visualizing missing value imputation**

Make a scatterplot of air temperature against water temperature. Highlight the points for which the air temperature was imputed in attempts 1 and 2 with a different color.

*(8 points)*

### F.1.3 Living conditions

This section begins to investigate the living conditions of fish at different locations and time periods. Continue using the updated dataset with the imputed missing values in attempts 1 and 2 of the previous section.

#### F.1.3.1 Air-water temperatures: Summary statistics

Use a single `groupby` statement to view the minimum, mean, standard deviation, and maximum air temperature and water temperature for each project during the month of August (use the `Month` column).

(5 points)

#### F.1.3.2 Air-water temperatures: visualizing yearly trend

Make lineplots showing maximum air temperature and water temperature by `Month` and `Region`. To construct `Region`, use the Pandas function `cut()` to satisfy the following conditions:

- Rows with a latitude lower than 42.4 should have *Southern* in the `Region` column
- Rows with a latitude between 42.4 and 42.8 should have *Central* in the `Region` column
- Rows with a latitude higher than 42.8 should have *Northern* in the `Region` column

You can have the month on the horizontal axis, the temperature on the vertical axis, different colors for different regions, and different styles (solid line / dotted line) to indicate air/water temperature.

Does anything in the visualization surprise you? Why or why not?

(14 points)

### F.1.4 Fish diversity

Finally let's focus on the stars of this survey—the fish, of course.

#### F.1.4.1 Top 3 species by Region

Let's continue using our `Region` categorization. Find the top three fish `Species` in each region by `Number Captured`.

(10 points)



#### **F.1.4.2 Species spread across Region**

Are certain fish only found in some regions? Visualize how many species are in all three regions, how many are in two of three, and how many were only captured in one region.

*(10 points)*

#### **F.1.4.3 Exclusive fishes by region**

What percentage of all species are exclusively captured in the Southern region? How about the Northern Region? And the Central region?

*(10 points)*

#### **Hint:**

1. Find the number of distinct regions in which each species is found.
2. Filter the species that are found only in one region.
3. Group the data, containing only the species found in (2), by region, count the number of unique species in each group, and divide by the total number of distinct species.

#### **F.1.4.4 Turbidity**

Turbidity (`Turbidity (ntu)`) quantifies the level of cloudiness in liquid. For fish in each of the three regions, is there a linear association between turbidity and number of fish caught? You may consider a correlation higher than 50% in magnitude as presence of a linear association.

*(5 points)*

#### **F.1.4.5 Fish dimensions**

Now let's turn to the length of fish captured, given by `Maximum (mm)` and `Minimum (mm)`. Find the overall maximum and minimum lengths of all fish in each region. Which region has the largest range in captured fish length?

*(4 points)*

## F.2 GDP, surplus, and compensation

The dataset *Real GDP.csv* contains the GDP of each US State for all years starting from 1997 until 2020. The data is at *State* level, i.e., each observation corresponds to a unique State.

The dataset *Surplus.csv* contains the surplus of each US State for all years starting from 1997 until 2020. The data is at *year* level, i.e., each observation corresponds to a unique year.

The dataset *Compensation.csv* contains *Compensation* and *Chain-type quantity indexes for real GDP* for each US State and year starting from 1997 to 2020. The dataset is at *Year-State-Description* level, i.e., each observation corresponds to a unique **Year-State-Description** combination where **Description** refers to either *Compensation* or *Chain-type quantity indexes for real GDP*.

### F.2.1 Combining datasets

Combine all these datasets to obtain a dataset at *State-Year* level, i.e., each observation corresponds to a unique **State-Year** combination. The combined dataset must contain the GDP, surplus, *Compensation*, and *Chain-type quantity indexes for real GDP* for each US State and all years starting from 1997 until 2020. *Note that each observation must contain the name of the US State, year, and the four values (GDP, surplus, compensation, and Chain-type quantity indexes for real GDP).*

**Hint:** Here is one way to do it:

1. Melt the GDP dataset to year-State level
2. Melt the Surplus dataset to year-State level
3. Pivot the compensation dataset to year-State level
4. Now that all the datasets are at the year-State level, merge them!

*(3 + 3 + 3 + 1 = 10 points)*

### F.2.2 Time trend: GDP with region

Merge the file *State\_region\_mapping.csv* with the dataset obtained in the previous question. Make a lineplot showing the mean GDP for each of the five regions with year. Do not display the confidence interval. Which two regions seems to have the least growth in GDP over the past 24 years?

*(5 points)*

## G Datasets & Templates

Datasets used in the book, and assignment / project templates can be found [here](#)